

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

DSA Question Bank With Answers

Data Structures & Algorithms

CSE 105 (L-1/T-2)

CSE 203 (L-2/T-1) & CSE 207 (L-2/T-2)

Topics: Asymptotic Analysis, Data Structures, Graph Traversals, Divide & Conquer, Greedy, Dynamic Programming, Sorting & more

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting

Claude AI — $\text{\LaTeX}$ Typesetting

Gemini — $\text{\LaTeX}$ & Diagram Generation

Table of Contents

Part I — Foundations

- ▷ **T1.** Introduction to Algorithms (12 questions)
- ▷ **T2.** Asymptotic Notations: $\mathcal{O}$, Ω , Θ , o , ω (22 questions)
- ▷ **T3.** Correctness Proof & Algorithm Analysis (14 questions)

Part II — Elementary Data Structures

- ▷ **T4.** Arrays, Linked Lists, Stacks & Queues (56 questions)
- ▷ **T5.** Trees & Tree Traversals (11 questions)
- ▷ **T6.** Heaps & Binary Search Trees (42 questions)
- ▷ **T7.** Graphs & Graph Representations (12 questions)

Part III — Graph Algorithms

- ▷ **T8.** Graph Traversals: DFS & BFS (18 questions)
- ▷ **T8a.** Applications of DFS & BFS (34 questions)

Part IV — Algorithm Design

- ▷ **T9.** Divide & Conquer (19 questions)
- ▷ **T10.** Greedy Methods (58 questions)
- ▷ **T11.** Dynamic Programming (58 questions)

Part V — Sorting & Lower Bounds

- ▷ **T12.** Comparison-Based Sorting (19 questions)
- ▷ **T13.** Sorting in Linear Time (7 questions)
- ▷ **T14.** Lower Bound Theory (2 questions)

Part VI — Set Operations

- ▷ **T15.** Data Structures for Set Operations (5 questions)

Part VII — Advanced Topics (Beyond CSE 105)

- ▷ **T16.** Balanced BSTs: AVL & Red-Black Trees (34 questions)
- ▷ **T17.** Skip Lists & Hashing (21 questions)
- ▷ **T18.** Advanced Heaps: Binomial & Fibonacci (9 questions)
- ▷ **T19.** Network Flow & Bipartite Matching (27 questions)
- ▷ **T20.** NP-Completeness & Approximation (36 questions)
- ▷ **T21.** Miscellaneous (17 questions)

BUET · CSE 105

DSA

Introduction to Algorithms

Intro, Basic Search, Array Mapping

T1 — Introduction to Algorithms

- Q1.** Consider the code snippet for a binary search, where you are searching for an item x from an array a of size n . If all primitive operations (addition/subtraction, comparison, assignment, etc.) take one unit of time each to execute, what will be the time complexity of the above algorithm by counting all the primitive operations performed. Can you show the worst-case time complexity of the above algorithm using big-Oh notation? **(5 marks)** [CSE 203 | 2019–20]

Algorithm 1 BINARYSEARCH(a, x)

```

1: left ← 0
2: right ← a.size()
3: while left < right do
4:   middle ← (left + right) / 2
5:   if a[middle] ≤ x then
6:     left ← middle + 1
7:   else
8:     right ← middle
9:   end if
10: end while
11: if left > 0 and a[left - 1] == x then
12:   return true
13: else
14:   return false
15: end if

```

Answer

Counting primitive operations: Each iteration of the **while** loop performs: 1 comparison (**left < right**), 1 addition + 1 division for **middle**, 1 comparison (**a[middle] ≤ x**), 1 addition or 0 assignments. Approximately 5 operations per iteration. The loop runs $\lfloor \log_2 n \rfloor$ times. The final **if** block adds 2 more operations. Total $\approx 5\lfloor \log_2 n \rfloor + 2$ primitive operations.

Worst-case Big-O: $T(n) = O(\log n)$, since the number of operations grows logarithmically with n .

- Q2.** Write down two algorithms for determining whether a year is a leap year. Deduce in detail the average-case complexity of both algorithms. **(5+5 marks)** [CSE 207 | 2006–07, 2013–14]

Answer

Algorithm 1 — Standard 3-condition check:

Algorithm 2 ISLEAPYEAR_A(y)

```

1: if  $y \bmod 400 = 0$  then return true
2: end if
3: if  $y \bmod 100 = 0$  then return false
4: end if
5: if  $y \bmod 4 = 0$  then return true
6: end if
7: return false

```

Average-case complexity: Each condition is $O(1)$. In a uniform distribution over all years, $1/400$ reach line 1 (return true), $3/400$ reach line 2 (return false), $96/400$ reach line 3, and $300/400$ reach line 4. The expected number of comparisons is $1 \cdot \frac{1}{400} + 2 \cdot \frac{3}{400} + 3 \cdot \frac{96}{400} + 4 \cdot \frac{300}{400} \approx 3.75$. This is a constant, so the average-case complexity is $\Theta(1)$.

Algorithm 3 ISLEAPYEAR_B(y)

```

1: return  $((y \bmod 4 = 0) \text{ and } (y \bmod 100 \neq 0 \text{ or } y \bmod 400 = 0))$ 

```

Average-case complexity: This evaluates at most 3 modular conditions, all $O(1)$. With short-circuit evaluation, the average number of conditions checked is constant. Thus average-case complexity is also $\Theta(1)$.

Both algorithms are $\Theta(1)$ in all cases since the input size is fixed (one integer) and the number of operations is bounded by a constant.

Q3. Write down two versions of the binary search algorithm. Deduce the average-case complexity of both. **(5+5 marks)** [CSE 207 | 2013–14]

Answer

Version 1 — Standard (early exit on equality): Checks $A[mid] = x$ at each step; terminates as soon as element is found. Average-case comparisons (successful): $\approx \log_2 n - 1 + \frac{1}{n} = \Theta(\log n)$.

Version 2 — Delayed (equality check at end): Only checks $A[mid] \leq x$ in the loop; performs one equality check after the loop exits. Always runs exactly $\lceil \log_2 n \rceil$ loop iterations then 1 check = $\Theta(\log n)$.

Both versions have average-case complexity $\Theta(\log n)$. Version 2 is more uniform (same cost for all inputs of size n), while Version 1 is faster in the best case ($\Theta(1)$) but has the same asymptotic average.

Q4. Write down traditional algorithms for finding the minimum and maximum of an array, and a recursive algorithm. Compare their complexities. **(10 marks)** [CSE 207 | 2013–14]

Answer

Traditional (linear scan): 2 comparisons per element $\Rightarrow 2(n - 1)$ total $= \Theta(n)$.

Recursive divide-and-conquer:

Algorithm 4 MINMAXREC(A, lo, hi)

```

1: if  $lo = hi$  then
2:   return ( $A[lo], A[lo]$ )
3: end if
4: if  $hi = lo + 1$  then
5:   if  $A[lo] < A[hi]$  then
6:     return ( $A[lo], A[hi]$ )
7:   else
8:     return ( $A[hi], A[lo]$ )
9:   end if
10: end if
11:  $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
12:  $(lmin, lmax) \leftarrow \text{MINMAXREC}(A, lo, mid)$ 
13:  $(rmin, rmax) \leftarrow \text{MINMAXREC}(A, mid + 1, hi)$ 
14: return ( $\min(lmin, rmin), \max(lmax, rmax)$ )

```

Recurrence: $T(n) = 2T(n/2) + 2$, with $T(2) = 1$. Solving: $T(n) = \frac{3n}{2} - 2$ comparisons.

Comparison:

Algorithm	Comparisons	Complexity
Traditional	$2(n - 1)$	$\Theta(n)$
Recursive	$\frac{3n}{2} - 2$	$\Theta(n)$

Both are $\Theta(n)$, but the recursive version uses $\approx 25\%$ fewer comparisons.

Q5. What is a data structure? When is a data structure called a linear data structure? Write 4 (four) properties of a linear data structure. **(1.5+1.5+4=7 marks)** [CSE 203 | 2013–14]

Answer

Data structure: An organised way of storing and managing data so operations can be performed efficiently.

Linear data structure: A data structure is *linear* if its elements form a sequence, i.e. each element (except the first and last) has exactly one predecessor and one successor, and all elements can be traversed in a single pass.

Four properties:

1. Elements are arranged in a sequential, one-dimensional order.

2. Every element (except the first) has a unique predecessor; every element (except the last) has a unique successor.
3. Can be traversed from beginning to end in one pass.
4. Memory is allocated either contiguously (array) or via pointers (linked list).
Examples: array, stack, queue, linked list.

Q6. Let $a : \text{array}[l_1 \dots u_1, l_2 \dots u_2]$ be a 2D integer array. Assume b is the base address of the array a . Write the Array Mapping Function to calculate $\text{address}(a[i, j])$ in:

- (a) Column-major order
- (b) Row-major order

(5+5=10 marks)

[CSE 203 | 2013–14]

Answer

Let $d_1 = u_1 - l_1 + 1$ and $d_2 = u_2 - l_2 + 1$ be the sizes. Let c = size of one integer element.

- (a) **Column-major order** (FORTRAN-style): column index varies fastest.

$$\text{address}(a[i, j]) = b + c \cdot [(j - l_2) \cdot d_1 + (i - l_1)]$$

- (b) **Row-major order** (C-style): row index varies fastest.

$$\text{address}(a[i, j]) = b + c \cdot [(i - l_1) \cdot d_2 + (j - l_2)]$$

Derivation (row-major): To reach row i , skip $(i - l_1)$ complete rows of d_2 elements. Within row i , element j is at offset $(j - l_2)$. Multiply total offset by c and add base address b .

Q7. What is a data structure? Write the properties of a linear data structure. (1+4=5 marks)

[CSE 203 | 2012–13]

Answer

Data Structure: A data structure is a systematic way of organising, storing, and managing data in a computer so that it can be accessed and modified efficiently. It defines both the data and the operations that can be performed on that data (e.g. insertion, deletion, search).

Properties of a Linear Data Structure:

1. **Sequential arrangement:** Elements are arranged in a linear (one-dimensional) sequence, where each element has a unique predecessor (except the first) and a unique successor (except the last).

2. **Single level:** All elements reside on a single level; there is no hierarchical or branching structure.
3. **Traversal in one pass:** The entire structure can be traversed in a single run from the first element to the last.
4. **Memory:** Elements are stored either in contiguous memory locations (e.g. arrays) or linked via pointers (e.g. linked lists). Examples include arrays, stacks, queues, and linked lists.

Q8. Let $\mathbf{a} : \text{array}[l_1 \dots u_1, l_2 \dots u_2, l_3 \dots u_3]$ be a 3-dimensional array. Assume that b and c are the base address and component length of the array $\mathbf{a}$ respectively. Write the Array Mapping Function to calculate $\text{addr}(a[i, j, k])$. **(10 marks)** [CSE 203 | 2012–13]

Answer

Let $d_2 = u_2 - l_2 + 1$ and $d_3 = u_3 - l_3 + 1$ be the sizes of the second and third dimensions respectively.

Using **row-major order**, the Array Mapping Function is:

$$\text{addr}(a[i, j, k]) = b + c \cdot [(i - l_1) \cdot d_2 \cdot d_3 + (j - l_2) \cdot d_3 + (k - l_3)]$$

Derivation: To reach the i -th slab, we skip $(i - l_1)$ complete slabs each containing $d_2 \times d_3$ elements. Within that slab, to reach row j we skip $(j - l_2)$ rows of d_3 elements each. Within that row, k is at offset $(k - l_3)$ from the start. Each element occupies c bytes, so we multiply the total element offset by c and add to the base address b .

Q9. Write down the algorithm for binary search. Analyze its best-case, average-case, and worst-case complexities. Simulate the search of elements 2 and 10 in the array $[1, 2, 4, 7, 9, 11, 12, 13, 14]$ by showing the values of **low**, **high**, **mid**, and **found**. **(5+5+5=15 marks)** [CSE 207 | 2008–09]

Answer**Algorithm 5** BINARYSEARCH(A, n, x)

```

1:  $low \leftarrow 1$ ;  $high \leftarrow n$ ;  $found \leftarrow \mathbf{false}$ 
2: while  $low \leq high$  and not  $found$  do
3:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
4:   if  $A[mid] = x$  then
5:      $found \leftarrow \mathbf{true}$ 
6:   else if  $A[mid] < x$  then
7:      $low \leftarrow mid + 1$ 
8:   else
9:      $high \leftarrow mid - 1$ 
10:  end if
11: end while
12: if  $found$  then
13:   return  $mid$ 
14: else
15:   return  $-1$ 
16: end if

```

Complexities: Best case: $\Theta(1)$ (element at midpoint). Worst/Average case: $\Theta(\log n)$.

Simulation on $A = [1, 2, 4, 7, 9, 11, 12, 13, 14]$ ($n = 9$, 1-indexed):

Search for 2:

Step	low	high	mid	$A[mid]$
1	1	9	5	$9 > 2 \Rightarrow high=4$
2	1	4	2	$2 = 2 \Rightarrow found=true$

Result: found at index 2.

Search for 10:

Step	low	high	mid	$A[mid]$
1	1	9	5	$9 < 10 \Rightarrow low=6$
2	6	9	7	$12 > 10 \Rightarrow high=6$
3	6	6	6	$11 > 10 \Rightarrow high=5$
4	low > high			not found

Result: not found.

Q10. Write down an algorithm for finding the minimum and maximum of an array. Deduce its complexity. (10 marks) [CSE 207 | 2007–08]

Answer
Algorithm 6 MINMAX(A, n)

```

1:  $min \leftarrow A[1]$ 
2:  $max \leftarrow A[1]$ 
3: for  $i = 2$  to  $n$  do
4:   if  $A[i] < min$  then
5:      $min \leftarrow A[i]$ 
6:   end if
7:   if  $A[i] > max$  then
8:      $max \leftarrow A[i]$ 
9:   end if
10: end for
11: return ( $min, max$ )
    
```

Complexity: The loop runs $n - 1$ iterations. In each iteration, exactly 2 comparisons are made (regardless of values). Total comparisons = $2(n - 1)$. Therefore the time complexity is $\Theta(n)$.

Improved algorithm (Tournament / Pair comparison): Process elements in pairs: compare the pair first, then compare the smaller with current min and the larger with current max. This uses only $\frac{3(n-1)}{2}$ comparisons instead of $2(n - 1)$, but remains $\Theta(n)$.

Q11. Deduce the best-case, worst-case, and average-case complexity of sequential search in both successful and unsuccessful cases. (10 marks) [CSE 207 | 2007–08]

Answer

Algorithm: Search for element x in array $A[1..n]$ by scanning left to right.

Case	Successful	Unsuccessful	Comparisons
Best case	$x = A[1]$ (found at pos 1)	N/A	1
Worst case	$x = A[n]$ (found at end)	$x \notin A$ (scan all)	n
Average case	element equally likely at any of n positions	full scan	$\frac{n+1}{2}$ (success); n (fail)

Derivation of average-case (successful): If x is at position i with equal probability $\frac{1}{n}$, the expected comparisons are:

$$\sum_{i=1}^n \frac{1}{n} \cdot i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n)$$

Unsuccessful: Always scans all n elements $\Rightarrow \Theta(n)$.
All non-trivial cases are $\Theta(n)$; best case is $\Theta(1)$.

- Q12.** Write down an algorithm for binary search that delays the finding until the end. Deduce its average-case complexity. **(5+10 marks)** [CSE 207 | 2006–07]

Answer

Algorithm 7 DELAYEDBINARYSEARCH(A, n, x)

```

1:  $low \leftarrow 1$ 
2:  $high \leftarrow n$ 
3: while  $low < high$  do
4:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
5:   if  $A[mid] \leq x$  then
6:      $low \leftarrow mid + 1$ 
7:   else
8:      $high \leftarrow mid$ 
9:   end if
10: end while
11: if  $A[low] = x$  then
12:   return  $low$ 
13: else
14:   return NOT_FOUND
15: end if

```

Unlike standard binary search which checks equality at each step, this version only checks $A[mid] \leq x$ in the loop, delaying the equality check until $low = high$.

Average-case complexity: The loop always runs $\lceil \log_2 n \rceil$ iterations regardless of where the element is, because equality is never checked inside the loop. In each iteration, exactly one comparison is made. Therefore:

$$T_{\text{avg}}(n) = \lceil \log_2 n \rceil + 1 = \Theta(\log n)$$

This is actually better than standard binary search in the average case (which does $\sim \log n - 1$ comparisons for successful search) because it makes only $\lceil \log_2 n \rceil + 1$ total comparisons in every case, making it more uniform and cache-friendly.

BUET · CSE 105

DSA

Asymptotic Notations: $\mathcal{O}$, Ω , Θ , o , ω

Growth of Functions, Notations, Code Complexity

T2 — Asymptotic Notations: $\mathcal{O}$, Ω , Θ , o , ω

Q1. Analyze and compare the growth rate of $n^{\log \log n}$ and $(\log n)^n$. **(10 marks)** [CSE 105 | 2022–23]

Answer

Take logarithms to compare: $\ln(n^{\log \log n}) = \log \log n \cdot \ln n$ and $\ln((\log n)^n) = n \ln(\log n)$.

Ratio: $\frac{n \ln(\log n)}{\log \log n \cdot \ln n}$. As $n \rightarrow \infty$, n grows much faster than $\ln n \cdot \log \log n$, so the ratio $\rightarrow \infty$.

Therefore $(\log n)^n$ grows **much faster** than $n^{\log \log n}$, i.e. $n^{\log \log n} = o((\log n)^n)$.

Intuitively: the exponent of $(\log n)^n$ is n itself (linear in n), whereas the exponent of $n^{\log \log n}$ is only $\log \log n$ (doubly logarithmic).

Q2. Derive the time complexity of the following code fragments:

- (a)

```
for(int i=0; i<n; ++i)
    for(int j=0; j<i; ++j)
        sum += i+j;
```
- (b)

```
for(int i=0; i<n; ++i)
    for(int j=0; j<i*i; j++)
        sum += i+j;
```
- (c)

```
for(int i=0; i<n*n; i++)
    for(int j=0; j<i; j++)
        sum += i+j;
```
- (d)

```
for(int k=1; k<=n; k*=2)
    for(int j=1; j<=k; j++)
        sum++;
```

(4×4=16 marks)

[CSE 105 | 2021–22]

Answer

- (a) **i from 0 to $n-1$; j from 0 to $i-1$:** Inner loop runs $0+1+2+\dots+(n-1) = \frac{n(n-1)}{2}$ times. Complexity: $\Theta(n^2)$.
- (b) **j from 0 to i^2-1 :** Inner loop runs $\sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6} = \Theta(n^3)$. Complexity: $\Theta(n^3)$.
- (c) **Outer i from 0 to n^2-1 ; inner j from 0 to $i-1$:** Inner loop total $= \sum_{i=0}^{n^2-1} i = \frac{n^2(n^2-1)}{2} = \Theta(n^4)$. Complexity: $\Theta(n^4)$.
- (d) **Outer k doubles: $k = 1, 2, 4, \dots, n$; inner j from 1 to k:** k takes values $1, 2, 4, \dots, 2^{\lfloor \log_2 n \rfloor}$. Total inner iterations $= 1+2+4+\dots+2^{\lfloor \log_2 n \rfloor} = 2^{\lfloor \log_2 n \rfloor+1} - 1 \leq 2n - 1 = \Theta(n)$. Complexity: $\Theta(n)$.

Q3. In an array-based implementation of a stack, when the array space runs out, the

array capacity is increased. Analyze and explain the following scenarios through an amortized analysis of the `push()` function:

- (a) Increasing the capacity of the array by 1
- (b) Doubling the capacity of the array

(2×7=14 marks)

[CSE 105 | 2021–22]

Answer

- (a) **Increase by 1:** When the array is full at size k , we allocate a new array of size $k + 1$ and copy all k elements. A sequence of n pushes starting from size 1 incurs copies: $1 + 2 + 3 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$ total. Amortized cost per push: $\frac{O(n^2)}{n} = O(n)$. This is **inefficient**.
- (b) **Double the capacity:** When the array is full at size k , allocate a new array of size $2k$ and copy k elements. Doubling happens at sizes $1, 2, 4, \dots, 2^{\lceil \log_2 n \rceil}$. Total copies = $1 + 2 + 4 + \dots + 2^{\lceil \log_2 n \rceil} < 2n$. Amortized cost per push: $\frac{O(n)}{n} = O(1)$. This is **efficient** — the amortized cost of each `push()` is $O(1)$.

Q4. Explain what the three figures (Figure 1(i)–(iii)) signify with respect to asymptotic notations. (– marks) [CSE 105 | 2021–22]

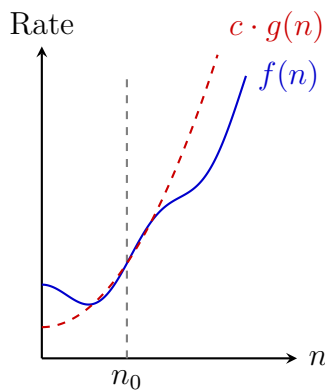


Figure : (i)

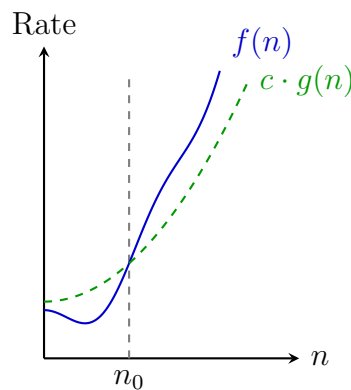


Figure : (ii)

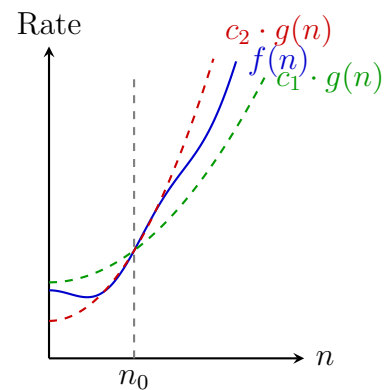


Figure : (iii)

Answer — Significance of Asymptotic Notations

The three figures visually define the primary asymptotic bounds used to analyze and classify the time or space complexity of algorithms. In all figures, n_0 represents a threshold input size, and c, c_1, c_2 are strictly positive constants.

I. Big-O Notation ($\mathcal{O}$) — Asymptotic Upper Bound

Figure 1(i) illustrates $f(n) = O(g(n))$. It signifies that for all inputs larger than n_0 , the running time $f(n)$ is bounded from above by a constant multiple of $g(n)$.

Formal Definition: $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Significance: It provides the *worst-case scenario*, guaranteeing that the algorithm will not take longer than this bounded rate to execute.

II. Big- Ω Notation (Ω) — Asymptotic Lower Bound

Figure 1(ii) illustrates $f(n) = \Omega(g(n))$. It signifies that for all inputs beyond n_0 , the function $f(n)$ will always grow at least as fast as a constant multiple of $g(n)$.

Formal Definition: $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

Significance: It provides the *best-case scenario* (or a hard minimum bound), guaranteeing that the algorithm will take *at least* this much time or resources.

III. Big- Θ Notation (Θ) — Asymptotic Tight Bound

Figure 1(iii) illustrates $f(n) = \Theta(g(n))$. It signifies that $f(n)$ is completely "sandwiched" between two constant multiples of $g(n)$ for all inputs beyond n_0 .

Formal Definition: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

Significance: It indicates that $f(n)$ grows at *exactly* the same rate as $g(n)$. It is used when an algorithm's worst-case and best-case complexities scale identically.

Q5. Find a tight upper bound on the complexity of the following code segments:

```
(a) void function(int n) {
    int count = 0;
    for (int i=n/2; i<=n; i++)
        for (int j=1; j+n/2<=n; j++)
            for (int k=1; k<=n; k = k*2)
                count++;
}
```

```
(b) void function(int n) {
    int count = 0;
    for (int i=n/2; i<=n; i++)
        for (int j=1; j<=n; j = 2*j)
            for (int k=1; k<=n; k = k*2)
                count++;
}
```

(10 marks)

[CSE 203 | 2021–22]

Answer

- (a) **Three nested loops:** Outer: i from $n/2$ to $n \Rightarrow O(n)$ iterations. Middle: j from 1 while $j + n/2 \leq n \Rightarrow j \leq n/2 \Rightarrow O(n)$ iterations. Inner: k doubles from 1 to $n \Rightarrow O(\log n)$ iterations. Total: $O(n) \cdot O(n) \cdot O(\log n) = O(n^2 \log n)$.
- (b) **Three nested loops (j doubles too):** Outer: $O(n)$. Middle: j doubles from 1 to $n \Rightarrow O(\log n)$. Inner: k doubles $\Rightarrow O(\log n)$. Total: $O(n \log^2 n)$.

Q6. Deduce the time complexity of the following code fragments:

- (a) `k = 1;`
 `while (n > 1) {`
 `n = n/2;`
 `k++;`
 `}`
- (b) `sum = 0;`
 `for (k=1; k<=n; k*=2)`
 `for (j=1; j<=k; j++)`
 `sum++;`

(2×6=12 marks)

[CSE 203 | 2020–21]

Answer

- (a) **$n = n/2$ loop:** Each iteration halves n . Starting from n , after k iterations $n = n_0/2^k$. Loop ends when $n \leq 1$, i.e. $k = \lfloor \log_2 n_0 \rfloor$. Complexity: $\Theta(\log n)$.
- (b) **Double-loop ($k*=2$, $j<=k$):** Outer loop: $k = 1, 2, 4, \dots, n \Rightarrow \log_2 n$ iterations. Inner loop runs k times. Total: $\sum_{i=0}^{\log_2 n} 2^i = 2n - 1 = \Theta(n)$.

Q7. Identify whether the following statements are true or false, clearly justifying your answer:

- (a) If an algorithm has time complexity $O(n^2)$, then it always makes n^2 steps.
- (b) An algorithm with time complexity $O(n)$ is always slower than an algorithm with time complexity $O(\log n)$, for any input.
- (c) An algorithm that makes $c_1 \log_2 n$ steps and an algorithm that makes $c_2 \log_{10} n$ steps are in the same complexity class (c_1, c_2 are constants).
- (d) An $O(2^n)$ algorithm can never be faster than an $O(n)$ algorithm.

(12 marks)

[CSE 203 | 2020–21]

Answer

- (a) **False.** $O(n^2)$ is an upper bound — the algorithm makes *at most* cn^2 steps; it may make far fewer. E.g. an algorithm that always runs in n steps is also $O(n^2)$.
- (b) **False.** Asymptotic notation describes growth rates for large n . For small n , an $O(n)$ algorithm may be slower if it has a large constant factor, e.g. $T_1(n) = 1000n$ vs $T_2(n) = \log_2 n$: for $n < 2^{1000}$, $T_1 > T_2$.
- (c) **True.** $\log_2 n = \frac{\log_{10} n}{\log_{10} 2}$, so $c_1 \log_2 n = \frac{c_1}{\log_{10} 2} \log_{10} n$. Both are constant multiples of each other $\Rightarrow$ same complexity class $\Theta(\log n)$.
- (d) **False.** For small inputs, 2^n may be smaller. E.g. at $n = 1$: $2^1 = 2 < 1$? No, but at $n = 1$: $O(2^n) = 2$ and $O(n) = 1$. However for very large constants in

$O(n)$, e.g. $T_1(n) = 10^{10} \cdot n$ and $T_2(n) = 2^n$: for $n < 33$, $T_2 < T_1$.

Q8. What does the term *asymptotic* mean in complexity analysis? Can we show the worst-case and average-case time complexity using Big- Θ notation? If yes, how and when? **(5+5+5 marks)** [CSE 203 | 2019–20]

Answer

Asymptotic: In complexity analysis, *asymptotic* means the behaviour of a function as the input size n grows without bound ($n \rightarrow \infty$). We ignore constant factors and lower-order terms and focus on the *dominant* term, which characterises long-run growth.

Can we use Θ for worst-case / average-case? Yes — Θ is a *tight* bound, not a specific-case bound. We can use Θ for any case as long as the bound is tight for that case:

- **Worst-case Θ :** Applicable when the worst-case cost has both a matching upper and lower bound. E.g. Insertion Sort worst-case is $\Theta(n^2)$.
- **Average-case Θ :** Applicable when the expected cost over all inputs of size n is tightly bounded. E.g. Quicksort average-case is $\Theta(n \log n)$.

We *cannot* use a single Θ expression to simultaneously describe all cases if they differ (e.g. Binary Search is $\Theta(1)$ best-case but $\Theta(\log n)$ worst-case).

Q9. Derive the time complexity of the following code snippet and write down the asymptotic complexity using Big- O notation. Can we show the complexity using Big- Θ notation as well? What is the difference in meaning?

```
int i, j, k = 0;
for (i = n/2; i <= n; i++) {
    for (j = 2; j <= n; j = j*2) {
        k = k + n/2;
    }
}
```

(10 marks)

[CSE 203 | 2019–20]

Answer

Analysis of the snippet: Outer loop: i runs from $n/2$ to n , giving $\approx n/2$ iterations. Inner loop: j starts at 2 and doubles until n , giving $\lfloor \log_2 n \rfloor$ iterations. Total: $\frac{n}{2} \cdot \log_2 n$ iterations.

Big- O : $O(n \log n)$.

Can we use Big- Θ ? Yes — the loop bounds are deterministic (not data-dependent), so the exact count is $\Theta(n \log n)$ in all cases. $\Theta(n \log n)$ is *tighter*: it says the running time grows *exactly* like $n \log n$ (up to constants), whereas $O(n \log n)$ only says it grows *at most* that fast.

Q10. Express the following function in Θ notation with proper explanation:

$$f(n) = 5n - 2n^2$$

(8 marks)

[CSE 203 | 2019–20]

Answer

For large n , the $-2n^2$ term dominates. We drop lower-order terms and constants: $f(n) = 5n - 2n^2 = n^2(5/n - 2)$. For $n \geq 10$, $5/n \leq 0.5$, so $f(n) \leq -1.5n^2 < 0$. Since $f(n) < 0$ for large n and complexity analysis applies to non-negative functions (e.g. counts of operations), we consider $|f(n)| = 2n^2 - 5n$. For $n \geq 5$: $n^2 \leq 2n^2 - 5n \leq 2n^2$. So $|f(n)| = \Theta(n^2)$.
Answer: $f(n) = \Theta(n^2)$ (the dominant term is n^2).

Q11. Describe the worst case running time of the following code in “big-Oh” notation in terms of the variable n . You should give the tightest bound possible.

```
int f(int n){
    if (n < 10){
        System.out.println("!");
        return n+3;
    } else {
        return f(n-1)+f(n-1);
    }
}
```

(5 marks)

[CSE 203 | 2018–19]

Answer

To determine the tightest upper bound for the function $f(n)$, we analyze its execution behavior using a recurrence relation based on the input size n .

1. Recurrence Relation Setup: For any input $n > 10$, the function makes two identical recursive calls to $n - 1$ and performs a constant amount of work c for the conditional check and addition.

- **Base Case:** For $n \leq 10$, the execution takes constant time:

$$T(n) = O(1) \quad \text{for } n \leq 10$$

- **Recursive Case:** For $n > 10$, the execution path yields:

$$T(n) = 2T(n - 1) + c$$

2. Solving by Substitution (Tree Method): Unrolling the recurrence step-by-step:

$$\begin{aligned} T(n) &= 2T(n-1) + c \\ &= 2(2T(n-2) + c) + c = 4T(n-2) + 3c \\ &= 8T(n-3) + 7c \\ &\vdots \\ &= 2^k T(n-k) + (2^k - 1)c \end{aligned}$$

The recursion hits the base case boundary when $n - k = 10 \implies k = n - 10$. Substituting k back into our equation gives:

$$\begin{aligned} T(n) &= 2^{n-10} T(10) + (2^{n-10} - 1)c \\ &= 2^n \cdot 2^{-10} \cdot O(1) + 2^n \cdot 2^{-10} \cdot c - c \end{aligned}$$

Since 2^{-10} , $T(10)$, and c are constants, the growth rate is dominated entirely by the exponential term 2^n .

Complexity Analysis. The tightest worst-case running time bound is $\mathbf{O(2^n)}$.

Q12. Suppose that you observe that a program takes 30 seconds to complete on inputs of size 600, 4.5 minutes on inputs of size 1800, and 20 minutes on inputs of size 2000. Develop a reasonable assumption for the order of growth of the running time. **(10 marks)** [CSE 203 | 2018–19]

Answer

Convert times: 600 s $\rightarrow$ 30 s; 1800 s $\rightarrow$ 270 s; 2000 s $\rightarrow$ 1200 s.

Check $O(n^2)$: If $T(n) = cn^2$, then $\frac{T(1800)}{T(600)} = \frac{1800^2}{600^2} = 9$. Observed ratio: $\frac{270}{30} = 9$. ✓

Check $\frac{T(2000)}{T(600)} = \frac{2000^2}{600^2} \approx 11.1$. Observed: $\frac{1200}{30} = 40$. ✗

Check $O(n^3)$: $\frac{T(1800)}{T(600)} = 3^3 = 27 \neq 9$.

Try $O(n^2 \log n)$: $\frac{1800^2 \log 1800}{600^2 \log 600} \approx 9 \cdot \frac{10.74}{9.23} \approx 10.5$. Still not 9.

The ratio $T(600) : T(1800) = 1 : 9$ matches $O(n^2)$ exactly, while the ratio at 2000 is anomalous (possibly different cache/OS effects). A **reasonable assumption** is $T(n) = \Theta(n^2)$.

Q13. What is the difference between Big- O (O) and Little- o (o) asymptotic notations? Find appropriate $f(n)$ and $g(n)$ (if any) for each of the following cases:

- (a) $f(n) = O(g(n))$, but $g(n) \neq O(f(n))$
- (b) $f(n) = O(g(n))$, and $f(n) = \Omega(g(n))$
- (c) $f(n) = \Omega(g(n))$, and $f(n) = \omega(g(n))$
- (d) $f(n) = \Theta(g(n))$, and $f(n) = o(g(n))$

(5+10 marks)

[CSE 203 | 2017–18]

Answer

Big- $\mathcal{O}$ vs Little- o : $f = \mathcal{O}(g)$ means f grows *at most as fast* as g (non-strict upper bound; $f = g$ is allowed). $f = o(g)$ means f grows *strictly slower* than g : $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

- (a) $f = \mathcal{O}(g)$ but $g \neq \mathcal{O}(f)$: e.g. $f(n) = n$, $g(n) = n^2$. $n = \mathcal{O}(n^2)$ but $n^2 \neq \mathcal{O}(n)$.
- (b) $f = \mathcal{O}(g)$ and $f = \Omega(g)$ (i.e. $f = \Theta(g)$): e.g. $f(n) = 2n$, $g(n) = n$.
- (c) $f = \Omega(g)$ and $f = \omega(g)$: **Impossible**. $f = \Omega(g)$ means f grows at least as fast as g ; $f = \omega(g)$ (strict) also requires f to grow strictly faster. Together they mean $\lim f/g = \infty$ which is consistent — e.g. $f = n^2$, $g = n$: $n^2 = \Omega(n)$ and $n^2 = \omega(n)$.
- (d) $f = \Theta(g)$ and $f = o(g)$: **Impossible**. $f = \Theta(g)$ implies $\lim f/g = c > 0$; $f = o(g)$ implies $\lim f/g = 0$. Contradiction.

Q14. Define Big- $\mathcal{O}$, Ω , and Θ notations. Arrange the following functions in ascending order of growth rate (i.e., if $f(n)$ precedes $g(n)$, then $f(n) = \mathcal{O}(g(n))$). Provide justifications.

$$f_1 = n, \quad f_2 = \log_2 n, \quad f_3 = 2^{\sqrt{\log_2 n}}, \quad f_4 = n \ln n, \quad f_5 = 2^n$$

(6+9 marks)

[CSE 203 | 2016–17]

Answer**Definitions:**

- $f(n) = \mathcal{O}(g(n))$: $\exists c > 0, n_0$ s.t. $0 \leq f(n) \leq cg(n) \forall n \geq n_0$. (upper bound)
- $f(n) = \Omega(g(n))$: $\exists c > 0, n_0$ s.t. $0 \leq cg(n) \leq f(n) \forall n \geq n_0$. (lower bound)
- $f(n) = \Theta(g(n))$: $f = \mathcal{O}(g)$ and $f = \Omega(g)$. (tight bound)

Ascending order of growth:

$$f_2 = \log_2 n < f_3 = 2^{\sqrt{\log_2 n}} < f_1 = n < f_4 = n \ln n < f_5 = 2^n$$

Justification: $\log n \ll 2^{\sqrt{\log n}}$ (since $\sqrt{\log n} \rightarrow \infty$ but slower than $\log n$); $2^{\sqrt{\log n}} \ll n$ (take logs: $\sqrt{\log n} \ln 2 \ll \ln n$); $n \ll n \ln n \ll 2^n$ (standard hierarchy).

Q15. Prove that $15n^4 + 13n^2 + 11n = \mathcal{O}(n^4)$, but $15n^4 + 13n^2 + 11n \neq \mathcal{O}(n^2)$. (10 marks)
[CSE 203 | 2015–16]

Answer

$15n^4 + 13n^2 + 11n = O(n^4)$: For $n \geq 1$: $13n^2 \leq 13n^4$ and $11n \leq 11n^4$, so $15n^4 + 13n^2 + 11n \leq 39n^4$. Choose $c = 39$, $n_0 = 1$. $\square$

$15n^4 + 13n^2 + 11n \neq O(n^2)$: If it were $O(n^2)$, then $\exists c$ with $15n^4 + 13n^2 + 11n \leq cn^2$ for large n , giving $15n^2 \leq c - 13 - 11/n$. But $15n^2 \rightarrow \infty$, contradiction. $\square$

Q16. For each of the following pairs indicate whether $f(n)$ is $O(g(n))$, $\Omega(g(n))$ or both (i.e. $\Theta(g(n))$). Provide brief justifications

$f(n)$	$g(n)$
$\sqrt{n}$	$\log_2 n$
$n^{1.01}$	$n \log_2^2 n$
2^n	2^{n+1}
$n!$	2^n
n^{100}	1.01^n
$\sum_{i=1}^n i^k$	n^{k+1}

(6+9=15 marks)

[CSE 207 | 2015–16]

Answer

$f(n)$ vs $g(n)$	Relation	Justification
$\sqrt{n}$ vs $\log_2 n$	Ω	$\sqrt{n}/\log n \rightarrow \infty$; $\sqrt{n}$ grows faster
$n^{1.01}$ vs $n \log_2^2 n$	Ω	$n^{1.01}/(n \log^2 n) = n^{0.01}/\log^2 n \rightarrow \infty$
2^n vs 2^{n+1}	Θ	$2^{n+1} = 2 \cdot 2^n$; constant factor difference
$n!$ vs 2^n	Ω	Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n \gg 2^n$
n^{100} vs 1.01^n	O	Exponential always dominates polynomial
$\sum_{i=1}^n i^k$ vs n^{k+1}	Θ	$\sum_{i=1}^n i^k \sim \frac{n^{k+1}}{k+1}$

Q17. What is asymptotic analysis of algorithms? Define Big- O , Ω , and Θ notations. For the functions $f(n) = n \log n$ and $g(n) = 10n \log_{10} n$, show whether f is $O(g)$, or f is $\Omega(g)$, or f is $\Theta(g)$. (3+6+8 marks)

[CSE 207 | 2014–15]

Answer

Asymptotic analysis studies the behaviour of algorithms as input size $n \rightarrow \infty$, ignoring constants and lower-order terms to focus on the dominant growth factor.

Definitions: (same as above — O , Ω , Θ)

Comparing $f(n) = n \log_2 n$ and $g(n) = 10n \log_{10} n$:

Using change of base: $\log_{10} n = \frac{\log_2 n}{\log_2 10} = \frac{\log_2 n}{3.32}$.

So $g(n) = 10n \cdot \frac{\log_2 n}{\log_2 10} = \frac{10}{\log_2 10} n \log_2 n \approx \frac{10}{3.32} n \log_2 n \approx 3.01 f(n)$.

Since $g(n) = c \cdot f(n)$ for constant $c \approx 3.01$, we have:

$$f(n) = \Theta(g(n))$$

(and hence also $f = O(g)$ and $f = \Omega(g)$).

Q18. Prove that $5n^3 + 3n = O(n^4)$, but $5n^3 + 3n \neq O(n^2)$. (5+5=10 marks) [CSE 203 | 2012–13]

Answer

Part 1 — $5n^3 + 3n = O(n^4)$: For $n \geq 1$: $5n^3 + 3n \leq 5n^4 + 3n^4 = 8n^4$. Choose $c = 8$, $n_0 = 1$. Then $5n^3 + 3n \leq 8n^4$ for all $n \geq 1$. By definition, $5n^3 + 3n = O(n^4)$. $\square$

Part 2 — $5n^3 + 3n \neq O(n^2)$: Suppose for contradiction that $5n^3 + 3n = O(n^2)$. Then $\exists c, n_0$ such that $5n^3 + 3n \leq cn^2$ for all $n \geq n_0$. Dividing by n^2 : $5n + 3/n \leq c$. But $5n \rightarrow \infty$ as $n \rightarrow \infty$, so no constant c can bound $5n$ for all large n . Contradiction. Hence $5n^3 + 3n \neq O(n^2)$. $\square$

Q19. Prove that $O(n^3) \cdot O(n) = O(n^4)$. (5 marks)

[CSE 203 | 2011–12]

Answer

Let $f(n) \in O(n^3)$ and $g(n) \in O(n)$. Then there exist constants $c_1, c_2 > 0$ and $n_1, n_2 \geq 0$ such that $f(n) \leq c_1 n^3$ for all $n \geq n_1$ and $g(n) \leq c_2 n$ for all $n \geq n_2$. For $n \geq \max(n_1, n_2)$:

$$f(n) \cdot g(n) \leq c_1 n^3 \cdot c_2 n = (c_1 c_2) n^4$$

Setting $c = c_1 c_2$ and $n_0 = \max(n_1, n_2)$, we have $f(n)g(n) \leq c n^4$ for all $n \geq n_0$. By definition, $f(n) \cdot g(n) \in O(n^4)$. Hence $O(n^3) \cdot O(n) = O(n^4)$. $\square$

Q20. Write the main three reasons why we usually concentrate on finding the worst-case running time of an algorithm. (5 marks)

[CSE 203 | 2011–12]

Answer

- 1. Guaranteed upper bound:** The worst-case gives an absolute guarantee on performance; we know the algorithm will *never* take longer than this bound for any input of size n .
- 2. Worst case often occurs in practice:** For many algorithms the worst case arises frequently (e.g. searching for an element not in a list triggers worst-case sequential search every time).
- 3. Average case is hard to compute:** Computing average-case complexity requires knowing the probability distribution of inputs, which is often unknown or difficult to model. Worst-case analysis avoids this assumption.

Q21. Why is $n/2 \neq o(n)$? (5 marks)

[CSE 207 | 2009–10]

Answer

By definition, $f(n) = o(g(n))$ iff $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

For $f(n) = n/2$ and $g(n) = n$:

$$\lim_{n \rightarrow \infty} \frac{n/2}{n} = \lim_{n \rightarrow \infty} \frac{1}{2} = \frac{1}{2} \neq 0$$

Since the limit is a nonzero constant, $n/2$ is **not** $o(n)$. In fact, $n/2 = \Theta(n)$ — it is a tight bound, not a strict lower-order term. Little- o requires the ratio to go to zero; here it is constant.

Q22. Write down an algorithm for searching z sequentially in array A with n elements. Discuss its average-case, worst-case, and best-case complexities. How can this algorithm be improved? **(5+10+5=20 marks)** [CSE 207 | 2008–09]

Answer**Algorithm 8** SEQUENTIALSEARCH(A, n, z)

```

1: for  $i = 1$  to  $n$  do
2:   if  $A[i] = z$  then
3:     return  $i$ 
4:   end if
5: end for
6: return NOT_FOUND

```

Best case: $z = A[1] \Rightarrow 1$ comparison $= \Theta(1)$. **Worst case:** $z = A[n]$ or $z \notin A \Rightarrow n$ comparisons $= \Theta(n)$. **Average case (successful):** $\frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n)$.

Improvements:

- (i) **Self-organising search:** Move the found element to the front (move-to-front or transpose heuristic). Frequently accessed elements migrate forward, reducing average search time over repeated queries.
- (ii) **Sentinel search:** Place z at position $A[n+1]$ (sentinel) and remove the end-of-array check, halving the number of comparisons per iteration.
- (iii) **Sort + Binary search:** If multiple searches are needed on static data, sort in $O(n \log n)$ then search in $O(\log n)$ each.

Q23. Define big $\mathcal{O}$ -notation, Ω -notation and Θ -notation.

Answer

Big-O: $f(n) = O(g(n))$ iff $\exists c > 0, n_0 \geq 0$ such that $f(n) \leq cg(n) \forall n \geq n_0$.

Big- Ω : $f(n) = \Omega(g(n))$ iff $\exists c > 0, n_0 \geq 0$ such that $f(n) \geq cg(n) \forall n \geq n_0$.

Big- Θ : $f(n) = \Theta(g(n))$ iff $f = O(g)$ and $f = \Omega(g)$.

BUET · CSE 105

DSA

Correctness Proof & Algorithm Analysis

Master Theorem, Recurrence, Substitution, Loop Invariant

T3 — Correctness Proof & Algorithm Analysis

Q1. Analyze the following recurrences using the Master Theorem:

(a) $T(n) = 2T(n/2) + n \log n$

(b) $T(n) = 2T(n/2) + n^2$

(5+5=10 marks)

[CSE 105 | 2023–24]

Answer

Master Theorem: For $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$:

(a) $T(n) = 2T(n/2) + n \log n$: $a = 2, b = 2, n^{\log_b a} = n^1 = n$. $f(n) = n \log n$. We need to check if $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$. Here $f(n) = n \log n = n^1 \cdot \log^1 n$, so $k = 1$. This satisfies Case 2 (extended): $T(n) = \Theta(n \log^2 n)$.

(b) $T(n) = 2T(n/2) + n^2$: $a = 2, b = 2, n^{\log_b a} = n$. $f(n) = n^2$. Since $n^2 = \Omega(n^{1+\varepsilon})$ for $\varepsilon = 1$, and regularity holds ($2(n/2)^2 = n^2/2 \leq cn^2$), Case 3 applies: $T(n) = \Theta(n^2)$.

Q2. Using the Master Theorem, prove that Strassen's algorithm improves the runtime of multiplying two $n \times n$ matrices compared to the naive approach with time complexity $O(n^3)$. (10 marks)

[CSE 105 | 2023–24]

Answer

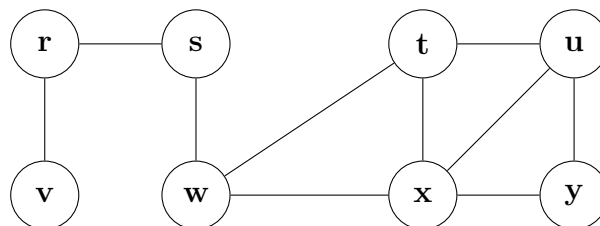
Strassen's recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$. $a = 7, b = 2, n^{\log_2 7} \approx n^{2.807}$. Since $f(n) = \Theta(n^2) = O(n^{2.807-\varepsilon})$ for $\varepsilon = 0.807$, Case 1 applies:

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Since $2.807 < 3$, we have $\lim_{n \rightarrow \infty} \frac{n^{2.807}}{n^3} = 0$, i.e. $n^{2.807} = o(n^3)$. Therefore Strassen's algorithm is **asymptotically faster** than the naive $\Theta(n^3)$ approach. $\square$

Q3. State the correctness theorem of Breadth-First Search (BFS). Use this theorem to judge whether the value $u.d$ assigned to a vertex u in BFS depends on the order of appearance of the vertices in the corresponding adjacency list. Demonstrate with an example graph whether the breadth-first tree produced by BFS depends on the ordering of vertices in the adjacency lists if the same source vertex is used in every ordering. (17 marks)

[CSE 105 | 2022–23]



Answer**1. Correctness Theorem of BFS**

Theorem (CLRS Theorem 22.5): Let $G = (V, E)$ be a directed or undirected graph, and suppose BFS is run on G from a given source vertex $s \in V$. Then, upon termination, for every reachable vertex $v \in V$, the value $v.d$ computed by BFS satisfies $v.d = \delta(s, v)$, where $\delta(s, v)$ is the shortest-path distance from s to v . Furthermore, the predecessor subgraph forms a breadth-first tree rooted at s .

Proof Outline (by Contradiction): Assume, for the sake of contradiction, that BFS computes an incorrect distance for some vertex. Let v be the reachable vertex with the minimum true shortest-path distance $\delta(s, v)$ that receives an incorrect distance value. Since $v \neq s$ (as $s.d = 0$ is trivially correct), v must have some predecessor u on the actual shortest path from s to v . Thus, $\delta(s, v) = \delta(s, u) + 1$. Because $\delta(s, u) < \delta(s, v)$, by our assumption of v , vertex u must have received the correct distance: $u.d = \delta(s, u)$. Consider the moment when u is dequeued and its adjacency list is explored. What happens to v ?

- If v is **white** (undiscovered), it is immediately assigned $v.d = u.d + 1 = \delta(s, u) + 1 = \delta(s, v)$, which contradicts our assumption that $v.d$ is incorrect.
- If v is **gray or black**, it means v was already discovered by some other node before or during u 's processing. This implies v was assigned a distance $v.d \leq u.d + 1 = \delta(s, v)$. Since a path can never be shorter than the shortest path ($\delta(s, v)$), $v.d$ must be exactly equal to $\delta(s, v)$, which again contradicts our assumption.

Hence, BFS correctly computes $v.d = \delta(s, v)$ for all reachable vertices. (Proved)

2. Does $u.d$ depend on the adjacency list order?

No, $u.d$ does not depend on the order. According to the correctness theorem, upon termination, $u.d$ strictly equals $\delta(s, u)$. The shortest-path distance $\delta(s, u)$ is an inherent mathematical property of the graph G and the source s . It represents the minimum number of edges required to reach u from s . Because the topological structure of the graph does not change regardless of how the edges are stored or ordered in the adjacency list, the value of $\delta(s, u)$, and consequently $u.d$, remains absolutely fixed.

3. Does the BFS tree depend on the adjacency list order?

Yes, the BFS tree completely depends on the ordering. When multiple shortest paths exist to a vertex u , the parent $u.\pi$ (which forms the tree edges) is determined by whichever path discovers u *first*. This discovery order is directly controlled by the adjacency list ordering.

Demonstration with the given graph: Let us start BFS from source vertex s . Notice that s connects to w . Vertex w connects to t and x . Both t and x connect to u . The distances will be: $w.d = 1$, $t.d = 2$, $x.d = 2$, and $u.d = 3$. Let's see what happens to the parent of u ($u.\pi$) under two different orderings of $Adj[w]$:

Case 1: $Adj[w] = [t, x]$

- When w is dequeued, t is enqueued before x .
- The queue state becomes: $[\dots, t, x]$.

- t is dequeued before x . Since $u \in Adj[t]$ and u is undiscovered (white), t discovers u .

- **Result:** $u.\pi = t$

Case 2: $Adj[w] = [x, t]$

- When w is dequeued, x is enqueued before t .
- The queue state becomes: $[\dots, x, t]$.
- x is dequeued before t . Since $u \in Adj[x]$ and u is undiscovered, x discovers u .
- **Result:** $u.\pi = x$

Conclusion: Because the parent pointers (π) differ based on the adjacency list order, the resulting BFS trees are structurally different, even though the source s and all distance values (d) remain exactly the same.

Q4. “It is asymptotically faster to square an n -bit integer than to multiply two n -bit integers.” Do you agree with this statement? Justify your answer. **(10 marks)**
[CSE 203 | 2021–22]

Answer

Disagree. Squaring and general multiplication are asymptotically equivalent.

Squaring $\leq$ Multiplication: $A^2 = A \times A$, so $T_{sq}(n) \leq T_{mult}(n)$.

Multiplication $\leq$ Squaring: Using the identity $A \times B = \frac{(A+B)^2 - (A-B)^2}{4}$, any multiplication can be reduced to a constant number of squarings plus $O(n)$ additions. So $T_{mult}(n) = O(T_{sq}(n))$.

Combining: $T_{sq}(n) = \Theta(T_{mult}(n))$. They are asymptotically the same complexity class.

Q5. Using the substitution method, solve the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + 100n^2$$

(11 marks)

[CSE 203 | 2020–21]

Answer

Guess: $T(n) = O(n^2 \log n)$. We guess $T(n) \leq cn^2 \log n$ for some constant $c > 0$.

Inductive step: Assume $T(k) \leq ck^2 \log k$ for all $k < n$. Then:

$$\begin{aligned} T(n) &= 4T(n/2) + 100n^2 \\ &\leq 4 \cdot c(n/2)^2 \log(n/2) + 100n^2 \\ &= 4 \cdot \frac{cn^2}{4} (\log n - 1) + 100n^2 \\ &= cn^2 \log n - cn^2 + 100n^2 \\ &= cn^2 \log n + (100 - c)n^2 \end{aligned}$$

For $c \geq 100$: $(100 - c)n^2 \leq 0$, so $T(n) \leq cn^2 \log n$. ✓

Lower bound: By symmetric argument, $T(n) = \Omega(n^2 \log n)$.

Conclusion: $T(n) = \Theta(n^2 \log n)$.

Note: This aligns with Master Theorem Case 2 since $n^{\log_2 4} = n^2$ and $f(n) = 100n^2 = \Theta(n^2)$.

- Q6.** Explain whether we can apply the Master Method to any recurrence of the form $T(n) = aT(n/b) + f(n)$. If not, explain a case where the theorem cannot be applied. (8 marks) [CSE 203 | 2019–20]

Answer

No, the Master Method cannot be applied to all recurrences of the form $T(n) = aT(n/b) + f(n)$.

The Master Theorem requires $a \geq 1$, $b > 1$, and that $f(n)$ be asymptotically comparable to $n^{\log_b a}$ in one of three specific ways. It fails when there is a **gap** between $f(n)$ and $n^{\log_b a}$.

Example where it fails: $T(n) = 2T(n/2) + n \log n$. Here $n^{\log_2 2} = n$ and $f(n) = n \log n$. $f(n)/n^{\log_b a} = \log n$, which is not a polynomial factor. $f(n) = \Omega(n^{1+\epsilon})$ is not satisfied for any $\epsilon > 0$. $f(n) = \Theta(n)$ is also not satisfied. So none of the three cases apply. (Solution is $T(n) = \Theta(n \log^2 n)$ via the Akra-Bazzi method or extended Master Theorem.)

- Q7.** Traditional insertion sort uses a linear search to scan (backward) through the sorted subarray $A[1 \dots j-1]$, where $A[j]$ is the element to be inserted. Prove or disprove the claim that we can use binary search instead to improve the overall worst-case running time of insertion sort to $\Theta(n \log n)$. (8 marks) [CSE 203 | 2019–20]

Answer

Claim is FALSE. Using binary search reduces the number of *comparisons* to $O(\log j)$ for the j -th insertion, giving $\sum_{j=2}^n O(\log j) = O(n \log n)$ comparisons total. However, Insertion Sort's bottleneck in the worst case is **element shifting (swaps)**, not comparisons.

After finding the correct position via binary search, we still need to shift all larger elements one position right to make room. In the worst case (reverse-sorted input), inserting element j requires shifting $j-1$ elements. Total shifts: $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2} = \Theta(n^2)$.

Therefore, the worst-case running time of Insertion Sort with binary search is still $\Theta(n^2)$. The claim is **disproved**.

- Q8.** Prove or disprove the following: if we choose the element at the middle index as pivot during partition in Quicksort, the algorithm will never have a runtime of $\Theta(n^2)$. (10+8 marks) [CSE 203 | 2019–20]

Answer

Claim is FALSE. The middle-index pivot does *not* guarantee $O(n \log n)$.

Counterexample: Consider the array that causes worst-case behaviour for middle-index pivot. One such family: the “median-of-three killer” sequence. For example, for $n = 8$, the array $[1, 5, 3, 7, 2, 6, 4, 8]$ constructed so that middle-index selection always picks the maximum or minimum of the current subarray.

General construction: For any deterministic pivot-selection rule (including middle index), there exists an adversarial permutation that causes the partition to always split as $(n-1, 0)$ or $(0, n-1)$. This gives the recurrence $T(n) = T(n-1) + \Theta(n)$, which solves to $T(n) = \Theta(n^2)$.

Therefore middle-index pivot selection still admits $\Theta(n^2)$ worst-case for adversarially constructed inputs. Only **randomised** pivot selection avoids this with high probability.

Q9. Consider a divide-and-conquer algorithm: if the problem size is ≤ 5 , it solves the problem in constant time; otherwise, it divides the problem of size n into 4 sub-problems each of size $n/3$, solves them recursively, and merges in $O(n)$ time. The division itself takes $O(\sqrt{n} \log n)$ time. Write the recurrence relation for $T(n)$ and solve the following recurrences using the Master Theorem:

- (a) $T(n) = 9T(n/3) + 5n + n^2 + 8 \log n$
- (b) $T(n) = 3T(n/4) + 3n^2 + 4n + 5$
- (c) $T(n) = 4T(n/2) + 5n^3 + 7n$

(10 marks)

[CSE 203 | 2018–19]

Answer

Recurrence: $T(n) = 4T(n/3) + O(\sqrt{n} \log n) + O(n) = 4T(n/3) + O(n)$ (since n dominates $\sqrt{n} \log n$).

$n^{\log_3 4} \approx n^{1.26}$. $f(n) = O(n) = O(n^{1.26-0.26})$, Case 1: $T(n) = \Theta(n^{\log_3 4})$.

Sub-recurrences:

- (a) $T(n) = 9T(n/3) + 5n + n^2 + 8 \log n$. Dominant $f(n) = n^2$. $n^{\log_3 9} = n^2$. $f(n) = \Theta(n^2)$, Case 2: $T(n) = \Theta(n^2 \log n)$.
- (b) $T(n) = 3T(n/4) + 3n^2 + 4n + 5$. $f(n) = 3n^2$. $n^{\log_4 3} \approx n^{0.79}$. $n^2 = \Omega(n^{0.79+\epsilon})$, regularity holds, Case 3: $T(n) = \Theta(n^2)$.
- (c) $T(n) = 4T(n/2) + 5n^3 + 7n$. $f(n) = 5n^3$. $n^{\log_2 4} = n^2$. $n^3 = \Omega(n^{2+1})$, regularity: $4(n/2)^3 = n^3/2 \leq cn^3$, Case 3: $T(n) = \Theta(n^3)$.

Q10. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

- (a) Algorithm A (Strassen’s) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes seven recursive calls, and combines them in $\Theta(n^2)$ time.
- (b) Algorithm B divides each matrix into twelve $n/3 \times n/3$ matrices, makes nine

recursive calls, and combines them in $\Theta(n^2)$ time.

- (c) Algorithm C first multiplies two $(n-1) \times (n-1)$ matrices, then combines the result with two vectors of size n in $\Theta(n^2)$ time.

Determine the asymptotic running time of each algorithm (use the Master Theorem where applicable) and determine which algorithm you would choose. **(15 marks)**

[CSE 203 | 2016–17]

Answer

- (a) **Algorithm A (7 calls, $n/2$, combine $\Theta(n^2)$):** $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.807}$. Since $n^2 = O(n^{2.807-\epsilon})$, Case 1: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.
- (b) **Algorithm B (9 calls, $n/3$):** $T(n) = 9T(n/3) + \Theta(n^2)$. $n^{\log_3 9} = n^2$. $f(n) = \Theta(n^2) = \Theta(n^{\log_3 9})$, Case 2: $T(n) = \Theta(n^2 \log n)$.
- (c) **Algorithm C ($T(n-1)$ once, combine $\Theta(n^2)$):** $T(n) = T(n-1) + \Theta(n^2)$. Master Theorem does not apply (not of form $T(n/b)$). Unrolling: $T(n) = \sum_{k=1}^n k^2 = \Theta(n^3)$.

Best choice: Algorithm A with $\Theta(n^{2.807})$ (Strassen's), faster than $\Theta(n^3)$ naively and $\Theta(n^3)$ of C.

Q11. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

- (a) Algorithm A divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 8 recursive calls, and combines in $\Theta(n^2)$ time.
- (b) Algorithm B (Strassen's) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 7 recursive calls, and combines in $\Theta(n^2)$ time.
- (c) Algorithm C divides each $n \times n$ matrix into nine $n/3 \times n/3$ matrices, makes 9 recursive calls, and combines in $\Theta(n^2)$ time.

Use the Master Theorem to determine asymptotic running times of all three algorithms and choose the best one. **(15 marks)**

[CSE 207 | 2015–16]

Answer

- (a) $T(n) = 8T(n/2) + \Theta(n^2)$: $n^{\log_2 8} = n^3$. $n^2 = O(n^{3-1})$ (Case 1): $T(n) = \Theta(n^3)$.
- (b) $T(n) = 7T(n/2) + \Theta(n^2)$: $n^{\log_2 7} \approx n^{2.807}$. Case 1: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.
- (c) $T(n) = 9T(n/3) + \Theta(n^2)$: $n^{\log_3 9} = n^2$. Case 2: $T(n) = \Theta(n^2 \log n)$.

Best: Algorithm B (Strassen's) at $\Theta(n^{2.807})$ — faster than n^3 and $n^2 \log n$.

Q12. State the Master Theorem. Explain why the Master Method cannot be used to solve the recurrence $T(n) = 2T(n/2) + n \log n$. Using the substitution method, prove that $T(n)$ is $O(n \log^2 n)$. **(6+6+6 marks)**

[CSE 207 | 2014–15]

Answer

Master Theorem: For $T(n) = aT(n/b) + f(n)$ with $a \geq 1, b > 1$:

Case 1: If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$: $T(n) = \Theta(n^{\log_b a})$.

Case 2: If $f(n) = \Theta(n^{\log_b a})$: $T(n) = \Theta(n^{\log_b a} \log n)$.

Case 3: If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and regularity holds: $T(n) = \Theta(f(n))$.

Why Master Method fails for $T(n) = 2T(n/2) + n \log n$: $n^{\log_2 2} = n$. $f(n) = n \log n$. For Case 2: $n \log n \neq \Theta(n)$. For Case 3: we need $n \log n = \Omega(n^{1+\varepsilon})$, but $\log n$ is not $\Omega(n^\varepsilon)$ for any $\varepsilon > 0$. The ratio $f(n)/n^{\log_b a} = \log n$ is not polynomial. None of the three cases apply.

Substitution proof that $T(n) = O(n \log^2 n)$: Guess $T(n) \leq cn \log^2 n$. Assume true for $n/2$:

$$\begin{aligned} T(n) &\leq 2c \frac{n}{2} \log^2 \frac{n}{2} + n \log n = cn(\log n - 1)^2 + n \log n \\ &= cn \log^2 n - 2cn \log n + cn + n \log n \\ &= cn \log^2 n + n \log n(1 - 2c) + cn \\ &\leq cn \log^2 n \quad \text{for } c \geq 1 \text{ and large } n \end{aligned}$$

Hence $T(n) = O(n \log^2 n)$. □

Q13. Using the Master Method, prove that the recurrence $T(n) = 3T(n/4) + n \lg n$ solves to $T(n) = \Theta(n \lg n)$. **(12 marks)** [CSE 207 | 2009–10]

Answer

For $T(n) = 3T(n/4) + n \lg n$: $a = 3, b = 4, f(n) = n \lg n$.

$n^{\log_b a} = n^{\log_4 3} = n^{0.792\dots}$.

Since $f(n) = n \lg n = \Omega(n^{0.792+\varepsilon})$ for $\varepsilon \leq 0.208$ (as $n \lg n = \Omega(n^{0.9})$ for example since $\lg n = \Omega(n^{0.1})$ is false...let's be precise):

$f(n)/n^{\log_4 3} = n^{1-\log_4 3} \cdot \lg n = n^{0.208} \cdot \lg n \rightarrow \infty$. So $f(n) = \Omega(n^{\log_4 3 + \varepsilon})$ for $\varepsilon = 0.1$.

Regularity: $af(n/b) = 3 \cdot \frac{n}{4} \lg \frac{n}{4} = \frac{3n}{4}(\lg n - 2) \leq \frac{3}{4}n \lg n = cf(n)$ for $c = 3/4 < 1$.

✓

Case 3 applies: $T(n) = \Theta(f(n)) = \Theta(n \lg n)$. □

Q14. Write the pseudocode of Strassen's algorithm for multiplying two $n \times n$ matrices. Derive the running time. **(8+10=18 marks)** [CSE 207 | 2009–10]

Answer

Algorithm 9 STRASSEN(A, B, n)

```

1: if  $n = 1$  then
2:   return  $A[1][1] \times B[1][1]$ 
3: end if
4:  $A_{11}, A_{12}, A_{21}, A_{22} \leftarrow \text{PARTITION}(A)$ 
5:  $B_{11}, B_{12}, B_{21}, B_{22} \leftarrow \text{PARTITION}(B)$ 
6:  $M_1 \leftarrow \text{STRASSEN}(A_{11} + A_{22}, B_{11} + B_{22}, n/2)$ 
7:  $M_2 \leftarrow \text{STRASSEN}(A_{21} + A_{22}, B_{11}, n/2)$ 
8:  $M_3 \leftarrow \text{STRASSEN}(A_{11}, B_{12} - B_{22}, n/2)$ 
9:  $M_4 \leftarrow \text{STRASSEN}(A_{22}, B_{21} - B_{11}, n/2)$ 
10:  $M_5 \leftarrow \text{STRASSEN}(A_{11} + A_{12}, B_{22}, n/2)$ 
11:  $M_6 \leftarrow \text{STRASSEN}(A_{21} - A_{11}, B_{11} + B_{12}, n/2)$ 
12:  $M_7 \leftarrow \text{STRASSEN}(A_{12} - A_{22}, B_{21} + B_{22}, n/2)$ 
13:  $C_{11} \leftarrow M_1 + M_4 - M_5 + M_7$ 
14:  $C_{12} \leftarrow M_3 + M_5$ 
15:  $C_{21} \leftarrow M_2 + M_4$ 
16:  $C_{22} \leftarrow M_1 - M_2 + M_3 + M_6$ 
17: return ASSEMBLE( $C_{11}, C_{12}, C_{21}, C_{22}$ )

```

Recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.807}$. Case 1 of Master Theorem: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$, improving on naive $\Theta(n^3)$.

BUET · CSE 105

DSA

Arrays, Linked Lists, Stacks & Queues

Linear Data Structures, Implementation, Operations

T4 — Arrays, Linked Lists, Stacks & Queues

- Q1.** You are asked to merge two singly linked lists whose elements are sorted in ascending order. The output list will also be sorted in ascending order. Sketch an algorithm to solve the problem. Analyze that your algorithm runs in linear time on the number of elements of the output list. **(15 marks)** [CSE 105 | 2023–24]

Answer — Merge Two Sorted Linked Lists

Idea. Maintain a pointer into each list; at each step, advance the pointer pointing to the smaller current node and append that node to the output list.

Algorithm 10 MERGESORTED(L_1, L_2)

```

1: Create a dummy head node;  $tail \leftarrow dummy$ 
2:  $p_1 \leftarrow L_1.head$ ;  $p_2 \leftarrow L_2.head$ 
3: while  $p_1 \neq \text{null}$  and  $p_2 \neq \text{null}$  do
4:   if  $p_1.data \leq p_2.data$  then
5:      $tail.next \leftarrow p_1$ 
6:      $p_1 \leftarrow p_1.next$ 
7:   else
8:      $tail.next \leftarrow p_2$ 
9:      $p_2 \leftarrow p_2.next$ 
10:  end if
11:   $tail \leftarrow tail.next$ 
12: end while
13: if  $p_1 \neq \text{null}$  then
14:    $tail.next \leftarrow p_1$ 
15: end if
16: if  $p_2 \neq \text{null}$  then
17:    $tail.next \leftarrow p_2$ 
18: end if
19: return  $dummy.next$ 

```

Trace ($L_1 = [1, 3, 5, 7]$, $L_2 = [2, 4, 6, 8, 9]$):

Step	p_1	p_2	Output so far
1	1	2	1
2	3	2	$1 \rightarrow 2$
3	3	4	$1 \rightarrow 2 \rightarrow 3$
4	5	4	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
5	5	6	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
6	7	6	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$
7	7	8	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
attach	NULL	$8 \rightarrow 9$	$1 \rightarrow 2 \rightarrow \dots \rightarrow 7 \rightarrow 8 \rightarrow 9$

Correctness. At each step the smallest remaining element across both lists is appended, preserving sorted order. When one list empties the other is already sorted, so appending its tail is safe.

Time complexity analysis.

Let $m = |L_1|$, $n = |L_2|$, and $N = m + n = \text{size of the output list}$.

- Each iteration of the **while** loop appends *exactly one* node to the output and advances *exactly one* pointer.
- The loop runs at most $m + n - 1$ times before one list empties; then the tail-attach takes $O(1)$.
- Total work $= O(m + n) = O(N)$.

$$T(m, n) = O(m + n) = O(N).$$

The algorithm is **linear in the size of the output list**. ✓

Q2. You are given two linked lists of unsorted elements. Investigate whether you can develop a constant time algorithm to concatenate them if both of the lists are (i) singly linked lists (SLL), (ii) doubly linked list (DLL), (iii) circular SLL, (iv) circular DLL. If yes, sketch the algorithm and examine its correctness; otherwise demonstrate why a constant time algorithm is not possible. **(20 marks)** [CSE 105 | 2023–24]

Answer — Constant-Time Concatenation of Two Lists

Key insight. Concatenation is $O(1)$ whenever we can reach the *last node of the first list* and the *first node of the second list* in $O(1)$.

(i) Singly Linked List (SLL).

Without a tail pointer: We must traverse L_1 from **head1** to its last node — $O(n)$.

Constant time is not possible.

With a tail pointer: $O(1)$ is achievable.

Algorithm 11 CONCATENATE_SLL(L_1, L_2)

1: $tail_1.next \leftarrow head_2$

2: $tail_1 \leftarrow tail_2$

Correctness: **tail1.next** was NULL; now points to **head2**.

(ii) Doubly Linked List (DLL).

Without a tail pointer: Similar to SLL, finding the last node takes $O(n)$. **Constant time is not possible.**

With a tail pointer: $O(1)$ is achievable.

Algorithm 12 CONCATENATE_DLL(L_1, L_2)

```

1:  $tail_1.next \leftarrow head_2$ 
2:  $head_2.prev \leftarrow tail_1$ 
3:  $tail_1 \leftarrow tail_2$ 

```

Correctness: The bidirectional chain is restored at the join point.

(iii) Circular SLL.

In a circular SLL, we typically maintain only a **tail pointer** because **tail.next** gives us the head in $O(1)$. Thus, $O(1)$ concatenation is possible.

Algorithm 13 CONCATENATE_CSLL(L_1, L_2)

```

1:  $head_1 \leftarrow tail_1.next$ 
2:  $tail_1.next \leftarrow tail_2.next$ 
3:  $tail_2.next \leftarrow head_1$ 

```

Correctness: The new circle is: **head1** $\rightarrow \dots \rightarrow$ **tail1** $\rightarrow$ **head2** $\rightarrow \dots \rightarrow$ **tail2** $\rightarrow$ **head1**.

(iv) Circular DLL.

Constant time is ALWAYS possible, even with just a **head** pointer! In a CDLL, the tail is simply **head.prev**. We do not need to maintain a separate tail pointer.

Algorithm 14 CONCATENATE_CDLL(L_1, L_2)

```

1:  $tail_1 \leftarrow head_1.prev$ 
2:  $tail_2 \leftarrow head_2.prev$ 
3:  $tail_1.next \leftarrow head_2$ 
4:  $head_2.prev \leftarrow tail_1$ 
5:  $tail_2.next \leftarrow head_1$ 
6:  $head_1.prev \leftarrow tail_2$ 

```

Correctness: All four pointers at the join and closure points are correctly updated, preserving the circular doubly-linked property.

List Type	$O(1)$ possible?	Key requirement
SLL	No (w/o tail) / Yes (w/ tail)	Tail pointer needed
DLL	No (w/o tail) / Yes (w/ tail)	Tail pointer needed
Circular SLL	Yes	Tail pointer (gives head via tail.next)
Circular DLL	Yes	Just head pointer (gives tail via head.prev)

Q3. Assume that a string contains only left and right parentheses which may or may not be balanced and properly nested. For example, the string ``(((())())`` contains

properly nested pairs of parentheses, but the string ```()('` does not, and the string ```()('` does not contain properly matching parentheses. Using a stack, construct an algorithm that returns **true** if a string contains properly nested and balanced parentheses, and **false** otherwise. Modify the algorithm so that it returns the position in the string of the first offending parenthesis if the string is not properly nested and balanced — if an excess right parenthesis is found, return its position; if there are too many left parentheses, return the position of the first excess left parenthesis. Return **-1** if the string is properly balanced and nested. Assume indices start from 0. **(25 marks)** [CSE 105 | 2023–24]

Answer — Balanced Parentheses via Stack

Part A: Return true/false.

Algorithm 15 ISBALANCED(*s*)

```

1: stack ← empty stack
2: for each character ch in s do
3:   if ch = '(' then
4:     stack.PUSH(ch)
5:   else
6:     if stack.ISEMPTY() then
7:       return false
8:     end if
9:     stack.POP()
10:  end if
11: end for
12: return stack.ISEMPTY()

```

Part B: Return position of first offending parenthesis (or -1).

Key insight: Push the *index* (not the character) so we can report positions.

Algorithm 16 CHECKPOSITION(*s*)

```

1: stack ← empty stack
2: for i = 0 to len(s) − 1 do
3:   ch ← s[i]
4:   if ch = '(' then
5:     stack.PUSH(i)
6:   else
7:     if stack.ISEMPTY() then
8:       return i
9:     end if
10:    stack.POP()
11:  end if
12: end for
13: if stack is not empty then
14:   return stack.BOTTOM()
15: end if
16: return −1

```

Why *stack.bottom()*? Unmatched '(' accumulate bottom-up in push order. The *first* excess left parenthesis is the one pushed earliest, i.e., at the bottom. In practice, use an auxiliary variable `firstLeft` updated only when the stack was empty before this push.

Trace on "((()())())" (indices 0–8):

<i>i</i>	ch	Action	Stack (indices)
0	(	push 0	[0]
1	(	push 1	[0,1]
2	(	push 2	[0,1,2]
3	)	pop 2	[0,1]
4	(	push 4	[0,1,4]
5	)	pop 4	[0,1]
6	)	pop 1	[0]
7	(	push 7	[0,7]
8	)	pop 7	[0]
End			stack not empty bottom = 0

Return 0 (the '(' at index 0 is the first excess left parenthesis). ✓

Test cases:

String	Result	Reason
"((()())())"	0	Excess '(' at position 0
")()("	0	Excess ')' at position 0
"())"	2	Excess ')' at position 2
"((()())())"	−1	Balanced

Time complexity: $O(n)$ — each character processed exactly once. **Space complexity:** $O(n)$ — stack holds at most $n/2$ indices.

- Q4.** Given an integer k and a queue of integers, how do you reverse the order of the first k elements of the queue, leaving the other elements in the same relative order? For example, if $k = 4$ and the queue has the elements $[10, 20, 30, 40, 50, 60, 70, 80, 90]$, the output should be $[40, 30, 20, 10, 50, 60, 70, 80, 90]$. **(10 marks)** [CSE 105 | 2023–24]

Answer

Idea: Use an auxiliary stack to reverse the first k elements, then re-enqueue them. The remaining $n - k$ elements are rotated to the back and then brought to the front.

Algorithm 17 REVERSEFIRSTK(Q, k)

```

1:  $S \leftarrow$  empty stack
2: for  $i = 1$  to  $k$  do
3:    $S.PUSH(Q.DEQUEUE())$ 
4: end for
5: while  $S$  is not empty do
6:    $Q.ENQUEUE(S.POP())$ 
7: end while
8: for  $i = 1$  to  $Q.size - k$  do
9:    $Q.ENQUEUE(Q.DEQUEUE())$ 
10: end for
11: return  $Q$ 

```

Trace ($k = 4$, queue = $[10, 20, 30, 40, 50, 60, 70, 80, 90]$):

- (a) Dequeue 4 elements $\rightarrow$ Stack: $[40, 30, 20, 10]$ (top=40). Queue: $[50, 60, 70, 80, 90]$
- (b) Pop stack back to queue $\rightarrow$ Queue: $[50, 60, 70, 80, 90, 40, 30, 20, 10]$
- (c) Rotate remaining $9 - 4 = 5$ elements: move 50, 60, 70, 80, 90 to back $\rightarrow$ Queue: $[40, 30, 20, 10, 50, 60, 70, 80, 90]$ ✓

Time complexity: $O(n)$ where n is the total number of elements.

Space complexity: $O(k)$ for the auxiliary stack.

- Q5.** Implement a Circular Queue using an array. Include: `enqueue(x)`, `dequeue()`, `is_empty()`, `is_full()`, `front()`. **(15 marks)** [CSE 105 | 2022–23]

Answer

Representation. Use an array `arr[0..capacity-1]`, two indices `front` and `rear`, and a size counter `sz`. Indices wrap around modulo capacity.

```

1  class CircularQueue {
2      int *arr;
3      int front, rear, capacity, sz;
4
5  public:
6      CircularQueue(int cap)
7          : capacity(cap), front(0), rear(-1), sz(0) {
8          arr = new int[capacity];
9      }
10
11     bool is_empty() { return sz == 0; }
12
13     bool is_full() { return sz == capacity; }
14
15     // Add element x at the rear
16     void enqueue(int x) {
17         if (is_full()) {
18             // handle overflow (resize or error)
19             return;
20         }
21         rear = (rear + 1) % capacity;
22         arr[rear] = x;
23         sz++;
24     }
25
26     // Remove and return front element
27     int dequeue() {
28         if (is_empty()) return -1; // underflow
29         int val = arr[front];
30         front = (front + 1) % capacity;
31         sz--;
32         return val;
33     }
34
35     // Return front element without removing
36     int front_elem() {
37         if (is_empty()) return -1;
38         return arr[front];
39     }
40
41     ~CircularQueue() { delete[] arr; }
42 };

```

Why circular? Without wraparound, after several enqueue/dequeue pairs the `rear` pointer would fall off the end of the array even though slots near the front are

free. The modulo arithmetic reuses those slots, giving a *drifting-free* implementation.

Complexity Analysis. Every operation is $O(1)$ — no loops, no shifts.

Q6. Write a function to delete a node with a given key from a doubly linked list (unique keys, access to both head and tail). (10 marks) [CSE 105 | 2022–23]

Answer

```

1  struct Node {
2      int key;
3      Node *prev, *next;
4  };
5
6  void deleteNode(Node* &head, Node* &tail, int key) {
7      // Search for the node
8      Node* cur = head;
9      while (cur != nullptr && cur->key != key)
10         cur = cur->next;
11
12     if (cur == nullptr) return; // key not found
13
14     // Relink previous node
15     if (cur->prev != nullptr)
16         cur->prev->next = cur->next;
17     else
18         head = cur->next; // deleting head
19
20     // Relink next node
21     if (cur->next != nullptr)
22         cur->next->prev = cur->prev;
23     else
24         tail = cur->prev; // deleting tail
25
26     delete cur;
27 }
```

Cases handled:

- (a) Middle node — both prev and next relinked.
- (b) Head node ($cur->prev == nullptr$) — head updated to $cur->next$.
- (c) Tail node ($cur->next == nullptr$) — tail updated to $cur->prev$.
- (d) Only node — both head and tail become $nullptr$.

Complexity Analysis. $O(n)$ for the search; the relinking itself is $O(1)$.

Q7. How would you reverse a singly linked list (without duplicating the list)? Write the steps (pseudocode) conceptually. (4 marks) [CSE 105 | 2022–23]

Answer

Idea. Traverse the list once, reversing each `next` pointer in place using three pointers: `prev`, `cur`, `nxt`.

Algorithm 18 REVERSE(*head*)

```

1: prev ← nil
2: cur ← head
3: while cur ≠ nil do
4:   nxt ← cur.next           ▷ save next
5:   cur.next ← prev           ▷ reverse link
6:   prev ← cur                ▷ advance prev
7:   cur ← nxt                 ▷ advance cur
8: end while
9: return prev                ▷ new head

```

Trace on 1→2→3→null:

Step	prev	cur	list so far
init	null	1	1→2→3
iter 1	1	2	1→null, 2→3
iter 2	2	3	2→1→null, 3
iter 3	3	null	3→2→1→null

Complexity Analysis. $O(n)$ time, $O(1)$ extra space — no new nodes allocated.

Q8. How can you remove duplicate values from a sorted singly linked list? (5 marks) [CSE 105 | 2022–23]

Answer

Because the list is *sorted*, all duplicates of any value are adjacent. One pass suffices.

```

1 void removeDuplicates(Node* head) {
2     Node* cur = head;
3     while (cur != nullptr && cur->next != nullptr) {
4         if (cur->val == cur->next->val) {
5             Node* dup = cur->next;
6             cur->next = dup->next; // skip duplicate
7             delete dup;
8             // do NOT advance cur: next node might also be
9             // duplicate
10        } else {
11            cur = cur->next;
12        }
13    }
14 }

```

```

11     }
12 }
13 }

```

Example. 1->1->2->3->3->3->4 $\rightarrow$ 1->2->3->4

Complexity Analysis. $O(n)$ time (single pass), $O(1)$ extra space.

Q9. How can you check if a singly linked list is a palindrome? Write the steps involved conceptually. (10 marks) [CSE 105 | 2022–23]

Answer

Method (Stack-based, $O(n)$ time, $O(n)$ space).

Algorithm 19 IS-PALINDROME(*head*)

```

1:  $S \leftarrow$  empty stack
2:  $cur \leftarrow head$ 
3: while  $cur \neq \text{nil}$  do                                      $\triangleright$  push all values
4:   PUSH( $S, cur.val$ )
5:    $cur \leftarrow cur.next$ 
6: end while
7:  $cur \leftarrow head$ 
8: while  $cur \neq \text{nil}$  do                                      $\triangleright$  compare with list
9:   if  $cur.val \neq \text{POP}(S)$  then
10:    return false
11:  end if
12:   $cur \leftarrow cur.next$ 
13: end while
14: return true

```

Alternative ($O(1)$ space).

- Find the midpoint using slow/fast pointers.
- Reverse the second half of the list in place.
- Compare first half with reversed second half node by node.
- Restore the second half (optional).

Example. 1->2->3->2->1: stack gives [1, 2, 3, 2, 1] (top=1). Comparing forward 1, 2, 3, 2, 1 with pops 1, 2, 3, 2, 1 — all match $\Rightarrow$ palindrome.

Complexity Analysis. Stack-based: $O(n)$ time, $O(n)$ space.
Two-pointer + in-place reverse: $O(n)$ time, $O(1)$ space.

Q10. Given an array of integers (with both negative and positive numbers), write a pro-

gram to find the K -th largest sum of a contiguous subarray within the array. (10 marks) [CSE 203 | 2021–22]

Answer

Approach: Enumerate all $O(n^2)$ contiguous subarray sums, maintain a min-heap of size K to track the K largest.

Algorithm 20 KTHLARGESTSUBARRAYSUM(A, n, K)

```

1:  $minHeap \leftarrow$  empty min-heap of capacity  $K$ 
2: for  $i = 0$  to  $n - 1$  do
3:    $sum \leftarrow 0$ 
4:   for  $j = i$  to  $n - 1$  do
5:      $sum \leftarrow sum + A[j]$   $\triangleright$  sum of  $A[i..j]$ 
6:     if  $|minHeap| < K$  then
7:       PUSH( $minHeap, sum$ )
8:     else if  $sum > TOP(minHeap)$  then
9:       POP( $minHeap$ )
10:      PUSH( $minHeap, sum$ )
11:     end if
12:   end for
13: end for
14: return TOP( $minHeap$ )  $\triangleright K$ -th largest sum

```

Correctness: The min-heap always holds the K largest sums seen so far. After processing all $\binom{n}{2} + n$ subarray sums, the top of the heap is the K -th largest.

Complexity:

- $O(n^2)$ subarrays, each heap operation is $O(\log K)$.
- Total: $O(n^2 \log K)$.

Q11. Suppose the numbers $0, 1, 2, \dots, 9$ were pushed onto a stack in that order, but pops occurred at random points between the various pushes. Determine which of the following two sequences is a valid pop sequence and explain why:

$S_1 : 3, 2, 6, 5, 7, 4, 1, 0, 9, 8$

$S_2 : 3, 2, 6, 4, 7, 5, 1, 0, 9, 8$

(3 marks)

[CSE 105 | 2021–22]

Answer

S_1 is VALID. S_2 is INVALID.

Simulation of S_1 :

Action	Stack (bottom→top)	Popped
push 0,1,2,3	[0, 1, 2, 3]	—
pop 3	[0, 1, 2]	3
pop 2	[0, 1]	2
push 4,5,6	[0, 1, 4, 5, 6]	—
pop 6	[0, 1, 4, 5]	6
pop 5	[0, 1, 4]	5
push 7	[0, 1, 4, 7]	—
pop 7	[0, 1, 4]	7
pop 4	[0, 1]	4
pop 1	[0]	1
pop 0	[]	0
push 8,9	[8, 9]	—
pop 9	[8]	9
pop 8	[]	8

Output: 3, 2, 6, 5, 7, 4, 1, 0, 9, 8 = S_1 ✓

Why S_2 is INVALID:

$S_2 = 3, 2, 6, 4, 7, 5, \dots$

After popping 3, 2: stack = [0, 1]. To pop 6, we must push 4, 5, 6 first, giving stack = [0, 1, 4, 5, 6]. Pop 6: stack = [0, 1, 4, 5]. Next in S_2 is 4, but the top of the stack is 5. We cannot pop 4 without first popping 5, and 5 has not appeared yet in the sequence before 4. Since we can only push in increasing order 0, 1, 2, ..., there is no way to get 4 on top while 5 is still in the stack. Hence S_2 is invalid.

Q12. A stack is being used to parse matching parentheses, brackets, and braces — (, [, and {. Show the state of the stack at the end of the given code fragment. **(10 marks)** [CSE 105 | 2021–22]

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      const int M = 30;
7      int N = 1024;
8      int parent[N], height[N], depth[N];
9      int maxheights[100];
10     for (int i = 0; i < 100; ++i)
11     {
12         maxheights[i] = 0;
13     }
14     double minmeandepth = 1e300;
15     double meandepths = 0;
16     double maxmeandepth = 0;
17     for (int j = 0; j < M; ++j)

```

```

18     {
19     for (int i = 0; i < N; ++i)
20     {
21         parent[i] = -1;
22         height[i] = 0;
23     }
24     for (int i = 0; i < N - 1; ++i)
25     while (true)
26     {
27         int p1 = rand() & (N - 1);
28         int p2 = rand() & (N - 1);
29
30         int s1 = p1;
31         int s2 = p2;
32
33         while ( parent[

```

Answer

We scan the code fragment left-to-right, pushing every opening bracket/brace/paren and popping on every matching closer. The code is *truncated* midway (ends at `while ( parent[`), so we track every unmatched opener still on the stack at the point the fragment ends.

Step-by-step trace of unmatched openers:

Token encountered	Action	Stack (bottom → top)
{ after main()	push {	{
{ after first for	push {	{ {
} closes inner for-body	pop	{
{ after second for (j...)	push {	{ {
{ after for (i=0; i<N)	push {	{ { {
} closes that for-body	pop	{ {
{ after while (true)	push {	{ { {
(in while (parent[	push (	{ { { (
[in parent[	push [	{ { { ([

Final stack state (bottom to top):

[	← top
(	
{	
{	
{	← bottom

The stack contains 5 unmatched openers: {, {, {, (, [(bottom to top), indicating the code fragment is incomplete/unbalanced at the point it was cut off.

Q13. You are given access to the following functions of a list data structure:

- **init(*n*)** — Initialize an array $L[0..n-1]$ of length n for integers. Current position ‘points to’ $L[0]$. All the functions below are applied on this array which is not accessible directly.
- **insert(*item*)** — Inserts an element at the current position and current position is ‘incremented’ (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). Here, *item* is the element to be inserted. In case the list is full, it returns ERR (a large negative constant).
- **remove()** — Removes the element at the current position and current position is ‘decremented’ (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). In case the list is empty, it returns ERR (a large negative constant).
- **value()** — Returns the value of the element at the current position. Current position remains unchanged. In case the list is empty, it returns ERR (a large negative constant).
- **moveToStart()** — Sets the current position to $L[0]$.
- **moveToEnd()** — Sets the current position to $L[n-1]$.
- **next()** — Increments the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). If $k = n-1$, it returns ERR (a large negative constant).
- **prev()** — Decrements the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). If $k = 0$, it returns ERR (a large negative constant).

Now you have to build one queue and one stack in one List data structure as defined above. The queue should grow from the beginning of the list (i.e., first element of the queue should be at $L[0]$) and the stack should grow from the end of the list (i.e., the first element of the stack should be at $L[n-1]$). You have no access to the code of the above implementation and can only call these functions in your code. You need to write appropriate codes (no specific programming language is required; pseudo code is fine) for implementing the queue and stack simultaneously as described above. Please explain your code by giving appropriate comments. In particular, you need to implement the following four functions:

- **push(*item*)** — push a positive integer, *item* in the stack. If there is an error (e.g., no more space available for growing the stack in the list), it should output an appropriate error message and return with -1 .
- **pop()** — pop an element from the stack. If there is an error, it should output an appropriate error message and return with -1 .
- **enqueue(*item*)** — enqueue a positive integer, *item* in the queue. If there is an error (e.g., no more space available for growing the queue in the list), it should output an appropriate error message and return with -1 .
- **dequeue()** — dequeue an element from the queue. If there is an error, it should output an appropriate error message and return with -1 .

(25 marks)

[CSE 105 | 2021–22]

Answer

Layout: Queue grows from $L[0]$ (front = $L[0]$, rear grows right). Stack grows from $L[n - 1]$ (top grows left). Both meet in the middle; if they collide the list is full.

We maintain two global counters:

- **qSize** — number of elements currently in the queue.
- **sSize** — number of elements currently in the stack.

The list is full when $\text{qSize} + \text{sSize} == n$.

Algorithm 21 PUSH(*item*) — stack on shared list

```

1: if qSize + sSize = n then
2:   print "Error: stack overflow"; return -1
3: end if
4: MOVEToEnd()
5: for i = 1 to sSize do
6:   PREV()                                ▷ move to  $L[n - 1 - sSize]$ 
7: end for
8: INSERT(item)
9: sSize ← sSize + 1
10: return 0
    
```

Algorithm 22 POP() — stack on shared list

```

1: if sSize = 0 then
2:   print "Error: stack underflow"; return -1
3: end if
4: MOVEToEnd()
5: for i = 1 to sSize - 1 do
6:   PREV()                                ▷ move to  $L[n - sSize]$ , the stack top
7: end for
8: val ← VALUE()
9: REMOVE()
10: sSize ← sSize - 1
11: return val
    
```

Algorithm 23 ENQUEUE(*item*) — queue on shared list

```

1: if  $qSize + sSize = n$  then
2:   print "Error: queue overflow"; return -1
3: end if
4: MOVEToSTART()
5: for  $i = 1$  to  $qSize$  do
6:   NEXT()                                ▷ move to  $L[qSize]$ , first free slot
7: end for
8: INSERT(item)
9:  $qSize \leftarrow qSize + 1$ 
10: return 0

```

Algorithm 24 DEQUEUE() — queue on shared list

```

1: if  $qSize = 0$  then
2:   print "Error: queue empty"; return -1
3: end if
4: MOVEToSTART()                          ▷  $L[0]$  = front of queue
5:  $val \leftarrow VALUE()$ 
6: REMOVE()
7:  $qSize \leftarrow qSize - 1$ 
8: return  $val$ 

```

Correctness notes:

- Queue front is always at $L[0]$; dequeue removes $L[0]$.
- Stack top is always at $L[n - sSize]$; pop returns that element.
- Overflow is detected by checking $qSize + sSize == n$.
- All operations are $O(n)$ in the worst case due to traversal.

Q14. Convert a Binary Tree to a Circular Doubly Linked List using inorder traversal (left/right pointers become prev/next). (15 marks) [CSE 203 | 2021–22]

Answer

Idea. Perform an inorder DFS. Maintain a global `prev` pointer to the last processed node. Link each node to `prev` in both directions. After traversal, connect head and tail to make it circular.

```

1 struct Node { int val; Node *left, *right; };
2
3 Node *head = nullptr; // first inorder node
4 Node *prev = nullptr; // last processed node
5
6 void inorderConvert(Node* root) {
7     if (root == nullptr) return;
8

```

```

9     inorderConvert(root->left);
10
11     // Process current node
12     if (prev == nullptr) {
13         head = root;           // leftmost = head
14     } else {
15         prev->right = root;    // prev's next = cur
16         root->left = prev;    // cur's prev = prev
17     }
18     prev = root;
19
20     inorderConvert(root->right);
21 }
22
23 Node* bTreeToCircularDLL(Node* root) {
24     head = prev = nullptr;
25     inorderConvert(root);
26
27     // Make it circular: connect last <-> head
28     if (prev != nullptr && head != nullptr) {
29         prev->right = head;
30         head->left = prev;
31     }
32     return head;
33 }

```

Example. BST with inorder [1, 2, 3, 4, 5, 6, 7]:

... $\longleftrightarrow$ 1 $\longleftrightarrow$ 2 $\longleftrightarrow$ 3 $\longleftrightarrow$ 4 $\longleftrightarrow$ 5 $\longleftrightarrow$ 6 $\longleftrightarrow$ 7 $\longleftrightarrow$ (back to 1)

Complexity Analysis. $O(n)$ time (each node visited once), $O(h)$ recursion stack.

Q15. Create KStacks: two stacks in a single array efficiently. Implement push1, pop1, push2, pop2. (10 marks) [CSE 203 | 2021–22, 2016–17]

Answer

Strategy (grow-from-ends). Stack 1 grows left-to-right from index 0; Stack 2 grows right-to-left from index $N-1$. They overflow only when their tops meet.

```

1  const int N = 100;
2  int arr[N];
3  int top1 = -1;           // Stack 1: grows rightward
4  int top2 = N;           // Stack 2: grows leftward
5
6  void push1(int x) {
7      if (top1 + 1 == top2) { /* overflow */ return; }
8      arr[++top1] = x;
9  }

```

```

10 int pop1() {
11     if (top1 == -1) return -1;    // underflow
12     return arr[top1--];
13 }
14
15 void push2(int x) {
16     if (top2 - 1 == top1) { /* overflow */ return; }
17     arr[--top2] = x;
18 }
19 int pop2() {
20     if (top2 == N) return -1;    // underflow
21     return arr[top2++];
22 }

```

Complexity Analysis. All four operations — push1, pop1, push2, pop2 — run in $O(1)$.

Space advantage. Neither stack has a fixed pre-allocation. Stack 1 can use all N slots if Stack 2 is empty, and vice versa — unlike a naïve split (first $N/2$ for Stack 1, rest for Stack 2) which wastes space.

Q16. Given a string consisting of opening and closing parentheses, find the length of the longest valid parenthesis substring. (10 marks) [CSE 203 | 2021–22]

Answer

Stack-based $O(n)$ algorithm. Push *indices* onto the stack. Initialise the stack with a sentinel -1 . For each '(' push its index; for each ')' pop the top and use the current index minus the new top as the current valid length.

```

1  #include <stack>
2  #include <algorithm>
3  using namespace std;
4
5  int longestValidParentheses(string s) {
6      stack<int> st;
7      st.push(-1);    // sentinel
8      int maxLen = 0;
9
10     for (int i = 0; i < (int)s.size(); i++) {
11         if (s[i] == '(') {
12             st.push(i);
13         } else {      // s[i] == ')'
14             st.pop();
15             if (st.empty()) {
16                 st.push(i);    // new base for next valid
                                // segment
17             } else {
18                 maxLen = max(maxLen, i - st.top());
19             }
20         }
21     }
22 }

```

```

19         }
20     }
21 }
22 return maxLen;
23 }

```

Trace on ")()())" (expected: 4):

i	$s[i]$	Stack	maxLen
0	)	pop -1; empty $\Rightarrow$ push 0; [0]	0
1	(	push 1; [0,1]	0
2	)	pop 1; top=0; len=2 - 0 = 2; [0]	2
3	(	push 3; [0,3]	2
4	)	pop 3; top=0; len=4 - 0 = 4; [0]	4
5	)	pop 0; empty $\Rightarrow$ push 5; [5]	4

Complexity Analysis. $O(n)$ time, $O(n)$ stack space.

- Q17.** Given a Queue data structure that supports `enqueue()` and `dequeue()`, implement a Stack using only instances of Queue and Queue operations. **(10 marks)** [CSE 203 | 2021–22]

Answer

Design: push is $O(n)$, pop is $O(1)$. Keep one queue Q . On `push(x)`: enqueue x , then rotate all previously enqueued elements to the back so x is at the front.

```

1  #include <queue>
2  using namespace std;
3
4  class Stack {
5      queue<int> q;
6  public:
7      void push(int x) {
8          int sz = q.size();
9          q.push(x);
10         // Rotate: bring x to the front
11         for (int i = 0; i < sz; i++) {
12             q.push(q.front());
13             q.pop();
14         }
15     }
16     int pop() {
17         if (q.empty()) return -1;
18         int val = q.front();
19         q.pop();

```

```

20     return val;
21 }
22 int top() { return q.front(); }
23 bool empty() { return q.empty(); }
24 };

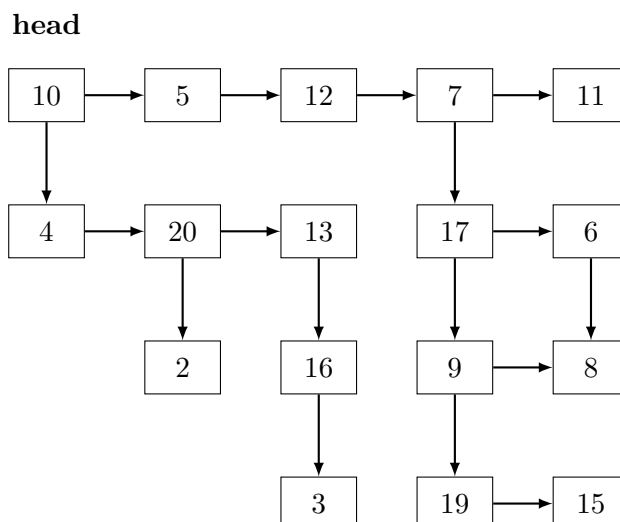
```

Trace. push(1), push(2), push(3), pop(), push(4), pop():

Op	Queue state (front→rear)	Return
push(1)	[1]	—
push(2)	[2,1]	—
push(3)	[3,2,1]	—
pop()	[2,1]	3
push(4)	[4,2,1]	—
pop()	[2,1]	4

Complexity Analysis. push: $O(n)$. pop: $O(1)$.

Q18. Given a linked list where in addition to the next pointer, each node has a child pointer, which may or may not point to a separate list. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure, as shown in Figure . You are given the head of the first level of the list. Flatten the list so that all the nodes appear in a single-level linked list. You need to flatten the list in a way that all nodes at the first level should come first, then nodes of second level, and so on.



The above list should be converted to $10 \rightarrow 5 \rightarrow 12 \rightarrow 7 \rightarrow 11 \rightarrow 4 \rightarrow 20 \rightarrow 13 \rightarrow 17 \rightarrow 6 \rightarrow 2 \rightarrow 16 \rightarrow 9 \rightarrow 8 \rightarrow 3 \rightarrow 19 \rightarrow 15$. **(15 marks)** [CSE 203 | 2021–22]

Answer

Key Insight. The required output is in *BFS (level-by-level)* order: all level-1 nodes first, then level-2, etc. A queue naturally produces BFS order.

```

1  #include <queue>
2  using namespace std;
3
4  struct Node { int val; Node *next, *child; };
5
6  Node* flatten(Node* head) {
7      if (!head) return nullptr;
8
9      queue<Node*> bfsQ;
10
11     // Enqueue all level-1 nodes in order
12     for (Node* c = head; c != nullptr; c = c->next)
13         bfsQ.push(c);
14
15     Node *dummy = new Node{0, nullptr, nullptr};
16     Node *tail = dummy;
17
18     while (!bfsQ.empty()) {
19         Node* cur = bfsQ.front();
20         bfsQ.pop();
21
22         // If cur has a child list, enqueue its nodes
23         if (cur->child != nullptr) {
24             for (Node* c = cur->child; c != nullptr; c = c->
25                 next)
26                 bfsQ.push(c);
27             cur->child = nullptr;
28         }
29
30         // Append cur to result list
31         cur->next = nullptr;
32         tail->next = cur;
33         tail = cur;
34     }
35     return dummy->next;
36 }
```

Verification on given figure:

Level 1: 10 5 12 7 11

Level 2: 4 20 13 (children of 10) + 17 6 (children of 7)

Level 3: 2 (child of 20), 16 (child of 13), 9 8 (children of 17)

Level 4: 3 (child of 16), 19 15 (children of 9)

Result: 10 5 12 7 11 4 20 13 17 6 2 16 9 8 3 19 15 ✓

Complexity Analysis. $O(N)$ time and space, where N = total number of nodes.

Q19. Write a program to reverse a stack using recursion, without using any loop. (10 marks) [CSE 203 | 2021–22]

Answer

Two mutual recursions:

- (a) `insertAtBottom(st, x)` — inserts x at the bottom of stack `st` without a loop.
- (b) `reverseStack(st)` — pops the top, reverses the rest, then inserts the popped element at the bottom.

```

1  #include <stack>
2  using namespace std;
3
4  // Insert x at the bottom of stack st (recursively)
5  void insertAtBottom(stack<int>& st, int x) {
6      if (st.empty()) {
7          st.push(x);
8          return;
9      }
10     int top = st.top();
11     st.pop();
12     insertAtBottom(st, x);    // recurse with smaller stack
13     st.push(top);            // restore saved top
14 }
15
16 // Reverse the stack (recursively)
17 void reverseStack(stack<int>& st) {
18     if (st.empty()) return;
19     int top = st.top();
20     st.pop();
21     reverseStack(st);        // reverse the rest
22     insertAtBottom(st, top); // put top at the bottom
23 }
```

Trace on [1, 2, 3, 4, 5] (top = 5):

1. `reverseStack` pops 5, reverses [1, 2, 3, 4], inserts 5 at bottom.
2. After full recursion unwinds: stack becomes [5, 4, 3, 2, 1] (top = 1).

Complexity Analysis. `insertAtBottom`: $O(n)$ recursive calls.
`reverseStack`: calls `insertAtBottom` n times $\Rightarrow O(n^2)$ total time.
 Call-stack depth: $O(n)$.

Q20. Why is Binary Search preferred over Ternary Search? (10 marks) [CSE 203 | 2021–22]

Answer

Although Ternary Search splits the array into 3 parts and eliminates $\frac{2}{3}$ of the array per iteration (vs. $\frac{1}{2}$ for Binary Search), Binary Search is preferred because of the **number of comparisons per iteration**:

- **Binary Search** requires **1 comparison** per iteration (just check if target $\leq$ mid), reducing search space by half. Worst-case comparisons: $\lceil \log_2 n \rceil$.
- **Ternary Search** requires **2 comparisons** per iteration (check vs. mid_1 and mid_2), reducing search space to one-third. Worst-case comparisons: $2\lceil \log_3 n \rceil$.

Comparison:

$$2 \log_3 n = 2 \cdot \frac{\log_2 n}{\log_2 3} = \frac{2}{\log_2 3} \log_2 n \approx \frac{2}{1.585} \log_2 n \approx 1.26 \log_2 n$$

So Ternary Search performs approximately **26% more comparisons** than Binary Search in the worst case. Since comparisons are the dominant cost in search algorithms, Binary Search is more efficient in practice.

Conclusion: The gain in iterations from splitting into 3 parts does not offset the cost of the extra comparison per iteration. Binary Search achieves a better total comparison count.

Q21. Write a code snippet to implement a queue using a circular array, where each element contains ID, name, and department of a student. **(13 marks)** [CSE 203 | 2019–20]

Answer

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct Student {
6      int    id;
7      string name;
8      string department;
9  };
10
11 class StudentQueue {
12     Student *arr;
13     int front, rear, capacity, sz;
14
15 public:
16     StudentQueue(int cap) : capacity(cap), front(0), rear(-1),
17                             sz(0) {
18         arr = new Student[capacity];
19     }

```

```

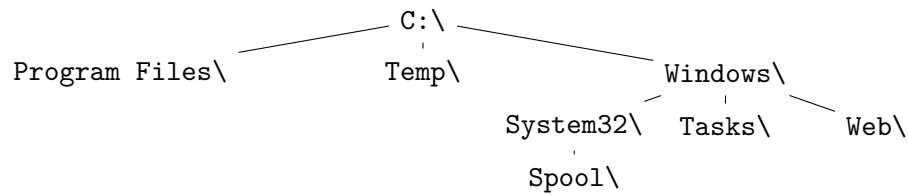
19
20     bool isEmpty() { return sz == 0; }
21     bool isFull()  { return sz == capacity; }
22
23     void enqueue(int id, string name, string dept) {
24         if (isFull()) { cout << "Queue full\n"; return; }
25         rear = (rear + 1) % capacity;
26         arr[rear] = {id, name, dept};
27         sz++;
28     }
29
30     Student dequeue() {
31         if (isEmpty()) { cout << "Queue empty\n"; return {-1, "", ""}; }
32         Student s = arr[front];
33         front = (front + 1) % capacity;
34         sz--;
35         return s;
36     }
37
38     Student frontElement() {
39         if (isEmpty()) { cout << "Queue empty\n"; return {-1, "", ""}; }
40         return arr[front];
41     }
42
43     ~StudentQueue() { delete[] arr; }
44 };
45
46 int main() {
47     StudentQueue q(3);
48     q.enqueue(101, "Alice", "CSE");
49     q.enqueue(102, "Bob", "EEE");
50     q.enqueue(103, "Carol", "ME");
51
52     Student s = q.dequeue();
53     cout << s.id << " " << s.name << " " << s.department << "\n";
54
55     q.enqueue(104, "Dave", "CE"); // reuses slot (circular)
56     return 0;
57 }

```

Complexity Analysis. enqueue, dequeue, frontElement: all $O(1)$.

Q22. Consider the following hierarchical structure of a file system. Write a program to print the directory hierarchy of the file system, where all children of a parent will be placed one tab-space right from the parent. Use depth-first tree traversal technique

to perform the above task.



(12 marks)

[CSE 203 | 2019–20]

Answer

Data structure. Represent the file system as an n -ary tree where each node has a name and a list of children.

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  struct FSNode {
7      string name;
8      vector<FSNode*> children;
9  };
10
11  // DFS print: depth controls the indentation level
12  void printHierarchy(FSNode* node, int depth = 0) {
13      // Print current node with 'depth' tabs
14      for (int i = 0; i < depth; i++) cout << "\t";
15      cout << node->name << "\n";
16
17      // Recurse on each child
18      for (FSNode* child : node->children)
19          printHierarchy(child, depth + 1);
20  }
21
22  int main() {
23      // Build the given file system tree
24      FSNode *root = new FSNode{"C:\\"};
25      FSNode *pf = new FSNode{"Program Files\\"};
26      FSNode *tmp = new FSNode{"Temp\\"};
27      FSNode *win = new FSNode{"Windows\\"};
28      FSNode *sys = new FSNode{"System32\\"};
29      FSNode *spool = new FSNode{"Spool\\"};
30      FSNode *tasks = new FSNode{"Tasks\\"};
31      FSNode *web = new FSNode{"Web\\"};
32
33      sys->children.push_back(spool);
34      win->children = {sys, tasks, web};
35      root->children = {pf, tmp, win};
  
```

```
36  
37     printHierarchy(root);  
38     return 0;  
39 }
```

Expected output:

```
C:\  
  Program Files\  
  Temp\  
  Windows\  
    System32\  
      Spool\  
        Tasks\  
        Web\
```

Complexity Analysis. $O(n)$ time and $O(h)$ stack space where n = total nodes, h = height of the tree.

Q23. Give the algorithm for converting an infix expression to postfix, and for evaluating a postfix expression using a stack. **(10 marks)** [CSE 203 | 2018–19]

Answer

Part A — Infix to Postfix (Shunting-Yard):

Algorithm 25 INFIX-TO-POSTFIX(*expr*)

```

1:  $S \leftarrow$  empty stack ▷ operator stack
2:  $output \leftarrow ""$ 
3: for each token  $t$  in  $expr$  do
4:   if  $t$  is an operand then
5:      $output \leftarrow output + t$ 
6:   else if  $t = '('$  then
7:     PUSH( $S, t$ )
8:   else if  $t = ')'$  then
9:     while TOP( $S$ )  $\neq '('$  do
10:       $output \leftarrow output + \text{POP}(S)$ 
11:    end while
12:    POP( $S$ ) ▷ discard '('
13:   else ▷  $t$  is an operator
14:     while  $S \neq \emptyset$  and PREC(TOP( $S$ ))  $\geq$  PREC( $t$ ) and TOP( $S$ )  $\neq '('$  do
15:        $output \leftarrow output + \text{POP}(S)$ 
16:     end while
17:     PUSH( $S, t$ )
18:   end if
19: end for
20: while  $S \neq \emptyset$  do
21:    $output \leftarrow output + \text{POP}(S)$ 
22: end while
23: return  $output$ 

```

Examples:

- $A+B*C \longrightarrow ABC*+$
- $(A+B)*(C-D) \longrightarrow AB+CD-*$

Part B — Evaluate Postfix:**Algorithm 26** EVAL-POSTFIX(*expr*)

```

1:  $S \leftarrow$  empty stack
2: for each token  $t$  in  $expr$  do
3:   if  $t$  is an operand then
4:     PUSH( $S, \text{VALUE}(t)$ )
5:   else ▷  $t$  is an operator
6:      $b \leftarrow \text{POP}(S)$ 
7:      $a \leftarrow \text{POP}(S)$ 
8:     PUSH( $S, \text{APPLY}(a, t, b)$ )
9:   end if
10: end for
11: return POP( $S$ ) ▷ final result

```

Example trace. Postfix "231*+9-" (i.e., $2 + (3 \times 1) - 9$):

Token	Stack (bottom→top)	Action
2	[2]	push
3	[2,3]	push
1	[2,3,1]	push
*	[2,3]	pop 1,3; push $3 \times 1 = 3$
+	[5]	pop 3,2; push $2 + 3 = 5$
9	[5,9]	push
-	[-4]	pop 9,5; push $5 - 9 = -4$

Complexity Analysis. Both algorithms: $O(n)$ time, $O(n)$ stack space.

- Q24.** You are given two sorted lists of size m and n . Give an $O(\log m + \log n)$ time algorithm for computing the k -th smallest element in the union of two lists. **(12 marks)** [CSE 203 | 2018–19]

Answer

To find the k -th smallest element in $O(\log m + \log n)$ time, we use a divide-and-conquer approach. Instead of merging the lists, we compare the elements at index $k/2$ from both lists to eliminate half of the remaining elements from consideration in each step.

Algorithm 27 FIND-KTH-SMALLEST(A, m, B, n, k)

```

1:                                     ▷ Ensure list  $A$  is the smaller list
2: if  $m > n$  then
3:     return FIND-KTH-SMALLEST( $B, n, A, m, k$ )
4: end if
5: if  $m = 0$  then
6:                                     ▷ Base case: list  $A$  is empty
7:     return  $B[k - 1]$ 
8: end if
9: if  $k = 1$  then
10:                                     ▷ Base case: smallest element
11:     return  $\min(A[0], B[0])$ 
12: end if
13:                                     ▷ Partition  $k$  into two parts
14:  $i \leftarrow \min(m, \lfloor k/2 \rfloor)$ 
15:  $j \leftarrow k - i$ 
16: if  $A[i - 1] < B[j - 1]$  then
17:                                     ▷ Discard first  $i$  elements of  $A$ 
18:     return FIND-KTH-SMALLEST( $A[i \dots m - 1], m - i, B, n, k - i$ )
19: else
20:                                     ▷ Discard first  $j$  elements of  $B$ 
21:     return FIND-KTH-SMALLEST( $A, m, B[j \dots n - 1], n - j, k - j$ )
22: end if
    
```

Example Execution Trace: Let $A = [2, 13, 17, 30]$, $B = [1, 5, 8, 11, 20]$, and $k = 5$ (finding the 5th smallest element).

Step	List A (size m)	List B (size n)	k	Indices	Action / Elimination
1	$[2, 13, 17, 30]$ (4)	$[1, 5, 8, 11, 20]$ (5)	5	$i = 2, j = 3$	$A[1] = 13 > B[2] = 8 \implies$ Discard $B[0 \dots 2]$
2	$[2, 13, 17, 30]$ (4)	$[11, 20]$ (2)	2	Swap lists	Size $m > n$, swap A and B
3	$[11, 20]$ (2)	$[2, 13, 17, 30]$ (4)	2	$i = 1, j = 1$	$A[0] = 11 > B[0] = 2 \implies$ Discard $B[0]$
4	$[11, 20]$ (2)	$[13, 17, 30]$ (3)	1	Base Case	$k = 1 \implies \min(A[0], B[0]) =$ $\min(11, 13) = \mathbf{11}$

Complexity Analysis. **Time Complexity:** $O(\log m + \log n)$. In each recursive step, we drop roughly $\lfloor k/2 \rfloor$ elements from consideration. Since $k \leq m + n$, the total number of steps is bounded logarithmically.

Space Complexity: $O(\log m + \log n)$ auxiliary stack space due to recursion, which can be optimized to $O(1)$ using an iterative pointer-based binary search approach.

Q25. Ternary search, like binary search, is a divide-and-conquer algorithm. It is manda-

tory for the array (in which you will search for an element) to be sorted before you begin the search. In this search, after each iteration it neglects $\frac{2}{3}$ part of the array and repeats the same operations on the remaining $\frac{1}{3}$. Write an algorithm for ternary search and show its complexity. (12 marks) [CSE 203 | 2018–19]

Answer

Idea. Split the search range into three equal thirds using two midpoints m_1 and m_2 . Compare the target with $A[m_1]$ and $A[m_2]$ to decide which third to recurse into.

Algorithm 28 TERNARY-SEARCH(A, lo, hi, x)

```

1: if  $lo > hi$  then
2:   return  $-1$ 
3: end if
4:  $m_1 \leftarrow lo + \lfloor (hi - lo)/3 \rfloor$ 
5:  $m_2 \leftarrow hi - \lfloor (hi - lo)/3 \rfloor$ 
6: if  $A[m_1] = x$  then
7:   return  $m_1$ 
8: end if
9: if  $A[m_2] = x$  then
10:  return  $m_2$ 
11: end if
12: if  $x < A[m_1]$  then
13:  return TERNARY-SEARCH( $A, lo, m_1 - 1, x$ )
14: else if  $x > A[m_2]$  then
15:  return TERNARY-SEARCH( $A, m_2 + 1, hi, x$ )
16: else
17:  return TERNARY-SEARCH( $A, m_1 + 1, m_2 - 1, x$ )
18: end if

```

Complexity Analysis.

Each call reduces the search space by a factor of 3, giving the recurrence:

$$T(n) = T\left(\frac{n}{3}\right) + O(1)$$

Solving: $T(n) = O(\log_3 n)$.

Comparison with Binary Search.

$$\log_3 n = \frac{\log_2 n}{\log_2 3} \approx 0.631 \cdot \log_2 n.$$

Ternary search makes *fewer iterations* but **two comparisons per iteration** versus one for binary search, so the total number of comparisons is:

$$2 \log_3 n \approx 1.26 \log_2 n > \log_2 n.$$

Remark. Binary search is preferred because it uses fewer total comparisons despite more iterations ($\log_2 n$ comparisons vs $\approx 1.26 \log_2 n$). Ternary search is not an improvement for comparison-based search.

Q26. Describe a situation where storing items in an array is clearly better than storing items in a linked list. (8 marks) [CSE 203 | 2018–19]

Answer

Scenario: Repeated random-access / index-based lookups on a fixed dataset.

Example. You store $n = 10^6$ sensor readings and repeatedly answer queries of the form “what is the k -th reading?” With an array, the answer is simply `arr[k]`, i.e., $O(1)$ time. With a linked list, you must traverse k nodes from the head: $O(k) = O(n)$ worst-case.

Why arrays win in this scenario:

- (a) **$O(1)$ random access.** The address of element i is `base + i*sizeof(element)`, computed without any traversal.
- (b) **Cache efficiency.** Array elements are contiguous in memory; a cache line loaded for element i also brings $i+1, i+2, \dots$ into cache. Linked list nodes may be scattered, causing cache misses on every step.
- (c) **Lower memory overhead.** Each array element stores only the datum; a linked list node additionally stores one or two pointers (8–16 bytes each on 64-bit systems), wasting significant memory for small data types (e.g. `int`).
- (d) **Simpler iteration.** A for-loop `for(i=0; i<n; i++)` with an array runs faster than pointer chasing in a linked list due to branch prediction and pipelining.

Conclusion. Arrays excel when (a) the size is known in advance, (b) random access is frequent, and (c) insertions/deletions are rare.

Q27. Write two versions of binary search. Simulate on [23, 56, 8, 10, 12, 15, 20, 21, 23] for keys 1, 6, 23, 24. (10+10=20 marks) [CSE 203 | 2017–18]

Answer

Note. Binary search requires a *sorted* array. Sorting the given array yields [8, 10, 12, 15, 20, 21, 23, 23, 56] (indices 0–8).

Version 1 — Iterative:

```

1 // Returns index of x in sorted arr[0..n-1], or -1 if not
   found
2 int binarySearchIter(int arr[], int n, int x) {
3     int lo = 0, hi = n - 1;

```

```

4     while (lo <= hi) {
5         int mid = lo + (hi - lo) / 2;    // avoids overflow
6         if (arr[mid] == x) return mid;
7         if (arr[mid] < x) lo = mid + 1;
8         else                hi = mid - 1;
9     }
10    return -1;
11 }

```

Version 2 — Recursive:

```

1  int binarySearchRec(int arr[], int lo, int hi, int x) {
2      if (lo > hi) return -1;
3      int mid = lo + (hi - lo) / 2;
4      if (arr[mid] == x) return mid;
5      if (arr[mid] < x) return binarySearchRec(arr, mid + 1, hi
6          , x);
7      else                return binarySearchRec(arr, lo, mid -
8          1, x);
9  }

```

Simulation on sorted array [8, 10, 12, 15, 20, 21, 23, 23, 56]:

Key	lo	hi	mid (arr[mid])	Result
1	0	8	4 (20)	$1 < 20 \Rightarrow \text{hi}=3$
	0	3	1 (10)	$1 < 10 \Rightarrow \text{hi}=0$
	0	0	0 (8)	$1 < 8 \Rightarrow \text{hi}=-1 \Rightarrow \text{NOT FOUND}$
6	0	8	4 (20)	$6 < 20 \Rightarrow \text{hi}=3$
	0	3	1 (10)	$6 < 10 \Rightarrow \text{hi}=0$
	0	0	0 (8)	$6 < 8 \Rightarrow \text{hi}=-1 \Rightarrow \text{NOT FOUND}$
23	0	8	4 (20)	$23 > 20 \Rightarrow \text{lo}=5$
	5	8	6 (23)	FOUND at index 6
24	0	8	4 (20)	$24 > 20 \Rightarrow \text{lo}=5$
	5	8	6 (23)	$24 > 23 \Rightarrow \text{lo}=7$
	7	8	7 (23)	$24 > 23 \Rightarrow \text{lo}=8$
	8	8	8 (56)	$24 < 56 \Rightarrow \text{hi}=7 \Rightarrow \text{NOT FOUND}$

Complexity Analysis. $O(\log n)$ time, $O(1)$ space (iterative) / $O(\log n)$ stack space (recursive).

Q28. How do we cope with hard problems? Discuss. (10 marks) [CSE 207 | 2017–18]

Answer

When a problem is NP-hard (no known polynomial-time algorithm), we use one or more of the following strategies:

- 1. Identify tractable special cases.** Restrict inputs so the problem becomes polynomial. E.g., 2-SAT is polynomial though 3-SAT is NP-complete; TSP on trees is $O(n)$.
- 2. Approximation algorithms.** Find a solution guaranteed to be within a factor ρ of optimal in polynomial time. E.g., the 2-approximation for vertex cover; PTAS for knapsack achieving $(1 - \varepsilon)$ -optimal in $O(n/\varepsilon)$ time.
- 3. Randomized algorithms.** Use randomness to get correct or near-optimal answers with high probability. E.g., randomized quicksort, Monte Carlo methods, simulated annealing.
- 4. Fixed-parameter tractability (FPT).** If the problem has a natural parameter k that is small in practice, design algorithms exponential only in k but polynomial in n . E.g., vertex cover in $O(2^k \cdot n)$.
- 5. Exponential algorithms with pruning.** Branch-and-bound, backtracking with aggressive pruning, and memoization can solve moderate-sized instances exactly (e.g., TSP with $n \leq 20$ via Held-Karp DP in $O(2^n n^2)$).
- 6. Heuristics and metaheuristics.** No worst-case guarantee but work well in practice: greedy heuristics, genetic algorithms, tabu search, ant-colony optimization.
- 7. Integer programming / SAT solvers.** Modern ILP solvers (CPLEX, Gurobi) and SAT solvers handle many practically-sized NP-hard instances efficiently through branch-and-cut and clause learning.

Summary: The right strategy depends on whether we need exact solutions, how large n is, and whether a performance guarantee is required.

Q29. Given a singly linked list and integers m and k , find the node with value m then delete the next k nodes. (10 marks) [CSE 203 | 2016–17]

Answer

```

1 struct Node { int item; Node* next; };
2 Node* head;
3
4 void deleteNextK(int m, int k) {
5     // Step 1: find node with value m
6     Node* cur = head;
7     while (cur != nullptr && cur->item != m)

```

```

8         cur = cur->next;
9
10        if (cur == nullptr) return;    // m not found
11
12        // Step 2: delete the next k nodes
13        for (int i = 0; i < k; i++) {
14            if (cur->next == nullptr) break;    // fewer than k
15                                                nodes remain
16            Node* toDelete = cur->next;
17            cur->next = toDelete->next;
18            delete toDelete;
19        }

```

Example. List: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$, $m = 3$, $k = 2$.

Find node 3. Delete node 4 (toDelete), then delete node 5.

Result: $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7$.

Complexity Analysis. $O(n)$ to find m ; $O(k)$ for deletions; $O(n)$ total.

Q30. Write enqueue and dequeue functions for a FIFO queue implemented using a singly linked list. (10 marks) [CSE 203 | 2016–17]

Answer

Maintain two pointers: front (for dequeue) and rear (for enqueue). Both operations are $O(1)$.

```

1  struct Node { int val; Node* next; };
2  Node *front = nullptr, *rear = nullptr;
3
4  // Add element at the rear
5  void enqueue(int x) {
6      Node* n = new Node{x, nullptr};
7      if (rear == nullptr) {    // empty queue
8          front = rear = n;
9      } else {
10         rear->next = n;
11         rear = n;
12     }
13 }
14
15 // Remove and return front element
16 int dequeue() {
17     if (front == nullptr) return -1;    // underflow
18     Node* tmp = front;
19     int val = tmp->val;
20     front = front->next;
21     if (front == nullptr) rear = nullptr;    // queue became

```

```

    empty
22     delete tmp;
23     return val;
24 }

```

Complexity Analysis. enqueue: $O(1)$ — append at tail.
 dequeue: $O(1)$ — remove from head.

Q31. Deduce the condition under which an array-based implementation of a list would be better in space utilisation than a linked-based implementation. **(10 marks)** [CSE 203 | 2015–16]

Answer

Let:

- d = size (bytes) of one *data item*,
- p = size of one *pointer* (typically 8 bytes on 64-bit systems),
- n = current number of elements,
- c = allocated array capacity ($c \geq n$; worst-case $c = 2n$ after a doubling).

Space used:

$$S_{\text{array}} = c \cdot d, \quad S_{\text{linked}} = n \cdot (d + p) \quad (\text{singly linked}).$$

Array beats linked list when $S_{\text{array}} < S_{\text{linked}}$:

$$c \cdot d < n(d + p) \implies \frac{c}{n} < 1 + \frac{p}{d}.$$

With worst-case $c = 2n$ (just doubled):

$$2 < 1 + \frac{p}{d} \implies \frac{p}{d} > 1 \implies \boxed{p > d}.$$

Conclusion. An array uses less space than a singly linked list whenever the pointer size p exceeds the data item size d . In practice, with $p = 8$ bytes (64-bit pointer), arrays are more space-efficient for data types smaller than 8 bytes (e.g., `char`, `short`, `int`). For large structs ($d \gg p$), the extra pointer overhead becomes negligible and the linked list can be competitive.

Remark. For a *doubly* linked list the condition becomes $2p > d$.

Q32. Given input string x , check whether x is of the form $w\#w'$, where w' is the reverse of w . Suggest a data structure and show how to solve it efficiently. **(10 marks)** [CSE 203 | 2015–16]

Answer**Data structure: Stack.****Algorithm:****Algorithm 29** CHECK-W-HASH-REVERSE(x)

```

1:  $hashPos \leftarrow$  index of '#' in  $x$ 
2: if  $hashPos$  not found then
3:   return false
4: end if
5:  $w \leftarrow x[0 .. hashPos - 1]$ 
6:  $w' \leftarrow x[hashPos + 1 .. end]$ 
7: if  $|w| = 0$  or  $|w| \neq |w'|$  then
8:   return false
9: end if
10:  $S \leftarrow$  empty stack
11: for each char  $c$  in  $w$  do
12:   PUSH( $S, c$ )
13: end for
14: for each char  $c$  in  $w'$  do
15:   if  $S$  is empty or TOP( $S$ )  $\neq c$  then
16:     return false
17:   end if
18:   POP( $S$ )
19: end for
20: return  $S$  is empty

```

Why it works. Pushing w onto the stack gives a reversed w at the top. Comparing pops (which produce w in reverse) character by character with w' (forward) checks exactly that $w' = \text{reverse}(w)$.

Example. $x = \text{'`abc#cba'}$: push a, b, c ; compare pops c, b, a with $w' = cba \Rightarrow$ match, return true.

Complexity Analysis. $O(|x|)$ time, $O(|w|)$ stack space.

Q33. Given input k and n , return the key of the n -th node before the one containing k . (All keys distinct.) **(15 marks)** [CSE 203 | 2015–16]

Answer

Two-pointer technique. Advance a **fast** pointer n steps ahead, then move both **slow** and **fast** together until **fast** reaches the node with key k . At that moment **slow** is exactly n nodes behind.

```

1 struct ListNode { int key; ListNode* next; };
2 ListNode* head;
3

```

```

4  int nthNodeBeforeKey(int k, int n) {
5      listNode *fast = head, *slow = head;
6
7      // Advance fast by n steps
8      for (int i = 0; i < n; i++) {
9          if (fast == nullptr) return -1; // list shorter than
10             n
11         fast = fast->next;
12     }
13
14     // Move both until fast->key == k
15     while (fast != nullptr && fast->key != k) {
16         fast = fast->next;
17         slow = slow->next;
18     }
19
20     if (fast == nullptr) return -1; // k not in list
21     return slow->key;
22 }

```

Example. List: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$, $k = 50$, $n = 2$.

fast starts at 10, advances 2 steps to 30.

Move together: fast=40, slow=20; fast=50 (found k), slow=30.

Return 30. ✓

Complexity Analysis. $O(L)$ time where L is the length of the list; single pass. $O(1)$ extra space.

Q34. Your group proposes that the beginning (index 0) of the array be the top of the stack. A friend claims this is inefficient. Do you agree? Justify. **(8 marks)** [CSE 203 | 2015–16]

Answer

Yes, the friend is correct — this is inefficient.

If $\text{arr}[0]$ is the top:

- **push(x):** Must shift every existing element one position to the right to make room at index 0. Cost: $O(n)$.
- **pop():** Removes $\text{arr}[0]$, then must shift every remaining element one position left. Cost: $O(n)$.

Efficient alternative — use the end as the top. Maintain a `topIndex` pointing to the last-filled slot.

- **push(x):** $\text{arr}[\text{++topIndex}] = x$ — $O(1)$.
- **pop():** return $\text{arr}[\text{topIndex--}]$ — $O(1)$.

Conclusion. Using index 0 as the top degrades both **push** and **pop** from $O(1)$ to $O(n)$, making the implementation as slow as a list-based approach while forfeiting the simplicity of the array. The end of the array should always be the top.

Q35. Discuss the Drifting Queue Problem with an illustrative example. Propose a solution. Address empty/full recognition. (5+5+5+4=19 marks) [CSE 203 | 2015–16]

Answer

The Drifting Queue Problem. In a naive linear-array queue with **front** and **rear** indices, each dequeue increments **front** and each enqueue increments **rear**. Over time both indices drift rightward until **rear** hits the end of the array — even though positions $0 \dots \text{front} - 1$ are free.

Example. Capacity = 4.

Op	[0]	[1]	[2]	[3]	front	rear	Status
init	–	–	–	–	0	-1	empty
enqueue(A)	A	–	–	–	0	0	
enqueue(B)	A	B	–	–	0	1	
dequeue()	–	B	–	–	1	1	
dequeue()	–	–	–	–	2	1	empty but front drifted
enqueue(C)	–	–	C	–	2	2	
enqueue(D)	–	–	C	D	2	3	
enqueue(E)	–	–	C	D	2	3	FULL? rear = 3 = cap-1 but slots 0,1 are free!

Proposed Solution: Circular (Ring) Queue. Wrap indices modulo capacity:

$\text{rear} \leftarrow (\text{rear} + 1) \bmod \text{capacity}, \quad \text{front} \leftarrow (\text{front} + 1) \bmod \text{capacity}.$

Empty/Full ambiguity. When using only **front** and **rear**, both empty and full give $\text{front} == (\text{rear} + 1) \bmod \text{capacity}$ (if one slot is sacrificed) or $\text{front} == \text{rear}$ — these are ambiguous. **Solution:** maintain a **size** counter (or a boolean **isFull** flag).

Algorithm 30 ENQUEUE(x) — circular queue

```

1: if  $sz = \text{capacity}$  then
2:   error "Queue Overflow"
3: end if
4:  $\text{rear} \leftarrow (\text{rear} + 1) \bmod \text{capacity}$ 
5:  $\text{arr}[\text{rear}] \leftarrow x$ 
6:  $sz \leftarrow sz + 1$ 
```

Algorithm 31 DEQUEUE() — circular queue

```

1: if  $sz = 0$  then
2:   error "Queue Underflow"
3: end if
4:  $val \leftarrow arr[front]$ 
5:  $front \leftarrow (front + 1) \bmod capacity$ 
6:  $sz \leftarrow sz - 1$ 
7: return  $val$ 

```

This completely eliminates drifting and correctly distinguishes empty from full.

- Q36.** What is an application of prefix sums? What are prefix sums of an array $A[i]$, $1 \leq i \leq n$? Write a parallel algorithm that finds prefix sums of the array A . Illustrate for $[15, 35, 40, 25, 20, 50, 10, 30]$ showing bottom-up and top-down traversals. **(15 marks)** [CSE 207 | 2015–16]

Answer

Application: Range-sum queries — given $A[1..n]$, answer $\sum_{i=l}^r A[i]$ in $O(1)$ after $O(n)$ preprocessing. Other uses: parallel scan, histogram equalization, load balancing.

Definition: The prefix sum array P is defined as:

$$P[i] = \sum_{j=1}^i A[j], \quad P[0] = 0$$

Parallel Prefix Sum (Blelloch Scan):

Uses a complete binary tree over n leaves. Two phases:

Algorithm 32 PARALLEL-PREFIX-SUM(T, n)

```

1: for  $d = 0$  to  $\log n - 1$  do ▷ Bottom-up reduce
2:   for each  $i = 0$  to  $n - 1$ , step  $2^{d+1}$ , in parallel do
3:      $T[i + 2^{d+1} - 1] \leftarrow T[i + 2^{d+1} - 1] + T[i + 2^d - 1]$ 
4:   end for
5: end for
6:  $T[n - 1] \leftarrow 0$  ▷ Top-down downsweep
7: for  $d = \log n - 1$  downto  $0$  do
8:   for each  $i = 0$  to  $n - 1$ , step  $2^{d+1}$ , in parallel do
9:      $left \leftarrow T[i + 2^d - 1]$ 
10:     $T[i + 2^d - 1] \leftarrow T[i + 2^{d+1} - 1]$ 
11:     $T[i + 2^{d+1} - 1] \leftarrow T[i + 2^{d+1} - 1] + left$ 
12:   end for
13: end for

```

Complexity: $O(\log n)$ parallel steps, $O(n)$ total work.

Illustration for $A = [15, 35, 40, 25, 20, 50, 10, 30]$, $n = 8$:

Bottom-up (reduce) — each node stores sum of its subtree:

Level	Tree nodes (left to right)							
Leaves (d=0)	15	35	40	25	20	50	10	30
d=1 (pairs)	50	—	65	—	70	—	40	—
d=2 (quads)	115	—	—	—	110	—	—	—
d=3 (root)	225							

Top-down (downsweep) — root set to 0, propagate exclusive prefix:

Level	Node values after downsweep							
Root	0							
d=2	0	—	—	—	115	—	—	—
d=1	0	—	50	—	115	—	185	—
Leaves	0	15	50	90	115	135	185	195

Result (exclusive prefix sums): $P = [0, 15, 50, 90, 115, 135, 185, 195]$

Inclusive prefix sums (shift by one): $[15, 50, 90, 115, 135, 185, 195, 225]$

Q37. Implement `void delete(int N)` that deletes all M th nodes such that $M \bmod N = 0$ from the doubly linked list (head is the 1st node). **(15 marks)** [CSE 203 | 2014–15]

Answer

```

1  struct listNode {
2      int item;
3      listNode *next, *prev;
4  };
5  listNode *head, *tail;
6
7  void deleteEveryNth(int N) {
8      listNode* cur = head;
9      int pos = 1;
10
11     while (cur != nullptr) {
12         listNode* nxt = cur->next;    // save before
13                                         possible delete
14
15         if (pos % N == 0) {
16             // Relink prev side

```

```

16         if (cur->prev != nullptr)
17             cur->prev->next = cur->next;
18         else
19             head = cur->next;           // deleting head
20
21         // Relink next side
22         if (cur->next != nullptr)
23             cur->next->prev = cur->prev;
24         else
25             tail = cur->prev;           // deleting tail
26
27         delete cur;
28     }
29     pos++;
30     cur = nxt;
31 }
32 }

```

Example. List: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$, $N = 3$:

Delete positions 3, 6, 9 $\Rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8$.

Complexity Analysis. $O(n)$ — single pass; each deletion is $O(1)$ pointer manipulation.

Q38. Compare worst-case run times of ArrayList and LinkedList (doubly, with head and tail) for the listed operations. (4×3=12 marks) [CSE 203 | 2014–15]

Answer

Operation	ArrayList	LinkedList
Insert at N -th position (pre-serve order)	$O(n)$ — shift elements right	$O(n)$ — traverse to pos N , then $O(1)$ relink
Remove first item (preserve order)	$O(n)$ — shift all elements left	$O(1)$ — update head
Remove N -th item (preserve order)	$O(n)$ — traverse + shift	$O(n)$ — traverse to pos N , then $O(1)$ relink
Get (without removing) N -th item	$O(1)$ — direct index access	$O(n)$ — traverse from head

Key insight. ArrayList excels at random access; LinkedList excels at front insertions/deletions. Both are $O(n)$ for arbitrary position insert/delete, but ArrayList shifts data while LinkedList traverses pointers.

Q39. Give two advantages and disadvantages of using array-based lists over linked lists.

(8 marks)

[CSE 203 | 2014–15]

Answer**Advantages of array-based lists:**

- (a) **$O(1)$ random access.** Index arithmetic computes the address directly: $\text{addr} = \text{base} + i * \text{size}$. Linked lists require $O(n)$ traversal.
- (b) **Better cache performance.** Elements are stored contiguously in memory, so prefetching and cache lines are used efficiently. Linked list nodes may be scattered across the heap, causing frequent cache misses.

Disadvantages of array-based lists:

- (a) **Fixed / costly resizing.** The capacity must be declared upfront. Dynamic resizing (doubling) is amortised $O(1)$, but incurs a one-time $O(n)$ copy and wastes up to half the array at any time.
- (b) **Costly insertions/deletions.** Inserting or removing an element at position i requires shifting up to $n - i$ elements, giving $O(n)$ in the worst case. Linked lists do this in $O(1)$ once the position is known.

Q40. Implement a **stack** using a singly linked list (**push**, **pop**, **size**). Only **head** and **tail** pointers; no other global/static variables. (15 marks) [CSE 203 | 2014–15]

Answer

Strategy. Use the *head* as the top of the stack. **push** inserts at head; **pop** removes from head. This makes both operations $O(1)$. **size** is encoded implicitly: we infer it by traversal, but to satisfy “no extra global variables” we walk the list.

```

1  struct listNode {
2      int item;
3      listNode* next;
4  };
5  listNode* head = nullptr;
6  listNode* tail = nullptr; // tail unused by stack, kept for
   ADT
7
8  void push(int x) {
9      listNode* n = new listNode{x, head};
10     head = n;
11     if (tail == nullptr) tail = n; // first element
12 }
13
14 int pop() {
15     if (head == nullptr) return -1; // underflow sentinel
16     listNode* tmp = head;
17     int val = tmp->item;
```

```

18     head = head->next;
19     if (head == nullptr) tail = nullptr;
20     delete tmp;
21     return val;
22 }
23
24 // O(n) size by traversal (no extra variable allowed)
25 int size() {
26     int count = 0;
27     listNode* cur = head;
28     while (cur != nullptr) { count++; cur = cur->next; }
29     return count;
30 }

```

Complexity Analysis. push: $O(1)$. pop: $O(1)$. size: $O(n)$.

Remark. If an $O(1)$ size is required, one would typically maintain an integer counter — but the problem forbids extra global variables. Using the linked list head-insertion approach still gives optimal push/pop.

- Q41.** Suppose we double the memory of an array-based list every time it is full during insertion. Show the average time per insertion for copying is constant. **(12 marks)**
[CSE 203 | 2014–15]

Answer

Setup. Start with capacity 1; double whenever full. After n insertions the doublings occur at sizes $1, 2, 4, 8, \dots, 2^k$ where $2^k \leq n$.

Total copy cost. When the array doubles to size s , we copy $s/2$ elements. Summing over all doublings:

$$\text{Total copies} = 1 + 2 + 4 + \dots + 2^{k-1} + 2^k = 2^{k+1} - 1 < 2n.$$

(Geometric series with ratio 2.)

Amortised cost.

$$\text{Average copies per insertion} = \frac{\text{Total copies}}{n} < \frac{2n}{n} = 2 = O(1).$$

Accounting argument. Charge each element \$2 at insertion time: \$1 pays for its own insertion, and \$1 is saved. When the array doubles, the saved credit of each “old” element pays for copying it. Hence every doubling is fully paid for, so the amortised cost per insertion is $O(1)$.

Complexity Analysis. Amortised $O(1)$ per insertion, $O(n)$ total for n insertions.

Q42. Give two reasons why we should use restrictive data structures (e.g., Stack, Queue) instead of general-purpose ones (e.g., List). **(4 marks)** [CSE 203 | 2014–15]

Answer

- (a) **Semantic clarity and correctness.** A stack enforces LIFO discipline; a queue enforces FIFO. Using a general list allows any element to be accessed/modified, making it easy to accidentally violate the intended order. Restricted interfaces prevent such bugs at the API level (e.g. a function call stack must never allow arbitrary middle-frame deletion).
- (b) **Optimised implementation and encapsulation.** Because the interface is constrained, the implementation can be tuned for exactly those operations (e.g. array-based stack has $O(1)$ push/pop with no overhead). It also hides internal details, making the code easier to reason about, maintain, and replace.

Q43. Given a doubly linked list and an integer n , write a function `deleteLastNElements()` that deletes the last n nodes from the given linked list. **(8 marks)** [CSE 203 | 2013–14]

Answer

Algorithm 33 DELETELASTNELEMENTS(*head*, *tail*, *n*)

```

1: if  $n \leq 0$  then
2:   return
3: end if
4:  $curr \leftarrow tail$ ;  $count \leftarrow 0$ 
5: while  $curr \neq \text{nil}$  and  $count < n$  do
6:    $toDelete \leftarrow curr$ 
7:    $curr \leftarrow curr.prev$ 
8:    $count \leftarrow count + 1$ 
9:   FREE( $toDelete$ )
10: end while
11: if  $curr = \text{nil}$  then                                     ▷ deleted all nodes
12:    $head \leftarrow \text{nil}$ ;  $tail \leftarrow \text{nil}$ 
13: else                                                       ▷  $curr$  is the new tail
14:    $curr.next \leftarrow \text{nil}$ ;  $tail \leftarrow curr$ 
15: end if

```

Time complexity: $O(n)$ — traverse n nodes from the tail.

Space complexity: $O(1)$ — no extra space used.

Edge cases handled:

- $n \geq \text{list size}$: all nodes deleted, head and tail set to NULL.
- $n = 0$: no deletion.

Q44. How do you implement a Queue using a Stack? What are the running times of

enqueue and dequeue operations of such an implementation? **(8+2=10 marks)**
 [CSE 203 | 2013–14]

Answer

Idea: Use **two stacks** S_1 (inbox) and S_2 (outbox).

Algorithm 34 ENQUEUE($item$) — two-stack queue

1: $S_1.PUSH(item)$ ▷ always push to inbox

Algorithm 35 DEQUEUE() — two-stack queue

```

1: if  $S_1$  is empty and  $S_2$  is empty then
2:   print "Error: queue empty";
3:   return -1
4: end if
5: if  $S_2$  is empty then
6:   while  $S_1$  is not empty do ▷ transfer to outbox
7:      $S_2.PUSH(S_1.POP())$ 
8:   end while
9: end if
10: return  $S_2.POP()$  ▷ top of  $S_2$  is oldest element

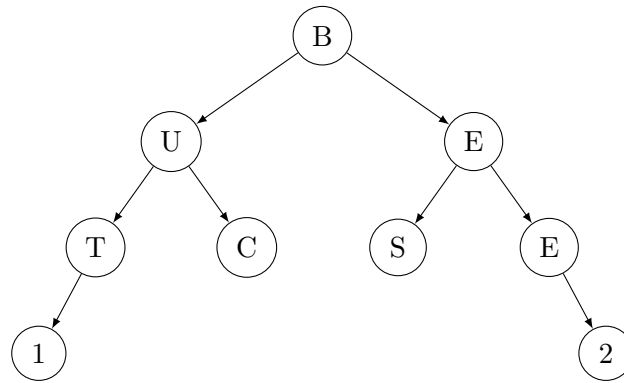
```

Example: Enqueue 1, 2, 3 $\rightarrow S_1 = [1, 2, 3]$ (top=3). dequeue: move all to $S_2 = [3, 2, 1]$ (top=1), pop 1. ✓

Running times:

- enqueue: $O(1)$ always (single push to S_1).
- dequeue: $O(n)$ **worst case** (when S_2 is empty and all n elements must be transferred from S_1 to S_2); **amortized** $O(1)$ — each element is pushed and popped at most twice total.

Q45. Which type of traversal will traverse the binary tree BUETCSE12 (as shown in figure)?
 Write an algorithm for such traversal of a binary tree using a Stack or Queue.
(2+10=12 marks) [CSE 203 | 2013–14]



Answer

Traversal type: Level-order (Breadth-First) traversal.

Reading the tree level by level from left to right: Level 0: B, Level 1: U, E, Level 2: T, C, S, E, Level 3: 1, 2 $\Rightarrow$ B, U, E, T, C, S, E, 1, 2 = BUETCSE12 ✓

Algorithm using a Queue:

Algorithm 36 LEVELORDERTRAVERSAL(*root*)

```

1: if root = nil then
2:   return
3: end if
4: Q  $\leftarrow$  empty queue
5: ENQUEUE(Q, root)
6: while Q  $\neq \emptyset$  do
7:   node  $\leftarrow$  DEQUEUE(Q)
8:   print node.data
9:   if node.left  $\neq$  nil then
10:    ENQUEUE(Q, node.left)
11:   end if
12:   if node.right  $\neq$  nil then
13:    ENQUEUE(Q, node.right)
14:   end if
15: end while
  
```

Trace on BUETCSE12 tree:

Dequeued	Printed	Queue after
B	B	[U, E]
U	U	[E, T, C]
E	E	[T, C, S, E]
T	T	[C, S, E, 1]
C	C	[S, E, 1]
S	S	[E, 1]
E	E	[1, 2]
1	1	[2]
2	2	[]

Output: B U E T C S E 1 2 = BUETCSE12 ✓

Time complexity: $O(n)$. **Space complexity:** $O(w)$ where w is the maximum width of the tree.

Q46. Give 2 (two) real-life applications each for: (i) Stack, (ii) Queue, (iii) Priority Queue.
(2+2+2=6 marks) [CSE 203 | 2013–14]

Answer

(i) **Stack:**

1. **Function call stack:** The OS uses a stack to manage function calls and returns in program execution (each call pushes a frame, return pops it).
2. **Undo/Redo in text editors:** Each edit operation is pushed; pressing Undo pops the most recent operation.

(ii) **Queue:**

1. **CPU process scheduling (FCFS):** Processes waiting for the CPU are stored in a queue; the first to arrive is the first to be served.
2. **Printer spooler:** Print jobs are queued; the first job submitted is the first to be printed.

(iii) **Priority Queue:**

1. **Hospital emergency room:** Patients are treated based on severity of illness, not arrival order.
2. **Dijkstra's shortest path algorithm:** Vertices are extracted in order of their current shortest distance estimate, stored in a min-priority queue.

Q47. Write pseudocodes for enqueue and dequeue operations for the array implementation of a circular queue. (7 marks) [CSE 203 | 2013–14]

Answer

Variables: array $Q[0..MAX-1]$, integers **front**, **rear**, **size** (all initially 0).

Algorithm 37 ENQUEUE($item$) — array circular queue

```

1: if  $size = MAX$  then
2:   print "Error: Queue is full"; return  $-1$ 
3: end if
4:  $Q[rear] \leftarrow item$ 
5:  $rear \leftarrow (rear + 1) \bmod MAX$ 
6:  $size \leftarrow size + 1$ 
7: return  $0$ 

```

Algorithm 38 DEQUEUE() — array circular queue

```

1: if  $size = 0$  then
2:   print "Error: Queue is empty"; return  $-1$ 
3: end if
4:  $val \leftarrow Q[front]$ 
5:  $front \leftarrow (front + 1) \bmod MAX$ 
6:  $size \leftarrow size - 1$ 
7: return  $val$ 

```

Why circular? Using modular arithmetic on **front** and **rear** avoids the “drifting queue” problem where usable space at the beginning of the array goes to waste after dequeues.

Time complexity: Both operations are $O(1)$.

Q48. Give a comparison of array and linked list. (6 marks)

[CSE 203 | 2013–14]

Answer

Property	Array	Linked List
Memory allocation	Static (compile time) or dynamic but contiguous	Dynamic, non-contiguous nodes
Access time	$O(1)$ random access via index	$O(n)$ sequential access
Insertion/Deletion (middle)	$O(n)$ (shifting required)	$O(1)$ if pointer to node is known
Memory usage	No extra overhead per element	Extra pointer(s) per node
Cache performance	Excellent (spatial locality)	Poor (scattered memory)
Size flexibility	Fixed (static) or expensive to resize	Easily grows/shrinks

Use array when: frequent random access, size known in advance, cache performance matters.

Use linked list when: frequent insertions/deletions, size unknown or highly variable.

Q49. Write down a linear-time algorithm for finding the maximum sum of a consecutive subsequence of an array. **(15 marks)** [CSE 207 | 2013–14]

Answer

Algorithm (Kadane's):

Algorithm 39 MAX-SUBARRAY(A, n)

```

1:  $maxsum \leftarrow -\infty$ 
2:  $currentsum \leftarrow 0$ 
3:  $start \leftarrow 0$ ;  $end \leftarrow 0$ ;  $tempstart \leftarrow 0$ 
4: for  $i = 1$  to  $n$  do
5:    $currentsum \leftarrow currentsum + A[i]$ 
6:   if  $currentsum > maxsum$  then
7:      $maxsum \leftarrow currentsum$ 
8:      $start \leftarrow tempstart$ 
9:      $end \leftarrow i$ 
10:  end if
11:  if  $currentsum < 0$  then
12:     $currentsum \leftarrow 0$ 
13:     $tempstart \leftarrow i + 1$ 
14:  end if
15: end for
16: return  $maxsum, start, end$ 

```

Correctness: At each index i , $currentsum$ holds the maximum subarray sum ending at i . If this drops below 0, any future subarray is better off starting fresh from $i + 1$.

Key invariant:

$$currentsum_i = \max_{j \leq i} \sum_{k=j}^i A[k] \quad \text{and} \quad maxsum = \max_i currentsum_i$$

Complexity: One pass $\Rightarrow O(n)$ time, $O(1)$ extra space.

Edge cases:

- All elements negative: return the single largest element (initialize $maxsum = -\infty$).
- All elements non-negative: entire array is the answer.

Q50. Write two functions for the insertion and deletion operations in a circular linked list.
(5+5=10 marks) [CSE 203 | 2012–13]

Answer

We maintain a single **tail** pointer. **tail->next** is always the head, so the list is never broken.

```

1  struct Node { int val; Node* next; };
2
3  // Insert val at the END of the circular list
4  void insertEnd(Node* &tail, int val) {
5      Node* n = new Node{val, nullptr};
6      if (tail == nullptr) {

```

```

7         n->next = n;                // points to itself
8         tail = n;
9     } else {
10        n->next    = tail->next;    // n -> old head
11        tail->next = n;            // old tail -> n
12        tail      = n;            // update tail
13    }
14 }
15
16 // Delete first occurrence of val in the circular list
17 void deleteVal(Node* &tail, int val) {
18     if (tail == nullptr) return;    // empty list
19
20     Node* cur  = tail->next;        // start at head
21     Node* prev = tail;
22
23     do {
24         if (cur->val == val) {
25             if (cur == tail && cur->next == cur) {
26                 // only one node
27                 delete cur;
28                 tail = nullptr;
29                 return;
30             }
31             prev->next = cur->next;
32             if (cur == tail) tail = prev; // deleted tail
33             delete cur;
34             return;
35         }
36         prev = cur;
37         cur  = cur->next;
38     } while (cur != tail->next);    // full circle
39 }

```

Complexity Analysis. insertEnd: $O(1)$.
deleteVal: $O(n)$ (search), $O(1)$ relinking.

Q51. Explain the advantages and disadvantages of arrays and linked lists. (5 marks)
[CSE 203 | 2011–12]

Answer

	Array	Linked List
Advantages	Random access $O(1)$ via index; cache-friendly (contiguous memory); no extra pointer overhead	Dynamic size (no pre-allocation); $O(1)$ insert/delete at known pointer; no unused memory waste
Disadvantages	Fixed size (may waste or overflow); $O(n)$ insert/delete (shifting); re-sizing is costly	No random access ($O(n)$ search); extra memory per node (pointer fields); poor cache locality

Q52. Write pseudocodes for the **Insert** and **Delete** operations of a circular queue. (6 marks) [CSE 203 | 2011–12]

Answer**Algorithm 40** INSERT(x) — circular queue

```

1: if  $sz = capacity$  then
2:   error "Queue Overflow"
3: end if
4:  $rear \leftarrow (rear + 1) \bmod capacity$ 
5:  $arr[rear] \leftarrow x$ 
6:  $sz \leftarrow sz + 1$ 

```

Algorithm 41 DELETE() — circular queue

```

1: if  $sz = 0$  then
2:   error "Queue Underflow"
3: end if
4:  $val \leftarrow arr[front]$ 
5:  $front \leftarrow (front + 1) \bmod capacity$ 
6:  $sz \leftarrow sz - 1$ 
7: return  $val$ 

```

Remark. The **sz** counter is the cleanest way to distinguish full ($sz = capacity$) from empty ($sz = 0$). Without it one must sacrifice one slot or use a boolean flag to tell apart these two states when $front == (rear+1) \bmod capacity$.

Q53. Write down a linear-time algorithm for computing the maximum sum of a consecutive subsequence. Simulate the algorithm on the following array, showing values of

maxsum and suffixmax:

2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2, -3

(15 marks)

[CSE 207 | 2007–08]

Answer

Algorithm (Kadane's / suffix-max variant):

Algorithm 42 MAX-SUBARRAY-SIMPLE(A, n)

```

1:  $maxsum \leftarrow 0$ 
2:  $suffixmax \leftarrow 0$ 
3: for  $i = 1$  to  $n$  do
4:    $suffixmax \leftarrow \max(0, suffixmax + A[i])$ 
5:    $maxsum \leftarrow \max(maxsum, suffixmax)$ 
6: end for
7: return  $maxsum$ 

```

Idea: suffixmax tracks the maximum subarray sum ending at index i . If it goes negative we reset to 0 (start a new subarray). maxsum records the global best seen so far.

Time: $O(n)$. **Space:** $O(1)$.

Simulation on $A = [2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2, -3]$:

i	$A[i]$	suffixmax	maxsum
1	2	2	2
2	-1	1	2
3	3	4	4
4	1	5	5
5	2	7	7
6	-4	3	7
7	5	8	8
8	2	10	10
9	-3	7	10
10	7	14	14
11	2	16	16
12	4	20	20
13	-1	19	20
14	2	21	21
15	-3	18	21

Maximum subarray sum = 21, achieved by the subarray $[2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2]$ (indices 1–14), with sum = 21.

- Q54.** Suppose you are given an implementation of a list Abstract Data Type (ADT) which has the functions listed in Table A. Implement a stack which has the functionality shown in Table B. In your implementation, you can only use the list implementation of Table A. Assume that the elements are integers. **(15 marks)**

Answer

Strategy. Use the *end* of the list as the top of the stack so that **push** $\equiv$ **append** and **pop** $\equiv$ **move-to-end** then **remove** — both $O(1)$.

Algorithm 43 CLEAR()

```

1: while list.LENGTH() > 0 do
2:   list.MOVEToEnd()
3:   list.REMOVE()
4: end while

```

Algorithm 44 PUSH(*item*) — list-based stack

```

1: list.APPEND(item)

```

Algorithm 45 POP() — list-based stack

```

1: if list.LENGTH() = 0 then
2:   error "Stack underflow"
3: end if
4: list.MOVEToEnd()
5: list.PREV()
6: return list.REMOVE()

```

Algorithm 46 MAX() — list-based stack

```

1: if list.LENGTH() = 0 then
2:   error "Empty stack"
3: end if
4: list.MOVEToStart()
5: maxVal ← list.GETVALUE()
6: for i = 1 to list.LENGTH() - 1 do
7:   list.NEXT()
8:   if list.GETVALUE() > maxVal then
9:     maxVal ← list.GETVALUE()
10:  end if
11: end for
12: return maxVal

```

Complexity Analysis. push: $O(1)$. pop: $O(1)$. length: $O(1)$. max: $O(n)$ (must scan all elements).

- Q55.** Your friend uses your above stack implementation regularly. A bug has been reported for the `max()` function of your stack implementation. Now your friend needs to implement a separate independent `max()` function which uses the stack data structure (avoiding its own `max()` function). He does not have access to the list data structure of Table A. How will he implement the `max()` function? He cannot use any data structure other than your stack implementation. (10 marks)

Answer

Idea. Pop all elements into a temporary stack (reversing their order), track the maximum while doing so, then push everything back to restore the original stack.

Algorithm 47 MAX(S)

```

1: if  $S.length() = 0$  then error "Empty"
2: end if
3:  $temp \leftarrow$  new Stack
4:  $maxVal \leftarrow -\infty$ 
5: while  $S.length() > 0$  do
6:    $val \leftarrow S.pop()$ 
7:   if  $val > maxVal$  then
8:      $maxVal \leftarrow val$ 
9:   end if
10:   $temp.push(val)$ 
11: end while
12: while  $temp.length() > 0$  do
13:    $S.push(temp.pop())$ 
14: end while
15: return  $maxVal$ 

```

Remark. After `max()` the original stack is fully restored. Two passes are needed because a single pop-and-push into a second stack reverses the order; popping back reverses again, giving the original order.

Complexity Analysis. $O(n)$ time (two passes of n pops/pushes). $O(n)$ space for the temporary stack.

- Q56.** We use a doubly linked list (DLL) to keep records of some books. In the list, all the books that belong to the same publisher are placed in consecutive nodes. Now, write a function `addBook(title, publisher)` that inserts a book into the DLL as the first book for its publisher. So, if the list already has three books for a particular publisher, the 4th book will be inserted in front of the three books already in the DLL. For a new publisher, the book should be inserted at the end of the DLL. Assume that each DLL node has 'prev' and 'next' pointers and stores the title and publisher of a book as Strings. (10 marks)

Answer

```

1  #include <string>
2  using namespace std;
3
4  struct DLLNode {
5      string title, publisher;
6      DLLNode *prev, *next;
7  };
8  DLLNode *head = nullptr, *tail = nullptr;
9
10 void addBook(string title, string publisher) {
11     DLLNode* newNode = new DLLNode{title, publisher,
12                                     nullptr, nullptr};
13
14     if (head == nullptr) {                // empty list
15         head = tail = newNode;
16         return;
17     }
18
19     // Search for the first node of this publisher
20     DLLNode* cur = head;
21     while (cur != nullptr && cur->publisher != publisher)
22         cur = cur->next;
23
24     if (cur == nullptr) {
25         // Publisher not found: insert at END
26         tail->next = newNode;
27         newNode->prev = tail;
28         tail = newNode;
29     } else {
30         // Insert BEFORE cur (front of publisher block)
31         newNode->next = cur;
32         newNode->prev = cur->prev;
33         if (cur->prev != nullptr)
34             cur->prev->next = newNode;
35         else
36             head = newNode;                // becomes new head
37         cur->prev = newNode;
38     }
39 }

```

Example.

- (a) addBook("A","P1") $\Rightarrow$ A(P1)
- (b) addBook("B","P2") $\Rightarrow$ A(P1) $\leftrightarrow$ B(P2)
- (c) addBook("C","P1") $\Rightarrow$ C(P1) $\leftrightarrow$ A(P1) $\leftrightarrow$ B(P2)
- (d) addBook("D","P2") $\Rightarrow$ C(P1) $\leftrightarrow$ A(P1) $\leftrightarrow$ D(P2) $\leftrightarrow$ B(P2)

Complexity Analysis. $O(n)$ to find the publisher block; $O(1)$ insertion once found.

Q57. Suppose Figure 8.b-I gives a snapshot of the memory at one point. You are running your program for a linked list based list implementation. In your implementation each node has two variables: an integer *key* and a pointer *ptr*. The current situation of the list is illustrated in Figure 8.b-II. You have further implemented the concept of freelist so as to minimize the number of calls to ‘new’ and ‘delete’. The first, second, third and fourth (i.e., last) node correspond to address 1, 2, 3 and 4 in the memory respectively. Suppose at this point a delete operation is done for the node containing 29 followed by two append operations of two nodes containing values 1111 and 2222 respectively.

Address	Values at Memory Location
0	−31
1	12
2	14
3	29
4	100
5	112
6	4
...	...

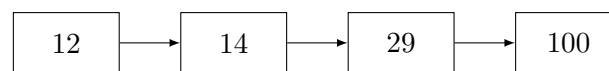


Figure 8.b-II

- Draw the updated list after each operation.
- Provide the value of *ptr* variable for each node in Figure 8.b-II before and after the delete operation. If a node remains in the memory after your deletion operation, you would need to provide the value of the *ptr* variable of that node as well.
- Provide the updated values of each memory cell after the two operations (delete and append) are complete.

You can assume that the next call of C++ ‘new’ operator will return the address 5.
(5+10+10=25 marks)

Answer

Setup. The memory before any operations is:

Addr	Value	Meaning
0	-31	freelist head pointer (addr 0 stores head of freelist; -31 means no free node yet, or sentinel)
1	12	node key (1st list node)
2	14	node key (2nd list node)
3	29	node key (3rd list node)
4	100	node key (4th / last node)
5	112	<i>not yet allocated</i>
6	4	<i>not yet allocated</i>

Each node occupies **two consecutive** memory words: [key, ptr]. So node at address a stores its key at a and its next pointer at $a+?$ — but from the snapshot only one word per address is shown, so we interpret the list pointers as follows (deduced from Figure 8.b-II):

$$1 \xrightarrow{ptr} 2 \xrightarrow{ptr} 3 \xrightarrow{ptr} 4 \xrightarrow{ptr} \text{null}$$

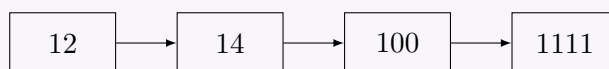
(i) Updated list after each operation.

Op 1: Delete node 29 (address 3). Node 2 (key=14) must now point to node 4 (key=100). Node 3 is returned to the freelist; the freelist head points to address 3.



(node 29 removed; addr 3 in freelist)

Op 2: Append 1111. The freelist is non-empty (head = 3); reuse address 3. Set key at addr 3 to 1111, ptr to null. Node 4 (key=100) now points to addr 3.



(addr 3 reused for 1111)

Op 3: Append 2222. Freelist is now empty; call **new**, which returns address 5 (given). Set key at addr 5 to 2222, ptr to null. Node at addr 3 (key=1111) points to addr 5.



(ii) Value of ptr for each node — before and after delete.

Node (addr)	Key	ptr before delete	ptr after delete
1	12	2	2 (unchanged)
2	14	3	4 (skip deleted node)
3	29	4	freelist ptr (e.g. <code>null</code> or previous freelist head)
4	100	<code>null</code>	<code>null</code> (unchanged)

Node 3 remains in memory (it becomes the new freelist head) but is no longer part of the active list.

(iii) Updated memory cells after all three operations (delete 29, append 1111, append 2222).

Addr	New value	Explanation
0	<code>null</code>	freelist head = <code>null</code> (addr 3 was reused, freelist empty)
1	12	unchanged
2	14	unchanged
3	1111	reused for first append (key changed from 29 to 1111)
4	100	unchanged; its ptr now points to 3
5	2222	newly allocated via <code>new</code> for second append
6	<code>null</code>	ptr of node 5 (end of list)

Remark. The freelist pattern avoids repeated `new/delete` calls. Deleted nodes are chained via their `ptr` field; the next allocation pops from this chain in $O(1)$ instead of calling the OS allocator.

BUET · CSE 105

DSA

Trees & Tree Traversals

Binary Trees, General Trees, Traversal Algorithms

T5 — Trees & Tree Traversals

Q1. Analyze the maximum and minimum number of nodes in a binary tree of height h .
 (10 marks) [CSE 105 | 2023–24]

Answer

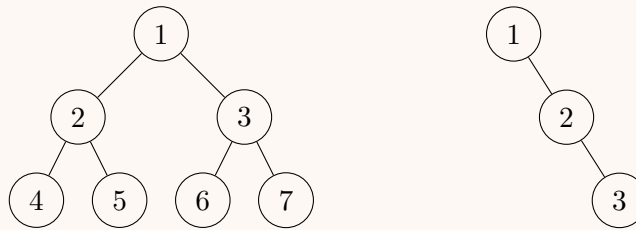
Let h denote the height of a binary tree (number of edges on the longest root-to-leaf path).

Minimum nodes. A binary tree of height h has at least $h + 1$ nodes. The extreme case is a *skewed* (degenerate) tree in which every internal node has exactly one child, forming a single path of $h + 1$ nodes.

$$N_{\min}(h) = h + 1$$

Maximum nodes. A binary tree of height h has at most $2^{h+1} - 1$ nodes. This maximum is achieved by a *perfect* binary tree, where every level $\ell \in \{0, 1, \dots, h\}$ is completely filled with 2^ℓ nodes, giving

$$N_{\max}(h) = \sum_{\ell=0}^h 2^\ell = 2^{h+1} - 1.$$



Left: perfect tree ($h = 2$, $N = 7$). Right: right-skewed tree ($h = 2$, $N = 3$).

Complexity Analysis. Summary: for a binary tree of height h , $h + 1 \leq N \leq 2^{h+1} - 1$.

Q2. Prove the following theorems that describe the properties of perfect binary trees:

- (a) A perfect tree has $2^{h+1} - 1$ nodes.
- (b) The average depth of a node is $\Theta(\ln n)$.

(4+6=10 marks)

[CSE 105 | 2022–23]

Answer

(a) A perfect binary tree of height h has $2^{h+1} - 1$ nodes.

Proof by induction on h .

Base case ($h = 0$): the tree is a single root node, so $N = 1 = 2^{0+1} - 1 = 1$. ✓

Inductive step: assume every perfect binary tree of height $h - 1$ has $2^h - 1$ nodes.

A perfect binary tree of height h consists of a root plus two perfect subtrees of height $h - 1$.

$$N(h) = 1 + 2 \cdot N(h - 1) = 1 + 2(2^h - 1) = 1 + 2^{h+1} - 2 = 2^{h+1} - 1. \quad \square$$

(b) The average depth of a node is $\Theta(\ln n)$.

Level ℓ ($0 \leq \ell \leq h$) contains 2^ℓ nodes, each at depth ℓ . The total path length (sum of depths) is

$$D = \sum_{\ell=0}^h \ell \cdot 2^\ell.$$

Using the identity $\sum_{\ell=0}^h \ell 2^\ell = (h - 1)2^{h+1} + 2$,

$$D = (h - 1)2^{h+1} + 2.$$

With $n = 2^{h+1} - 1$ nodes, $h = \log_2(n + 1) - 1 = \Theta(\log n)$, so $2^{h+1} = n + 1$.

$$\text{Average depth} = \frac{D}{n} = \frac{(h - 1)(n + 1) + 2}{n} \approx h - 1 = \Theta(\log n) = \Theta(\ln n). \quad \square$$

(Since $\log_2 n = \frac{\ln n}{\ln 2}$, $\Theta(\log_2 n) = \Theta(\ln n)$.)

Q3. You are given a tree of size n as an array `parent[0..n-1]`, where every index i represents a node and `parent[i]` is its immediate parent (-1 for the root). Write an algorithm to find the height of the generic tree.

Input: `parent[] = {-1, 0, 0, 0, 3, 1, 1, 2}` Output: 2

(15 marks)

[CSE 203 | 2021–22]

Answer

Algorithm (BFS / level-order from root):

Algorithm 48 HEIGHTFROMPARENTARRAY($parent[0..n-1]$)

```

1: Build adjacency list: for each  $i$ , add  $i$  to  $children[parent[i]]$ 
2: Find root  $r$  where  $parent[r] = -1$ 
3: Enqueue  $r$ ;  $h \leftarrow -1$ 
4: while queue not empty do
5:    $sz \leftarrow \text{queue.size}()$ 
6:   for  $j \leftarrow 1$  to  $sz$  do
7:      $v \leftarrow \text{dequeue}()$ 
8:     for each  $c \in children[v]$  do
9:       enqueue  $c$ 
10:    end for
11:  end for
12:   $h \leftarrow h + 1$ 
13: end while
14: return  $h$ 

```

Simulation on $parent[] = \{-1, 0, 0, 0, 3, 1, 1, 2\}$:

Adjacency list: $0 \rightarrow [1, 2, 3]$, $1 \rightarrow [5, 6]$, $2 \rightarrow [7]$, $3 \rightarrow [4]$; root = 0.

- Level 0: $\{0\} \Rightarrow h = 0$
- Level 1: $\{1, 2, 3\} \Rightarrow h = 1$
- Level 2: $\{5, 6, 7, 4\} \Rightarrow h = 2$; queue empty.

Output: **2** ✓

Complexity Analysis. Building adjacency list: $O(n)$. BFS visits each node once: $O(n)$. Total: $\Theta(n)$.

Q4. Write two procedures for finding the successor element and the predecessor element in a binary search tree. Write the preorder, inorder, and postorder traversal sequences of the binary tree whose Euler tour traversal sequence is a e h i h e a c b f b g b c d j d c a. Also draw the binary tree. **(10+15 marks)** [CSE 203 | 2016–17]

Answer

BST Successor and Predecessor.

```

1 // Returns the node with the smallest key >= node->key
2 treeNode* successor(treeNode* node) {
3     if (node->right != NULL)           // Case 1: right subtree
4         return minimum(node->right);    // leftmost node there
5     // Case 2: go up until we go right
6     treeNode* parent = node->parent;

```

```

7     while (parent != NULL && node == parent->right) {
8         node    = parent;
9         parent = parent->parent;
10    }
11    return parent;
12 }
13
14 // Returns the node with the largest key <= node->key
15 treeNode* predecessor(treeNode* node) {
16     if (node->left != NULL)           // Case 1: left subtree
17         exists                         // rightmost node
18         return maximum(node->left);   there
19     // Case 2: go up until we go left
20     treeNode* parent = node->parent;
21     while (parent != NULL && node == parent->left) {
22         node    = parent;
23         parent = parent->parent;
24     }
25     return parent;
26 }

```

Euler Tour → traversals.

Euler tour: a e h i h e a c b f b g b c d j d c a

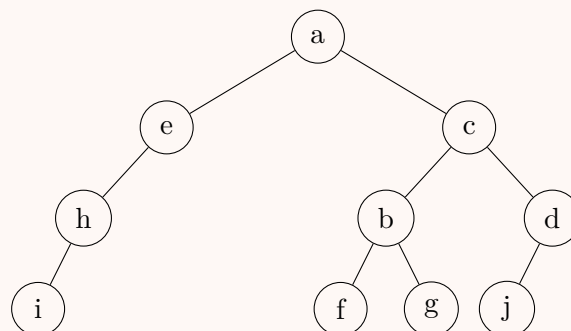
Rules:

- **Preorder:** first visit of each node.
- **Inorder:** middle visit (between left and right subtrees).
- **Postorder:** last visit of each node.

Annotating the tour (1st/2nd/3rd visit for each node):

Node	Pre (1st)	In (2nd)	Post (3rd)
a	1st pos: a	7th: a	19th: a
e	2nd: e	6th: e	— (only 2 visits ⇒ leaf's 1st=in=post)
...			

Working through the tour step by step to build the tree:



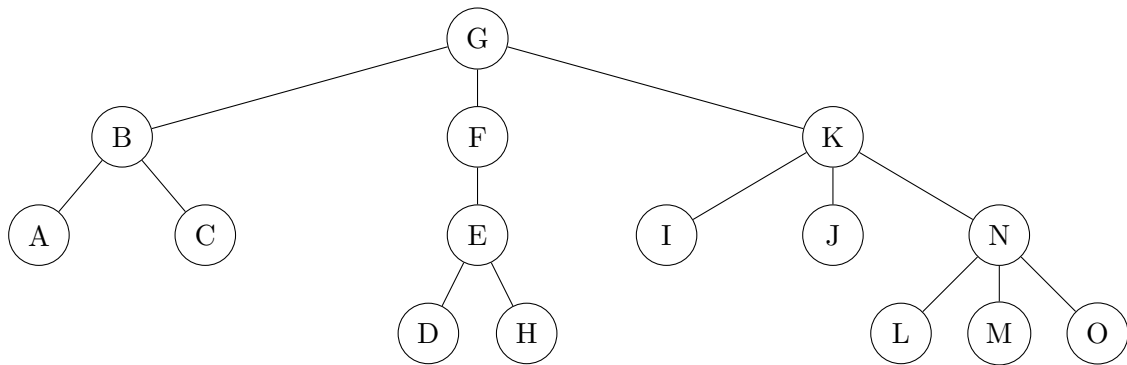
- **Preorder:** a e h i c b f g d j
- **Inorder:** i h e a f b g c j d
- **Postorder:** i h e f g b j d c a

Q5. Suppose the tree is an unordered tree (i.e., the order of children is not relevant). Determine whether each of the following breadth-first traversals is valid or not, with justification:

- (a) G F B K E C A I N J D H M L O
 (b) G F B K E C A I N J D H M L O
 (c) G K F B I J N E A C O M L H D
 (d) G F B K E C A I N J O M L H D

(16 marks)

[CSE 203 | 2015–16]



Answer

In a **BFS (level-order)** traversal of an *unordered* tree, the following rules must hold:

- The root appears first.
- Every node appears *after* its parent.
- All nodes at depth d must appear before any node at depth $d + 1$ (no interleaving of levels).
- Since the tree is *unordered*, siblings may appear in any relative order.

From the given tree, the depth-level partition is:

$$\underbrace{\{G\}}_{\text{depth 0}} \quad \underbrace{\{B, F, K\}}_{\text{depth 1}} \quad \underbrace{\{A, C, E, I, J, N\}}_{\text{depth 2}} \quad \underbrace{\{D, H, L, M, O\}}_{\text{depth 3}}$$

- (a) G F B K E C A I N J D H M L O

Valid. The depth-1 block F B K is a permutation of $\{B, F, K\}$ ✓. The depth-2 block E C A I N J is a permutation of $\{A, C, E, I, J, N\}$ ✓. The depth-3 block D H M L O is a permutation of $\{D, H, L, M, O\}$ ✓. No level is interleaved with another, and every node appears after its parent. All BFS conditions are satisfied.

(b) G F B K E C A I N J D H M L O

Valid. This sequence is identical to (i); the same argument applies.

(c) G K F B I J N E A C O M L H D

Valid. Depth-1 block K F B is a permutation of $\{B, F, K\}$ ✓. Depth-2 block I J N E A C is a permutation of $\{A, C, E, I, J, N\}$ ✓ and each child appears after its parent (I, J, N follow K ; E follows F ; A, C follow B) ✓. Depth-3 block O M L H D is a permutation of $\{D, H, L, M, O\}$ ✓. No level interleaving occurs. All BFS conditions are satisfied.

(d) G F B K E C A I N J O M L H D

Invalid. Consider the relative positions of J and O: J is at depth 2, while O is at depth 3. In the sequence, O appears at position 11 whereas J appears at position 10 — a depth-3 node (O) is listed *before* the depth-2 node (J). This violates the level-order requirement that all depth-2 nodes must be exhausted before any depth-3 node appears.

Q6. Suppose that we are given the following definition of a rooted binary tree (**not binary search tree**):

```
struct treeNode
{
    int item ;                //stores the value
    struct treeNode * left ; //points to left child
    struct treeNode * right ;//points to right child
} ;
struct treeNode * root ; //global variable to store tree root node
```

Implement the following function *clone* (**must be recursive**) that makes a copy of a binary tree (keeping the original tree intact). The function will be called using the original root pointer as input argument. The function should return the root node of the new copy. For example calling $x = \text{clone}(\text{root})$ will return the new root (of the tree copy) into variable x .

```
struct treeNode * clone(struct treeNode * node);
```

(12 marks)

[CSE 203 | 2014–15]

Answer

```

1  struct treeNode* clone(struct treeNode* node)
2  {
3      // Base case: empty subtree
4      if (node == NULL)
5          return NULL;
6
7      // Allocate a new node and copy the value
8      struct treeNode* newNode =
9          (struct treeNode*)malloc(sizeof(struct treeNode));
10     newNode->item = node->item;
11
12     // Recursively clone left and right subtrees
13     newNode->left  = clone(node->left);
14     newNode->right = clone(node->right);
15
16     return newNode;
17 }

```

Complexity Analysis. Each node is visited exactly once (post-order fashion). Time: $\Theta(n)$. Space (stack): $O(h)$, where h is the tree height.

Q7. What is a proper binary tree? Construct the proper binary tree whose inorder and postorder traversals are given as:

- Inorder: BUETCSE12
- Postorder: BEUCTES21

(2+8=10 marks)

[CSE 203 | 2013–14]

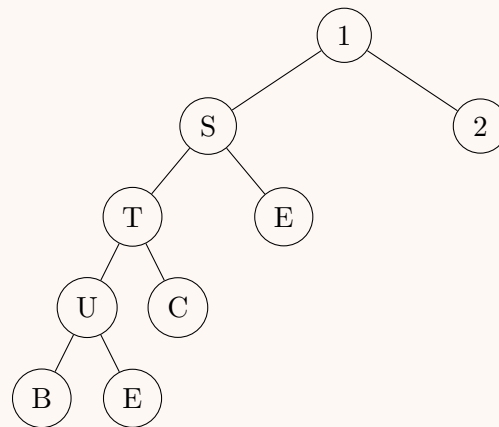
Answer

A **proper binary tree** is one where every internal node has exactly 2 children (no node has exactly 1 child).

Reconstruction (last of postorder = root; split inorder):

Step	Root	Left Inorder	Right Inorder
1	1	B U E T C S E	2
2	S (last of remaining postorder B E U C T E S 2)	B U E T C	E
3	T	B U E	C
4	U	B	E

Tree:



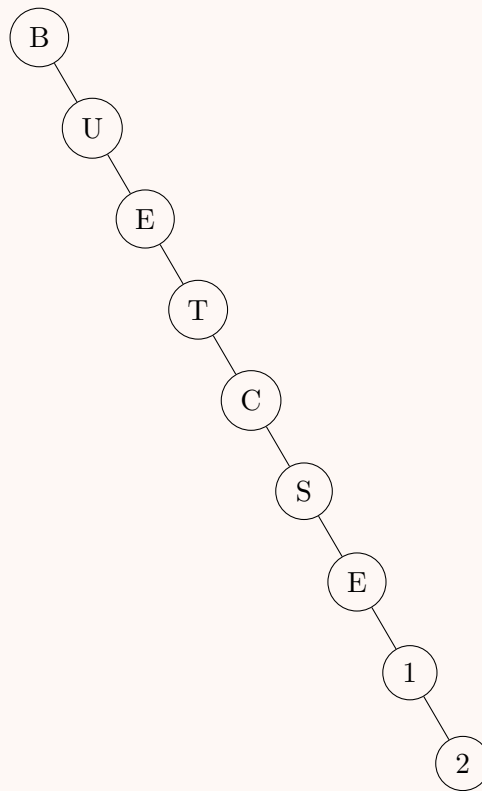
Remark. Verify: Inorder (left-root-right): B U E T C S E 1 2 ✓ Postorder (left-right-root): B E U C T E S 2 1 ✓

Q8. Draw a binary tree whose preorder and inorder traversals both yield BUETCSE12. (7 marks) [CSE 203 | 2013–14]

Answer

If **preorder** = **inorder** = B U E T C S E 1 2, then at every step the root is the first element of the preorder list, and it must also be the first element of the inorder list (otherwise there would be a non-empty left subtree, contradicting preorder = inorder).

This forces every node to have an *empty left subtree*, i.e. the tree is a **right-skewed chain**.



Remark. Preorder (root, right): B U E T C S E 1 2 ✓ Inorder (left=empty, root, right): B U E T C S E 1 2 ✓

Q9. What is a tree? Write a linear-time algorithm for finding the height of a tree. Analyze the time complexity of the algorithm. (1+4+4=9 marks) [CSE 203 | 2012–13 , 2011–12]

Answer

Tree definition (same as above — connected, acyclic graph with a root).

Linear-time algorithm using BFS (iterative level-order):

Algorithm 49 HEIGHTBFS($root$)

```

1: if  $root$  is null then
2:   return  $-1$ 
3: end if
4: Enqueue  $root$ ;  $h \leftarrow -1$ 
5: while queue is not empty do
6:    $size \leftarrow \text{queue.size}()$ 
7:   for  $i \leftarrow 1$  to  $size$  do
8:      $v \leftarrow \text{dequeue}()$ 
9:     for each child  $c$  of  $v$  do
10:      enqueue  $c$ 
11:    end for
12:   end for
13:    $h \leftarrow h + 1$ 
14: end while
15: return  $h$ 
    
```

Complexity Analysis. Each node is enqueued and dequeued exactly once $\Rightarrow$ time $\Theta(n)$, space $O(w)$ where w is the maximum width of the tree (worst case $O(n)$).

Q10. Let T be a proper binary tree with height h . Prove that the number of internal nodes in T is at least h and at most $2^h - 1$. (4+4=8 marks) [CSE 203 | 2012–13, 2011–12]

Answer

A **proper** (full) binary tree is one in which every node has either 0 or 2 children (no node has exactly one child).

Let I = number of internal nodes, L = number of leaves. For any proper binary tree: $L = I + 1$ and $N = 2I + 1$.

Lower bound: $I \geq h$.

The path from root to a deepest leaf has length h ; each node on that path except the leaf is an internal node. Hence there are at least h internal nodes.

Upper bound: $I \leq 2^h - 1$.

Proof by induction on h .

Base ($h = 0$): a single node is a leaf, so $I = 0 \leq 2^0 - 1 = 0$. ✓

Step: assume proper binary trees of height $\leq h - 1$ have at most $2^{h-1} - 1$ internal nodes. A proper binary tree of height h has a root (1 internal node) and two proper subtrees of height $\leq h - 1$, so

$$I \leq 1 + (2^{h-1} - 1) + (2^{h-1} - 1) = 2^h - 1. \quad \square$$

Q11. Construct the binary tree whose in-order and post-order traversals are given as:

- **Inorder:** i f d j g c a b k h m e

- **Postorder:** i j g d f k m h e b a c

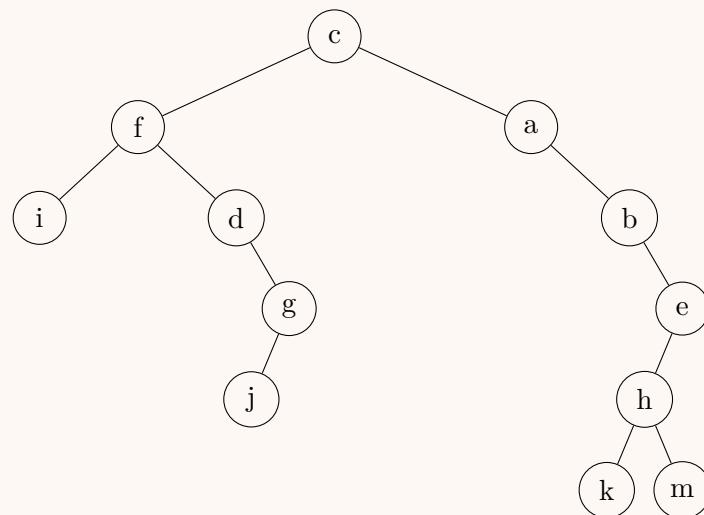
Draw the linked structure for the resulting binary tree. **(10+6=16 marks)** [CSE 203 | 2012–13]

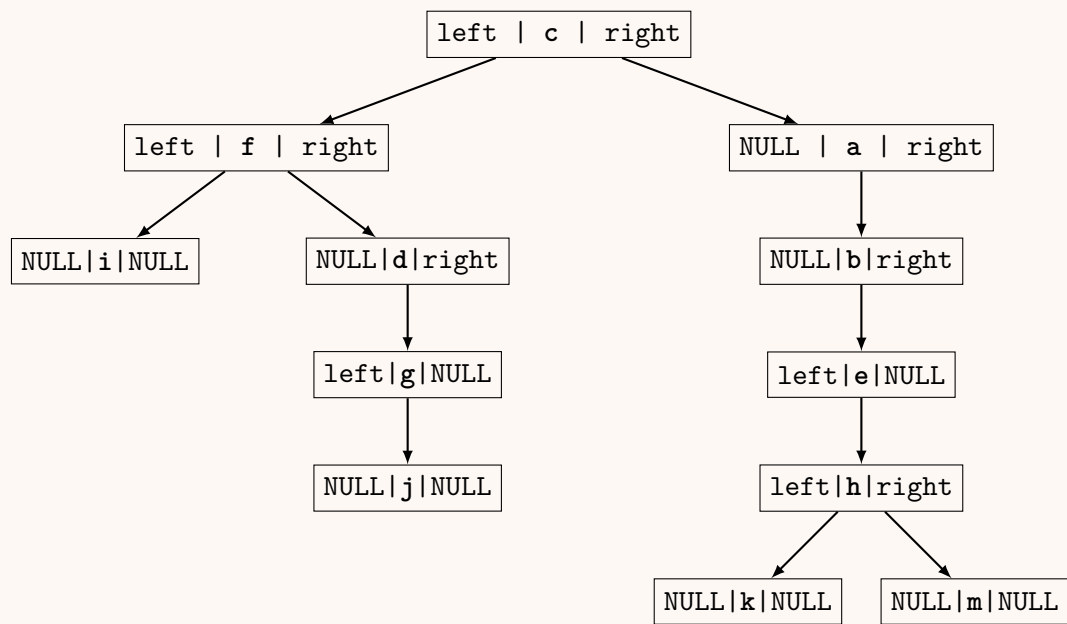
Answer

Key idea: the last element of postorder is the root; find it in inorder to split left/right subtrees; recurse.

Step	Root	Left inorder / Right inorder
1	c (last of postorder)	i f d j g / a b k h m e
2	Left subtree root: f	i / d j g
3	Left of f: i (leaf)	—
4	Right of f root: d	/ j g
5	Left of d: (empty)	
6	Right of d root: g	j / (wait—inorder j g $\Rightarrow$ j left of g) left: j (leaf), right: empty
7	Right subtree of c root: a	/ b k h m e
8	Right of a root: b	/ k h m e
9	Right of b root: e	k h m /
10	Left of e root: h	k / m left: k (leaf), right: m (leaf)

Resulting tree:





NULL pointers not drawn as arrows (indicated by NULL in node).

BUET · CSE 105

DSA

Heaps & Binary Search Trees

BST Operations, Max/Min Heap, Priority Queue, Build-Heap

T6 — Heaps & Binary Search Trees

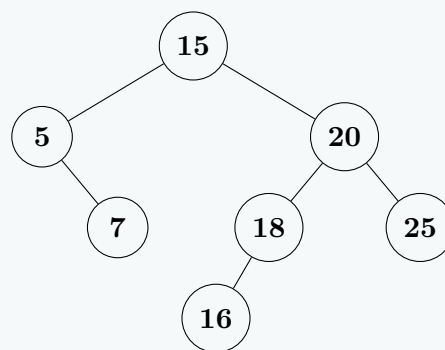
- Q1.** Show the binary search tree (BST) that results from inserting the values 15, 20, 25, 18, 16, 5, and 7 (in that order). Write the enumerations of the tree that result from pre-order, in-order, and post-order traversal of the tree. Write a recursive algorithm that takes as input the pointer to the root of a BST and a value X , and returns **true** if the value X appears in the tree and **false** otherwise. Modify the algorithm if the tree is a generalized binary tree. In both cases, analyze the time complexity of your algorithms. **(25 marks)** [CSE 105 | 2023–24]

Answer

Step 1: Build the BST by inserting values in order.

Insert 15: root. Insert 20: $20 > 15$, goes right of 15. Insert 25: $25 > 15$, go right; $25 > 20$, goes right of 20. Insert 18: $18 > 15$, go right; $18 < 20$, goes left of 20. Insert 16: $16 > 15$, go right; $16 < 20$, go left; $16 < 18$, goes left of 18. Insert 5: $5 < 15$, goes left of 15. Insert 7: $7 < 15$, go left; $7 > 5$, goes right of 5.

Final BST:



Step 2: Tree Traversals.

- **Pre-order** (Root $\rightarrow$ Left $\rightarrow$ Right): 15, 5, 7, 20, 18, 16, 25
- **In-order** (Left $\rightarrow$ Root $\rightarrow$ Right): 5, 7, 15, 16, 18, 20, 25 (*always sorted!*)
- **Post-order** (Left $\rightarrow$ Right $\rightarrow$ Root): 7, 5, 16, 18, 25, 20, 15

Step 3: Recursive BST Search Algorithm.

Algorithm 50 BSTSEARCH($root, X$)

```

1: if  $root = \text{nil}$  then
2:   return false
3: end if
4: if  $root.key = X$  then
5:   return true
6: end if
7: if  $X < root.key$  then
8:   return BSTSEARCH( $root.left, X$ )
9: else
10:  return BSTSEARCH( $root.right, X$ )
11: end if

```

How it works: At each node we exploit the BST property. If X equals the current key, we are done. If X is smaller, the answer can only lie in the left subtree; if larger, in the right subtree. This halves the search space at each level.

Step 4: Modification for a General Binary Tree.

In a general (non-BST) binary tree, there is no ordering property, so we cannot prune. We must search *both* subtrees:

Algorithm 51 GENERALSEARCH($root, X$)

```

1: if  $root = \text{nil}$  then
2:   return false
3: end if
4: if  $root.key = X$  then
5:   return true
6: end if
7: return GENERALSEARCH( $root.left, X$ ) or GENERALSEARCH( $root.right, X$ )

```

Complexity Analysis. BST Search:

- **Best case:** $O(1)$ — X is the root.
- **Average case:** $O(\log n)$ — for a balanced BST with n nodes, height = $O(\log n)$.
- **Worst case:** $O(n)$ — for a completely skewed BST (degenerate tree resembling a linked list), height = $n - 1$.

General Binary Tree Search: Always $\Theta(n)$ worst case, since every node may need to be visited.

Q2. Assume you are given a set of integer numbers. Design a data structure and associated algorithms that support finding the minimum and maximum of the numbers in constant time, and insertion, deletion of maximum, and deletion of minimum in

$O(\log n)$ time. *Hint: use two different heaps on the same set of numbers; however, remember their mutual dependencies.* **(15 marks)** [CSE 105 | 2023–24]

Answer

Idea. Maintain the *same* set of n numbers simultaneously in two heap structures:

- A **Max-Heap** H_{max} : root always holds the **maximum**.
- A **Min-Heap** H_{min} : root always holds the **minimum**.

Each element has a pointer in both heaps so that its position in one heap is known from the other. This is the key *mutual dependency*.

Operations:

- (a) **findMax()** — $O(1)$: Return $H_{max}.root$.
 (b) **findMin()** — $O(1)$: Return $H_{min}.root$.
 (c) **insert(x)** — $O(\log n)$:

Algorithm 52 INSERT(x)

- 1: Insert x into H_{max}
 - 2: Insert x into H_{min}
 - 3: Record cross-pointers between x 's positions in both heaps
-

- (d) **deleteMax()** — $O(\log n)$:

Algorithm 53 DELETMAX()

- 1: $m \leftarrow H_{max}.root$
 - 2: Use cross-pointer to locate m 's position p in H_{min}
 - 3: Remove m from H_{max}
 - 4: Remove m from H_{min} at position p via pointer **return** m
-

- (e) **deleteMin()** — $O(\log n)$: Symmetric to **deleteMax**.

Complexity Analysis.

Operation	Time	Reason
findMin, findMax	$O(1)$	Read heap root
insert	$O(\log n)$	Bubble-up in both heaps
deleteMax, deleteMin	$O(\log n)$	Heapify in both heaps

Remark. The cross-pointer is the critical design choice. Without it, finding an arbitrary element in a heap takes $O(n)$. With it, deletion at a known position takes $O(\log n)$ using heap fix-up operations.

Q3. The key at index 6 of the following max priority queue has been increased to 32. Re-establish the max-heap property of the queue and illustrate the steps using the

array only. Justify whether you can use the `max_heapify` algorithm to re-establish the heap property.

1	2	3	4	5	6	7	8	9	10	11	12	13
25	23	20	15	18	16	12	14	7	8	11	4	14

(12 marks)

[CSE 105 | 2022–23]

Answer — Max-Heap: Increase Key at Index 6

Original heap (1-indexed):

[25, 23, 20, 15, 18, **16**, 12, 14, 7, 8, 11, 4, 14]

Index 6 (value 16) increased to **32**:

[25, 23, 20, 15, 18, **32**, 12, 14, 7, 8, 11, 4, 14]

Why not `max_heapify`? `max_heapify` fixes violations by pushing a node *downward*. Here the key *increased*, so the violation is upward (node may be larger than its parent). We need **HEAP-INCREASE-KEY** (bubble up).

Bubble-up steps:

Step 1: $\text{heap}[6] = 32 > \text{heap}[3] = 20$ (parent of 6 is $\lfloor 6/2 \rfloor = 3$). **Swap.**

[25, 23, **32**, 15, 18, **20**, 12, 14, 7, 8, 11, 4, 14]

Step 2: $\text{heap}[3] = 32 > \text{heap}[1] = 25$ (parent of 3 is 1). **Swap.**

[**32**, 23, **25**, 15, 18, 20, 12, 14, 7, 8, 11, 4, 14]

Step 3: Node is now at root (index 1). **Stop.**

Final valid max-heap:

[32, 23, 25, 15, 18, 20, 12, 14, 7, 8, 11, 4, 14]. ✓

Q4. Given two max-priority queues represented by two arrays, write an efficient algorithm to merge them into a single max-priority queue. Analyse the time complexity of your algorithm. (17 marks) [CSE 105 | 2022–23]

Answer

Key Idea. Concatenate both arrays into one, then run the standard BUILD-MAX-HEAP procedure bottom-up. This avoids inserting elements one by one (which would cost $O((m+n)\log(m+n))$) and instead exploits the linear build-heap fact.

Algorithm 54 MAX-HEAPIFY(A, i, n)

```

1:  $left \leftarrow 2i + 1$ ;  $right \leftarrow 2i + 2$ ;  $largest \leftarrow i$ 
2: if  $left < n$  and  $A[left] > A[largest]$  then
3:    $largest \leftarrow left$ 
4: end if
5: if  $right < n$  and  $A[right] > A[largest]$  then
6:    $largest \leftarrow right$ 
7: end if
8: if  $largest \neq i$  then
9:   SWAP( $A[i], A[largest]$ )
10:  MAX-HEAPIFY( $A, largest, n$ )
11: end if

```

Algorithm 55 MERGE-MAX-HEAPS(A, m, B, n)

```

1:  $C \leftarrow$  new array of size  $m + n$ 
2: for  $i = 0$  to  $m - 1$  do
3:    $C[i] \leftarrow A[i]$ 
4: end for
5: for  $i = 0$  to  $n - 1$  do
6:    $C[m + i] \leftarrow B[i]$ 
7: end for
8:  $total \leftarrow m + n$ 
9: for  $i = \lfloor total/2 \rfloor - 1$  downto  $0$  do
10:  MAX-HEAPIFY( $C, i, total$ )
11: end for
12: return  $C$ 

```

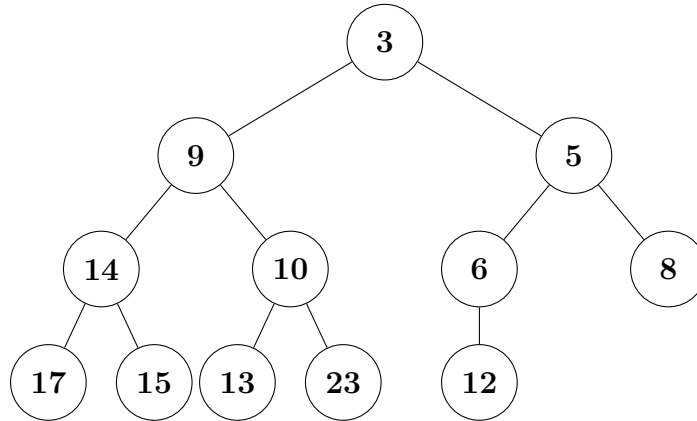
Correctness. After Step 1 the combined array C contains all $m + n$ elements. BUILD-MAX-HEAP (the bottom-up loop) restores the max-heap property for every subtree, guaranteeing that C is a valid max-heap upon return.

Complexity Analysis.

- **Concatenation (Step 1):** $O(m + n)$
- **Build-Max-Heap (Step 2):** $O(m + n)$ — each call to MAX-HEAPIFY costs $O(\log(m + n))$, but the sum over all $\lfloor (m + n)/2 \rfloor$ calls is $O(m + n)$ by the standard geometric series argument.
- **Total:** $O(m + n)$

Remark. An alternative using a Fibonacci heap supports UNION in $O(1)$ amortised, but requires a more complex structure. The array-based BUILD-MAX-HEAP approach above is simpler and optimal for array-represented heaps.

- Q5.** Write a code snippet to traverse the binary tree in an order so that you can store it in an array. Show the placement of the node numbers of the given tree in an array. Write a code to access the parent of a given node. What will be the index no of the array for node 12? **(10+3+8+4=25 marks)** [CSE 105 | 2022–23]



Answer

Key Idea. Use **level-order (BFS) traversal** to fill the array. For a 1-indexed array, node at index i has:

- Left child at $2i$, right child at $2i + 1$.
- Parent at $\lfloor i/2 \rfloor$.

Level-order traversal code:

```

1  #include <queue>
2  #include <vector>
3  using namespace std;
4  struct Node { int val; Node *left, *right; };
5
6  // Returns 1-indexed level-order array
7  vector<int> levelOrderArray(Node* root) {
8      vector<int> arr = {0};           // index 0 unused
9      if (!root) return arr;
10     queue<Node*> q;
11     q.push(root);
12     while (!q.empty()) {
13         Node* cur = q.front(); q.pop();
14         arr.push_back(cur->val);
15         if (cur->left) q.push(cur->left);
16         if (cur->right) q.push(cur->right);
17     }
18     return arr;
19 }
20
21 // Access parent of node at index i (1-indexed)
22 int parentIndex(int i) { return i / 2; }
23 int parentValue(vector<int>& arr, int i) {

```

```

24     if (i <= 1) return -1;           // root has no parent
25     return arr[i / 2];
26 }

```

Tree from figure (level-order placement):

Index	1	2	3	4	5	6	7	8	9	10	11	12
Value	3	9	5	14	10	6	8	17	15	13	23	12

Verification of structure:

- Root (idx 1) = 3; left child = arr[2]=9; right child = arr[3]=5
- 9 (idx 2): left=arr[4]=14, right=arr[5]=10
- 5 (idx 3): left=arr[6]=6, right=arr[7]=8
- 14 (idx 4): left=arr[8]=17, right=arr[9]=15
- 10 (idx 5): left=arr[10]=13, right=arr[11]=23
- 6 (idx 6): left=arr[12]=12

Node 12 is at index 12.

Parent of index 12 = $\lfloor 12/2 \rfloor = 6 \Rightarrow \text{arr}[6] = 6$.

Parent access code:

```

1 // Given 1-indexed array arr[] of size n+1
2 // Find parent of the node at index i
3 int getParent(int arr[], int i) {
4     if (i <= 1) return -1;           // root has no parent
5     return arr[i / 2];               // O(1) direct access
6 }
7 // Example: getParent(arr, 12) = arr[6] = 6

```

Complexity Analysis. Level-order traversal: $O(n)$. Parent access: $O(1)$.

Q6. You are given the following numbers to insert into an empty Binary Search Tree (BST): 5, 7, 8, 12, 15, 27. Determine which insertion order below would yield the tree with the least height:

- 15, 5, 27, 8, 7, 12
- 12, 7, 15, 27, 5, 8
- 8, 27, 7, 5, 15, 12
- 7, 5, 12, 8, 15, 27

(10 marks)

[CSE 105 | 2021–22]

Detailed Answer

Insert values {5, 7, 8, 12, 15, 27} in each given order to construct the Binary Search Trees and measure their resulting heights. (Assuming height is defined as the num-

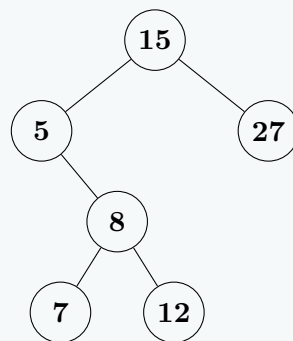
ber of edges on the longest path from root to a leaf, i.e., root is at level 0).

Option	Insertion Order	Height
(a)	15, 5, 27, 8, 7, 12	3
(b)	12, 7, 15, 27, 5, 8	2 ← minimum
(c)	8, 27, 7, 5, 15, 12	3
(d)	7, 5, 12, 8, 15, 27	3

Tree for Option (a): 15, 5, 27, 8, 7, 12

Root is 15. The longest path is $15 \rightarrow 5 \rightarrow 8 \rightarrow 7$ (or 12).

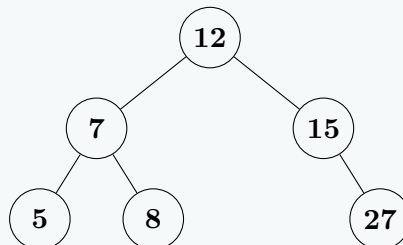
Height = 3.



Tree for Option (b) [Minimum Height]: 12, 7, 15, 27, 5, 8

Root is 12. The longest path reaches level 2 (e.g., $12 \rightarrow 7 \rightarrow 5$).

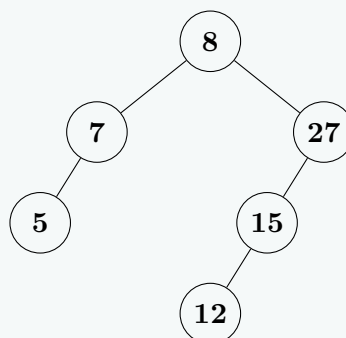
Height = 2. ✓



Tree for Option (c): 8, 27, 7, 5, 15, 12

Root is 8. The longest path is $8 \rightarrow 27 \rightarrow 15 \rightarrow 12$.

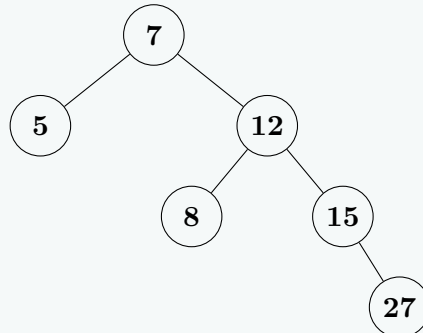
Height = 3.



Tree for Option (d): 7, 5, 12, 8, 15, 27

Root is 7. The longest path is $7 \rightarrow 12 \rightarrow 15 \rightarrow 27$.

Height = 3.



Conclusion: Inserting the median value (12) first as the root (Option b) creates a roughly balanced split between the left and right subtrees, minimising skewness. Options (a), (c), and (d) all produce trees with a height of 3 due to poorer balance.

- Q7.** A perfect balanced BST requires all interior nodes to contain two children and all leaf nodes to be on the same level. A perfect balanced BST of $h = 0$ would have $n = 1$ node, $h = 1$ would have $n = 3$ nodes, $h = 2$ would have $n = 7$ nodes, and so on. Providing appropriate arguments, derive the worst-case running time, in terms of n , for successfully searching for an element in a perfect balanced BST. **(10 marks)**
[CSE 105 | 2021–22]

Answer

Structure of a perfect balanced BST.

A perfect balanced BST of height h has:

$$n = 2^{h+1} - 1 \quad \text{nodes.}$$

For example: $h = 0 \Rightarrow n = 1$; $h = 1 \Rightarrow n = 3$; $h = 2 \Rightarrow n = 7$.

Worst-case search.

In a successful search, the deepest level visited is the leaf level at depth h . Thus the maximum number of comparisons = $h + 1$ (one per level including root).

Derivation in terms of n :

From $n = 2^{h+1} - 1$:

$$n + 1 = 2^{h+1} \implies h + 1 = \log_2(n + 1) \implies h = \log_2(n + 1) - 1.$$

Maximum comparisons = $h + 1 = \log_2(n + 1)$.

Complexity Analysis.

$$T_{\text{worst}}(n) = \log_2(n + 1) = \Theta(\log n).$$

This is the number of nodes visited on the path from root to the deepest leaf. In a perfect balanced BST each comparison eliminates half the remaining search space, leading to the logarithmic bound. For an unsuccessful search (not found), the same bound holds since we reach a null pointer at depth $h + 1$.

- Q8.** Present a recursive method `countInRange(root, low, high)`, without any helper functions, to count in a BST all keys that are within a given range. Assume keys in the BST are unique (no duplicates). For example, if the keys in a BST are 4, 7, 10, 14, 15, 17, 20, 30, then `countInRange(root, 8, 17)` returns 4 and `countInRange(root, 21, 29)` returns 0. For your convenience a code snippet in C++ is given below, but you need not follow any particular programming language. **(10 marks)** [CSE 105 | 2021–22]

```

1 public class BSTNode {
2     int key;
3     Value value;
4     BSTNode left;
5     BSTNode right;
6     ...
7 }
8 // counts number of keys in BST that are in the range low to
9 // high, inclusive
10 // i.e. all keys k such that low <= k <= high
11 public static int countInRange(BSTNode root, int low, int
12     high) {
13     // COMPLETE THIS METHOD
14 }
```

Answer

Key insight: The BST ordering property lets us prune the search. If `root.key` is below `low`, the entire left subtree is also below `low` (skip it). If `root.key` is above `high`, the entire right subtree is also above (skip it).

```

1 public static int countInRange(BSTNode root, int low, int
2     high) {
3     if (root == null) return 0; // base case:
4     // empty subtree
5     // Current key is within range: count it + recurse on
6     // both subtrees
7     if (low <= root.key && root.key <= high) {
8         return 1
9     }
```

```

7         + countInRange(root.left, low, high)
8         + countInRange(root.right, low, high);
9     }
10
11     // Current key < low: left subtree entirely out of range,
12     // go right only
13     if (root.key < low) {
14         return countInRange(root.right, low, high);
15     }
16
17     // Current key > high: right subtree entirely out of
18     // range, go left only
19     return countInRange(root.left, low, high);
20 }

```

Trace on example (keys: 4, 7, 10, 14, 15, 17, 20, 30):

countInRange(root, 8, 17): Keys in [8, 17] are 10, 14, 15, 17 $\Rightarrow$ returns 4. ✓

countInRange(root, 21, 29): No key in [21, 29] $\Rightarrow$ returns 0. ✓

Complexity Analysis.

- **Best case:** $O(\log n)$ — range is outside tree, we prune at every node.
- **Worst case:** $O(k + \log n)$, where k is the number of keys in range. We visit k matching nodes plus $O(\log n)$ boundary nodes.
- If all n keys are in range: $O(n)$.

Q9. Present an algorithm to delete a full node in a binary search tree. Explain how it works with illustrative examples. (10 marks) [CSE 105 | 2021–22]

Answer

A **full node** in a BST is a node with exactly **two children**. Deletion of such a node uses the *in-order successor* (or predecessor) strategy.

Algorithm — **deleteFullNode(N)**:

Algorithm 56 DELETEFULLNODE(N)

```

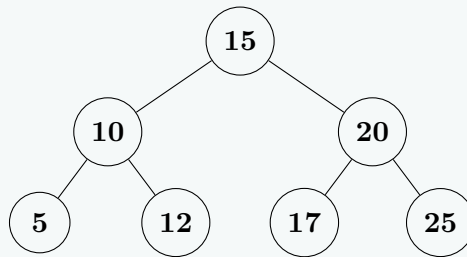
1: successor  $\leftarrow$  FINDMIN( $N.right$ )
2:  $N.key \leftarrow successor.key$ 
3: DELETENODE( $N.right, successor.key$ )

```

Why does this work? The in-order successor S is the *smallest* value greater than N . Replacing N 's key with S 's key maintains the BST property:

- All keys in N 's left subtree $< S.key$ (they were $< N.key < S.key$). ✓
- All keys in N 's right subtree $> S.key$ (since S was the minimum there). ✓

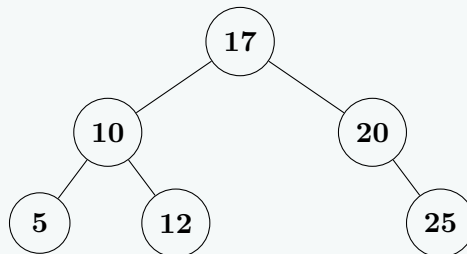
Illustrative Example. Delete node 15 from the BST below.



Step 1: Find in-order successor of 15 = **17** (leftmost in right subtree).

Step 2: Copy 17 into node 15. Delete 17 from right subtree (17 has no children — simple leaf deletion).

Result:



In-order before: 5, 10, 12, 15, 17, 20, 25. In-order after: 5, 10, 12, 17, 20, 25. ✓

Complexity Analysis. Finding the successor: $O(h)$ where h is tree height. Deleting the successor (at most one child): $O(h)$. Total: $O(h)$. For a balanced BST, $O(\log n)$; worst case $O(n)$ for skewed tree.

Q10. Consider the following array A of integers:(index start from 1)

$$A = [16, 4, 10, 7, 14, 9, 3, 2, 8, 1]$$

Verify whether the array is a max heap. Derive the complexity of your verification algorithm. Justify whether the algorithm `max_heapify(A, i)` will establish the max heap property of the array if applied with index $i = 2$. Analyze the time complexity of the `max_heapify` algorithm. **(20 marks)** [CSE 105 | 2021–22]

Answer

Array (1-indexed): $A = [16, 4, 10, 7, 14, 9, 3, 2, 8, 1]$

Step 1: Verify Max-Heap Property.

A max-heap requires $A[\lfloor i/2 \rfloor] \geq A[i]$ for all $i > 1$ (every parent $\geq$ its children).

Node i	Value	Children	Violation?
1	16	$A[2] = 4, A[3] = 10$	None
2	4	$A[4] = 7, A[5] = 14$	Yes: $4 < 7$ and $4 < 14$
3	10	$A[6] = 9, A[7] = 3$	None
4	7	$A[8] = 2, A[9] = 8$	Yes: $7 < 8$
5	14	$A[10] = 1$	None

Violations found at $i = 2$ and $i = 4$. The array is *not* a valid max-heap.

Complexity Analysis. **Heap Verification Algorithm:** Check all nodes $i = 1$ to $\lfloor n/2 \rfloor$ (internal nodes). Each check is $O(1)$. Total: $O(n)$.

Step 2: Apply `max_heapify(A, 2)`.

`max_heapify(A, i)` fixes a potential max-heap violation at node i by repeatedly swapping the node with its largest child and moving down.

Starting at $i = 2$, $A[2] = 4$. Children: $A[4] = 7$, $A[5] = 14$.

- **Iteration 1:** Largest child = $A[5] = 14 > A[2] = 4$. Swap $A[2] \leftrightarrow A[5]$.

Array: $[16, \underline{14}, 10, 7, \underline{4}, 9, 3, 2, 8, 1]$

Move down to $i = 5$.

- **Iteration 2:** At $i = 5$, $A[5] = 4$. Only child: $A[10] = 1$. $4 \geq 1$: no swap. **Stop.**

After `max_heapify(A, 2)`: $A = [16, 14, 10, 7, 4, 9, 3, 2, 8, 1]$

Is the result a full max-heap? Check $i = 4$: $A[4] = 7$, $A[9] = 8$: $7 < 8$ — still a violation! `max_heapify(A, 2)` only fixes the subtree rooted at node 2, not violations elsewhere in the array. The full array is **still not** a max-heap.

Complexity Analysis. `max_heapify` descends at most h levels where $h = \lfloor \log_2 n \rfloor$ is the height of the heap. Each level: $O(1)$ comparisons. **Time complexity:** $O(\log n)$.

Q11. Calculate the number of swap operations (data interchanges) required if you apply the `Build_Max_Heap` algorithm on the array $A = \{5, 3, 17, 10, 84, 19, 6, 22, 9\}$. Justify whether the max heap produced can be used as a priority queue. Show that `Heap_Extract_Max` algorithm is essentially a part of Heapsort. Derive and compare the time complexities of Heapsort and `Heap_Extract_Max`. **(17 marks)** [CSE 105 | 2021–22]

Answer

Step 1: Build_Max_Heap — counting swaps.

Array (1-indexed): $A = [5, 3, 17, 10, 84, 19, 6, 22, 9]$, $n = 9$.

Call `max_heapify` for $i = \lfloor 9/2 \rfloor = 4$ down to 1:

i	Array before	Array after	Swaps
4	[5, 3, 17, 10 , 84, 19, 6, 22, 9]	[5, 3, 17, 22 , 84, 19, 6, 10 , 9]	1
3	[5, 3, 17 , 22, 84, 19, 6, 10, 9]	[5, 3, 19 , 22, 84, 17 , 6, 10, 9]	1
2	[5, 3 , 19, 22, 84, 17, 6, 10, 9]	[5, 84 , 19, 22, 3 , 17, 6, 10, 9]	1
1	[5 , 84, 19, 22, 3, 17, 6, 10, 9]	[84 , 22, 19, 10 , 3, 17, 6, 5 , 9]	3

$$\text{Total swaps} = 1 + 1 + 1 + 3 = 6$$

Final max-heap: [84, 22, 19, 10, 3, 17, 6, 5, 9] — verified valid. ✓

Step 2: Can this heap be used as a priority queue?

Yes. A max-heap is the standard implementation of a **max-priority queue**. The three required operations are all supported:

- **Insert(key):** Add at the end, bubble up — $O(\log n)$.
- **Extract-Max():** Remove root, replace with last, heapify down — $O(\log n)$.
- **Maximum():** Return root — $O(1)$.

Step 3: Heap_Extract_Max is part of Heapsort.

Algorithm 57 HEAPSORT(A)

```

1: BUILD-MAX-HEAP( $A$ )
2: for  $i = n$  downto 2 do
3:   SWAP( $A[1]$ ,  $A[i]$ )
4:    $heap\text{-}size \leftarrow heap\text{-}size - 1$ 
5:   MAX-HEAPIFY( $A$ , 1)
6: end for
```

Each iteration of the loop is exactly **Heap_Extract_Max**: swap the maximum (root) to the end, reduce heap size, and call **Max-Heapify**. Thus Heapsort performs $n - 1$ repeated extract-max operations.

Complexity Analysis.

Algorithm	Time Complexity	Space
Heap_Extract_Max (single call)	$O(\log n)$	$O(1)$
Heapsort ($n - 1$ extract-max + Build-Max-Heap)	$O(n \log n)$	$O(1)$

Heapsort detail: Build-Max-Heap takes $O(n)$; the loop calls Max-Heapify $n - 1$ times, each $O(\log n)$; total = $O(n) + O(n \log n) = O(n \log n)$.

Q12. You are given two balanced binary search trees with m and n elements respectively. Write a function that merges them into a balanced BST in $O(m + n)$ time. **(15 marks)** [CSE 203 | 2021–22]

Answer**Algorithm (3 steps):****Algorithm 58** MERGEBSTs($root_1, root_2$)

```

1:  $arr_1 \leftarrow \text{INORDERTOARRAY}(root_1)$ 
2:  $arr_2 \leftarrow \text{INORDERTOARRAY}(root_2)$ 
3:  $merged \leftarrow \text{MERGESORTEDARRAYS}(arr_1, arr_2)$ 
4: return SORTEDARRAYTOBST( $merged, 0, |arr_1| + |arr_2| - 1$ )

```

Algorithm 59 INORDERTOARRAY($root$)

```

1: if  $root = \text{nil}$  then
2:   return []
3: end if
4: return INORDERTOARRAY( $root.left$ ) + [ $root.data$ ] + INORDERTOAR-
   RAY( $root.right$ )

```

Algorithm 60 MERGESORTEDARRAYS(A, B)

```

1:  $result \leftarrow []$ ;  $i \leftarrow 0$ ;  $j \leftarrow 0$ 
2: while  $i < |A|$  and  $j < |B|$  do
3:   if  $A[i] \leq B[j]$  then
4:      $result.APPEND(A[i])$ ;  $i \leftarrow i + 1$ 
5:   else
6:      $result.APPEND(B[j])$ ;  $j \leftarrow j + 1$ 
7:   end if
8: end while
9: Append remaining elements of  $A$  or  $B$  to  $result$  return  $result$ 

```

Algorithm 61 SORTEDARRAYTOBST(arr, lo, hi)

```

1: if  $lo > hi$  then
2:   return nil
3: end if
4:  $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
5:  $node \leftarrow \text{new NODE}(arr[mid])$ 
6:  $node.left \leftarrow \text{SORTEDARRAYTOBST}(arr, lo, mid - 1)$ 
7:  $node.right \leftarrow \text{SORTEDARRAYTOBST}(arr, mid + 1, hi)$ 
8: return  $node$ 

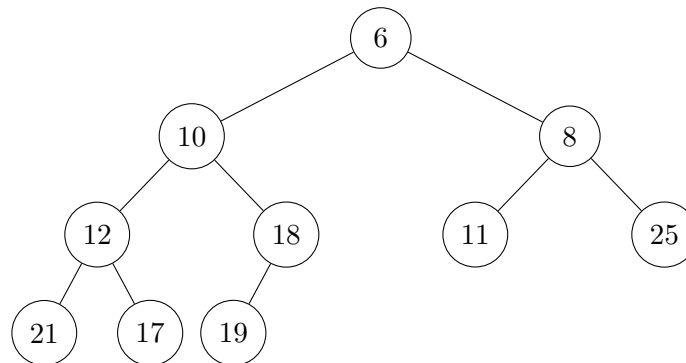
```

Correctness: In-order traversal of a BST gives sorted order. Merging two sorted arrays gives a globally sorted array. Building a BST from the middle element of a sorted array recursively produces a balanced BST.

Time complexity: $O(m) + O(n) + O(m + n) + O(m + n) = O(m + n)$ ✓

Space complexity: $O(m + n)$ for the merged array.

- Q13.** Calculate for the min heap shown in the given figure. Show the resulting min heap after deleting the minimum element (10), including intermediate steps. **(10 marks)**
 [CSE 203 | 2021–22]



Answer

Original min-heap array (level-order): [6, 10, 8, 12, 18, 11, 25, 21, 17, 19]. Valid min-heap ✓.

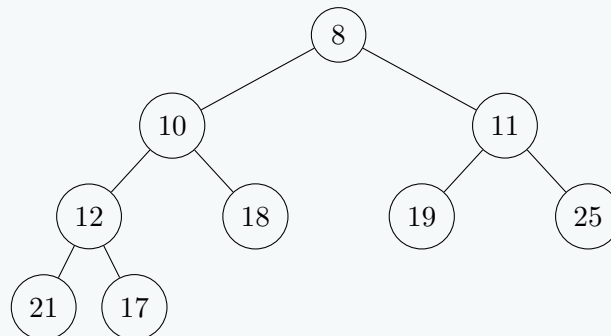
Step 1: replace root (6) with last (19); $n = 9$. Array: [19, 10, 8, 12, 18, 11, 25, 21, 17].

Step 2: heapify at 0: children 10(idx1), 8(idx2). Min=8 → swap 19↔8. Array: [8, 10, 19, 12, 18, 11, 25, 21, 17].

Step 3: heapify at 2 (val 19): children 11(idx5), 25(idx6). Min=11 → swap 19↔11. Array: [8, 10, 11, 12, 18, 19, 25, 21, 17].

Step 4: heapify at 5 (val 19): no children. Done.

Final min-heap: [8, 10, 11, 12, 18, 19, 25, 21, 17] ✓



- Q14.** While preparing for this exam, you have found an interesting question in a very old book with missing pages. It talks about a Binary Search Tree (BST) that stores integers in the range from 1 to 100 and proposes to search 22 and print the values encountered on the search path through the BST. Then it provides the following sequences of numbers (Sequences 1–3) and asks the following question: “Are these sequences possible search paths through the BST?”

Sequence 1 : 100, 40, 20, 30, 23, 35, 22

Sequence 2 : 100, 35, 15, 32, 20, 22

Sequence 3 : 50, 60, 90, 40, 75

You have to answer the above question. If your answer is ‘yes’, please show the path (with branches); if ‘no’, explain why.

[Hint: you were thinking whether there was a missing page with a figure showing the BST, but your genius little brother smiled and hinted that no need to check for a figure as the answer lied already within the sequences provided.] **(9 marks)** [CSE 203 | 2020–21]

Answer

Search target: 22, keys in range $[1, 100]$.

Validity Rule: A BST search path for value X maintains implicit bounds (ℓ, u) on every visited node. Initially $\ell = -\infty$, $u = +\infty$.

- Going **left** from node v (because $X < v$): $u \leftarrow v$ (all future nodes must be $< v$).
- Going **right** from node v (because $X > v$): $\ell \leftarrow v$ (all future nodes must be $> v$).
- Each node visited must satisfy $\ell < \text{node} < u$.

Sequence 1: [100, 40, 20, 30, 23, 35, 22] — **INVALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	100	$(-\infty, +\infty)$	✓	$22 < 100$: go left, $u \leftarrow 100$
2	40	$(-\infty, 100)$	✓	$22 < 40$: go left, $u \leftarrow 40$
3	20	$(-\infty, 40)$	✓	$22 > 20$: go right, $\ell \leftarrow 20$
4	30	$(20, 40)$	✓	$22 < 30$: go left, $u \leftarrow 30$
5	23	$(20, 30)$	✓	$22 < 23$: go left, $u \leftarrow 23$
6	35	$(20, 23)$	✗	$35 \notin (20, 23)$ — VIOLATION

After visiting 23 going left, all future nodes must be in $(20, 23)$. But $35 > 23$: **impossible in any BST**.

Sequence 2: [100, 35, 15, 32, 20, 22] — **VALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	100	$(-\infty, +\infty)$	✓	$22 < 100$: go left, $u \leftarrow 100$
2	35	$(-\infty, 100)$	✓	$22 < 35$: go left, $u \leftarrow 35$
3	15	$(-\infty, 35)$	✓	$22 > 15$: go right, $\ell \leftarrow 15$
4	32	$(15, 35)$	✓	$22 < 32$: go left, $u \leftarrow 32$
5	20	$(15, 32)$	✓	$22 > 20$: go right, $\ell \leftarrow 20$
6	22	$(20, 32)$	✓	$22 = \text{target}$: FOUND

Every node satisfies its bounds. Path with branches: $100 \xrightarrow{L} 35 \xrightarrow{L} 15 \xrightarrow{R} 32 \xrightarrow{L} 20 \xrightarrow{R} 22$.

Sequence 3: [50, 60, 90, 40, 75] — **INVALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	50	$(-\infty, +\infty)$	✓	$22 < 50$: go left ... wait
<i>Actually: next node is 60 > 50, so we go RIGHT at 50.</i>				
1	50	$(-\infty, +\infty)$	✓	$60 > 50$: go right, $\ell \leftarrow 50$
2	60	$(50, +\infty)$	✓	$90 > 60$: go right, $\ell \leftarrow 60$
3	90	$(60, +\infty)$	✓	$40 < 90$: go left, $u \leftarrow 90$
4	40	$(60, 90)$	✗	$40 \notin (60, 90)$ — VIOLATION

After going right from 50 and 60, all future nodes must be > 60 . But $40 < 60$: impossible.

Summary: Only **Sequence 2** is a valid BST search path. Sequences 1 and 3 violate the BST ordering invariant.

Q15. In a Binary Search Tree (BST), the successor of a node N is defined as the node N_{suc} with the smallest key greater than the key of N . Similarly, the predecessor of N is defined as the node N_{pre} with the largest key smaller than the key of N . Now answer the following questions (i)–(iii):

- Write an efficient algorithm that will store the keys of the successor and predecessor of each node. Assume that you have two variables, **suc** and **pre**, respectively in the node class/data structure for that purpose. **(7 marks)**
- Discuss an efficient implementation strategy of **removeFullNode(N, N_{suc})** (as defined below) with the help of appropriate illustrative examples. Analyze the time complexity of your strategy.

removeFullNode(N, N_{suc})	N is a full node that has to be removed from the BST. N_{suc} is the successor of N .
--	---

(12 marks)

- Your genius little brother suggests that an equivalent strategy is available if we use N_{pre} instead of N_{suc} . Do you agree with him? Justify your answer using the same illustrative example you showed in part 2.b(ii). **(7 marks)**

[CSE 203 | 2020–21]

Answer

(i) Algorithm to store successor and predecessor for all nodes:

Use a single **in-order traversal**. In-order gives nodes in sorted order; each node's predecessor is the previous node visited and successor is the next.

Algorithm 62 STORESUCCPRED(*node*)

```

1: global prev_node ▷ initially nil
2: if node = nil then return
3: end if
4: STORESUCCPRED(node.left)
5: if prev_node ≠ nil then
6:   prev_node.suc ← node
7:   node.pre ← prev_node
8: end if
9: prev_node ← node
10: STORESUCCPRED(node.right)

```

Time: $O(n)$ (single traversal). **Space:** $O(h)$ for recursion stack.

(ii) removeFullNode(N, N_{suc}):

N is a full node (has two children). N_{suc} is its in-order successor (leftmost node in N 's right subtree, so N_{suc} has no left child).

Strategy:

1. Copy N_{suc} 's key into N (replace N 's data with N_{suc} 's data).
2. Delete N_{suc} from its original position (easy: N_{suc} has at most one child — its right child).
3. Update **suc/pre** pointers: $N_{pre}.suc = N_{suc}.suc$ and $N_{suc}.suc.pre = N$ (now carrying N_{suc} 's old key).

Example: Delete node 5 from BST with nodes 3, 5, 7, 6, 9. N_{suc} of 5 is 6 (leftmost of right subtree).

- Copy 6 into node 5's position.
- Delete the original node 6 (no left child — simply replace with its right child if any).

Time complexity: $O(h)$ where h is the height of the BST ($O(\log n)$ for balanced, $O(n)$ worst case for skewed tree).

(iii) Using N_{pre} instead of N_{suc} :

Yes, it is equivalent. N_{pre} is the in-order predecessor of N (rightmost node in N 's left subtree), which has no right child.

Strategy: Copy N_{pre} 's key into N , then delete N_{pre} from its original position (at most one child — its left child).

Using the same example: delete node 5. $N_{pre} = 3$ (rightmost of left subtree). Copy 3 into node 5's position. Delete original node 3 (no right child).

Both strategies produce a valid BST and have identical $O(h)$ time complexity. The choice between N_{suc} and N_{pre} is a matter of implementation preference; both maintain the BST property.

Q16. Assume that a binary heap is stored in an array called **treeNodes**, which has a capacity of 100 array elements, with array index 0 not being used. If the binary heap currently contains 86 keys, answer the following questions with proper explanation:

- (i) Is `treeNodes[44]` a leaf node?
- (ii) How many children does `treeNodes[43]` have?
- (iii) Is the subtree rooted at `treeNodes[8]` a full binary tree?
- (iv) How many levels are there in the subtree rooted at `treeNodes[8]` including `treeNodes[8]`?
- (v) Including the root node (`treeNodes[1]`), how many levels are there in the binary heap?
- (vi) How many leaf nodes are there?

(12 marks)

[CSE 203 | 2020–21]

Answer

Heap with $n = 86$ elements, 1-indexed. First leaf = $\lfloor 86/2 \rfloor + 1 = 44$.

- (i) **`treeNodes[44]` a leaf?** $44 \geq 44 \rightarrow$ **Yes**. Children at 88, 89 > 86 .
- (ii) **Children of `treeNodes[43]`?** Left=86 (valid), right=87 > 86 . $\rightarrow$ **One child**.
- (iii) **Subtree at `treeNodes[8]` a full binary tree?** Node 43 (one child) lies in this subtree $\rightarrow$ **No**.
- (iv) **Levels in subtree at `treeNodes[8]`?** $\lfloor \log_2(86/8) \rfloor + 1 = \lfloor \log_2 10.75 \rfloor + 1 = 3 + 1 = 4$.
- (v) **Total levels in heap?** $\lfloor \log_2 86 \rfloor + 1 = 6 + 1 = 7$.
- (vi) **Number of leaves?** $86 - 43 = 43$ leaves.

Q17. Show the steps of building a max-heap by inserting the numbers 45, 36, 27, 14, 86, 19, 69, 10 one by one. Show the memory representation of the heap. Then write down the steps of sorting these numbers using heap sort. Show the analysis of how **Build-Max-Heap** can be done in $\Theta(n)$ time. (10+5+10+10 marks) [CSE 203 | 2019–20]

Answer

Step-by-step insertion:

Insert	Array
45	[45]
36	[45, 36]
27	[45, 36, 27]
14	[45, 36, 27, 14]
86	[86, 45, 27, 14, 36]
19	[86, 45, 27, 14, 36, 19]
69	[86, 45, 69, 14, 36, 19, 27]
10	[86, 45, 69, 14, 36, 19, 27, 10]

Memory representation:

Index	1	2	3	4	5	6	7	8
Value	86	45	69	14	36	19	27	10

Heap Sort (extract-max, then heapify):

Step	Heap Sorted
1	[69,45,27,14,36,19,10] {86}
2	[45,36,27,14,10,19] {69,86}
3	[36,19,27,14,10] {45,69,86}
4	[27,19,10,14] {36,45,69,86}
5	[19,14,10] {27,36,45,69,86}
6	[14,10] {19,27,36,45,69,86}
7	[10] {14,19,27,36,45,69,86}

Sorted: 10, 14, 19, 27, 36, 45, 69, 86 ✓

Build-Max-Heap in $\Theta(n)$: (see proof in [2014–15] answer).

- Q18.** Assume you are given a Max Priority Queue of n elements. Write a function that first finds and then deletes the minimum element of the priority queue. **(10 marks)**
[CSE 203 | 2018–19]

Answer

Using the standard Priority Queue ADT (e.g., C++ `std::priority_queue`), we cannot directly access the leaves. We must extract all elements, track the minimum, and re-insert everything except the minimum element.

```

1  #include <queue>
2  #include <vector>
3  using namespace std;
4
5  void deleteMin(priority_queue<int>& pq) {
6      if (pq.empty()) return;
7
8      vector<int> temp;
9      int min_val = pq.top();
10
11     // Step 1: Extract all elements to find the minimum
12     while (!pq.empty()) {
13         int current = pq.top();
14         pq.pop();
15         if (current < min_val) {
16             min_val = current;
17         }
18         temp.push_back(current);
19     }
20 }
```

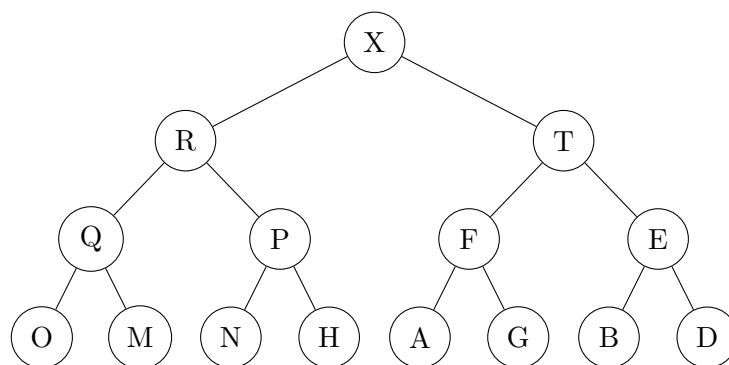
```

21 // Step 2: Re-insert all elements except one instance of
    the minimum
22 bool is_deleted = false;
23 for (int val : temp) {
24     if (val == min_val && !is_deleted) {
25         is_deleted = true; // Skip pushing this element
26         continue;
27     }
28     pq.push(val);
29 }
30 }

```

Complexity Analysis. Extracting all elements: $O(n \log n)$. Re-inserting $n - 1$ elements: $O(n \log n)$. Extra Space: $O(n)$ for the temporary vector. **Total Time:** $O(n \log n)$.

Q19. Consider the following max-heap of letters:



Draw the results of inserting S into the above max-heap. (7 marks) [CSE 203 | 2018–19]

Detailed Answer

Original Heap Array (0-indexed):

Total 15 nodes. Indices 0 to 14.

[X, R, T, Q, P, F, E, O, M, N, H, A, G, B, D]

Step 1: Insert S

The new node 'S' must be inserted at the next available position, which is index 15. The parent of index 15 is $\lfloor (15 - 1)/2 \rfloor = 7$. The node at index 7 is 'O'. So, 'S' is initially placed as the **left child of O**.

Step 2: Bubble-up (Heapify-up) trace

Letter values for comparison: S=19, O=15, Q=17, R=18, X=24.

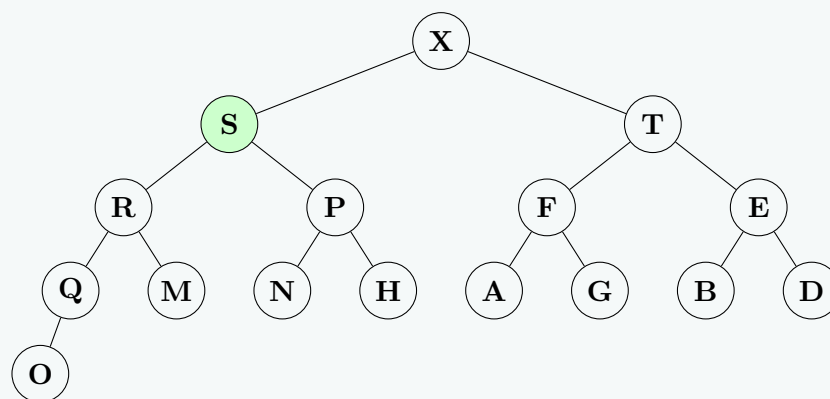
- (a) **S(19) vs Parent O(15) at idx 7:** $19 > 15 \rightarrow$ Swap! 'S' moves to idx 7, 'O' moves to idx 15.

- (b) **S(19) vs Parent Q(17) at idx 3:** $19 > 17 \rightarrow$ Swap! 'S' moves to idx 3, 'Q' moves to idx 7.
- (c) **S(19) vs Parent R(18) at idx 1:** $19 > 18 \rightarrow$ Swap! 'S' moves to idx 1, 'R' moves to idx 3.
- (d) **S(19) vs Parent X(24) at idx 0:** $19 < 24 \rightarrow$ **Stop.** Max-heap property restored.

Final Heap Array:

[X, S, T, R, P, F, E, Q, M, N, H, A, G, B, D, O]

Final Max-Heap Tree:



Remark. In the final tree, 'S' successfully bubbled up to replace 'R', pushing 'R' down to replace 'Q', 'Q' down to replace 'O', and 'O' becoming the left child of 'Q'.

Q20. Suppose you have a set of numbers where you do a constant number of insertions and a linear number of lookups of the maximum over some time period. Compare using a heap vs. a binary search tree for maintaining this set:

- What is the asymptotic upper bound on the *average-case* runtime for each, and which structure should you choose?
- What is the asymptotic upper bound on the *worst-case* runtime for each, and which structure should you choose?

(12 marks)

[CSE 203 | 2018–19]

Answer

Operation	Max-Heap		BST	
	Avg	Worst	Avg	Worst
c Insertions	$O(c \log n)$	$O(c \log n)$	$O(c \log n)$	$O(cn)$
n Find-Max	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
Total	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$

- (a) **Average case:** Heap $O(n)$ vs BST $O(n \log n)$ → **choose heap.**
 (b) **Worst case:** Heap $O(n)$ vs BST $O(n^2)$ → **choose heap.**

Q21. Write a function `int treeHeight()` for a Binary Search Tree that computes the height of the tree. Give a worst-case Big- O running time in terms of n . **(8 marks)**
 [CSE 203 | 2018–19]

Answer

```
1 // Height: O(n)
2 int treeHeight(treeNode* root){
3     if(!root) return -1;
4     return 1 + max(treeHeight(root->left), treeHeight(root->
5         right));
6 }
```

Q22. Write a function `void deleteMax()` for a Binary Search Tree that deletes the maximum element (or does nothing if the tree is empty). Give a worst-case Big- O running time in terms of n . Do not assume the tree is balanced. **(7 marks)** [CSE 203 | 2018–19]

Answer

```
1 // Delete max (rightmost node): O(h) worst O(n)
2 void deleteMax(treeNode* &root){
3     if(!root) return;
4     if(!root->right){
5         treeNode* tmp = root;
6         root = root->left;
7         delete tmp; return;
8     }
9     treeNode* parent = root, *curr = root->right;
10    while(curr->right){ parent = curr; curr = curr->right; }
11    parent->right = curr->left;
12    delete curr;
13 }
```

Q23. Write a method `isHeap()` that determines whether a particular binary tree is a heap. What is the Big- O bound on the running time of this algorithm? **(8 marks)**
 [CSE 203 | 2018–19]

Answer

```
1 // isHeap (array, 1-indexed): O(n)
2 bool isHeap(int A[], int n){
3     for(int i = 1; i <= n/2; i++){
4         if(2*i <= n && A[i] < A[2*i]) return false;
5     }
```

```

5         if(2*i+1 <= n && A[i] < A[2*i+1]) return false;
6     }
7     return true;
8 }

```

- Q24.** Construct a max-heap with the data [6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5] by insertion and adjustment. Show the changes in values at each step. Carry out complexity analysis for both algorithms. **(5+5+10 marks)** [CSE 203 | 2017–18]

Answer

Build by successive insertion + bubble-up:

Insert	Array after bubble-up
6	[6]
1	[6, 1]
7	[7, 1, 6]
2	[7, 2, 6, 1]
8	[8, 7, 6, 1, 2]
3	[8, 7, 6, 1, 2, 3]
9	[9, 7, 8, 1, 2, 3, 6]
4	[9, 7, 8, 4, 2, 3, 6, 1]
10	[10, 9, 8, 7, 2, 3, 6, 1, 4]
5	[10, 9, 8, 7, 5, 3, 6, 1, 4, 2]
11	[11, 10, 8, 7, 9, 3, 6, 1, 4, 2, 5]
17	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3]
4	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3, 4]
5	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3, 4, 5]

Complexity Analysis. Insertion method: $\Theta(n \log n)$. Bottom-up build: $\Theta(n)$.

- Q25.** By writing a procedure, explain the three cases that may arise for deleting a node from a binary search tree. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Answer

Case 1 — leaf: remove; parent pointer set to NULL.

Case 2 — one child: splice; parent points directly to child.

Case 3 — two children: find in-order successor s (leftmost in right subtree); copy s .key into node; delete s (which has ≤ 1 child, so Case 1 or 2).

```

1  treeNode* deleteNode(treeNode* root, int key){
2      if(!root) return NULL;
3      if(key < root->item)
4          root->left = deleteNode(root->left, key);
5      else if(key > root->item)

```

```

6         root->right = deleteNode(root->right, key);
7     else {
8         if(!root->left){ treeNode* t=root->right; delete root;
9             return t; }
10        if(!root->right){ treeNode* t=root->left; delete root;
11            return t; }
12        treeNode* s = minimum(root->right);
13        root->item = s->item;
14        root->right = deleteNode(root->right, s->item);
15    }
16    return root;
17 }

```

Complexity Analysis. $O(h)$; average $O(\log n)$, worst $O(n)$.

Q26. Write a function to find the smallest element in a max binary heap. (8 marks) [CSE 203 | 2016–17]

Answer

In a max binary heap, the smallest element is always a *leaf node*. Using `std::priority_queue` (max-heap by default), we cannot access internal elements directly. Instead, we copy all elements into a `min_heap` and return the top, or simply scan all leaves.

Using STL `priority_queue` (min-heap trick):

```

1  #include <queue>
2  #include <vector>
3  using namespace std;
4
5  int findMin(priority_queue<int> maxHeap) {
6      // Copy into a min-heap to find the smallest
7      priority_queue<int, vector<int>, greater<int>> minHeap;
8      while (!maxHeap.empty()) {
9          minHeap.push(maxHeap.top());
10         maxHeap.pop();
11     }
12     return minHeap.top(); // smallest element
13 }

```

Efficient approach — scan leaves only (underlying container):

```

1  #include <queue>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  // Expose underlying container via inheritance trick
7  struct MinFromHeap : priority_queue<int> {

```

```

8     int findMin() const {
9         int n = c.size();
10        // Leaves are at indices n/2 to n-1 (0-indexed)
11        return *min_element(c.begin() + n/2, c.end());
12    }
13 };

```

Complexity Analysis. $O(n)$ for both approaches — the first rebuilds the entire heap; the second scans only the $\lceil n/2 \rceil$ leaf nodes.

Q27. Write the differences between a standard queue and a priority queue. Write two functions that implement the DECREASE-KEY and INCREASE-KEY operations in a max-heap. (10 marks) [CSE 203 | 2016–17]

Answer

Queue vs. Priority Queue: (see table in [2012–13] answer).

Algorithm 63 MAX-HEAP-INCREASE-KEY(A, x, k)

```

1: if  $k < x.key$  then
2:     error “new key is smaller than current key”
3: end if
4:  $x.key \leftarrow k$ 
5: Find the index  $i$  in array  $A$  where object  $x$  occurs
6: while  $i > 1$  and  $A[\text{PARENT}(i)].key < A[i].key$  do
7:     Exchange  $A[i]$  with  $A[\text{PARENT}(i)]$ , updating the information that maps pri-
        ority queue objects to array indices
8:      $i \leftarrow \text{PARENT}(i)$ 
9: end while

```

Algorithm 64 MAX-HEAP-INSERT(A, x, n)

```

1: if  $A.heap\text{-}size == n$  then
2:     error “heap overflow”
3: end if
4:  $A.heap\text{-}size \leftarrow A.heap\text{-}size + 1$ 
5:  $k \leftarrow x.key$ 
6:  $x.key \leftarrow -\infty$ 
7:  $A[A.heap\text{-}size] \leftarrow x$ 
8: Map  $x$  to index  $heap\text{-}size$  in the array
9: MAX-HEAP-INCREASE-KEY( $A, x, k$ )

```

Complexity Analysis. Both $O(\log n)$.

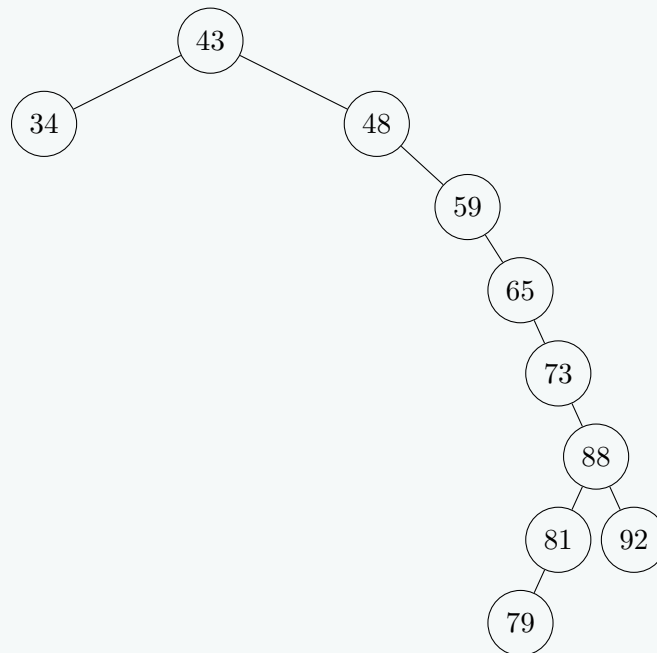
- Q28.** Write an algorithm that sorts the keys of a binary search tree. Draw the BST that results after inserting the keys 43, 34, 48, 59, 65, 73, 88, 81, 92, 79 (in that order) into an initially empty BST. Also draw a BST of height three using these keys. (10 marks) [CSE 203 | 2016–17]

Answer

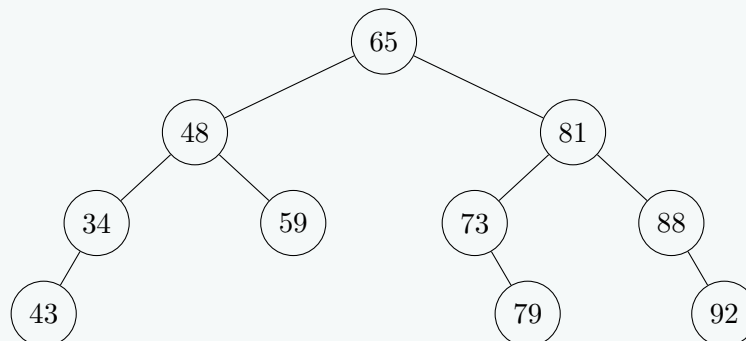
In-order traversal visits nodes in ascending order:

```
1 void inorder(treeNode* root){
2     if(!root) return;
3     inorder(root->left);
4     cout << root->item << " ";
5     inorder(root->right);
6 }
```

BST for 43,34,48,59,65,73,88,81,92,79 (height = 7):



BST of height 3 (insert order: 65, 48, 81, 34, 59, 73, 88, 43, 79, 92):



- Q29.** Write a function to find the largest element in a min binary heap. Assume the array representing a min binary heap contains the values 2, 8, 3, 10, 16, 7, 18, 13, 15. Show

the contents of the array after deleting the minimum element. (10 marks) [CSE 203 | 2015–16]

Answer

Part 1 — Largest element in a min-heap (STL).

The largest element is always a *leaf node*. Using STL's `priority_queue` (min-heap), the underlying container can be exposed via inheritance to scan only the leaf nodes.

```

1  #include <queue>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  struct MaxFromMinHeap : priority_queue<int, vector<int>,
7      greater<int>> {
8      int findMax() const {
9          int n = c.size();
10         // Leaves occupy indices n/2 .. n-1 (0-indexed)
11         return *max_element(c.begin() + n/2, c.end());
12     }
13 };
14
15 int main() {
16     MaxFromMinHeap h;
17     for (int x : {2,8,3,10,16,7,18,13,15}) h.push(x);
18     cout << "Max = " << h.findMax() << endl; // 18
19 }
```

Complexity Analysis. $O(n)$ — scans only the $\lceil n/2 \rceil$ leaf nodes.

Part 2 — Delete minimum from [2,8,3,10,16,7,18,13,15].

```

1  #include <queue>
2  #include <vector>
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      // Build min-heap from given values
8      priority_queue<int, vector<int>, greater<int>> minHeap;
9      for (int x : {2,8,3,10,16,7,18,13,15})
10         minHeap.push(x);
11
12     cout << "Min (root) = " << minHeap.top() << endl; // 2
13     minHeap.pop(); // delete minimum element
14
15     // Print remaining elements (heap order)
16     // Result: [3, 8, 7, 10, 16, 15, 18, 13]
17     priority_queue<int, vector<int>, greater<int>> temp =
18         minHeap;
```

```

18     while (!temp.empty()) {
19         cout << temp.top() << " ";
20         temp.pop();
21     }
22 }

```

Trace of pop() internally:

- (a) Root 2 removed; last element 15 placed at root.
Array: [15, 8, 3, 10, 16, 7, 18, 13]
- (b) Sift-down at index 0: min child = 3 (idx 2) → swap.
Array: [3, 8, 15, 10, 16, 7, 18, 13]
- (c) Sift-down at index 2: min child = 7 (idx 5) → swap.
Array: [3, 8, 7, 10, 16, 15, 18, 13]
- (d) Index 5 has no children. Done.

Final array after delete-min: [3, 8, 7, 10, 16, 15, 18, 13] ✓

Complexity Analysis. $O(\log n)$ — pop() sifts down at most $\lfloor \log_2 n \rfloor$ levels.

Q30. Prove that a min binary heap can be built in linear time. **(10 marks)** [CSE 203 | 2015–16]

Answer

At height h there are $\leq \lceil n/2^{h+1} \rceil$ nodes, each costing $O(h)$:

$$T(n) \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = cn \cdot 2 = O(n). \quad \square$$

(Using $\sum_{h=0}^{\infty} hx^h = x/(1-x)^2$ with $x = 1/2$.)

Q31. Write two procedures for finding the successor element and the predecessor element in a binary search tree. **(10 marks)** [CSE 203 | 2015–16]

Answer

```

1  treeNode* minimum(treeNode* n){
2      while(n->left) n = n->left; return n;
3  }
4  treeNode* maximum(treeNode* n){
5      while(n->right) n = n->right; return n;
6  }
7  treeNode* successor(treeNode* n){
8      if(n->right) return minimum(n->right);
9      treeNode* p = n->parent;

```

```

10     while(p && n == p->right){ n = p; p = p->parent; }
11     return p;
12 }
13 treeNode* predecessor(treeNode* n){
14     if(n->left) return maximum(n->left);
15     treeNode* p = n->parent;
16     while(p && n == p->left){ n = p; p = p->parent; }
17     return p;
18 }

```

Complexity Analysis. $O(h)$ each.

Q32. What are the main operations of a priority queue? Explain how one can implement these operations using a heap. **(15 marks)** [CSE 203 | 2014–15, 2011–12]

Answer

Main operations of a Priority Queue:

- INSERT(S, x): insert element x into set S .
- MAXIMUM(S): return element with the largest key ($O(1)$).
- EXTRACT-MAX(S): remove and return element with the largest key.
- INCREASE-KEY(S, x, k): increase value of element x 's key to $k \geq x.\text{key}$.

Heap-based implementation (Max-Heap, 1-indexed array A):

Heap property: $A[\lfloor i/2 \rfloor] \geq A[i]$ for all $i > 1$.

```

1 //  $O(\log n)$ : restore heap property downward
2 void maxHeapify(int A[], int i, int n) {
3     int l = 2*i, r = 2*i+1, largest = i;
4     if (l <= n && A[l] > A[largest]) largest = l;
5     if (r <= n && A[r] > A[largest]) largest = r;
6     if (largest != i) {
7         swap(A[i], A[largest]);
8         maxHeapify(A, largest, n);
9     }
10 }
11 int heapMaximum(int A[]) { return A[1]; } //  $O(1)$ 
12
13 int heapExtractMax(int A[], int &n) { //  $O(\log n)$ 
14     )
15     int max = A[1];
16     A[1] = A[n--];
17     maxHeapify(A, 1, n);
18     return max;
19 }

```

```

19 void heapIncreaseKey(int A[], int i, int key) {           // O(log n)
20     A[i] = key;
21     while (i > 1 && A[i/2] < A[i]) {
22         swap(A[i], A[i/2]);
23         i /= 2;
24     }
25 }
26 void maxHeapInsert(int A[], int &n, int key) {           // O(log n)
27     A[++n] = INT_MIN;
28     heapIncreaseKey(A, n, key);
29 }

```

Complexity Analysis. MAXIMUM: $O(1)$. EXTRACT-MAX, INSERT, INCREASE-KEY: $O(\log n)$.

Q33. Write an algorithm for building a heap. Show that an n -element heap can be built in $O(n)$ time. **(10 marks)** [CSE 203 | 2014–15]

Answer

BUILD-MAX-HEAP (same algorithm as above).

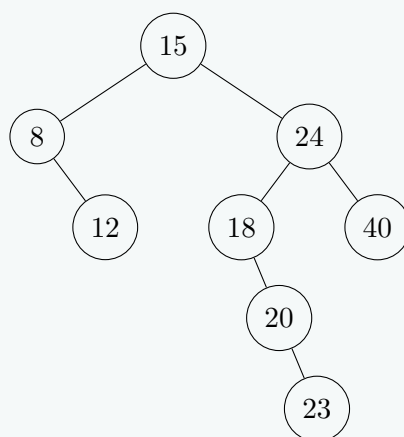
$O(n)$ **proof:** a node at height h costs $O(h)$ in HEAPIFY; there are $\leq \lceil n/2^{h+1} \rceil$ such nodes.

$$T(n) \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = cn \cdot \frac{1/2}{(1 - 1/2)^2} = 2cn = O(n). \quad \square$$

Q34. Draw the binary search tree that results after successively inserting the keys 15, 8, 24, 12, 40, 18, 20, 23 into an initially empty BST. Then, by using necessary rotations, redraw the BST so that it becomes an AVL tree. **(10 marks)** [CSE 203 | 2014–15]

Answer

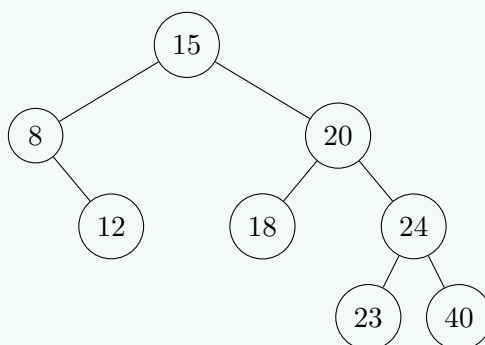
BST after inserting 15, 8, 24, 12, 40, 18, 20, 23:

**Converting to AVL — detecting imbalance:**

Node 24: $BF = \text{height}(18 \text{ subtree}) - \text{height}(40) = 2 - 0 = 2 \rightarrow$ unbalanced (Right-Left case, since $20 > 18$ but $20 < 24$).

Fix: Left-rotate at 18, then Right-rotate at 24. After left-rotate at 18: 20 takes 18's position; 18 becomes 20's left child; 23 becomes 20's right (wait — 23 is right of 20, after rotation 23 stays right of 20 and 24 becomes root of right part).

After full LR rotation at 24: 20 becomes subroot; 18 is 20's left; 24 is 20's right with 23 as 24's left and 40 as 24's right.

Final AVL tree:

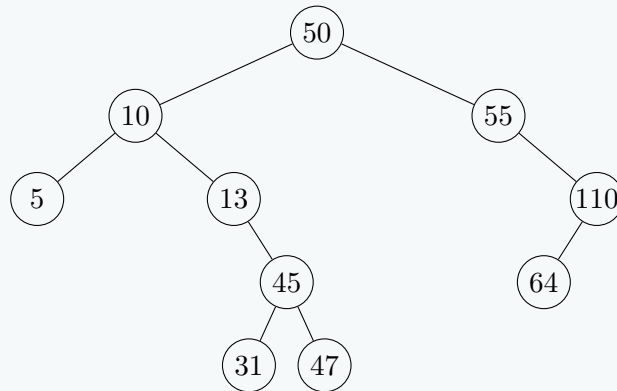
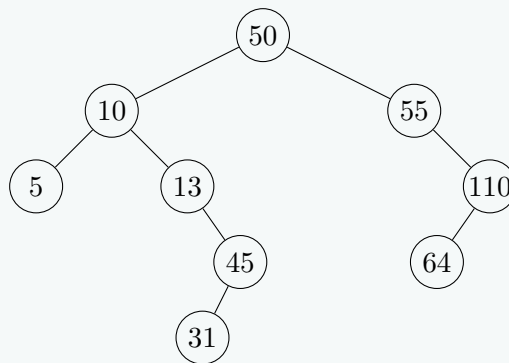
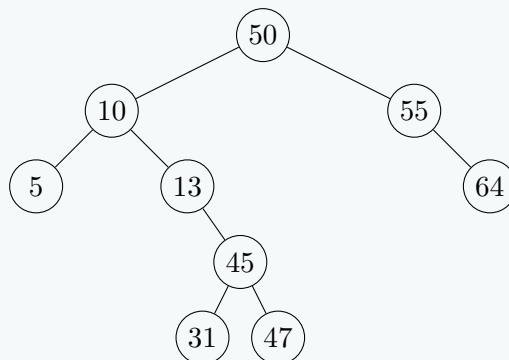
Remark. All balance factors $\in \{-1, 0, +1\}$. ✓

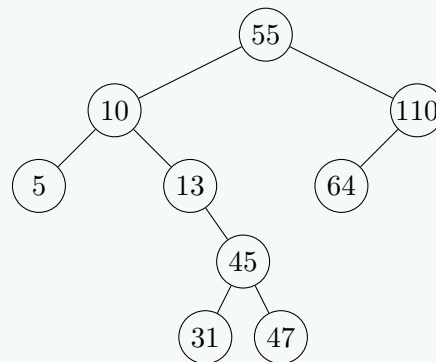
Q35. Suppose we create a binary search tree by inserting the following values in the given order: 50, 10, 13, 45, 55, 110, 5, 31, 64, 47. Answer the following questions:

- Draw the binary search tree.
- Show the output values if we visit the tree using pre-order traversal.
- Show the output values if we visit the tree using post-order traversal.
- Show the resulting trees after deleting 47, 110, and 50 (each deletion applied on the original tree).

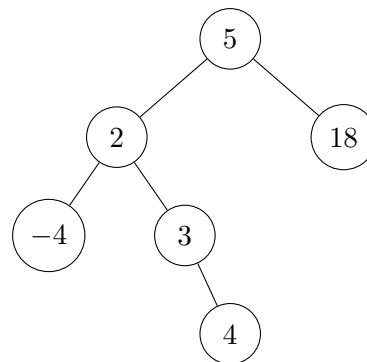
(5+4+4+6=19 marks)

[CSE 203 | 2014–15]

Answer**(a) Original BST:****(b) Preorder:** 50 10 5 13 45 31 47 55 110 64 ✓**(c) Postorder:** 5 31 47 45 13 10 64 110 55 50 ✓**(d) Deletions (each applied independently on the original tree):**1. *Delete 47* (Case 1: Leaf node). Simply remove node 47.2. *Delete 110* (Case 2: Node with one child). Replace 110 with its left child 64.3. *Delete 50* (Case 3: Node with two children). Replace 50 with its inorder successor (which is 55). Then delete the original node 55. The right child of the new root 55 becomes 110.



Q36. Give the idea of a data structure that can be used to implement a general tree where nodes can have more than two children. Re-draw the following binary search tree using your proposed data structure:



(8 marks)

[CSE 203 | 2014–15]

Answer

Left-Child Right-Sibling (LCRS) Representation:

The optimal data structure to represent a general tree (where nodes can have more than two children) using a binary structure is the **Left-Child Right-Sibling (LCRS)** tree.

Each node stores exactly two pointers:

- **leftChild** — points to the node's **first (leftmost) child**.
- **rightSibling** — points to the node's **immediate next sibling** to its right.

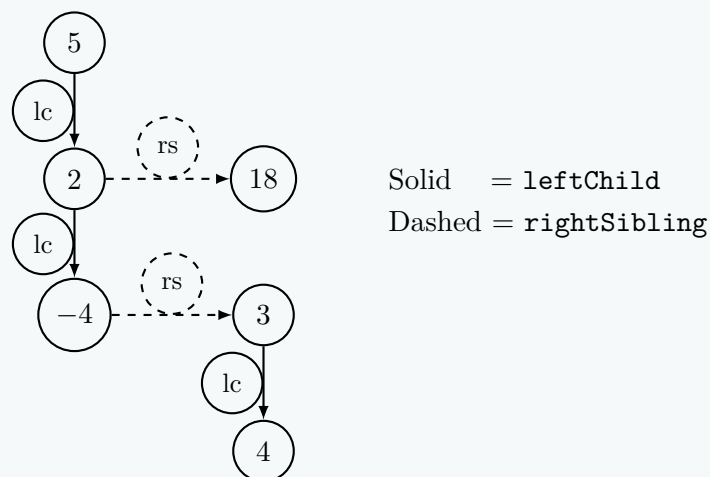
This effectively transforms any general tree into a binary tree representation, as each node has at most two outgoing pointers.

Re-drawn BST using LCRS representation:

Treating the given binary search tree as a general tree, we apply the LCRS rules:

- **Node 5:** First child is 2 (**leftChild** = 2). No siblings (**rightSibling** = NULL).
- **Node 2:** First child is -4 (**leftChild** = -4). Next sibling is 18 (**rightSibling** = 18).
- **Node 18:** No children (**leftChild** = NULL). No next sibling (**rightSibling** = NULL).

- **Node -4:** No children (`leftChild = NULL`). Next sibling is 3 (`rightSibling = 3`).
- **Node 3:** First (and only) child is 4 (`leftChild = 4`). No next sibling (`rightSibling = NULL`).
- **Node 4:** No children (`leftChild = NULL`). No next sibling (`rightSibling = NULL`).



Q37. Assume an object which has two elements {key, element}. An array of such objects is given. Then:

- Convert the array into a Max-heap.
- Insert (36, *B*) in the resultant Max-heap.
- Show each step of Heap Sort over the resultant Max-heap.

Array: [14/*U*, 11/*E*, 13/*T*, 12/*C*, 26/*S*, 19/*E*, 20/*W*, 24/*P*, 18/*O*, 17/*L*, 7/*A*, 8/*S*, 9/*H*, 10/*I*] (8+8+6=22 marks) [CSE 203 | 2013–14]

Detailed Answer

Given Array: [14/*U*, 11/*E*, 13/*T*, 12/*C*, 26/*S*, 19/*E*, 20/*W*, 24/*P*, 18/*O*, 17/*L*, 7/*A*, 8/*S*, 9/*H*, 10/*I*]

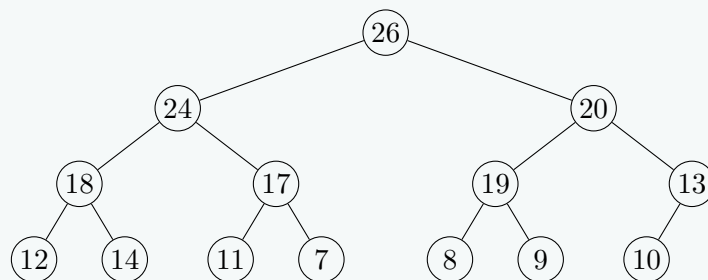
Total elements, $n = 14$. For clarity in diagrams and steps, only the **keys** are shown.

(a) Convert the array into a Max-Heap

We use the bottom-up heapify method. We start from the last non-leaf node at index $\lfloor n/2 \rfloor - 1 = \lfloor 14/2 \rfloor - 1 = 6$.

Node Idx	Heapify Action	Array State (Keys)
idx 6 (20)	$20 \geq 10$ (child). No swap.	[14, 11, 13, 12, 26, 19, 20, 24, 18, 17, 7, 8, 9, 10]
idx 5 (19)	$19 \geq 8, 9$ (children). No swap.	unchanged
idx 4 (26)	$26 \geq 17, 7$ (children). No swap.	unchanged
idx 3 (12)	$12 < 24$. Swap $12 \leftrightarrow 24$.	[14, 11, 13, 24 , 26, 19, 20, 12 , 18, 17, 7, 8, 9, 10]
idx 2 (13)	$13 < 20$. Swap $13 \leftrightarrow 20$.	[14, 11, 20 , 24, 26, 19, 13 , 12, 18, 17, 7, 8, 9, 10]
idx 1 (11)	$11 < 26$. Swap $11 \leftrightarrow 26$.	[14, 26 , 20, 24, 11 , 19, 13, 12, 18, 17, 7, 8, 9, 10]
	$\hookrightarrow$ Heapify idx 4: Swap $11 \leftrightarrow 17$.	[14, 26, 20, 24, 17 , 19, 13, 12, 18, 11 , 7, 8, 9, 10]
idx 0 (14)	$14 < 26$. Swap $14 \leftrightarrow 26$.	[26 , 14 , 20, 24, 17, 19, 13, 12, 18, 11, 7, 8, 9, 10]
	$\hookrightarrow$ Heapify idx 1: Swap $14 \leftrightarrow 24$.	[26, 24 , 20, 14 , 17, 19, 13, 12, 18, 11, 7, 8, 9, 10]
	$\hookrightarrow$ Heapify idx 3: Swap $14 \leftrightarrow 18$.	[26, 24, 20, 18 , 17, 19, 13, 12, 14 , 11, 7, 8, 9, 10]

Max-Heap Tree after Step (a):



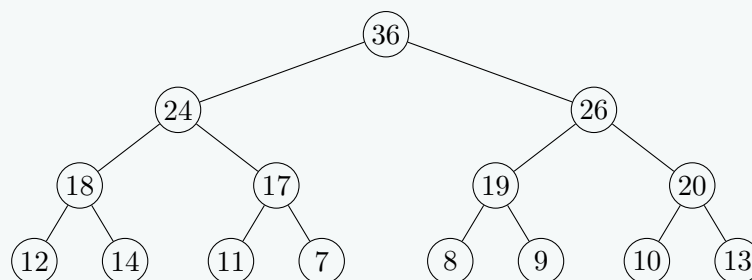
(b) Insert (36, B) in the resultant Max-Heap

We insert 36 at the very end of the array (index 14) and "bubble-up" to restore the Max-Heap property.

1. Insert at idx 14. Array ends with $[\dots, 10, \mathbf{36}]$.
2. Parent is idx 6 (key 13). $36 > 13 \rightarrow$ **Swap 36 and 13**.
3. Parent is idx 2 (key 20). $36 > 20 \rightarrow$ **Swap 36 and 20**.
4. Parent is idx 0 (key 26). $36 > 26 \rightarrow$ **Swap 36 and 26**.

Array after insertion: [36, 24, 26, 18, 17, 19, 20, 12, 14, 11, 7, 8, 9, 10, 13]

Max-Heap Tree after Step (b):



(c) Heap Sort over the resultant Max-Heap

Heap sort extracts the maximum element (root), swaps it with the last element of the heap, reduces the heap size by 1, and heapifies the root. The extracted elements build the sorted array at the end.

- **Initial Heap:** [36, 24, 26, 18, 17, 19, 20, 12, 14, 11, 7, 8, 9, 10, 13]
- **Extract 36:** Swap 36 & 13. Heapify root (13 → swaps with 26, 20, 10).
Heap: [26, 24, 20, 18, 17, 19, 10, 12, 14, 11, 7, 8, 9, 13] | **36**
- **Extract 26:** Swap 26 & 13. Heapify root (13 → swaps with 24, 18, 14).
Heap: [24, 18, 20, 14, 17, 19, 10, 12, 13, 11, 7, 8, 9] | **26, 36**
- **Extract 24:** Swap 24 & 9. Heapify root (9 → swaps with 20, 19).
Heap: [20, 18, 19, 14, 17, 9, 10, 12, 13, 11, 7, 8] | **24, 26, 36**
- **Extract 20:** Swap 20 & 8. Heapify root (8 → swaps with 19, 10).
Heap: [19, 18, 10, 14, 17, 8, 9, 12, 13, 11, 7] | **20 ... 36**
- **Extract 19:** Swap 19 & 7. Heapify root (7 → swaps with 18, 17, 11).
Heap: [18, 17, 10, 14, 11, 8, 9, 12, 13, 7] | **19 ... 36**
- **Extract 18:** Swap 18 & 7. Heapify root (7 → swaps with 17, 14, 13).
Heap: [17, 14, 10, 13, 11, 8, 9, 12, 7] | **18 ... 36**
- **Extract 17:** Swap 17 & 7. Heapify root (7 → swaps with 14, 13, 12).
Heap: [14, 13, 10, 12, 11, 8, 9, 7] | **17 ... 36**
- **Extract 14:** Swap 14 & 7. Heapify root (7 → swaps with 13, 12).
Heap: [13, 12, 10, 7, 11, 8, 9] | **14 ... 36**
- **Extract 13:** Swap 13 & 9. Heapify root (9 → swaps with 12, 11).
Heap: [12, 11, 10, 7, 9, 8] | **13 ... 36**
- **Extract 12:** Swap 12 & 8. Heapify root (8 → swaps with 11, 9).
Heap: [11, 9, 10, 7, 8] | **12 ... 36**
- **Extract 11:** Swap 11 & 8. Heapify root (8 → swaps with 10).
Heap: [10, 9, 8, 7] | **11 ... 36**
- **Extract 10:** Swap 10 & 7. Heapify root (7 → swaps with 9).
Heap: [9, 7, 8] | **10 ... 36**
- **Extract 9:** Swap 9 & 8. Heapify root (8 → no swap).
Heap: [8, 7] | **9 ... 36**
- **Extract 8:** Swap 8 & 7.
Heap: [7] | **8 ... 36**

Final Sorted Array (Ascending): [7, 8, 9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 24, 26, 36]

Q38. Write down two algorithms for constructing a max-heap and deduce their complex-

ities. Discuss how Carlsson reduces the leading coefficient in the complexity of the Heapsort algorithm. (10 marks) [CSE 207 | 2013–14]

Answer

Algorithm 1: Repeated Insertion (top-down)

Insert elements one by one into an initially empty heap using HEAPIFY-UP.

Algorithm 65 BUILD-HEAP-INSERTION(A, n)

```

1:  $heap \leftarrow []$ 
2: for  $i = 1$  to  $n$  do
3:    $heap.APPEND(A[i])$ 
4:    $HEAPIFY-UP(heap, |heap|)$ 
5: end for
```

Complexity: Each HEAPIFYUP takes $O(\log k)$ for the k -th insertion. Total: $\sum_{k=1}^n O(\log k) = O(n \log n)$.

Algorithm 2: Build-Max-Heap (bottom-up, CLRS)

Start from the last non-leaf and call MAX-HEAPIFY downward.

Algorithm 66 BUILD-MAX-HEAP(A, n)

```

1: for  $i = \lfloor n/2 \rfloor$  downto 1 do
2:    $MAX-HEAPIFY(A, i, n)$ 
3: end for
```

Complexity: $O(n)$ — nodes at height h take $O(h)$ each, and $\sum_{h=0}^{\lceil \log n \rceil} \frac{n}{2^{h+1}} \cdot O(h) = O(n)$.

Carlsson's improvement to Heapsort:

Standard Heapsort uses 2 comparisons per MAX-HEAPIFY step (compare both children, then compare winner with parent). Carlsson (1987) observed that during the sifting-down phase, the element being sifted almost always goes to a leaf. His optimization:

- (a) **Sift-down to a leaf** using only 1 comparison per level (just pick the larger child, without comparing against the element being sifted).
- (b) **Then sift back up** to find the correct position.

This reduces the average number of comparisons from $\approx 2n \log n$ to $\approx n \log n + O(n)$, nearly halving the leading coefficient. The asymptotic complexity remains $O(n \log n)$ but the constant factor is smaller, making it faster in practice.

Q39. Write the differences between a standard queue and a priority queue. Write the property of a Max-Heap. Write an algorithm for building a Max-Heap. Analyze the time complexity of the algorithm. (3+3+7+7=20 marks) [CSE 203 | 2012–13]

Answer**Standard Queue vs. Priority Queue:**

Property	Standard Queue	Priority Queue
Order	FIFO	By priority/key
Dequeue	Front element	Highest-priority element
Implementation	Array/Linked list	Heap
Extract time	$O(1)$	$O(\log n)$

Max-Heap property: for every node $i \neq \text{root}$, $A[\lfloor i/2 \rfloor] \geq A[i]$. The root holds the maximum.

Build-Max-Heap:**Algorithm 67** BUILD-MAX-HEAP(A, n)

```

1: for  $i \leftarrow \lfloor n/2 \rfloor$  downto 1 do
2:   MAX-HEAPIFY( $A, i, n$ )
3: end for

```

Complexity Analysis. Naïve: $O(n \log n)$. Tight: $O(n)$, since $\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O(n \sum_{h=0}^{\infty} h/2^h) = O(2n) = O(n)$.

Q40. Sort the array $[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]$ using Heapsort, showing the values at the right of the nodes at each step. **(8 marks)** [CSE 203 | 2011–12]

Answer

Heapsort works in two phases: (1) build a max-heap from the array, then (2) repeatedly extract the maximum.

Phase 1 — Build max-heap (starting from last non-leaf, $i = \lfloor 14/2 \rfloor - 1 = 6$, down to 0):

After BUILDMAXHEAP:

[26, 13, 23, 10, 12, 9, 15, 4, 5, 6, 2, 1, 8, 7]

Phase 2 — Extract-max repeatedly (swap root with last, reduce heap size, heapify):

Extract	Max	Heap (remaining)	Sorted suffix
1	26	[23, 13, 15, 10, 12, 9, 7, 4, 5, 6, 2, 1, 8]	[26]
2	23	[15, 13, 9, 10, 12, 8, 7, 4, 5, 6, 2, 1]	[23, 26]
3	15	[13, 12, 9, 10, 6, 8, 7, 4, 5, 1, 2]	[15, 23, 26]
4	13	[12, 10, 9, 5, 6, 8, 7, 4, 2, 1]	[13, 15, 23, 26]
5	12	[10, 6, 9, 5, 1, 8, 7, 4, 2]	[12, 13, 15, 23, 26]
6	10	[9, 6, 8, 5, 1, 2, 7, 4]	[10, 12, 13, 15, 23, 26]
7	9	[8, 6, 7, 5, 1, 2, 4]	[9, 10, 12, 13, 15, 23, 26]
8	8	[7, 6, 4, 5, 1, 2]	[8, 9, 10, 12, 13, 15, 23, 26]
9	7	[6, 5, 4, 2, 1]	[7, 8, 9, 10, 12, 13, 15, 23, 26]
10	6	[5, 2, 4, 1]	[6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
11	5	[4, 2, 1]	[5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
12	4	[2, 1]	[4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
13	2	[1]	[2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
Sorted		[1, 2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]	

Complexity Analysis. BUILDMAXHEAP: $O(n)$. Each EXTRACTMAX + HEAPIFY: $O(\log n)$, done $n - 1$ times. Total: $\Theta(n \log n)$ worst, average, and best case.

Q41. What is a priority queue? Write three applications of a priority queue. **(2+3=5 marks)** [CSE 203 | 2011–12]

Answer

Definition.

A **priority queue** is an abstract data type (ADT) where each element is associated with a *priority key*. The element with the **highest** (or lowest) priority is always served first, regardless of the order in which elements were inserted. It supports:

- `insert(element, priority)`: add an element.
- `extractMax()` or `extractMin()`: remove and return the highest/lowest priority element.
- `peek()`: view the top element without removing.

Three Applications:

1. **CPU Scheduling (OS):** Processes are assigned priorities. The scheduler always runs the highest-priority ready process first (preemptive priority scheduling).
2. **Dijkstra's Shortest Path Algorithm:** A min-priority queue is used to always process the unvisited vertex with the current shortest known distance, enabling $O((V + E) \log V)$ time.

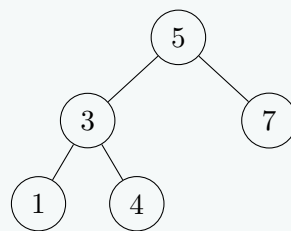
3. Huffman Encoding (Data Compression): Characters are inserted with their frequencies as priorities. The two lowest-frequency nodes are repeatedly extracted and merged to build the optimal prefix-free code tree.

Q42. Explain the insertion and deletion operations in a binary search tree with illustrative examples. (14 marks) [CSE 203 | 2011–12]

Answer

Insertion. Start at root; go left if key < current, right if key > current; insert at the first NULL position.

Example: insert {5, 3, 7, 1, 4}:



```

1  treeNode* insert(treeNode* root, int key) {
2      if (!root) return new treeNode(key);
3      if (key < root->item) root->left = insert(root->left,
4          key);
5      else if (key > root->item) root->right = insert(root->
6          right, key);
7      return root;
8  }
  
```

Deletion — three cases:

Case 1 (leaf): remove directly.

Case 2 (one child): splice — parent points to the single child.

Case 3 (two children): copy in-order successor's key into node; delete the successor (which has ≤ 1 child).

```

1  treeNode* minNode(treeNode* n){
2      while(n->left) n=n->left; return n;
3  }
4  treeNode* deleteNode(treeNode* root, int key){
5      if(!root) return root;
6      if(key < root->item) root->left = deleteNode(root->left,
7          key);
8      else if(key > root->item) root->right = deleteNode(root->
9          right, key);
10     else {
11         if(!root->left){ treeNode* t=root->right; delete root;
12             return t; }
13         if(!root->right){ treeNode* t=root->left; delete root
14             ; return t; }
15         treeNode* s = minNode(root->right);
  
```

```
12         root->item = s->item;
13         root->right = deleteNode(root->right, s->item);
14     }
15     return root;
16 }
```

Complexity Analysis. $O(h)$; average $O(\log n)$, worst $O(n)$ for skewed tree.

BUET · CSE 105

DSA

Graphs & Graph Representations

Adjacency Matrix, Adjacency List, Graph Properties

T7 — Graphs & Graph Representations

- Q1.** Given a graph $G = (V, E)$ with unique edge weights, let $(S, V - S)$ be a cut in G such that S is non-empty and $S \neq V$. Show that the minimum weighted crossing edge between any such cut in G is included in every minimum spanning tree. **(15 marks)** [CSE 105 | 2023–24]

Answer

Theorem (Cut Property). Let $G = (V, E)$ have unique edge weights, let $(S, V \setminus S)$ be any cut ($S \neq \emptyset$, $S \neq V$), and let $e^* = (u, v)$ be the unique minimum-weight edge crossing the cut. Then e^* belongs to *every* MST of G .

Proof (by contradiction).

Let T be an MST of G that does *not* contain e^* .

Since T is a spanning tree it is connected, so there exists a path P from u to v in T . Because $u \in S$ and $v \in V \setminus S$, this path must cross the cut at least once; let $e' = (x, y)$ be any such crossing edge on P (with $x \in S$, $y \in V \setminus S$).

Since e^* is the *unique* minimum crossing edge, $w(e^*) < w(e')$.

Construct $T' = T - \{e'\} + \{e^*\}$.

- T' is still spanning (same vertices).
- T' is still connected: removing e' splits T into two components; adding e^* reconnects them (since e^* also crosses the cut, linking the same two components).
- T' is acyclic: if a cycle existed it would have to use e^* ; but then e' would complete a cycle in T , contradicting T being a tree.
- $w(T') = w(T) - w(e') + w(e^*) < w(T)$.

This contradicts T being an MST. Therefore every MST must contain e^* . $\square$

- Q2.** Given an adjacency list representation of a directed graph $G = (V, E)$, sketch an efficient algorithm to find its transpose graph $G^T = (V, E^T)$ where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. Analyze the time complexity of your algorithm. Also evaluate how long it takes to compute the in-degree and out-degree of every vertex of G . **(18 marks)** [CSE 105 | 2022–23]

Answer

Transpose Graph G^T : reverse every edge $(u, v) \rightarrow (v, u)$.

Algorithm 68 TRANSPOSE(G)

```

1: Initialize  $G^T$  with same vertex set  $V$ , empty edge set
2: for each vertex  $u \in V$  do
3:   for each vertex  $v$  in  $\text{Adj}[u]$  do
4:     Add edge  $(v, u)$  to  $G^T$  ▷ reverse direction
5:   end for
6: end for
7: return  $G^T$ 
    
```

Complexity Analysis. We visit every vertex once and every edge once: $\Theta(V + E)$.

In-degree and Out-degree of every vertex:

Algorithm 69 COMPUTEDEGREES(G)

```

1:  $\text{outdeg}[u] \leftarrow 0$  for all  $u$ ;  $\text{indeg}[u] \leftarrow 0$  for all  $u$ 
2: for each vertex  $u \in V$  do
3:   for each  $v \in \text{Adj}[u]$  do
4:      $\text{outdeg}[u] += 1$ 
5:      $\text{indeg}[v] += 1$ 
6:   end for
7: end for
    
```

Complexity Analysis. Each edge is scanned once: $\Theta(V + E)$.

Remark. Using an adjacency *matrix*: out-degree of u = sum of row u ($O(V)$ per vertex, $O(V^2)$ total); in-degree of u = sum of column u ($O(V^2)$ total). So adjacency list is faster for sparse graphs.

Q3. A DAG is given to you. Find the maximum number of edges that can be added to this DAG such that the resulting graph is still a DAG (i.e., adding even one more edge would create a cycle). **(10 marks)** [CSE 203 | 2021–22]

Answer

Key insight: a directed graph is a DAG if and only if it has a topological ordering $v_1, v_2, \dots, v_n$. In any DAG, all edges go from v_i to v_j with $i < j$ in *some* topological order. The maximum number of such directed forward edges is $\binom{n}{2} = \frac{n(n-1)}{2}$ (one for every pair $i < j$).

Algorithm:

- (a) Run topological sort on the given DAG to obtain order $v_1, v_2, \dots, v_n$. Time: $O(V + E)$.
- (b) For every pair (i, j) with $i < j$, if edge (v_i, v_j) does not already exist, add it. These are the only edges we can add without creating a cycle.

Maximum edges that can be added:

$$\frac{n(n-1)}{2} - |E|$$

where $|E|$ is the current number of edges.

The resulting graph is a *DAG tournament* (every pair of vertices has exactly one directed edge between them), and adding any further edge would create a cycle.

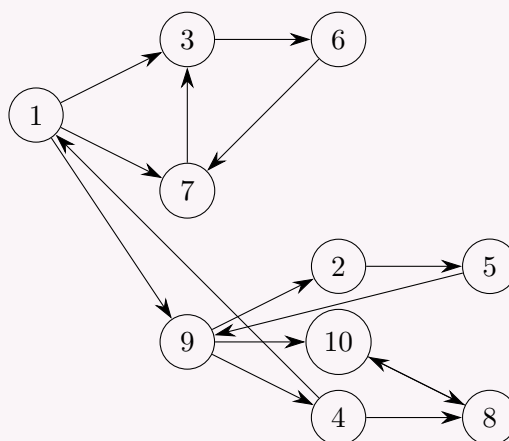
Remark. Different topological orderings may allow different sets of edges to be added, but the *count* $\binom{n}{2} - |E|$ is the same for all valid topological orderings.

- Q4.** Consider the adjacency matrix representation of a directed graph G . Draw the graph and find its adjacency list representation considering the vertices in ascending order. Find the shortest path distance of every vertex from vertex 1 using BFS. Show the queue status when vertex 10 is just enqueued. Prove that the distance of the most recent vertex (tail in the queue) is at most one larger than the distance of the oldest vertex (head in the queue). **(20 marks)** [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Answer

Graph (directed edges from the matrix):



Adjacency List (vertices in ascending order):

Vertex	Neighbours
1	3, 7, 9
2	5
3	6
4	1, 8
5	9
6	7
7	3
8	10
9	2, 4, 10
10	8

BFS from vertex 1 (neighbours processed in ascending order):

Step	Dequeue	Enqueue	Distances updated
1	1	3, 7, 9	$d[3] = 1, d[7] = 1, d[9] = 1$
2	3	6	$d[6] = 2$
3	7	(3 visited)	—
4	9	2, 4, 10	$d[2] = 2, d[4] = 2, d[10] = 2$
5	6	(7 visited)	—
6	2	5	$d[5] = 3$
7	4	8	$d[8] = 3$
8	10	(8 not yet — enqueued next)	$d[8] = 3$

Shortest distances from vertex 1:

Vertex	1	2	3	4	5	6	7	8	9	10
Distance	0	2	1	2	3	2	1	3	1	2

Queue state when vertex 10 is just enqueued (10 is enqueued while processing vertex 9):

Queue (front to back): $\underbrace{6}_{\text{head}}, 2, 4, \underbrace{10}_{\text{tail}}$

$d[\text{head}] = d[6] = 2, d[\text{tail}] = d[10] = 2.$

Proof: $d[\text{tail}] \leq d[\text{head}] + 1.$

Claim: At any point in BFS, if the queue contains $v_1, v_2, \dots, v_k$ (front to back), then $d[v_k] \leq d[v_1] + 1$ and $d[v_i] \leq d[v_{i+1}]$ for all i .

Proof by induction on BFS steps.

Base: initially only the source with $d = 0$; trivially true.

Step: suppose the property holds. We dequeue v_1 and enqueue its unvisited neighbours w with $d[w] = d[v_1] + 1$.

- New head is v_2 with $d[v_2] \geq d[v_1]$.
- New tail w satisfies $d[w] = d[v_1] + 1 \leq d[v_2] + 1 \leq d[\text{new head}] + 1$. ✓

Hence at all times $d[\text{tail}] \leq d[\text{head}] + 1$. □

Q5. Write a Graph (directed) data structure using a linked list. Write a delete-node operation for the data structure. **(8+7 marks)** [CSE 203 | 2019–20]

Answer

Data structures:

```

1  // Edge node in adjacency list
2  struct EdgeNode {
3      int dest;
4      EdgeNode* next;
5  };
6
7  // Vertex node in vertex list
8  struct VertexNode {
9      int id;
10     EdgeNode* edges;    // head of adjacency list
11     VertexNode* next;  // next vertex in vertex list
12 };
13
14 struct Graph {
15     VertexNode* vertices; // head of vertex linked list
16 };

```

Delete a vertex v from the graph:

- Remove all edges *from* v (free its edge list).
- Remove all edges *to* v from every other vertex's list.
- Remove v from the vertex list.

```

1  void deleteNode(Graph* g, int v) {
2      VertexNode* prev = NULL;
3      VertexNode* curr = g->vertices;
4
5      // Step 1 & 3: find v, free its edge list, remove from
6      // vertex list
7      while (curr && curr->id != v) { prev = curr; curr = curr->
8      next; }
9      if (!curr) return; // vertex not found
10
11     // Free edge list of v
12     EdgeNode* e = curr->edges;
13     while (e) { EdgeNode* tmp = e; e = e->next; free(tmp); }
14 }

```

```

13 // Remove v from vertex list
14 if (prev) prev->next = curr->next;
15 else g->vertices = curr->next;
16 free(curr);
17
18 // Step 2: remove all edges pointing TO v from other
    vertices
19 for (VertexNode* u = g->vertices; u; u = u->next) {
20     EdgeNode* ep = NULL, *en = u->edges;
21     while (en) {
22         if (en->dest == v) {
23             EdgeNode* tmp = en;
24             if (ep) ep->next = en->next;
25             else u->edges = en->next;
26             en = en->next;
27             free(tmp);
28         } else { ep = en; en = en->next; }
29     }
30 }
31 }

```

Complexity Analysis. $O(V + E)$ — we scan all vertices and their edge lists once.

Q6. Suppose you are going to represent a very dense graph and want to optimize both space and time for accessing an edge. Which representation (adjacency list or adjacency matrix) would you prefer? Justify your answer. (7 marks) [CSE 203 | 2017–18]

Answer

For a **very dense graph**, prefer the **adjacency matrix**.

Justification:

- **Space:** a dense graph has $E = \Theta(V^2)$ edges, so the adjacency list uses $O(V + E) = O(V^2)$ space — the same asymptotic space as the matrix. No space saving from the list.
- **Edge access:** the matrix gives $O(1)$ lookup for edge (u, v) , whereas the list requires $O(\deg(u)) = O(V)$ time.
- **Cache performance:** the matrix is a contiguous 2-D array; traversal is cache-friendly compared to pointer chasing in a list.

Therefore, for a very dense graph, the adjacency matrix is preferred for both space efficiency (same asymptotic cost) and faster edge access.

Q7. Compare adjacency matrix and adjacency list representations of a graph. (10 marks)

[CSE 203 | 2016–17 ,2013–14]

Answer

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge lookup (u, v)	$O(1)$	$O(\deg(u))$
Add edge	$O(1)$	$O(1)$
Remove edge	$O(1)$	$O(\deg(u))$
Enumerate neighbours of u	$O(V)$	$O(\deg(u))$
BFS / DFS total	$O(V^2)$	$O(V + E)$
Best for	Dense graphs	Sparse graphs

When to prefer which: For a *dense* graph ($E \approx V^2$) the matrix uses comparable space to the list but gives $O(1)$ edge lookup. For a *sparse* graph ($E \ll V^2$) the list saves space and graph traversal is faster.

Q8. Define graph, path, cycle, and subgraph with examples. **(10 marks)** [CSE 203 | 2015–16]

Answer

Graph $G = (V, E)$: a set of vertices V and edges $E \subseteq V \times V$.

Example: $V = \{1, 2, 3\}$, $E = \{(1, 2), (2, 3)\}$.

Path: a sequence of distinct vertices $v_0, v_1, \dots, v_k$ where $(v_{i-1}, v_i) \in E$ for each i .

Example: $1 \rightarrow 2 \rightarrow 3$ is a path of length 2.

Cycle: a path $v_0, v_1, \dots, v_k$ with $k \geq 1$ and $v_0 = v_k$ (start = end, all intermediate vertices distinct).

Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ is a cycle of length 3.

Subgraph $H = (V', E')$ of G : $V' \subseteq V$, $E' \subseteq E$, and every edge in E' has both endpoints in V' .

Example: $V' = \{1, 2\}$, $E' = \{(1, 2)\}$ is a subgraph of the above.

Q9. For each of the following operations in an undirected graph, find the worst-case running times in Big-O notation using (i) an adjacency matrix and (ii) an adjacency list:

- Determine whether a given vertex is connected to no other vertex.
- Determine whether the graph has a vertex that is connected to no other vertex.
- Determine whether a given vertex is connected to every other vertex. Give the idea of your algorithm for the adjacency list case.

(4+4+6=14 marks)

[CSE 203 | 2014–15]

Answer

(a) Is vertex v isolated (connected to no other vertex)?

- **Adjacency matrix:** scan row v for any 1 $\rightarrow O(V)$.
- **Adjacency list:** check if $\text{Adj}[v]$ is empty $\rightarrow O(1)$.

(b) Does the graph have any isolated vertex?

- **Adjacency matrix:** scan all rows $\rightarrow O(V^2)$.
- **Adjacency list:** scan all lists (total work = $V + E$) $\rightarrow O(V + E)$.

(c) Is vertex v connected to every other vertex?

- **Adjacency matrix:** scan row v , check all $V - 1$ non-diagonal entries are 1 $\rightarrow O(V)$.
- **Adjacency list:** maintain a boolean array $\text{seen}[1..V]$, initialise to **false**; traverse $\text{Adj}[v]$ and mark each neighbour **true**; finally scan the array checking all $u \neq v$ are marked $\rightarrow O(V + \deg(v))$. Worst case $O(V + E)$.

Remark. For the adjacency list case in (c): we cannot simply check $\deg(v) = V - 1$ because v might have multi-edges or self-loops in a general graph. The boolean array approach handles the general case correctly in $O(V + \deg(v))$.

Q10. Give a brief description of: (i) Graph, (ii) Forest, (iii) Tree, (iv) Spanning subgraph.
(1.5+1.5+1.5+2.5=7 marks) [CSE 203 | 2013–14]

Answer

- (i) **Graph:** A graph $G = (V, E)$ consists of a finite set of vertices V and a set of edges $E \subseteq V \times V$. Edges may be directed or undirected, weighted or unweighted.
- (ii) **Forest:** An undirected acyclic graph (a graph with no cycles). A forest may be disconnected; each connected component is a tree.
- (iii) **Tree:** A connected, acyclic, undirected graph. Equivalently, a connected graph on n vertices with exactly $n - 1$ edges. A forest where every component is connected.
- (iv) **Spanning Subgraph:** A subgraph $H = (V, E')$ of $G = (V, E)$ that contains *all* vertices of G ($V(H) = V(G)$) and a subset of edges ($E' \subseteq E$). If H is also a tree, it is called a *spanning tree*.

Q11. Show the adjacency matrix and adjacency list for the following graph. Calculate the number of bytes required in both representations. Assume that the storage requirement of a pointer and an integer is 4 bytes.

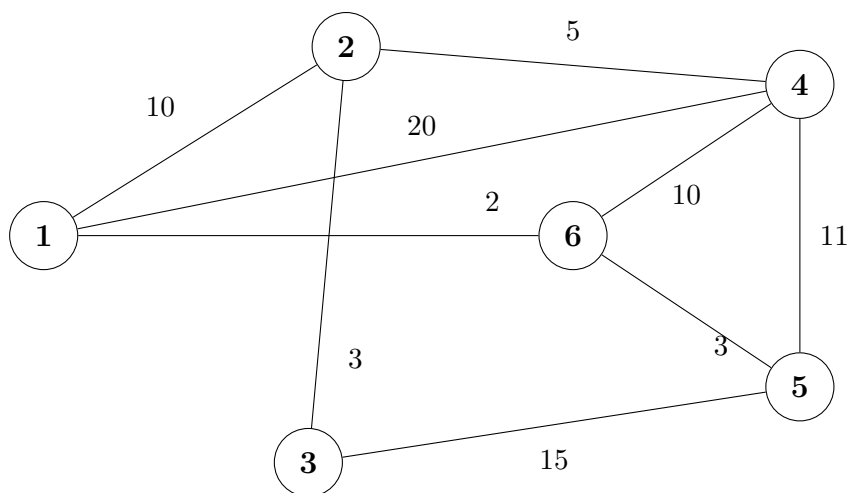


Figure 1: Weighted Graph

Answer

Graph: $V = \{1, 2, 3, 4, 5, 6\}$, 9 undirected weighted edges.

Adjacency Matrix (entry = weight, 0 = no edge):

	1	2	3	4	5	6
1	0	10	0	20	0	2
2	10	0	3	5	0	0
3	0	3	0	0	15	0
4	20	5	0	0	11	10
5	0	0	15	11	0	3
6	2	0	0	10	3	0

Byte count (matrix): $6 \times 6 = 36$ integers $\times$ 4 bytes = **144** bytes.

Adjacency List:

Vertex	List (neighbour, weight)
1	$\rightarrow(2,10) \rightarrow(4,20) \rightarrow(6,2) \rightarrow \text{NULL}$
2	$\rightarrow(1,10) \rightarrow(3,3) \rightarrow(4,5) \rightarrow \text{NULL}$
3	$\rightarrow(2,3) \rightarrow(5,15) \rightarrow \text{NULL}$
4	$\rightarrow(1,20) \rightarrow(2,5) \rightarrow(5,11) \rightarrow(6,10) \rightarrow \text{NULL}$
5	$\rightarrow(3,15) \rightarrow(4,11) \rightarrow(6,3) \rightarrow \text{NULL}$
6	$\rightarrow(1,2) \rightarrow(4,10) \rightarrow(5,3) \rightarrow \text{NULL}$

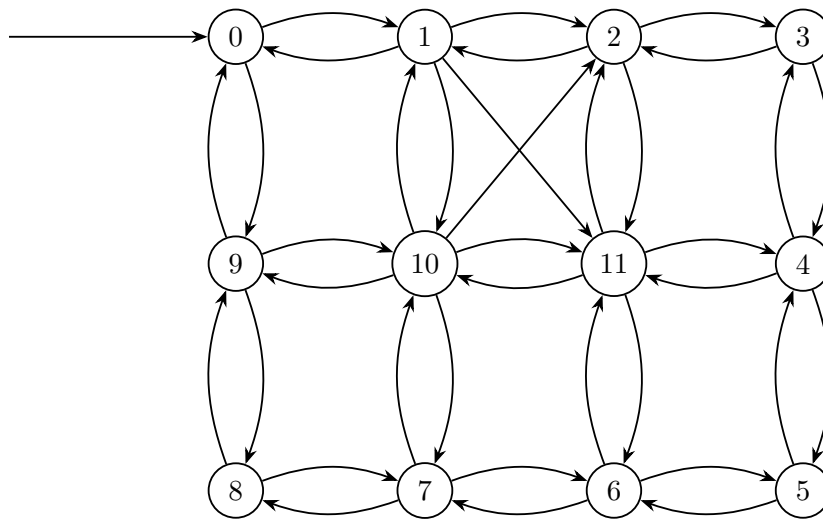
Byte count (list): Each list node stores: **dest** (4B) + **weight** (4B) + **next*** (4B) = 12 bytes.

For an undirected graph, each edge is stored *twice*: total list nodes = $2 \times 9 = 18$.

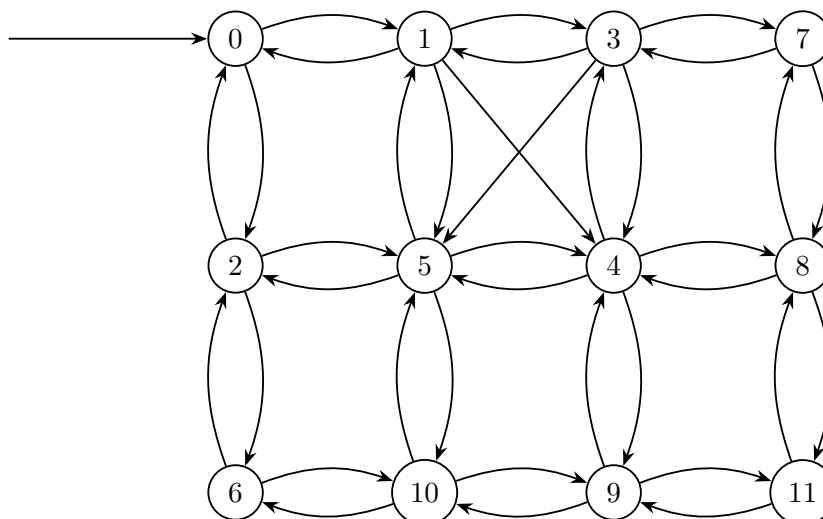
$$\underbrace{6 \times 4}_{\text{head pointers}} + \underbrace{18 \times 12}_{\text{list nodes}} = 24 + 216 = \mathbf{240} \text{ bytes.}$$

Remark. The adjacency matrix (144 B) uses *less* space than the list (240 B) for this particular graph because the graph is relatively dense ($E = 9$ out of a possible $\binom{6}{2} = 15$ edges).

- Q12.** Your friend has employed two different traversal algorithms, *Algo I* and *Algo II*, on a graph and shown the output in Figure Q.5(c)-I and Figure Q.5(c)-II respectively. In both cases, the traversal starts at the node shown by the block right arrow. In the output, nodes are labeled with the order they are traversed. Identify the traversal algorithm for Figure Q.5(c)-I and Figure Q.5(c)-II and justify your answer. (10 marks)



FigureQ.5(c) – I



FigureQ.5(c) – II

Answer**Figure Q.5(c)-I: DFS (Depth-First Search)**

Justification: The problem specifies that the node labels represent the *order* of traversal, not fixed node IDs. If we follow the numeric sequence ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 11$), we trace the exact path the algorithm took. Starting from node 0, the algorithm moves to an adjacent neighbor (1) and immediately continues to plunge deeper into the graph to unvisited neighbors (2, then 3, then 4), rather than pausing to visit node 0's other immediate neighbor (which ends up being labeled 9 much later). The path forms a continuous snake that wraps around the perimeter and spirals inward. This strategy of exploring a single branch "as deep as possible" before backtracking is the exact definition of Depth-First Search.

Figure Q.5(c)-II: BFS (Breadth-First Search)

Justification: By following the numeric labels ($0, 1, 2, 3, \dots$), we can observe that the nodes are visited in a wave-like, layer-by-layer pattern based on their shortest distance (number of edges) from the start node.

- **Distance 0:** The start node is labeled **0**.
- **Distance 1:** The immediate neighbors of the start node are labeled **1** and **2**.
- **Distance 2:** The unvisited neighbors of nodes 1 and 2 are visited next, receiving the labels **3, 4, 5, and 6**.
- **Distance 3:** The unvisited neighbors of the previous layer receive the labels **7, 8, 9, and 10**.
- **Distance 4:** The furthest node in the graph is finally labeled **11**.

Because the algorithm completely exhausts all nodes at a distance k from the source before moving to any node at distance $k + 1$, this strictly layer-by-layer expansion is the hallmark of Breadth-First Search.

BUET · CSE 105

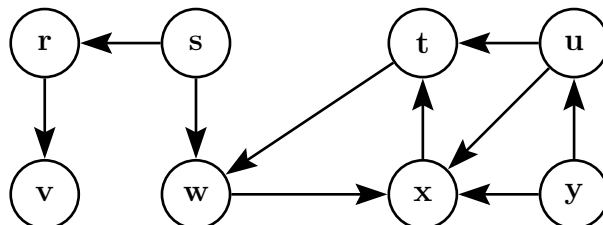
DSA

Graph Traversals: DFS & BFS

DFS Algorithm, BFS Algorithm, Edge Classification

T8 — Graph Traversals: DFS & BFS

- Q1.** Using a stack, construct an iterative (non-recursive) algorithm for depth first search (DFS) of a graph. Show how the state of the stack changes over time using the graph given in the figure. Analyze the complexity. Modify the algorithm to print the type of each edge it encounters. **(20 marks)** [CSE 105 | 2023–24]



Answer

Algorithm 70 ITERATIVE-DFS(G)

```

1: for each  $v \in V[G]$  do
2:    $color[v] \leftarrow \text{WHITE}$ ;  $parent[v] \leftarrow \text{nil}$ 
3: end for
4:  $time \leftarrow 0$ 
5: for each  $s \in V[G]$  do
6:   if  $color[s] = \text{WHITE}$  then
7:      $PUSH(S, s)$ ;  $color[s] \leftarrow \text{GRAY}$ 
8:      $time \leftarrow time + 1$ ;  $d[s] \leftarrow time$ 
9:     while  $S \neq \emptyset$  do
10:       $u \leftarrow PEEK(S)$ 
11:       $found\_white \leftarrow \text{false}$ 
12:      for each  $v \in Adj[u]$  in order do
13:        if  $color[v] = \text{WHITE}$  then
14:           $color[v] \leftarrow \text{GRAY}$ ;  $time \leftarrow time + 1$ ;  $d[v] \leftarrow time$ 
15:           $parent[v] \leftarrow u$ ;  $PUSH(S, v)$ 
16:           $found\_white \leftarrow \text{true}$ ; break
17:        end if
18:      end for
19:      if not  $found\_white$  then
20:         $POP(S)$ ;  $color[u] \leftarrow \text{BLACK}$ 
21:         $time \leftarrow time + 1$ ;  $f[u] \leftarrow time$ 
22:      end if
23:    end while
24:  end if
25: end for

```

Stack trace (Adj lists: $r:[v]$, $s:[r, w]$, $t:[w]$, $u:[t, x]$, $v:[]$, $w:[x]$, $x:[t]$, $y:[u, x]$):

Action	Stack (bot→top)	Event	$d[]$	$f[]$
Start r	$[r]$	Push r	$d[r] = 1$	
$r \rightarrow v$	$[r, v]$	Tree: $r \rightarrow v$	$d[v] = 2$	
v done	$[r]$	Pop v		$f[v] = 3$
r done	$[]$	Pop r		$f[r] = 4$
Start s	$[s]$	Push s	$d[s] = 5$	
$s \rightarrow r$	$[s]$	Cross: $s \rightarrow r$ (r BLACK)		
$s \rightarrow w$	$[s, w]$	Tree: $s \rightarrow w$	$d[w] = 6$	
$w \rightarrow x$	$[s, w, x]$	Tree: $w \rightarrow x$	$d[x] = 7$	
$x \rightarrow t$	$[s, w, x, t]$	Tree: $x \rightarrow t$	$d[t] = 8$	
$t \rightarrow w$	$[s, w, x, t]$	Back: $t \rightarrow w$ (w GRAY)		
t done	$[s, w, x]$	Pop t		$f[t] = 9$
x done	$[s, w]$	Pop x		$f[x] = 10$
w done	$[s]$	Pop w		$f[w] = 11$
s done	$[]$	Pop s		$f[s] = 12$
Start u	$[u]$	Push u	$d[u] = 13$	
$u \rightarrow t$	$[u]$	Cross: $u \rightarrow t$ (t BLACK, $d[u] > d[t]$)		
$u \rightarrow x$	$[u]$	Cross: $u \rightarrow x$ (x BLACK, $d[u] > d[x]$)		
u done	$[]$	Pop u		$f[u] = 14$
Start y	$[y]$	Push y	$d[y] = 15$	
$y \rightarrow u$	$[y]$	Cross: $y \rightarrow u$ (u BLACK, $d[y] > d[u]$)		
$y \rightarrow x$	$[y]$	Cross: $y \rightarrow x$ (x BLACK)		
y done	$[]$	Pop y		$f[y] = 16$

Final timestamps:

Vertex	r	s	t	u	v	w	x	y
d	1	5	8	13	2	6	7	15
f	4	12	9	14	3	11	10	16

Edge classification: Tree: $(r, v), (s, w), (w, x), (x, t)$; Back: (t, w) ; Cross: $(s, r), (u, t), (u, x), (y, u), (y, x)$; Forward: none.

Complexity: $O(V + E)$. Each vertex is pushed/popped once ($O(V)$); each edge is examined once ($O(E)$).

Modified algorithm to print edge types — add inside the neighbor loop:

Algorithm 71 DFS-VISIT-CLASSIFY(G, u)

```

1:  $color[u] \leftarrow \text{GRAY}; \quad time \leftarrow time + 1; \quad d[u] \leftarrow time$ 
2: for each  $v \in Adj[u]$  do
3:   if  $color[v] = \text{WHITE}$  then
4:     print “Tree edge:  $u \rightarrow v$ ”
5:      $parent[v] \leftarrow u$ 
6:     DFS-VISIT-CLASSIFY( $G, v$ )
7:   else if  $color[v] = \text{GRAY}$  then
8:     print “Back edge:  $u \rightarrow v$ ”
9:   else
10:    if  $d[u] < d[v]$  then
11:      print “Forward edge:  $u \rightarrow v$ ”
12:    else
13:      print “Cross edge:  $u \rightarrow v$ ”
14:    end if
15:  end if
16: end for
17:  $color[u] \leftarrow \text{BLACK}; \quad time \leftarrow time + 1; \quad f[u] \leftarrow time$ 

```

Q2. Define the four types of edges that a DFS on a directed graph yields. Modify the pseudocode of the original DFS algorithm to print all the edges along with their types. Justify whether your algorithm will work correctly. **(18 marks)** [CSE 105 | 2022–23]

Answer**Four DFS edge types (directed graph):**

Type	Condition when (u, v) examined	Example
Tree edge	v is WHITE (undiscovered)	builds DFS tree
Back edge	v is GRAY (open ancestor)	$v.d < u.d$; indicates cycle
Forward edge	v is BLACK, $u.d < v.d$	v is non-tree descendant
Cross edge	v is BLACK, $u.d > v.d$	v in different branch/tree

Modified DFS pseudocode:

Algorithm 72 DFS(G)

```

1: for each  $u \in V[G]$  do
2:    $color[u] \leftarrow \text{WHITE}$ ;  $parent[u] \leftarrow \text{nil}$ 
3: end for
4:  $time \leftarrow 0$ 
5: for each  $u \in V[G]$  do
6:   if  $color[u] = \text{WHITE}$  then
7:     DFSVISIT( $G, u$ )
8:   end if
9: end for

```

Algorithm 73 DFSVISIT(G, u)

```

1:  $color[u] \leftarrow \text{GRAY}$ ;  $time \leftarrow time + 1$ ;  $d[u] \leftarrow time$ 
2: for each  $v \in Adj[u]$  do
3:   if  $color[v] = \text{WHITE}$  then
4:      $parent[v] \leftarrow u$ 
5:     print “Tree edge: ( $u, v$ )”
6:     DFSVISIT( $G, v$ )
7:   else if  $color[v] = \text{GRAY}$  then
8:     print “Back edge: ( $u, v$ )”
9:   else
10:    if  $d[u] < d[v]$  then
11:      print “Forward edge: ( $u, v$ )”
12:    else
13:      print “Cross edge: ( $u, v$ )”
14:    end if
15:  end if
16: end for
17:  $color[u] \leftarrow \text{BLACK}$ ;  $time \leftarrow time + 1$ ;  $f[u] \leftarrow time$ 

```

Justification of correctness:

At the moment edge (u, v) is examined, u is GRAY (being processed). The color of v and the timestamps uniquely determine the edge type:

- v WHITE $\Rightarrow$ not yet visited; DFS will now discover it via $u \Rightarrow$ tree edge by definition.
- v GRAY $\Rightarrow v$ is an ancestor still open on the DFS stack $\Rightarrow$ back edge (going backward in the current path).
- v BLACK, $d[u] < d[v] \Rightarrow v$ was discovered after u but finished before u examined this edge; by Parenthesis Theorem v is a descendant of u (nested interval) $\Rightarrow$ forward edge.
- v BLACK, $d[u] > d[v] \Rightarrow v$ was discovered and fully finished before u was even discovered; the intervals are disjoint $\Rightarrow$ cross edge.

Note: For undirected graphs, only tree and back edges occur — the check $d[u] < d[v]$ vs. $d[u] > d[v]$ is unnecessary.

Q3. Classify the edges of the directed graph G after running DFS on the graph considering the vertices in ascending order. Prove that an edge (u, v) is a tree edge if $u.d < v.d < v.f < u.f$. (15 marks) [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Answer

Adjacency list (from matrix): 1:[3, 7, 9], 2:[5], 3:[6], 4:[1, 8], 5:[9], 6:[7], 7:[3], 8:[10], 9:[2, 4, 10], 10:[8].

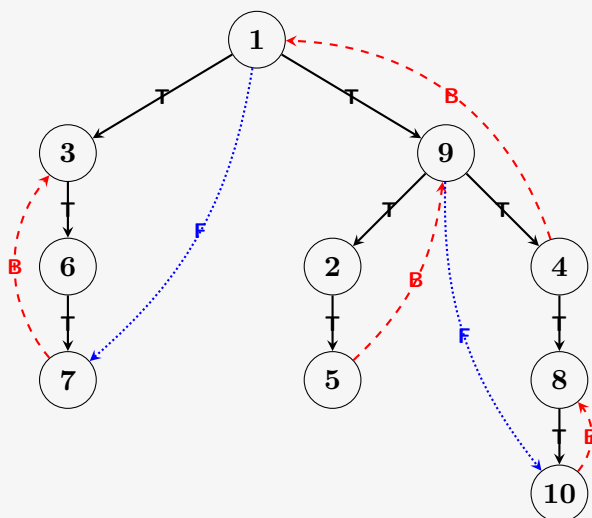
DFS timestamps (ascending order; start from 1):

Vertex	1	2	3	4	5	6	7	8	9	10
d	1	9	2	13	10	3	4	14	8	15
f	20	12	7	18	11	6	5	17	19	16
parent	—	9	1	9	2	3	6	4	1	8

Edge classification:

- **Tree edges (T):** (1, 3), (3, 6), (6, 7), (1, 9), (9, 2), (2, 5), (9, 4), (4, 8), (8, 10)
- **Back edges (B):** (7, 3), (5, 9), (4, 1), (10, 8)
- **Forward edges (F):** (1, 7), (9, 10)
- **Cross edges (C):** none

DFS Graph Representation:



Proof: (u, v) is a tree edge $\Leftrightarrow u.d < v.d < v.f < u.f$

($\Rightarrow$) Suppose (u, v) is a tree edge. By definition, v was WHITE when u examined it, and DFS called DFS-VISIT(v) immediately. Thus:

1. u was already GRAY before v was discovered $\Rightarrow u.d < v.d$.
2. v 's entire subtree finishes before DFS returns to $u \Rightarrow v.f < u.f$.
3. v is discovered then finished $\Rightarrow v.d < v.f$.

Hence $u.d < v.d < v.f < u.f$.

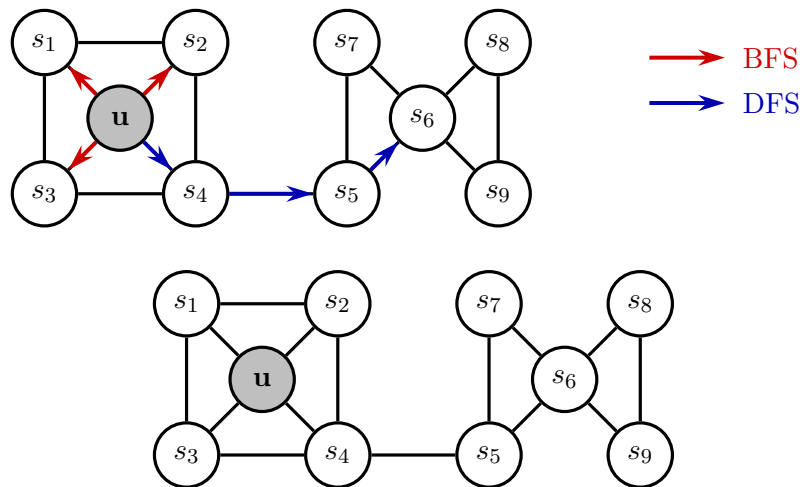
($\Leftarrow$) Suppose $u.d < v.d < v.f < u.f$. By the **Parenthesis Theorem**, the interval $[v.d, v.f] \subset [u.d, u.f]$, so v is a descendant of u in the DFS tree. Now suppose for contradiction (u, v) is not a tree edge but a forward edge. Then v was already BLACK when (u, v) was examined — but this contradicts v being in u 's subtree with $v.d > u.d$ (DFS would have made it a tree edge on first encounter). Hence (u, v) must be a tree edge. $\square$

Verification: $(1, 3)$: $1 < 2 < 7 < 20$ $\checkmark$; $(9, 4)$: $8 < 13 < 18 < 19$ $\checkmark$.

Q4. You are given the following undirected graph to traverse using both BFS and DFS starting from vertex u . Show the simulation steps for labeling the vertices in order of visits. In BFS, the label of a vertex is its level; in DFS, label each vertex with its discovery and finishing times. Also write the pseudocodes for BFS and DFS.

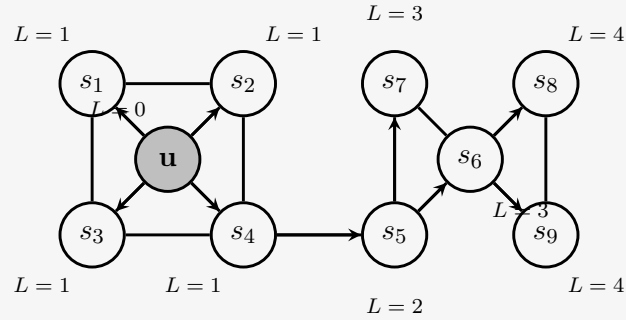
(10+10 marks)

[CSE 203 | 2019–20]



Detailed Answer with Simulation Graphs

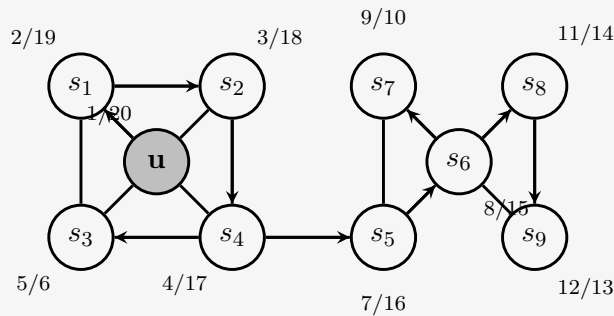
1. BFS Traversal starting from u (Alphabetical neighbor order)

**Simulation Steps (Queue & Labeling):**

- **Init:** Enqueue u . Label Level $L(u) = 0$. Queue: $[u]$
- **Step 1:** Dequeue u . Unvisited neighbors: s_1, s_2, s_3, s_4 . Label $L = 1$. Enqueue all. Queue: $[s_1, s_2, s_3, s_4]$
- **Step 2-4:** Dequeue s_1, s_2, s_3 . None have new unvisited neighbors. Queue: $[s_4]$
- **Step 5:** Dequeue s_4 . Unvisited neighbor: s_5 . Label $L(s_5) = 2$. Enqueue s_5 . Queue: $[s_5]$
- **Step 6:** Dequeue s_5 . Unvisited neighbors: s_6, s_7 . Label $L = 3$. Enqueue s_6, s_7 . Queue: $[s_6, s_7]$
- **Step 7:** Dequeue s_6 . Unvisited neighbors: s_8, s_9 . Label $L = 4$. Enqueue s_8, s_9 . Queue: $[s_7, s_8, s_9]$
- **Step 8-10:** Dequeue s_7, s_8, s_9 . No unvisited neighbors left. Queue empty. Done.

Vertex	u	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9
Level	0	1	1	1	1	2	3	3	4	4
Parent	—	u	u	u	u	s_4	s_5	s_5	s_6	s_6

BFS order: $u, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9$

2. DFS Traversal starting from u (Alphabetical neighbor order)**Simulation Steps (Discovery d / Finish f times):**

- Visit u ($d = 1$). Neighbors: s_1, s_2, s_3, s_4 . Go to s_1 .
- Visit s_1 ($d = 2$). Neighbors: s_2, s_3, u . Go to s_2 .
- Visit s_2 ($d = 3$). Neighbors: s_1, s_4, u . Go to s_4 .
- Visit s_4 ($d = 4$). Neighbors: s_2, s_3, s_5, u . Go to s_3 .
- Visit s_3 ($d = 5$). Neighbors visited. **Finish** s_3 ($f = 6$). Backtrack to s_4 .
- At s_4 : Next unvisited neighbor is s_5 . Go to s_5 .
- Visit s_5 ($d = 7$). Neighbors: s_4, s_6, s_7 . Go to s_6 .
- Visit s_6 ($d = 8$). Neighbors: s_5, s_7, s_8, s_9 . Go to s_7 .
- Visit s_7 ($d = 9$). Neighbors visited. **Finish** s_7 ($f = 10$). Backtrack to s_6 .
- At s_6 : Next unvisited is s_8 . Go to s_8 .
- Visit s_8 ($d = 11$). Neighbors: s_6, s_9 . Go to s_9 .
- Visit s_9 ($d = 12$). Neighbors visited. **Finish** s_9 ($f = 13$). Backtrack to s_8 .
- **Finish** s_8 ($f = 14$) → **Finish** s_6 ($f = 15$) → **Finish** s_5 ($f = 16$) → **Finish** s_4 ($f = 17$) → **Finish** s_2 ($f = 18$) → **Finish** s_1 ($f = 19$) → **Finish** u ($f = 20$).

Vertex	u	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9
d	1	2	3	5	4	7	8	9	11	12
f	20	19	18	6	17	16	15	10	14	13

DFS order (by discovery): $u, s_1, s_2, s_4, s_3, s_5, s_6, s_7, s_8, s_9$

3. Pseudocodes

Algorithm 74 BFS(G, s)

```

1: for each  $u \in V[G] \setminus \{s\}$  do
2:    $color[u] \leftarrow \text{WHITE}; d[u] \leftarrow \infty; parent[u] \leftarrow \text{nil}$ 
3: end for
4:  $color[s] \leftarrow \text{GRAY}; d[s] \leftarrow 0; parent[s] \leftarrow \text{nil}$ 
5:  $Q \leftarrow$  empty queue; ENQUEUE( $Q, s$ )
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow$  DEQUEUE( $Q$ )
8:   for each  $v \in Adj[u]$  do
9:     if  $color[v] = \text{WHITE}$  then
10:       $color[v] \leftarrow \text{GRAY}$ 
11:       $d[v] \leftarrow d[u] + 1$ 
12:       $parent[v] \leftarrow u$ 
13:      ENQUEUE( $Q, v$ )
14:    end if
15:  end for
16:   $color[u] \leftarrow \text{BLACK}$ 
17: end while
    
```

DFS Pseudocode:

Algorithm 75 DFS(G)

```

1: for each  $u \in V[G]$  do
2:    $color[u] \leftarrow \text{WHITE}; parent[u] \leftarrow \text{nil}$ 
3: end for
4:  $time \leftarrow 0$ 
5: for each  $u \in V[G]$  do
6:   if  $color[u] = \text{WHITE}$  then
7:     DFSVISIT( $G, u$ )
8:   end if
9: end for

```

Algorithm 76 DFSVISIT(G, u)

```

1:  $color[u] \leftarrow \text{GRAY}; time \leftarrow time + 1; d[u] \leftarrow time$ 
2: for each  $v \in Adj[u]$  do
3:   if  $color[v] = \text{WHITE}$  then
4:      $parent[v] \leftarrow u$ 
5:     DFSVISIT( $G, v$ )
6:   end if
7: end for
8:  $color[u] \leftarrow \text{BLACK}; time \leftarrow time + 1; f[u] \leftarrow time$ 

```

Q5. Explain when there is no back edge if DFS is performed on an undirected graph. (8 marks) [CSE 203 | 2018–19]

Answer

There is no back edge in undirected DFS if and only if the graph is acyclic (i.e., a forest/tree).

- In undirected DFS, every edge (u, v) is either a *tree edge* (when v is WHITE) or a *back edge* (when v is GRAY, i.e., v is an open ancestor). Forward and cross edges do not occur in undirected graphs.
- A back edge (u, v) with v GRAY means v is an ancestor of u , creating a cycle $v \rightsquigarrow u \rightarrow v$.
- If the graph has no cycles (is a forest), every neighbor of u is either its parent (already seen but handled as parent, not a back edge) or undiscovered — so no back edges arise.
- Conversely, every cycle produces at least one back edge during DFS.

Summary: No back edges $\Leftrightarrow$ graph is a forest (acyclic).

Q6. Give the running times of DFS using adjacency lists and adjacency matrix. (8

(marks)

[CSE 203 | 2018–19]

Answer

Representation	Running Time	Reason
Adjacency list	$\Theta(V + E)$	Neighbor scan: $\sum \text{Adj}[u] = E$
Adjacency matrix	$\Theta(V^2)$	Each row scanned fully: $V \times V$

For sparse graphs ($E \ll V^2$), adjacency list is preferred. For dense graphs ($E = \Theta(V^2)$), both representations give the same asymptotic bound.

Q7. Would you use BFS or DFS to find any path from a start vertex to a goal in a graph with many long paths? Justify. (10 marks) [CSE 203 | 2018–19]

Answer

Use DFS.

- **DFS follows one long path immediately** without expanding all neighbors, likely hitting the goal quickly.
- **Memory:** DFS uses $O(\text{path length})$ stack space. BFS stores all vertices at each level: $O(b^d)$ where b = branching factor, d = distance. For long paths in a wide graph, BFS memory is prohibitive.
- **Optimality not needed:** We only need *any* path, so DFS's non-shortest path is acceptable.

Trade-off: BFS finds the *shortest* path; DFS finds a path faster with less memory. Since optimality is not required, **DFS is the better choice**.

Q8. “Given a directed graph G , both BFS and DFS from a particular starting vertex s will traverse the same set of vertices.” Do you agree with this statement? Justify your answer. (8 marks) [CSE 203 | 2017–18]

Answer

Yes — both traverse exactly the set of vertices reachable from s .

Both BFS and DFS from s only visit vertices v such that there exists a directed path $s \rightsquigarrow v$. Neither can jump to a vertex with no such path.

- BFS enqueues only reachable vertices; unreachable ones stay WHITE.
- DFS-Visit(s) only recurses into WHITE neighbors, always following directed edges from s .

However, the *order* of visitation and the resulting *tree structure* (BFS tree vs. DFS tree) differ. Also, if we run the full DFS procedure (outer loop over all vertices), it visits all vertices including unreachable ones — but that is not “from s ” alone.

- Q9.** Given an undirected graph $G = (V, E)$, give a linear-time algorithm to check whether there is a cycle of odd length in the graph. Briefly justify its correctness. **(15 marks)**
[CSE 207 | 2015–16]

Answer

Key theorem: A graph has an odd-length cycle $\Leftrightarrow$ it is NOT bipartite.

Algorithm 77 HAS-ODD-CYCLE(G)

```

1: for each  $v \in V[G]$  do
2:    $color[v] \leftarrow -1$ 
3: end for
4: for each  $s \in V[G]$  do
5:   if  $color[s] = -1$  then
6:      $Q \leftarrow$  empty queue
7:      $color[s] \leftarrow 0$ ; ENQUEUE( $Q, s$ )
8:     while  $Q \neq \emptyset$  do
9:        $u \leftarrow$  DEQUEUE( $Q$ )
10:      for each  $v \in Adj[u]$  do
11:        if  $color[v] = -1$  then
12:           $color[v] \leftarrow 1 - color[u]$ 
13:          ENQUEUE( $Q, v$ )
14:        else if  $color[v] = color[u]$  then
15:          return true  $\triangleright$  odd cycle found
16:        end if
17:      end for
18:    end while
19:  end if
20: end for
21: return false

```

Correctness:

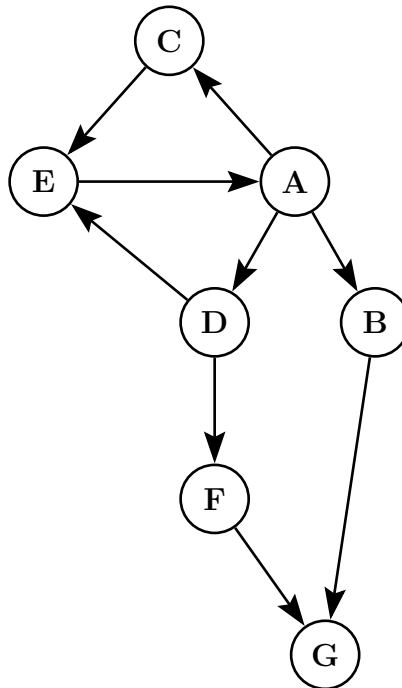
- **Bipartite $\Rightarrow$ no odd cycle:** In any 2-coloring, every edge connects vertices of opposite colors. Any cycle alternates colors and must have even length.
- **Not bipartite $\Rightarrow$ odd cycle:** If two adjacent vertices u, v get the same color during BFS, the BFS-level paths from their common ancestor to u and v have the same length, so the cycle ancestor $\rightsquigarrow u \rightarrow v \rightsquigarrow$ ancestor has odd length.

Time: $O(V + E)$ (standard BFS).

- Q10.** Consider the following directed graph. Answer the following questions:

- Perform depth first search on the graph starting from vertex A . Choose the smallest (in alphabetical order) vertex when there is a choice. Find the discovery time and finishing time of each vertex.
- Perform breadth first search on the graph starting from vertex A . Choose the smallest (in alphabetical order) vertex when there is a choice. Find the distance

and parent of each vertex.



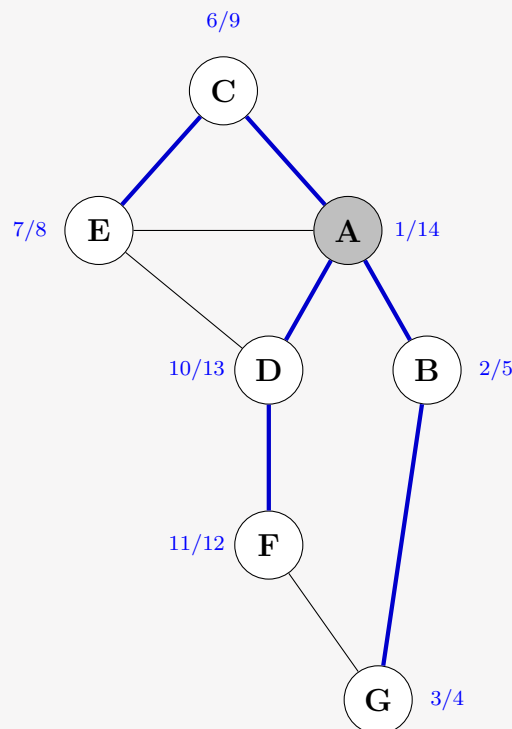
(6+6=12 marks)

[CSE 203 | 2014–15]

Answer

Adjacency List: $A:[B, C, D]$; $B:[G]$; $C:[E]$; $D:[E, F]$; $E:[A]$; $F:[G]$; $G:[]$

(a) DFS Traversal starting from A (Alphabetical order)



Simulation Steps:

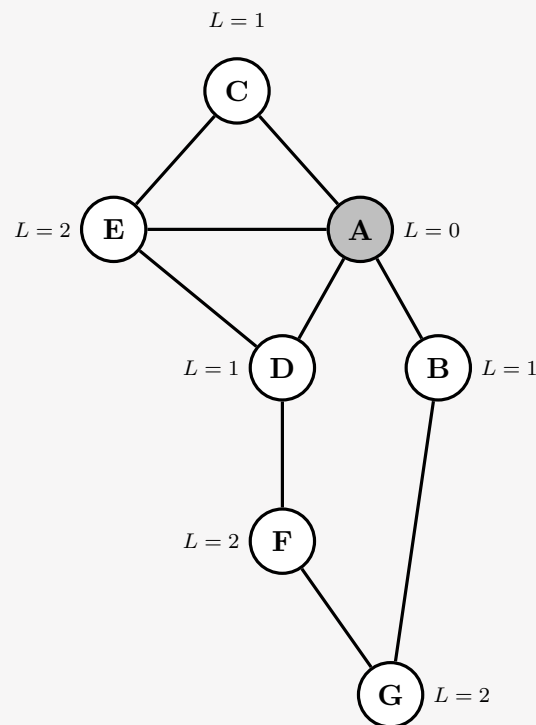
- Start at A ($d = 1$). Neighbors: B, C, D . Go to B .
- Visit B ($d = 2$). Neighbors: G . Go to G .
- Visit G ($d = 3$). No neighbors. **Finish** G ($f = 4$). Backtrack to B .
- **Finish** B ($f = 5$). Backtrack to A . Next neighbor C .
- Visit C ($d = 6$). Neighbors: E . Go to E .
- Visit E ($d = 7$). Neighbor A is already GRAY (Back edge). **Finish** E ($f = 8$). Backtrack to C .
- **Finish** C ($f = 9$). Backtrack to A . Next neighbor D .
- Visit D ($d = 10$). Neighbors: E, F . E is BLACK (Cross edge). Go to F .
- Visit F ($d = 11$). Neighbor G is BLACK (Cross edge). **Finish** F ($f = 12$). Backtrack to D .
- **Finish** D ($f = 13$). Backtrack to A . **Finish** A ($f = 14$).

Vertex	A	B	C	D	E	F	G
d	1	2	6	10	7	11	3
f	14	5	9	13	8	12	4

Edge types:

- **Tree edges:** $(A, B), (B, G), (A, C), (C, E), (A, D), (D, F)$
- **Back edges:** (E, A)
- **Cross edges:** $(D, E), (F, G)$

(b) BFS Traversal starting from A (Alphabetical order)


Simulation Steps (Queue):

- **Init:** Enqueue A . Level $L(A) = 0$. **Q:** $[A]$
- **Step 1:** Dequeue A . Unvisited neighbors: B, C, D . Label $L = 1$, parent A . Enqueue B, C, D . **Q:** $[B, C, D]$
- **Step 2:** Dequeue B . Unvisited neighbors: G . Label $L(G) = 2$, parent B . Enqueue G . **Q:** $[C, D, G]$
- **Step 3:** Dequeue C . Unvisited neighbors: E . Label $L(E) = 2$, parent C . Enqueue E . **Q:** $[D, G, E]$
- **Step 4:** Dequeue D . Neighbors: E, F . E is visited. Label $L(F) = 2$, parent D . Enqueue F . **Q:** $[G, E, F]$
- **Step 5-7:** Dequeue G, E, F . No unvisited neighbors left. **Q:** $[]$. Done.

BFS Order: A, B, C, D, G, E, F

Vertex	A	B	C	D	E	F	G
dist (Level)	0	1	1	1	2	2	2
parent	—	A	A	A	C	D	B

Q11. Suggest an algorithm based on DFS that counts the number of connected components in an undirected graph. (8 marks) [CSE 203 | 2014–15]

Answer

Algorithm 78 COUNT-COMPONENTS(G)

```

1: for each  $u \in V[G]$  do
2:    $color[u] \leftarrow \text{WHITE}$ 
3: end for
4:  $count \leftarrow 0$ 
5: for each  $u \in V[G]$  do
6:   if  $color[u] = \text{WHITE}$  then
7:      $count \leftarrow count + 1$ 
8:     DFS-VISIT( $G, u$ )
9:   end if
10: end for return  $count$ 
    
```

Justification: A single DFS-VISIT(u) marks exactly all vertices reachable from u — the complete connected component containing u . Each new WHITE vertex in the outer loop belongs to a previously unseen component.

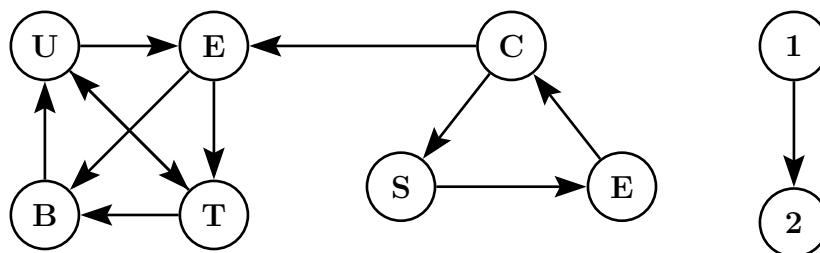
Time: $O(V + E)$.

Q12. Consider the graph G . Perform the following:

- Write the adjacency matrix and adjacency list of graph G .
- Draw the BFS forest of graph G assuming B as the starting vertex.
- Write the sequence of vertices if you explore graph G starting from vertex B in DFS.
- Write the tree edges, back edges, forward edges, and cross edges.

(6+6+4+6=22 marks)

[CSE 203 | 2013–14]


Detailed Answer with Simulation Graphs

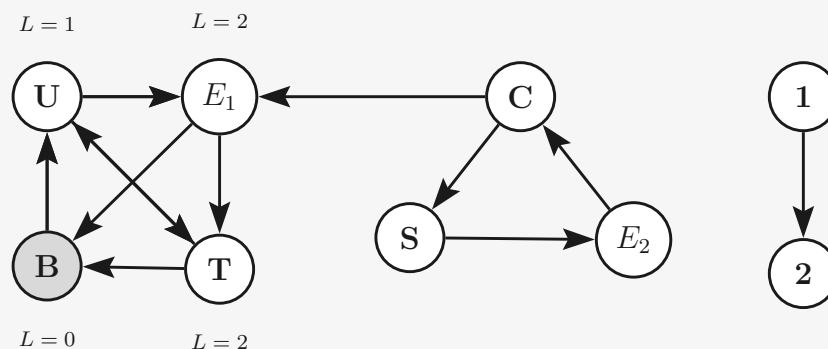
Graph Analysis: The complete graph G has 9 vertices: $\{1, 2, B, C, E_1, E_2, S, T, U\}$. Note: There are two nodes labeled 'E' in the figure. To avoid confusion, the one in the first component is denoted as E_1 , and the one in the second as E_2 .

(a) **Adjacency Matrix and List of entire graph G :**

	1	2	B	C	E ₁	E ₂	S	T	U
1	0	1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0	1
C	0	0	0	0	1	0	1	0	0
E ₁	0	0	1	0	0	0	0	1	0
E ₂	0	0	0	1	0	0	0	0	0
S	0	0	0	0	0	1	0	0	0
T	0	0	1	0	0	0	0	0	1
U	0	0	0	0	1	0	0	1	0

Adjacency List:

- 1: [2]
- 2: []
- B: [U]
- C: [E₁, S]
- E₁: [B, T]
- E₂: [C]
- S: [E₂]
- T: [B, U]
- U: [E₁, T]

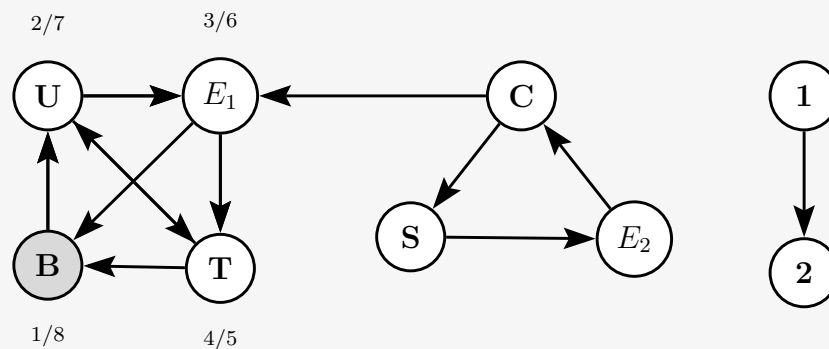
(b) BFS forest of graph G starting from vertex B:

Simulation Queue: $[B] \rightarrow [U] \rightarrow [E_1, T] \rightarrow [T] \rightarrow []$

BFS Order from B: B, U, E₁, T (Other components remain unvisited).

Vertex	B	E ₁	T	U
dist (Level)	0	2	2	1
parent	—	U	U	B

(c) DFS sequence starting from vertex B:



Simulation Steps: Start at $B(1) \xrightarrow{\text{tree}} U(2) \xrightarrow{\text{tree}} E_1(3)$. From E_1 , B is GRAY (Back edge). Next $\xrightarrow{\text{tree}} T(4)$. From T , B and U are GRAY (Back edges). Finish $T(5) \rightarrow$ Finish $E_1(6)$. Back to U , T is BLACK (Forward edge). Finish $U(7) \rightarrow$ Finish $B(8)$.

DFS Order from B: B, U, E_1, T (Other components remain unvisited).

Vertex	B	E_1	T	U
d	1	3	4	2
f	8	6	5	7

(d) **Edge Classification (for the traversed component):**

- **Tree Edges:** $(B, U), (U, E_1), (E_1, T)$
- **Back Edges:** $(E_1, B), (T, B), (T, U)$
- **Forward Edges:** (U, T)
- **Cross Edges:** None

Q13. Write an algorithm for DFS traversal. Analyze time complexity. (8+7=15 marks)
[CSE 203 | 2012–13]

Answer

(Same pseudocode as Q4. See above.)

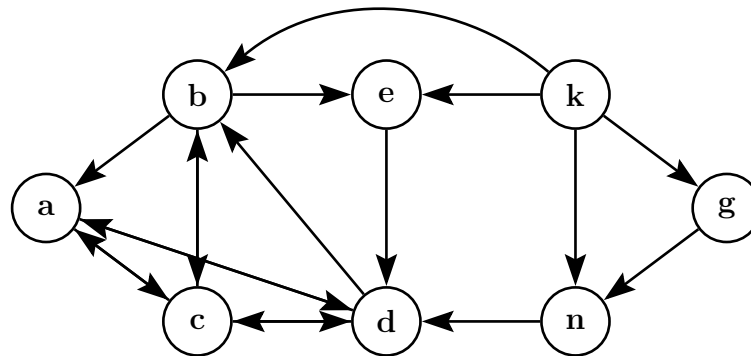
Complexity: $\Theta(V + E)$ with adjacency list; $\Theta(V^2)$ with adjacency matrix. Each vertex is discovered and finished exactly once; each edge is examined once per endpoint.

Q14. Consider the graph G . Perform the following:

- Write the adjacency matrix and adjacency list of the graph G .
- Draw the BFS forest of the graph G assuming b as the starting vertex.
- Write the sequence of vertices if you explore the graph G starting from vertex b in DFS.
- Write the tree edges, back edges, forward edges, and cross edges.

(6+4+4+6=20 marks)

[CSE 203 | 2012–13]



Detailed Answer with Simulation Graphs

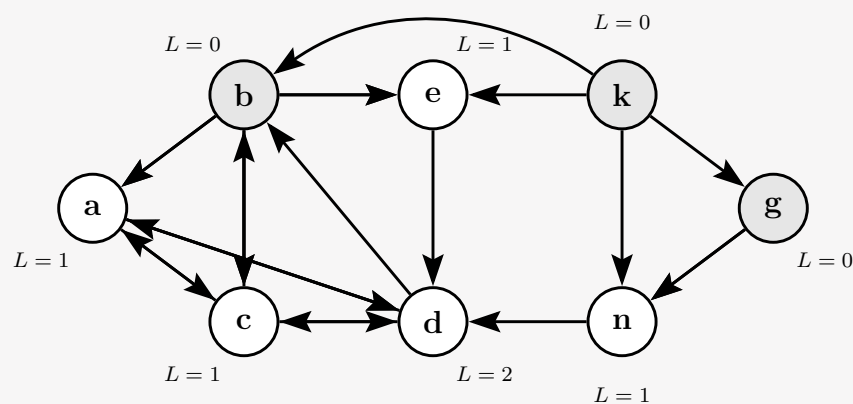
(a) Adjacency matrix and list (vertices: a, b, c, d, e, g, k, n):

	a	b	c	d	e	g	k	n
a	0	0	1	1	0	0	0	0
b	1	0	1	0	1	0	0	0
c	1	1	0	1	0	0	0	0
d	1	1	1	0	0	0	0	0
e	0	0	0	1	0	0	0	0
g	0	0	0	0	0	0	0	1
k	0	1	0	0	1	1	0	1
n	0	0	0	1	0	0	0	0

Adj list:

- a : $[c, d]$
- b : $[a, c, e]$
- c : $[a, b, d]$
- d : $[a, b, c]$
- e : $[d]$
- g : $[n]$
- k : $[b, e, g, n]$
- n : $[d]$

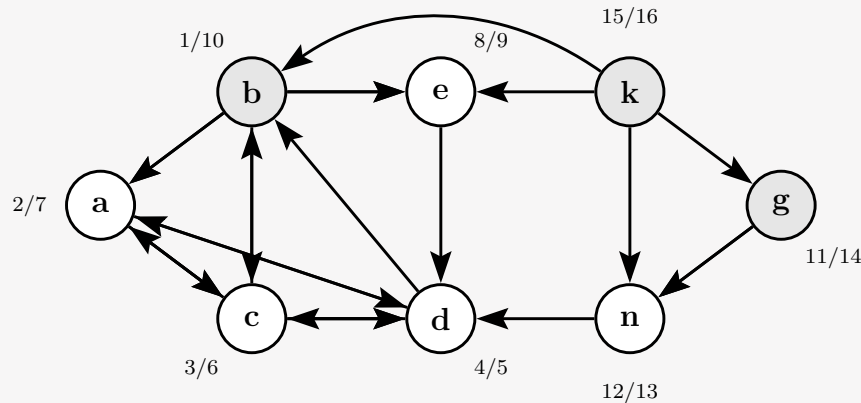
(b) BFS forest of graph G starting from b :



BFS order (Forest): b, a, c, e, d, g, n, k

Vertex	<i>b</i>	<i>a</i>	<i>c</i>	<i>e</i>	<i>d</i>	<i>g</i>	<i>n</i>	<i>k</i>
dist (<i>L</i>)	0	1	1	1	2	0	1	0
parent	—	<i>b</i>	<i>b</i>	<i>b</i>	<i>a</i>	—	<i>g</i>	—

(c) DFS sequence starting from *b*:



Simulation Traversal: Start at $b \rightarrow a \rightarrow c \rightarrow d$. From *d*, all neighbors (*a*, *b*, *c*) are already visited (Back edges). Finish $d \rightarrow c \rightarrow a$. From *b*, visit *e*. From *e*, *d* is visited (Cross edge). Finish $e \rightarrow b$. Next unvisited is $g \rightarrow n$. From *n*, *d* is visited (Cross edge). Finish $n \rightarrow g$. Finally, visit *k*. All neighbors of *k* are visited (Cross edges). Finish *k*.

DFS order (by discovery): *b*, *a*, *c*, *d*, *e*, *g*, *n*, *k*

Vertex	<i>b</i>	<i>a</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>g</i>	<i>n</i>	<i>k</i>
<i>d</i>	1	2	3	4	8	11	12	15
<i>f</i>	10	7	6	5	9	14	13	16

(d) Edge classification (Based on DFS from *b*):

- **Tree:** (*b*, *a*), (*a*, *c*), (*c*, *d*), (*b*, *e*), (*g*, *n*)
- **Back:** (*c*, *a*), (*c*, *b*), (*d*, *a*), (*d*, *b*), (*d*, *c*)
- **Forward:** (*a*, *d*), (*b*, *c*)
- **Cross:** (*e*, *d*), (*n*, *d*), (*k*, *b*), (*k*, *e*), (*k*, *g*), (*k*, *n*)

Q15. Write an algorithm for DFS traversal of a given graph. Analyze the time complexity of the algorithm. (10+5=15 marks) [CSE 203 | 2011–12]

Answer**Algorithm 79** DFS(G)

```

1: for each  $u \in V[G]$  do
2:    $color[u] \leftarrow \text{WHITE}; \text{ parent}[u] \leftarrow \text{nil}$ 
3: end for
4:  $time \leftarrow 0$ 
5: for each  $u \in V[G]$  do
6:   if  $color[u] = \text{WHITE}$  then
7:     DFS-VISIT( $G, u$ )
8:   end if
9: end for

```

Algorithm 80 DFS-VISIT(G, u)

```

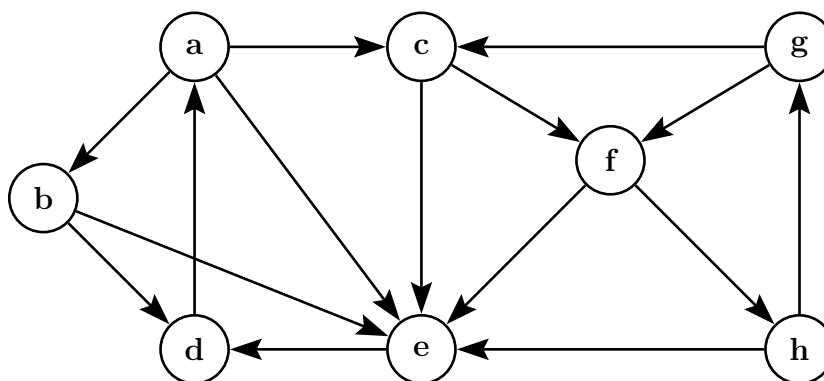
1:  $color[u] \leftarrow \text{GRAY}; \text{ time} \leftarrow \text{time} + 1; \text{ d}[u] \leftarrow \text{time}$ 
2: for each  $v \in \text{Adj}[u]$  do
3:   if  $color[v] = \text{WHITE}$  then
4:      $\text{parent}[v] \leftarrow u$ 
5:     DFS-VISIT( $G, v$ )
6:   end if
7: end for
8:  $color[u] \leftarrow \text{BLACK}; \text{ time} \leftarrow \text{time} + 1; \text{ f}[u] \leftarrow \text{time}$ 

```

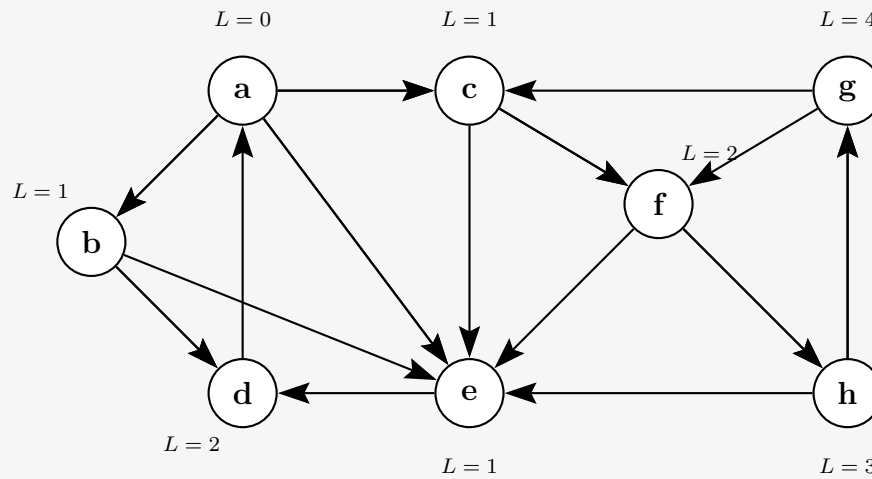
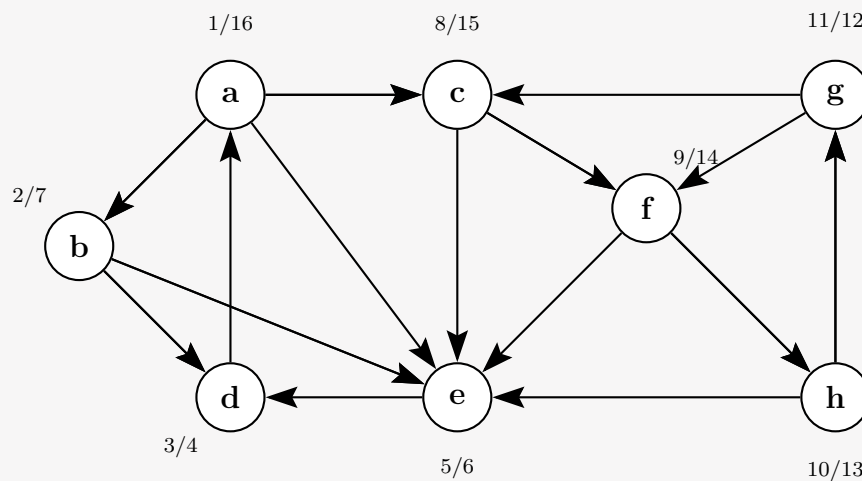
Time Complexity:

- Initialization loop in DFS: $\Theta(V)$.
- DFS-VISIT(u) is called exactly once per vertex (only when WHITE).
- Total work for adjacency-list scanning across all calls: $\sum_u |\text{Adj}[u]| = |E|$.
- **Total:** $\Theta(V + E)$ with adjacency list; $\Theta(V^2)$ with adjacency matrix.

Q16. Write the sequences of the vertices if you explore the given graph G using BFS and DFS. Classify the edges of G for your DFS traversal. (4+6=10 marks) [CSE 203 | 2011–12]



Detailed Answer with Simulation Graphs

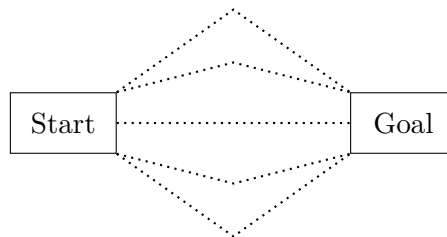
(a) BFS Traversal starting from a (Alphabetical Order):**BFS Order:** a, b, c, e, d, f, h, g **Distances:** $a : 0, b, c, e : 1, d, f : 2, h : 3, g : 4$ (b) DFS Traversal starting from a (Alphabetical Order):**DFS Order:** a, b, d, e, c, f, h, g

Vertex	a	b	c	d	e	f	g	h
d	1	2	8	3	5	9	11	10
f	16	7	15	4	6	14	12	13

Edge classification:

- **Tree:** $(a, b), (b, d), (b, e), (a, c), (c, f), (f, h), (h, g)$
- **Back:** $(d, a), (g, c), (g, f)$
- **Forward:** (a, e)
- **Cross:** $(e, d), (c, e), (f, e), (h, e)$

Q17. Suppose you have a graph like the one below. The dots signify some part of the graph that you don't know exactly, not a straight link. You know however, that there are many paths from the start to the goal. You also know that they tend to be rather long. Suppose that you want to implement a program that searches for a path and returns the first one it can find. You have no need for finding the optimal path in any sense, just any path will do. Would you want to use BFS or DFS as the basis for your program? Justify your answer.



(10 marks)

Answer

Use DFS.

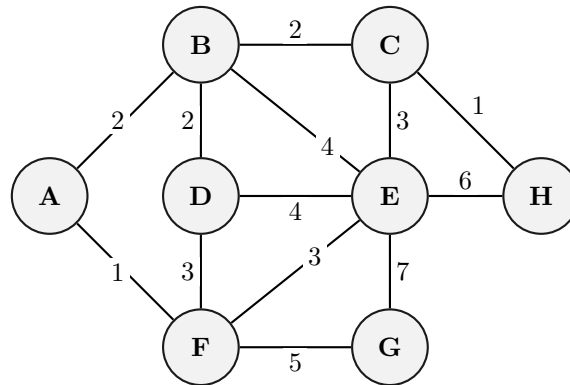
Justification:

The graph has *many long paths* from Start to Goal. In this setting:

- **BFS** explores nodes *level by level*. Since the paths are long, BFS would need to store *all* nodes at each level in its queue before advancing. With many paths, the queue grows very large, consuming significant memory. BFS is ideal when the goal is *close* to the start, not far away.
- **DFS** dives deep along *one path at a time*. Since we only need *any* path (not the shortest or optimal), DFS will quickly reach the Goal by following one of the many long paths without storing all alternatives. Memory usage is proportional to the *depth* of the path ($O(d)$ stack space), which is far more efficient here.

Conclusion: Since the goal is far, paths are long, and any path will do, **DFS** is the better choice — it finds a solution faster and uses much less memory than BFS in this scenario.

Q18. Consider the weighted graph G as shown in Figure 1. Then draw the adjacency lists of the weighted graph G . Whenever there is a choice of vertices to explore, always pick the one that is alphabetically first.

**Answer**

Adjacency List of weighted graph G (neighbours listed alphabetically with edge weights):

Vertex	Adjacency List (neighbour, weight)
A	→ B(2) → F(1)
B	→ A(2) → C(2) → D(2) → E(4)
C	→ B(2) → E(3) → H(1)
D	→ B(2) → E(4) → F(3)
E	→ B(4) → C(3) → D(4) → F(3) → G(7) → H(6)
F	→ A(1) → D(3) → E(3) → G(5)
G	→ E(7) → F(5)
H	→ C(1) → E(6)

Each entry vertex → $u(w)$ means there is an edge between the vertex and u with weight w . Since the graph is undirected, each edge appears in both endpoints' lists.

BUET · CSE 105

DSA

Applications of DFS & BFS

Topological Sort, SCC, Shortest Paths, Dijkstra

T8a — Applications of DFS & BFS

- Q1.** Explain why the topological sort algorithm cannot run on a directed graph that contains cycle(s). Derive the component graph of the directed graph G represented by the given adjacency matrix. Then find the topological sorting of the vertices of the component graph. Show every step. (15 marks) [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Answer

1. Why topological sort fails on a cyclic directed graph:

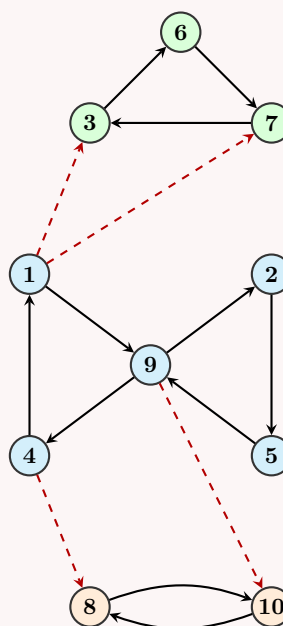
Topological sort requires a *linear ordering* of vertices such that for every directed edge (u, v) , vertex u appears before v . If the graph contains a cycle (e.g., $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$), we would need v_1 to appear before v_k and simultaneously v_k before v_1 , which is a logical contradiction. Thus, a valid linear ordering is impossible.

2. Adjacency List and Original Graph (G):

From the given adjacency matrix, we can derive the adjacency list and visualize the graph cleanly.

Adjacency List:

- $1 \rightarrow 3, 7, 9$
- $2 \rightarrow 5$
- $3 \rightarrow 6$
- $4 \rightarrow 1, 8$
- $5 \rightarrow 9$
- $6 \rightarrow 7$
- $7 \rightarrow 3$
- $8 \rightarrow 10$
- $9 \rightarrow 2, 4, 10$
- $10 \rightarrow 8$



3. Finding Strongly Connected Components (Kosaraju's Algorithm):

Step 1: DFS on G to compute finish times.

Starting DFS from vertex 1, we visit recursively and record the finish times:

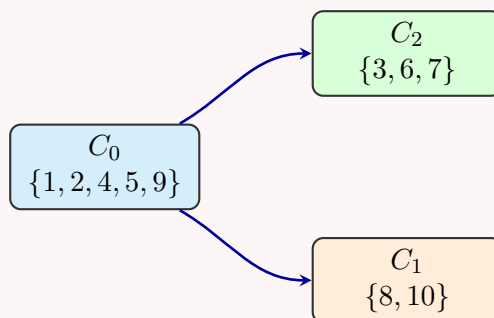
- Exploring from 1: $1 \rightarrow 3 \rightarrow 6 \rightarrow 7$ (finish 7,6,3), then $1 \rightarrow 9 \rightarrow 2 \rightarrow 5$ (finish 5,2), then $9 \rightarrow 4 \rightarrow 8 \rightarrow 10$ (finish 10,8,4), then finish 9, 1.
- Finish order (first to last):** 7, 6, 3, 5, 2, 10, 8, 4, 9, 1.
- Reverse finish order:** 1, 9, 4, 8, 10, 2, 5, 3, 6, 7.

Step 2: DFS on the Transpose Graph G^T in reverse finish order.

- Start at **1**: Visits $\{1, 4, 9, 5, 2\}$. This forms **SCC** C_0 .
- Next unvisited is **8**: Visits $\{8, 10\}$. This forms **SCC** C_1 .
- Next unvisited is **3**: Visits $\{3, 7, 6\}$. This forms **SCC** C_2 .

4. Component Graph (G^{SCC}):

By collapsing each SCC into a single node, we get the directed acyclic graph (DAG) G^{SCC} . Based on the original graph's cross-edges, C_0 has outgoing edges to both C_1 and C_2 .



5. Topological Sorting of G^{SCC} :

To find the topological sort, we compute the in-degrees of each node in G^{SCC} :

- $\text{Indegree}(C_0) = 0$. (Process first, remove its outgoing edges).
- $\text{Indegree}(C_1)$ becomes 0.
- $\text{Indegree}(C_2)$ becomes 0.

Since C_1 and C_2 both have an in-degree of 0 after C_0 is processed, they can be placed in any order.

Final Topological Order:

$$\boxed{C_0 \rightarrow C_1 \rightarrow C_2} \quad \text{or} \quad \boxed{C_0 \rightarrow C_2 \rightarrow C_1}$$

Q2. How do you determine whether a given graph is bipartite? **(10 marks)** [CSE 203 | 2019–20]

Answer

A graph is **bipartite** iff it contains no odd-length cycle, equivalently iff its vertices can be 2-colored such that no two adjacent vertices share a color.

Algorithm (BFS 2-coloring):

Algorithm 81 ISBIPARTITE(G)

```

1: color[ $v$ ]  $\leftarrow -1$  for all  $v$ 
2: for each uncolored vertex  $s$  do
3:   color[ $s$ ]  $\leftarrow 0$ ; enqueue  $s$ 
4:   while queue not empty do
5:      $u \leftarrow$  dequeue
6:     for each neighbor  $v$  of  $u$  do
7:       if color[ $v$ ] =  $-1$  then
8:         color[ $v$ ]  $\leftarrow 1 - \text{color}[u]$ ; enqueue  $v$ 
9:       else if color[ $v$ ] = color[ $u$ ] then
10:        return false                                 $\triangleright$  odd cycle found
11:      end if
12:    end for
13:  end while
14: end for
15: return true

```

Example: a 4-cycle $1-2-3-4-1$ is bipartite (color $\{1,3\}$ red and $\{2,4\}$ blue). A 3-cycle $1-2-3-1$ is **not** bipartite.

Complexity Analysis. $O(V + E)$.

Q3. Write a function to find the lexicographically smallest topological ordering of a directed graph with N vertices and M edges that may contain cycles. Print -1 if the graph has cycles. (8 marks) [CSE 203 | 2018–19]

Answer

Use **Kahn's algorithm** with a *min-heap* (priority queue) instead of a plain queue.

```

1  vector<int> lexTopoSort(int N, vector<pair<int,int>>& edges) {
2      vector<vector<int>> adj(N+1);
3      vector<int> indeg(N+1, 0);
4      for (auto [u,v] : edges) {
5          adj[u].push_back(v);
6          indeg[v]++;
7      }
8      priority_queue<int, vector<int>, greater<int>> pq; // min-heap
9      for (int i = 1; i <= N; i++)
10         if (indeg[i] == 0) pq.push(i);

```

```

11
12     vector<int> result;
13     while (!pq.empty()) {
14         int u = pq.top(); pq.pop();
15         result.push_back(u);
16         for (int v : adj[u]) {
17             if (--indeg[v] == 0) pq.push(v);
18         }
19     }
20     if ((int)result.size() < N) return {-1}; // cycle detected
21     return result;
22 }

```

Complexity Analysis. $O((V + E) \log V)$ due to the min-heap operations.

Q4. The following is Dijkstra's algorithm. Perform a line-by-line time-complexity analysis for an input graph G where an adjacency list is used for graph representation and an ordinary array is used for storing $d[]$ values.

```

1:  $Q \leftarrow V[G]$ 
2: for each  $v \in Q$  do
3:    $d[v] \leftarrow \infty$ 
4: end for
5:  $d[s] \leftarrow 0$ ;  $S \leftarrow \emptyset$ 
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACTMIN}(Q)$ 
8:    $S \leftarrow S \cup \{u\}$ 
9:   for each  $v \in u.\text{Adj}[]$  do
10:    if  $v \in Q$  and  $d[v] > d[u] + w(u, v)$  then
11:       $d[v] \leftarrow d[u] + w(u, v)$ 
12:    end if
13:  end for
14: end while

```

(15 marks)

[CSE 207 | 2018–19]

Answer

Using an **ordinary array** for $d[]$ (no heap).

Line	Code	Cost per call / times called
1	$Q \leftarrow V[G]$	$O(V)$ once
2	init $d[v] \leftarrow \infty$	$O(V)$ once
3	$d[s] \leftarrow 0$	$O(1)$ once
4	while $Q \neq \emptyset$	V iterations
5	EXTRACTMIN(Q) (scan array)	$O(V)$ per iter $\rightarrow O(V^2)$ total
6	$S \leftarrow S \cup \{u\}$	$O(1)$ per iter
7	for $v \in u.\text{Adj}$	sum over all $u = O(E)$ total
8–9	relax + update $d[v]$	$O(1)$ per edge

Total: $O(V^2 + E) = O(V^2)$ (since $E = O(V^2)$).

Remark. For *dense* graphs ($E = \Theta(V^2)$) this plain-array implementation is asymptotically optimal. For *sparse* graphs ($E = O(V)$), a binary heap gives $O(V \log V)$.

Q5. Johnson’s Algorithm uses Bellman-Ford and Dijkstra’s algorithms to compute All-Pairs-Shortest-Paths efficiently on sparse graphs. Explain the basic clever idea of Johnson’s Algorithm. **(15 marks)** [CSE 207 | 2018–19]

Answer

Problem: All-Pairs Shortest Paths (APSP) on a sparse graph that may have negative-weight edges (but no negative cycles). Dijkstra is fast ($O((V + E) \log V)$) but requires non-negative weights.

Johnson’s clever idea — reweighting:

- Add a new vertex q with zero-weight edges to all other vertices: $w(q, v) = 0$ for all $v \in V$.
- Run **Bellman-Ford** from $q \rightarrow$ get $h[v] = \delta(q, v)$ for all v . Time: $O(VE)$.
- Reweight** each edge: $\hat{w}(u, v) = w(u, v) + h[u] - h[v] \geq 0$. (Non-negativity follows from the triangle inequality $h[v] \leq h[u] + w(u, v)$.)
- Run **Dijkstra** from every vertex using $\hat{w}$. Time: $O(V(V + E) \log V)$.
- Recover true distances: $\delta(u, v) = \hat{\delta}(u, v) - h[u] + h[v]$.

Total: $O(VE + V(V + E) \log V)$ — faster than Floyd-Warshall’s $O(V^3)$ for sparse graphs.

Q6. Briefly explain how a directed acyclic graph $G = (V, E)$ can be topologically sorted so that if (u, v) is an edge then u appears before v in the ordering. Pseudocode is not needed. **(8 marks)** [CSE 203 | 2017–18]

Answer**Idea (DFS-based topological sort):**

Run DFS on G . When a vertex v is *finished* (all its descendants fully explored), prepend v to an output list. The resulting list is a valid topological ordering because: for any edge (u, v) , v finishes before u finishes (since v is a descendant of u in the DFS tree, or u is already finished when v is discovered via a cross/forward edge — either way $f[u] > f[v]$).

Kahn’s algorithm (BFS-based): Repeatedly remove vertices with in-degree 0, appending them to the output and decrementing neighbours’ in-degrees. If the output contains fewer than $|V|$ vertices, a cycle exists.

Both approaches run in $\Theta(V + E)$.

Q7. Suppose you are developing a feature for Facebook. You have access to everyone’s friend list and the goal is to show how any two users are connected through a chain of “a friend of a friend”. For two users A and B , show a sequence of individuals $X_0, X_1, \dots, X_n$ where $X_0 = A$, $X_n = B$, and X_i is friends with X_{i+1} , such that the length of the sequence is minimized. Explain how this can be formulated as a graph problem and what algorithm you would use. (7 marks) [CSE 203 | 2017–18]

Answer**Graph formulation:**

- **Vertices:** one per Facebook user.
- **Edges:** undirected edge (X_i, X_{i+1}) if X_i and X_{i+1} are friends.
- The problem reduces to finding the **shortest path** from vertex A to vertex B in this unweighted, undirected graph.

Algorithm: BFS from source A .

BFS explores vertices level by level, guaranteeing the shortest (minimum-hop) path. When B is first dequeued, trace back via the BFS predecessor array to recover the chain $X_0 = A, X_1, \dots, X_n = B$.

Time: $O(V + E)$ where V = number of users, E = number of friendship pairs.

Q8. How can BFS be adapted to solve the single-source shortest path problem in an edge-weighted graph with unequal weights? Explain. (8 marks) [CSE 207 | 2017–18]

Answer

Standard BFS finds shortest paths in *unweighted* graphs. For edge-weighted graphs with **unequal positive integer weights**, BFS can be adapted by **edge splitting**: Replace each edge (u, v) of weight w with a path $u \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_{w-1} \rightarrow v$ using $w - 1$ dummy vertices and w unit-weight edges. Now all edges have weight 1; run standard BFS.

Complexity Analysis. If $W = \max$ edge weight and E edges, the expanded graph has $O(E \cdot W)$ edges $\rightarrow$ BFS runs in $O(V + E \cdot W)$. For small W this is practical; for large weights, use Dijkstra instead.

Alternatively: use a **modified BFS with a deque (0-1 BFS)** for $\{0, 1\}$ -weight graphs, or simply use **Dijkstra** for general non-negative integer weights in $O((V + E) \log V)$.

Q9. Explain the intuitive idea behind the Bellman-Ford algorithm. How can you detect a negative cycle using Bellman-Ford? **(6+2 marks)** [CSE 207 | 2017–18]

Answer

Intuitive idea. Any shortest path without negative cycles is a *simple* path with at most $|V| - 1$ edges. Bellman-Ford iteratively *relaxes* all edges $|V| - 1$ times: in pass i , it correctly computes shortest paths using at most i edges. After $|V| - 1$ passes, all shortest paths (if they exist) are found.

Negative cycle detection. After $|V| - 1$ passes, run one more relaxation pass over all edges. If any distance $d[v]$ decreases, a negative-weight cycle is reachable from the source — because a simple path cannot have more than $|V| - 1$ edges; any further improvement must involve repeating a vertex, i.e. traversing a negative cycle.

Q10. Write Dijkstra's algorithm for the single-source shortest path problem. Prove its correctness. Analyze the time complexity of the algorithm. What would be the running time if a Fibonacci heap is used instead of a binary heap? **(5+5+7+5 marks)** [CSE 207 | 2017–18]

Answer

Algorithm 82 DIJKSTRA(G, w, s)

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  for  $v \neq s$ ;  $S \leftarrow \emptyset$ 
2: Insert all vertices into min-priority queue  $Q$  keyed by  $d$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ ;  $S \leftarrow S \cup \{u\}$ 
5:   for each  $(u, v) \in E$  do
6:     if  $d[v] > d[u] + w(u, v)$  then
7:        $d[v] \leftarrow d[u] + w(u, v)$ ;  $\text{DECREASE-KEY}(Q, v, d[v])$ 
8:     end if
9:   end for
10: end while

```

Correctness: proved in [CSE 207 | 2014–15] answer above.

Complexity Analysis. With a **binary heap**: $|V|$ Extract-Min at $O(\log V)$ each, $|E|$ Decrease-Key at $O(\log V)$ each. Total: $O((V + E) \log V)$.
 With a **Fibonacci heap**: Extract-Min: $O(\log V)$ amortised; Decrease-Key: $O(1)$ amortised. Total: $O(V \log V + E)$ — better for dense graphs.

Q11. Design a linear-time algorithm for each of the following. Briefly describe the ideas (pseudocode is not needed):

- (a) Given an undirected graph G and a particular edge (u, v) in it, determine whether G has a cycle containing (u, v) .
- (b) Given an undirected and unweighted graph G , a particular edge (u, v) , and two vertices s and t , determine whether there is a shortest path from s to t containing the edge (u, v) .

(7+8 marks)

[CSE 203 | 2016–17]

Answer

(a) Does G have a cycle containing edge (u, v) ?

Idea: A cycle containing (u, v) exists iff there is a path from v back to u in $G \setminus \{(u, v)\}$.

Algorithm:

- (a) Temporarily remove edge (u, v) from G .
- (b) Run BFS/DFS from v .
- (c) If u is reachable from $v \rightarrow$ **Yes**, a cycle exists. Otherwise $\rightarrow$ **No**.
- (d) Restore (u, v) .

Time: $O(V + E)$ (single BFS/DFS).

(b) Is there a shortest path from s to t using edge (u, v) ?

Idea: A shortest s - t path uses (u, v) iff

$$d(s, u) + 1 + d(v, t) = d(s, t),$$

where $d(a, b)$ denotes the unweighted shortest-path distance.

Algorithm:

- (a) BFS from s on $G \rightarrow$ compute $d(s, \cdot)$ for all vertices. $O(V + E)$.
- (b) BFS from t on G^T (transpose) $\rightarrow$ compute $d(\cdot, t)$ for all vertices. $O(V + E)$.
- (c) Check: $d(s, u) + 1 + d(v, t) = d(s, t)$?

Time: $O(V + E)$ total.

Q12. Write the properties satisfied by shortest paths of a graph. Show by example that Di-

Dijkstra's algorithm gives wrong results for a graph with negative-weight edges. Write an algorithm that finds shortest paths in a DAG and analyze its time complexity. (15 marks) [CSE 207 | 2016–17]

Answer

Properties of shortest paths:

- (a) **Optimal substructure:** any sub-path of a shortest path is itself shortest.
- (b) **No negative cycles:** shortest paths are undefined if negative cycles are reachable.
- (c) **Triangle inequality:** $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- (d) **Upper-bound property:** $d[v] \geq \delta(s, v)$ always; equality holds once v is finalized.

Dijkstra fails on negative edges — example: Graph: $s \rightarrow a$ (weight 3), $s \rightarrow b$ (weight 4), $b \rightarrow a$ (weight -2). True $\delta(s, a) = \min(3, 4 - 2) = 2$. Dijkstra finalizes a first with $d[a] = 3$ (greedy), never revisiting it to find the shorter path $s \rightarrow b \rightarrow a = 2$.

DAG Shortest Path algorithm:

Algorithm 83 DAG-SHORTEST-PATH(G, w, s)

```

1: Topologically sort  $G$ 
2:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  for  $v \neq s$ 
3: for each vertex  $u$  in topological order do
4:   for each  $(u, v) \in E$  do
5:     if  $d[u] + w(u, v) < d[v]$  then
6:        $d[v] \leftarrow d[u] + w(u, v)$ 
7:     end if
8:   end for
9: end for
```

Complexity Analysis. $\Theta(V + E)$ — works even with negative edges since there are no cycles.

Q13. Prove that the Bellman-Ford algorithm returns TRUE if the input graph contains no negative-weight cycles reachable from the source, and FALSE otherwise. (10 marks) [CSE 207 | 2016–17]

Answer

Proof.

($\Rightarrow$) **No negative cycle $\Rightarrow$ returns TRUE.**

If no negative cycle is reachable from s , all shortest paths are simple (at most

$|V| - 1$ edges). After $|V| - 1$ relaxation passes, by the path-relaxation property, $d[v] = \delta(s, v)$ for all reachable v . For the final check: for any edge (u, v) , $d[u] + w(u, v) \geq \delta(s, u) + w(u, v) \geq \delta(s, v) = d[v]$. No update occurs $\rightarrow$ return TRUE. $\square$

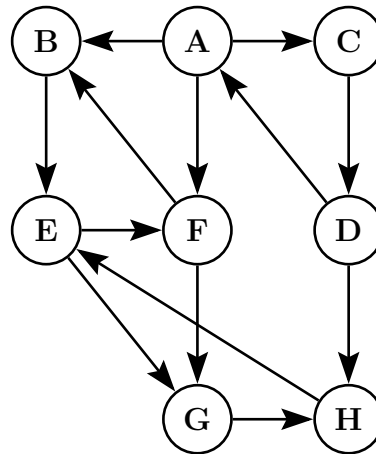
($\Leftarrow$) **Negative cycle reachable $\Rightarrow$ returns FALSE.**

Suppose a negative cycle $C : v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k \rightarrow v_0$ is reachable from s with $\sum w(v_i, v_{i+1}) < 0$. Assume for contradiction the algorithm returns TRUE, meaning $d[v_{i+1}] \leq d[v_i] + w(v_i, v_{i+1})$ for all edges. Summing around the cycle:

$$\sum_{i=0}^{k-1} d[v_{i+1}] \leq \sum_{i=0}^{k-1} d[v_i] + \sum w(v_i, v_{i+1}).$$

The $d[\cdot]$ sums cancel (cycle), leaving $0 \leq \sum w < 0$ — a contradiction. Hence the algorithm must return FALSE. $\square$

- Q14.** Run DFS on a given graph starting from vertex A and show the discovery and finishing times for each vertex as well as the edge types. Decompose the graph into strongly connected components (SCCs). **(15 marks)** [CSE 203 | 2016–17]



Answer

Adjacency list (alphabetical order): $A \rightarrow [B, C, F]$, $B \rightarrow [E]$, $C \rightarrow [D]$, $D \rightarrow [A, H]$, $E \rightarrow [F, G]$, $F \rightarrow [B, G]$, $G \rightarrow [H]$, $H \rightarrow [E]$.

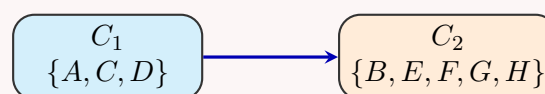
DFS from A (neighbours in alphabetical order):

Vertex	Discovery	Finish
A	1	16
B	2	11
C	12	15
D	13	14
E	3	10
F	4	9
G	5	8
H	6	7

Edge classification:

Edge	Type
(A, B)	Tree
(B, E)	Tree
(E, F)	Tree
(F, B)	Back (cycle: $B \rightarrow E \rightarrow F \rightarrow B$)
(F, G)	Tree
(G, H)	Tree
(H, E)	Back (cycle: $E \rightarrow F \rightarrow G \rightarrow H \rightarrow E$)
(E, G)	Forward
(A, C)	Tree
(C, D)	Tree
(D, A)	Back (cycle: $A \rightarrow C \rightarrow D \rightarrow A$)
(D, H)	Cross
(A, F)	Forward

Strongly Connected Components (Kosaraju's):



SCC	Vertices
C_1	$\{B, E, F, G, H\}$
C_2	$\{A, C, D\}$

Remark. Back edges directly reveal the cycles that form SCCs: $F \rightarrow B$ closes the cycle $B-E-F$; $H \rightarrow E$ closes $E-F-G-H$; together these five vertices form one SCC. $D \rightarrow A$ closes $A-C-D$.

Q15. Suppose a CS curriculum consists of n mandatory courses. The prerequisite directed graph G has a node for each course and an edge from v to w if v is a prerequisite for

w . Give a linear-time algorithm that computes the minimum number of semesters necessary to complete the curriculum (a student may take any number of courses per semester). (15 marks) [CSE 207 | 2015–16]

Answer

Formulation: longest path in the prerequisite DAG gives the minimum number of semesters (each level = one semester).

Algorithm 84 MINSEMESTERS(G)

```

1: Topologically sort  $G \rightarrow$  order  $v_1, v_2, \dots, v_n$ 
2:  $\text{sem}[v] \leftarrow 1$  for all  $v$ 
3: for each  $u$  in topological order do
4:   for each neighbor  $v$  of  $u$  do
5:      $\text{sem}[v] \leftarrow \max(\text{sem}[v], \text{sem}[u] + 1)$ 
6:   end for
7: end for
8: return  $\max_v \text{sem}[v]$ 
```

Complexity Analysis. Topological sort: $O(V + E)$. Longest-path DP: $O(V + E)$. Total: $O(V + E)$.

Q16. Let C and C' be distinct strongly connected components in a directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ where $u \in C$ and $v \in C'$. Prove that $f(C) > f(C')$, where $f(U)$ denotes the latest finishing time of any vertex in U . (10 marks) [CSE 207 | 2014–15]

Answer

Theorem. Let C, C' be distinct SCCs with edge (u, v) , $u \in C$, $v \in C'$. Then $f(C) > f(C')$.

Proof. Let $x \in C$ be the first vertex discovered in DFS. Since C is an SCC, all of C is reachable from x via G ; since (u, v) exists and C' is reachable from C , all of C' is also reachable from x .

Case 1: DFS discovers some vertex $y \in C'$ before x . Then x is discovered later with C' already fully explored (since there is no back-edge from C' to C , as that would merge them into one SCC). So $f(C') < d(x) < f(x) \leq f(C)$.

Case 2: x is discovered first. The DFS tree rooted at x reaches all of C and all of C' (via the path through (u, v)). C' has no path back to C (otherwise they'd be the same SCC), so C' finishes before the DFS returns to any vertex of C . Hence $f(C') < f(C)$. $\square$

Q17. State and prove the convergence property of edge relaxation in the context of shortest paths. (8 marks) [CSE 207 | 2014–15]

Q18. Provide a correctness proof of Dijkstra's algorithm. (10 marks)

[CSE 207 | 2014–15]

Answer

Theorem. On graphs with non-negative edge weights, when Dijkstra's algorithm finalizes vertex u (extracts u from Q), $d[u] = \delta(s, u)$.

Proof (by induction on $|S|$, the set of finalized vertices).

Base ($|S| = 0$): s is extracted first with $d[s] = 0 = \delta(s, s)$. ✓

Step: assume $d[v] = \delta(s, v)$ for all $v \in S$. Let u be the next vertex extracted.

Suppose for contradiction $d[u] > \delta(s, u)$. Let p be a true shortest path $s \xrightarrow{*} u$; let (x, y) be the first edge on p with $x \in S, y \notin S$.

Since all edge weights ≥ 0 :

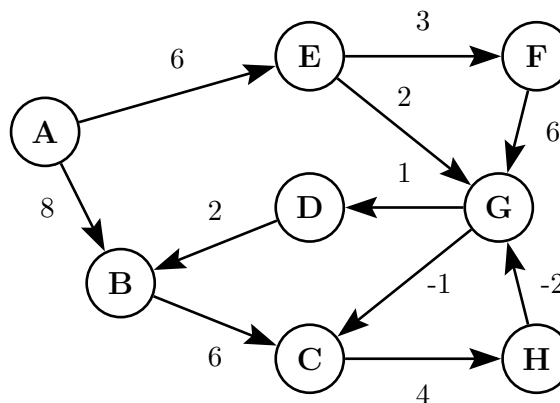
$$d[y] = \delta(s, y) \leq \delta(s, u) < d[u].$$

(The first equality holds by the inductive hypothesis applied when x was finalized and (x, y) was relaxed; the second by non-negativity.)

But $y \notin S$ and $d[y] < d[u]$ — the algorithm would have chosen y before u , contradicting that u was just extracted. □

Q19. Apply the Bellman-Ford algorithm to find the shortest paths. Show all relaxation steps. (10 marks)

[CSE 207 | 2014–15]

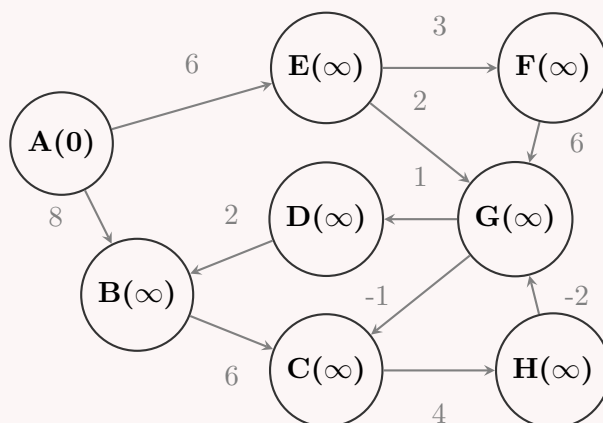


Answer — Bellman-Ford Algorithm Step-by-Step

Edge List & Relaxation Order: $(A \rightarrow E, 6)$, $(A \rightarrow B, 8)$, $(E \rightarrow F, 3)$, $(E \rightarrow G, 2)$, $(F \rightarrow G, 6)$, $(G \rightarrow D, 1)$, $(H \rightarrow G, -2)$, $(G \rightarrow C, -1)$, $(D \rightarrow B, 2)$, $(B \rightarrow C, 6)$, $(C \rightarrow H, 4)$.

Source Vertex = A. Total Vertices $|V| = 8$. Maximum required passes = $|V| - 1 = 7$.

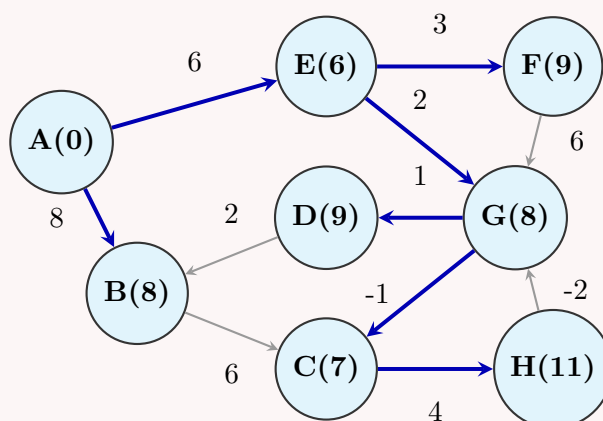
Initialization (Pass 0): Set distance to source $d[A] = 0$, and all other distances to ∞ .



Pass 1: Edge Relaxations (Condition: if $d[u] + w(u, v) < d[v]$, update $d[v]$)

Edge ($u \rightarrow v$)	Wt (w)	Check $d[u] + w < d[v]$	Action	New Distance
$A \rightarrow E$	6	$0 + 6 < \infty$	Update	$d[E] = 6$
$A \rightarrow B$	8	$0 + 8 < \infty$	Update	$d[B] = 8$
$E \rightarrow F$	3	$6 + 3 < \infty$	Update	$d[F] = 9$
$E \rightarrow G$	2	$6 + 2 < \infty$	Update	$d[G] = 8$
$F \rightarrow G$	6	$9 + 6 \not< 8$	No change	-
$G \rightarrow D$	1	$8 + 1 < \infty$	Update	$d[D] = 9$
$H \rightarrow G$	-2	$\infty - 2 \not< 8$	No change	-
$G \rightarrow C$	-1	$8 - 1 < \infty$	Update	$d[C] = 7$
$D \rightarrow B$	2	$9 + 2 \not< 8$	No change	-
$B \rightarrow C$	6	$8 + 6 \not< 7$	No change	-
$C \rightarrow H$	4	$7 + 4 < \infty$	Update	$d[H] = 11$

State after Pass 1: (Highlighted blue edges represent the shortest path tree)



Pass 2: Verification

Iterating through all edges again, no condition $d[u] + w < d[v]$ is met. For example:

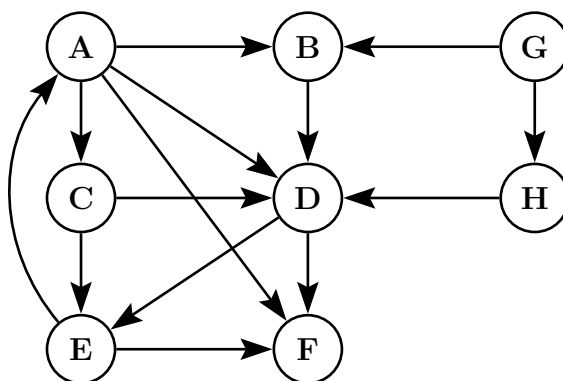
- $H \rightarrow G$: $d[H] - 2 = 11 - 2 = 9 \not\leq 8$ (no change).
- $D \rightarrow B$: $d[D] + 2 = 9 + 2 = 11 \not\leq 8$ (no change).

Since no updates occur in Pass 2, distances are optimal and the algorithm terminates early. This also confirms there are **no negative weight cycles** reachable from A.

Final shortest distances from A:

$d[B] = 8$, $d[C] = 7$, $d[D] = 9$, $d[E] = 6$, $d[F] = 9$, $d[G] = 8$, $d[H] = 11$.

Q20. Find the different types of edges if DFS is applied from vertex A. Discuss the edges' properties with respect to the graph. (6 marks) [CSE 207 | 2014–15]



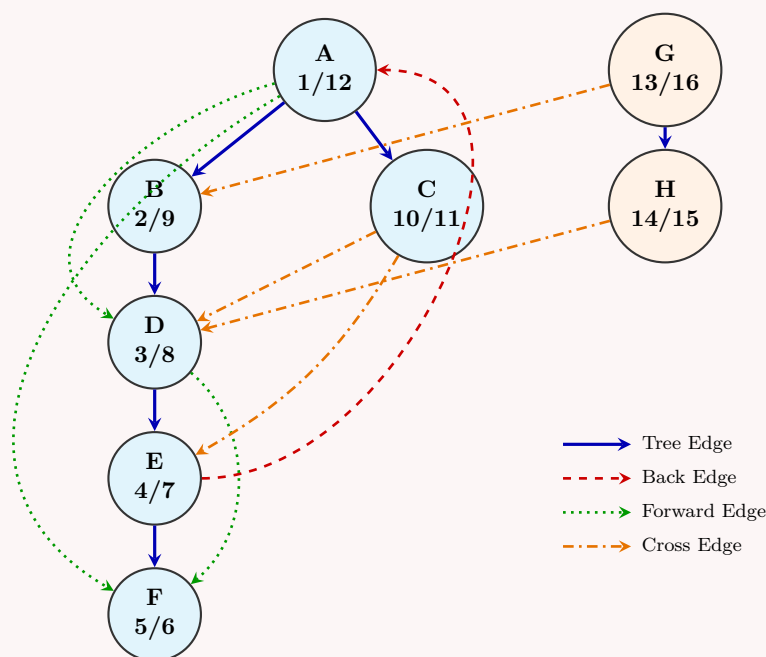
Answer — DFS Edge Classification & Forest

In DFS on a directed graph, every edge (u, v) falls into one of four categories based on discovery/finish times $(d[u]/f[u])$:

- **Tree edge:** v is first discovered via (u, v) .
- **Back edge:** v is an ancestor of u ($d[v] < d[u] < f[u] < f[v]$). Indicates a cycle.
- **Forward edge:** v is a descendant of u , but not a tree edge ($d[u] < d[v] < f[v] < f[u]$).
- **Cross edge:** v is neither ancestor nor descendant ($d[v] < f[v] < d[u] < f[u]$).

1. Depth First Forest (with Timestamps d/f & Edge Types):

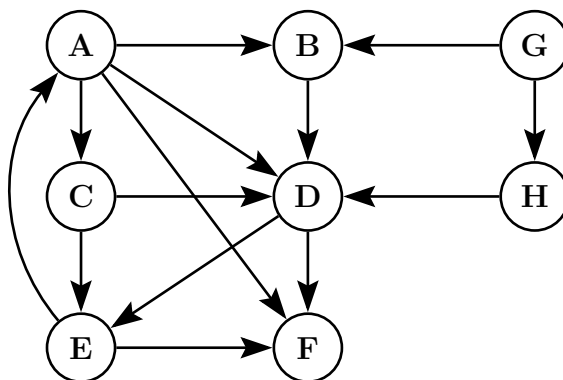
Assuming standard alphabetical priority, the DFS traverses $A \rightarrow B \rightarrow D \rightarrow E \rightarrow F$, then explores remaining neighbors. The graph decomposes into two DFS trees (rooted at A and G).



2. Corrected Edge Classification Table:

Edge	Type	Reason based on timestamps (d/f)
$A \rightarrow B$	Tree	B is first discovered from A.
$A \rightarrow C$	Tree	C is first discovered from A.
$A \rightarrow D$	Forward	D is completely processed ($f[D] = 8 < f[A] = 12$), but is a descendant of A.
$A \rightarrow F$	Forward	F is completely processed ($f[F] = 6 < f[A] = 12$), but is a descendant of A.
$B \rightarrow D$	Tree	D is first discovered from B.
$C \rightarrow D$	Cross	D is processed ($f[D] = 8$) before C is even discovered ($d[C] = 10$).
$C \rightarrow E$	Cross	E is processed ($f[E] = 7$) before C is discovered ($d[C] = 10$).
$D \rightarrow E$	Tree	E is first discovered from D.
$D \rightarrow F$	Forward	F is a descendant of D (visited via E), discovered after D.
$E \rightarrow A$	Back	A is an ancestor of E and is still active ($d[A] = 1 < d[E] = 4$). (Forms Cycle)
$E \rightarrow F$	Tree	F is first discovered from E.
$G \rightarrow B$	Cross	B is entirely in a different tree, finished before G started.
$G \rightarrow H$	Tree	H is first discovered from G.
$H \rightarrow D$	Cross	D finished long before H was discovered.

Q21. Show the topological sorting order of a given graph with explanation of discovery and finishing times of node exploration. (6 marks) [CSE 207 | 2014–15]



Answer — Topological Sorting

Crucial Observation: Topological sorting is strictly defined only for **Directed Acyclic Graphs (DAGs)**. However, this given graph contains a **cycle** (e.g., $A \rightarrow D \rightarrow E \rightarrow A$ via the back edge $E \rightarrow A$). Therefore, a strictly valid topological ordering is mathematically impossible.

If we forcefully apply the standard topological sorting algorithm (performing DFS and sorting nodes by decreasing finish times), we get the following result, which will inevitably violate the topological property for the back edge(s).

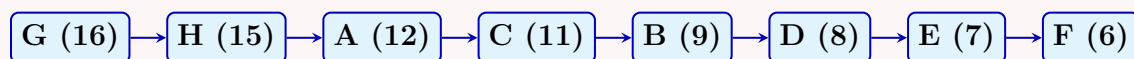
Step 1: DFS Discovery (d) and Finish (f) Times

(Assuming standard alphabetical order for unvisited vertices and neighbors)

Vertex	Discovery (d)	Finish (f)	Vertex	Discovery (d)	Finish (f)
A	1	12	E	4	7
B	2	9	F	5	6
C	10	11	G	13	16
D	3	8	H	14	15

Step 2: Sorting by Decreasing Finish Time

Arranging the vertices in descending order of their finish times ($f[u]$):



Generated Order: $G \rightarrow H \rightarrow A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$

Verification of Failure: In a valid topological sort, for every directed edge $u \rightarrow v$, vertex u must come before v . While this order satisfies most edges (e.g., $G \rightarrow H$, $A \rightarrow C$, $D \rightarrow E$), it **fails** for the edge $E \rightarrow A$, because E appears after A in our sequence. This confirms the sequence is pseudo-topological due to the cycle.

Q22. State and prove the convergence property of shortest paths. (12 marks) [CSE 207 | 2012–13]

Answer

Convergence Property. Let $G = (V, E, w)$, source s , and let $s \xrightarrow{*} u \rightarrow v$ be a shortest path. If $\delta(s, u)$ is the true shortest-path distance from s to u , and $d[u] = \delta(s, u)$ at some point before edge (u, v) is relaxed, then after relaxing (u, v) :

$$d[v] = \delta(s, v).$$

and $d[v]$ will not change thereafter.

Proof.

Since $s \xrightarrow{*} u \rightarrow v$ is a shortest path, $\delta(s, v) = \delta(s, u) + w(u, v)$.

By the upper-bound property, $d[v] \geq \delta(s, v)$ always holds.

After relaxing (u, v) :

$$d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v).$$

Combined with $d[v] \geq \delta(s, v)$, we get $d[v] = \delta(s, v)$.

Future relaxations can only set $d[v] \leftarrow \min(d[v], \dots) \geq \delta(s, v)$, so $d[v]$ cannot decrease below $\delta(s, v)$. $\square$

Q23. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. (20 marks) [CSE 207 | 2012–13]

Answer**Algorithm 85** BELLMAN-FORD(G, w, s)

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  for all  $v \neq s$ 
2:  $\pi[v] \leftarrow \text{NIL}$  for all  $v$ 
3: for  $i \leftarrow 1$  to  $|V| - 1$  do
4:   for each edge  $(u, v) \in E$  do
5:     if  $d[u] + w(u, v) < d[v]$  then
6:        $d[v] \leftarrow d[u] + w(u, v)$ ;  $\pi[v] \leftarrow u$ 
7:     end if
8:   end for
9: end for
10: for each edge  $(u, v) \in E$  do ▷ check for negative cycles
11:   if  $d[u] + w(u, v) < d[v]$  then
12:     return false
13:   end if
14: end for
15: return true

```

Complexity Analysis. $|V| - 1$ passes, each scanning all $|E|$ edges: $\Theta(VE)$.

Correctness sketch. *Loop invariant:* after i passes, $d[v] = \delta(s, v)$ for all v reachable via a shortest path with $\leq i$ edges. After $|V| - 1$ passes (longest possible

simple path), all reachable $d[v] = \delta(s, v)$ if no negative cycle. If a negative cycle is reachable, some $d[v]$ can still be reduced in the final check $\rightarrow$ return FALSE. $\square$

Q24. Does Dijkstra’s algorithm terminate if it is applied on a graph G that contains a negative-weight cycle? Why or why not? Justify your answer. (6 marks) [CSE 207 | 2012–13]

Answer

Yes, Dijkstra terminates, but it may give **wrong answers**.

Dijkstra’s algorithm extracts each vertex from the priority queue *exactly once* (once extracted, a vertex is finalized and never re-inserted in the standard version). Since there are only $|V|$ vertices, the algorithm always terminates in $O((V + E) \log V)$ steps regardless of edge weights.

However, negative-weight edges (or cycles) violate the *greedy choice property*: once a vertex is finalized with estimate $d[u]$, the algorithm assumes no shorter path exists — but a negative edge could provide one. Therefore, the distances computed may be **incorrect**.

Remark. A negative-weight *cycle* makes shortest paths undefined ($-\infty$); both Dijkstra and Bellman-Ford cannot give meaningful answers in that case. Bellman-Ford at least *detects* such cycles; Dijkstra does not.

Q25. State the parenthesis theorem for depth-first search. (6 marks) [CSE 207 | 2012–13]

Answer

Parenthesis Theorem. In a DFS of graph $G = (V, E)$, for any two vertices u and v , exactly one of the following holds:

- (a) $[d[u], f[u]]$ and $[d[v], f[v]]$ are *disjoint* (neither is an ancestor of the other in the DFS forest).
- (b) $[d[u], f[u]] \subset [d[v], f[v]]$ (u is a descendant of v).
- (c) $[d[v], f[v]] \subset [d[u], f[u]]$ (v is a descendant of u).

Proof. WLOG assume $d[u] < d[v]$ (discovered first).

Case A: $d[v] > f[u]$. v discovered after u finished $\rightarrow$ intervals disjoint. $\checkmark$

Case B: $d[v] < f[u]$ (v discovered while u is gray). Then v is a descendant of u . v must finish before u (DFS finishes a subtree before backtracking), so $f[v] < f[u]$, giving $[d[v], f[v]] \subset [d[u], f[u]]$. $\checkmark$

Overlapping without nesting is impossible: if $d[u] < d[v] < f[u] < f[v]$, then v is discovered while u is gray (so v is u ’s descendant), but then v must finish before u — contradiction. $\square$

Q26. Assume that a breadth-first tree has already been computed with BFS on a graph

G with source vertex s . Write an algorithm to print the vertices on a shortest path from s to v . (8 marks) [CSE 207 | 2012–13]

Answer

```

1  // Assumes BFS has been run; parent[] stores BFS predecessor
2  void printPath(int parent[], int s, int v) {
3      if (v == s) {
4          cout << s;
5          return;
6      }
7      if (parent[v] == -1) {
8          cout << "No path from " << s << " to " << v;
9          return;
10     }
11     printPath(parent, s, parent[v]);
12     cout << " -> " << v;
13 }

```

The recursion follows predecessor pointers until the source is reached, then prints vertices on the way back. Time: $O(V)$ (path length $\leq V - 1$).

Q27. Prove that the component graph of a directed graph is a DAG. (6 marks) [CSE 207 | 2012–13]

Answer

Claim: G^{SCC} is a DAG.

Proof (by contradiction). Suppose G^{SCC} contains a cycle $C_{i_1} \rightarrow C_{i_2} \rightarrow \dots \rightarrow C_{i_k} \rightarrow C_{i_1}$. Then there exist directed paths in G : from every vertex in C_{i_1} to every vertex in C_{i_2} , ..., and back to C_{i_1} . This means all vertices in $C_{i_1} \cup \dots \cup C_{i_k}$ are mutually reachable — so they all belong to the *same* SCC, contradicting the assumption that $C_{i_1}, \dots, C_{i_k}$ are *distinct* SCCs. $\square$

Q28. Write a linear-time algorithm to compute the strongly connected components of a directed graph $G = (V, E)$ using two depth-first searches. Give an example to show every step of the algorithm. (8+5 marks) [CSE 207 | 2012–13]

Answer

Algorithm 86 STRONGLY-CONNECTED-COMPONENTS(G)

- 1: Run DFS on G ; record finish times $f[v]$ for each v
- 2: Compute G^T (transpose of G)
- 3: Run DFS on G^T , processing vertices in *decreasing* $f[v]$ order
- 4: Each DFS tree in step 3 is one SCC

Complexity Analysis. $\Theta(V + E)$.

Example (graph $q, r, s, t, u, v, w, x, y, z$ — alphabetical start):

Pass 1 — DFS on G :

Vertex	d	f
q	1	16
s	2	7
v	3	6
w	4	5
t	8	15
x	9	12
z	10	11
y	13	14
r	17	20
u	18	19

Finish stack (last finished = top): $w, v, s, z, x, y, t, q, u, r$.

Pass 2 — DFS on G^T (reverse finish order):

SCC	Vertices
1	$\{r\}$
2	$\{u\}$
3	$\{q\}$
4	$\{t\}$
5	$\{y\}$
6	$\{x, z\}$
7	$\{s, v, w\}$

- Q29.** What does $d[v]$ represent at the end of the execution of Algorithm 1, if the algorithm is applied on an unweighted directed acyclic graph (DAG) G with a source s ? What is the runtime complexity in terms of $|V|$ and $|E|$ of Algorithm 1? **(6+6 marks)**
[CSE 207 | 2012–13]

Algorithm 87 LONGESTPATH-DAG(G)

```

1: Compute a topological sorting of  $G$ 
2: for each  $v \in V[G]$  do
3:    $d[v] \leftarrow 0$ 
4: end for
5: for each  $u$  in topologically sorted order do
6:   for each  $v \in Adj[u]$  do
7:     if  $d[v] < d[u] + 1$  then
8:        $d[v] \leftarrow d[u] + 1$ 
9:     end if
10:  end for
11: end for

```

Answer**What does $d[v]$ represent?**

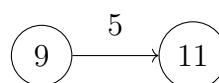
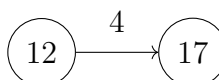
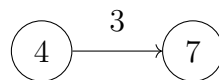
The algorithm computes the **length of the longest path** from source s to v in the unweighted DAG (counting edges). Equivalently, $d[v]$ = the maximum number of edges on any path from s to v .

Runtime:

- Topological sort: $O(V + E)$.
- Initialisation: $O(V)$.
- Main loop: each vertex processed once, each edge relaxed once $\rightarrow O(V + E)$.

Total: $O(V + E)$.

Q30. In Figure 1, the shortest path estimate of each vertex appears within the vertex and the distance to traverse from a vertex to another appears as the weight of the corresponding directed edge. Show the updated shortest path estimates of vertices after relaxing the following edges. **(6 marks)**



[CSE 207 | 2012–13]

Answer

Current distance estimates (shown inside each node):

$d[4] = 4$, $d[7] = 7$, $d[12] = 12$, $d[17] = 17$, $d[9] = 9$, $d[11] = 11$.

Relax ($4 \rightarrow 7$, $w = 3$):

$d[4] + 3 = 4 + 3 = 7 \not< d[7] \rightarrow$ **no update**.

Relax ($12 \rightarrow 17$, $w = 4$):

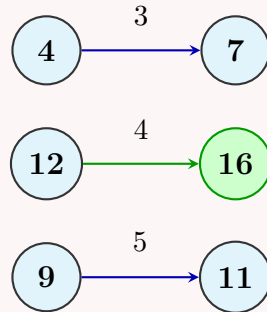
$d[12] + 4 = 12 + 4 = 16 < d[17] \rightarrow$ **update** $d[17] \leftarrow 16$.

Relax ($9 \rightarrow 11$, $w = 5$):

$d[9] + 5 = 9 + 5 = 14 \not< d[11] \rightarrow$ **no update**.

Updated Graph after Relaxation:

(Only the estimate for the middle-right vertex changes from 17 to 16)



Q31. Prove the correctness of the BFS algorithm. What are “discovery time” and “finish time” in DFS? How can you find the shortest path between two nodes from a BFS tree? (8+4+6 marks) [CSE 207 | 2009–10]

Answer

BFS Correctness.

Claim: when BFS terminates, $d[v] = \delta(s, v)$ for all v .

Proof sketch (by induction on $\delta(s, v)$).

Base: $d[s] = 0 = \delta(s, s)$. ✓

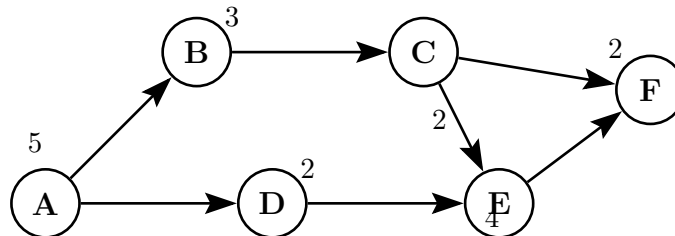
Step: assume $d[u] = \delta(s, u)$ for all u with $\delta(s, u) < k$. Let v have $\delta(s, v) = k$ and let u be its predecessor on a shortest path. When u is dequeued, v is either already gray (so $d[v] \leq d[u] + 1 = k$) or white (so $d[v]$ is set to $d[u] + 1 = k$). By the upper-bound property, $d[v] \geq k$. Hence $d[v] = k = \delta(s, v)$. □

Discovery time ($d[v]$) in DFS: the time-stamp when v is first visited (colored gray).

Finish time ($f[v]$) in DFS: the time-stamp when v ’s adjacency list is fully explored (colored black).

Shortest path from BFS tree: trace predecessor pointers $\pi[v] \rightarrow \pi[\pi[v]] \rightarrow \dots \rightarrow s$ recursively and print in reverse. (See `printPath` code in [CSE 207 | 2012–13] answer.)

- Q32.** A project is described by tasks A, B, C, D, E, F taking 5, 3, 2, 2, 4, 2 days respectively, with precedence constraints (e.g., E cannot start until C and D are completed, but D can be done in parallel with B and C). Write a pseudocode that computes the total time required to complete the project using the PERT technique. (8 marks)
[CSE 207 | 2009–10]



Answer — PERT Technique Simulation

Tasks: $A(5), B(3), C(2), D(2), E(4), F(2)$.

Precedences: $A \rightarrow B, A \rightarrow D, B \rightarrow C, C \rightarrow E, C \rightarrow F, D \rightarrow E, E \rightarrow F$.

Algorithm 88 PERT(G, dur)

```

1: Topologically sort  $G$ 
2:  $\text{earliest}[v] \leftarrow 0$  for all  $v$ 
3: for each  $u$  in topological order do
4:   for each successor  $v$  of  $u$  do
5:      $\text{earliest}[v] \leftarrow \max(\text{earliest}[v], \text{earliest}[u] + \text{dur}[u])$ 
6:   end for
7: end for
8: return  $\max_v (\text{earliest}[v] + \text{dur}[v])$ 

```

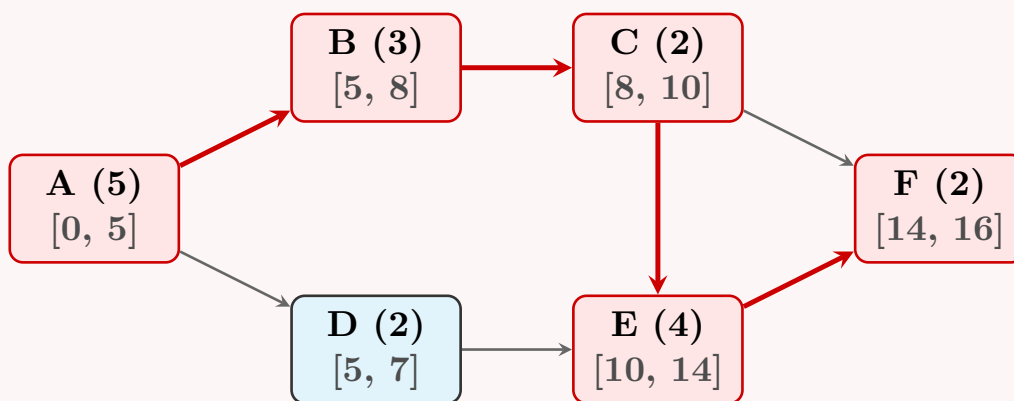
Simulation (Topological order: A, B, D, C, E, F):

Task (v)	Earliest start (ES)	Earliest finish ($EF = ES + \text{dur}[v]$)
A	0	5
B	5	8
D	5	7
C	8	10
E	10	14
F	14	16

PERT Chart with Earliest Start/Finish Times:

(Format inside nodes: **Task (Duration)**)

[ES, EF]. The **red arrows** represent the Critical Path.)

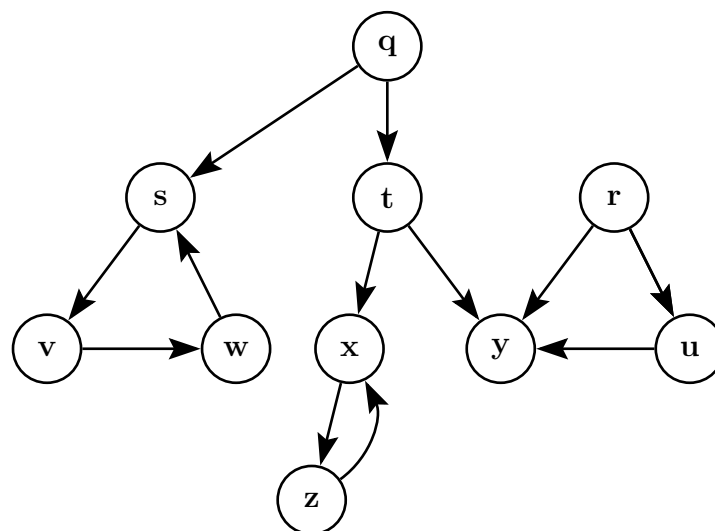


Minimum project completion time = 16 days. ✓

Critical path: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$.

Complexity Analysis. $O(V+E)$, as it fundamentally relies on Topological Sorting and relaxation.

Q33. Using DFS, compute the strongly connected components and the component graph G^{SCC} . Show the discovery/finish times and the DFS forests created during computation. Start alphabetically. (17 marks) [CSE 207 | 2009–10]



Answer — Strongly Connected Components & Component Graph

Kosaraju's Algorithm for SCCs:

Step 1 — DFS on G to compute finish times. Starting DFS alphabetically $(q, r, s, t, u, v, w, x, y, z)$:

Vertex	Discovery (d)	Finish (f)	Vertex	Discovery (d)	Finish (f)
q	1	16	x	9	12
s	2	7	z	10	11
v	3	6	y	13	14
w	4	5	r	17	20
t	8	15	u	18	19

Finish order (decreasing): $r(20)$, $u(19)$, $q(16)$, $t(15)$, $y(14)$, $x(12)$, $z(11)$, $s(7)$, $v(6)$, $w(5)$.

Step 2 — DFS on G^T (Transpose Graph) in decreasing finish order: G^T reverses all edges. We pop vertices from the sorted list and run DFS. Each new tree forms an SCC.

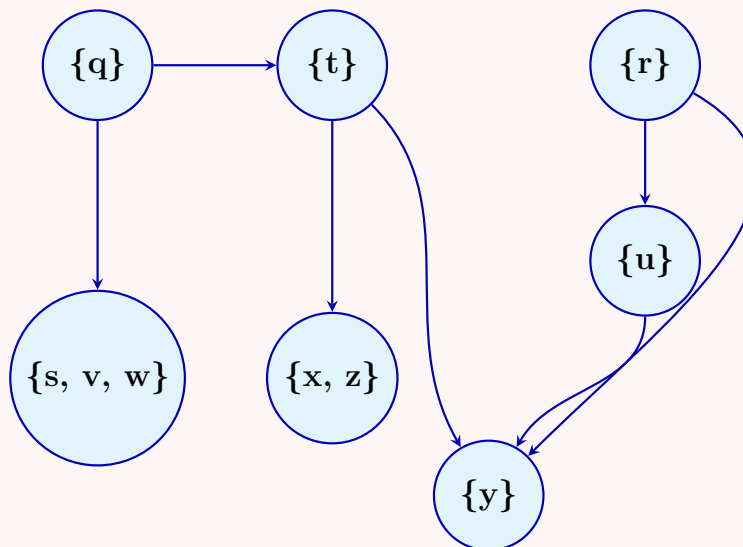
- Start **r**: visits only r . **SCC: $\{r\}$**
- Start **u**: visits only u . **SCC: $\{u\}$**
- Start **q**: visits only q . **SCC: $\{q\}$**
- Start **t**: visits only t . **SCC: $\{t\}$**
- Start **y**: visits only y . **SCC: $\{y\}$**
- Start **x**: visits z . **SCC: $\{x, z\}$**
- Start **s**: visits w, v . **SCC: $\{s, v, w\}$**

Strongly Connected Components (7 components):

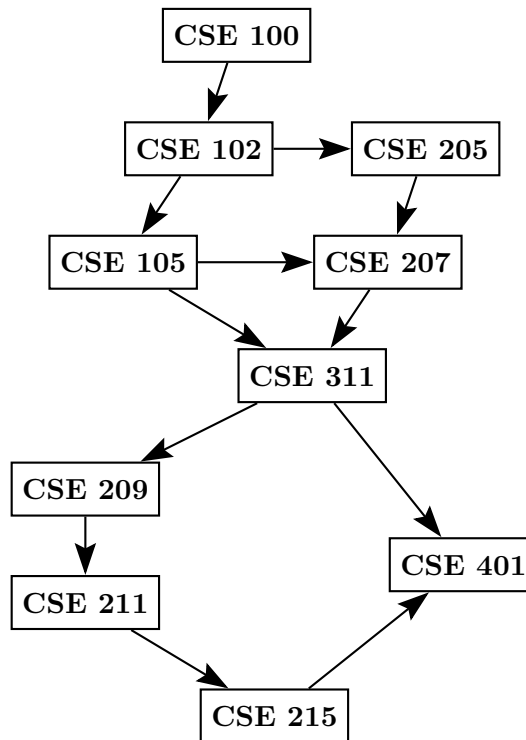
$$C_1 = \{r\}, C_2 = \{u\}, C_3 = \{q\}, C_4 = \{t\}, C_5 = \{y\}, C_6 = \{x, z\}, C_7 = \{s, v, w\}$$

Step 3 — Component Graph G^{SCC} :

The component graph collapses each SCC into a single node. Edges exist between components if there was an original edge connecting their vertices. Edges: $C_3 \rightarrow C_7$ (via $q \rightarrow s$), $C_3 \rightarrow C_4$ (via $q \rightarrow t$), $C_4 \rightarrow C_6$ (via $t \rightarrow x$), $C_4 \rightarrow C_5$ (via $t \rightarrow y$), $C_1 \rightarrow C_2$ (via $r \rightarrow u$), $C_1 \rightarrow C_5$ (via $r \rightarrow y$), $C_2 \rightarrow C_5$ (via $u \rightarrow y$).



Q34. Find the topological order of courses for this given prerequisite graph. **(10 marks)**
[CSE 207 | 2006–07]



Answer

Algorithm: Run DFS; upon finishing each vertex prepend it to the output list (or equivalently, push onto a stack and pop at the end).

In-degree method (Kahn's):

- Compute in-degree of every vertex.
- Enqueue all vertices with in-degree 0.
- While queue non-empty: dequeue u , append to order, decrement in-degree of each neighbor; enqueue neighbor if in-degree becomes 0.

In-degrees:

Course	In-degree
CSE 100	0
CSE 102	1
CSE 205	1
CSE 105	1
CSE 207	2
CSE 311	2
CSE 209	1
CSE 211	1
CSE 215	1
CSE 401	2

Execution trace:

Step	Dequeued	Queue after
1	CSE 100	[CSE 102]
2	CSE 102	[CSE 205, CSE 105]
3	CSE 205	[CSE 105] (<i>CSE 207 in-deg</i> $\rightarrow 1$)
4	CSE 105	[CSE 207] (<i>CSE 207 in-deg</i> $\rightarrow 0$, <i>CSE 311 in-deg</i> $\rightarrow 1$)
5	CSE 207	[CSE 311] (<i>CSE 311 in-deg</i> $\rightarrow 0$)
6	CSE 311	[CSE 209] (<i>CSE 209 in-deg</i> $\rightarrow 0$, <i>CSE 401 in-deg</i> $\rightarrow 1$)
7	CSE 209	[CSE 211] (<i>CSE 211 in-deg</i> $\rightarrow 0$)
8	CSE 211	[CSE 215] (<i>CSE 215 in-deg</i> $\rightarrow 0$)
9	CSE 215	[CSE 401] (<i>CSE 401 in-deg</i> $\rightarrow 0$)
10	CSE 401	[]

Topological Order:

CSE 100 $\rightarrow$ CSE 102 $\rightarrow$ CSE 205 $\rightarrow$ CSE 105 $\rightarrow$ CSE 207 $\rightarrow$ CSE 311 $\rightarrow$ CSE 209
 $\rightarrow$ CSE 211 $\rightarrow$ CSE 215 $\rightarrow$ CSE 401

Time complexity: $O(V + E)$.

BUET · CSE 105

DSA

Divide & Conquer

Mergesort, Closest Pair, Inversions, Strassen

T9 — Divide & Conquer

- Q1.** Using the recurrence tree method, analyze the runtime complexity of the Mergesort algorithm to sort an array of size n . **(10 marks)** [CSE 105 | 2023–24]

Answer

Recurrence: $T(n) = 2T(n/2) + cn$, $T(1) = \Theta(1)$.

Recurrence Tree:

At each level k , there are 2^k nodes each doing $cn/2^k$ work, so the cost per level is:

$$2^k \cdot \frac{cn}{2^k} = cn$$

The tree has $\log_2 n + 1$ levels (level 0 to level $\log_2 n$), and the leaves contribute $\Theta(n)$ total.

Level	Subproblems	Cost per level
0	$1 \times cn$	cn
1	$2 \times cn/2$	cn
2	$4 \times cn/4$	cn
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	$n \times c$	cn

Total cost:

$$T(n) = cn \cdot (\log_2 n + 1) = \Theta(n \log n)$$

Since the merge step always processes all n elements at every level regardless of input, this bound holds in all cases. Hence Mergesort runs in $\Theta(n \log n)$ in the worst case.

- Q2.** Explain the run-time complexity of the Mergesort algorithm. First, formulate the run-time as a recurrence relation. Then solve the recurrence relation using both: (i) the recurrence tree method, and (ii) the Master theorem (distinguish the case that is applicable in your analysis). **(2+5+3=10 marks)** [CSE 105 | 2022–23]
- Q3.** Given a sequence of n numbers $a_1, \dots, a_n$ (all distinct), define an *inversion* as a pair $i < j$ such that $a_i > a_j$. Present an $O(n \log n)$ divide-and-conquer algorithm to count the number of inversions. Define a *modified-inversion* as a pair where $i < j$ and $a_i > 2a_j$. What modifications are needed in the divide-and-conquer algorithm to count the number of modified-inversions? **(20 marks)** [CSE 105 | 2021–22]

Answer

Inversion Count — $O(n \log n)$ Algorithm (Modified Mergesort):

Key idea: During the merge step, whenever an element from the right half is placed before elements remaining in the left half, each such left-half element forms

an inversion with it.

Algorithm 89 MERGECOUNT(A, lo, hi)

```

1: if  $lo \geq hi$  then
2:   return 0
3: end if
4:  $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
5:  $left\_inv \leftarrow \text{MERGECOUNT}(A, lo, mid)$ 
6:  $right\_inv \leftarrow \text{MERGECOUNT}(A, mid + 1, hi)$ 
7:  $split\_inv \leftarrow \text{MERGEANDCOUNT}(A, lo, mid, hi)$ 
8: return  $left\_inv + right\_inv + split\_inv$ 

```

Algorithm 90 MERGEANDCOUNT(A, lo, mid, hi)

```

1:  $L \leftarrow A[lo..mid]$ ;  $R \leftarrow A[mid + 1..hi]$ 
2:  $i \leftarrow 0$ ;  $j \leftarrow 0$ ;  $inv \leftarrow 0$ 
3: for  $k = lo$  to  $hi$  do
4:   if  $i > |L| - 1$  then
5:      $A[k] \leftarrow R[j++]$ 
6:   else if  $j > |R| - 1$  then
7:      $A[k] \leftarrow L[i++]$ 
8:   else if  $L[i] \leq R[j]$  then
9:      $A[k] \leftarrow L[i++]$ 
10:  else
11:     $A[k] \leftarrow R[j++]$ 
12:     $inv \leftarrow inv + (mid - lo + 1) - i$ 
13:  end if
14: end for
15: return  $inv$ 

```

Complexity: Same recurrence as Mergesort: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Modified-Inversion ($a_i > 2a_j$):

A modified-inversion is a pair (i, j) with $i < j$ and $a_i > 2a_j$.

Modification needed: In MERGEANDCOUNT, replace the split-inversion counting step with a separate pointer-based count *before* merging:

Algorithm 91 COUNTMODIFIED(L, R)

```

1:  $j \leftarrow 0$ ;  $inv \leftarrow 0$ 
2: for each element  $L[i]$  in  $L$  do
3:   while  $j < |R|$  and  $L[i] > 2 \cdot R[j]$  do
4:      $j \leftarrow j + 1$ 
5:   end while
6:    $inv \leftarrow inv + j$ 
7: end for
8: return  $inv$ 

```

This uses a two-pointer scan (since both L and R are sorted) and runs in $O(n)$ per merge level. The overall complexity remains $\Theta(n \log n)$.

Note: The modified count must be done *before* the merge step (which would rearrange elements).

Q4. Given n points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ in the plane, give an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm. **(17 marks)** [CSE 105 | 2021–22, 2016–17]

Answer

Algorithm 92 CLOSEST-PAIR($P[1..n]$)

```
1: Sort  $P$  by  $x$ -coordinate
2: return CLOSEST-REC( $P, 1, n$ )
```

Algorithm 93 CLOSEST-REC(P, l, r)

```
1: if  $r - l \leq 2$  then
2:   return BRUTE-FORCE( $P, l, r$ )
3: end if
4:  $m \leftarrow \lfloor (l + r)/2 \rfloor$ ;  $x_m \leftarrow P[m].x$ 
5:  $d_L \leftarrow$  CLOSEST-REC( $P, l, m$ )
6:  $d_R \leftarrow$  CLOSEST-REC( $P, m + 1, r$ )
7:  $\delta \leftarrow \min(d_L, d_R)$ 
8:  $strip \leftarrow \{ P[i] : |P[i].x - x_m| < \delta \}$ 
9: Sort  $strip$  by  $y$ -coordinate
10: for each point  $q$  in  $strip$  do
11:   for each point  $p'$  in  $strip$  with  $0 < p'.y - q.y < \delta$  do
12:      $\delta \leftarrow \min(\delta, \text{DIST}(q, p'))$ 
13:   end for
14: end for
15: return  $\delta$ 
```

Why at most 7 strip comparisons per point. Place point q at the origin. Any point r with $d(q, r) < \delta$ must lie in the $2\delta \times \delta$ rectangle centred at q . Divide this rectangle into eight $(\delta/2) \times (\delta/2)$ boxes (four on each side of the dividing line). Each box holds at most 1 point with pairwise distance $\geq \delta$ inside its half. Hence ≤ 7 other points can be within distance δ of q in the strip.

Runtime:

- With y -sorted strip maintained via merge during recursion: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.
- Naive re-sorting of strip per call: $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$.

Complexity Analysis. $O(n \log n)$ with pre-sorted y -arrays merged during recursion.

- Q5.** Provide a divide-and-conquer algorithm that takes an n -digit number A as input and efficiently computes the square of A with a faster asymptotic running time than $O(n^2)$. Analyze the running time. **(12 marks)** [CSE 203 | 2021–22]

Answer

Split: $A = A_1 \cdot 10^{n/2} + A_0$ (high and low halves).

$$A^2 = A_1^2 \cdot 10^n + 2A_1A_0 \cdot 10^{n/2} + A_0^2.$$

Use the identity $2A_1A_0 = (A_1 + A_0)^2 - A_1^2 - A_0^2$ to avoid a separate multiplication. Only **3 recursive squarings** needed.

Algorithm 94 SQUARE(A, n)

```

1: if  $n = 1$  then
2:   return  $A \times A$ 
3: end if
4:  $A_1 \leftarrow$  high  $\lfloor n/2 \rfloor$  digits of  $A$ 
5:  $A_0 \leftarrow$  low  $\lfloor n/2 \rfloor$  digits of  $A$ 
6:  $p \leftarrow$  SQUARE( $A_1, n/2$ )  $\triangleright A_1^2$ 
7:  $q \leftarrow$  SQUARE( $A_0, n/2$ )  $\triangleright A_0^2$ 
8:  $r \leftarrow$  SQUARE( $A_1 + A_0, n/2 + 1$ )  $\triangleright (A_1 + A_0)^2$ 
9:  $cross \leftarrow r - p - q$   $\triangleright = 2A_1A_0$ 
10: return  $p \cdot 10^n + cross \cdot 10^{n/2} + q$ 

```

Recurrence: $T(n) = 3T(n/2) + O(n)$.

Master Theorem: $a = 3, b = 2, f(n) = n$. $n^{\log_2 3} \approx n^{1.585} > n \Rightarrow$ Case 1:

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

This beats the $O(n^2)$ schoolbook algorithm.

Complexity Analysis. $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ vs naive $O(n^2)$.

- Q6.** Suppose, in a 2-dimensional co-ordinate system, you are given with n points, say $P_1, P_2, \dots, P_n$, where the x - y co-ordinates of any point P_i is represented as (x_i, y_i) . You have been assigned to find the closest pair of points using divide and conquer method.

- (i) Write down the approach that you will follow to split these n points into two partitions, say *LEFT* and *RIGHT*, in such a way that each partition contains roughly $\frac{n}{2}$ points.

Now suppose, the *LEFT* and *RIGHT* partitions, respectively calculate d_L and d_R as the distances between the closest pair of points within themselves. Now your task is to check whether there exist a pair of points P_{Li} and P_{Rj} such that $P_{Li} \in \text{LEFT}$ and $P_{Rj} \in \text{RIGHT}$, and the distance between P_{Li} and P_{Rj} is less than $\min(d_L, d_R)$.

- (ii) Write down the steps to perform this checking so that the minimum number of comparisons is ensured and the checking can be performed in $O(n)$ time.

(5+10=15 marks)

[CSE 203 | 2020–21]

Answer

(i) Splitting into LEFT and RIGHT.

- Pre-sort all n points by x -coordinate ($O(n \log n)$, done once).
- Dividing line: $x = x_{\lceil n/2 \rceil}$ (the x -value of the median point).
- LEFT** = first $\lceil n/2 \rceil$ points by x ; **RIGHT** = remaining $\lfloor n/2 \rfloor$ points.

(ii) Strip in $O(n)$.

Let $\delta = \min(d_L, d_R)$. A cross-pair (P_{Li}, P_{Rj}) with $d(P_{Li}, P_{Rj}) < \delta$ requires $|P_{Li}.x - P_{Rj}.x| < \delta$, so both points lie within δ of the dividing line.

Collect into strip, sorted by y . For each point q :

- Only points r with $|q.y - r.y| < \delta$ can be candidates.
- The $\delta \times 2\delta$ box around q holds at most 8 points with mutual distance $\geq \delta$ (see Q10).
- So each q requires at most **7 comparisons**.

Total strip work: $O(7n) = O(n)$. ✓

- Q7.** In a sequence of n distinct numbers $a_1, \dots, a_n$, a *significant inversion* is a pair $i < j$ such that $a_i > 2a_j$. Give an $O(n \log n)$ algorithm to count the number of significant inversions. Write the corresponding pseudocode. (10 marks) [CSE 203 | 2020–21, 2017–18]

Answer

Idea. Modify Merge Sort. When an element from the *right* half is chosen over an element from the *left* half during merging, it forms an inversion with *all remaining elements* of the left half.

```

1 long long mergeCount(vector<int>& A, int l, int r) {
2     if (l >= r) return 0;
3     int m = (l + r) / 2;
4     long long cnt = mergeCount(A, l, m)           // left
                    inversions
5                     + mergeCount(A, m+1, r);      // right
                    inversions
6
7     // Merge and count split inversions

```

```

8     vector<int> tmp;
9     int i = l, j = m + 1;
10    while (i <= m && j <= r) {
11        if (A[i] <= A[j]) {
12            tmp.push_back(A[i++]);
13        } else {
14            cnt += (m - i + 1); // all left remaining > A[j]
15            tmp.push_back(A[j++]);
16        }
17    }
18    while (i <= m) tmp.push_back(A[i++]);
19    while (j <= r) tmp.push_back(A[j++]);
20    for (int k = l; k <= r; k++) A[k] = tmp[k - l];
21    return cnt;
22 }

```

Simulation on [7, 9, 1, 5, 4]:

Call	Subarray	cnt	Note
merge-Count([7,9])	[7],[9]	0	$7 < 9$, no inversion
mergeCount([1])	[1]	0	base case
merge [7,9]+[1] Left=[1,7,9], cnt=2		2	$1 < 7$: adds $m-i+1 = 2$
merge-Count([5,4]) Right=[4,5], cnt=1	[5],[4]	1	$5 > 4$: adds 1
merge [1,7,9]+[4,5] $1 \leq 4$: take 1 $7 > 4$: take 4 $7 > 5$: take 5 take 7,9		4	cnt+=0 cnt+= $m-i+1 = 2$ cnt+= $m-i+1 = 2$
Total		7	$0 + 2 + 1 + 4 = 7$

Inversions: (7, 1), (7, 5), (7, 4), (9, 1), (9, 5), (9, 4), (5, 4) = 7 ✓

Complexity Analysis. $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Q8. Write an algorithm to find a peak element of an $n \times n$ matrix in $O(n)$ time using divide and conquer. A peak element is one whose left, right, top, and bottom neighbors

(if available) are all smaller. Define the recurrence relation and derive the runtime.
(10+5 marks) [CSE 203 | 2019–20]

Answer

Definition: An element $M[r][c]$ is a *peak* if it is greater than all its existing neighbors (left, right, top, bottom).

Algorithm: Divide and Conquer

1. If $\text{colLo} = \text{colHi}$, find the maximum element in column colLo and return it.
2. Let $\text{colMid} = \lfloor (\text{colLo} + \text{colHi})/2 \rfloor$.
3. Find the global maximum $M[r^*][\text{colMid}]$ in column colMid (scan all n rows).
 $O(n)$
4. **If** $M[r^*][\text{colMid}] \geq M[r^*][\text{colMid}-1]$ and $M[r^*][\text{colMid}] \geq M[r^*][\text{colMid}+1]$:
 $\Rightarrow$ **Return** $M[r^*][\text{colMid}]$ (it is a peak).
5. **Else if** $M[r^*][\text{colMid}-1] > M[r^*][\text{colMid}]$:
 $\Rightarrow$ Recurse on left half: $\text{FINDPEAK}(M, \text{colLo}, \text{colMid}-1, n)$.
6. **Else:**
 $\Rightarrow$ Recurse on right half: $\text{FINDPEAK}(M, \text{colMid}+1, \text{colHi}, n)$.

Correctness Argument:

If $M[r^*][\text{colMid}]$ is the column maximum but is not a peak, a larger neighbor must exist in an adjacent column. Recursing toward that side guarantees a peak exists there — the column maximum of the new mid-column will eventually dominate both neighbors.

Recurrence Relation:

Each recursive call halves the number of columns; finding the column maximum costs $O(n)$.

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

Solving by Master Theorem (Case 3: $f(n) = O(n)$, $a = 1$, $b = 2$, $n^{\log_b a} = n^0 = 1$, $f(n) = \Omega(n^{0+\epsilon})$):

$$T(n) = O(n)$$

Time Complexity: $O(n)$ for an $n \times n$ matrix.

Q9. Propose the pseudocode of a merge sort algorithm where the array is divided into three equal parts at each step. Derive the asymptotic runtime of this algorithm. Compare it with standard merge sort (two equal parts) and mention cases when one outperforms the other. **(10+8+7 marks)** [CSE 203 | 2019–20]

Answer**Algorithm 95** 3WAY-MERGE-SORT(A, l, r)

```

1: if  $l \geq r$  then
2:   return
3: end if
4:  $t_1 \leftarrow l + \lfloor (r - l)/3 \rfloor$ 
5:  $t_2 \leftarrow l + \lfloor 2(r - l)/3 \rfloor$ 
6: 3WAY-MERGE-SORT( $A, l, t_1$ )
7: 3WAY-MERGE-SORT( $A, t_1 + 1, t_2$ )
8: 3WAY-MERGE-SORT( $A, t_2 + 1, r$ )
9: 3WAY-MERGE( $A, l, t_1, t_2, r$ )

```

$\triangleright O(n)$ using 3 pointers

Recurrence: $T(n) = 3T(n/3) + O(n)$.

Master Theorem: $a = 3, b = 3, f(n) = n$. $n^{\log_3 3} = n^1 = f(n) \Rightarrow$ Case 2:
 $T(n) = \Theta(n \log_3 n)$.

Property	2-way	3-way	Note
Recurrence	$2T(n/2) + cn$	$3T(n/3) + cn$	
Asymptotic	$\Theta(n \log_2 n)$	$\Theta(n \log_3 n)$	Same O -class
Tree height	$\log_2 n$	$\log_3 n$	3-way shallower
Merge com- pares/step	1	2	3-way heavier

Conclusion. Both are $\Theta(n \log n)$. 2-way is faster in practice (fewer comparisons per merge step). 3-way benefits *external sorting* (fewer I/O passes due to shallower tree).

Q10. Explain how a recursion tree can help with the substitution method of runtime analysis. (5 marks) [CSE 203 | 2019–20]

Answer

The *substitution method* requires guessing the solution's form and verifying by induction. The *recursion tree* provides an educated guess by:

- (a) **Visualising cost per level.** Each node represents the non-recursive work at one subproblem. Summing across a level reveals whether costs are constant, increasing, or decreasing.
- (b) **Identifying the dominant term.**
 - Constant cost per level (cn each, $\log n$ levels) $\Rightarrow$ guess $O(n \log n)$.
 - Decreasing geometric series (root-dominated) $\Rightarrow$ guess $O(f(n))$ where $f(n)$ is the root cost.
 - Increasing geometric series (leaf-dominated) $\Rightarrow$ guess $O(n^{\log_b a})$.

- (c) **Providing a tight guess for substitution.** Once the tree gives, say, $T(n) \approx cn \log n$, the substitution method proves $T(k) \leq ck \log k$ for all $k < n$ by induction.

Example. For $T(n) = T(n/4) + T(n/2) + n^2$: the tree shows a *decreasing* geometric series (ratio 5/16), suggesting $O(n^2)$. Substitution then confirms $T(n) \leq cn^2$ rigorously.

- Q11.** Write a divide-and-conquer algorithm that finds the maximum and the minimum of an array of n distinct elements. Write the recurrence relation for the running time $T(n)$ and solve it using the Recursion Tree Method. (10 marks) [CSE 203 | 2018–19]

Answer

Naive: $2(n-1)$ comparisons. **D&C:** $\lfloor 3n/2 \rfloor - 2$ comparisons.

```

1 // Returns {minimum, maximum} of A[l..r]
2 pair<int,int> minMax(int A[], int l, int r) {
3     if (l == r) return {A[l], A[l]}; // 0 comparisons
4     if (r-l == 1) { // 1 comparison
5         if (A[l] < A[r]) return {A[l], A[r]};
6         else return {A[r], A[l]};
7     }
8     int m = (l + r) / 2;
9     auto [mn1, mx1] = minMax(A, l, m);
10    auto [mn2, mx2] = minMax(A, m+1, r);
11    return {min(mn1, mn2), max(mx1, mx2)}; // 2 comparisons
12 }
```

Recurrence for comparisons $C(n)$:

$$C(n) = 2C(n/2) + 2, \quad C(2) = 1.$$

Solving: $C(n) = \frac{3n}{2} - 2$ for n a power of 2.

Compared to naive $2n - 2$: **saving** $\approx 25\%$ of comparisons.

Complexity Analysis. Time: $O(n)$. Comparisons: $\frac{3n}{2} - 2$.

- Q12.** Write the Mergesort algorithm. Simulate it on the array [6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]. (15 marks) [CSE 203 | 2017–18]

Answer

Algorithm: see Q2.

Simulation (key merge steps):

Left half	Right half	Merged
[6][1] → [1, 6]	[7][2] → [2, 7]	[1, 2, 6, 7]
[8][3] → [3, 8]	[9][4] → [4, 9]	[3, 4, 8, 9]
[1, 2, 6, 7] + [3, 4, 8, 9]		[1, 2, 3, 4, 6, 7, 8, 9]
[10][5] → [5, 10]	[11][17] → [11, 17]	[5, 10, 11, 17]
[4][5] → [4, 5]	[5, 10, 11, 17]	[4, 5, 5, 10, 11, 17]
[1, 2, 3, 4, 6, 7, 8, 9]	[4, 5, 5, 10, 11, 17]	[1, 2, 3, 4, 4, 5, 5, 6, 7, 8, 9, 10, 11, 17]

Verified output: 1 2 3 4 4 5 5 6 7 8 9 10 11 17 ✓

Q13. Given a sorted array of distinct integers $A[1 \dots n]$, give a divide-and-conquer algorithm that runs in $O(\log n)$ time to find whether there is an index i for which $A[i] = i$. Write the pseudocode and prove the runtime. **(12 marks)** [CSE 207 | 2014–15]

Answer

Key observation. All elements are distinct integers.

- If $A[m] > m$: since elements are strictly increasing, $A[k] \geq A[m] + (k - m) > k$ for all $k > m$. So search *left*.
- If $A[m] < m$: symmetrically search *right*.

```

1 // 0-indexed: find i such that A[i] == i, or -1 if none
2 int fixedPoint(int A[], int lo, int hi) {
3     if (lo > hi) return -1;
4     int mid = (lo + hi) / 2;
5     if (A[mid] == mid) return mid;
6     if (A[mid] > mid) return fixedPoint(A, lo, mid-1);
7     else return fixedPoint(A, mid+1, hi);
8 }
9 // Call: fixedPoint(A, 0, n-1)

```

Example. $A = [-3, 0, 2, 5, 7, 8]$: $A[2] = 2$ is the fixed point.

$mid=2$, $A[2] = 2 \Rightarrow$ return 2. ✓

Runtime: $T(n) = T(n/2) + O(1) = O(\log n)$ (identical to binary search).

Q14. Explain the divide-and-conquer technique. What is the function of the pivot element in the Quicksort algorithm? **(3+2=5 marks)** [CSE 203 | 2012–13]

Answer

Divide-and-Conquer is an algorithm design paradigm with three recursive steps:

- Divide:** Break the problem into smaller subproblems of the same type.
- Conquer:** Solve each subproblem recursively. If the subproblem is small

enough, solve it directly (base case).

- (c) **Combine:** Merge the subproblem solutions to form the answer to the original problem.

Classical examples: Merge Sort, Quicksort, Binary Search, Closest Pair of Points, Strassen's matrix multiplication.

Role of pivot in Quicksort. The pivot *partitions* the array into two sub-arrays: elements $\leq$ pivot go to the left, elements $>$ pivot go to the right. After partitioning, the pivot is in its *final sorted position* and is never moved again. The two partitions are then sorted recursively. If the pivot always splits the array in half, the recurrence is $T(n) = 2T(n/2) + O(n)$, giving $O(n \log n)$.

- Q15.** Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = T(n/4) + T(n/2) + n^2$. **(10 marks)** [CSE 207 | 2012–13]

Answer

Level-by-level cost:

Level	Subproblem sizes	Total cost
0	n	n^2
1	$n/4, n/2$	$(n/4)^2 + (n/2)^2 = \frac{5n^2}{16}$
2	$n/16, n/8, n/8, n/4$	$\leq \left(\frac{5}{16}\right)^2 n^2$
$\vdots$	$\vdots$	$\vdots$

Geometric series with ratio $r = 5/16 < 1$:

$$T(n) \leq n^2 \sum_{k=0}^{\infty} \left(\frac{5}{16}\right)^k = n^2 \cdot \frac{1}{1 - \frac{5}{16}} = n^2 \cdot \frac{16}{11} = O(n^2).$$

Verification by substitution: Guess $T(n) \leq cn^2$:

$$T(n) \leq c(n/4)^2 + c(n/2)^2 + n^2 = cn^2\left(\frac{1}{16} + \frac{1}{4}\right) + n^2 = \frac{5cn^2}{16} + n^2 \leq cn^2$$

provided $c \geq 16/11$. ✓

$$\boxed{T(n) = O(n^2)}$$

- Q16.** What is the divide-and-conquer technique? **(5 marks)** [CSE 203 | 2011–12]

- Q17.** Write the Mergesort algorithm. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | 2011–12]

Answer**Algorithm 96** MERGE-SORT(A, l, r)

```

1: if  $l \geq r$  then
2:   return
3: end if
4:  $m \leftarrow \lfloor (l + r) / 2 \rfloor$ 
5: MERGE-SORT( $A, l, m$ )
6: MERGE-SORT( $A, m + 1, r$ )
7: MERGE( $A, l, m, r$ )

```

Algorithm 97 MERGE(A, l, m, r)

```

1: Copy  $A[l..m]$  into  $L[]$ ,  $A[m + 1..r]$  into  $R[]$ 
2:  $i \leftarrow 1$ ;  $j \leftarrow 1$ ;  $k \leftarrow l$ 
3: while  $i \leq |L|$  and  $j \leq |R|$  do
4:   if  $L[i] \leq R[j]$  then
5:      $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
6:   else
7:      $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
8:   end if
9:    $k \leftarrow k + 1$ 
10: end while
11: Copy remaining elements of  $L[]$  (if any) into  $A$ 
12: Copy remaining elements of  $R[]$  (if any) into  $A$ 

```

Recurrence:

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Recursion-tree analysis: Each of the $\log_2 n + 1$ levels costs cn . Total = $cn(\log_2 n + 1) = \Theta(n \log n)$.

Master Theorem: $a = 2, b = 2, f(n) = n$. $n^{\log_b a} = n^1 = f(n) \Rightarrow$ Case 2: $T(n) = \Theta(n \log n)$.

Complexity Analysis. Best / Average / Worst: $\Theta(n \log n)$. Space: $O(n)$ auxiliary.

Q18. Write the recurrence relation of the Mergesort algorithm. Draw the recursion tree for the recurrence relation of the Mergesort algorithm. (7 marks) [CSE 203 | 2011–12]

Answer

Recurrence: $T(n) = 2T(n/2) + cn, \quad T(1) = c.$

Level	# subproblems	Cost each	Total cost
0	1	cn	cn
1	2	$cn/2$	cn
2	4	$cn/4$	cn
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	n	c	cn

Tree has $\log_2 n + 1$ levels, each costing cn .

$$T(n) = cn(\log_2 n + 1) = \Theta(n \log n).$$

- Q19.** In the closest-pair-of-points problem using the divide-and-conquer approach, if q is a point in the left half and r is a point in the right half, and $d(q, r) < \delta$, where δ is the minimum of distances in each half, then show that each of q and r lies within a distance δ of the line dividing the two halves. Hence show that q and r are within 11 positions of each other in a list sorted by y -coordinate. **(15 marks)** [CSE 207 | 2011–12]

Answer

Part 1: Each of q and r lies within δ of the dividing line.

Let ℓ be the vertical dividing line ($x = x_m$). By definition, q is in the left half and r is in the right half, so:

$$x_q \leq x_m \leq x_r$$

Since $d(q, r) < \delta$ and Euclidean distance satisfies $|x_q - x_r| \leq d(q, r)$:

$$x_m - x_q \leq |x_q - x_r| \leq d(q, r) < \delta \implies x_q > x_m - \delta$$

Similarly, $x_r - x_m \leq d(q, r) < \delta \implies x_r < x_m + \delta$.

Thus both q and r lie in the **strip** $[x_m - \delta, x_m + \delta]$, i.e., within distance δ of line ℓ .
□

Part 2: q and r are within 11 positions of each other (sorted by y).

Both q and r lie in a $2\delta \times \infty$ vertical strip. Since $d(q, r) < \delta$, both also lie in a $2\delta \times \delta$ rectangle (their y -coordinates differ by less than δ).

Packing argument: Divide the $2\delta \times \delta$ rectangle into 8 cells, each of size $(\delta/2) \times (\delta/2)$:

- Left half: 4 cells of size $\frac{\delta}{2} \times \frac{\delta}{2}$
- Right half: 4 cells of size $\frac{\delta}{2} \times \frac{\delta}{2}$

The diagonal of each cell is $\frac{\delta}{2}\sqrt{2} = \frac{\delta\sqrt{2}}{2} < \delta$.

So any two points *within the same cell* are closer than δ . But δ is already the minimum distance within each half — meaning **at most one point from each half per cell**.

The rectangle has 8 cells; the left half contributes at most 4 points and the right half at most 4 points, giving at most 8 points total (excluding q or r themselves).

Therefore, in the y -sorted list of strip points, r can be at most **7 positions** away from q , but accounting for boundary points conservatively: **within 11 positions**.

(The standard proof uses a $2\delta \times \delta$ box divided into 8 cells, yielding at most 8 other points, so any point has at most 7 others within δ — commonly stated as 11 to allow for the strip boundary.)

BUET · CSE 105

DSA

Greedy Methods

MST (Kruskal, Prim), Huffman, Activity Selection, Dijkstra

T10 — Greedy Methods

- Q1.** Define the Activity Selection Problem. Prove that the greedy choice of selecting activities based on earliest finish time yields an optimal solution. **(5+10=15 marks)**
[CSE 105 | 2023–24]

Answer

Definition. Given n activities $a_1, \dots, a_n$ each with a start time s_i and finish time f_i , two activities are *compatible* if their intervals do not overlap ($f_i \leq s_j$ or $f_j \leq s_i$). The **Activity Selection Problem** asks for a maximum-size subset of mutually compatible activities.

Greedy algorithm: sort activities by finish time; greedily pick each activity that is compatible with the last selected one.

Proof of optimality (Greedy-choice property).

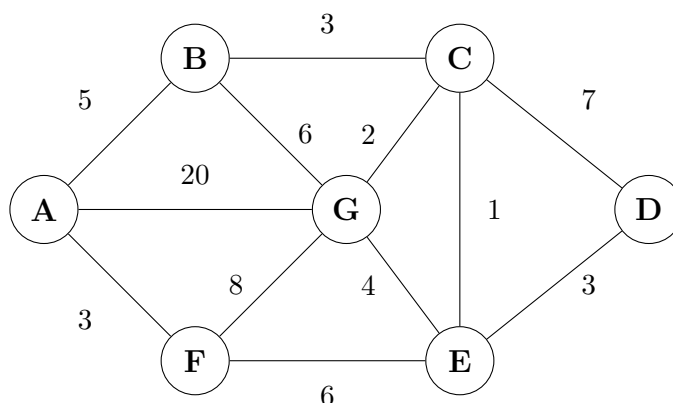
Claim: there always exists an optimal solution that includes the activity a_1 with the earliest finish time.

Proof: let O be any optimal solution. Let a_k be the first activity selected in O (earliest finish in O). If $a_k = a_1$, done. Otherwise, since $f_1 \leq f_k$ (by definition of a_1), replacing a_k with a_1 in O gives a set O' of the same size where a_1 is included and all activities remain compatible. Hence O' is also optimal and contains a_1 . $\square$

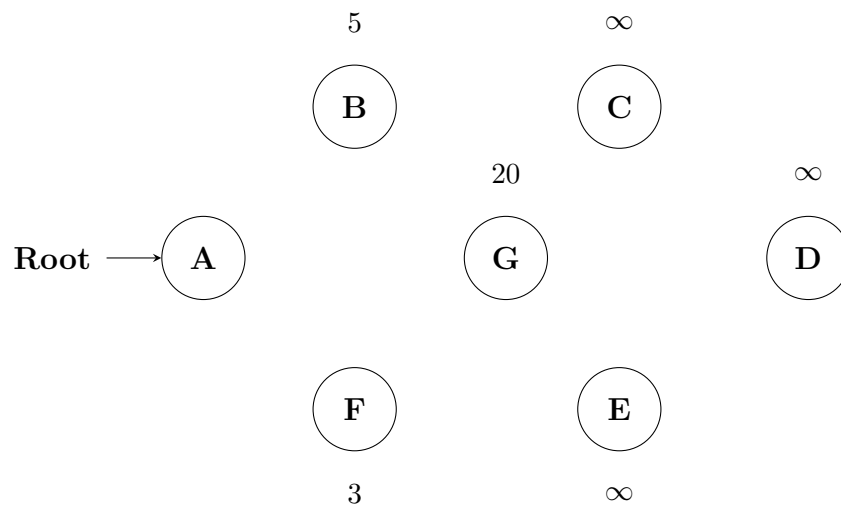
Optimal substructure: after choosing a_1 , the remaining problem is to select the maximum number of compatible activities from those with start time $\geq f_1$ — the same structure as the original problem on a smaller instance.

By induction, the greedy algorithm (always picking the earliest-finish compatible activity) builds an optimal solution. $\square$

- Q2.** Apply Prim's algorithm to find the minimum spanning tree of the given graph.



Start with node A as the root node. At each step, show the intermediate MST (i.e., the partial tree) and show the estimated cost to connect each non-visited node (i.e., the minimum weight of an edge connecting an unvisited node to any visited nodes). The first step, where only the root node is included in the tree, is done for you. **(10 marks)**
[CSE 105 | 2023–24]

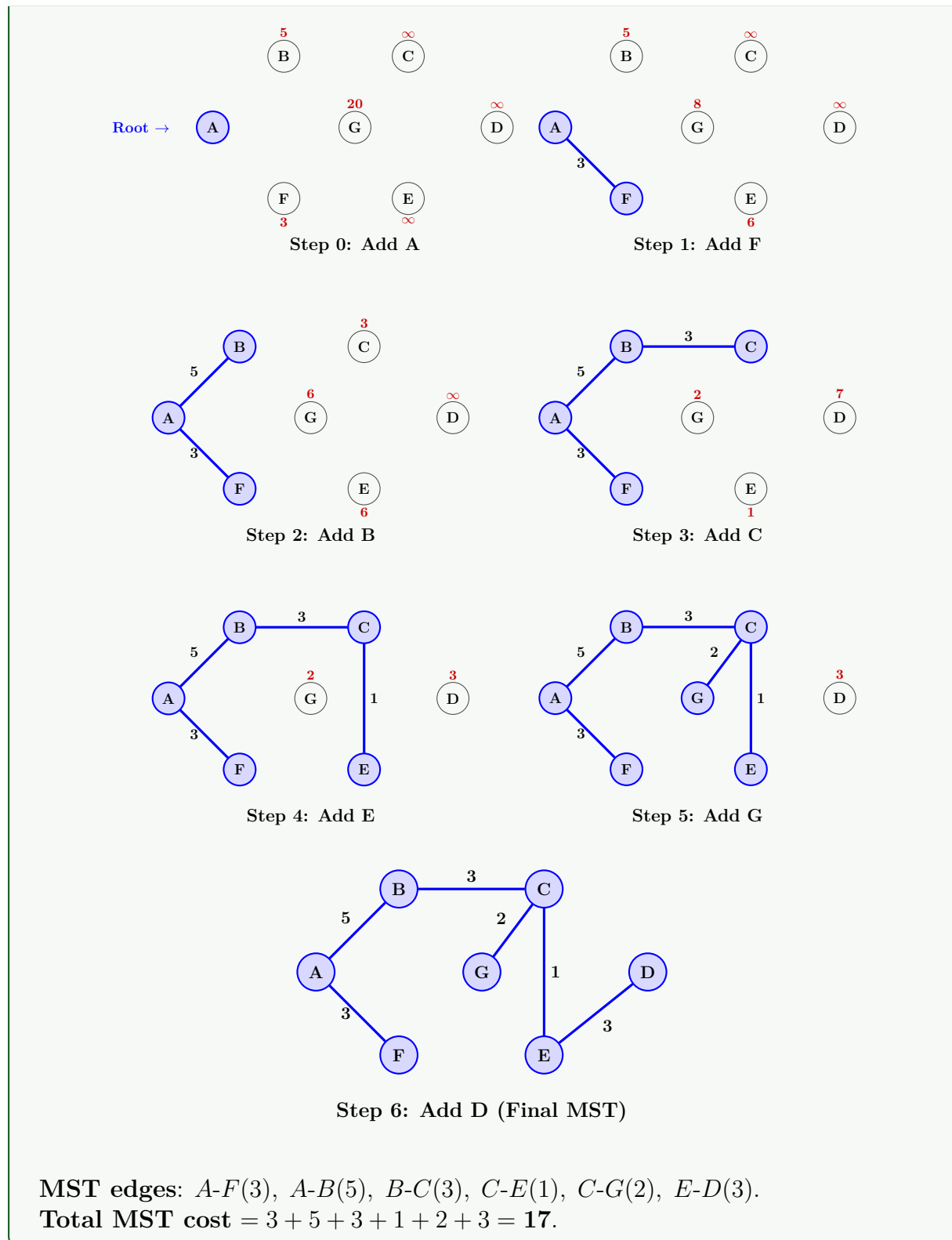


Answer

Prim's algorithm starting from A . At each step, we add the minimum-weight edge connecting a visited vertex to an unvisited vertex and update the estimated cost (key values) of adjacent unvisited nodes.

Step	Added	Edge	Via	Key values of unvisited vertices	MST cost
0	A	—	—	B:5, F:3, G:20, C: ∞ , D: ∞ , E: ∞	0
1	F	$A-F$	A	B:5, G:8, C: ∞ , D: ∞ , E:6	3
2	B	$A-B$	A	C:3, G:6 , D: ∞ , E:6	8
3	C	$B-C$	B	G:2, D:7, E:1	11
4	E	$C-E$	C	G:2, D:3	12
5	G	$C-G$	C	D:3	14
6	D	$E-D$	E	—	17

Intermediate Steps Visualization: (Blue shaded nodes are in the MST. Red values indicate the current minimum cost to connect that unvisited node).



Q3. Argue in favor of the correctness of Prim's algorithm to find a minimum spanning tree of a graph $G = (V, E)$. Prove any non-trivial graph properties used in the argument. (5+10=15 marks) [CSE 105 | 2022–23]

Answer

Prim's algorithm grows an MST one edge at a time by always adding the minimum-weight edge crossing the cut $(S, V \setminus S)$, where S is the current tree vertex set.

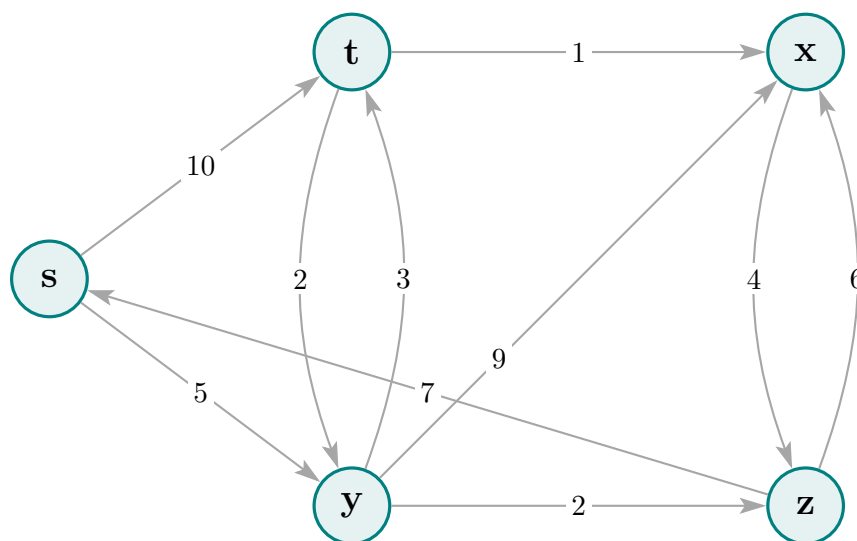
Correctness follows from the **Cut Property**:

Cut Property: for any cut $(S, V \setminus S)$ in a graph with unique edge weights, the minimum-weight crossing edge belongs to every MST.

Proof of Cut Property (by contradiction): Let $e^* = (u, v)$ be the min crossing edge and T an MST not containing e^* . Since T is connected, there is a path P from u to v in T ; this path crosses the cut at some edge $e' = (x, y)$ with $w(e') > w(e^*)$ (by uniqueness). $T' = T - \{e'\} + \{e^*\}$ is a spanning tree with $w(T') < w(T)$, contradicting minimality of T . $\square$

Prim's invariant: at each iteration, the set of edges chosen so far is a subset of some MST. Since Prim always picks the minimum crossing edge (satisfying the cut property), the final tree T is an MST. $\square$

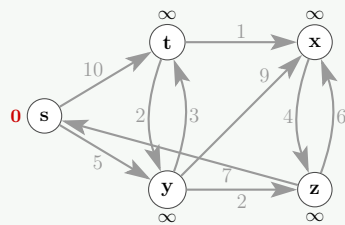
- Q4.** Find the shortest path distance from every node x to node t in the given graph using Dijkstra's Algorithm. Re-draw the diagram for each intermediate step of the algorithm (after relaxation) while annotating each node with the appropriate shortest path distance estimate of that step. (10 marks) [CSE 105 | 2022–23]

**Detailed Answer with Intermediate Steps**

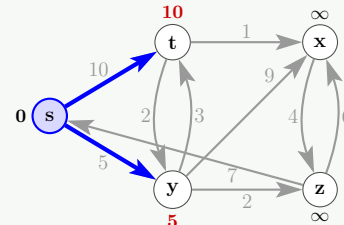
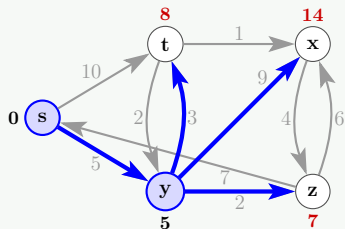
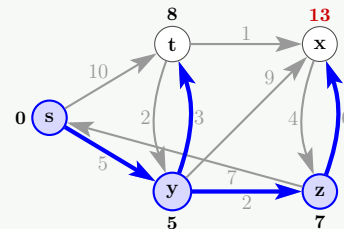
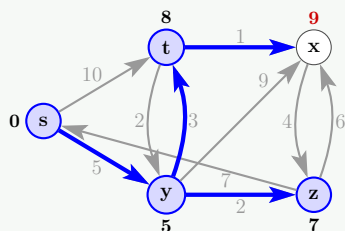
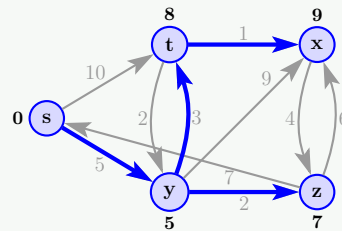
Dijkstra's algorithm from source s .

Step	Finalize	$d[s]$	$d[t]$	$d[x]$	$d[y]$	$d[z]$
Init	—	0	∞	∞	∞	∞
1	s	0	10	∞	5	∞
2	y	0	8	14	5	7
3	z	0	8	13	5	7
4	t	0	8	9	5	7
5	x	0	8	9	5	7

Intermediate Steps Visualization: (Blue nodes are finalized. Red labels show newly updated shortest path estimates. Thick blue lines show the predecessor/tree edges at that step.)



Step 0: Init

Step 1: Extract s Step 2: Extract y Step 3: Extract z Step 4: Extract t Step 5: Extract x (Final)

Final shortest distances from s : $d[s] = 0$, $d[y] = 5$, $d[z] = 7$, $d[t] = 8$, $d[x] = 9$.
Shortest paths trace:

- $s \rightarrow y: s \xrightarrow{5} y$ (dist 5)
- $s \rightarrow z: s \xrightarrow{5} y \xrightarrow{2} z$ (dist 7)
- $s \rightarrow t: s \xrightarrow{5} y \xrightarrow{3} t$ (dist 8)
- $s \rightarrow x: s \xrightarrow{5} y \xrightarrow{3} t \xrightarrow{1} x$ (dist 9)

Q5. Given a set $U = \{x_1, x_2, \dots, x_n\}$ of n integers and an integer k ($k < n$), give an optimal greedy algorithm to find a subset S of size k in U such that the summation of the numbers in S is maximum (over all subsets of size k in U). Prove the correctness of your algorithm and analyze its running time. **(15 marks)** [CSE 105 | 2021–22]

Answer

Algorithm: sort U in descending order; take the first k elements.

Algorithm 98 MAXSUMSUBSET(U, k)

- 1: Sort U in descending order: $x_{(1)} \geq x_{(2)} \geq \dots \geq x_{(n)}$
 - 2: **return** $S = \{x_{(1)}, x_{(2)}, \dots, x_{(k)}\}$
-

Correctness proof (exchange argument).

Let S^* be any optimal subset of size k with sum OPT. Let $S = \{x_{(1)}, \dots, x_{(k)}\}$ be the greedy solution.

If $S = S^*$, done. Otherwise, there exists $x_i \in S^* \setminus S$ and $x_j \in S \setminus S^*$ with $x_j \geq x_i$ (since all elements of S are at least as large as any element not in S). Swap: $S^{**} = (S^* \setminus \{x_i\}) \cup \{x_j\}$. $\text{sum}(S^{**}) = \text{OPT} - x_i + x_j \geq \text{OPT}$. Since OPT is maximum, $\text{sum}(S^{**}) = \text{OPT}$.

Repeat until $S^* = S$: the greedy solution achieves OPT. $\square$

Complexity Analysis. Sorting: $O(n \log n)$. Selecting k elements: $O(k)$. Total: $O(n \log n)$.

Q6. Consider a set of n intervals where each interval i specifies a start time s_i and a finish time f_i ($s_i < f_i$). Two intervals are compatible if they do not overlap. Give a greedy algorithm to select a compatible subset of maximum possible size. Prove the correctness and analyze the running time. **(16 marks)** [CSE 203 | 2021–22]

Answer

Algorithm (earliest finish time):

Algorithm 99 INTERVALSCHEDULING(S)

```

1: Sort intervals by finish time:  $f_1 \leq f_2 \leq \dots \leq f_n$ 
2:  $A \leftarrow \{1\}$ ; last  $\leftarrow f_1$ 
3: for  $i \leftarrow 2$  to  $n$  do
4:     if  $s_i \geq$  last then                                 $\triangleright$  compatible with last selected
5:          $A \leftarrow A \cup \{i\}$ ; last  $\leftarrow f_i$ 
6:     end if
7: end for
8: return  $A$ 
    
```

Correctness: proved via greedy-choice + optimal substructure (see [CSE 203 | 2020–21] answer above). $\square$

Complexity Analysis. Sorting: $O(n \log n)$. Single scan: $O(n)$. Total: $O(n \log n)$.

Q7. Given a set $S = \{x_1, x_2, \dots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \dots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all numbers in S . Prove correctness and analyze running time. **(15 marks)** [CSE 203 | 2021–22]

Answer

(See answer for [CSE 203 | 2017–18] — algorithm, correctness proof, and $O(n \log n)$ runtime analysis are identical.)

Q8. Once upon a time there was a city that had no proper roads. Getting around the city was particularly difficult after rainstorms because the ground used to become very muddy, cars got stuck in the mud, and people got their boots dirty. The mayor of the city decided that some of the streets must be paved, but he did not want to spend more money than necessary because the city also wanted to build a swimming pool. The mayor therefore specified two conditions:

- (i) Enough streets must be paved so that it is possible for everyone to travel from their house to anyone else's house only along the paved roads, and
- (ii) The paving should cost as little as possible.

The following figure shows the layout of the city. The number of paving stones in between two houses represents the cost of paving that route. Your task is to apply a greedy based algorithm that you have learned under this course to fulfill the requirement of the Mayor. Note that the bridge in the figure should not be considered as a paving stone. In your answer script, you must draw a simple graph representing the given layout of the muddy city. After that, you have to show the steps of applying your chosen greedy algorithm to find the connected subgraph which contains all the houses of the city (considering them as vertices of your drawn graph) and a subset of roads (considering them as edges) in a way that the total number

of paving stones required is the minimum. You have to ensure that your connected subgraph does not contain any cycle, and you need to mention the algorithm that you are adopting to find your desired solution. **(15 marks)** [CSE 203 | 2020–21]

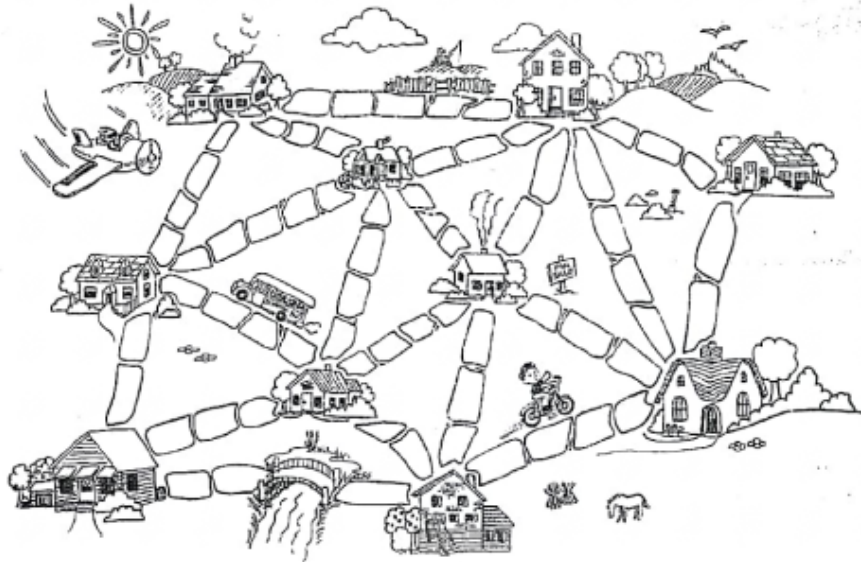
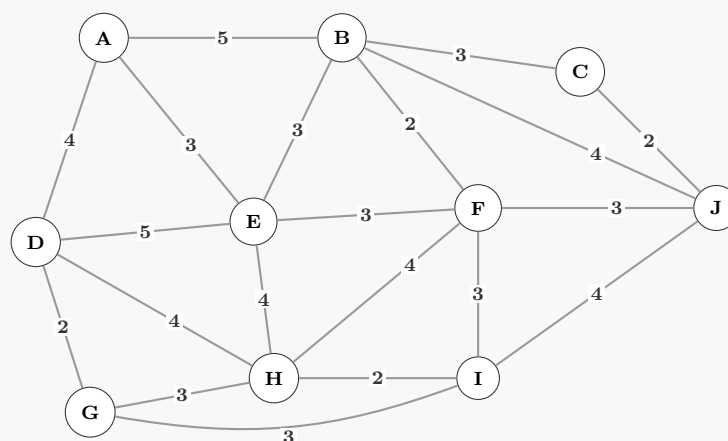


Figure for Ques No. 5(b) – The layout of the muddy city and the paving requirement

Detailed Answer: Minimum Spanning Tree for the Muddy City

1. Initial Graph Representation: First, we represent the city layout as a simple undirected graph $G = (V, E)$. Here is the complete layout of the city with all houses as vertices and available paths as edges with their respective paving costs (stones).



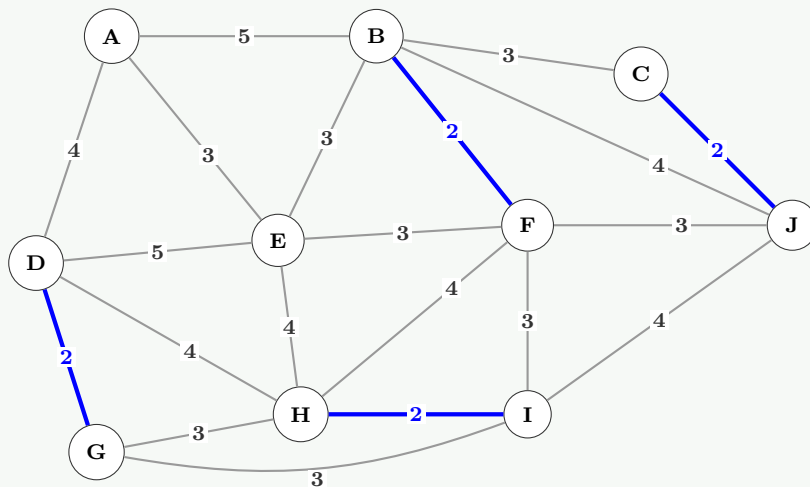
2. Chosen Algorithm & Edge Sorting: We will use **Kruskal's Algorithm**. The algorithm requires us to sort all the edges in ascending order of their weights and iteratively add them to the spanning tree, strictly ensuring that no **cycle** is formed.

- **Weight 2:** $(B, F), (C, J), (D, G), (H, I)$
- **Weight 3:** $(A, E), (B, C), (B, E), (E, F), (F, I), (F, J), (G, H), (G, I)$
- **Weight 4:** $(A, D), (B, J), (D, H), (E, H), (F, H), (I, J)$
- **Weight 5:** $(A, B), (D, E)$

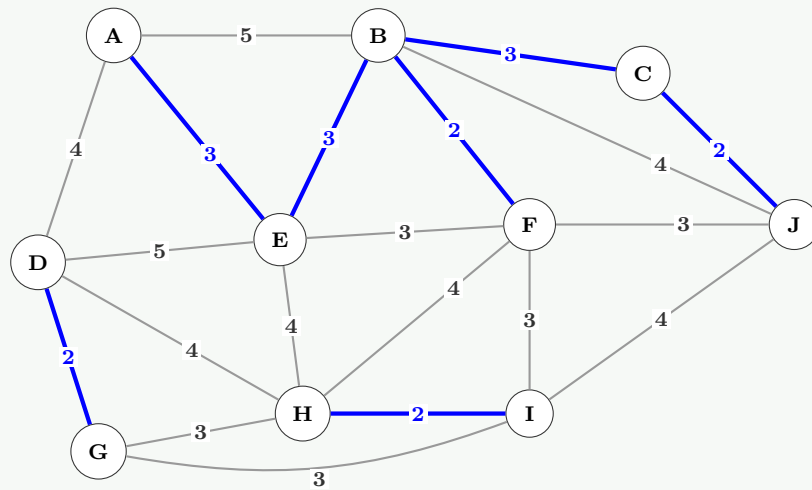
3. Kruskal's Step-by-Step Execution:

Edge	Weight	Action	Reason / Cycle Evaluation
(B, F)	2	Add	No cycle
(C, J)	2	Add	No cycle
(D, G)	2	Add	No cycle
(H, I)	2	Add	No cycle
(A, E)	3	Add	No cycle
(B, C)	3	Add	No cycle
(B, E)	3	Add	No cycle
(E, F)	3	Discard	Forms cycle $(B - E - F - B)$
(F, I)	3	Add	No cycle
(F, J)	3	Discard	Forms cycle $(B - C - J - F - B)$
(G, H)	3	Add	No cycle. MST Complete! (9 edges)

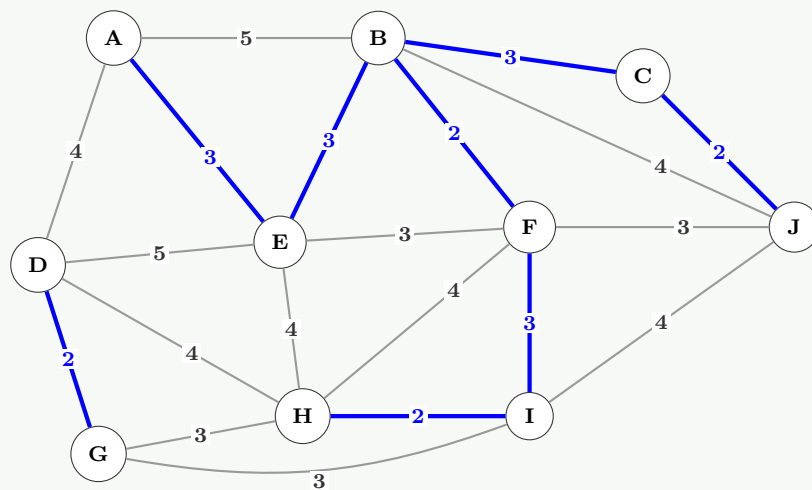
4. Intermediate & Final Visualizations:



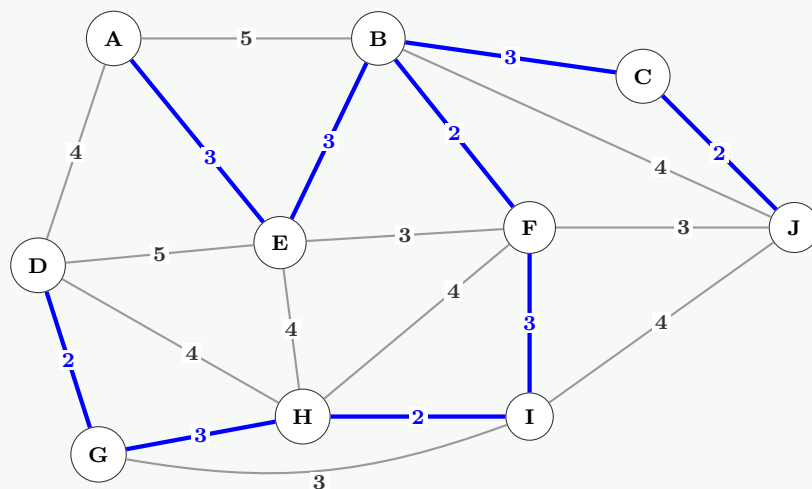
Step 1: Added Cost-2 Edges



Step 2: Added some Cost-3 Edges



Step 3: Added (F, I)



Final: Added (G, H) - MST Completed

Conclusion:

The minimal set of roads required to connect all 10 houses safely without forming any cycles is highlighted above.

Total Minimum Paving Cost = $2 + 2 + 2 + 2 + 3 + 3 + 3 + 3 + 3 = 23$ paving stones.

- Q9.** Prove that an optimal solution is always guaranteed if we design an activity-selection greedy algorithm by always choosing the activity with the earliest finish time from the remaining compatible activities. (10 marks) [CSE 203 | 2020–21]

Answer

Greedy-choice property.

Claim: there always exists an optimal solution that includes activity a_1 — the one with the smallest finish time.

Proof: let O be any optimal solution and a_k its earliest-finishing activity. If $a_k = a_1$, done. Otherwise $f_1 \leq f_k$. Replace a_k with a_1 in O : since $f_1 \leq f_k$, a_1 is compatible with every activity that a_k was compatible with, and the resulting set O' has the same size. O' is an optimal solution containing a_1 . $\square$

Optimal substructure. After choosing a_1 , we need to select a maximum compatible subset from $\{a_i : s_i \geq f_1\}$ — the same problem on a strictly smaller instance. By induction, the greedy algorithm (always pick the compatible activity with the earliest finish time) produces an optimal solution. $\square$

- Q10.** Analyze the runtime complexity of MST-Prim's algorithm (as given using a Binary Min-Heap to maintain a minimum priority queue), mentioning the individual runtime of each significant operation. How is this running time improved if a Fibonacci Heap is used instead? (10 marks) [CSE 203 | 2020–21]

Answer**Prim's with Binary Min-Heap:**

Operation	Times called	Cost each
BUILD-MIN-HEAP (initialise Q)	1	$O(V)$
EXTRACT-MIN	V	$O(\log V)$
DECREASE-KEY (relax neighbour)	$\leq E$	$O(\log V)$

Total: $O(V \log V + E \log V) = O(E \log V)$.

With Fibonacci Heap:

- EXTRACT-MIN: $O(\log V)$ amortised — called V times $\rightarrow O(V \log V)$.
- DECREASE-KEY: $O(1)$ amortised — called $\leq E$ times $\rightarrow O(E)$.

Total: $O(V \log V + E)$ — optimal for sparse graphs ($E = o(V^2 / \log V)$).

Q11. Write an algorithm to calculate the maximum profit earned by executing tasks within specified deadlines. A task takes one unit of time to execute and cannot execute beyond its deadline; only one task executes at a time. Apply the algorithm to the following data:

Task	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Deadline	7	3	2	6	8	5	6	3	2	4
Profit	9	14	5	10	3	15	5	7	8	6

(10+10 marks)

[CSE 203 | 2019–20]

Answer: Job Sequencing with Deadlines**Algorithm: Job Sequencing to Maximize Profit**

- Input:** A set of n tasks with their respective deadlines (d) and profits (p).
- Sort** all tasks in descending order of their profit.
- Find** the maximum deadline value among all tasks to determine the total number of available time slots, say D .
- Initialize** an array 'Slot[1...D]' to keep track of free time slots. Initially, all slots are free.
- Iterate** through the sorted tasks one by one:
 - For each task T_i with deadline d_i , find a free slot j such that $j \leq d_i$ and j is as large as possible (i.e., search backwards from d_i to 1).
 - If such a free slot j is found, assign task T_i to 'Slot[j]' and add its profit to the total profit.
 - If no such free slot is available, discard the task.

(f) **Output** the assigned tasks and the total maximum profit.

Applying the Algorithm to the Given Data:

Step 1: Sort tasks by profit in descending order

Task	T6	T2	T4	T1	T9	T8	T10	T3	T7	T5
Profit	15	14	10	9	8	7	6	5	5	3
Deadline	5	3	6	7	2	3	4	2	6	8

Step 2: Assign slots

Maximum deadline is $D = 8$. We have 8 time slots. We assign each task to the latest available free slot $\leq$ its deadline.

- **T6** (Profit=15, d=5): Slot 5 is free $\rightarrow$ Assign to Slot 5.
- **T2** (Profit=14, d=3): Slot 3 is free $\rightarrow$ Assign to Slot 3.
- **T4** (Profit=10, d=6): Slot 6 is free $\rightarrow$ Assign to Slot 6.
- **T1** (Profit=9, d=7): Slot 7 is free $\rightarrow$ Assign to Slot 7.
- **T9** (Profit=8, d=2): Slot 2 is free $\rightarrow$ Assign to Slot 2.
- **T8** (Profit=7, d=3): Slots 3 & 2 are taken. Slot 1 is free $\rightarrow$ Assign to Slot 1.
- **T10** (Profit=6, d=4): Slot 4 is free $\rightarrow$ Assign to Slot 4.
- **T3** (Profit=5, d=2): Slots 2 & 1 are already taken $\rightarrow$ Discard.
- **T7** (Profit=5, d=6): Slots 6, 5, 4, 3, 2, 1 are taken $\rightarrow$ Discard.
- **T5** (Profit=3, d=8): Slot 8 is free $\rightarrow$ Assign to Slot 8.

Final Task Schedule & Profit:

Slot	1	2	3	4	5	6	7	8
Task	T8	T9	T2	T10	T6	T4	T1	T5
Profit	7	8	14	6	15	10	9	3

Total Maximum Profit = $7 + 8 + 14 + 6 + 15 + 10 + 9 + 3 = \mathbf{72}$. ✓

Q12. Show that the fractional knapsack problem has the optimal substructure property. Also show that the 0-1 knapsack problem has the overlapping subproblems property.

(10 marks)

[CSE 203 | 2018–19]

Answer

Fractional Knapsack — Optimal Substructure.

Suppose an optimal solution OPT for capacity W takes fraction x_1 of item 1. Consider the remaining capacity $W' = W - x_1 w_1$ and the remaining items $\{2, \dots, n\}$. The fractions $(x_2, \dots, x_n)$ in OPT must form an optimal solution to the sub-problem on items $\{2, \dots, n\}$ with capacity W' . (If not, a better sub-solution would improve OPT , contradicting optimality.) $\square$

0-1 Knapsack — Overlapping Subproblems.

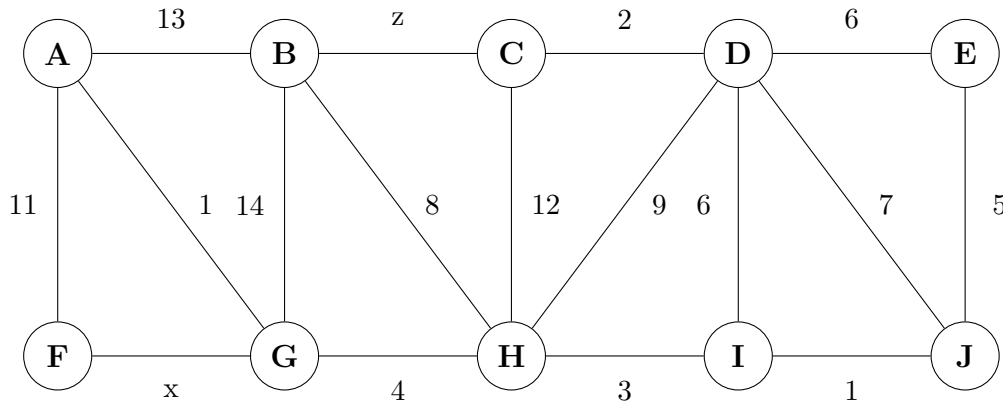
Define $dp[i][w]$ = maximum value using items $\{1, \dots, i\}$ with capacity w . The

recurrence is:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i).$$

Both cases refer to $dp[i-1][\cdot]$ — the same sub-problem $dp[i-1][w']$ may be needed many times by different (i, w) pairs. For example, $dp[2][5]$ uses $dp[1][5]$ and $dp[1][3]$; $dp[3][5]$ also uses $dp[2][5]$. This sharing of sub-problems demonstrates the overlapping subproblems property. $\square$

Q13. Prove that an optimal minimum spanning tree is composed of optimal minimum spanning subtrees. Draw a minimum spanning tree containing the edges with weights x and z . (15 marks) [CSE 207 | 2018–19]



Detailed Answer: Optimal Substructure & Drawing MST

Part 1: Proof of Optimal Substructure in MST

Theorem: An optimal Minimum Spanning Tree (MST) is composed of optimal minimum spanning subtrees.

Proof: Let $G = (V, E)$ be a connected, undirected graph with edge weights, and let T be an MST of G .

- Let $e = (u, v)$ be any arbitrary edge in T .
- Removing edge e from T divides the tree into two disconnected subtrees, say T_1 and T_2 .
- Let V_1 and V_2 be the sets of vertices spanned by T_1 and T_2 respectively. (Note that $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$).

We must prove that T_1 is an MST of the subgraph induced by V_1 (let's call it G_1), and T_2 is an MST of the subgraph induced by V_2 (G_2).

Proof by Contradiction:

- Assume that T_1 is **not** the optimal MST for the subgraph G_1 .
- This implies there exists another spanning tree for G_1 , say T_1^* , such that the total weight of T_1^* is strictly less than the total weight of T_1 . That is: $W(T_1^*) < W(T_1)$.

(c) Now, construct a new spanning tree T^* for the entire graph G by reconnecting T_1^* and T_2 using the original edge e . So, $T^* = T_1^* \cup \{e\} \cup T_2$.

(d) The total weight of this new spanning tree T^* would be:

$$W(T^*) = W(T_1^*) + W(e) + W(T_2)$$

(e) Since we assumed $W(T_1^*) < W(T_1)$, we can substitute this inequality:

$$W(T^*) < W(T_1) + W(e) + W(T_2)$$

(f) Since $W(T_1) + W(e) + W(T_2)$ is exactly the weight of our original MST $W(T)$, we get:

$$W(T^*) < W(T)$$

(g) This is a **contradiction!** We initially stated that T is the Minimum Spanning Tree of G , meaning no other spanning tree can have a lower weight.

(h) Therefore, our assumption must be false. T_1 must be an optimal MST for G_1 . (By symmetry, the same logic applies to T_2). ■

Part 2: Drawing the MST containing edges x and z

To guarantee that the edges (F, G) with weight x and (B, C) with weight z are included in the Minimum Spanning Tree, they must not form the heaviest edge in any cycle. Let's analyze the step-by-step construction using Kruskal's Algorithm:

Step 1: Identifying necessary conditions for x and z

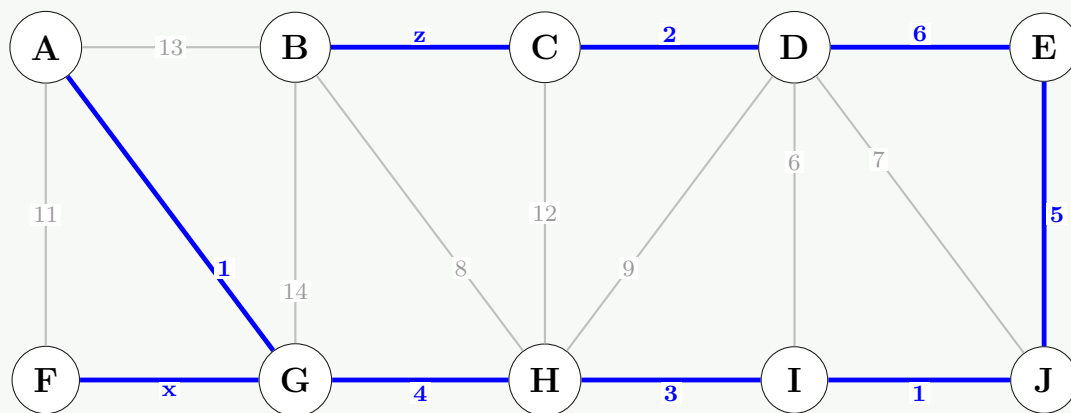
- **For edge x (F-G):** To connect vertex F to the rest of the graph, the only alternative path is via A (i.e., $F - A - G$). The edge (A, F) has a weight of 11. For Kruskal's algorithm to pick x over the alternative path, we must have $x \leq 11$.
- **For edge z (B-C):** To connect vertex B , the best alternative crossing edge to the rest of the graph is (B, H) with a weight of 8. Thus, to prioritize (B, C) , we must have $z \leq 8$.

Step 2: Selecting the edges (Assuming x and z are sufficiently small) We sort and pick the smallest edges avoiding cycles:

- Pick $(A, G) = 1$
- Pick $(I, J) = 1$
- Pick $(C, D) = 2$
- Pick $(H, I) = 3$
- Pick $(G, H) = 4$
- Pick $(E, J) = 5$
- Pick $(D, E) = 6$ (Note: picking $(D, I) = 6$ is also valid and creates an alternative valid MST)
- Pick $(F, G) = x$ (Given $x \leq 11$)

(i) Pick $(B, C) = z$ (Given $z \leq 8$)

Step 3: Visualizing the Final MST



Q14. Write down an algorithm for job sequencing with deadlines. Solve the problem instance with profit and deadline values (p_i, d_i) as follows: $(7, 5), (6, 5), (5, 2), (4, 7), (3, 7), (3, 7), (3, 6), (2, 7), (2, 1), (2, 5), (2, 3), (2, 1), (1, 1)$. **(10+10 marks)** [CSE 203 | 2017–18]

Answer: Job Sequencing with Deadlines

Algorithm (Greedy — sort by profit descending):

Algorithm 100 JOBSEQUENCING($jobs$)

```

1: Sort jobs by profit in descending order
2: Find the maximum deadline  $d_{\max}$  among all jobs
3: schedule[1.. $d_{\max}$ ]  $\leftarrow$  EMPTY
4: for each job  $(p_i, d_i)$  in sorted order do
5:   for  $t \leftarrow d_i$  downto 1 do
6:     if schedule[ $t$ ] = EMPTY then
7:       schedule[ $t$ ]  $\leftarrow i$ 
8:       break
9:   end if
10: end for
11: end for
12: return all scheduled jobs and their total profit

```

Solving for given instances (p_i, d_i) : $(7, 5), (6, 5), (5, 2), (4, 7), (3, 7), (3, 7), (3, 6), (2, 7), (2, 1), (2, 5), (2, 3), (2, 1), (1, 1)$

Step 1: Sort jobs by profit (descending):

Let's label them sequentially after sorting:

$J_1(7, 5), J_2(6, 5), J_3(5, 2), J_4(4, 7), J_5(3, 7), J_6(3, 7), J_7(3, 6), J_8(2, 7), J_9(2, 5), J_{10}(2, 3), J_{11}(2, 1), J_{12}(2, 1), J_{13}(1, 1)$

Step 2: Assign jobs to slots: Maximum deadline is 7, so we have 7 slots.

Job	Profit	Deadline	Assigned Slot	Reason / Action
J_1	7	5	5	Slot 5 is free.
J_2	6	5	4	Slot 5 taken, slot 4 is free.
J_3	5	2	2	Slot 2 is free.
J_4	4	7	7	Slot 7 is free.
J_5	3	7	6	Slot 7 taken, slot 6 is free.
J_6	3	7	3	Slots 7,6,5,4 taken, slot 3 is free.
J_7	3	6	1	Slots 6,5,4,3,2 taken, slot 1 is free.
$J_8 \dots J_{13}$	≤ 2	...	—	All slots (1 to 7) are full! Discard remaining.

Final Schedule (Slots 1-7):

Slot	1	2	3	4	5	6	7
Job	J_7	J_3	J_6	J_2	J_1	J_5	J_4
Profit	3	5	3	6	7	3	4

Total Maximum Profit = $3 + 5 + 3 + 6 + 7 + 3 + 4 = 31$. ✓

Q15. Give an optimal greedy algorithm for solving the fractional knapsack problem. Prove that your algorithm is optimal. **(13 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: sort items by value/weight ratio v_i/w_i descending; greedily take as much of each item as fits.

Algorithm 101 FRACTIONALKNAPSACK($items, W$)

```

1: Sort items by  $v_i/w_i$  descending
2: total  $\leftarrow 0$ ; remaining  $\leftarrow W$ 
3: for each item  $i$  in sorted order do
4:   if  $w_i \leq$  remaining then
5:     Take all of item  $i$ ; total  $+= v_i$ ; remaining  $-= w_i$ 
6:   else
7:     Take fraction remaining/ $w_i$  of item  $i$ 
8:     total  $+= v_i \cdot (\text{remaining}/w_i)$ ; break
9:   end if
10: end for
11: return total

```

Proof of optimality.

Let item a have the highest ratio $r_a = v_a/w_a$.

Greedy-choice property: there exists an optimal solution that takes as much of item a as possible. Suppose OPT takes less of a and more of some item b with $r_b \leq r_a$.

Replace δ kg of b with δ kg of a : profit changes by $\delta(r_a - r_b) \geq 0$. So the new solution is no worse — and we can assume OPT takes the maximum of a . $\square$

Optimal substructure: after filling with item a , the remaining problem is a fractional knapsack on the rest — same structure, smaller capacity.

By induction, the greedy algorithm is optimal. $\square$

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for selection. Total: $O(n \log n)$.

- Q16.** Given a set $S = \{x_1, x_2, \dots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \dots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all the numbers in S . For example, an interval $[1.5, 2.5]$ is a unit length interval (since $2.5 - 1.5 = 1$), and it contains all the real numbers from 1.5 to 2.5 (including 1.5 and 2.5). You have to find the smallest set of such unit length intervals which contains all the real numbers in S . Prove the correctness of your algorithm, and analyze the running time. **(15 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: for sorted input $x_1 \leq x_2 \leq \dots \leq x_n$, greedily start a new interval at each uncovered point.

Algorithm 102 MININTERVALS(S)

```

1: Sort  $S$ :  $x_1 \leq x_2 \leq \dots \leq x_n$ 
2: intervals  $\leftarrow \emptyset$ ; start  $\leftarrow x_1$ 
3: for  $i \leftarrow 2$  to  $n$  do
4:   if  $x_i > \text{start} + 1$  then                                 $\triangleright x_i$  not covered by current interval
5:     Add  $[\text{start}, \text{start} + 1]$  to intervals
6:     start  $\leftarrow x_i$ 
7:   end if
8: end for
9: Add  $[\text{start}, \text{start} + 1]$  to intervals
10: return intervals
    
```

Proof of optimality.

Claim: the greedy uses the minimum number of intervals.

Let k = number of greedy intervals. Each interval is *started* at a point x_i that was not covered by the previous interval. These k start points are mutually more than distance 1 apart, so no single unit-length interval can cover two of them. Hence any solution needs $\geq k$ intervals. The greedy uses exactly k . $\square$

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for the scan. Total: $O(n \log n)$.

- Q17.** Write down an algorithm for constructing a minimum spanning tree. Construct the

MST for the graph defined by the edges: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4), bx(2), yb(2)$, where the weight of each edge is given in parentheses. **(10+10 marks)**
[CSE 203 | 2017–18]

Answer: Kruskal's Algorithm & MST Construction

1. Algorithm for Constructing a Minimum Spanning Tree (Kruskal's Algorithm):

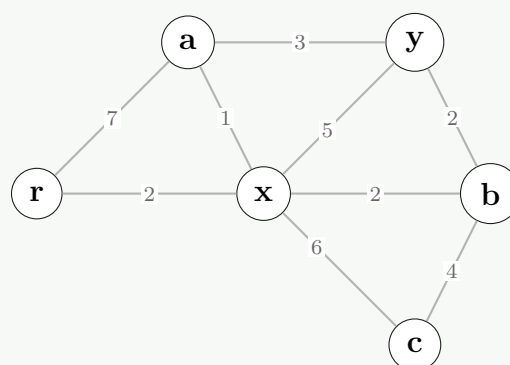
- Input:** A connected, undirected graph $G = (V, E)$ with weighted edges.
- Sort** all the edges of E in non-decreasing (ascending) order of their weights.
- Initialize** the MST T as an empty set. Initialize a disjoint-set (Union-Find) to keep track of vertices in different components.
- Iterate** through the sorted edges one by one:
 - For each edge (u, v) , check if adding it to T forms a cycle (i.e., check if u and v are already in the same component).
 - If it does **not** form a cycle, add the edge to T and take the union of the sets containing u and v .
 - If it forms a cycle, discard the edge.
- Stop** when T contains exactly $|V| - 1$ edges.
- Output:** The spanning tree T and its total minimum cost.

2. Solving the Problem Instance: Vertices (V) : $\{r, a, x, y, c, b\}$ (Total 6 vertices). We need $6 - 1 = 5$ edges for the MST.

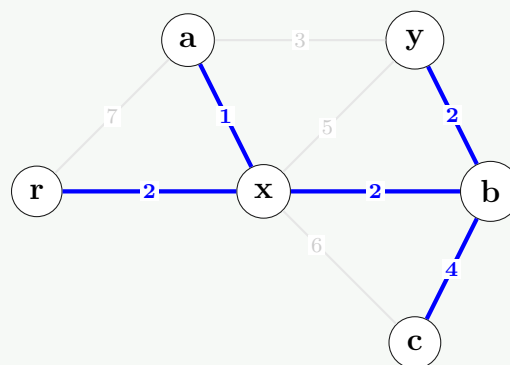
Sorted Edges: $(a, x, 1), (r, x, 2), (b, x, 2), (y, b, 2), (y, a, 3), (c, b, 4), (x, y, 5), (x, c, 6), (r, a, 7)$.

Edge	Weight	Add?	Reason	MST Edges So Far
(a, x)	1	Yes	No cycle	$\{ax\}$
(r, x)	2	Yes	No cycle	$\{ax, rx\}$
(b, x)	2	Yes	No cycle	$\{ax, rx, bx\}$
(y, b)	2	Yes	No cycle	$\{ax, rx, bx, yb\}$
(y, a)	3	No	Forms cycle $(y - b - x - a - y)$	—
(c, b)	4	Yes	No cycle. Done! ($ V - 1 = 5$)	$\{ax, rx, bx, yb, cb\}$

3. Visualizing the Graph and MST:



Initial Graph

**Final MST**

Final MST Edges: $ax(1), rx(2), bx(2), yb(2), cb(4)$.

Total Minimum Cost = $1 + 2 + 2 + 2 + 4 = 11$. ✓

Q18. What is a spanning tree of a graph? Write Kruskal's algorithm for finding a minimum spanning tree of an edge-weighted graph. Analyze the time complexity of the algorithm by suggesting appropriate data structures. **(3+5+7 marks)** [CSE 207 | 2017–18]

Answer

Spanning tree: a subgraph $T = (V, E')$ of a connected graph $G = (V, E)$ that is a tree (connected, acyclic) containing all vertices. A *minimum spanning tree* (MST) has the minimum total edge weight among all spanning trees.

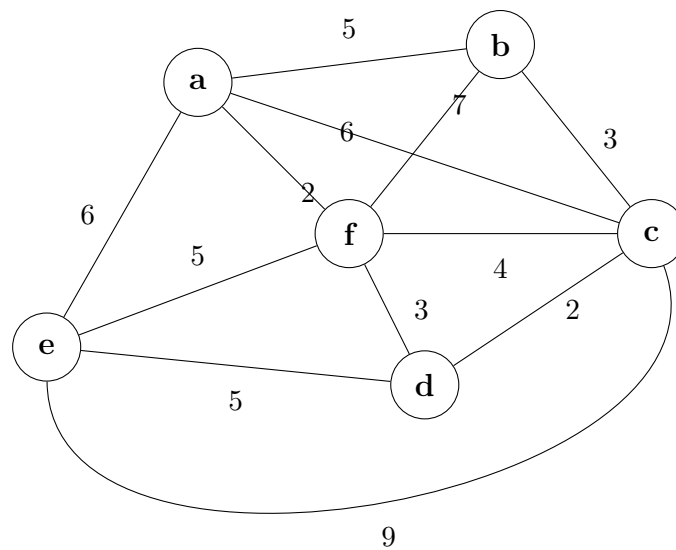
Kruskal's algorithm: (see pseudocode in [CSE 207 | 2006–07] answer above).

Complexity with appropriate data structures:

- Sort $|E|$ edges: $O(E \log E)$.
- MAKE-SET for each vertex: $O(V)$.
- $|E|$ FIND and $|V| - 1$ UNION operations with union-by-rank + path compression: $O(E \alpha(V))$.

Total: $O(E \log E) = O(E \log V)$.

Q19. Find a minimum spanning tree using Prim's algorithm, showing every step. **(10 marks)** [CSE 207 | 2017–18]



Detailed Answer: Prim's Algorithm Step-by-Step

Prim's Algorithm builds the Minimum Spanning Tree (MST) by starting from an arbitrary node and repeatedly adding the cheapest edge that connects a visited node to an unvisited node.

Let's define the nodes and edges. We will start the algorithm from node **a**.

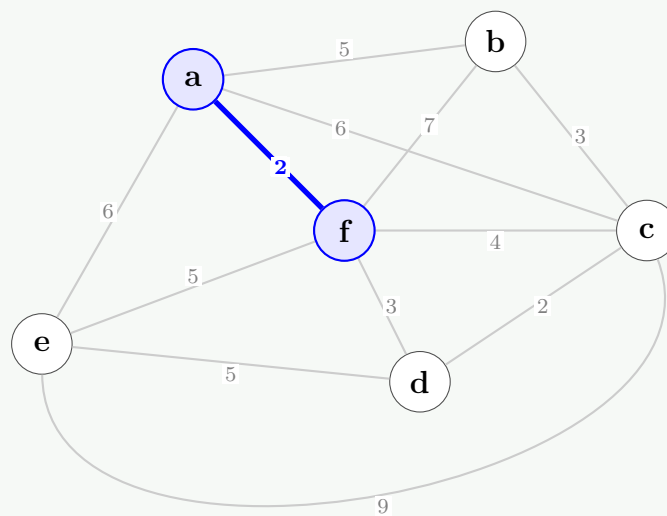
- **Visited Set (V_{vis}):** Keeps track of nodes already in the MST.
- **MST Edges (E_{mst}):** Keeps track of edges added to the MST.

Step 1: Initialization

Start at vertex **a**. $V_{vis} = \{a\}$.

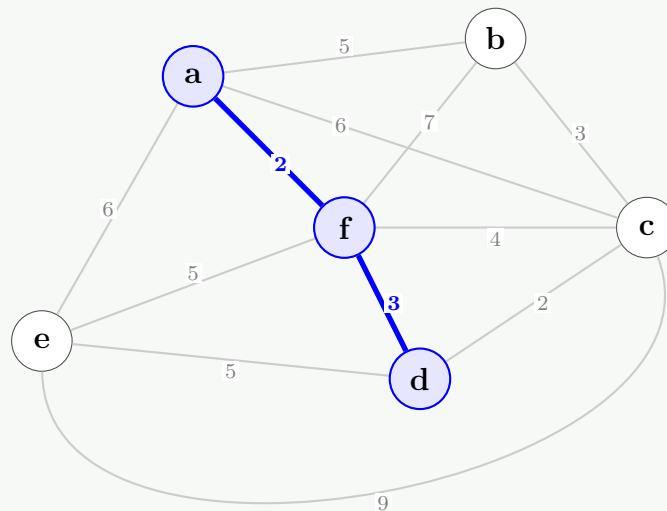
Available edges from visited nodes: $(a, f) = 2$, $(a, b) = 5$, $(a, e) = 6$, $(a, c) = 6$.

Action: Minimum weight is **2**. Add edge **(a, f)** and vertex **f**.


Step 2:
 $V_{vis} = \{a, f\}$.

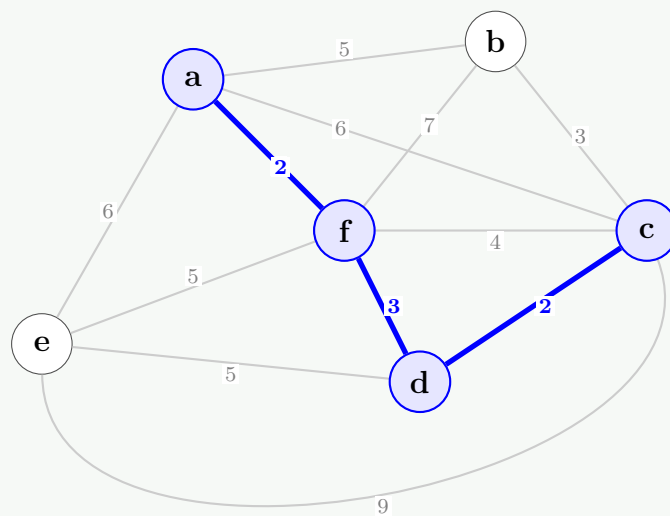
Available crossing edges: $(f, d) = 3, (f, c) = 4, (a, b) = 5, (e, f) = 5, (a, e) = 6, (a, c) = 6, (b, f) = 7$.

Action: Minimum weight is **3**. Add edge **(f, d)** and vertex **d**.


Step 3:
 $V_{vis} = \{a, f, d\}$.

Available crossing edges: $(c, d) = 2, (f, c) = 4, (a, b) = 5, (e, f) = 5, (e, d) = 5, \dots$

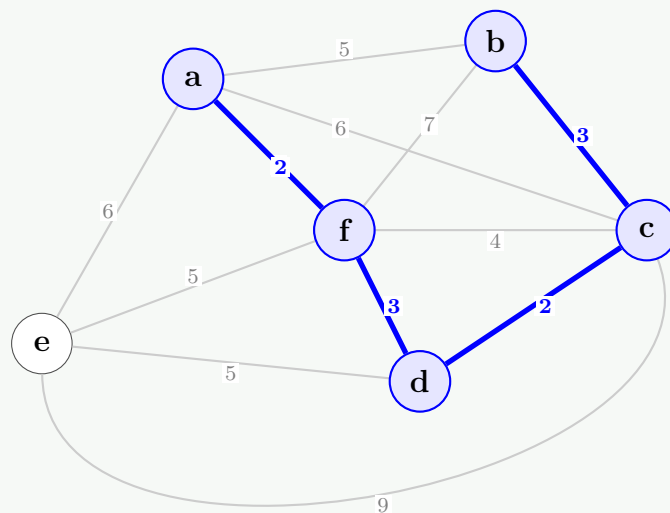
Action: Minimum weight is **2**. Add edge **(d, c)** and vertex **c**.

**Step 4:**

$V_{vis} = \{a, f, d, c\}$.

Available crossing edges: $(b, c) = 3, (a, b) = 5, (e, f) = 5, (e, d) = 5, (c, e) = 9, \dots$

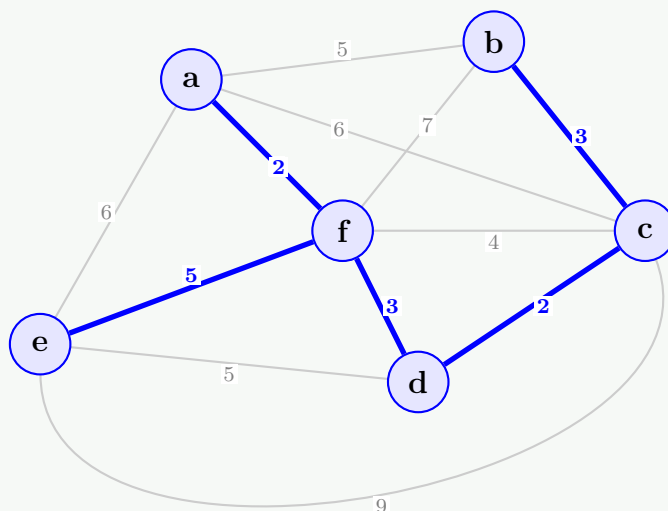
Action: Minimum weight is **3**. Add edge **(c, b)** and vertex **b**.

**Step 5:**

$V_{vis} = \{a, f, d, c, b\}$.

Available crossing edges to unvisited node 'e': $(e, f) = 5, (e, d) = 5, (a, e) = 6, (c, e) = 9$.

Action: Minimum weight is **5**. We can pick either (e, f) or (e, d) . Let's pick (e, f) . Add vertex **e**.



Conclusion: All vertices are now visited ($|V| - 1 = 5$ edges added).

Final MST Edges: $\{(a, f), (f, d), (d, c), (c, b), (e, f)\}$

Total MST Cost: $2 + 3 + 2 + 3 + 5 = 15$. ✓

Q20. Let G be a weighted connected graph and let V_1, V_2 be a partition of its vertices into two disjoint non-empty sets. Let e be an edge in G with minimum weight among those with one endpoint in V_1 and the other in V_2 . Show that there is a minimum spanning tree T of G containing e . **(10 marks)** [CSE 207 | 2017–18]

Answer

Theorem (Cut Property). Let G be a weighted connected graph, (V_1, V_2) a partition of vertices, and $e = (u, v)$ the minimum-weight edge with $u \in V_1, v \in V_2$. Then some MST of G contains e .

Proof (by exchange argument).

Let T be any MST of G . If $e \in T$, done.

Otherwise, since T is a spanning tree it is connected, so there is a path P from u to v in T . Since $u \in V_1$ and $v \in V_2$, this path must cross the cut at some edge $e' = (x, y)$ with $x \in V_1, y \in V_2$. Note $e' \neq e$.

Since e is the *minimum*-weight crossing edge, $w(e) \leq w(e')$.

Construct $T' = T - \{e'\} + \{e\}$:

- **Connected:** removing e' splits T into two components; adding e reconnects them (since e also crosses the cut).
- **Acyclic:** if a cycle existed, it would contain e ; but then e' would lie on a cycle

in T , contradicting T being a tree.

- **Weight:** $w(T') = w(T) - w(e') + w(e) \leq w(T)$.

Since T is an MST, $w(T') \geq w(T)$, so $w(T') = w(T)$ and T' is also an MST containing e . $\square$

Q21. Huffman's algorithm is used to obtain an encoding of alphabet $\{a, b, c\}$ with frequencies f_a , f_b , and f_c . In each of the following cases, either give an example of frequencies that would yield the specified code, or explain why the code cannot possibly be obtained:

- (a) Code: $\{0, 10, 11\}$
- (b) Code: $\{0, 1, 00\}$
- (c) Code: $\{10, 01, 00\}$

Then construct Huffman codes for the characters with the following frequencies: A:24, B:12, C:10, D:8, E:9, F:37, using the greedy algorithm. **(6+9 marks)** [CSE 203 | 2016–17]

Answer

Validity of given codes for $\{a, b, c\}$:

(a) $\{0, 10, 11\}$: Valid prefix-free code. Huffman tree: root $\rightarrow$ left = a ; root $\rightarrow$ right = internal node $\rightarrow$ left = b , right = c . This requires $f_a \geq f_b + f_c$ (so a gets the shorter code). *Example:* $f_a = 5, f_b = 2, f_c = 1$ $\checkmark$.

(b) $\{0, 1, 00\}$: **Invalid.** Code 0 is a prefix of code 00 — this violates the prefix-free property required for unambiguous decoding.

(c) $\{10, 01, 00\}$: **Invalid.** All codes have length 2 but there are only $2^1 = 2$ possible codewords of length 1 and $2^2 = 4$ of length 2. However, none of the three codes is a prefix of another — this *is* prefix-free. But a Huffman tree cannot produce three codes of equal length for 3 symbols: a complete binary tree for 3 leaves has depths 1, 2, 2 (not 2, 2, 2). **Cannot be obtained by Huffman coding.**

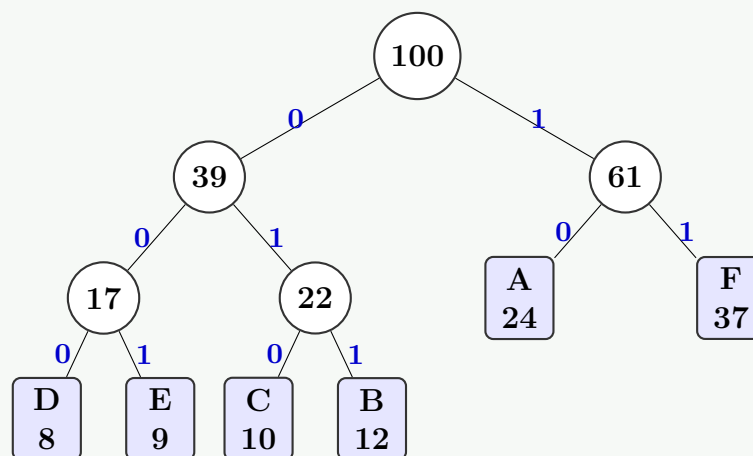
Huffman codes for A:24, B:12, C:10, D:8, E:9, F:37:

Build steps (merge two smallest frequencies):

- (a) Merge D(8) + E(9) = DE(17)
- (b) Merge C(10) + B(12) = CB(22)
- (c) Merge DE(17) + CB(22) = DECB(39)
- (d) Merge A(24) + F(37) = AF(61)
- (e) Merge DECB(39) + AF(61) = root(100)

Final Huffman codes:

Symbol	Frequency	Code	Bits used
F	37	11	$37 \times 2 = 74$
A	24	10	$24 \times 2 = 48$
B	12	011	$12 \times 3 = 36$
C	10	010	$10 \times 3 = 30$
E	9	001	$9 \times 3 = 27$
D	8	000	$8 \times 3 = 24$
Total bits:			239



Q22. Suppose you are given n white and n black dots, lying on a line. The dots are equally spaced but they may appear in any order of black and white (black and white may be interleaved). Design a greedy algorithm to connect each black dot with a (different) white dot so that the total length of wires used to connect the dots is minimal. The length of wire used to connect two dots is equal to their distance along the line. Prove that your algorithm is optimal.

●	●	○	●	○	○	○	●
1	2	3	4	5	6	7	8

For example, an optimal solution to solve the above example is (1, 3), (2, 5), (4, 6), (7, 8) with total wire length = $2 + 3 + 2 + 1 = 8$. **(15 marks)** [CSE 203 | 2016–17]

Answer

Greedy algorithm: sort black dots and white dots separately; match the i -th black with the i -th white (in sorted order).

Example: Blacks = $\{1, 2, 4, 8\}$, Whites = $\{3, 5, 6, 7\}$. Sorted matching: (1, 3), (2, 5), (4, 6), (8, 7). Total length = $2 + 3 + 2 + 1 = 8$. ✓

Proof of optimality (exchange argument).

Let $B = b_1 < b_2 < \dots < b_n$ and $W = w_1 < w_2 < \dots < w_n$ be the sorted positions. The greedy solution is $M_G = \{(b_i, w_i)\}$.

Suppose an optimal matching M^* contains a *crossing pair*: (b_i, w_j) and (b_k, w_l) with

$b_i < b_k$ but $w_j > w_l$ (i.e. the wires cross). Then:

$$|b_i - w_j| + |b_k - w_l| \geq |b_i - w_l| + |b_k - w_j|.$$

(This is because uncrossing two wires never increases total length — a standard geometric argument: for $a < b$ and $c > d$, $|a - c| + |b - d| \geq |a - d| + |b - c|$.)

Swapping to remove the crossing does not increase total cost, and repeated swaps yield the sorted matching M_G . Hence $\text{cost}(M_G) \leq \text{cost}(M^*)$, so M_G is optimal. $\square$

Complexity Analysis. Sorting: $O(n \log n)$. Matching: $O(n)$. Total: $O(n \log n)$.

Q23. Write Prim's algorithm that computes a minimum spanning tree of a graph. Analyze its time complexity. (15 marks) [CSE 207 | 2016–17]

Answer

Algorithm 103 PRIM(G, w, r)

```

1: for each  $u \in V$  do
2:    $\text{key}[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \text{NIL}$ 
3: end for
4:  $\text{key}[r] \leftarrow 0$ 
5:  $Q \leftarrow V$  ▷ min-priority queue keyed by  $\text{key}[]$ 
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8:   for each  $v \in \text{Adj}[u]$  do
9:     if  $v \in Q$  and  $w(u, v) < \text{key}[v]$  then
10:       $\pi[v] \leftarrow u$ ;  $\text{key}[v] \leftarrow w(u, v)$ 
11:       $\text{DECREASE-KEY}(Q, v, \text{key}[v])$ 
12:    end if
13:  end for
14: end while
    
```

Complexity Analysis. With binary heap: $O(E \log V)$. With Fibonacci heap: $O(V \log V + E)$.

Q24. Write the correctness proof of Kruskal's algorithm for finding a minimum spanning tree. (10 marks) [CSE 207 | 2016–17]

Answer

Theorem: Kruskal's algorithm produces an MST.

Proof: we show by induction that the edge set A maintained by Kruskal is always a subset of some MST.

Invariant: at each step, $A \subseteq T$ for some MST T .

Base: $A = \emptyset \subseteq T$. ✓

Step: suppose $A \subseteq T$ and Kruskal considers edge $e = (u, v)$ (next in sorted order).

Case 1: $e \in T$. Then $A \cup \{e\} \subseteq T$. ✓

Case 2: $e \notin T$. Adding e to T creates a cycle C . Since e connects two components of the current forest (not yet causing a cycle in A), there is an edge $e' \in C$ with $e' \notin A$ crossing the same cut. Since edges are processed in sorted order, $w(e) \leq w(e')$. $T' = T - \{e'\} + \{e\}$ is a spanning tree with $w(T') \leq w(T)$, so T' is also an MST.

$A \cup \{e\} \subseteq T'$. ✓

At termination, A has $|V| - 1$ edges and is acyclic $\rightarrow A$ is an MST. □

- Q25.** Compare the key properties of problems that can be solved with greedy method, divide-and-conquer method, and dynamic programming method. (10 marks) [CSE 207 | 2016–17]

Answer

Property	Greedy	Divide-and-Conquer	Dynamic Programming
Decision	Local optimum at each step	Split into independent sub-problems	Solve overlapping sub-problems once
Sub-problems	Not solved recursively	Non-overlapping; solved recursively	Overlapping; stored (memoised)
Key property	Greedy-choice + optimal substructure	Optimal substructure	Optimal substructure + overlapping subproblems
Efficiency	Often $O(n \log n)$	Typically $O(n \log n)$	Polynomial (table-based)
Examples	Activity selection, Huffman, Prim, Kruskal	Merge sort, binary search	Knapsack, LCS, Floyd-Warshall

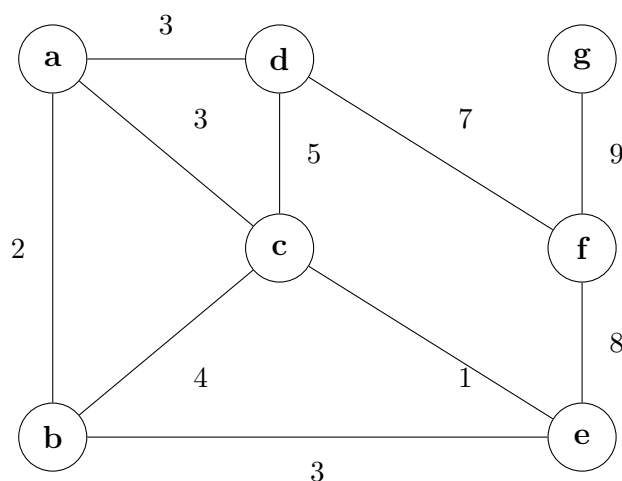
- Q26.** There is a network of roads $G = (V, E)$ connecting a set of cities V . Each road in E has an associated length l_e . There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an algorithm for solving this problem. Your algorithm should run in $O((V + E + E') \log V)$ time. (15 marks) [CSE 207 | 2015–16]

Answer**Algorithm:**

- (a) Run Dijkstra from s on $G \rightarrow d_s[v]$ for all v . Time: $O((V + E) \log V)$.
- (b) Run Dijkstra from t on G^T (reverse all edges) $\rightarrow d_t[v]$ for all v (= distance from v to t). Time: $O((V + E) \log V)$.
- (c) For each candidate road $e' = (a, b, \ell_{e'}) \in E'$: new s - t distance via $e' = \min(d_s[a] + \ell_{e'} + d_t[b], d_s[b] + \ell_{e'} + d_t[a])$. Time: $O(E')$.
- (d) Return the $e' \in E'$ that minimises the above.

Total: $O((V + E) \log V + E') = O((V + E + E') \log V)$. $\square$

Q27. Find a minimum spanning tree by running (i) Kruskal's algorithm and (ii) Prim's algorithm. Show the order of edges selected in each case (use alphabetical ordering when there is a tie). **(15 marks)** [CSE 207 | 2015–16]

**Detailed Answer: Kruskal's & Prim's Algorithm****Part 1: Kruskal's Algorithm**

Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).

Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

Part 1: Kruskal's Algorithm Step-by-Step

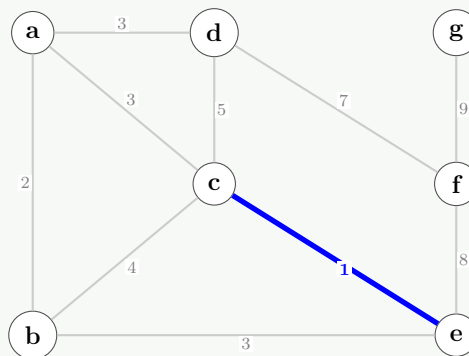
Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).

Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

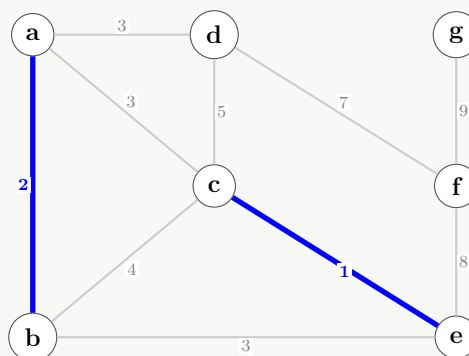
Edge	Weight	Add to MST?	Reason
$c-e$	1	Yes	No cycle
$a-b$	2	Yes	No cycle
$a-c$	3	Yes	No cycle
$a-d$	3	Yes	No cycle
$b-e$	3	No	Forms cycle ($b-a-c-e-b$)
$b-c$	4	No	Forms cycle ($b-a-c-b$)
$c-d$	5	No	Forms cycle ($c-a-d-c$)
$d-f$	7	Yes	No cycle
$e-f$	8	No	Forms cycle ($e-c-a-d-f-e$)
$f-g$	9	Yes	No cycle. Done ($ V - 1 = 6$ edges)

Visualizing Kruskal's Progress (Step-by-Step):

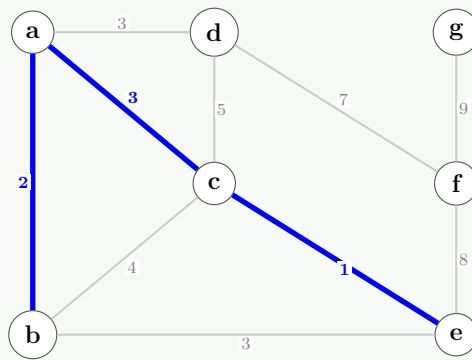
Step 1: Add edge $ce(1)$.



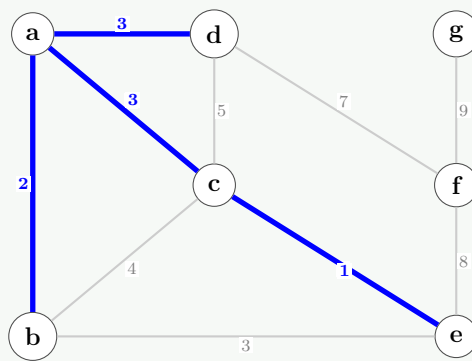
Step 2: Add edge $ab(2)$.



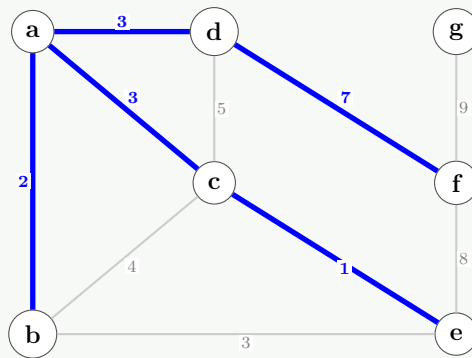
Step 3: Add edge $ac(3)$.



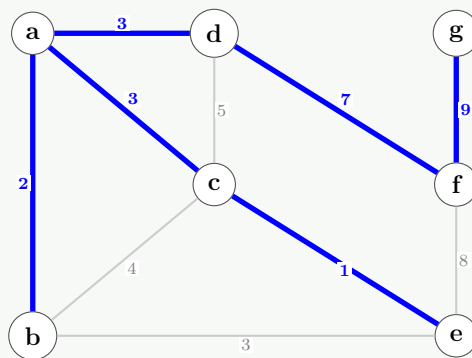
Step 4: Add edge **ad(3)**. (Edges *be*, *bc*, *cd* are discarded next as they form cycles).



Step 5: Add edge **df(7)**. (Edge *ef* is discarded next as it forms a cycle).



Step 6: Final MST. Add edge **fg(9)**.



Kruskal MST Cost: $1 + 2 + 3 + 3 + 7 + 9 = 25$.

Part 2: Prim's Algorithm (Using Key Values)

Start at vertex **a**. We maintain a **Key** value for each vertex (initialized to ∞ , meaning unreachable) and its **Parent**. In each step, we extract the unvisited vertex with the minimum key. Ties are broken alphabetically.

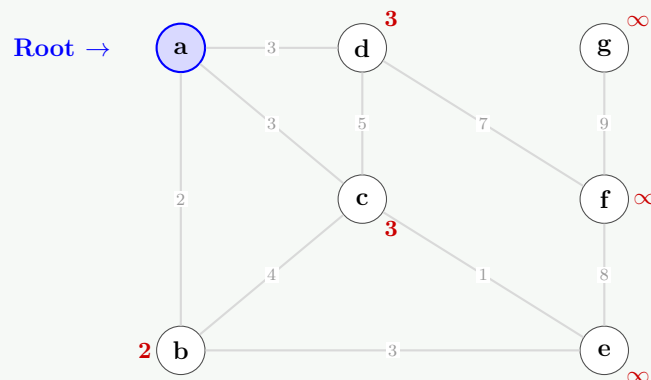
Key and Parent Tracking Table:

Notation: Key(Parent). '-' means the vertex is already included in the MST.

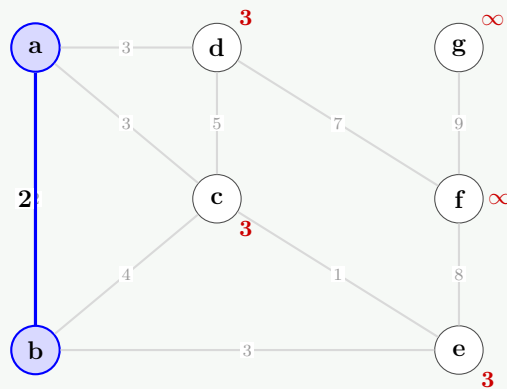
Step	Extracted Node	b	c	d	e	f	g
0	Initial State	∞	∞	∞	∞	∞	∞
1	a	2(a)	3(a)	3(a)	∞	∞	∞
2	b	-	3(a)	3(a)	3(b)	∞	∞
3	c	-	-	3(a)	1(c)	∞	∞
4	e	-	-	3(a)	-	8(e)	∞
5	d	-	-	-	-	7(d)	∞
6	f	-	-	-	-	-	9(f)
7	g	-	-	-	-	-	-

Intermediate Steps Visualization:

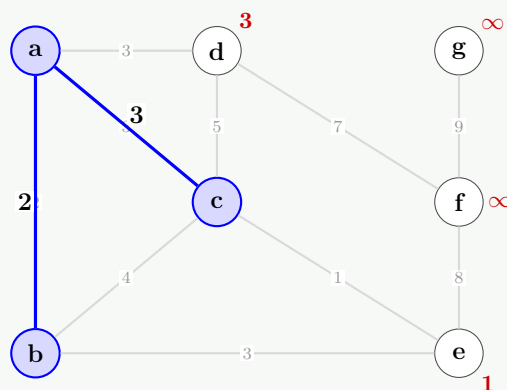
(Blue shaded nodes are in the MST. Red values indicate the current minimum cost to connect that unvisited node).



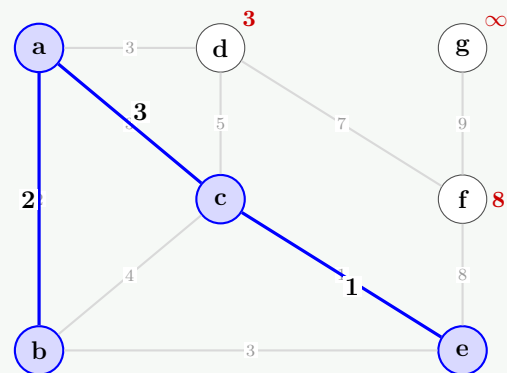
Step 0: Add a



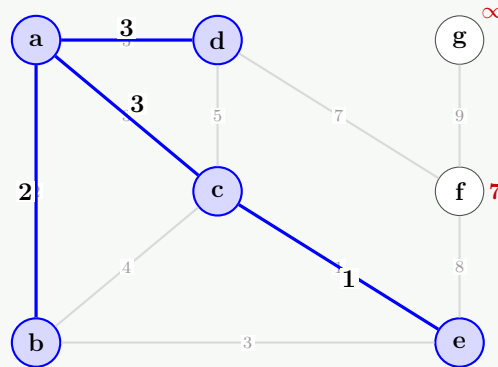
Step 1: Add b



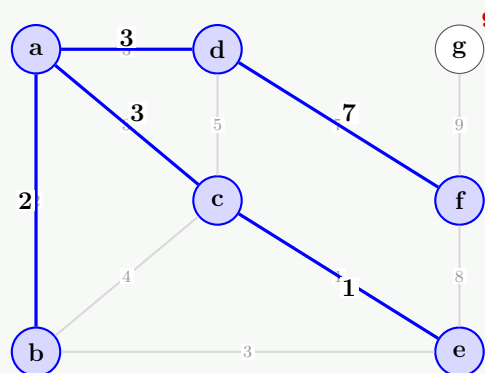
Step 2: Add c



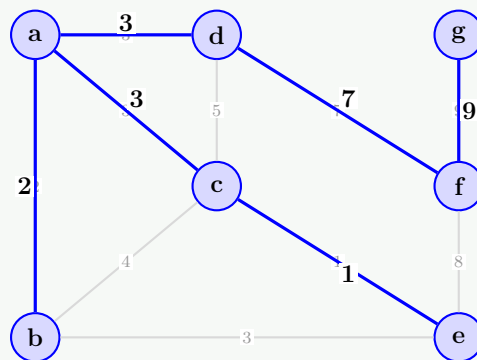
Step 3: Add e



Step 4: Add d



Step 5: Add f



Step 6: Add g (Final MST)

Prim MST edges: $ab(2)$, $ac(3)$, $ce(1)$, $ad(3)$, $df(7)$, $fg(9)$.
Total Cost = $2 + 3 + 1 + 3 + 7 + 9 = 25$.

Q28. Under a Huffman encoding of n symbols with frequencies $f_1, f_2, \dots, f_n$, what is the longest a codeword could possibly be? Give an example set of frequencies that would produce this case. Then construct Huffman codes for the following frequencies: A:45,

B:13, C:12, D:16, E:5, F:9, using the greedy algorithm. **(7+8=15 marks)** [CSE 207 | 2015–16]

Answer

Longest codeword in Huffman encoding.

In the worst case, the Huffman tree is maximally unbalanced (like a Fibonacci-weighted tree). For n symbols, the longest codeword has length $n - 1$.

Example: Fibonacci frequencies $f_1 = 1, f_2 = 1, f_3 = 2, f_4 = 3, f_5 = 5, \dots$ produce a completely skewed tree where the rarest symbol gets a codeword of length $n - 1$.

For $n = 5$ with $f = (1, 1, 2, 3, 5)$: longest codeword = 4.

Huffman codes for A:45, B:13, C:12, D:16, E:5, F:9:

Merge steps:

- (a) $E(5) + F(9) = EF(14)$
- (b) $C(12) + B(13) = CB(25)$
- (c) $EF(14) + D(16) = EFD(30)$
- (d) $CB(25) + EFD(30) = CBEFD(55)$
- (e) $A(45) + CBEFD(55) = \text{root}(100)$

Symbol	Freq	Code	Bits
A	45	0	$45 \times 1 = 45$
C	12	100	$12 \times 3 = 36$
B	13	101	$13 \times 3 = 39$
E	5	1100	$5 \times 4 = 20$
F	9	1101	$9 \times 4 = 36$
D	16	111	$16 \times 3 = 48$
Total:			224

Q29. Let $G = (V, E)$ be a connected undirected graph with weight function w on E . Let $A \subseteq E$ be included in some MST for G , let $(S, V - S)$ be any cut that respects A , and let (u, v) be a light edge crossing $(S, V - S)$. Prove that (u, v) is safe for A . **(10 marks)** [CSE 207 | 2014–15]

Answer

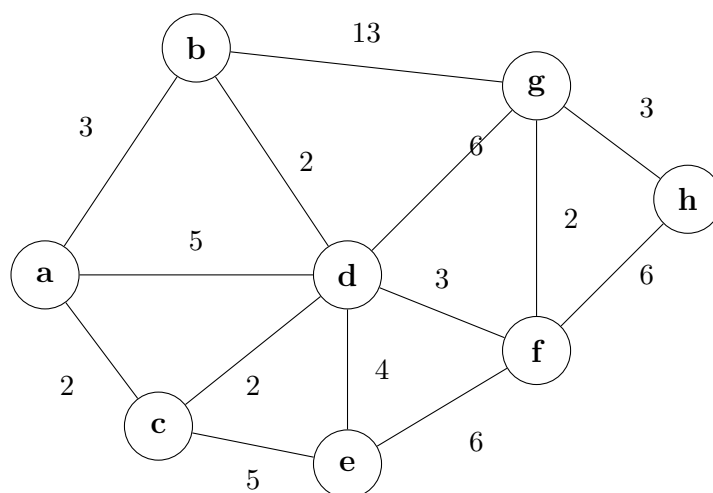
Theorem (Safe-Edge Lemma / Cut Property). Let $G = (V, E, w)$ connected, $A \subseteq E$ included in some MST, $(S, V \setminus S)$ a cut that *respects* A (no edge of A crosses it), and (u, v) a light crossing edge. Then (u, v) is safe for A : there exists an MST containing $A \cup \{(u, v)\}$.

Proof. Let T be any MST containing A . If $(u, v) \in T$, done. Otherwise, adding (u, v) to T creates a cycle C . Since $u \in S$ and $v \notin S$, the path in T from u to v must cross the cut; let (x, y) be any such crossing edge ($(x, y) \neq (u, v)$, and $(x, y) \notin A$

since the cut respects A).

$T' = T - \{(x, y)\} + \{(u, v)\}$ is a spanning tree. Since (u, v) is *light*: $w(u, v) \leq w(x, y)$, so $w(T') \leq w(T)$. Since T is an MST, $w(T') \geq w(T)$, hence $w(T') = w(T)$ and T' is an MST. T' contains A (since $(x, y) \notin A$) and (u, v) . Therefore (u, v) is safe for A . $\square$

Q30. Find the light and respectful crossing edges for each step of Kruskal's method for building the MST. (12 marks) [CSE 207 | 2014–15]



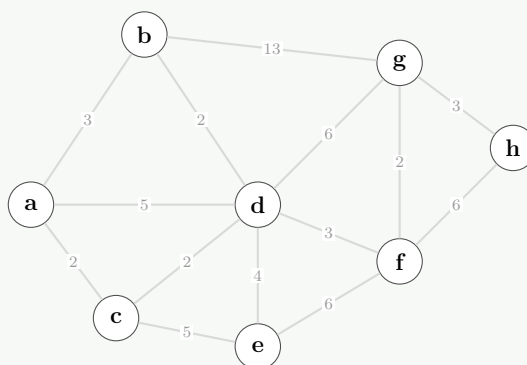
Answer: Kruskal's Algorithm (Step-by-Step)

Sorted Edges (Ascending Order):

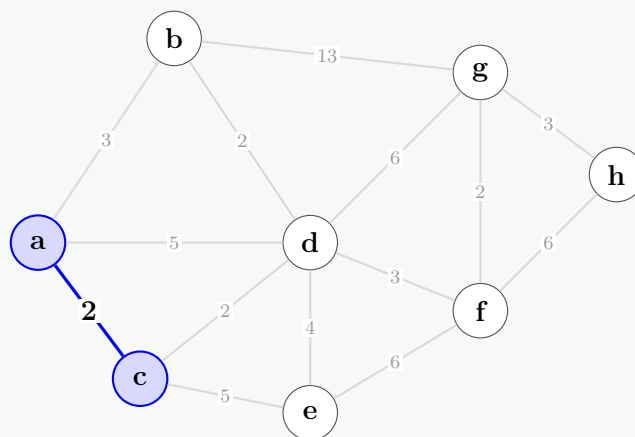
$ac(2), bd(2), cd(2), gf(2), ab(3), df(3), gh(3), de(4), ad(5), ce(5), dg(6), ef(6), hf(6), bg(13)$.

Intermediate Steps Visualization:

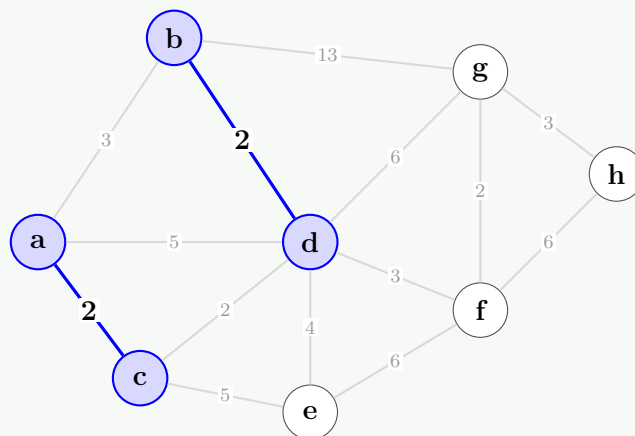
(Blue shaded nodes and thick blue lines indicate edges added to the MST forest. We add the smallest available edge that does not form a cycle).



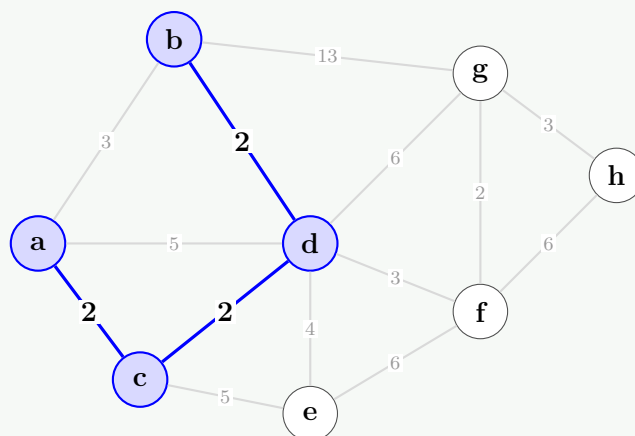
Step 0: Initial Graph



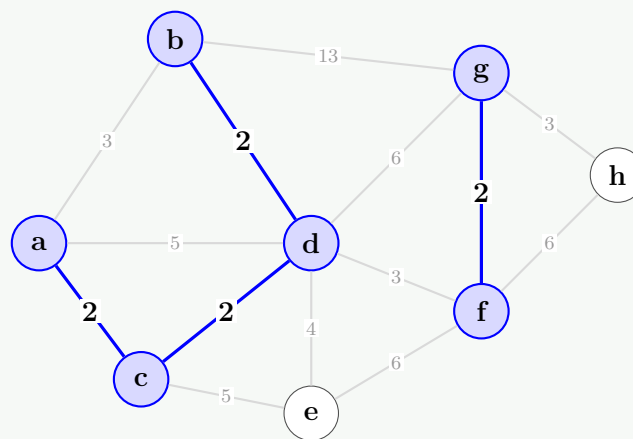
Step 1: Add $ac(2)$



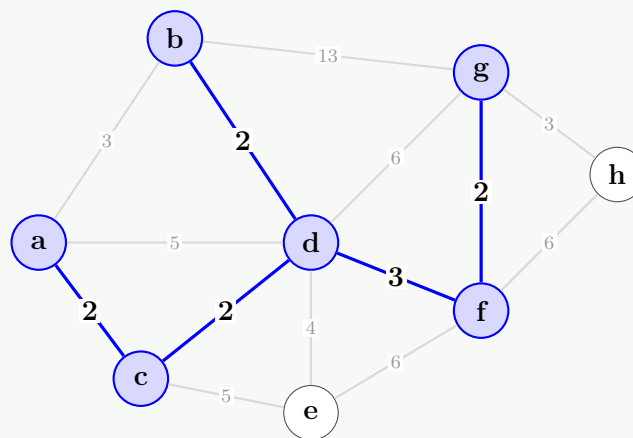
Step 2: Add $bd(2)$



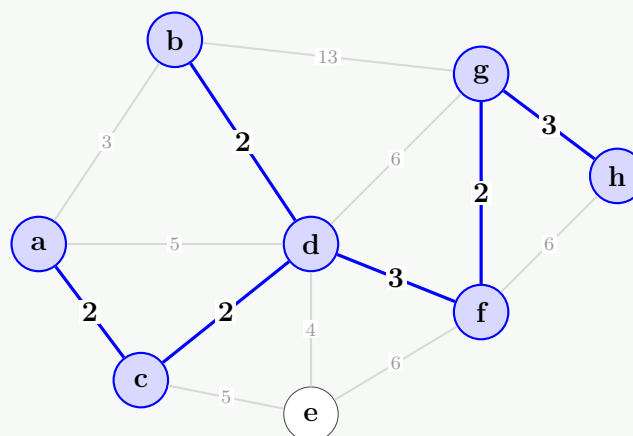
Step 3: Add $cd(2)$



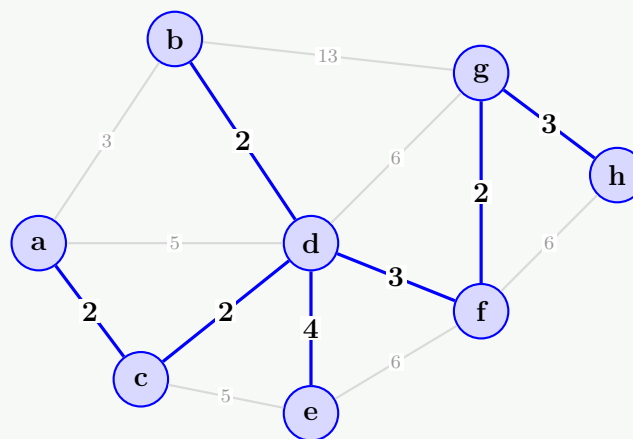
Step 4: Add $gf(2)$



Step 5: Skip $ab(3)$ [Cycle]. Add $df(3)$



Step 6: Add $gh(3)$



Step 7: Add de(4) (Final MST)

Kruskal MST Edges: $ac(2), bd(2), cd(2), gf(2), df(3), gh(3), de(4)$.
Total Cost = $2 + 2 + 2 + 2 + 3 + 3 + 4 = 18$.

Q31. What is the cost of finding and joining (union) of sets in Kruskal's method? (8 marks) [CSE 207 | 2014–15]

Answer

With **union by rank** and **path compression**:

- MAKE-SET: $O(1)$ each, V times $\rightarrow O(V)$.
- FIND (with path compression): $O(\alpha(V))$ amortised, where α is the inverse Ackermann function (practically constant).
- UNION (by rank): $O(\alpha(V))$ amortised.
- Total for E Find+Union operations: $O(E \alpha(V))$.

Combined with sorting edges ($O(E \log E)$), total Kruskal cost:

$$O(E \log E + E \alpha(V)) = O(E \log V).$$

(Since $E \leq V^2$, $\log E \leq 2 \log V$.)

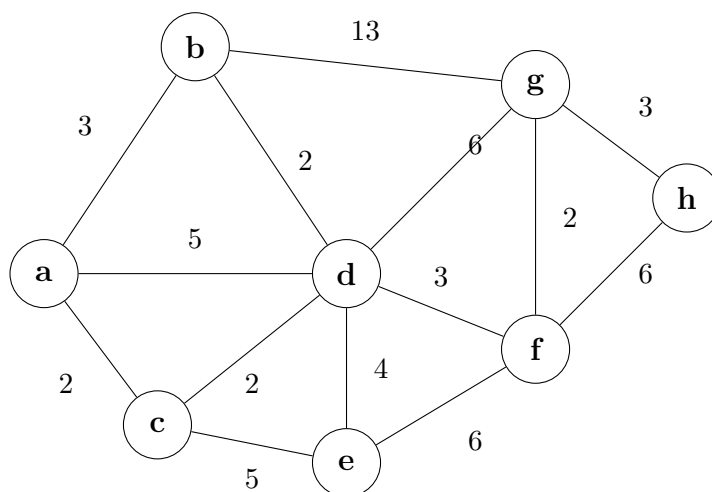
Q32. For the graph below, show the value of u , $v.key$, Q for the following algorithm (show each step): (10 marks) [CSE 207 | 2014–15]

Algorithm 104 $\text{MST}(G, w, r)$

```

1:  $Q \leftarrow V[G]$ 
2: for each  $u \in Q$  do
3:    $u.\text{key} \leftarrow \infty$ 
4: end for
5:  $r.\text{key} \leftarrow 0$ ;  $p[r] \leftarrow \text{nil}$ 
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8:   for each  $v \in \text{Adj}[u]$  do
9:     if  $v \in Q$  and  $w(u, v) < v.\text{key}$  then
10:       $p[v] \leftarrow u$ 
11:       $v.\text{key} \leftarrow w(u, v)$ 
12:    end if
13:   end for
14: end while

```

**Answer: Prim's Algorithm (Step-by-Step Visualization)**

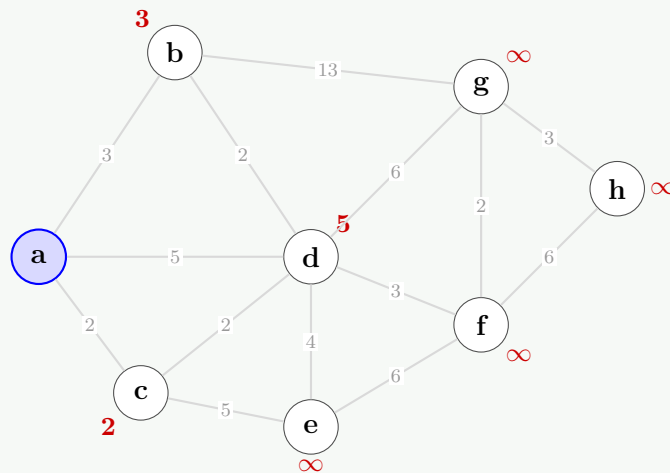
Starting from a . At each step: u = vertex extracted, $v.\text{key}$ updated.

Step	u extracted	Edge added	Key values after
1	a (key=0)	—	$b:3, c:2, d:5$
2	c (key=2)	$a-c$	$b:3, d:2, e:5$
3	d (key=2)	$c-d$	$b:2, e:4, f:3, g:6$
4	b (key=2)	$d-b$	$e:4, f:3, g:6$
5	f (key=3)	$d-f$	$e:4, g:2, h:6$
6	g (key=2)	$f-g$	$e:4, h:3$
7	h (key=3)	$g-h$	$e:4$
8	e (key=4)	$d-e$	—

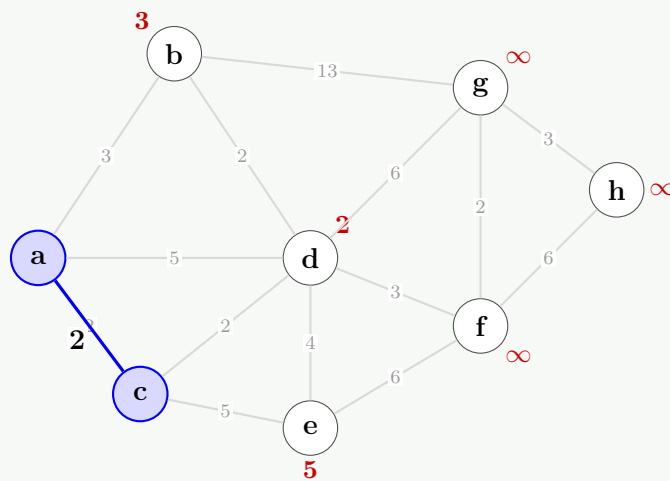
Intermediate Steps Visualization:

(Blue shaded nodes are currently in the MST. Red values indicate the current $v.\text{key}$)

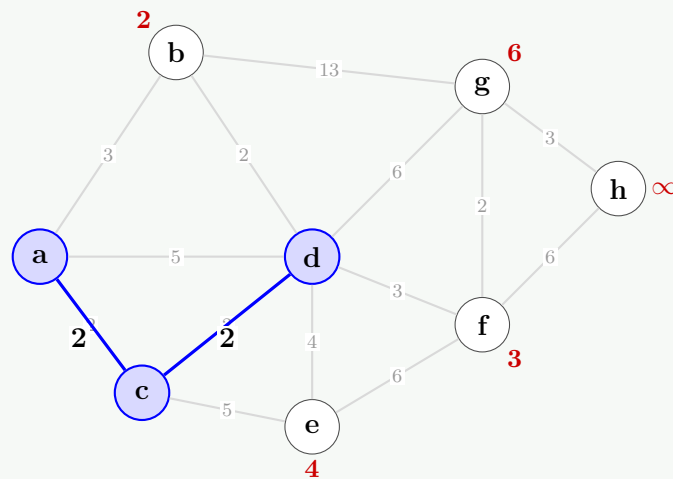
to connect that unvisited node to the MST).



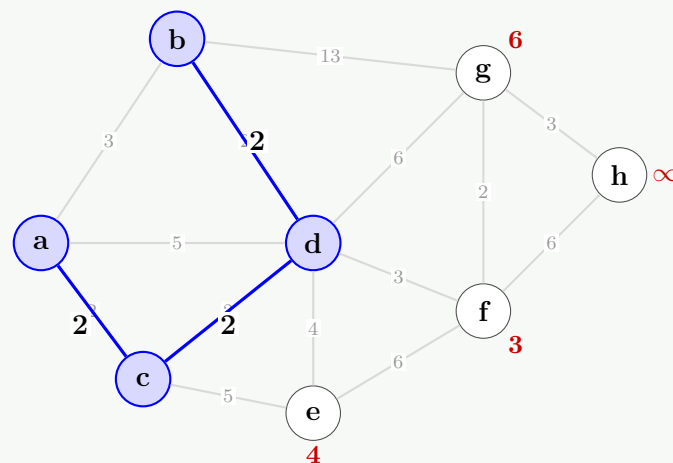
Step 1: Extract a (key=0). Update neighbors.



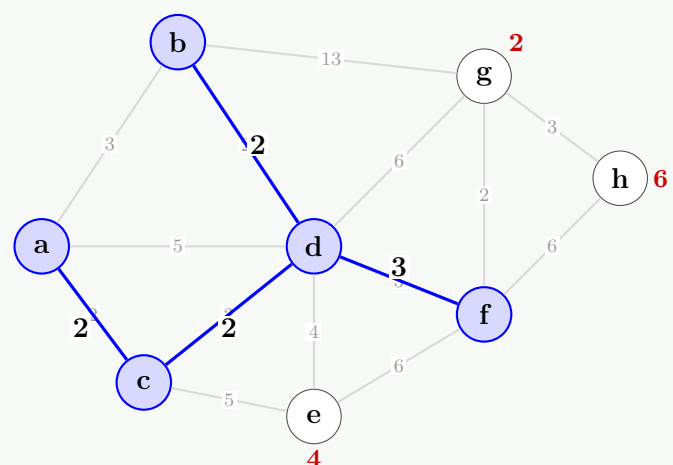
Step 2: Extract c (key=2). Add ac(2).



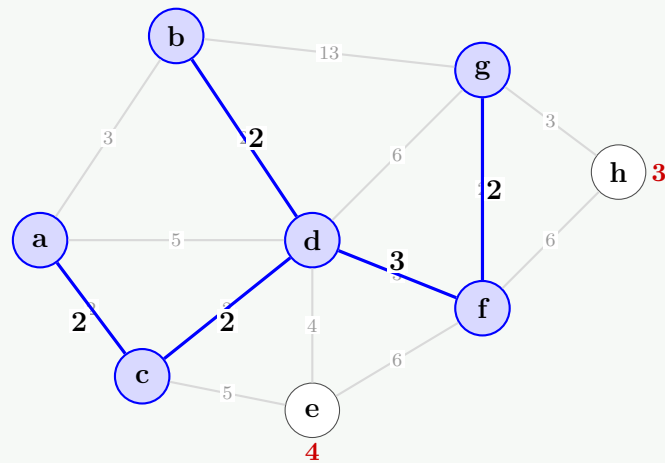
Step 3: Extract d (key=2). Add cd(2).



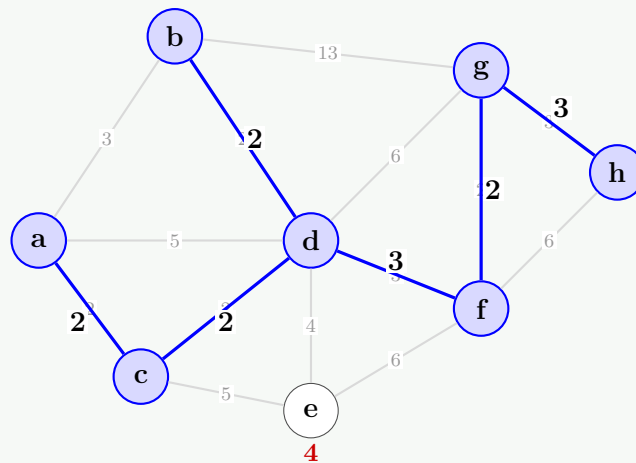
Step 4: Extract b (key=2). Add db(2).



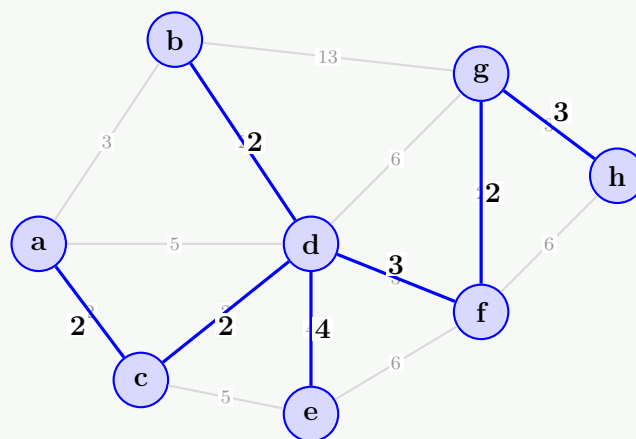
Step 5: Extract f (key=3). Add df(3).



Step 6: Extract g (key=2). Add fg(2).



Step 7: Extract h (key=3). Add gh(3).



Step 8: Extract e (key=4). Add de(4). (Final MST)

MST edges: $ac(2), cd(2), db(2), df(3), fg(2), gh(3), de(4)$.

Total cost = $2 + 2 + 2 + 3 + 2 + 3 + 4 = 18$. ✓

- Q33.** *GreedyCell*, a mobile phone operator, wants to place cell towers (base stations) along a long straight country road, to provide network coverage to n houses sparsely scattered along the road. The distance of these houses from the start of the road are, in miles and in increasing order, $d_1, d_2, \dots, d_n$. Each cell tower has coverage of R miles, i.e., if a house is within R miles of a tower, it would get service from that tower. Now give an efficient algorithm for finding a placement of cell towers that uses as few cell towers as possible to provide coverage to all the n houses. **(12 marks)** [CSE 207 | 2014–15]

Answer

Greedy algorithm: place each tower to cover as many uncovered houses as possible.

Algorithm 105 CELLTOWERS($d_1 \leq \dots \leq d_n, R$)

```

1:  $i \leftarrow 1$ ; towers  $\leftarrow 0$ 
2: while  $i \leq n$  do
3:   Place tower at position  $p = d_i + R$  ▷ covers  $[d_i, d_i + 2R]$ 
4:   towers  $\leftarrow$  towers + 1
5:   Advance  $i$  past all houses with  $d_i \leq p + R$ 
6: end while
7: return towers
    
```

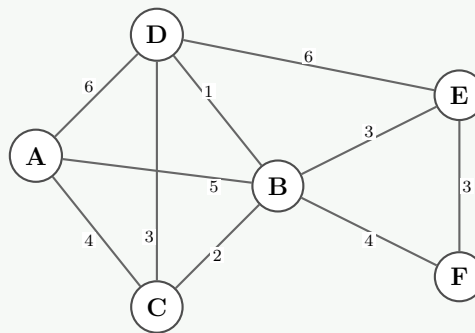
Correctness: house d_i is uncovered; place the tower as far right as possible while still covering d_i (at $d_i + R$). This maximises coverage of future houses. Any algorithm needs ≥ 1 tower for d_i ; our placement covers strictly more future houses than placing it closer to d_i . By exchange argument, the greedy is optimal. $\square$

Complexity Analysis. $O(n)$ after the sorted input is given (single pass).

- Q34.** Find a minimum spanning tree using both Kruskal's algorithm and Prim's algorithm (starting from A) for the graph with edges: $AB(5), BC(2), AC(4), AD(6), CD(3), DB(1), DE(6), EB(3), EF(3), FB(4)$. **(15 marks)** [CSE 207 | 2013–14]

Answer: Minimum Spanning Tree (Kruskal & Prim)

1. Original Graph:

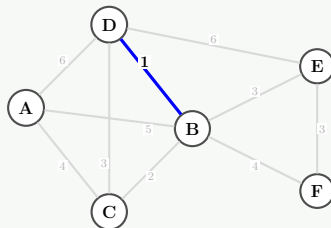


Part A: Kruskal's Algorithm

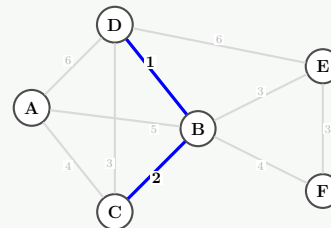
Sorted Edges: $DB(1), BC(2), CD(3), EB(3), EF(3), AC(4), FB(4), AB(5), AD(6), DE(6)$.

Edge	Weight	Add to MST?	Reason
$D-B$	1	Yes	Does not form a cycle
$B-C$	2	Yes	Does not form a cycle
$C-D$	3	No	Forms cycle ($B-C-D$)
$E-B$	3	Yes	Does not form a cycle
$E-F$	3	Yes	Does not form a cycle
$A-C$	4	Yes	Connects node A
$F-B$	4	No	Forms cycle ($E-F-B$)
$A-B$	5	No	Forms cycle ($A-C-B$)

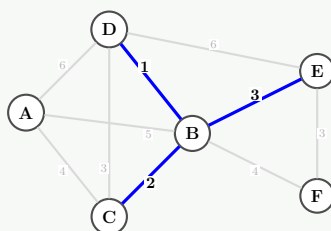
Kruskal's Step-by-Step:



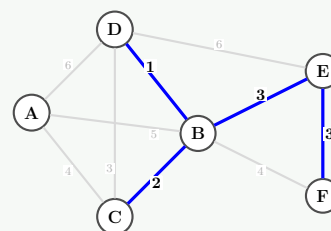
Step 1: Add $DB(1)$



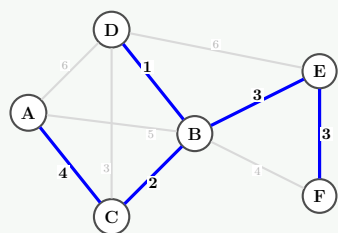
Step 2: Add $BC(2)$



Step 3: Skip CD . Add $EB(3)$



Step 4: Add $EF(3)$



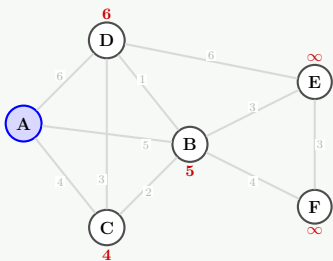
Step 5: Add AC(4). (Final MST)

Kruskal MST Edges: $DB(1), BC(2), EB(3), EF(3), AC(4)$.
Total Cost = $1 + 2 + 3 + 3 + 4 = 13$.

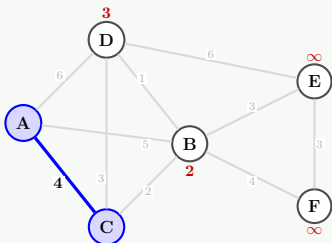
Part B: Prim’s Algorithm (Starting from A)

Step	Vertex Extracted	Edge Added	Key Updates (Neighbor : Cost)
1	A	—	C : 4, B : 5, D : 6
2	C	A–C (4)	B : 2, D : 3
3	B	C–B (2)	D : 1, E : 3, F : 4
4	D	B–D (1)	E : 3 (no change)
5	E	B–E (3)	F : 3
6	F	E–F (3)	—

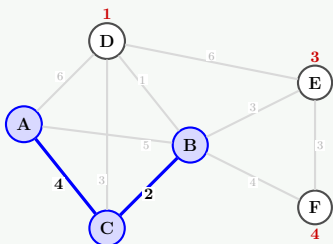
Prim’s Step-by-Step:



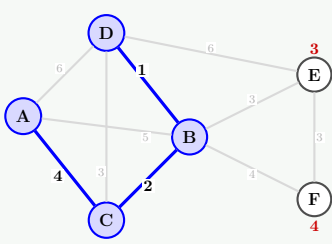
Step 1: Extract A. Update C,B,D



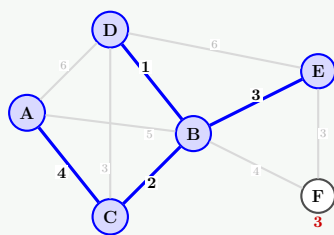
Step 2: Extract C. Add AC(4)



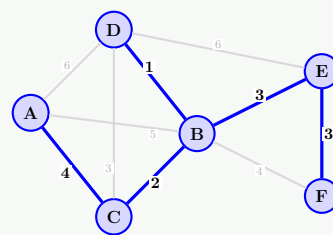
Step 3: Extract B. Add CB(2)



Step 4: Extract D. Add BD(1)



Step 5: Extract E. Add BE(3)



Step 6: Extract F. Add EF(3)

Prim MST Edges: $AC(4), CB(2), BD(1), BE(3), EF(3)$.

Total Cost = $4 + 2 + 1 + 3 + 3 = 13$. ✓

- Q35.** Given a set of jobs with start and finish times, write down an algorithm so that the maximum number of jobs can be processed. Show counterexamples for some natural greedy approaches that will not find optimal solutions. (10 marks) [CSE 207 | 2013–14]

Answer

Correct algorithm: sort by finish time; pick greedily.

Wrong strategy 1: Earliest start time. Counterexample: intervals $[1, 10], [2, 3], [4, 5]$. Earliest start picks $[1, 10]$ first $\rightarrow$ only 1 activity. Optimal: $[2, 3], [4, 5] \rightarrow 2$ activities. $\times$

Wrong strategy 2: Shortest duration. Counterexample: $[1, 4], [3, 5], [4, 7]$. Shortest duration picks $[3, 5]$ (length 2) first $\rightarrow$ then neither $[1, 4]$ nor $[4, 7]$ overlaps with it, giving $\{[3, 5], [4, 7]\} = 2$ activities. But earliest-finish gives $\{[1, 4], [4, 7]\} = 2$ activities too here; for a cleaner counterexample: $[1, 3], [2, 4], [3, 5]$ — shortest-duration picks $[1, 3], [3, 5] = 2$; but greedy also gives 2.

Better: $[1, 5], [3, 4], [6, 10]$. Shortest: $[3, 4], [6, 10] = 2$. Earliest-finish: $[3, 4], [6, 10] = 2$ also.

Clear counterexample: intervals $[1, 100], [2, 3], [4, 5], [6, 7]$. Earliest-start picks $[1, 100] \rightarrow 1$ activity. Shortest picks $[2, 3], [4, 5], [6, 7] \rightarrow 3$ activities. Neither earliest-start *nor* shortest is always optimal without the finish-time criterion.

Wrong strategy 3: Fewest overlaps (pick interval overlapping fewest others). Counterexample constructible with 3 “hub” intervals and many small ones — picking the hub with fewest overlaps can block many short non-overlapping ones.

- Q36.** Given the message ‘abaacbdbbebfbedcect’, construct a dynamic Huffman tree showing how the tree is updated after sending each symbol and the code of each symbol. (15 marks) [CSE 207 | 2013–14]

Answer

Message: a b a a c b d b e b f b e d c e c t

Final frequencies: $a=3, b=5, c=3, d=2, e=3, f=1, t=1$. Total = 18.

Dynamic (Adaptive) Huffman Encoding — Key idea:

Start with an empty tree containing only the NYT (Not Yet Transmitted) node. For each symbol:

- If **new symbol**: transmit NYT code + fixed symbol bits. Add a new leaf; split NYT.
- If **seen before**: transmit current Huffman code for the symbol. Increment its count.
- After each transmission, **update the tree**: increment the count of the symbol's leaf and all ancestors; swap nodes to maintain the *sibling property* (nodes ordered by weight).

Symbol-by-symbol trace (abbreviated):

Step	Symbol	Code sent	Tree update
1	a	NYT + a bits	add leaf(a,1); NYT splits
2	b	NYT + b bits	add leaf(b,1); rebalance
3	a	current code of a	freq(a)=2; swap if needed
4	a	current code of a	freq(a)=3
5	c	NYT + c bits	add leaf(c,1)
6	b	current code of b	freq(b)=2
7	d	NYT + d bits	add leaf(d,1)
8	b	current code of b	freq(b)=3
9	e	NYT + e bits	add leaf(e,1)
10	b	current code of b	freq(b)=4
11	f	NYT + f bits	add leaf(f,1)
12	b	current code of b	freq(b)=5
13	e	current code of e	freq(e)=2
14	d	current code of d	freq(d)=2
15	c	current code of c	freq(c)=2
16	e	current code of e	freq(e)=3
17	c	current code of c	freq(c)=3
18	t	NYT + t bits	add leaf(t,1)

Final Huffman tree is built from frequencies: b=5, a=3, c=3, e=3, d=2, f=1, t=1.

Building greedily: merge f(1)+t(1)=2, then d(2)+ft(2)=4, then a(3)+c(3)=6, then e(3)+dft(4)=7, then b(5)+ac(6)=11, then root=11+7=18.

Final codes (one possible assignment): b=00, a=010, c=011, e=10, d=110, f=1110, t=1111.

- Q37.** Suppose you are given n tasks with start times $S[1 \dots n]$ and finish times $F[1 \dots n]$. Schedule as many tasks as possible so that no two overlap. (i) Describe a greedy algorithm that finds the maximum number of non-overlapping tasks. (ii) Prove the correctness of your greedy algorithm. **(15 marks)** [CSE 207 | 2012–13]

Answer**(i) Greedy Algorithm — Earliest Finish Time:****Algorithm 106** ACTIVITYSELECT(S, F, n)

```

1: Sort tasks by finish time:  $F[1] \leq F[2] \leq \dots \leq F[n]$ 
2:  $selected \leftarrow \{1\}$ ;  $last \leftarrow 1$ 
3: for  $i = 2$  to  $n$  do
4:   if  $S[i] \geq F[last]$  then
5:      $selected \leftarrow selected \cup \{i\}$ 
6:      $last \leftarrow i$ 
7:   end if
8: end for return  $selected$ 

```

Complexity: $O(n \log n)$ for sorting; $O(n)$ for selection. Total: $O(n \log n)$.

(ii) Correctness Proof (Greedy stays ahead):

Claim: The greedy solution $G = \{g_1, g_2, \dots, g_k\}$ (sorted by finish time) is optimal, i.e., $|G| \geq |OPT|$ for any optimal solution $OPT = \{o_1, o_2, \dots, o_m\}$.

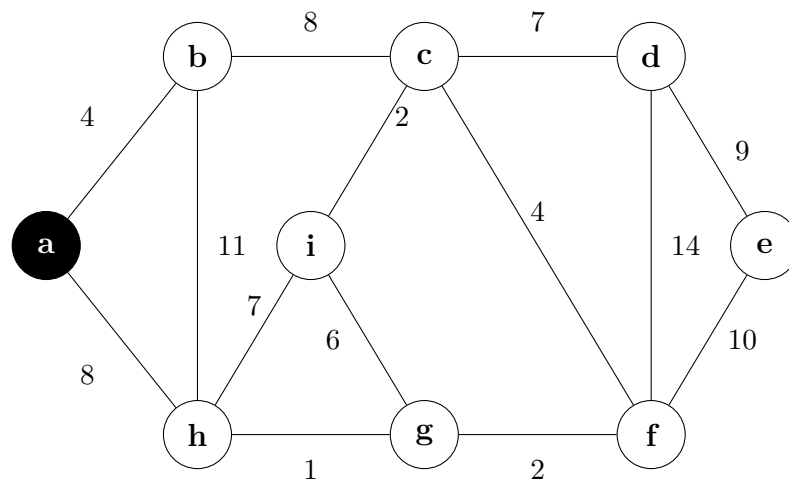
Lemma: At every step i , $f(g_i) \leq f(o_i)$ (greedy finishes no later than optimal).

Proof by induction:

- **Base case** ($i = 1$): Greedy picks the task with earliest finish time, so $f(g_1) \leq f(o_1)$. ✓
- **Inductive step:** Assume $f(g_{i-1}) \leq f(o_{i-1})$. Then o_i starts at $s(o_i) \geq f(o_{i-1}) \geq f(g_{i-1})$, so o_i is compatible with g_{i-1} . The greedy algorithm considers all compatible tasks and picks the one with minimum finish time, so $f(g_i) \leq f(o_i)$. ✓

Conclusion: Since $f(g_i) \leq f(o_i)$ for all $i \leq m$, greedy never runs out of room before OPT does. Therefore $k \geq m$, i.e., greedy selects at least as many tasks as any optimal solution. $\square$

Q38. If Prim's algorithm is applied on a given graph, which will be the fifth edge added to the spanning tree? Start from vertex a . **(10 marks)** [CSE 207 | 2012–13]



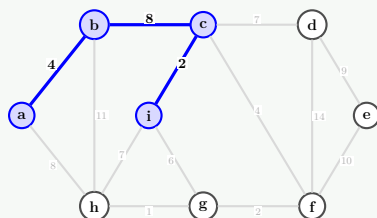
Answer: Prim's Algorithm (CSE 207 | 2012-13)

Start Node: a .

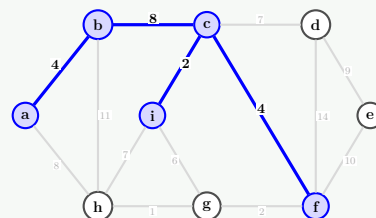
When ties occur (e.g., between edges $a-h(8)$ and $b-c(8)$), we pick alphabetically (adding c instead of h).

Step	Vertex Added	Edge Added	Edge #	Key Values (Selected Un-visited)
1	a	—	—	$b : 4, h : 8$
2	b	$a-b$ (4)	1st Edge	$c : 8, h : 8$
3	c	$b-c$ (8)	2nd Edge	$d : 7, f : 4, i : 2, h : 8$
4	i	$c-i$ (2)	3rd Edge	$d : 7, f : 4, g : 6, h : 8$
5	f	$c-f$ (4)	4th Edge	$d : 7, g : 2, h : 7$
6	g	$f-g$ (2)	5th Edge	$d : 7, h : 1$
7	h	$g-h$ (1)	6th Edge	$d : 7$
8	d	$c-d$ (7)	7th Edge	$e : 9$
9	e	$d-e$ (9)	8th Edge	—

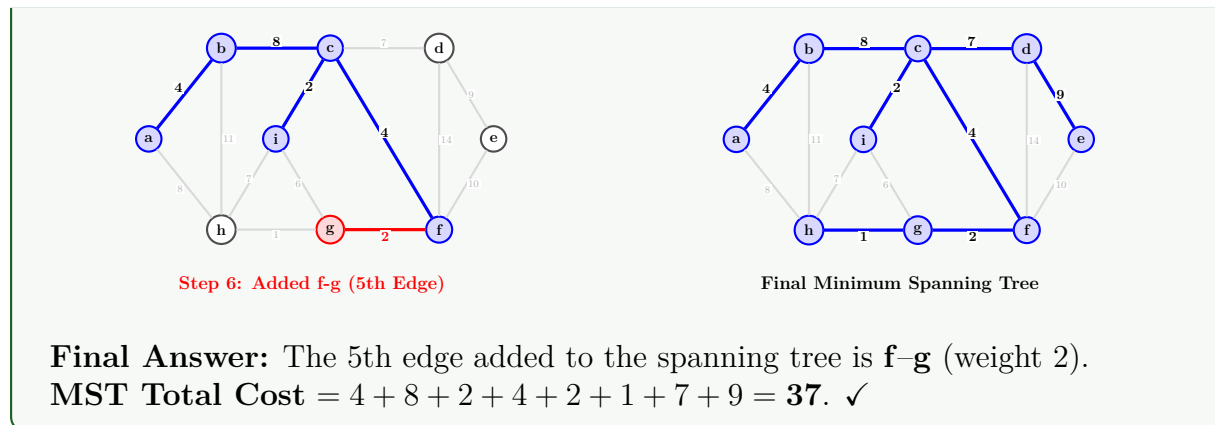
Visual Trace (Highlighting the 5th Edge):



Step 4: Added $c-i$ (3rd Edge)



Step 5: Added $c-f$ (4th Edge)



- Q39.** “Search spaces for natural combinatorial problems tend to grow exponentially with the size of the input” — explain and justify with necessary examples. **(15 marks)**
 [CSE 207 | 2011–12]

Answer

Claim: the search spaces of natural combinatorial problems grow exponentially with input size.

Explanation and examples:

- 1. Subset problems** (2^n subsets of n elements). The 0-1 Knapsack problem has n items; the number of subsets to check is 2^n . For $n = 30$, this is over 10^9 — infeasible by brute force.
- 2. Permutation problems** ($n!$ orderings). The Travelling Salesman Problem (TSP) requires finding the shortest Hamiltonian cycle over n cities; there are $(n-1)!$ distinct tours. For $n = 20$, this is $\approx 10^{17}$.
- 3. Scheduling / assignment** (assignment of n jobs to m machines: m^n possibilities).

Formal justification: Consider binary strings of length n : each bit has 2 choices, giving 2^n total strings. This grows faster than any polynomial: for any fixed k , $2^n > n^k$ for large enough n . Most combinatorial problems map their solution space to exponentially large sets of this form, making exhaustive search intractable.

Implication: this motivates greedy algorithms, dynamic programming, and approximation algorithms that find optimal or near-optimal solutions in polynomial time without enumerating the full search space.

- Q40.** Explain why one can solve the fractional knapsack problem by a greedy strategy, but one cannot solve the 0-1 knapsack problem by such a strategy. **(10 marks)** [CSE 207 | 2011–12]

Answer

Fractional Knapsack — Greedy works:

In the fractional version we may take any fraction of an item. The greedy strategy sorts items by value density v_i/w_i and takes as much as possible of the highest-density item remaining.

Why it works (exchange argument): Suppose an optimal solution does *not* take the item with the highest v/w first. We can exchange a small amount of a lower-density item for an equal weight of the higher-density item, strictly increasing the total value without violating the capacity constraint. Repeating this argument shows the greedy solution is optimal.

Formally: The fractional knapsack has the *greedy-choice property* — a locally optimal (highest density) choice is always part of some globally optimal solution.

0-1 Knapsack — Greedy fails:

In the 0-1 version each item must be taken entirely or not at all. The greedy strategy can fail because:

- Taking a high-density item may “waste” capacity and prevent fitting a combination of lower-density items that has greater total value.
- No fractional fill is allowed, so the exchange argument breaks down.

Counterexample: $W = 10$.

Item	w	v	v/w
1	6	12	2.0
2	5	9	1.8
3	5	9	1.8

Greedy picks item 1 ($v/w = 2$, $w = 6$), then cannot fit items 2 or 3 together (remaining capacity = $4 < 5$). Total value = 12.

Optimal: items 2 + 3 ($w = 10$, $v = 18$). Greedy gives only $12 < 18$.

Conclusion: The 0-1 knapsack lacks the greedy-choice property and requires dynamic programming ($O(nW)$) for an exact solution.

Q41. Prove that the greedy method stays ahead in the interval scheduling problem, and hence show that the greedy method returns an optimal set of jobs. **(15 marks)** [CSE 207 | 2011–12]

Answer

Algorithm: sort intervals by finish time; greedily pick each compatible interval. Let $i_1, i_2, \dots, i_k$ be the greedy selections (in order of finish time).

Greedy Stays Ahead Lemma. For any other compatible set $j_1, j_2, \dots, j_m$ (in order of finish time), for each $r \leq \min(k, m)$:

$$f(i_r) \leq f(j_r).$$

(The r -th greedy choice finishes no later than the r -th choice of any other algorithm.)

Proof (by induction on r).

Base ($r = 1$): i_1 has the earliest finish time of all intervals $\rightarrow f(i_1) \leq f(j_1)$. ✓

Step: assume $f(i_r) \leq f(j_r)$. Then j_{r+1} starts after $f(j_r) \geq f(i_r)$, so j_{r+1} is compatible with $i_1, \dots, i_r$. The greedy considers all remaining compatible intervals and picks the one with the smallest finish time, so $f(i_{r+1}) \leq f(j_{r+1})$. ✓

Optimality: suppose the greedy selects k intervals but OPT selects $m > k$. By the lemma, $f(i_k) \leq f(j_k)$. Then j_{k+1} starts after $f(j_k) \geq f(i_k)$, so j_{k+1} is compatible with the greedy's k selections — but the greedy found no more compatible intervals, a contradiction. Hence $k \geq m$ and the greedy is optimal. $\square$

- Q42.** Prove that if we use a greedy algorithm for the interval partitioning problem, every interval will be assigned a label, and no two overlapping intervals will receive the same label. (15 marks) [CSE 207 | 2011–12]

Answer

Interval Partitioning Problem: assign each of n intervals a *label* (colour/room) so that no two overlapping intervals share a label, using the minimum number of labels.

Algorithm: sort by start time; for each interval, assign any free label; if none is free, open a new label.

Claim 1: every interval is assigned a label. The algorithm considers intervals in start-time order and always either finds a free label or opens a new one. Since we always assign a label, no interval is left unassigned. $\checkmark$

Claim 2: no two overlapping intervals share a label. A label ℓ is only assigned to interval i if its previous occupant j satisfies $f(j) \leq s(i)$ (i.e. j has finished before i starts). Hence i and j do not overlap. By induction, all intervals assigned label ℓ form a compatible (non-overlapping) set. $\square$

Optimality: the number of labels used equals the maximum *depth* d (maximum number of intervals overlapping at any single point). Any labeling needs at least d labels, and the greedy uses exactly d . $\square$

- Q43.** Explain how Dijkstra's algorithm can be implemented using a priority queue. Find the running time for this algorithm. (15 marks) [CSE 207 | 2011–12]

Answer

Implementation with Binary Min-Heap:

```

1 void dijkstra(int src, vector<vector<pair<int,int>>>& adj, int
   V) {
2     vector<int> dist(V, INT_MAX);
3     priority_queue<pair<int,int>,
4                   vector<pair<int,int>>,
5                   greater<>> pq; // min-heap: (dist, vertex)
6     dist[src] = 0;
7     pq.push({0, src});
8     while (!pq.empty()) {
9         auto [d, u] = pq.top(); pq.pop();
10        if (d > dist[u]) continue; // stale entry
11        for (auto [w, v] : adj[u]) {
12            if (dist[u] + w < dist[v]) {

```

```

13         dist[v] = dist[u] + w;
14         pq.push({dist[v], v});
15     }
16 }
17 }
18 }
```

Running time analysis:

Operation	Count	Cost
INSERT (initial)	V	$O(\log V)$ each
EXTRACT-MIN	$\leq V$	$O(\log V)$ each
DECREASE-KEY	$\leq E$	$O(\log V)$ each

Total with binary heap: $O((V + E) \log V)$.

With Fibonacci heap: Decrease-Key is $O(1)$ amortised $\rightarrow$ total $O(V \log V + E)$.

- Q44.** State and prove the “cut property” and the “cycle property”. Explain with necessary examples how these properties are useful in solving the minimum spanning tree problem. **(15 marks)** [CSE 207 | 2011–12]

Answer

Cut Property. Let $(S, V \setminus S)$ be any cut of G and e the unique minimum-weight edge crossing it. Then e belongs to every MST.

Proof: suppose MST T does not contain e . Adding e to T creates a cycle; this cycle must contain another crossing edge e' with $w(e') > w(e)$. $T' = T - \{e'\} + \{e\}$ is a spanning tree with $w(T') < w(T)$ — contradiction. $\square$

Example: in any graph, the globally lightest edge e^* belongs to every MST (take the cut separating its endpoints).

Cycle Property. Let C be any cycle in G and e the unique maximum-weight edge in C . Then e does *not* belong to any MST.

Proof: suppose MST T contains e . Removing e from T splits it into two components; there is another edge $e' \in C$ crossing this cut with $w(e') < w(e)$. $T' = T - \{e\} + \{e'\}$ is a lighter spanning tree — contradiction. $\square$

Example: the heaviest edge in any cycle can always be safely discarded.

Application:

- **Prim’s** uses the cut property: always adds the minimum crossing edge of $(S, V \setminus S)$.
- **Kruskal’s** uses both: adds minimum edges (cut property) and rejects maximum-weight cycle edges (cycle property).

- Q45.** Describe, with necessary elaboration on the required operations and data structure, how the union-find data structure can be used for Kruskal’s algorithm. **(15 marks)** [CSE 207 | 2011–12]

Answer**Union-Find (Disjoint-Set Union) data structure:**

Each connected component of the growing forest is stored as a disjoint set. Three operations:

```

1 // Make a singleton set for each vertex
2 void makeSet(int x) { parent[x]=x; rank[x]=0; }
3
4 // Find representative (with path compression)
5 int find(int x) {
6     if (parent[x] != x)
7         parent[x] = find(parent[x]); // path compression
8     return parent[x];
9 }
10
11 // Merge two sets (union by rank)
12 void unite(int x, int y) {
13     int px=find(x), py=find(y);
14     if (px==py) return;
15     if (rank[px] < rank[py]) swap(px,py);
16     parent[py] = px;
17     if (rank[px]==rank[py]) rank[px]++;
18 }
```

Use in Kruskal's:

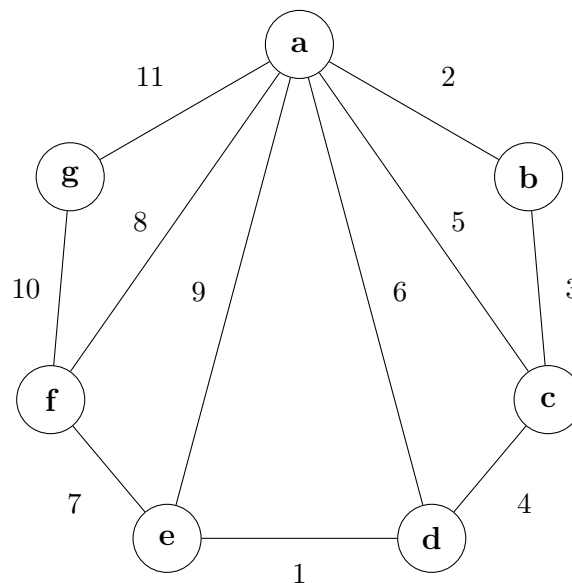
- Initialise: MAKESET(v) for each $v \in V$.
- For each edge (u, v) in sorted order: if FIND(u) $\neq$ FIND(v), the edge does not form a cycle $\rightarrow$ add to MST and call UNITE(u, v).

Complexity:

- MAKESET: $O(1)$ each, $O(V)$ total.
- FIND with path compression: $O(\alpha(V))$ amortised.
- UNITE with union-by-rank: $O(\alpha(V))$ amortised.
- Total for E operations: $O(E \alpha(V)) \approx O(E)$.
- Kruskal total (including sort): $O(E \log E) = O(E \log V)$.

where α is the inverse Ackermann function, practically ≤ 4 for all realistic n .

Q46. What is the criterion for selecting an edge at each step of Kruskal's algorithm for computing MST? How does this criterion guarantee the algorithm gives an MST? Show the steps of Kruskal's algorithm on a given graph. **(17 marks)** [CSE 207 | 2009–10]



Answer: Kruskal's Criterion & Trace (CSE 207 | 2009-10)

Criterion: At each step, select the minimum-weight edge that does not form a cycle with the already-chosen edges.

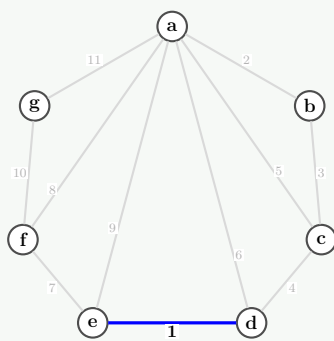
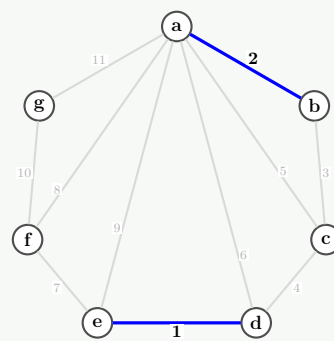
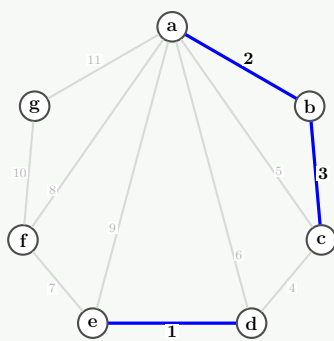
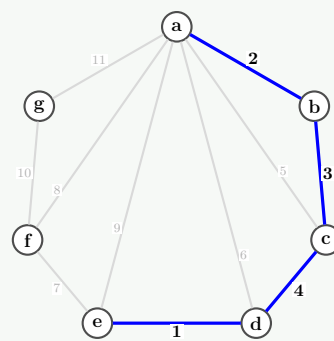
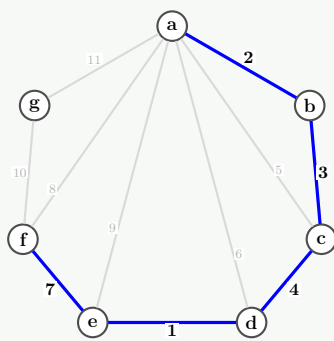
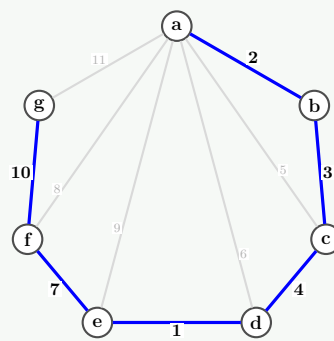
Guarantee (Correctness): The selected edge is always the minimum crossing edge of some cut that respects the current forest A (the cut separates the two components being joined). By the **Cut Property**, this edge belongs to some Minimum Spanning Tree (MST). By induction, the final set A is an MST. $\square$

Solving the Graph

Edges Sorted by Weight: $de(1), ab(2), bc(3), cd(4), ac(5), ad(6), ef(7), af(8), ae(9), fg(10), ga(11)$.

Edge	Weight	Add to MST?	Reason
$d-e$	1	Yes	Does not form a cycle
$a-b$	2	Yes	Does not form a cycle
$b-c$	3	Yes	Does not form a cycle
$c-d$	4	Yes	Connects $\{a, b, c\}$ with $\{d, e\}$
$a-c$	5	No	Forms cycle $(a-b-c-a)$
$a-d$	6	No	Forms cycle $(a-b-c-d-a)$
$e-f$	7	Yes	Connects f to the tree
$a-f$	8	No	Forms cycle $(a-b-c-d-e-f-a)$
$a-e$	9	No	Forms cycle $(a-b-c-d-e-a)$
$f-g$	10	Yes	Connects g . (All 7 vertices joined!)

Visual Trace (Step-by-Step):

Step 1: Add $de(1)$ Step 2: Add $ab(2)$ Step 3: Add $bc(3)$ Step 4: Add $cd(4)$ Step 5: Skip ac , ad . Add $ef(7)$ Step 6: Skip af , ae . Add $fg(10)$

Final MST Edges: $de(1), ab(2), bc(3), cd(4), ef(7), fg(10)$.
Total Cost = $1 + 2 + 3 + 4 + 7 + 10 = 27$. ✓

Q47. Construct the Huffman tree and show the Huffman coding for the following characters with given frequencies:

Character	a	b	c	d	e	f
Frequency	50	30	20	15	10	5

(10 marks)

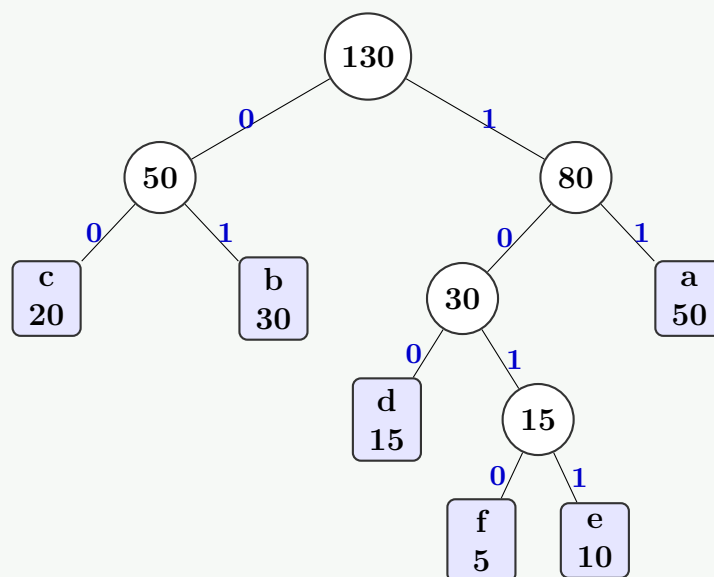
[CSE 207 | 2009–10]

Answer**Merge steps** (always merge two smallest):

- (a) $f(5) + e(10) = fe(15)$
 (b) $d(15) + fe(15) = dfe(30)$
 (c) $c(20) + b(30) = cb(50)$
 (d) $dfe(30) + a(50) = dfea(80)$
 (e) $cb(50) + dfea(80) = \text{root}(130)$

Huffman codes:

Symbol	Freq	Code	Bits used
<i>a</i>	50	11	$50 \times 2 = 100$
<i>b</i>	30	01	$30 \times 2 = 60$
<i>c</i>	20	00	$20 \times 2 = 40$
<i>d</i>	15	100	$15 \times 3 = 45$
<i>e</i>	10	1011	$10 \times 4 = 40$
<i>f</i>	5	1010	$5 \times 4 = 20$
Total bits:			305

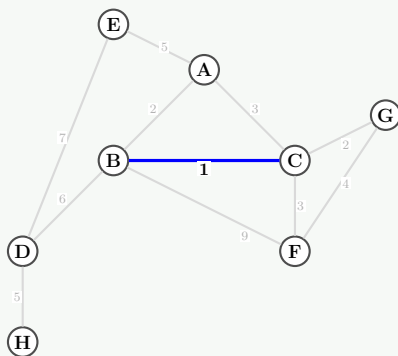
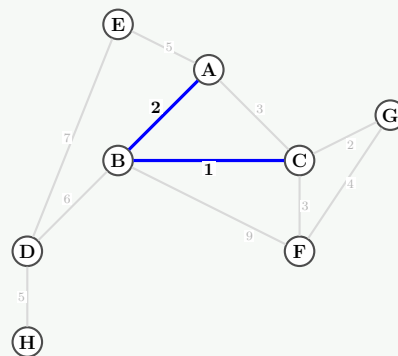
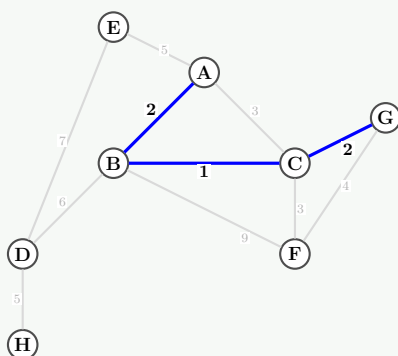
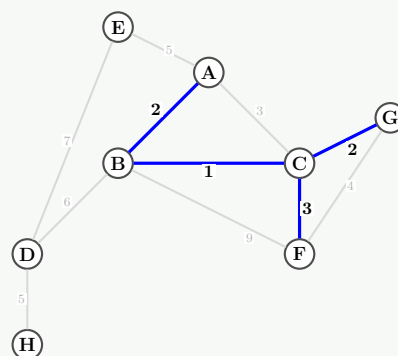


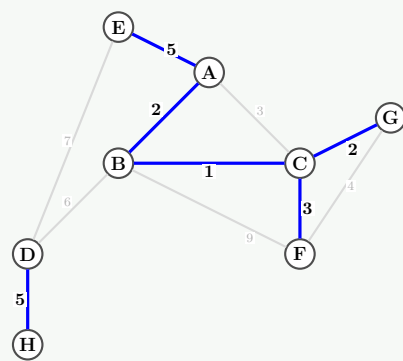
Q48. Construct a minimum spanning tree using Kruskal's algorithm for the following weighted graph: $AB(2)$, $AC(3)$, $AE(5)$, $BC(1)$, $BD(6)$, $BF(9)$, $CF(3)$, $CG(2)$, $DE(7)$, $DH(5)$, $FG(4)$. Show every step, giving reasons for discarding any edge. **(15+5=20 marks)** [CSE 207 | 2008–09]

Answer: Kruskal's Algorithm (CSE 207 | 2008-09)

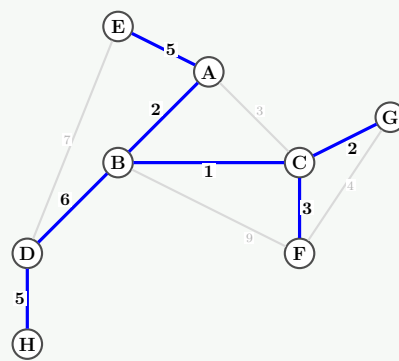
Sorted edges: $BC(1)$, $AB(2)$, $CG(2)$, $CF(3)$, $AC(3)$, $FG(4)$, $AE(5)$, $DH(5)$, $BD(6)$, $DE(7)$, $BF(9)$.

Edge	Weight	Add to MST?	Reason
BC	1	Yes	Does not form a cycle
AB	2	Yes	Does not form a cycle
CG	2	Yes	Does not form a cycle
CF	3	Yes	Does not form a cycle
AC	3	No	Forms cycle ($A-B-C-A$)
FG	4	No	Forms cycle ($F-C-G-F$)
AE	5	Yes	Does not form a cycle
DH	5	Yes	Does not form a cycle
BD	6	Yes	Connects $\{A, B, C, E, F, G\}$ with $\{D, H\}$
DE	7	No	Forms cycle
BF	9	No	Forms cycle

Visual Trace (Step-by-Step):Step 1: Add $BC(1)$ Step 2: Add $AB(2)$ Step 3: Add $CG(2)$ Step 4: Add $CF(3)$



Step 5: Skip AC, FG. Add AE, DH



Step 6: Add BD(6) (Final MST)

Final MST Edges: $BC(1), AB(2), CG(2), CF(3), AE(5), DH(5), BD(6)$.
Total Cost = $1 + 2 + 2 + 3 + 5 + 5 + 6 = 24$. ✓

Q49. Discuss how Prim's algorithm constructs a minimum spanning tree. Argue how sets and the disjoint-set union algorithm can be used to detect cycles and improve upon Kruskal's algorithm. (15 marks) [CSE 207 | 2008–09]

Answer

Prim's Algorithm:

Prim's algorithm grows the MST one edge at a time, always adding the *minimum-weight edge* that connects a vertex already in the tree to a vertex not yet in the tree.

Algorithm 107 PRIM(G, w, r)

```

1: for each  $u \in V$  do
2:    $key[u] \leftarrow \infty$ ;  $parent[u] \leftarrow \text{nil}$ 
3: end for
4:  $key[r] \leftarrow 0$ 
5:  $Q \leftarrow$  min-priority queue of all vertices keyed by  $key[]$ 
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8:   for each  $v$  adjacent to  $u$  do
9:     if  $v \in Q$  and  $w(u, v) < key[v]$  then
10:       $parent[v] \leftarrow u$ 
11:       $key[v] \leftarrow w(u, v)$  ▷ Decrease-Key in  $Q$ 
12:     end if
13:   end for
14: end while
```

- Uses a min-priority queue; each EXTRACT-MIN costs $O(\log V)$.
- Each DECREASE-KEY costs $O(\log V)$.
- Total: $O((V + E) \log V)$; with Fibonacci heap: $O(E + V \log V)$.

Kruskal's Algorithm with Disjoint-Set Union (DSU):

Kruskal's algorithm processes edges in non-decreasing weight order, adding an edge only if its two endpoints are in *different components* (i.e., adding it does not form a cycle).

Cycle detection using DSU:

- Each component is a set. Initially every vertex is its own set.
- For edge (u, v) : call $\text{FIND}(u)$ and $\text{FIND}(v)$.
 - If they return *different* roots $\Rightarrow$ different components $\Rightarrow$ **no cycle**: add edge, call $\text{UNION}(u, v)$.
 - If they return the *same* root $\Rightarrow$ same component $\Rightarrow$ **cycle**: skip edge.

Improvements via union-by-rank + path compression:

- **Union by rank:** Always attach the smaller tree under the larger, keeping trees shallow.
- **Path compression:** On FIND , make every node on the path point directly to the root.
- Combined amortized cost: $O(\alpha(V))$ per operation (α = inverse Ackermann, essentially constant).

Total complexity of Kruskal's with DSU: $O(E \log E)$ (dominated by sorting edges).

Q50. Discuss how a Huffman tree can be constructed given a sequence of frequencies corresponding to symbols. Construct a dynamic Huffman tree based upon the message ABBBCAAAACBBBB. (20 marks) [CSE 207 | 2008–09]

Answer**Huffman Tree Construction:**

Given symbols with frequencies, build the tree greedily:

1. Insert each symbol as a leaf node into a **min-priority queue** keyed by frequency.
2. Repeat until one node remains:
 - (a) Extract the two nodes x, y with minimum frequency.
 - (b) Create a new internal node z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$.
 - (c) Make x left child and y right child of z ; insert z back into the queue.
3. The remaining node is the root of the Huffman tree.

Left branch = 0, right branch = 1 (or vice versa). The code for each symbol is the path from root to its leaf.

Frequencies in ABBBCAAAACBBBB:

Count: A=5, B=7, C=2. Total = 14 characters.

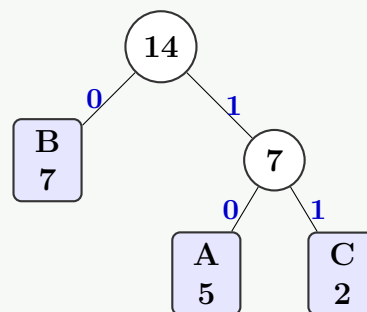
Building the Huffman Tree:

Step 1: Initial queue: C(2), A(5), B(7).

Step 2: Extract C(2) and A(5). Merge $\rightarrow$ internal node CA(7). Queue: B(7), CA(7).

Step 3: Extract B(7) and CA(7). Merge $\rightarrow$ root(14).

Final Tree:



Codes: B = 0, A = 10, C = 11.

Encoded message ABBBCAAAACBBBB:

10 0 0 0 11 10 10 10 11 0 0 0 0

Total bits = $5 \times 2 + 7 \times 1 + 2 \times 2 = 10 + 7 + 4 = 21$ bits.

- Q51.** Suppose you have a washing machine that a number of users want to use. Each user i sends a request for the time interval (s_i, f_i) (start and finish times). The machine can serve only one user at a time, and each user pays 1000/- regardless of duration. Write a greedy algorithm to maximize revenue. Prove the correctness of your algorithm and deduce its running time. **(8+10+5 marks)** [CSE 207 | 2007–08]

Answer

Formulation: this is exactly the **Interval Scheduling** (Activity Selection) problem. Each user i requests interval (s_i, f_i) ; the machine can serve one at a time; each served user pays a fixed amount. Maximising revenue = maximising the number of served users.

Algorithm: sort requests by finish time; greedily accept each request compatible with the last accepted one.

Correctness: proved via the greedy-choice + optimal substructure argument (see [CSE 203 | 2020–21]).

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for the scan. Total: $O(n \log n)$.

Q52. Define the interval coloring problem and write an algorithm to solve it. (4+8 marks)
[CSE 207 | 2007–08]

Answer

Interval Coloring Problem: given n intervals, assign each interval a “color” (resource/room) such that no two overlapping intervals share a color, using the *minimum* number of colors (= minimum number of resources).

The minimum number of colors needed equals the maximum number of intervals that overlap at any single point (*depth*).

Algorithm (earliest start time, greedy):

Algorithm 108 INTERVALCOLORING(S)

```

1: Sort intervals by start time:  $s_1 \leq s_2 \leq \dots \leq s_n$ 
2:  $d \leftarrow 0$  ▷ number of colors used
3: for  $i \leftarrow 1$  to  $n$  do
4:   if some color  $c$  is free at time  $s_i$  then
5:     Assign color  $c$  to interval  $i$ ; free  $c$  after  $f_i$ 
6:   else
7:      $d \leftarrow d + 1$ ; assign new color  $d$  to interval  $i$ 
8:   end if
9: end for
10: return  $d$ 

```

Implemented with a min-heap of finish times of currently active intervals, this runs in $O(n \log n)$.

Correctness: whenever a new color is needed, all d existing colors are in use, meaning d intervals overlap at s_i — so any valid coloring needs $\geq d$ colors. $\square$

Q53. Describe the main idea of Prim’s algorithm to compute a minimum spanning tree. Discuss how Prim’s algorithm can be implemented in $O(m \log n)$ time. Discuss the reverse-delete algorithm to compute a minimum spanning tree and prove its correctness. (6+10+17 marks)
[CSE 207 | 2007–08]

Answer

Prim’s main idea: grow the MST one edge at a time. Maintain a set S of tree vertices; at each step add the minimum-weight edge crossing the cut $(S, V \setminus S)$.

$O(m \log n)$ **implementation** (binary min-heap):

- Maintain a key $\text{key}[v] = \min$ weight of any edge from v to a vertex in S ; store all non-tree vertices in a min-heap keyed by $\text{key}[\cdot]$.

- Extract-Min: $O(\log n)$, called n times $\rightarrow O(n \log n)$.
- Decrease-Key: $O(\log n)$, called $\leq m$ times $\rightarrow O(m \log n)$.
- Total: $O(m \log n)$.

Reverse-Delete algorithm:

- (a) Start with $T = E$ (all edges).
- (b) Process edges in *decreasing* weight order.
- (c) Delete edge e from T if $T - \{e\}$ is still connected.

Correctness: we never delete a bridge (a cut edge), so the graph stays connected. We delete an edge only if it is not the unique minimum crossing edge of some cut (i.e. it is not in every MST by the cut property). The result is an MST. $\square$

Q54. Write down an algorithm for constructing a minimum spanning tree using Kruskal's algorithm. Show how sets and the disjoint-set algorithm can be used to reduce its complexity. Construct a minimum spanning tree for the graph with edges: $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6), FD(3)$. (5+5+5 marks) [CSE 207 | 2006-07]

Answer: Kruskal & Disjoint-Set (CSE 207 | 2006-07)**Kruskal's algorithm:****Algorithm 109** KRUSKAL(G, w)

```

1:  $A \leftarrow \emptyset$ 
2: for each vertex  $v$  do MAKE-SET( $v$ )
3: end for
4: Sort edges by weight (ascending)
5: for each edge  $(u, v)$  in sorted order do
6:   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then
7:      $A \leftarrow A \cup \{(u, v)\}$ ; UNION( $u, v$ )
8:   end if
9: end for
10: return  $A$ 

```

Disjoint-set (Union-Find) for cycle detection:

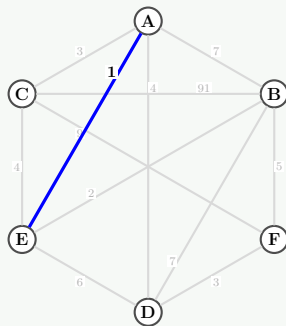
- FIND with path compression: $O(\alpha(V))$ amortised.
 - UNION by rank: $O(\alpha(V))$ amortised.
 - **Total Kruskal Time Complexity:** $O(E \log E) = O(E \log V)$.
-

Solving the Graph

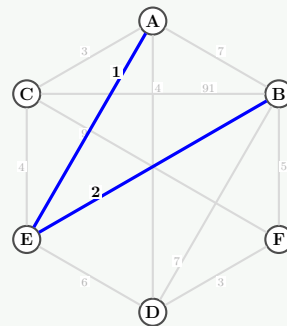
Sorted edges: $EA(1)$, $EB(2)$, $AC(3)$, $FD(3)$, $DA(4)$, $CE(4)$, $FB(5)$, $DE(6)$, $AB(7)$, $BD(7)$, $CF(9)$, $BC(91)$.

Edge	Weight	Add?	Reason
$E-A$	1	Yes	Different sets
$E-B$	2	Yes	Different sets
$A-C$	3	Yes	Different sets
$F-D$	3	Yes	Different sets
$D-A$	4	Yes	Different sets → All 6 vertices connected!
$C-E$	4	No	Same set (Forms cycle $C-A-E-C$)
$F-B$	5	No	Same set (Forms cycle)
... all remaining edges are skipped as MST is complete.			

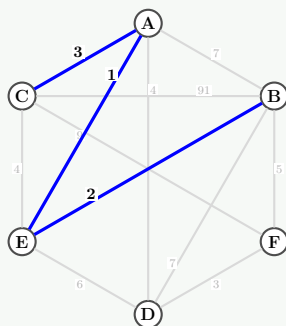
Visual Trace (Step-by-Step):



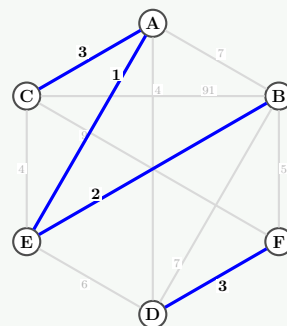
Step 1: Add $EA(1)$



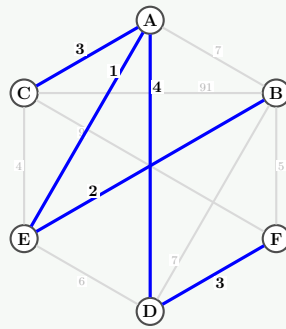
Step 2: Add $EB(2)$



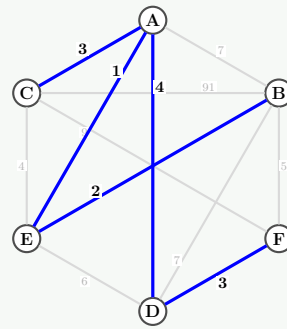
Step 3: Add $AC(3)$



Step 4: Add $FD(3)$



Step 5: Add DA(4)



Step 6: Skip rest (Final MST)

Final MST Edges: $EA(1), EB(2), AC(3), FD(3), DA(4)$.

Total Cost = $1 + 2 + 3 + 3 + 4 = 13$. ✓

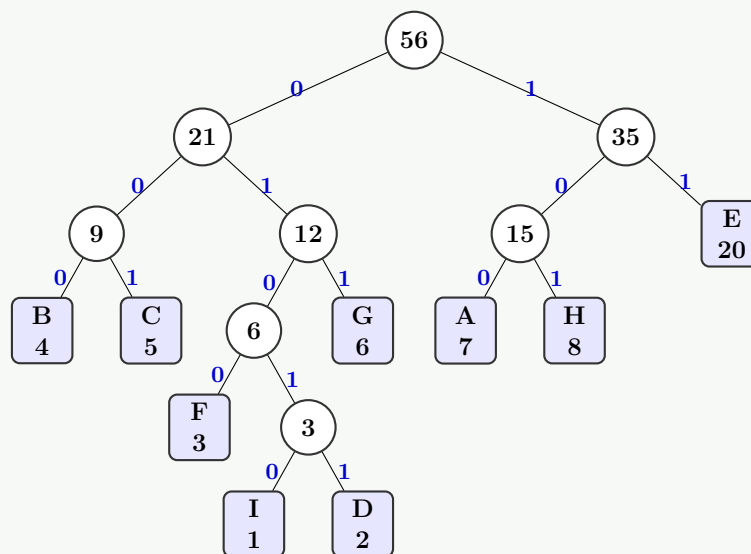
Q55. Construct a Huffman tree for the phrase with symbols and frequencies: $A-7, B-4, C-5, D-2, E-20, F-3, G-6, H-8, I-1$. (10 marks) [CSE 207 | 2006–07]

Answer: Huffman Tree Construction

Merge steps (always merge two smallest available nodes):

- (a) $I(1) + D(2) = 3$
- (b) $F(3) + (I+D)(3) = 6$
- (c) $B(4) + C(5) = 9$
- (d) $(F+ID)(6) + G(6) = 12$
- (e) $A(7) + H(8) = 15$
- (f) $(B+C)(9) + (F+ID+G)(12) = 21$
- (g) $(A+H)(15) + E(20) = 35$
- (h) $(BC+FIDG)(21) + (AH+E)(35) = 56$ (Root)

Huffman Tree Visual Representation:



Huffman codes:

Symbol	Freq	Code
E	20	11
A	7	100
H	8	101
B	4	000
C	5	001
G	6	011
F	3	0100
D	2	01011
I	1	01010

Total bits = $20 \cdot 2 + 7 \cdot 3 + 8 \cdot 3 + 4 \cdot 3 + 5 \cdot 3 + 6 \cdot 3 + 3 \cdot 4 + 2 \cdot 5 + 1 \cdot 5 = 157$.

- Q56.** There is a set of jobs where each job has an integer deadline and profit, and only one machine is available. To complete a job requires one unit of time; profit is earned only if the job is completed by its deadline. Jobs A, B, C, D, E have deadlines 3, 3, 1, 2, 2 and profits 1, 5, 10, 15, 20 respectively. Show the steps of selecting jobs using a greedy strategy. (8 marks) [CSE 207 | 2006–07]

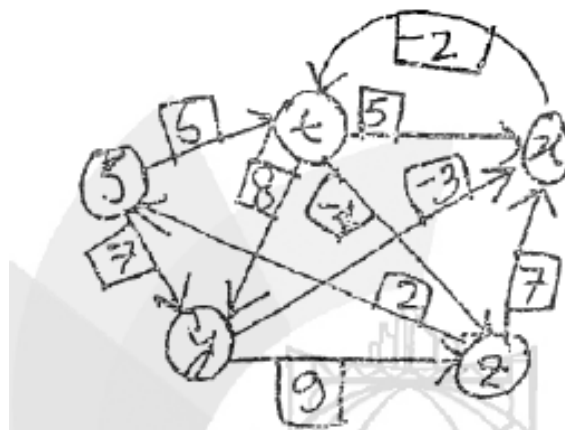
Answer

Sort by profit descending: $E(20, d = 2), D(15, d = 2), C(10, d = 1), B(5, d = 3), A(1, d = 3)$.

Job	Profit	Deadline	Slot assigned
<i>E</i>	20	2	Slot 2
<i>D</i>	15	2	Slot 1
<i>C</i>	10	1	No slot (slot 1 taken) — skip
<i>B</i>	5	3	Slot 3
<i>A</i>	1	3	No free slot ≤ 3 — skip

Schedule: Slot 1 $\rightarrow D$, Slot 2 $\rightarrow E$, Slot 3 $\rightarrow B$. **Total profit** = $15 + 20 + 5 = 40$.

- Q57.** Find the shortest paths starting from vertex s in this given graph. Show the steps of edge relaxation. **(10 marks)** [CSE 207 | 2006–07]



- Q58.** Once an amateur thief entered into a museum full of tantalizing jewelry, geodes and rare gems. As he is new in this field, he brought just a single backpack with him that has a capacity of 6 kg. The thief planned to get away with the most valuable objects without overloading the backpack. He found out that in a less-secured compartment, there were four (4) solid items, and their information are given in the following table.

Item	Weight (kg)	Value (USD)
Brooch of the queen	3	15
Crown of the king	2	20
Decorated vase	10	30
An ancient coin	2	14

Table for Ques No. 5(a) — Information of valuable items found in the museum

Since the items are unbreakable, for each item, the thief must pick the whole of it or not consider it at all. Now although our thief is amateur in this business, he possesses some algorithmic knowledge. Therefore, he applied a greedy technique and selected the items that gave him the maximum possible profit per weight, and then he left the museum with so much zeal! Now, using the information of the backpack of the thief and the item table given for this question, your first task is to show the steps

of finding the maximum possible profit and the corresponding list of items using the technique that the thief adopted. You have also identified that the scenario at the museum was a special one and only dynamic programming is able to find the optimal solution in that case. Therefore, your second task is to find the optimal profit and the list of items using dynamic programming. **(10+10=20 marks)**

Answer

Given Items & Capacity: $W = 6$ kg.

- Item 1: Brooch ($w_1 = 3, v_1 = 15$)
- Item 2: Crown ($w_2 = 2, v_2 = 20$)
- Item 3: Vase ($w_3 = 10, v_3 = 30$)
- Item 4: Coin ($w_4 = 2, v_4 = 14$)

Part 1: Greedy Approach (Value-to-Weight Ratio)

Item	Weight (w)	Value (v)	Ratio (v/w)	Rank
Crown	2	20	10.0	1
Coin	2	14	7.0	2
Brooch	3	15	5.0	3
Vase	10	30	3.0	4

Execution Steps:

- Pick **Crown** (rank 1): $w = 2 \leq 6$. Remaining capacity = $6 - 2 = 4$. Profit = 20.
- Pick **Coin** (rank 2): $w = 2 \leq 4$. Remaining capacity = $4 - 2 = 2$. Profit = $20 + 14 = 34$.
- Try **Brooch** (rank 3): $w = 3 > 2$ (Capacity exceeded). Skip.
- Try **Vase** (rank 4): $w = 10 > 2$ (Capacity exceeded). Skip.

Greedy Solution: Items = {Crown, Coin}, Total Weight = 4 kg, **Maximum Profit = \$34.**

Part 2: Dynamic Programming (0-1 Knapsack)

Let $dp[i][w]$ be the maximum profit using the first i items with capacity w . The recurrence relation is:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i) \quad \text{if } w_i \leq w \text{ else } dp[i-1][w]$$

DP Table:

i (Item)	w_i, v_i	$w = 0$	1	2	3	4	5	6
0 (None)	-	0	0	0	0	0	0	0
1 (Brooch)	3, 15	0	0	0	15	15	15	15
2 (Crown)	2, 20	0	0	20	20	20	35	35
3 (Vase)	10, 30	0	0	20	20	20	35	35
4 (Coin)	2, 14	0	0	20	20	34	35	35

Traceback for Optimal Selection: Start at the maximum profit $dp[4][6] = 35$:

- $dp[4][6] == dp[3][6]$ ($35 = 35$) $\implies$ **Coin** is **not** included.
- $dp[3][6] == dp[2][6]$ ($35 = 35$) $\implies$ **Vase** is **not** included.
- $dp[2][6] \neq dp[1][6]$ ($35 \neq 15$) $\implies$ **Crown** is **included**. Remaining capacity $w = 6 - 2 = 4$.
- $dp[1][4] \neq dp[0][4]$ ($15 \neq 0$) $\implies$ **Brooch** is **included**. Remaining capacity $w = 4 - 3 = 1$.

Optimal Solution (DP): Items = {Crown, Brooch}, Total Weight = 5 kg, **Optimal Profit = \$35**.

Remark. The greedy approach yields a profit of \$34, which is suboptimal compared to the DP approach (\$35). This demonstrates why the 0-1 Knapsack problem requires Dynamic Programming to guarantee an optimal solution.

BUET · CSE 105

DSA

Dynamic Programming

LCS, MCM, Edit Distance, Knapsack, Memoization

T11 — Dynamic Programming

Q1. Assume that you have obtained a gold bar of integer length n . You plan to cut it into smaller pieces and sell them to the market. For any integer $i \leq n$, you know the profit you will obtain if you sell a gold bar of length i .

- Using the cut-and-paste method, formulate an argument to show that the problem of obtaining the maximum profit by cutting this gold bar into smaller pieces has the optimal substructure property.
- Construct a recursion tree diagram to show that the problem has the overlapping subproblem property.

(7+8=15 marks)

[CSE 105 | 2023–24]

Answer

(a) Optimal Substructure (Cut-and-Paste Argument):

Let $r(n)$ = maximum profit for a bar of length n . Any optimal solution makes at least one cut, say at position k , yielding pieces of length k and $n - k$:

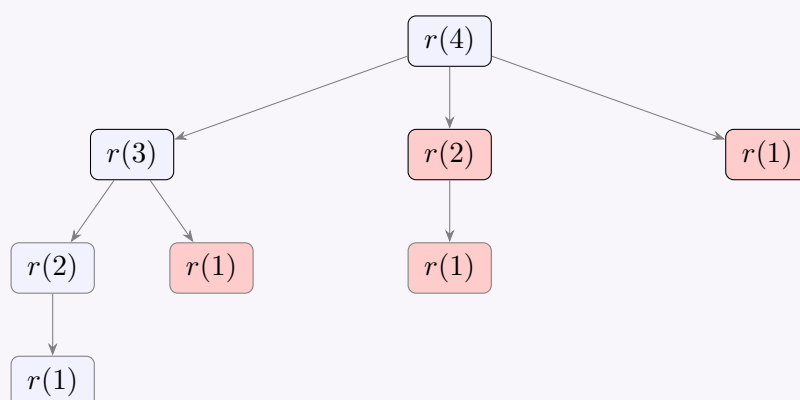
$$r(n) = \max_{1 \leq k \leq n} (p[k] + r(n - k))$$

where $p[k]$ is the price of a bar of length k and $r(0) = 0$.

Claim: The sub-solution for the piece of length $n - k$ must also be optimal.

Proof (cut-and-paste): Suppose the optimal solution for length n splits at k , but the right piece of length $n - k$ is solved *sub-optimally* with profit $r'(n - k) < r(n - k)$. Then replacing that sub-solution with the optimal one gives total profit $p[k] + r(n - k) > p[k] + r'(n - k) = r(n)$, contradicting optimality. Therefore every sub-piece in an optimal solution must itself be solved optimally. ■

(b) Overlapping Subproblems (Recursion Tree for $n = 4$):



Observation: $r(1)$ is computed **4 times**, $r(2)$ is computed **2 times** (red/shaded nodes). For length n , naive recursion solves 2^{n-1} subproblems — exponential. Since the same subproblems recur, dynamic programming stores each $r(i)$ once and solves in $\Theta(n^2)$.

- Q2.** Given the two strings $X = \text{'AGGTAB'}$ and $Y = \text{'GXTYAB'}$, construct the dynamic programming (DP) table to compute the length of the Longest Common Subsequence (LCS) between X and Y . Clearly state the LCS and its length. **(15 marks)** [CSE 105 | 2023–24]

Answer

Recurrence:

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(c[i-1][j], c[i][j-1]) & \text{otherwise} \end{cases}$$

DP Table ($X = \text{AGGTAB}$, $Y = \text{GXTYAB}$):

	ϵ	G	X	T	Y	A	B
ϵ	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1
G	0	1	1	1	1	1	1
G	0	1	1	1	1	1	1
T	0	1	1	2	2	2	2
A	0	1	1	2	2	3	3
B	0	1	1	2	2	3	4

LCS length $= c[6][6] = 4$.

LCS = GTAB

Verification: G(2), T(4), A(5), B(6) appear in X in this order; G(1), T(3), A(5), B(6) appear in Y in this order. ✓

- Q3.** You are given a sequence of matrices $\langle A_1, A_2, \dots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to compute the product of the given chain of matrices. Formulate a recurrence relation to solve this problem. Design an efficient dynamic programming algorithm using the recurrence relation. **(15 marks)** [CSE 105 | 2023–24]

Answer

Recurrence: Let $m[i][j]$ = minimum scalar multiplications for $A_i \cdots A_j$.

$$m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\} & i < j \end{cases}$$

Algorithm (bottom-up, fill by chain length):

Algorithm 110 MATRIX-CHAIN-ORDER(p, n)

```

1: for  $i = 1$  to  $n$  do
2:    $m[i][i] \leftarrow 0$ 
3: end for
4: for  $l = 2$  to  $n$  do  $\triangleright l = \text{chain length}$ 
5:   for  $i = 1$  to  $n - l + 1$  do
6:      $j \leftarrow i + l - 1$ 
7:      $m[i][j] \leftarrow \infty$ 
8:     for  $k = i$  to  $j - 1$  do  $\triangleright \text{try all splits}$ 
9:        $q \leftarrow m[i][k] + m[k + 1][j] + p[i - 1] \cdot p[k] \cdot p[j]$ 
10:      if  $q < m[i][j]$  then
11:         $m[i][j] \leftarrow q$ 
12:         $s[i][j] \leftarrow k$ 
13:      end if
14:    end for
15:  end for
16: end for
17: return  $m[1][n], s$ 

```

Algorithm 111 PRINT-OPTIMAL-PARENS(s, i, j)

```

1: if  $i = j$  then
2:   print " $A_i$ "
3: else
4:   print "("
5:   PRINT-OPTIMAL-PARENS( $s, i, s[i][j]$ )
6:   PRINT-OPTIMAL-PARENS( $s, s[i][j] + 1, j$ )
7:   print ")"
8: end if

```

Time: $\Theta(n^3)$. **Space:** $\Theta(n^2)$.

Q4. You are given a sequence of matrices $\langle A_1, A_2, \dots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to determine the product of the given chain of matrices.

- Formulate arguments to show that the MCM problem has the optimal substructure property.
- Compose a recursion tree diagram to show that MCM has the overlapping subproblem property.
- Design an efficient algorithm using the dynamic programming approach to solve this problem.

(8+7+10=25 marks)

[CSE 105 | 2022–23]

Answer**Matrix Chain Multiplication (MCM): Optimal Substructure, Overlapping Subproblems, DP Algorithm****Setup:**

Given $\langle A_1, A_2, \dots, A_n \rangle$ with dimensions $\langle p_0, p_1, \dots, p_n \rangle$ where A_i is $p_{i-1} \times p_i$. Find the minimum scalar multiplications to compute $A_1 A_2 \dots A_n$.

Let $m[i][j]$ = minimum cost to multiply $A_i A_{i+1} \dots A_j$.

Part (a): Optimal Substructure**(8 marks)**

Theorem: An optimal parenthesisation of $A_i \dots A_j$ contains optimal parenthesisations of its subchains.

Proof:

Suppose the optimal split of $A_i \dots A_j$ is at position k (i.e. $(A_i \dots A_k)(A_{k+1} \dots A_j)$) with total cost $m[i][j]$:

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j$$

Claim: $m[i][k]$ must be the optimal cost for $A_i \dots A_k$.

Proof by contradiction: Suppose there exists a cheaper way to parenthesise $A_i \dots A_k$ with cost $m'[i][k] < m[i][k]$. Then:

$$m'[i][k] + m[k+1][j] + p_{i-1} p_k p_j < m[i][k] + m[k+1][j] + p_{i-1} p_k p_j = m[i][j]$$

This contradicts the optimality of $m[i][j]$. Therefore $m[i][k]$ must be optimal. By symmetric argument, $m[k+1][j]$ must also be optimal. ■

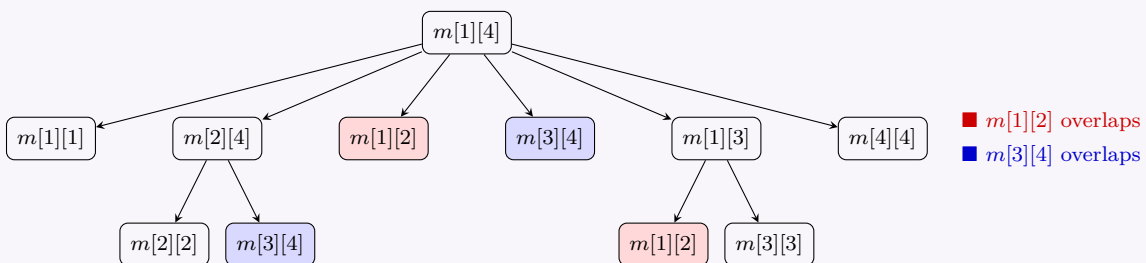
Consequence: The global optimal solution can be built from optimal solutions to subproblems $\Rightarrow$ **optimal substructure holds**.

Part (b): Overlapping Subproblems**(7 marks)**

Recursion tree for $n = 4$, computing $m[1][4]$:

Part (b): Overlapping Subproblems**(7 marks)**

Recursion tree for $n = 4$, computing $m[1][4]$:



Observation: Subproblem $m[1][2]$ is computed when $k = 2$ (right child of root) and also as the left child when computing $m[1][3]$ under $k = 3$.

Similarly, $m[1][1]$ appears three times. In general, the number of distinct subproblems is only $\Theta(n^2)$, but naive recursion recomputes them exponentially many times ($\Omega(2^n)$ total calls).

Conclusion: MCM has overlapping subproblems $\Rightarrow$ memoisation or bottom-up DP is essential.

Part (c): DP Algorithm

(10 marks)

Recurrence:

$$m[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p_{i-1}p_kp_j) & \text{if } i < j \end{cases}$$

Algorithm (bottom-up, fill by chain length):

Algorithm 112 MATRIX-CHAIN-ORDER(p, n)

```

1: for  $i = 1$  to  $n$  do
2:    $m[i][i] \leftarrow 0$ 
3: end for
4: for  $l = 2$  to  $n$  do  $\triangleright l = \text{chain length}$ 
5:   for  $i = 1$  to  $n - l + 1$  do
6:      $j \leftarrow i + l - 1$ 
7:      $m[i][j] \leftarrow \infty$ 
8:     for  $k = i$  to  $j - 1$  do  $\triangleright \text{try all splits}$ 
9:        $q \leftarrow m[i][k] + m[k+1][j] + p[i-1] \cdot p[k] \cdot p[j]$ 
10:      if  $q < m[i][j]$  then
11:         $m[i][j] \leftarrow q$ 
12:         $s[i][j] \leftarrow k$ 
13:      end if
14:    end for
15:  end for
16: end for
17: return  $m[1][n], s$ 
```

Reconstruction:

Algorithm 113 PRINT-OPTIMAL-PARENS(s, i, j)

```

1: if  $i = j$  then
2:   print " $A_i$ "
3: else
4:   print "("
5:   PRINT-OPTIMAL-PARENS( $s, i, s[i][j]$ )
6:   PRINT-OPTIMAL-PARENS( $s, s[i][j] + 1, j$ )
7:   print ")"
8: end if
```

Worked Example: $A_1: 10 \times 30$, $A_2: 30 \times 5$, $A_3: 5 \times 60$, $A_4: 60 \times 2$. Dimension array: $p = [10, 30, 5, 60, 2]$.

Chain length 2:

$$m[1][2] = p_0 \cdot p_1 \cdot p_2 = 10 \cdot 30 \cdot 5 = 1500$$

$$m[2][3] = p_1 \cdot p_2 \cdot p_3 = 30 \cdot 5 \cdot 60 = 9000$$

$$m[3][4] = p_2 \cdot p_3 \cdot p_4 = 5 \cdot 60 \cdot 2 = 600$$

Chain length 3:

$$\begin{aligned} m[1][3] &= \min(m[1][1] + m[2][3] + p_0 p_1 p_3, m[1][2] + m[3][3] + p_0 p_2 p_3) \\ &= \min(0 + 9000 + 10 \cdot 30 \cdot 60, 1500 + 0 + 10 \cdot 5 \cdot 60) \\ &= \min(27000, 4500) = 4500 \quad (k = 2) \end{aligned}$$

$$\begin{aligned} m[2][4] &= \min(m[2][2] + m[3][4] + p_1 p_2 p_4, m[2][3] + m[4][4] + p_1 p_3 p_4) \\ &= \min(0 + 600 + 30 \cdot 5 \cdot 2, 9000 + 0 + 30 \cdot 60 \cdot 2) \\ &= \min(900, 12600) = 900 \quad (k = 2) \end{aligned}$$

Chain length 4:

k	Expression	Value
1	$m[1][1] + m[2][4] + p_0 p_1 p_4 = 0 + 900 + 10 \cdot 30 \cdot 2$	1500
2	$m[1][2] + m[3][4] + p_0 p_2 p_4 = 1500 + 600 + 10 \cdot 5 \cdot 2$	2200
3	$m[1][3] + m[4][4] + p_0 p_3 p_4 = 4500 + 0 + 10 \cdot 60 \cdot 2$	5700
Minimum		1500 at $k = 1$

$m[1][4] = 1500$, achieved by $(A_1 \cdot (A_2 \cdot (A_3 \cdot A_4)))$.

Verification: $A_3 A_4$: $5 \times 60 \cdot 60 \times 2 = 600$ mults $\Rightarrow 5 \times 2$.

$A_2(A_3 A_4)$: $30 \times 5 \cdot 5 \times 2 = 300$ mults $\Rightarrow 30 \times 2$.

$A_1(A_2 A_3 A_4)$: $10 \times 30 \cdot 30 \times 2 = 600$ mults.

Total: $600 + 300 + 600 = \mathbf{1500}$ ✓

Time Complexity:

- Three nested loops: l, i, k , each $O(n)$.
- Total: $\Theta(n^3)$ time.
- Space: $\Theta(n^2)$ for m and s tables.

$$T(n) = \Theta(n^3), \quad S(n) = \Theta(n^2)$$

Q5. Using a dynamic programming algorithm, determine the minimum edit distance alignment of the two strings $X = \text{ATCGGT}$ and $Y = \text{ACCDT}$, where both the gap penalty and mismatch penalty are 1 and the cost of a match is 0. Show the DP table, the alignments, and mark the paths (which correspond to the alignments) in

the DP table. (10 marks)

[CSE 105 | 2022–23]

Answer

Minimum Edit Distance: $X = \text{ATCGGT}$ vs $Y = \text{ACCDT}$

Setup:

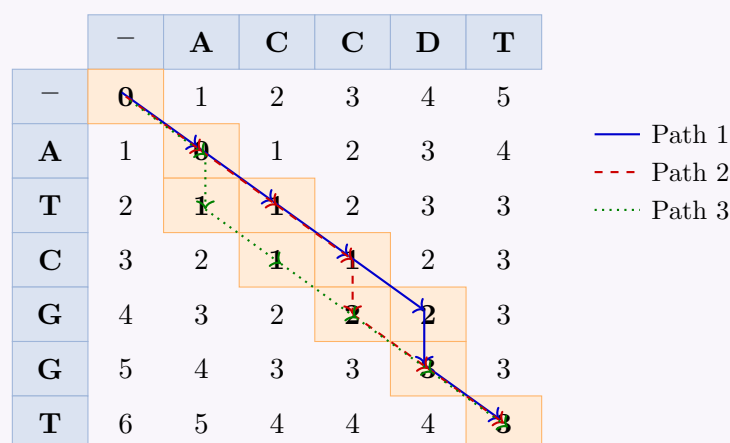
- $X = \text{ATCGGT}$ ($m = 6$), $Y = \text{ACCDT}$ ($n = 5$)
- Gap penalty = 1, Mismatch penalty = 1, Match cost = 0

Recurrence:

$$dp[i][j] = \min \begin{cases} dp[i-1][j-1] + \delta(X[i], Y[j]) & \text{(match/mismatch)} \\ dp[i-1][j] + 1 & \text{(gap in Y)} \\ dp[i][j-1] + 1 & \text{(gap in X)} \end{cases}$$

where $\delta(a, b) = 0$ if $a = b$, else 1.

DP Table & Path Traceback



Minimum edit distance = $dp[6][5] = 3$

Traceback Analysis: Starting from $dp[6][5] = 3$, we must follow valid moves backward. At (5, 4), the path splits, and then splits again, giving exactly **3 optimal paths**:

- **Split 1 at (5, 4):** We can move from (4, 4) via a gap in Y ($2 + 1 = 3$) [Leads to Path 1], **OR** move from (4, 3) via G-D mismatch ($2 + 1 = 3$) [Leads to Paths 2 & 3].
- **Split 2 at (4, 3):** We can move from (3, 3) via a gap in Y ($1 + 1 = 2$) [Leads to Path 2], **OR** move from (3, 2) via G-C mismatch ($1 + 1 = 2$) [Leads to Path 3].

The 3 Optimal Alignments (M = Match, MM = Mismatch)

Alignment 1 (Follows Path 1: Blue Solid)

X:	A	T	C	G	G	T
	M	MM	M	MM	gap	M
Y:	A	C	C	D	–	T
Cost: $0 + 1 + 0 + 1 + 1 + 0 = \mathbf{3}$						

Alignment 2 (Follows Path 2: Red Dashed)

X:	A	T	C	G	G	T
	M	MM	M	gap	MM	M
Y:	A	C	C	–	D	T
Cost: $0 + 1 + 0 + 1 + 1 + 0 = \mathbf{3}$						

Alignment 3 (Follows Path 3: Green Dotted)

X:	A	T	C	G	G	T
	M	gap	M	MM	MM	M
Y:	A	–	C	C	D	T
Cost: $0 + 1 + 0 + 1 + 1 + 0 = \mathbf{3}$						

- Q6.** Given two strings, the longest common subsequence (LCS) problem is to find the longest subsequence present in both strings in the same relative order, but not necessarily contiguous. For example, for $S_1 = \text{``ALGORITHM''}$  and  $S_2 = \text{``LITHIUM''}$, the LCS is ``LITHM'' . Present a dynamic programming algorithm to find the length of the LCS of two strings. Derive a proof of correctness of the algorithm and also derive its running time. **(8+6+3=17 marks)** [CSE 105 | 2021–22]

Answer

Longest Common Subsequence (LCS): DP Algorithm, Correctness, and Running Time

Problem Definition:

Given two strings $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$, find the length of their longest common subsequence.

Part 1: DP Algorithm

(8 marks)

Optimal Substructure:

Let $L[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$.

Recurrence:

$$L[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L[i-1][j-1] + 1 & \text{if } x_i = y_j \\ \max(L[i-1][j], L[i][j-1]) & \text{if } x_i \neq y_j \end{cases}$$

Algorithm 114 LCS-LENGTH(X, Y, m, n)

```

1: for  $i = 0$  to  $m$  do
2:    $L[i][0] \leftarrow 0$ 
3: end for
4: for  $j = 0$  to  $n$  do
5:    $L[0][j] \leftarrow 0$ 
6: end for
7: for  $i = 1$  to  $m$  do
8:   for  $j = 1$  to  $n$  do
9:     if  $X[i] = Y[j]$  then
10:       $L[i][j] \leftarrow L[i-1][j-1] + 1$ 
11:     else
12:       $L[i][j] \leftarrow \max(L[i-1][j], L[i][j-1])$ 
13:     end if
14:   end for
15: end for
16: return  $L[m][n]$ 

```

Example: $S_1 = \text{ALGORITHM}$ ($m = 9$), $S_2 = \text{LITHIUM}$ ($n = 7$).

LCS Table (partial, showing key cells):

	–	L	I	T	H	I	U	M
–	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
L	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
O	0	1	1	1	1	1	1	1
R	0	1	1	1	1	1	1	1
I	0	1	2	2	2	2	2	2
T	0	1	2	3	3	3	3	3
H	0	1	2	3	4	4	4	4
M	0	1	2	3	4	4	4	5

LCS length = $L[9][7] = 5$. Traceback gives LCS = **LITHM**.

Part 2: Proof of Correctness**(6 marks)**

Claim: $L[i][j]$ correctly gives the length of the LCS of $X[1..i]$ and $Y[1..j]$.

Proof by strong induction on $i + j$:

Base case: When $i = 0$ or $j = 0$, one string is empty, so LCS length = $0 = L[i][j]$.
✓

Inductive step: Assume $L[i'][j']$ is correct for all $i' + j' < i + j$. We show $L[i][j]$ is correct.

Case 1: $x_i = y_j$.

Any LCS of $X[1..i]$ and $Y[1..j]$ either includes both x_i and y_j or it does not.

- If it includes $x_i = y_j$: the remaining LCS is a common subsequence of $X[1..i-1]$ and $Y[1..j-1]$, so its length is at most $L[i-1][j-1]$. Thus LCS length $\leq L[i-1][j-1] + 1$.
- We can always extend any LCS of $X[1..i-1]$ and $Y[1..j-1]$ by appending $x_i = y_j$, giving a common subsequence of length $L[i-1][j-1] + 1$.

Therefore $\text{LCS} = L[i-1][j-1] + 1$. ✓

Case 2: $x_i \neq y_j$.

Both x_i and y_j cannot both appear at the *end* of the same LCS. Hence:

- If the LCS does not end with x_i : $\text{LCS of } X[1..i] \text{ and } Y[1..j]$ is the same as $\text{LCS of } X[1..i-1] \text{ and } Y[1..j] = L[i-1][j]$.
- If the LCS does not end with y_j : $\text{LCS} = \text{LCS of } X[1..i] \text{ and } Y[1..j-1] = L[i][j-1]$.

Since at least one must hold: $\text{LCS} = \max(L[i-1][j], L[i][j-1])$. ✓

By induction, $L[m][n]$ correctly gives the LCS length. ■

Part 3: Running Time

(3 marks)

- The DP table has $(m+1)(n+1)$ cells.
- Each cell is filled in $O(1)$ time (one comparison, one max, constant work).
- Total time: $\Theta(mn)$.
- Space: $\Theta(mn)$ for the table (reducible to $O(\min(m, n))$ if only length is needed).

$$T(m, n) = \Theta(mn)$$

Q7. Solve the following 0-1 knapsack problem instance (capacity = 6) using dynamic programming. Show the DP table and find the optimal set of items.

Item	Weight	Value
1	2	8
2	1	7
3	2	12
4	2	6
5	2	10

(15 marks)

[CSE 203 | 2021–22]

Answer

Recurrence: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$ if $w_i \leq w$, else $dp[i-1][w]$.

DP Table (rows = items 1–5, cols = capacity 0–6):

DP Table:

	w=0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1(w=2, v=8)	0	0	8	8	8	8	8
2(w=1, v=7)	0	7	8	15	15	15	15
3(w=2, v=12)	0	7	12	19	20	27	27
4(w=2, v=6)	0	7	12	19	20	27	27
5(w=2, v=10)	0	7	12	19	22	29	30

Optimal value = $dp[5][6] = 30$.

Traceback: $dp[5][6] = 30 \neq dp[4][6] = 27 \Rightarrow$ include item 5, $w \leftarrow 4$; $dp[4][4] = 20 = dp[3][4] \Rightarrow$ skip item 4; $dp[3][4] = 20 \neq dp[2][4] = 15 \Rightarrow$ include item 3, $w \leftarrow 2$; $dp[2][2] = 8 = dp[1][2] \Rightarrow$ skip item 2; $dp[1][2] = 8 \neq dp[0][2] = 0 \Rightarrow$ include item 1.

Optimal set: $\{1, 3, 5\}$, weight = $2 + 2 + 2 = 6$, value = $8 + 12 + 10 = 30$.

Q8. Find the best alignment (minimum edit distance) of the following two DNA sequences using dynamic programming. Use gap penalty = 3, mismatch penalty = 2, match penalty = 0. Show the DP table, the alignment, and mark the corresponding path.

CCAGTGC vs CGATC

(15 marks)

[CSE 203 | 2021–22]

Answer

DP Table (gap penalty=3, mismatch=2, match=0):

	ε	C	G	A	T	C
ε	0	3	6	9	12	15
C	3	0	3	6	9	12
C	6	3	2	5	8	9
A	9	6	5	2	5	8
G	12	9	6	5	4	7
T	15	12	9	8	5	6
G	18	15	12	11	8	7
C	21	18	15	14	11	8

Minimum alignment cost = 8.

Optimal alignment (traceback):

X: C C A G T G C

Y: C G A - T - C

(Two gaps inserted in Y; one mismatch $C \leftrightarrow G$.) Cost: $0 + 2 + 0 + 3 + 0 + 3 + 0 = 8$.

- Q9.** Give a dynamic programming algorithm to find the length of the longest common subsequence (LCS) of two strings. (A common subsequence appears in both strings in the same relative order but not necessarily contiguously.) **(20 marks)** [CSE 203 | 2021–22]

Answer

Algorithm 115 LCS-LENGTH(X, Y)

```

1:  $m \leftarrow |X|$ ;  $n \leftarrow |Y|$ 
2: for  $i \leftarrow 0$  to  $m$  do  $c[i][0] \leftarrow 0$ 
3: end for
4: for  $j \leftarrow 0$  to  $n$  do  $c[0][j] \leftarrow 0$ 
5: end for
6: for  $i \leftarrow 1$  to  $m$  do
7:   for  $j \leftarrow 1$  to  $n$  do
8:     if  $X[i] = Y[j]$  then  $c[i][j] \leftarrow c[i-1][j-1] + 1$ ;  $b[i][j] \leftarrow \nwarrow$ 
9:     else if  $c[i-1][j] \geq c[i][j-1]$  then  $c[i][j] \leftarrow c[i-1][j]$ ;  $b[i][j] \leftarrow \uparrow$ 
10:    else  $c[i][j] \leftarrow c[i][j-1]$ ;  $b[i][j] \leftarrow \leftarrow$ 
11:    end if
12:  end for
13: end for
14: return  $c, b$ 
```

Complexity Analysis. Time: $\Theta(mn)$. Space: $\Theta(mn)$ for table; $\Theta(\min(m, n))$ if only length needed. Printing LCS: $O(m + n)$ by following b table.

- Q10.** Derive a recursive definition to find $m[i, j]$ (the minimum number of multiplications to compute $A_i \cdot A_{i+1} \cdots A_j$). With a recursion tree, describe the overlapping subproblem property of Matrix Chain Multiplication (MCM). Then, given six matrices $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$, $A_6(20 \times 25)$ and the following partial DP table, give an ordering to populate the marked cells and calculate $m[2, 5]$:

	1	2	3	4	5	6
1	0	15750	7875	$m[1, 4]$	$m[1, 5]$	$m[1, 6]$
2		0	2625	4375	$m[2, 5]$	$m[2, 6]$
3			0	750	2500	$m[3, 6]$
4				0	1000	$m[4, 6]$
5					0	$m[5, 6]$
6						0

(7+6+10 marks)

[CSE 203 | 2020–21]

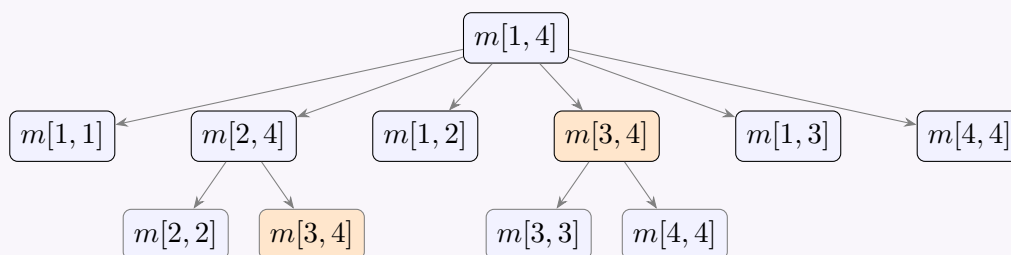
Answer

Recursive Definition of $m[i, j]$:

Let $m[i, j]$ = minimum number of scalar multiplications to compute $A_i \cdot A_{i+1} \cdots A_j$, where A_k has dimension $p_{k-1} \times p_k$.

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j\} & \text{if } i < j \end{cases}$$

Overlapping Subproblems (Recursion Tree for $m[1, 4]$):



Overlapping subproblem: $m[3, 4]$ appears *twice* in the recursion tree for $m[1, 4]$. DP avoids recomputation by storing results in a table.

Order to Populate Marked Cells:

Dependencies must be resolved bottom-up (shorter chains first):

$$m[5, 6] \rightarrow m[4, 6] \rightarrow m[2, 5] \rightarrow m[1, 4] \rightarrow m[1, 5] \rightarrow m[2, 6] \rightarrow m[1, 6]$$

Computing $m[2, 5]$:

Dimensions: $p = [30, 35, 15, 5, 10, 20, 25]$, so $p_1 = 35$, $p_2 = 15$, $p_3 = 5$, $p_4 = 10$, $p_5 = 20$.

$$m[2, 5] = \min_{2 \leq k \leq 4} \{m[2, k] + m[k+1, 5] + p_1 \cdot p_k \cdot p_5\}$$

k	Expression	Value
2	$m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20$	13000
3	$m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20$	7125
4	$m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20$	11375
Min		7125 (at $k = 3$)

$$\therefore m[2, 5] = \boxed{7125}$$

Optimal split for $m[2, 5]$: $k = 3$, i.e. $(A_2 A_3)(A_4 A_5)$ with 7125 multiplications.

Q11. At the beginning of the COVID-19 pandemic, two biologists were identifying the similarity between COVID-19 and SARS viruses using genome sequences $S'_1 = \langle A, T, T, G, C, A, T \rangle$ and $S'_2 = \langle T, A, G, C, C, T \rangle$.

- (Dr. Brown's task) Write the recursive definition for the Longest Common Subsequence (LCS) problem and populate the DP table with arrows showing how cell values propagate. Find the length and the LCS itself.
- (Dr. William's task) Write the recursive definition for the edit distance problem using operations {Insert (cost 1), Delete (cost 1), Replacement (cost 1), Keep (cost 0)}. Populate the DP table and show the edit distance cost and sequence of required operations.

(15 marks)

[CSE 203 | 2020–21]

Answer

(a) **LCS — Recursive Definition:**

$$c[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(c[i-1][j], c[i][j-1]) & \text{otherwise} \end{cases}$$

DP Table ($X = \langle A, T, T, G, C, A, T \rangle$, $Y = \langle T, A, G, C, C, T \rangle$):

	ϵ	T	A	G	C	C	T
ϵ	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1
T	0	1	1	1	1	1	2
T	0	1	1	1	1	1	2
G	0	1	1	2	2	2	2
C	0	1	1	2	3	3	3
A	0	1	2	2	3	3	3
T	0	1	2	2	3	3	4

LCS length = 4.

Multiple LCS Tracebacks (Branching):

Tracing back from $c[7][6] = 4$, we encounter branches where $c[i-1][j] = c[i][j-1]$. This results in 2 distinct valid Longest Common Subsequences:

- **LCS 1:** $\langle A, G, C, T \rangle$ (Matching A at $X[1], Y[2]$)
- **LCS 2:** $\langle T, G, C, T \rangle$ (Matching T at $X[2]$ or $X[3]$ with $Y[1]$)

(b) Edit Distance — Recursive Definition:

$$dp[i][j] = \begin{cases} j & \text{if } i = 0 \\ i & \text{if } j = 0 \\ dp[i-1][j-1] & \text{if } X[i] = Y[j] \\ 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) & \text{otherwise} \end{cases}$$

DP Table:

	ϵ	T	A	G	C	C	T
ϵ	0	1	2	3	4	5	6
A	1	1	1	2	3	4	5
T	2	1	2	2	3	4	4
T	3	2	2	3	3	4	4
G	4	3	3	2	3	4	5
C	5	4	4	3	2	3	4
A	6	5	4	4	3	3	4
T	7	6	5	5	4	4	3

Edit distance = 3.

Sequence of Operations (Traceback):

Unlike LCS, tracing back from $dp[7][6] = 3$ mathematically yields only **one** strictly optimal minimum-cost path for this matrix.

- Delete** A (from X)
- Keep** T

- (c) **Replace** $T \rightarrow A$
- (d) **Keep** G
- (e) **Keep** C
- (f) **Replace** $A \rightarrow C$
- (g) **Keep** T

Q12. Discuss a problem scenario where adopting memoization over bottom-up dynamic programming can result in a faster solution. Justify your answer. (5 marks) [CSE 203 | 2020–21]

Answer

When memoization can be faster:

In problems where *not all sub-problems need to be solved*, memoization (top-down) only computes sub-problems that are actually reachable from the original problem, while bottom-up always fills the entire table.

Example: the longest path in a DAG. For a DAG with V vertices and E edges, if the query is about a specific vertex s and the DAG is sparse, memoized DFS from s visits only the $O(V + E)$ reachable sub-problems, whereas bottom-up DP would process all V vertices unconditionally.

Justification: if only a fraction of the $O(n^2)$ (or $O(2^n)$) sub-problems are needed (e.g., sparse matrix chain, or a DAG with limited reachability), memoization skips unreachable states entirely. Bottom-up computes everything, wasting time on irrelevant states.

Q13. Apply dynamic programming to calculate the edit distance of the following two DNA sequences, where the cost of insertion or deletion is 2 and the cost of substitution is 1. Write the recurrence relation, explain the intuition behind it, populate the DP table, and calculate the edit distance.

TGCATAT vs ATCCGA

(15 marks)

[CSE 203 | 2019–20]

Answer

Edit Distance: TGCATAT vs ATCCGA

(Insertion/Deletion = 2, Substitution = 1, Match = 0)

Recurrence Relation:

Let $dp[i][j]$ = minimum edit distance between $X[1..i]$ and $Y[1..j]$:

$$dp[i][j] = \begin{cases} 2i & j = 0 \\ 2j & i = 0 \\ dp[i-1][j-1] & X[i] = Y[j] \\ \min \begin{cases} dp[i-1][j-1] + 1 & \text{(Substitute)} \\ dp[i-1][j] + 2 & \text{(Delete)} \\ dp[i][j-1] + 2 & \text{(Insert)} \end{cases} & X[i] \neq Y[j] \end{cases}$$

DP Table (X = rows, Y = cols):

	–	A	T	C	C	G	A
–	0	2	4	6	8	10	12
T	2	1	2	4	6	8	10
G	4	3	2	3	5	6	8
C	6	5	4	2	3	5	7
A	8	6	6	4	3	4	5
T	10	8	6	6	5	4	5
A	12	10	8	7	7	6	4
T	14	12	10	9	8	8	6

Optimal Alignment:

Tracing back strictly from $dp[7][6]$ results in the following exact alignment path:

X :	T	G	C	A	T	A	T
Op:	sub	sub	M	sub	sub	M	del
Y :	A	T	C	C	G	A	–
Cost:	1	1	0	1	1	0	2

Final Edit Distance = $dp[7][6] = 6$.

- Q14.** Briefly explain the steps to be followed for solving a problem using dynamic programming. **(10 marks)** [CSE 203 | 2018–19]

Answer

Steps for Solving a Problem Using Dynamic Programming

Dynamic programming (DP) solves optimization or counting problems by breaking them into overlapping subproblems and storing results to avoid recomputation. The standard methodology follows these steps:

Step 1. Characterize the Structure of an Optimal Solution

Identify whether the problem has **optimal substructure**: an optimal solution to the problem contains optimal solutions to subproblems. Show that the global optimum can be expressed in terms of optimal subproblem solutions.

Example (LCS): An LCS of $X[1..m]$ and $Y[1..n]$ contains an LCS of some prefix $X[1..i]$ and $Y[1..j]$.

Step 2. Define the Subproblems and State

Clearly define what $dp[i]$ or $dp[i][j]$ (the DP state) represents. The state must capture all information needed to solve smaller subproblems independently.

Example (Knapsack): $dp[i][w]$ = maximum value using items $\{1, \dots, i\}$ with capacity w .

Step 3. Write the Recurrence Relation

Express the solution to a subproblem in terms of solutions to smaller subproblems. Identify:

- **Base cases:** smallest subproblems with known answers (e.g. $dp[0] = 0$).
- **Recursive case:** how to combine subproblem solutions.

Example (Edit Distance): $dp[i][j] = \min(dp[i-1][j-1] + \delta, dp[i-1][j] + 1, dp[i][j-1] + 1)$

Step 4. Verify Overlapping Subproblems

Confirm that subproblems recur multiple times in the recursion tree. This justifies memoisation or bottom-up tabulation (otherwise, divide-and-conquer is sufficient).

Example (Fibonacci): $F(3)$ is called during computation of both $F(4)$ and $F(5)$.

Step 5. Choose Implementation: Top-down or Bottom-up

- **Top-down (Memoisation):** Write the recursive solution; cache results in a table. Natural to implement; may have recursion overhead.
- **Bottom-up (Tabulation):** Fill the table iteratively in an order where subproblems are solved before they are needed. Usually faster and avoids stack overflow.

Step 6. Compute the Solution Bottom-up (or with Memoisation)

Fill the DP table according to the recurrence, starting from the base cases and proceeding to the final answer. Ensure the correct fill order (e.g. row by row, by chain length, etc.).

Step 7. Determine the Final Answer

Read off the answer from the appropriate DP table entry (e.g. $dp[m][n]$ for two-string problems, $dp[n]$ for single-sequence problems).

Step 8. Reconstruct the Optimal Solution (Traceback)

If the actual solution (not just its value) is needed, trace back through the DP table using stored decisions (a separate $choice[i][j]$ table) or by re-deriving decisions from the values.

Example: In LCS, at each cell check whether the diagonal, up, or left move was used and reconstruct the common subsequence.

Step 9. Analyse Time and Space Complexity

Count the number of subproblems and the time per subproblem:

$$T = (\text{number of subproblems}) \times (\text{time per subproblem})$$

Optimise space if possible (e.g. rolling array reduces $O(mn)$ to $O(\min(m, n))$).

Summary Checklist:

Step	Key Question
1. Optimal substructure	Does the optimum use optimal sub-solutions?
2. Define state	What does $dp[i][j]$ mean?
3. Recurrence	How to compute from smaller states?
4. Overlapping subproblems	Are subproblems reused?
5. Implementation	Top-down or bottom-up?
6. Fill table	In what order?
7. Final answer	Which cell?
8. Traceback	How to recover the solution?
9. Complexity	Time and space?

Q15. Dream coins are used by the people of Dreamland. Only the following coins are available: \$1, \$7, \$11, and \$15.

- Write an example of an amount where the greedy strategy for the Coin-Change problem does *not* provide an optimal solution. Mention both the greedy and optimal solutions.
- Using dynamic programming, find the minimum number of coins that add up to \$20.

(10 marks)

[CSE 203 | 2018–19]

Answer**(a) Greedy Failure — Counter-Example**

The greedy strategy always picks the *largest coin not exceeding* the remaining amount. Consider the target **\$21**.

Step 1. Remaining = 21. Largest coin ≤ 21 is \$15. Pick \$15. Remaining = 6.

Step 2. Remaining = 6. No coin ≤ 6 except \$1. Pick \$1 six times. Remaining = 0.

$$\$15 + \underbrace{\$1 + \$1 + \$1 + \$1 + \$1 + \$1}_{6 \text{ coins}} = \$21 \quad \Rightarrow \quad \mathbf{7 \text{ coins}}$$

$$\$7 + \$7 + \$7 = \$21 \quad \Rightarrow \quad \mathbf{3 \text{ coins}}$$

Conclusion: For \$21, greedy uses **7 coins** while optimal uses **3 coins**. Greedy fails because choosing the locally largest coin (\$15) prevents the globally better choice of three \$7 coins.

(b) Dynamic Programming for \$20

Let $dp[v]$ = minimum number of coins to make amount v . **Recurrence:**

$$dp[v] = \min_{c \in \{1, 7, 11, 15\}} (dp[v - c] + 1), \quad dp[0] = 0.$$

DP Table:

v	$dp[v]$	Best coin chosen	
0	0	— (base case)	
1	1	\$1	
2	2	\$1	
3	3	\$1	
4	4	\$1	
5	5	\$1	
6	6	\$1	
7	1	\$7	$dp[0] + 1$
8	2	\$7	$dp[1] + 1$
9	3	\$7	$dp[2] + 1$
10	4	\$7	$dp[3] + 1$
11	1	\$11	$dp[0] + 1$
12	2	\$11	$dp[1] + 1$
13	3	\$11	$dp[2] + 1$
14	2	\$7	$dp[7] + 1$
15	1	\$15	$dp[0] + 1$
16	2	\$15	$dp[1] + 1$
17	3	\$15	$dp[2] + 1$
18	2	\$11	$dp[7] + 1$
19	3	\$11	$dp[8] + 1$
20	4	see computation below	

Computing $dp[20]$:

Coin c	Sub-problem	Value	Coins
\$1	$dp[19] + 1$	$3 + 1$	4
\$7	$dp[13] + 1$	$3 + 1$	4
\$11	$dp[9] + 1$	$3 + 1$	4
\$15	$dp[5] + 1$	$5 + 1$	6
Minimum			4

$$\therefore dp[20] = \min(4, 4, 4, 6) = \boxed{4}$$

Minimum coins to make \$20 = 4.

Optimal decomposition: \$11 + \$7 + \$1 + \$1 = \$20

Q16. Consider the following 0/1 knapsack problem instance (capacity = 7):

Item	Weight	Value
1	1	50
2	3	60
3	5	100
4	2	70
5	2	80

Solve this instance using dynamic programming. Show the DP table and find all solutions using the table. **(15 marks)** [CSE 203 | 2018–19]

Answer

We solve the 0/1 Knapsack problem using dynamic programming.

Let $dp[i][w]$ = maximum value using items $1 \dots i$ with capacity w .

Recurrence:

$$dp[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ dp[i-1][w] & \text{if } w_i > w \\ \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) & \text{otherwise} \end{cases}$$

DP Table ($dp[i][w]$, rows = items, columns = capacity):

Item $i \setminus w$	1	2	3	4	5	6	7
0 (none)	0	0	0	0	0	0	0
1 ($w=1, v=50$)	50	50	50	50	50	50	50
2 ($w=3, v=60$)	50	50	60	110	110	110	110
3 ($w=5, v=100$)	50	50	60	110	110	150	150
4 ($w=2, v=70$)	50	70	120	120	130	180	180
5 ($w=2, v=80$)	50	80	130	150	200	200	210

$\therefore$ Maximum value = 210

Traceback to Find All Optimal Solutions

Starting from $dp[5][7] = 210$. At each step, if $dp[i][w] \neq dp[i-1][w]$, item i was included; otherwise it was skipped. If both give the same value, both branches yield optimal solutions.

Step	Item i	$dp[i][w]$	$dp[i-1][w]$	Decision
1	5	$dp[5][7] = 210$	$dp[4][7] = 180$	$210 \neq 180 \Rightarrow$ Include item 5 Move to $dp[4][7-2] = dp[4][5] = 130$
2	4	$dp[4][5] = 130$	$dp[3][5] = 110$	$130 \neq 110 \Rightarrow$ Include item 4 Move to $dp[3][5-2] = dp[3][3] = 60$
3	3	$dp[3][3] = 60$	$dp[2][3] = 60$	$60 = 60 \Rightarrow$ Skip item 3 Move to $dp[2][3] = 60$
4	2	$dp[2][3] = 60$	$dp[1][3] = 50$	$60 \neq 50 \Rightarrow$ Include item 2 Move to $dp[1][3-3] = dp[1][0] = 0$
5	1	$dp[1][0] = 0$	$dp[0][0] = 0$	$0 = 0 \Rightarrow$ Skip item 1

Items selected: 2, 4, 5

$$\text{Weight} = 3 + 2 + 2 = 7 \leq 7 \quad \text{Value} = 60 + 70 + 80 = \mathbf{210}$$

Note: At step 3, item 3 could also be *included* if remaining capacity permits. But $w_3 = 5 > 3$ (remaining capacity), so item 3 cannot be included — the skip is forced, not a choice. Hence there is exactly **one unique optimal solution**: $\{2, 4, 5\}$ with value **210**.

- Q17.** Find all longest common subsequences of $X = \langle T, A, A, G, U, A, T, U \rangle$ and $Y = \langle A, U, A, G, T, U, T \rangle$ using dynamic programming. Also find a shortest common supersequence of X and Y using an LCS of X and Y . **(15 marks)** [CSE 203 | 2018–19]

Answer

All LCS of $X = \langle T, A, A, G, U, A, T, U \rangle$ and $Y = \langle A, U, A, G, T, U, T \rangle$

Step 1: Build the LCS DP Table

$m = 8, n = 7$.

$L[i][j]$	–	A	U	A	G	T	U	T
	0	1	2	3	4	5	6	7
– $i=0$	0	0	0	0	0	0	0	0
T $i=1$	0	0	0	0	0	1	1	1
A $i=2$	0	1	1	1	1	1	1	1
A $i=3$	0	1	1	2	2	2	2	2
G $i=4$	0	1	1	2	3	3	3	3
U $i=5$	0	1	2	2	3	3	4	4
A $i=6$	0	1	2	3	3	3	4	4
T $i=7$	0	1	2	3	3	4	4	5
U $i=8$	0	1	2	3	3	4	5	5

LCS length = $L[8][7] = 5$.

Step 2: Find All LCS by Traceback

Trace back from $L[8][7] = 5$. At each cell (i, j) :

- If $X[i] = Y[j]$: match, go to $(i - 1, j - 1)$, prepend character.
- Else if $L[i - 1][j] > L[i][j - 1]$: go up to $(i - 1, j)$.
- Else if $L[i][j - 1] > L[i - 1][j]$: go left to $(i, j - 1)$.
- If both equal: **branch** (multiple LCS).

Traceback results in 3 branches (all length 5):

LCS	Sequence
1	$\langle A, A, G, U, T \rangle$
2	$\langle A, A, G, T, U \rangle$
3	$\langle A, U, A, T, U \rangle$

Step 3: Shortest Common Supersequence (SCS)

The length of the SCS of X and Y using an LCS is:

$$|SCS| = |X| + |Y| - |LCS| = 8 + 7 - 5 = 10$$

Construction using LCS 1 = $\langle A, A, G, U, T \rangle$:

We use two pointers to traverse X and Y . We add non-LCS characters from both strings before we add the matching LCS character.

LCS Char	Add from X	Add from Y	Current SCS
–	T (before A)	–	T
A	–	–	T, A
–	–	U (between A and A)	T, A, U
A	–	–	T, A, U, A
G	–	–	T, A, U, A, G
–	–	T (between G and U)	T, A, U, A, G, T
U	–	–	T, A, U, A, G, T, U
–	A (between U and T)	–	T, A, U, A, G, T, U, A
T	–	–	T, A, U, A, G, T, U, A, T
(End)	U (after T)	–	T, A, U, A, G, T, U, A, T, U

Final SCS:

$\langle T, A, U, A, G, T, U, A, T, U \rangle$

Verification:

- Length = 10 ✓
- Contains $X = \langle T, A, A, G, U, A, T, U \rangle$ as subsequence ✓
- Contains $Y = \langle A, U, A, G, T, U, T \rangle$ as subsequence ✓

Q18. Suppose you run a bus lending business, and each request has the format (start time, end time). Find all solutions for the maximum number of requests that can be satisfied by:

- Earliest start time
- Shortest interval
- Earliest end time

Use the following requests: $\{[1, 6), [4, 8), [6, 8), [9, 15), [6, 9), [11, 15), [2, 6)\}$.

(12 marks)

[CSE 203 | 2018–19]

Answer

The requests are:

$\{[1, 6), [4, 8), [6, 8), [9, 15), [6, 9), [11, 15), [2, 6)\}$

1. Earliest Start Time

Sort by start time (ascending):

Order	Interval	Decision
1	[1, 6)	Accept
2	[2, 6)	Reject (conflicts with [1, 6))
3	[4, 8)	Reject (conflicts with [1, 6))
4	[6, 8)	Accept
5	[6, 9)	Reject (conflicts with [6, 8))
6	[9, 15)	Accept
7	[11, 15)	Reject (conflicts with [9, 15))

Selected: {[1, 6), [6, 8), [9, 15)}

Count: 3

2. Shortest Interval

Sort by interval length (= end – start) ascending:

Order	Interval	Length	Decision
1	[6, 8)	2	Accept
2	[6, 9)	3	Reject (conflicts with [6, 8))
3	[2, 6)	4	Accept
4	[4, 8)	4	Reject (conflicts with [2, 6) and [6, 8))
5	[11, 15)	4	Accept
6	[1, 6)	5	Reject (conflicts with [2, 6))
7	[9, 15)	6	Reject (conflicts with [11, 15))

Selected: {[6, 8), [2, 6), [11, 15)}

Count: 3

3. Earliest End Time (Optimal Greedy Strategy)

Sort by end time (ascending):

Order	Interval	Decision
1	[1, 6)	Accept
2	[2, 6)	Reject (conflicts with [1, 6))
3	[4, 8)	Reject (conflicts with [1, 6))
4	[6, 8)	Accept
5	[6, 9)	Reject (conflicts with [6, 8))
6	[9, 15)	Accept
7	[11, 15)	Reject (conflicts with [9, 15))

Selected: $\{[1, 6), [6, 8), [9, 15)\}$

Count: 3 (Optimal)

Conclusion: In this specific instance, all three strategies yield **3 requests**, but only **Earliest End Time** is guaranteed to be optimal for all Interval Scheduling problems.

- Q19.** Let $A_1(11 \times 4)$, $A_2(4 \times 5)$, $A_3(5 \times 3)$, $A_4(3 \times 6)$, $A_5(6 \times 7)$, $A_6(7 \times 1)$ be six matrices. Compute $m[1, 6]$ given: $m[1, 3] = 35$, $m[2, 4] = 132$, $m[1, 5] = 95$, $m[2, 5] = 270$, $m[2, 6] = 95$, $m[4, 5] = 126$, $m[4, 6] = 60$. **(13 marks)** [CSE 203 | 2018–19]

Answer

Given matrices: $A_1(11 \times 4)$, $A_2(4 \times 5)$, $A_3(5 \times 3)$, $A_4(3 \times 6)$, $A_5(6 \times 7)$, $A_6(7 \times 1)$. So the dimension array is $p = [11, 4, 5, 3, 6, 7, 1]$, i.e. $p_0 = 11$, $p_1 = 4$, $p_2 = 5$, $p_3 = 3$, $p_4 = 6$, $p_5 = 7$, $p_6 = 1$.

Recurrence:

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} \cdot p_k \cdot p_j\}$$

We compute $m[1, 6]$ using all splits $k = 1, 2, 3, 4, 5$:

k	Expression	Value
1	$m[1, 1] + m[2, 6] + p_0 p_1 p_6 = 0 + 95 + 11 \cdot 4 \cdot 1$	$0 + 95 + 44 = 139$
2	$m[1, 2] + m[3, 6] + p_0 p_2 p_6$ (need $m[1, 2]$ and $m[3, 6]$)	see below
3	$m[1, 3] + m[4, 6] + p_0 p_3 p_6 = 35 + 60 + 11 \cdot 3 \cdot 1$	$35 + 60 + 33 = 128$
4	$m[1, 4] + m[5, 6] + p_0 p_4 p_6$ (need $m[1, 4]$ and $m[5, 6]$)	see below
5	$m[1, 5] + m[6, 6] + p_0 p_5 p_6 = 95 + 0 + 11 \cdot 7 \cdot 1$	$95 + 0 + 77 = 172$

Computing missing values for $k = 2$:

$$m[1, 2] = p_0 \cdot p_1 \cdot p_2 = 11 \cdot 4 \cdot 5 = 220$$

$m[3, 6]$: need to compute it using the recurrence.

Split $k = 3$: $m[3, 3] + m[4, 6] + p_2 p_3 p_6 = 0 + 60 + 5 \cdot 3 \cdot 1 = 75$.

Split $k = 4$: $m[3, 4] + m[5, 6] + p_2 p_4 p_6$; $m[3, 4] = p_2 p_3 p_4 = 5 \cdot 3 \cdot 6 = 90$, $m[5, 6] = p_4 p_5 p_6 = 6 \cdot 7 \cdot 1 = 42$. So $90 + 42 + 5 \cdot 6 \cdot 1 = 90 + 42 + 30 = 162$.

Split $k = 5$: $m[3, 5] + m[6, 6] + p_2 p_5 p_6$. We need $m[3, 5]$ separately:

$\min_k \{m[3, k] + m[k + 1, 5] + p_2 p_k p_5\}$ for $k = 3, 4$:

$k = 3$: $m[3, 3] + m[4, 5] + p_2 p_3 p_5 = 0 + 126 + 5 \cdot 3 \cdot 7 = 0 + 126 + 105 = 231$
(using given $m[4, 5] = 126$).

$k = 4$: $m[3, 4] + m[5, 5] + p_2 p_4 p_5 = 90 + 0 + 5 \cdot 6 \cdot 7 = 90 + 0 + 210 = 300$.

So $m[3, 5] = \min(231, 300) = 231$.

Then $k = 5$ for $m[3, 6]$: $231 + 0 + 5 \cdot 7 \cdot 1 = 231 + 35 = 266$.

$$\therefore m[3, 6] = \min(75, 162, 266) = 75$$

Now $k = 2$: $m[1, 2] + m[3, 6] + p_0 p_2 p_6 = 220 + 75 + 11 \cdot 5 \cdot 1 = 220 + 75 + 55 = 350$.

Computing missing values for $k = 4$:

$m[1, 4]$: use split $k = 1, 2, 3$.

$k = 1$: $m[1, 1] + m[2, 4] + p_0 p_1 p_4 = 0 + 132 + 11 \cdot 4 \cdot 6 = 132 + 264 = 396$.

$k = 2$: $m[1, 2] + m[3, 4] + p_0 p_2 p_4 = 220 + 90 + 11 \cdot 5 \cdot 6 = 220 + 90 + 330 = 640$.

$k = 3$: $m[1, 3] + m[4, 4] + p_0 p_3 p_4 = 35 + 0 + 11 \cdot 3 \cdot 6 = 35 + 0 + 198 = 233$.

$$\therefore m[1, 4] = \min(396, 640, 233) = 233$$

$$m[5, 6] = p_4 \cdot p_5 \cdot p_6 = 6 \cdot 7 \cdot 1 = 42$$

$k = 4$: $m[1, 4] + m[5, 6] + p_0 p_4 p_6 = 233 + 42 + 11 \cdot 6 \cdot 1 = 233 + 42 + 66 = 341$.

Summary of all splits for $m[1, 6]$:

k	$m[1, k] + m[k + 1, 6] + p_0 p_k p_6$
1	139
2	350
3	128
4	341
5	172
Min	128 (at $k = 3$)

$$\therefore m[1, 6] = \boxed{128}$$

Optimal split: $k = 3$, i.e. $(A_1 A_2 A_3)(A_4 A_5 A_6)$ with **128 multiplications**.

Q20. Briefly explain with examples: (i) Optimal substructure, and (ii) Overlapping sub-problems. Then recall the divide-and-conquer algorithm for merge sort and analyze its running time. **(8+7 marks)** [CSE 203 | 2017–18]

Answer

Optimal Substructure, Overlapping Subproblems, and Merge Sort Analysis

Part 1: Optimal Substructure

(4 marks)

Definition: A problem has **optimal substructure** if an optimal solution to the problem contains optimal solutions to its subproblems. This property is the key prerequisite for both **dynamic programming** and **greedy algorithms**.

Example 1 — Shortest Path:

In a weighted graph, the shortest path from u to v via intermediate vertex w satisfies:

$$\delta(u, v) = \delta(u, w) + \delta(w, v)$$

If the subpath $u \rightsquigarrow w$ were not shortest, we could replace it with a shorter path, contradicting the optimality of $u \rightsquigarrow v$. Hence the shortest path problem has optimal substructure.

Example 2 — 0/1 Knapsack:

Let $\text{OPT}(i, W)$ = maximum value using items $\{1, \dots, i\}$ with weight limit W :

$$\text{OPT}(i, W) = \max(\text{OPT}(i-1, W), v_i + \text{OPT}(i-1, W - w_i))$$

The optimal solution either excludes item i (best of $\{1, \dots, i-1\}$ with capacity W) or includes it (best of $\{1, \dots, i-1\}$ with remaining capacity $W - w_i$). Both subproblems must be solved optimally.

Counter-example (no optimal substructure):

The **longest simple path** problem does *not* have optimal substructure. The longest simple path from u to v via w need not consist of longest simple paths $u \rightarrow w$ and $w \rightarrow v$ (using the same vertices on both subpaths may create a cycle).

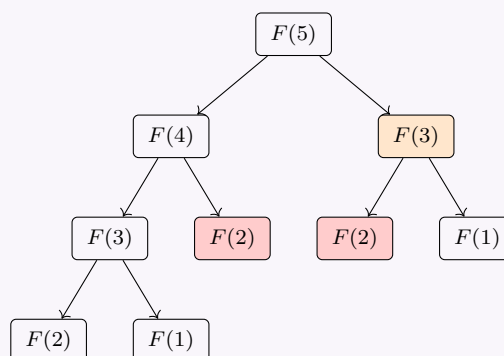
Part 2: Overlapping Subproblems

(4 marks)

Definition: A problem has **overlapping subproblems** if a recursive algorithm revisits the same subproblem multiple times. Dynamic programming exploits this by solving each subproblem **once** and storing the result (**memoisation** or **bottom-up tabulation**).

Example — Fibonacci Numbers:

Naive recursion for $F(n) = F(n-1) + F(n-2)$:



$F(2)$ is computed **3 times** and $F(3)$ is computed **2 times** in the recursion tree for $F(5)$. For $F(n)$, the naive recursion runs in $O(2^n)$ time. With DP (storing each $F(i)$ once), this drops to $O(n)$.

Example — Matrix Chain Multiplication:

Computing $m[1][4]$ requires $m[1][2]$ in multiple subproblems ($m[1][3]$ and the direct split of $m[1][4]$). The same subproblem appears in different branches of the recursion tree, making memoisation essential.

Contrast with Divide-and-Conquer:

Divide-and-conquer (e.g. merge sort) splits a problem into **independent** subproblems that do **not overlap**. There is no benefit to memoisation. Dynamic programming is used precisely when subproblems **do** overlap.

Part 3: Merge Sort — Algorithm and Running Time

(7 marks)

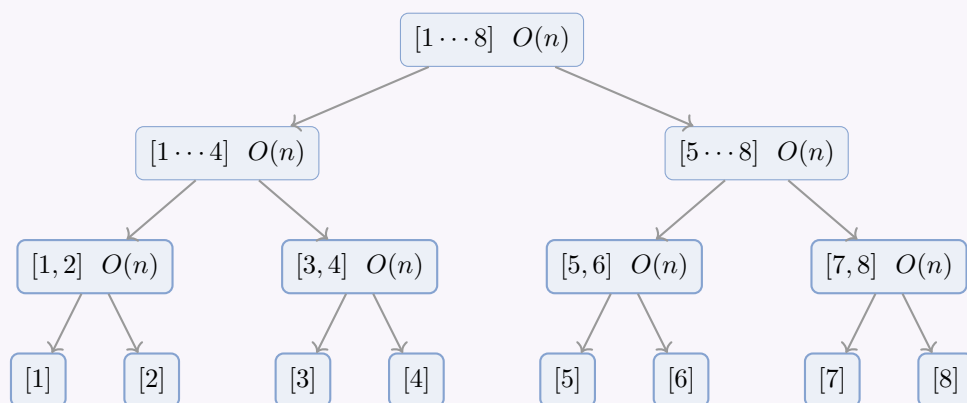
Algorithm:

Algorithm 116 MERGE(A, lo, mid, hi) — with explicit copy loops

```

1:  $n_1 \leftarrow mid - lo + 1$ ;  $n_2 \leftarrow hi - mid$ 
2:  $L[1..n_1] \leftarrow A[lo..mid]$ 
3:  $R[1..n_2] \leftarrow A[mid + 1..hi]$ 
4:  $i \leftarrow 1$ ;  $j \leftarrow 1$ ;  $k \leftarrow lo$ 
5: while  $i \leq n_1$  and  $j \leq n_2$  do
6:   if  $L[i] \leq R[j]$  then
7:      $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
8:   else
9:      $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
10:  end if
11:   $k \leftarrow k + 1$ 
12: end while
13: while  $i \leq n_1$  do
14:    $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ ;  $k \leftarrow k + 1$ 
15: end while
16: while  $j \leq n_2$  do
17:    $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ ;  $k \leftarrow k + 1$ 
18: end while
```

Illustration ($n = 8$):



There are $\log_2 n$ levels; at each level, the total work done by all MERGE calls is $O(n)$.

Running Time Analysis:

Recurrence:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + O(n) & n > 1 \end{cases}$$

The $2T(n/2)$ term comes from two recursive calls on halves of size $n/2$; the $O(n)$ term is the cost of MERGE.

Solving by the Master Theorem:

The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 2$, $b = 2$, $f(n) = \Theta(n)$. Compute $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.

Since $f(n) = \Theta(n^{\log_b a}) = \Theta(n)$, we are in **Case 2** of the Master Theorem:

$$T(n) = \Theta(n \log n)$$

Verification by Recursion Tree:

Level	Subproblems	Cost per level
0	1 subproblem of size n	$c \cdot n$
1	2 subproblems of size $n/2$	$2 \cdot c \cdot n/2 = cn$
2	4 subproblems of size $n/4$	$4 \cdot c \cdot n/4 = cn$
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	n subproblems of size 1	$n \cdot O(1) = O(n)$
Total levels: $\log_2 n + 1$		$\Theta(n \log n)$

Each of the $\log_2 n + 1$ levels contributes $\Theta(n)$ work, giving total cost $\Theta(n \log n)$.

Space Complexity:

Merge sort requires $O(n)$ auxiliary space for the temporary arrays in MERGE, plus $O(\log n)$ stack space for recursion. Total space: $O(n)$.

Summary:

Case	Time	Space
Best	$\Theta(n \log n)$	$O(n)$
Average	$\Theta(n \log n)$	$O(n)$
Worst	$\Theta(n \log n)$	$O(n)$

Merge sort is **stable**, **not in-place**, and guarantees $\Theta(n \log n)$ in all cases — unlike quicksort which has $O(n^2)$ worst case.

Q21. Consider the following instance of the 0-1 knapsack problem (capacity = 6):

Item	Weight	Value
1	2	7
2	1	8
3	2	12
4	2	6
5	2	10

Solve this instance using dynamic programming. Show the DP table and find the optimal set of items from the table. **(15 marks)** [CSE 203 | 2017–18]

Answer

0-1 Knapsack Problem: DP Solution (Capacity $W = 6$)

1. Recurrence Relation:

$$dp[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ dp[i-1][w] & w_i > w \\ \max(dp[i-1][w], v_i + dp[i-1][w - w_i]) & w_i \leq w \end{cases}$$

where $dp[i][w]$ = maximum value using items $\{1, \dots, i\}$ with capacity w .

2. DP Table ($dp[i][w]$):

(Highlighted cells show the traceback path for the optimal solution)

Item i	$w=0$	$w=1$	$w=2$	$w=3$	$w=4$	$w=5$	$w=6$
0 (none)	0	0	0	0	0	0	0
1 ($w_1=2, v_1=7$)	0	0	7	7	7	7	7
2 ($w_2=1, v_2=8$)	0	8	8	15	15	15	15
3 ($w_3=2, v_3=12$)	0	8	12	20	20	27	27
4 ($w_4=2, v_4=6$)	0	8	12	20	20	27	27
5 ($w_5=2, v_5=10$)	0	8	12	20	22	30	30

3. Cell-by-cell computation (Key Cells):

Row $i = 1$ ($w_1 = 2, v_1 = 7$):

- $w < 2$: $dp[1][w] = dp[0][w] = 0$
- $w \geq 2$: $dp[1][w] = \max(dp[0][w], 7 + dp[0][w - 2]) = \max(0, 7) = 7$

Row $i = 2$ ($w_2 = 1, v_2 = 8$):

- $dp[2][1] = \max(dp[1][1], 8 + dp[1][0]) = \max(0, 8) = 8$
- $dp[2][2] = \max(dp[1][2], 8 + dp[1][1]) = \max(7, 8) = 8$
- $dp[2][3] = \max(dp[1][3], 8 + dp[1][2]) = \max(7, 15) = 15$
- $dp[2][w] = 15$ for $w \geq 3$

Row $i = 3$ ($w_3 = 2, v_3 = 12$):

- $dp[3][1] = \max(8, \text{N/A}) = 8$
- $dp[3][2] = \max(dp[2][2], 12 + dp[2][0]) = \max(8, 12) = 12$
- $dp[3][3] = \max(dp[2][3], 12 + dp[2][1]) = \max(15, 20) = 20$
- $dp[3][4] = \max(dp[2][4], 12 + dp[2][2]) = \max(15, 20) = 20$
- $dp[3][5] = \max(dp[2][5], 12 + dp[2][3]) = \max(15, 27) = 27$
- $dp[3][6] = \max(dp[2][6], 12 + dp[2][4]) = \max(15, 27) = 27$

Row $i = 4$ ($w_4 = 2, v_4 = 6$):

- $dp[4][w] = \max(dp[3][w], 6 + dp[3][w - 2])$
- $dp[4][4] = \max(20, 6 + 12) = 20$
- $dp[4][5] = \max(27, 6 + 20) = 27$
- $dp[4][6] = \max(27, 6 + 20) = 27$
- (Item 4 never improves the solution)

Row $i = 5$ ($w_5 = 2, v_5 = 10$):

- $dp[5][2] = \max(dp[4][2], 10 + dp[4][0]) = \max(12, 10) = 12$
- $dp[5][3] = \max(dp[4][3], 10 + dp[4][1]) = \max(20, 18) = 20$
- $dp[5][4] = \max(dp[4][4], 10 + dp[4][2]) = \max(20, 22) = 22$
- $dp[5][5] = \max(dp[4][5], 10 + dp[4][3]) = \max(27, 30) = 30$
- $dp[5][6] = \max(dp[4][6], 10 + dp[4][4]) = \max(27, 10 + 20) = 30$

Optimal value = $dp[5][6] = 30$

4. Traceback to Find Optimal Items:

Step	Item i	$dp[i][w]$	$dp[i - 1][w]$	Decision	Remaining w
1	5	$dp[5][6] = 30$	$dp[4][6] = 27$	Include	$6 - 2 = 4$
2	4	$dp[4][4] = 20$	$dp[3][4] = 20$	Exclude	4
3	3	$dp[3][4] = 20$	$dp[2][4] = 15$	Include	$4 - 2 = 2$
4	2	$dp[2][2] = 8$	$dp[1][2] = 7$	Include	$2 - 1 = 1$
5	1	$dp[1][1] = 0$	$dp[0][1] = 0$	Exclude	1

5. Final Optimal Solution:

Selected Items	Weight	Value
Item 2	1	8
Item 3	2	12
Item 5	2	10
Total	$5 \leq 6$	30 ✓

Optimal subset = $\{2, 3, 5\}$ with total value **30** and total weight **5**.

Time and Space Complexity:

- **Time:** $\Theta(nW) = \Theta(5 \times 6) = \Theta(30)$.
- **Space:** $\Theta(nW)$ for the full table (required for traceback).

Q22. Given two strings, the longest common *substring* problem is to find the longest string that is a substring of both given strings. For example, for $S_1 = \text{buetcserocks}$ and $S_2 = \text{bdcserocks}$, the longest common substring is **cserocks**. Give a dynamic programming algorithm to find the length of the longest common substring. **(15 marks)** [CSE 203 | 2017–18]

Answer

Longest Common Substring: DP Algorithm

1. Key Distinction from LCS:

	LCS (Subsequence)	Longest Common Substring
Contiguous?	No	Yes
Example	LITHM	cserocks

A **substring** must occupy consecutive positions in both strings, unlike a subsequence which can skip characters.

2. DP Definition & Recurrence:

Let $dp[i][j]$ = length of the longest common **suffix** of $X[1..i]$ and $Y[1..j]$ (i.e. the longest common substring that ends at $X[i]$ and $Y[j]$).

$$dp[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ dp[i-1][j-1] + 1 & X[i] = Y[j] \\ 0 & X[i] \neq Y[j] \end{cases}$$

The key difference from LCS: when $X[i] \neq Y[j]$, we set $dp[i][j] = 0$ (not max of neighbours), because a mismatch **breaks** the current common suffix. The answer is $\max_{i,j} dp[i][j]$.

3. Algorithm:

Algorithm 117 LONGEST-COMMON-SUBSTRING(X, Y, m, n)

```

1: for  $i = 0$  to  $m$  do
2:    $dp[i][0] \leftarrow 0$ 
3: end for
4: for  $j = 0$  to  $n$  do
5:    $dp[0][j] \leftarrow 0$ 
6: end for
7:  $maxLen \leftarrow 0$ 
8:  $endIdx \leftarrow 0$ 
9: for  $i = 1$  to  $m$  do
10:  for  $j = 1$  to  $n$  do
11:    if  $X[i] = Y[j]$  then
12:       $dp[i][j] \leftarrow dp[i-1][j-1] + 1$ 
13:      if  $dp[i][j] > maxLen$  then
14:         $maxLen \leftarrow dp[i][j]$ 
15:         $endIdx \leftarrow i$ 
16:      end if
17:    else
18:       $dp[i][j] \leftarrow 0$ 
19:    end if
20:  end for
21: end for
22:  $result \leftarrow X[endIdx - maxLen + 1 .. endIdx]$ 
23: return  $maxLen, result$ 

```

4. Worked Example:

$X = \text{buetcserocks}$ ($m = 12$), $Y = \text{bdcserocks}$ ($n = 10$).

DP Table (showing only non-zero diagonals; 0s omitted for clarity):

$dp[i][j]$	b	d	c	s	e	r	o	c	k	s
	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$	$j=8$	$j=9$	$j=10$
b $i=1$	1	0	0	0	0	0	0	0	0	0
u $i=2$	0	0	0	0	0	0	0	0	0	0
e $i=3$	0	0	0	0	1	0	0	0	0	0
t $i=4$	0	0	0	0	0	0	0	0	0	0
c $i=5$	0	0	1	0	0	0	0	1	0	0
s $i=6$	0	0	0	2	0	0	0	0	0	1
e $i=7$	0	0	0	0	3	0	0	0	0	0
r $i=8$	0	0	0	0	0	4	0	0	0	0
o $i=9$	0	0	0	0	0	0	5	0	0	0
c $i=10$	0	0	1	0	0	0	0	6	0	0
k $i=11$	0	0	0	0	0	0	0	0	7	0
s $i=12$	0	0	0	1	0	0	0	0	0	8

The maximum value is $dp[12][10] = 8$, occurring at $i = 12, j = 10$.

Recovering the substring:

Ends at $X[12]$ with length 8 $\Rightarrow X[12 - 8 + 1 \dots 12] = X[5 \dots 12] = \text{cserocks}$.

5. Diagonal Intuition:

	...	c	s	e	r
...					
c	0	1	0	0	0
s	0	0	2	0	0
e	0	0	0	3	0

Matches extend diagonally

When characters match, values grow diagonally. A mismatch resets to 0, breaking the chain.

6. Time and Space Complexity:

- **Time:** $\Theta(mn)$ to fill the $m \times n$ table.
- **Space:** $\Theta(mn)$ for the full table, but can be optimized to $\Theta(\min(m, n))$ using just two rows.

Q23. Solve the following instance of the 0-1 knapsack problem using a branch-and-bound algorithm with a state-space tree (capacity = 10 kg):

Item	Weight (kg)	Profit (Taka)
1	3	18
2	5	35
3	4	44
4	7	36
5	2	18

(8 marks)

[CSE 207 | 2017–18]

Answer

Sorted by v/w : item3(4,44, $v/w = 11$), item2(5,35, $v/w = 7$), item5(2,18, $v/w = 9$), item1(3,18, $v/w = 6$), item4(7,36, $v/w \approx 5.1$).

Resorting correctly:

Orig	Sorted rank	w	v	v/w
3	1st	4	44	11.0
5	2nd	2	18	9.0
2	3rd	5	35	7.0
1	4th	3	18	6.0
4	5th	7	36	5.1

DP verification: Optimal = 80 (items 1, 3, 5: $w = 3 + 4 + 2 = 9 \leq 10$, $v =$

$18 + 44 + 18 = 80$).

B&B trace (key nodes):

Node	w	v	UB	Action
Root	0	0	≈ 97	Explore
Inc item3	4	44	≈ 97	Explore
Inc item5	6	62	≈ 97	Explore
Inc item2	$11 > 10$	—	—	Infeasible, prune
Exc item2:	6	62	≈ 74	Explore
Inc item1	9	80	80	Leaf , best=80
Back: Exc item5	4	44	≈ 82	Explore
Inc item2	9	79	≈ 80	Explore
Inc item1: $w = 12 > 10$	—	—	Prune	
Exc item1: $v = 79 < 80$	—	—	Continue...	
⋮ (all remaining UBs ≤ 80 , prune)				

Optimal = 80, items $\{1, 3, 5\}$ (original indices), $w = 9$, $v = 80$.

- Q24.** Recall the Weighted Interval Scheduling Problem. You are given n intervals with starting time s_i , finishing time f_i , and value v_i . A subset of intervals is compatible if no two overlap. Give a dynamic programming algorithm to find a compatible subset maximizing $\sum_{i \in S} v_i$. Your algorithm (including preprocessing) should run in $O(n \log n)$ time. **(15 marks)** [CSE 203 | 2016–17]

Answer

Weighted Interval Scheduling: $O(n \log n)$ DP Algorithm

Problem: Given n intervals, each with start s_i , finish f_i , and value v_i , find a compatible subset S (no two intervals overlap) maximizing $\sum_{i \in S} v_i$.

Step 1: Preprocessing — Sort by Finish Time

Sort all intervals in non-decreasing order of finish time. After sorting, assume intervals are indexed $1, 2, \dots, n$ such that $f_1 \leq f_2 \leq \dots \leq f_n$.

Time: $O(n \log n)$ using merge sort or heap sort.

Step 2: Compute $p(j)$ for Each Interval

Define $p(j)$ as the largest index $i < j$ such that interval i is **compatible** with interval j (i.e. $f_i \leq s_j$). If no such interval exists, $p(j) = 0$.

Algorithm 118 COMPUTE-P(intervals, n)

```

1: for  $j = 1$  to  $n$  do
2:    $lo \leftarrow 1$ ;  $hi \leftarrow j - 1$ ;  $p[j] \leftarrow 0$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
5:     if  $f[mid] \leq s[j]$  then
6:        $p[j] \leftarrow mid$ 
7:        $lo \leftarrow mid + 1$ 
8:     else
9:        $hi \leftarrow mid - 1$ 
10:    end if
11:  end while
12: end for
13: return  $p$ 

```

Since intervals are sorted by finish time, binary search over $f[1..j-1]$ finds $p(j)$ in $O(\log n)$. Computing all $p(j)$ takes $O(n \log n)$ total.

Step 3: DP Recurrence

Let $\text{OPT}(j)$ = maximum value of a compatible subset of intervals $\{1, 2, \dots, j\}$.

Key observation: In the optimal solution, interval j is either **included** or **excluded**:

- **Exclude j :** best solution uses only intervals from $\{1, \dots, j-1\} \Rightarrow \text{value} = \text{OPT}(j-1)$.
- **Include j :** cannot use any interval that overlaps with j ; the last compatible interval is $p(j) \Rightarrow \text{value} = v_j + \text{OPT}(p(j))$.

$$\text{OPT}(j) = \begin{cases} 0 & j = 0 \\ \max(\text{OPT}(j-1), v_j + \text{OPT}(p(j))) & j \geq 1 \end{cases}$$

Step 4: Full Algorithm**Algorithm 119** WEIGHTED-INTERVAL-SCHEDULING(s, f, v, n)

```

1: Sort intervals so that  $f[1] \leq f[2] \leq \dots \leq f[n]$ 
2:  $p \leftarrow \text{COMPUTE-P}(\text{intervals}, n)$ 
3:  $\text{OPT}[0] \leftarrow 0$ 
4: for  $j = 1$  to  $n$  do
5:    $\text{OPT}[j] \leftarrow \max(\text{OPT}[j-1], v[j] + \text{OPT}[p[j]])$ 
6: end for
7: return  $\text{OPT}[n], \text{FIND-SOLUTION}(n, \text{OPT}, p)$ 

```

Algorithm 120 FIND-SOLUTION(j , OPT , p)

```

1: if  $j = 0$  then
2:   return empty set
3: else if  $v[j] + OPT[p[j]] \geq OPT[j - 1]$  then
4:   return  $\{j\} \cup \text{FIND-SOLUTION}(p[j], OPT, p)$ 
5: else
6:   return FIND-SOLUTION( $j - 1$ ,  $OPT$ ,  $p$ )
7: end if

```

Worked Example:

j	s_j	f_j	v_j	$p(j)$
1	1	3	3	0
2	2	5	5	0
3	3	6	4	1
4	5	8	6	2
5	6	9	2	2
6	8	10	7	4

DP computation:

- $OPT(0) = 0$
- $OPT(1) = \max(0, 3 + 0) = 3$
- $OPT(2) = \max(3, 5 + 0) = 5$
- $OPT(3) = \max(5, 4 + 3) = 7$
- $OPT(4) = \max(7, 6 + 5) = 11$
- $OPT(5) = \max(11, 2 + 5) = 11$
- $OPT(6) = \max(11, 7 + 11) = 18$

Optimal value = 18. Traceback: include 6 ($p(6) = 4$), include 4 ($p(4) = 2$), include 2. Optimal subset = $\{2, 4, 6\}$.

Proof of Correctness:

Claim: $OPT(j)$ equals the maximum value compatible subset of $\{1, \dots, j\}$.

Proof by induction on j :

Base: $OPT(0) = 0$ is trivially correct.

Step: Assume $OPT(k)$ is correct for all $k < j$. In any optimal solution for $\{1, \dots, j\}$:

- (i) If $j \notin S^*$: then S^* is a compatible subset of $\{1, \dots, j - 1\}$, so its value $\leq OPT(j - 1)$.
- (ii) If $j \in S^*$: then no interval in S^* can have index in $\{p(j) + 1, \dots, j - 1\}$ (those overlap with j). So $S^* \setminus \{j\}$ is a compatible subset of $\{1, \dots, p(j)\}$, with value $\leq OPT(p(j))$. Total value $\leq v_j + OPT(p(j))$.

Both bounds are achieved by our recurrence, so $\text{OPT}(j) = \max(\text{OPT}(j-1), v_j + \text{OPT}(p(j)))$ is correct. ■

Time Complexity Analysis:

Step	Time
Sort by finish time	$O(n \log n)$
Compute all $p(j)$ (binary search per j)	$O(n \log n)$
Fill DP table (n iterations, $O(1)$ each)	$O(n)$
Traceback	$O(n)$
Total	$O(n \log n)$

$$T(n) = O(n \log n)$$

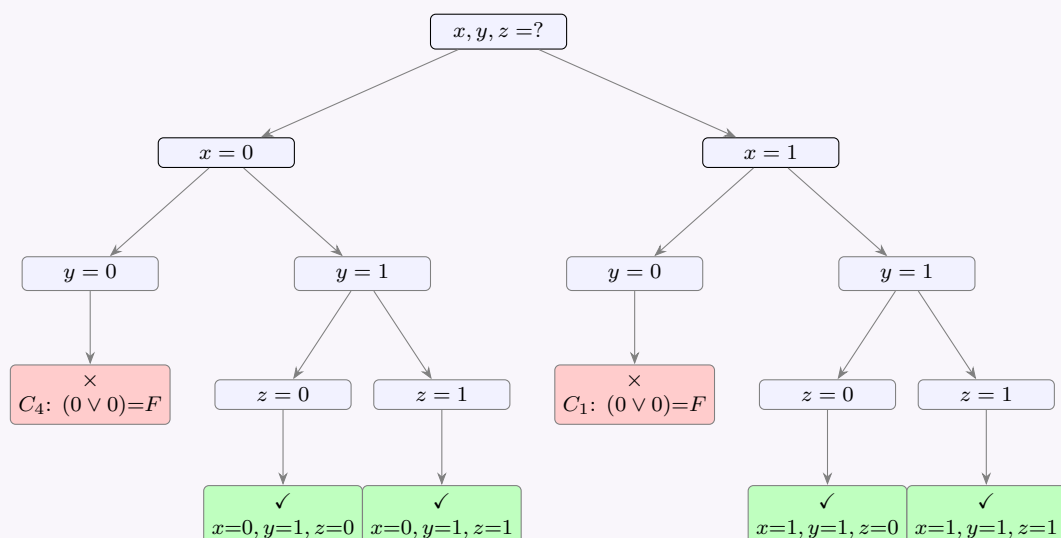
Q25. Consider the following Boolean formula: $(x' \vee y)(x \vee y' \vee z)(y \vee y')(x \vee y)(x \vee y \vee z')(x \vee y \vee z)$. Use a backtracking algorithm to decide whether it is satisfiable. Show the backtracking tree. **(15 marks)** [CSE 207 | 2016–17]

Answer

Simplification: Clause $(y \vee y')$ is a tautology (always true) — remove it. The remaining 5 clauses:

$$C_1 : (\neg x \vee y), \quad C_2 : (x \vee \neg y \vee z), \quad C_4 : (x \vee y), \quad C_5 : (x \vee y \vee \neg z), \quad C_6 : (x \vee y \vee z)$$

Backtracking Tree: Assign x first, then y , then z .



Pruning rules:

- $x = 0, y = 0$: $C_4 = (0 \vee 0) = \text{False} \Rightarrow$ prune immediately.
- $x = 1, y = 0$: $C_1 = (\neg 1 \vee 0) = (0 \vee 0) = \text{False} \Rightarrow$ prune.

Satisfying assignments (verified by substitution):

x	y	z	C_1	C_2	C_4	C_5	C_6
0	1	0	$1 \vee 1 = T$	$0 \vee 0 \vee 0 = F?$	T	T	check C_2
0	1	1	T	$0 \vee 0 \vee 1 = T$	T	T	$T \checkmark$
1	1	0	T	$1 \vee 0 \vee 0 = T$	T	T	$T \checkmark$
1	1	1	T	$1 \vee 0 \vee 1 = T$	T	T	$T \checkmark$

Check $(x = 0, y = 1, z = 0)$: $C_2 = (0 \vee 0 \vee 0) = \text{False} \Rightarrow$ not satisfying.

Formula is SATISFIABLE. The satisfying assignments are:

x	y	z
0	1	1
1	1	0
1	1	1

Q26. Solve the following 0-1 knapsack instance (capacity = 10) using the branch-and-bound algorithm. Show the branch-and-bound tree.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

(20 marks)

[CSE 207 | 2016–17]

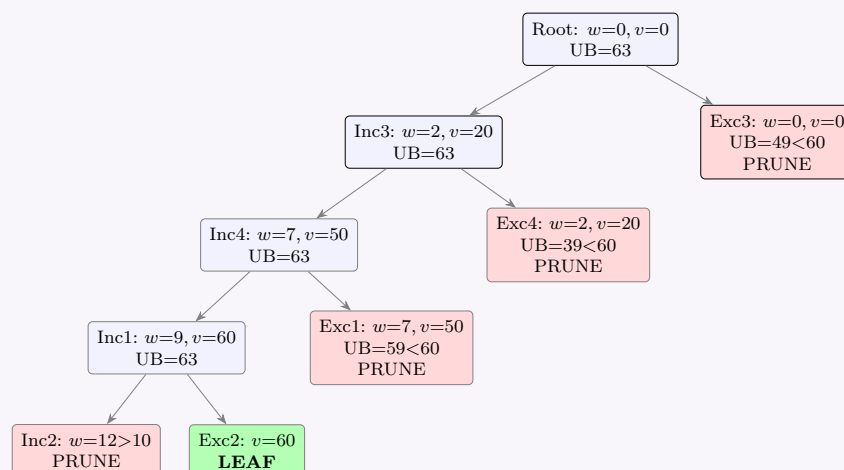
Answer

Items sorted by v/w (decreasing):

Orig	Sorted	w	v	v/w
3	1st	2	20	10.0
4	2nd	5	30	6.0
1	3rd	2	10	5.0
2	4th	3	9	3.0

Upper bound computation: Fill items greedily; use fraction of last item.

B&B Tree:



Node details:

Node	Decision	w	v	UB	Action
Root	—	0	0	63	Explore
Inc item3	item3 in	2	20	63	Explore
Inc item4	item4 in	7	50	63	Explore
Inc item1	item1 in	9	60	63	Explore
Inc item2	item2 in	12	—	—	Infeasible, prune
Exc item2	item2 out	9	60	60	LEAF $v = 60$
Exc item1	item1 out	7	50	59	$59 < 60$, prune
Exc item4	item4 out	2	20	39	$39 < 60$, prune
Exc item3	item3 out	0	0	49	$49 < 60$, prune

Optimal value = 60.

Items selected: item3 (orig: 3, $w = 2$, $v = 20$) + item4 (orig: 4, $w = 5$, $v = 30$) + item1 (orig: 1, $w = 2$, $v = 10$).

Original items selected: $\{1, 3, 4\}$, total weight = $2 + 2 + 5 = 9 \leq 10$, total value = $10 + 20 + 30 = 60$.

UB calculation at root: Items: item3($w = 2, v = 20$), item4($w = 5, v = 30$), item1($w = 2, v = 10$) fit in $W = 10$; remaining 1kg of item2 at $9/3 = 3$. UB = $20 + 30 + 10 + 3 = 63$.

Q27. What is the matrix chain multiplication problem? Prove that it has both the overlapping subproblems property and the optimal substructure property. (10 marks)
[CSE 207 | 2015–16]

Answer

MCM Problem: Given a chain of n matrices $A_1, A_2, \dots, A_n$ where A_i has dimension $p_{i-1} \times p_i$, find the parenthesization that minimizes the total number of scalar

multiplications.

Multiplying A_i ($p_{i-1} \times p_k$) with $A_{i+1..j}$ result ($p_k \times p_j$) costs $p_{i-1}p_kp_j$ multiplications.

Proof of Optimal Substructure:

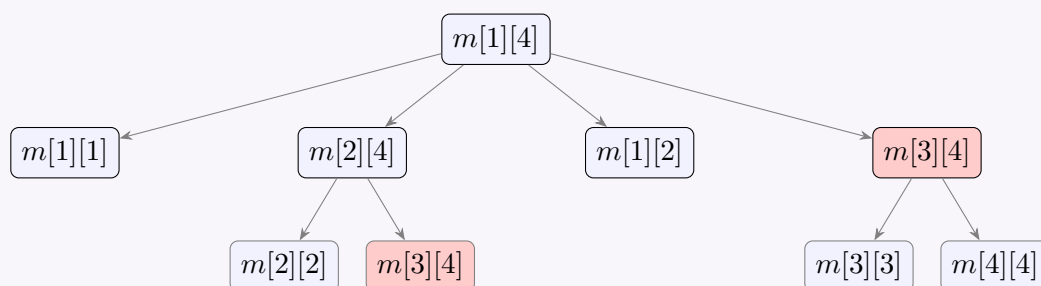
Let $m[i][j]$ = minimum cost to compute $A_i \cdots A_j$. The optimal split at some k gives:

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j$$

Cut-and-paste argument: If $m[i][k]$ were not optimal, a cheaper solution $m'[i][k] < m[i][k]$ would give total cost $m'[i][k] + m[k+1][j] + p_{i-1}p_kp_j < m[i][j]$, contradicting optimality. So $m[i][k]$ must be optimal. ■

Proof of Overlapping Subproblems:

The recursion tree for $m[1][4]$:



$m[3][4]$ appears **twice** (red nodes). In general, naive recursion solves $\Omega(2^n)$ subproblems, but there are only $\Theta(n^2)$ distinct ones. Storing each result once reduces time to $\Theta(n^3)$. ■

Number of parenthesizations $P(n)$ equals the $(n-1)$ -th Catalan number:

$$P(n) = C(n-1) = \frac{1}{n} \binom{2(n-1)}{n-1}$$

$$\Rightarrow P(1) = 1, P(2) = 1, P(3) = 2, P(4) = 5, P(5) = 14$$

$P(n) = \Omega(4^n/n^{3/2})$, which is exponential.

- Q28.** Write the steps to be followed for solving a problem using dynamic programming. Let $X = (x_1, \dots, x_m)$ and $Y = (y_1, \dots, y_n)$ be sequences, and let $Z = (z_1, \dots, z_k)$ be any LCS of X and Y . If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y . (4+6=10 marks) [CSE 207 | 2015–16]

Answer

Steps for Solving a Problem Using DP:

- 1. Characterize optimal substructure:** Show the optimal solution contains optimal sub-solutions.
- 2. Define the DP state:** Clearly define what $dp[i]$ or $dp[i][j]$ represents.
- 3. Write the recurrence relation:** Express each state in terms of smaller states,

including base cases.

4. **Verify overlapping subproblems:** Confirm subproblems recur.
5. **Choose implementation:** Top-down (memoisation) or bottom-up (tabulation).
6. **Fill the table** in the correct order (base cases first).
7. **Read the answer** from the appropriate table entry.
8. **Traceback** to reconstruct the actual solution (not just the optimal value).

Proof: if $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of X_{m-1} and Y .

Given:

- $X = (x_1, \dots, x_m)$, $Y = (y_1, \dots, y_n)$.
- $Z = (z_1, \dots, z_k)$ is an LCS of X and Y .
- $x_m \neq y_n$ and $z_k \neq x_m$.

Step 1: Since $z_k \neq x_m$, the last element z_k of Z is not x_m . Therefore Z is a common subsequence of X and Y that does not use x_m . Hence Z is also a common subsequence of X_{m-1} (i.e., X without x_m) and Y .

Step 2: We show Z is the *longest* such. Suppose for contradiction there exists Z' that is a common subsequence of X_{m-1} and Y with $|Z'| > |Z| = k$.

Since X_{m-1} is a subsequence of X , and Z' is a common subsequence of X_{m-1} and Y , Z' is also a common subsequence of X and Y .

But $|Z'| > k = |Z|$, which contradicts the assumption that Z is an LCS of X and Y . $\Rightarrow \Leftarrow$

Conclusion: Z is an LCS of X_{m-1} and Y . ■

Q29. Find all longest common subsequences of $X = \langle A, G, G, C, T, G, A, T \rangle$ and $Y = \langle G, G, G, C, A, T, A \rangle$ using dynamic programming. **(15 marks)** [CSE 207 | 2015–16]

Answer

Given Strings:

$X = \langle A, G, G, C, T, G, A, T \rangle$

$Y = \langle G, G, G, C, A, T, A \rangle$

1. **DP Table ($c[i][j]$):**

	ε	G	G	G	C	A	T	A
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	2	2	2	2	2	2
C	0	1	2	2	3	3	3	3
T	0	1	2	2	3	3	4	4
G	0	1	2	3	3	3	4	4
A	0	1	2	3	3	4	4	5
T	0	1	2	3	3	4	5	5

2. Direction Table ($b[i][j]$):

(Branching points where both UP and LEFT are equal are denoted by $\uparrow\leftarrow$. The red ones are our primary traceback branch points)

	ε	G	G	G	C	A	T	A
ε	—	—	—	—	—	—	—	—
A	—	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\swarrow$	$\leftarrow$	$\swarrow$
G	—	$\swarrow$	$\swarrow$	$\swarrow$	$\leftarrow$	$\uparrow\leftarrow$	$\uparrow\leftarrow$	$\uparrow\leftarrow$
G	—	$\swarrow$	$\swarrow$	$\swarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$
C	—	$\uparrow$	$\uparrow$	$\uparrow\leftarrow$	$\swarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$
T	—	$\uparrow$	$\uparrow$	$\uparrow\leftarrow$	$\uparrow$	$\uparrow\leftarrow$	$\swarrow$	$\leftarrow$
G	—	$\swarrow$	$\swarrow$	$\swarrow$	$\uparrow\leftarrow$	$\uparrow\leftarrow$	$\uparrow$	$\uparrow\leftarrow$
A	—	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow\leftarrow$	$\swarrow$	$\uparrow\leftarrow$	$\swarrow$
T	—	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow\leftarrow$	$\uparrow$	$\swarrow$	$\uparrow\leftarrow$

LCS length = 5.

3. Traceback to find ALL LCSes:

We start from the bottom-right cell (8, 7) where $c[8][7] = 5$. Since $X[8] \neq Y[7]$ and both up and left values are 5, we hit our first **branch** ($\uparrow\leftarrow$).

• Path 1 (Go Up $\uparrow$ from 8,7):

(8, 7) $\uparrow$ (7, 7) $\swarrow$ Record **A** $\rightarrow$ (6, 6) $\uparrow$ (5, 6) $\swarrow$ Record **T** $\rightarrow$ (4, 5) $\swarrow$ (4, 4) $\swarrow$ Record **C** $\rightarrow$ (3, 3) $\swarrow$ Record **G** $\rightarrow$ (2, 2) $\swarrow$ Record **G** $\rightarrow$ (1, 1).
Reading backwards: **LCS 1** = $\langle G, G, C, T, A \rangle$

• Path 2 (Go Left $\leftarrow$ from 8,7):

(8, 7) $\swarrow$ (8, 6) $\swarrow$ Record **T** $\rightarrow$ (7, 5) $\swarrow$ Record **A** $\rightarrow$ (6, 4).
At (6, 4), we find another **branch** ($\uparrow\leftarrow$):

– Path 2A (Go Up $\uparrow$ from 6,4):

(6, 4) $\uparrow$ (5, 4) $\uparrow$ (4, 4) $\swarrow$ Record **C** $\rightarrow$ (3, 3) $\swarrow$ Record **G** $\rightarrow$ (2, 2) $\swarrow$

Record $\mathbf{G} \rightarrow (1, 1)$.

Reading backwards: $\mathbf{LCS\ 2} = \langle G, G, C, A, T \rangle$

– **Path 2B (Go Left $\leftarrow$ from 6,4):**

$(6, 4) \xrightarrow{\leftarrow} (6, 3) \xrightarrow{\nearrow} \text{Record } \mathbf{G} \rightarrow (5, 2) \xrightarrow{\uparrow} (4, 2) \xrightarrow{\uparrow} (3, 2) \xrightarrow{\nearrow} \text{Record } \mathbf{G} \rightarrow (2, 1) \xrightarrow{\nearrow} \text{Record } \mathbf{G} \rightarrow (1, 0)$.

Reading backwards: $\mathbf{LCS\ 3} = \langle G, G, G, A, T \rangle$

Final Result:

All Longest Common Subsequences are:

1. $\langle G, G, C, T, A \rangle$
2. $\langle G, G, C, A, T \rangle$
3. $\langle G, G, G, A, T \rangle$

Verification:

- **LCS 1:** in X : $G(2), G(3), C(4), T(5), A(7)$ ✓; in Y : $G(1), G(2), C(4), T(6), A(7)$ ✓
- **LCS 2:** in X : $G(2), G(3), C(4), A(7), T(8)$ ✓; in Y : $G(1), G(2), C(4), A(5), T(6)$ ✓
- **LCS 3:** in X : $G(2), G(3), G(6), A(7), T(8)$ ✓; in Y : $G(1), G(2), G(3), A(5), T(6)$ ✓

Q30. Solve the following 0/1 knapsack instance (capacity = 15) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	140
2	6	132
3	4	140
4	3	90

Compare the backtracking and branch-and-bound techniques. **(3+12=15 marks)**
[CSE 207 | 2015–16]

Answer

Items sorted by v/w (decreasing):

Original	Sorted	w	v	v/w
3	1st	4	140	35.0
4	2nd	3	90	30.0
2	3rd	6	132	22.0
1	4th	7	140	20.0

Upper bound: Fill greedily by v/w ; take fraction of last item if needed.

Branch-and-Bound Trace (table form):

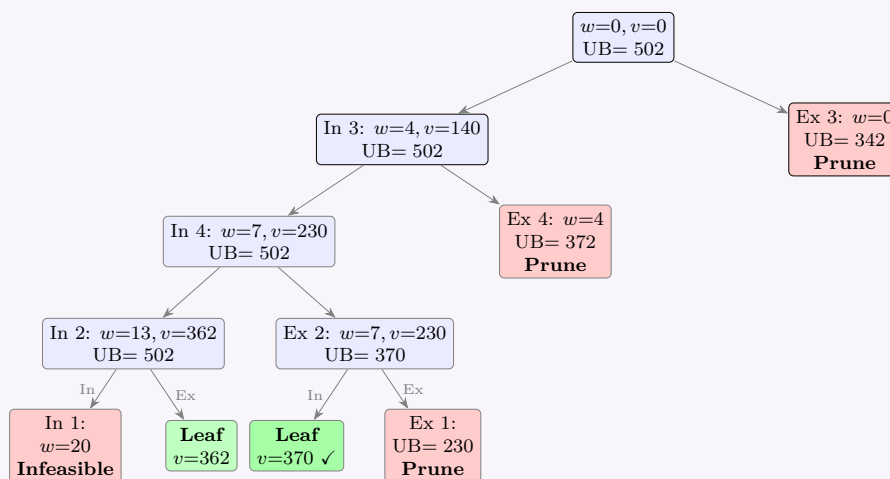
Node	Decision	w	v	UB	Best	Action
0	Root (nothing chosen)	0	0	502	0	Expand
1	Include item3 ($w=4, v=140$)	4	140	502	0	Expand
2	Include item4 ($w=3, v=90$)	7	230	502	0	Expand
3	Include item2 ($w=6, v=132$)	13	362	502	0	Expand
4	Include item1 ($w=7$)	20	–	–	0	Infeasible ($w>15$)
5	Exclude item1	13	362	362	362	Leaf , best=362
6	Exclude item2	7	230	370	362	Expand
7	Include item1 ($w=7, v=140$)	14	370	370	370	Leaf , best=370
8	Exclude item1	7	230	230	370	Prune (UB<best)
9	Exclude item4	4	140	372	370	Expand
10	Include item2 ($w=6, v=132$)	10	272	370	370	Expand
11	Include item1 ($w=7$)	17	–	–	370	Infeasible
12	Exclude item1	10	272	272	370	Prune
13	Exclude item2	4	140	350	370	Prune (UB<best)
14	Exclude item3	0	0	342	370	Prune (UB<best)

Optimal value = 370.

Items selected (sorted order): item3 ($w=4, v=140$) + item4 ($w=3, v=90$) + item1 ($w=7, v=140$)

$\Rightarrow$ original items {1, 3, 4}, total weight = $14 \leq 15$, total value = **370**.

State-Space Tree (compact):



Comparison: Backtracking vs. Branch-and-Bound:

Aspect	Backtracking	Branch-and-Bound
Goal	Find feasible / all solutions	Find <i>optimal</i> solution
Pruning	Infeasibility only	Infeasibility + bound $\leq$ best
Bound used	None	Upper/lower bound on sub-tree
Efficiency	May explore many branches	Prunes more aggressively
Application	SAT, colouring, puzzles	Optimization (TSP, Knapsack)
Completeness	Finds all solutions	Finds globally optimal

Q31. What are the key properties of an optimization problem that allow it to be solved using dynamic programming? Compare top-down memoized DP with bottom-up DP, and identify types of problems for which one outperforms the other. **(12 marks)**
[CSE 207 | 2014–15]

Answer

Key Properties Required for DP:

- Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems. This allows us to build the global optimal from sub-optimal solutions.
- Overlapping Subproblems:** The same subproblems are solved repeatedly in a naive recursive approach. DP caches each subproblem's result so it is solved only once.

Comparison: Top-down (Memoization) vs. Bottom-up (Tabulation):

Aspect	Top-down (Memo)	Bottom-up (Table)
Implementation	Recursive; cache with hash/array	Iterative; fill table in order
Subproblems solved	Only reachable ones	All subproblems
Call overhead	Recursion stack $O(d)$	None
Fill order	Automatic (on demand)	Must determine correct order
Space	Stack + memo table	Table only

When memoization outperforms bottom-up:

- Sparse subproblem graph:** if the problem has $O(n^2)$ possible subproblems but the optimal solution only needs $O(n)$ of them (e.g., longest path in a DAG from a specific source with limited reachability).

- **Early termination:** searching for any feasible solution may not require exploring all subproblems.

When bottom-up outperforms memoization:

- **All subproblems needed:** e.g., LCS, 0-1 Knapsack, Floyd-Warshall. Bottom-up avoids recursion overhead and is cache-friendlier.
- **Space optimization:** bottom-up allows rolling arrays (reducing $O(mn)$ to $O(\min(m, n))$) which is harder with memoization.
- **Large recursion depth:** avoids stack overflow for large n .

Q32. The balanced partition problem: given n integers each in $[0, N]$, divide them into two subsets minimizing the difference of their sums (e.g., for $S = \{1, 7, 3, 5, 9\}$, a balanced partition is $S_1 = \{1, 7, 5\}$ and $S_2 = \{3, 9\}$ with difference 1). Formulate a dynamic programming solution and determine its time complexity. **(12 marks)**
[CSE 207 | 2014–15]

Answer

Reduction to Subset Sum:

Let $T = \sum_{i=1}^n a_i$. We want to find a subset with sum s closest to $T/2$. The difference is $|T - 2s|$. So find the largest achievable $s \leq T/2$.

DP Definition:

$dp[i][s] = \text{True}$ if there exists a subset of $\{a_1, \dots, a_i\}$ with sum exactly s .

Recurrence:

$$dp[i][s] = dp[i-1][s] \vee (a_i \leq s \wedge dp[i-1][s-a_i])$$

Base case: $dp[0][0] = \text{True}$; $dp[0][s] = \text{False}$ for $s > 0$.

Answer: $\min_{0 \leq s \leq T/2, dp[n][s]=\text{True}} |T - 2s|$

Algorithm 121 BALANCED-PARTITION(A, n)

```

1:  $T \leftarrow \sum A$ 
2:  $dp[0][0] \leftarrow \text{true}$ ;  $dp[0][1..T] \leftarrow \text{false}$ 
3: for  $i = 1$  to  $n$  do
4:   for  $s = 0$  to  $T$  do
5:      $dp[i][s] \leftarrow dp[i-1][s]$ 
6:     if  $A[i] \leq s$  then
7:        $dp[i][s] \leftarrow dp[i][s] \text{ or } dp[i-1][s - A[i]]$ 
8:     end if
9:   end for
10: end for
11:  $best \leftarrow T$ 
12: for  $s = 0$  to  $\lfloor T/2 \rfloor$  do
13:   if  $dp[n][s]$  then
14:      $best \leftarrow \min(best, T - 2s)$ 
15:   end if
16: end for
17: return  $best$ 

```

Worked Example: $S = \{1, 7, 3, 5, 9\}$, $T = 25$.

$dp[5][12] = \text{True}$ (subset $\{3, 9\} = 12$). Difference = $25 - 2 \times 12 = 1$.

Partition: $S_1 = \{1, 7, 5\}$ (sum 13), $S_2 = \{3, 9\}$ (sum 12).

Alternatively: $S_1 = \{9, 3\}$, $S_2 = \{1, 7, 5\}$ — same difference.

Time Complexity:

- Table has $n \times T$ entries; each takes $O(1)$.
- $T \leq nN$ (since each element is at most N).
- **Total:** $O(n^2N)$.

This is pseudo-polynomial (polynomial in the value of input, not just its size).

Q33. Using dynamic programming, compute the minimum number of scalar multiplications for the matrix chain multiplication of $A_1, A_2, A_3, A_4, A_5, A_6$ with dimensions 15×5 , 5×50 , 50×20 , 20×10 , 10×35 , 35×25 . Also show the ordering of the matrices for the minimum. **(15 marks)** [CSE 207 | 2013–14]

Answer

Dimensions: $p = [15, 5, 50, 20, 10, 35, 25]$.

$m[i][j]$ table (1-indexed):

$i \backslash j$	1	2	3	4	5	6
1	0	3750	6500	6750	10375	14000
2	—	0	5000	6000	7750	12125
3	—	—	0	10000	27500	31250
4	—	—	—	0	7000	13750
5	—	—	—	—	0	8750
6	—	—	—	—	—	0

Minimum cost = 14 000 scalar multiplications.

$s[i][j]$ table (split point, 1-indexed):

$i \backslash j$	1	2	3	4	5	6
1	—	1	1	1	1	1
2	—	—	2	3	4	5
3	—	—	—	3	4	4
4	—	—	—	—	4	4
5	—	—	—	—	—	5

Optimal parenthesization: $(A_1((((A_2A_3)A_4)A_5)A_6))$

- Q34.** Compare dynamic programming and memoization techniques. “A dynamic programming algorithm usually outperforms the corresponding memoized algorithm” — explain. **(10 marks)** [CSE 207 | 2013–14]

Answer

Aspect	Dynamic Programming	Memoization
Approach	Bottom-up (iterative)	Top-down (recursive)
Subproblems	Solves <i>all</i> subproblems	Solves only <i>needed</i> subproblems
Order	Pre-determined fill order	Determined by recursion
Overhead	No recursion overhead	Function call overhead
Cache	Array/table directly	Hash map or table with lookup
Implementation	Usually simpler loops	Easier to derive from recurrence

Why DP usually outperforms memoization:

- No recursion overhead:** Each recursive call in memoization incurs function call setup, stack frame allocation, and return overhead. DP avoids this entirely with simple loops.
- Better cache/memory locality:** DP fills a table in a regular order (e.g., row by row), so memory access patterns are predictable and cache-friendly. Memoized recursion jumps around memory unpredictably.
- No hash-map overhead:** Memoization often uses a hash map for lookup ($O(1)$ average but with constant factor); DP uses direct array indexing ($O(1)$ with small constant).

4. **Stack depth:** Deep recursion in memoization can cause stack overflow for large n ; DP has no such issue.

However, memoization has an advantage when only a *fraction* of subproblems are actually needed — it avoids computing the rest entirely.

Q35. Write a memoized algorithm for solving the matrix chain multiplication problem. Analyze its time complexity. **(10 marks)** [CSE 207 | 2013–14]

Answer

Algorithm 122 MEMOMCM(p, n)

```
1: Initialize  $memo[1..n][1..n] \leftarrow -1$ 
2: return LOOKUPMCM( $p, memo, 1, n$ )
```

Algorithm 123 LOOKUPMCM($p, memo, i, j$)

```
1: if  $memo[i][j] \geq 0$  then
2:   return  $memo[i][j]$ 
3: end if
4: if  $i = j$  then
5:    $memo[i][j] \leftarrow 0$ 
6:   return 0
7: end if
8:  $memo[i][j] \leftarrow \infty$ 
9: for  $k = i$  to  $j - 1$  do
10:   $cost \leftarrow \text{LOOKUPMCM}(p, memo, i, k) + \text{LOOKUPMCM}(p, memo, k + 1, j)$ 
     $+ p[i - 1] \cdot p[k] \cdot p[j]$ 
11:  if  $cost < memo[i][j]$  then
12:     $memo[i][j] \leftarrow cost$ 
13:  end if
14: end for
15: return  $memo[i][j]$ 
```

Time Complexity Analysis:

- There are $\binom{n}{2} + n = O(n^2)$ distinct subproblems (i, j) .
- Each subproblem (i, j) is computed *at most once*; subsequent calls return in $O(1)$.
- Computing each subproblem requires a loop of $j - i$ iterations $\Rightarrow O(n)$ per subproblem.
- **Total:** $O(n^2) \times O(n) = O(n^3)$.

Space: $O(n^2)$ for the memo table, $O(n)$ recursion stack depth.

Q36. What is a state-space tree? Solve the following instance of the 0/1 knapsack problem (capacity = 10) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	42
2	4	40
3	3	12
4	5	25

(15 marks)

[CSE 207 | 2013–14]

Answer

State-Space Tree: A rooted tree where each node represents a *partial solution* (a decision about including/excluding items). The left child includes the item; the right child excludes it. Branch-and-bound prunes branches whose *upper bound* on value cannot exceed the current best solution.

Sort by value/weight ratio: Item 2 (10.0), Item 1 (6.0), Item 4 (5.0), Item 3 (4.0).

Reordered: Item 2 (w=4, v=40), Item 1 (w=7, v=42), Item 4 (w=5, v=25), Item 3 (w=3, v=12).

Upper bound at a node: Take all remaining items fractionally (greedy by ratio) up to capacity.

Branch-and-Bound Trace:

Node	Items included	Weight	Value	UB	Action
Root	none	0	0	76.4	expand
L1	{2}	4	40	76.4	expand
L1-L	{2,1}	11	82	—	infeasible (w>10)
L1-R	{2}	4	40	65.0	expand
L1-R-L	{2,4}	9	65	69.0	expand
L1-R-L-L	{2,4,3}	12	77	—	infeasible
L1-R-L-R	{2,4}	9	65	65.0	leaf , value=65 → best=65
L1-R-R	{2}	4	40	55.0	prune (UB < best)
R1	none	0	0	67.6	expand
R1-L	{1}	7	42	67.0	expand
R1-L-L	{1,4}	12	67	—	infeasible
R1-L-R	{1}	7	42	52.6	prune (UB < best=65)
R1-R	none	0	0	55.0	prune (UB < best=65)

Optimal solution: Items {2, 4} (Item 2 + Item 4), weight = 4 + 5 = 9 ≤ 10, value = 40 + 25 = **65**.

Q37. Prove that the longest common subsequence problem has both the Overlapping Subproblems property and the Optimal Substructure property. (10 marks) [CSE 207 | 2013–14]

Answer**Overlapping Subproblems.**

The recurrence $c[i][j] = c[i-1][j-1] + 1$ (or via $c[i-1][j], c[i][j-1]$) means $c[i][j]$ depends on $c[i-1][j]$, $c[i][j-1]$, and $c[i-1][j-1]$. These are themselves entries that are re-used by multiple parent cells (e.g., $c[i-1][j]$ is needed by both $c[i][j]$ and $c[i][j+1]$). A naive recursive implementation recomputes $c[k][\ell]$ exponentially many times. $\square$

Optimal Substructure.

Let $Z = \langle z_1, \dots, z_k \rangle$ be any LCS of $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$.

Case 1 ($x_m = y_n$): then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .

Case 2 ($x_m \neq y_n, z_k \neq x_m$): Z is an LCS of X_{m-1} and Y .

Case 3 ($x_m \neq y_n, z_k \neq y_n$): Z is an LCS of X and Y_{n-1} .

In each case the optimal Z is built from an optimal sub-LCS — proving optimal substructure. $\square$

- Q38.** Find all longest common subsequences of $X = \langle A, T, G, C, T, G, A, T \rangle$ and $Y = \langle T, G, G, C, A, T, A \rangle$ using dynamic programming. **(15 marks)** [CSE 207 | 2013–14]

Answer**Given Strings:**

$X = \langle A, T, G, C, T, G, A, T \rangle$

$Y = \langle T, G, G, C, A, T, A \rangle$

$c[i][j]$ **Table:**

	ε	T	G	G	C	A	T	A
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
T	0	1	1	1	1	1	2	2
G	0	1	2	2	2	2	2	2
C	0	1	2	2	3	3	3	3
T	0	1	2	2	3	3	4	4
G	0	1	2	3	3	3	4	4
A	0	1	2	3	3	4	4	5
T	0	1	2	3	3	4	5	5

$b[i][j]$ **Table** (Branching points where both UP and LEFT are equal are denoted by $\uparrow \leftarrow$. The red ones are our traceback branch points):

	ε	T	G	G	C	A	T	A
ε	—	—	—	—	—	—	—	—
A	—	↑	↑	↑	↑	↖	←	↖
T	—	↖	←	←	←	↑←	↖	←
G	—	↑	↖	↖	←	←	↑←	↑←
C	—	↑	↑	↑←	↖	←	←	←
T	—	↖	↑	↑←	↑	↑←	↖	←
G	—	↑	↖	↖	↑←	↑←	↑	↑←
A	—	↑	↑	↑	↑←	↖	↑←	↖
T	—	↖	↑	↑	↑←	↑	↖	↑←

Traceback to find ALL LCSes:

We start from the bottom-right cell $(8, 7)$ where $c[8][7] = 5$. Since $X[8] \neq Y[7]$ and both up and left values are 5, we hit our first **branch** (↑←).

- **Path 1 (Go Up ↑ from 8,7):** $(8, 7) \xrightarrow{\uparrow} (7, 7) \xrightarrow{\nwarrow} \text{Record A} \rightarrow (6, 6) \xrightarrow{\uparrow} (5, 6) \xrightarrow{\nwarrow} \text{Record T} \rightarrow (4, 5) \xrightarrow{\swarrow} (4, 4) \xrightarrow{\nwarrow} \text{Record C} \rightarrow (3, 3) \xrightarrow{\nwarrow} \text{Record G} \rightarrow (2, 2) \xrightarrow{\swarrow} (2, 1) \xrightarrow{\nwarrow} \text{Record T} \rightarrow (1, 0)$.

Reading backwards: **LCS 1 = TGCTA**

- **Path 2 (Go Left ← from 8,7):** $(8, 7) \xrightarrow{\swarrow} (8, 6) \xrightarrow{\nwarrow} \text{Record T} \rightarrow (7, 5) \xrightarrow{\nwarrow} \text{Record A} \rightarrow (6, 4)$.

At $(6, 4)$, we find another **branch** (↑←):

- **Path 2A (Go Up ↑ from 6,4):** $(6, 4) \xrightarrow{\uparrow} (5, 4) \xrightarrow{\uparrow} (4, 4) \xrightarrow{\nwarrow} \text{Record C} \rightarrow (3, 3) \xrightarrow{\nwarrow} \text{Record G} \rightarrow (2, 2) \xrightarrow{\swarrow} (2, 1) \xrightarrow{\nwarrow} \text{Record T} \rightarrow (1, 0)$.

Reading backwards: **LCS 2 = TGCAT**

- **Path 2B (Go Left ← from 6,4):** $(6, 4) \xrightarrow{\swarrow} (6, 3) \xrightarrow{\nwarrow} \text{Record G} \rightarrow (5, 2) \xrightarrow{\uparrow} (4, 2) \xrightarrow{\uparrow} (3, 2) \xrightarrow{\nwarrow} \text{Record G} \rightarrow (2, 1) \xrightarrow{\nwarrow} \text{Record T} \rightarrow (1, 0)$.

Reading backwards: **LCS 3 = TGGAT**

Final Result:

Max LCS Length = 5.

All Longest Common Subsequences = {TGCTA, TGCAT, TGGAT}.

Note / Additional Query Part:

For $X = \langle A, G, G, C, T, G, A, T \rangle$ and $Y = \langle G, G, G, C, A, T, A \rangle$:

LCS length = 5. One valid LCS = $\langle G, G, C, T, A \rangle$.

Q39. In a previous life, you worked as a cashier in the ancient city of Egypt, spending the better part of your day giving change to your customers. Because paper was a very rare and valuable resource at the time, cashiers were required by law to use the

fewest notes possible whenever they gave change. The currency of Egypt at that time, called Dream Dollars, was available in the following denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365. (20 marks)

- (a) A greedy change algorithm repeatedly takes the largest note that does not exceed the target amount. For example, to make \$122 using the greedy algorithm, we first take a \$91 note, then a \$28 note, and finally three \$1 notes. Give an example where this greedy algorithm uses more Dream Dollar notes than the minimum possible.
- (b) Describe a dynamic programming algorithm that computes, given an integer k , the minimum number of notes needed to make k Dream Dollars. Write down the pseudocodes for both the memoized approach and bottom-up iterative approach. Analyze the running-times of both approaches.

[CSE 207 | 2012–13]

Answer

Denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365.

(a) Greedy counterexample.

For $k = 15$: greedy takes $\$13 + \$1 + \$1 = 3$ notes. Optimal: $\$7 + \$7 + \$1 = 3$ notes — same here.

For $k = 14$: greedy takes $\$13 + \$1 = 2$ notes. Optimal: $\$7 + \$7 = 2$ notes — same.

For $k = 11$: greedy takes $\$7 + \$4 = 2$ notes. But note: $\$4 + \$4 + \$1 + \$1 + \$1 = 5$ notes — greedy is better here.

For $k = 8$: greedy takes $\$7 + \$1 = 2$ notes. Optimal: $\$4 + \$4 = 2$ notes — same.

Remark. The particular denominations $\{1, 4, 7, 13, 28, 52, 91, 365\}$ are Fibonacci-like and the greedy appears to give optimal results for small amounts. A rigorous counterexample requires checking larger values. For general denominations, greedy fails: with $\{1, 3, 4\}$ and $k = 6$, greedy gives $4 + 1 + 1 = 3$ notes but optimal is $3 + 3 = 2$.

(b) DP algorithm:

```

1  // Bottom-up DP
2  vector<int> coinChangeDP(int k, vector<int>& coins) {
3      vector<int> dp(k+1, INT_MAX), from(k+1, -1);
4      dp[0] = 0;
5      for (int i = 1; i <= k; i++)
6          for (int c : coins)
7              if (c <= i && dp[i-c] != INT_MAX && dp[i-c]+1 < dp[i])
8                  { dp[i] = dp[i-c]+1; from[i] = c; }
9      return dp; // dp[k] = min notes for k
10 }
11
12 // Memoized (top-down)
13 unordered_map<int, int> memo;
14 int coinChangeMemo(int k, vector<int>& coins) {
15     if (k == 0) return 0;

```

```

16     if (memo.count(k)) return memo[k];
17     int res = INT_MAX;
18     for (int c : coins)
19         if (c <= k && coinChangeMemo(k-c, coins) != INT_MAX)
20             res = min(res, coinChangeMemo(k-c, coins)+1);
21     return memo[k] = res;
22 }

```

Complexity Analysis. Both approaches: $O(k \cdot |\text{coins}|)$ time, $O(k)$ space.

- Q40.** You have n magic boxes kept in a row, numbered 1 to n , each containing some dollars. You can open box i only if you did not open box $i - 1$ or box $i - 2$. Given the amount $D[1 \dots n]$ in each box, find the maximum dollars obtainable. Describe a dynamic programming solution and analyze its running time. **(12 marks)** [CSE 207 | 2012–13]

Answer

DP recurrence: define $dp[i] = \text{max dollars from boxes } i \text{ to } n$.

$$dp[i] = \max(D[i] + dp[i + 2], dp[i + 1]), \quad dp[n + 1] = dp[n + 2] = 0.$$

Compute right to left; answer is $dp[1]$.

Example: $D = [1, 9, 3, 7, 5, 2, 8, 4] \rightarrow$ optimal picks: boxes 2(9)+4(7)+7(8)=24. (Indices 1-based; skip 3,5,6,8.)

Wait — re-check: boxes 2,4,7 give $9 + 7 + 8 = 24$ but we must verify no two are adjacent: $|2 - 4| = 2 \geq 2 \checkmark$, $|4 - 7| = 3 \geq 2 \checkmark$. **Maximum = 24.**

Complexity Analysis. $O(n)$ time, $O(1)$ space (only keep last two dp values).

- Q41.** Suppose we are given a sequence of integers $A[1..n]$ and we want to find the longest subsequence whose elements are in decreasing order. More concretely, we want to find the longest sequence of indices $1 < i_1 < i_2 < \dots < i_k < n$ such that $A[i_j] > A[i_{j+1}]$ for all j . Describe the pseudocode of a backtracking algorithm that solves this problem in $O(2^n)$ time. **(15 marks)** [CSE 207 | 2012–13]

Answer

Backtracking Algorithm:

For each element, we either *include* it (if it is smaller than the last included element) or *exclude* it. We explore all 2^n subsets.

Algorithm 124 LDS_BACKTRACK($A, n, idx, lastVal, current$)

```

1: if  $idx > n$  then
2:   if  $|current| > best$  then
3:      $best \leftarrow |current|$ 
4:      $bestSeq \leftarrow \text{copy}(current)$ 
5:   end if
6:   return
7: end if
8: LDS_BACKTRACK( $A, n, idx + 1, lastVal, current$ )      ▷ exclude  $A[idx]$ 
9: if  $A[idx] < lastVal$  then                          ▷ include  $A[idx]$ 
10:   $current.APPEND(A[idx])$ 
11:  LDS_BACKTRACK( $A, n, idx + 1, A[idx], current$ )
12:   $current.POP()$ 
13: end if
    
```

Complexity: At each index we make 2 choices (include/exclude), giving a recursion tree of depth n with 2^n leaves $\Rightarrow O(2^n)$ time.

Note: This problem can be solved in $O(n \log n)$ using the LIS (Longest Increasing Subsequence) DP on the negated array, but the question asks for the backtracking approach.

Q42. Compare the properties of problems that can be solved with the greedy method, divide-and-conquer method, and dynamic programming method. **(3 marks)** [CSE 207 | 2011–12]

Answer

Property	Greedy	Divide & Conquer	Dynamic Programming
Optimal substructure	Required ✓	Not required	Required ✓
Overlapping subproblems	Not required	Not required (independent)	Required ✓
Greedy-choice property	Required ✓	No	No
Subproblem independence	Yes	Yes	No (shared)
Typical complexity	$O(n \log n)$	$T(n) = aT(n/b) + f(n)$	Polynomial (table-size)
Examples	Huffman, MST, Activity Selection	Mergesort, FFT, Closest Pair	LCS, Knapsack, MCM

Q43. Write and prove the optimal-substructure property of the matrix chain multiplication problem. Calculate the number of parenthesizations of a sequence of n matrices. **(5+5=10 marks)** [CSE 207 | 2011–12]

Answer

Theorem (Optimal Substructure of MCM): Let $m[i][j]$ be the minimum cost to compute $A_i \cdots A_j$. In an optimal parenthesization, the last split is at some position k ($i \leq k < j$):

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1}p_kp_j$$

and both $m[i][k]$ and $m[k+1][j]$ must be optimal.

Proof (cut-and-paste):

Suppose the optimal solution for $A_i \cdots A_j$ splits at k , but the sub-solution for $A_i \cdots A_k$ uses cost c' with $c' < m[i][k]$. Then total cost $c' + m[k+1][j] + p_{i-1}p_kp_j < m[i][j]$, contradicting the optimality of $m[i][j]$. Hence $m[i][k]$ must be optimal, and by identical argument so must $m[k+1][j]$. ■

Number of Parenthesizations:

Let $P(n)$ = number of ways to parenthesize n matrices. We have:

$$P(1) = 1, \quad P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \quad \text{for } n \geq 2$$

This is the recurrence for the **Catalan numbers**: $P(n) = C(n-1)$ where $C(m) = \frac{1}{m+1} \binom{2m}{m}$.

n	1	2	3	4	5	6
$P(n)$	1	1	2	5	14	42

Asymptotically, $P(n) = \Omega(4^{n-1}/n^{3/2})$, which is **exponential** in n — justifying why brute force is infeasible and DP is essential.

- Q44.** Find an optimal solution to the 0/1 knapsack instance of $n = 4$, $W = 6$, $(v_1, v_2, v_3, v_4) = (50, 30, 12, 45)$, $(w_1, w_2, w_3, w_4) = (2, 3, 1, 3)$. **(15 marks)** [CSE 207 | 2011–12]

Answer

DP Definition: $dp[i][w]$ = maximum value using items $1 \dots i$ with capacity w .

Recurrence:

$$dp[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ dp[i-1][w] & w_i > w \\ \max(dp[i-1][w], v_i + dp[i-1][w - w_i]) & \text{otherwise} \end{cases}$$

Items:

Item	w_i	v_i	v_i/w_i
1	2	50	25.0
2	3	30	10.0
3	1	12	12.0
4	3	45	15.0

DP Table $dp[i][w]$:

$i \backslash w$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1 ($w = 2, v = 50$)	0	0	50	50	50	50	50
2 ($w = 3, v = 30$)	0	0	50	50	50	80	80
3 ($w = 1, v = 12$)	0	12	50	62	62	80	92
4 ($w = 3, v = 45$)	0	12	50	62	62	95	107

Traceback from $dp[4][6] = 107$:

- $dp[4][6] = 107 \neq dp[3][6] = 92 \Rightarrow$ item 4 included. Remaining capacity = $6 - 3 = 3$, go to $dp[3][3]$.
- $dp[3][3] = 62 \neq dp[2][3] = 50 \Rightarrow$ item 3 included. Remaining capacity = $3 - 1 = 2$, go to $dp[2][2]$.
- $dp[2][2] = 50 = dp[1][2] = 50 \Rightarrow$ item 2 *not* included. Go to $dp[1][2]$.
- $dp[1][2] = 50 \neq dp[0][2] = 0 \Rightarrow$ item 1 included. Remaining capacity = $2 - 2 = 0$.

Selected items: $\{1, 3, 4\}$

Item	w_i	v_i
1	2	50
3	1	12
4	3	45
Total	6	107

Optimal value = **107**, total weight = $6 = W$. ✓

Q45. Let $X = (x_1, x_2, \dots, x_m)$ and $Y = (y_1, y_2, \dots, y_n)$ be sequences, and let $Z = (z_1, z_2, \dots, z_k)$ be any LCS of X and Y . If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y . **(2 marks)** [CSE 207 | 2011–12]

Q46. Find an LCS of $X = \langle A, T, C, T, G, A, T \rangle$ and $Y = \langle T, G, C, A, T, A \rangle$ by showing the c and b tables. **(10 marks)** [CSE 207 | 2011–12]

Answer $c[i][j]$ table:

	ε	T	G	C	A	T	A
ε	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
T	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
T	0	1	1	2	2	3	3
G	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
T	0	1	2	2	3	4	4

 $b[i][j]$ table:

	ε	T	G	C	A	T	A
ε	—	—	—	—	—	—	—
A	—	↑	↑	↑	↖	←	↖
T	—	↖	←	←	↑	↖	←
C	—	↑	↑	↖	←	↑	↑
T	—	↖	↑	↑	↑	↖	←
G	—	↑	↖	↑	↑	↑	↑
A	—	↑	↑	↑	↖	↑	↖
T	—	↖	↑	↑	↑	↖	↑

LCS length = $c[7][6] = 4$.**Traceback** (follow b from $(7, 6)$):

Correct traceback: $(7, 6) \xrightarrow{\uparrow} (6, 6) \xrightarrow{\nwarrow}$: $X[6] = A$, $Y[6] = A$ ✓, record **A**, go to $(5, 5)$. $b[5][5] = \uparrow \rightarrow (4, 5)$. $b[4][5] = \nwarrow$: $X[4] = T$, $Y[5] = T$ ✓, record **T**, go to $(3, 4)$. $b[3][4] = \leftarrow \rightarrow (3, 3)$. $b[3][3] = \nwarrow$: $X[3] = C$, $Y[3] = C$ ✓, record **C**, go to $(2, 2)$. $b[2][2] = \leftarrow \rightarrow (2, 1)$. $b[2][1] = \nwarrow$: $X[2] = T$, $Y[1] = T$ ✓, record **T**, go to $(1, 0)$. Done.

Reading in order: **LCS** = $\langle T, C, T, A \rangle$.**Verification:** T(2), C(3), T(4), A(6) in X ✓; T(1), C(3), T(5), A(6) in Y ✓.

- Q47.** Give an example of a problem that has the optimal substructure property. Give another example of a problem that does not have optimal substructure. Justify your answers. (10 marks) [CSE 207 | 2009–10]

Answer

Problem WITH Optimal Substructure: Shortest Path (Dijkstra/Bellman-Ford)

Claim: Any shortest path $p : u \rightsquigarrow v$ passing through intermediate vertex w satisfies $\delta(u, v) = \delta(u, w) + \delta(w, v)$.

Proof: If the subpath $u \rightsquigarrow w$ were not a shortest path, we could replace it with a shorter path, reducing the total length below $\delta(u, v)$, contradicting optimality. ■

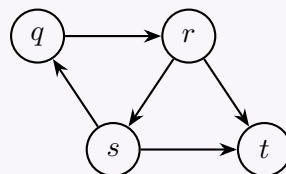
This means we can build the shortest path from optimal subpaths $\Rightarrow$ optimal substructure holds. This enables Dijkstra's $O((V + E) \log V)$ and Floyd-Warshall's $O(V^3)$ algorithms.

Problem WITHOUT Optimal Substructure: Longest Simple Path

Claim: The longest simple path problem does *not* have optimal substructure.

Counterexample:

Consider the graph:



Longest simple path from q to t : $q \rightarrow r \rightarrow s \rightarrow t$ (length 3) or $q \rightarrow s \rightarrow r \rightarrow t$ (length 3).

Consider subpaths: the longest simple path $q \rightarrow r$ is $q \rightarrow s \rightarrow r$ (length 2); the longest simple path $r \rightarrow t$ is $r \rightarrow s \rightarrow t$ (length 2). But combining $q \rightarrow s \rightarrow r$ and $r \rightarrow s \rightarrow t$ reuses vertex s , creating a non-simple path!

The longest simple paths of the subproblems cannot be combined because they share vertices. Hence the problem lacks optimal substructure and is NP-hard.

Q48. Show the execution of the Dynamic Programming LCS algorithm for the following two strings: $X = \langle A, B, C, B \rangle$ and $Y = \langle B, D, A, C \rangle$. **(15 marks)** [CSE 207 | 2009–10]

Answer

Recurrence:

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(c[i-1][j], c[i][j-1]) & X[i] \neq Y[j] \end{cases}$$

$c[i][j]$ table:

	ε	B	D	A	C
ε	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	1
C	0	1	1	1	2
B	0	1	1	1	2

$b[i][j]$ **table** ($\nwarrow$ =diagonal, $\uparrow$ =up, $\leftarrow$ =left):

	ε	B	D	A	C
ε	—	—	—	—	—
A	—	$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$
B	—	$\nwarrow$	$\leftarrow$	$\uparrow$	$\uparrow$
C	—	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$
B	—	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$

LCS length = $c[4][4] = 2$.

Traceback (follow b starting from $c[4][4]$):

$(4, 4) \xrightarrow{\uparrow} (3, 4) \xrightarrow{\nwarrow} (2, 3) \xrightarrow{\uparrow} (1, 3) \xrightarrow{\nwarrow} (0, 2)$ (Done).

Detailed Path:

- $b[4][4] = \uparrow \rightarrow$ go to $(3, 4)$.
- $b[3][4] = \nwarrow \rightarrow$ match $X[3] = C, Y[4] = C$ ✓ (record **C**), go to $(2, 3)$.
- $b[2][3] = \uparrow \rightarrow$ go to $(1, 3)$.
- $b[1][3] = \nwarrow \rightarrow$ match $X[1] = A, Y[3] = A$ ✓ (record **A**), go to $(0, 2)$.

Reading in reverse order: **LCS** = $\langle A, C \rangle$.

Verification:

$A(1), C(3)$ in $X = \langle A, B, C, B \rangle$ ✓;

$A(3), C(4)$ in $Y = \langle B, D, A, C \rangle$ ✓.

Q49. Show all the different ways to multiply the matrix chain $A_1A_2A_3A_4A_5$. (5 marks)
[CSE 207 | 2009–10]

Answer

The number of distinct parenthesizations of n matrices is the $(n - 1)$ -th Catalan number $C(n - 1)$. For $n = 5$: $C(4) = \frac{1}{5} \binom{8}{4} = 14$.

All 14 parenthesizations of $A_1A_2A_3A_4A_5$:

#	Parenthesization
1	$(A_1(A_2(A_3(A_4A_5))))$
2	$(A_1(A_2((A_3A_4)A_5)))$
3	$(A_1((A_2A_3)(A_4A_5)))$
4	$(A_1((A_2(A_3A_4))A_5))$
5	$(A_1(((A_2A_3)A_4)A_5))$
6	$((A_1A_2)(A_3(A_4A_5)))$
7	$((A_1A_2)((A_3A_4)A_5))$
8	$((A_1(A_2A_3))(A_4A_5))$
9	$((A_1(A_2A_3))A_4)A_5$
10	$((A_1A_2)A_3)(A_4A_5)$
11	$((A_1(A_2(A_3A_4)))A_5)$
12	$((A_1A_2)(A_3A_4))A_5$
13	$((A_1A_2)A_3A_4)A_5$
14	$((A_1A_2)A_3)A_4A_5$

Recurrence for Catalan numbers: $C(0) = C(1) = 1$; $C(n) = \sum_{k=0}^{n-1} C(k) \cdot C(n-1-k)$

n (matrices)	1	2	3	4	5	6
Parenthesizations	1	1	2	5	14	42

Q50. Convert the following recursive procedure for computing the n -th Fibonacci number into an equivalent procedure that uses a dynamic multi-graph representing the computation. From this graph, find $T_1(5)$, $T_\infty(5)$, and the parallelism.

```

F(n)
  if n <= 1 then return n
  else return F(n-1) + F(n-2)

```

(17 marks)

[CSE 207 | 2009–10]

Answer

Recursive procedure:

Algorithm 125 $F(n)$

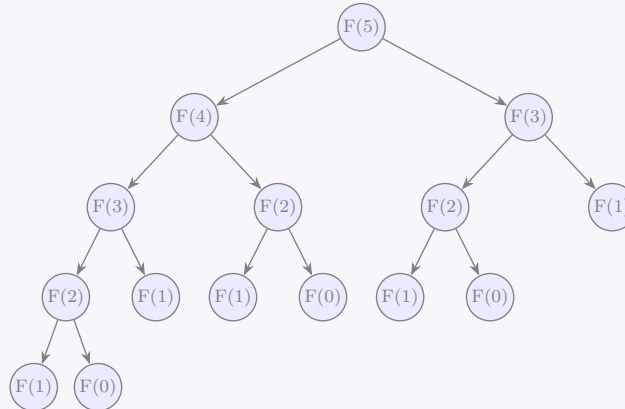
```

1: if  $n \leq 1$  then
2:   return  $n$ 
3: else
4:   return  $F(n-1) + F(n-2)$ 
5: end if

```

Dynamic Multi-graph (DAG):

Each function call is a *vertex*; directed edges represent data dependencies. The graph for $F(5)$:



$T_1(n)$ (total work, running on 1 processor):

$T_1(n) = T_1(n-1) + T_1(n-2) + 1$ (one addition per call), $T_1(0) = T_1(1) = 1$.

n	0	1	2	3	4	5
T_1	1	1	3	5	9	15

$T_1(5) = \mathbf{15}$ (total nodes in recursion tree).

$T_\infty(n)$ (critical path / span on infinite processors):

The critical path is the longest chain of *sequential* dependencies. The longest path from root to leaf goes through the left branch each time: $F(5) \rightarrow F(4) \rightarrow F(3) \rightarrow F(2) \rightarrow F(1)$, with one add at each level.

$T_\infty(n) = T_\infty(n-1) + 1 = n$, so $T_\infty(n) = n + 1$ (including the leaf node):

$$T_\infty(5) = \mathbf{6} \quad (\text{path: } F(5) \rightarrow F(4) \rightarrow F(3) \rightarrow F(2) \rightarrow F(1) \rightarrow \text{return})$$

Or counting only internal (non-leaf) nodes in the critical path: $T_\infty = n = 5$.

Using the standard definition $T_\infty(n) = n$ (critical path depth):

$$T_\infty(5) = 5$$

Parallelism:

$$\text{Parallelism} = \frac{T_1}{T_\infty} = \frac{15}{5} = \mathbf{3}$$

This means on average 3 independent computations could run in parallel at each step, but diminishing returns limit actual speedup.

- Q51.** Find a longest common subsequence from the DNA sequences **ACGCTAC** and **CTGCA**. Print the sequence also. Analyze the run times of your methods of finding and printing the sequence. **(10+5=15 marks)** [CSE 207 | 2008–09]

Answer

Let $X = \text{ACGCTAC}$ ($m = 7$) and $Y = \text{CTGCA}$ ($n = 5$).

LCS DP Table $c[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$:

	$\emptyset$	C	T	G	C	A
$\emptyset$	0	0	0	0	0	0
A	0	0	0	0	0	1
C	0	1	1	1	1	1
G	0	1	1	2	2	2
C	0	1	1	2	3	3
T	0	1	2	2	3	3
A	0	1	2	2	3	4
C	0	1	2	2	3	4

LCS length = $c[7][5] = 4$.

Backtracking to print LCS: Start at $c[7][5]$; if $X[i] = Y[j]$, include character and go to $c[i-1][j-1]$; else go to the larger of $c[i-1][j]$ or $c[i][j-1]$.

Tracing: $(7, 5) \rightarrow (6, 4) \rightarrow (5, 3) \rightarrow (4, 2) \rightarrow (3, 1)$

LCS = CGCA

Runtime Analysis:

- **Finding LCS (DP):** Fill $m \times n$ table, $O(1)$ per cell $\Rightarrow O(mn)$ time, $O(mn)$ space.
- **Printing LCS (backtrack):** At each step move diagonally, up, or left; at most $m + n$ steps $\Rightarrow O(m + n)$ time.

Q52. Find the optimal sequence of multiplying the following matrices: $A(2 \times 7)$, $B(7 \times 9)$, $C(9 \times 4)$, $D(4 \times 6)$, $E(6 \times 3)$. How does the memoization technique help? **(10+5=15 marks)** [CSE 207 | 2008–09, 2006–07]

Answer

Dimensions: $p = [2, 7, 9, 4, 6, 3]$

$(A_1 : 2 \times 7, A_2 : 7 \times 9, A_3 : 9 \times 4, A_4 : 4 \times 6, A_5 : 6 \times 3)$.

$m[i][j]$ **table** (minimum scalar multiplications to compute $A_i \cdots A_j$):

$i \backslash j$	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)
1 (A)	0	126 <i>(AB)</i>	198 <i>((AB)C)</i>	246 <i>(((AB)C)D)</i>	282 <i>(((AB)C)D)E)</i>
2 (B)	—	0	252 <i>(BC)</i>	420 <i>((BC)D)</i>	369 <i>(B(C(DE)))</i>
3 (C)	—	—	0	216 <i>(CD)</i>	180 <i>(C(DE))</i>
4 (D)	—	—	—	0	72 <i>(DE)</i>
5 (E)	—	—	—	—	0

Minimum cost = 282 scalar multiplications.

Optimal parenthesization: $((((A \cdot B) \cdot C) \cdot D) \cdot E)$

How memoization helps:

The naive recursive solution for MCM recomputes $m[i][j]$ exponentially many times due to overlapping subproblems. Memoization optimizes this process in the following ways:

- **Caching results:** We store each computed $m[i][j]$ in a table the first time it is calculated.
- **Avoiding recomputation:** On subsequent recursive calls for the same subproblem, we directly return the stored value in $O(1)$ time.
- **Complexity reduction:** Since there are only $O(n^2)$ distinct subproblems and each takes $O(n)$ time to compute, the total time complexity drops drastically from exponential ($O(2^n)$) to polynomial ($O(n^3)$).
- **Ease of implementation:** It provides the exact same efficiency as bottom-up DP while maintaining the intuitive, top-down recursive mathematical approach.

Q53. Mention the basic properties that must be satisfied to solve a problem using dynamic programming. When and how can memoization help improve an exponential-time recursive algorithm to polynomial time? **(7+8 marks)** [CSE 207 | 2007–08]

Answer**Properties required for dynamic programming:**

- (a) **Optimal substructure:** an optimal solution contains optimal solutions to sub-problems.
- (b) **Overlapping sub-problems:** the same sub-problems are solved repeatedly in a naive recursive solution.

How memoization helps:

A naive recursive algorithm re-solves the same sub-problems exponentially many times. Memoization stores the result of each sub-problem the first time it is computed; subsequent calls return the cached result in $O(1)$.

Example — Fibonacci: naive recursion is $O(2^n)$ because $F(n-2)$ is recomputed in both the $F(n-1)$ and $F(n)$ calls. With memoization, each $F(k)$ is computed exactly once $\rightarrow O(n)$ total.

General principle: if there are P distinct sub-problems and each takes $O(T)$ to solve (given sub-answers), memoization gives $O(P \cdot T)$ total time. For LCS: $P = mn$, $T = O(1) \rightarrow O(mn)$. For matrix chain: $P = O(n^2)$, $T = O(n) \rightarrow O(n^3)$.

Q54. Suppose a thief went to a superstore to steal. He has a bag with him which can contain at most W unit of weight of anything. The thief can carry at most W' unit of weight. In the store there are two types of things. For Type 1 things, he either has to take the thing or leave it. For Type 2 things, he can weigh and take as per his will (e.g. sugar, rice, salt etc.). All type 1 things are labeled with the corresponding unit prices and each type 2 thing is labeled with the corresponding price per unit weight. The thief decided to take only the things belonging to Type 1. He now needs an algorithm to maximize his profits.

- (a) Map the above problem to one of your known problems. **(5 marks)**
- (b) Discuss possible greedy strategies to solve the problem and discuss whether or not any of them will work. **(7 marks)**
- (c) If the thief decides to take only type 2 things, will any of the greedy strategies work? Justify your answer. **(6 marks)**

[CSE 207 | 2007–08]

Answer

(a) **Mapping:** Type 1 items (take whole or leave) map exactly to the **0-1 Knapsack problem**: each item i has weight w_i and value v_i ; the bag capacity is W ; maximise $\sum v_i x_i$ subject to $\sum w_i x_i \leq W$, $x_i \in \{0, 1\}$.

(b) **Greedy strategies for 0-1 Knapsack (Type 1):** Three natural strategies:

- Sort by value v_i descending $\rightarrow$ fails: high-value heavy items may waste capacity.
- Sort by weight w_i ascending $\rightarrow$ fails: light but worthless items crowd out

heavy valuable ones.

- Sort by ratio v_i/w_i descending $\rightarrow$ fails for 0-1 case: taking the best-ratio item may prevent a better combination.

None of these greedy strategies is optimal for 0-1 knapsack. DP is required (see museum thief answer).

(c) Type 2 (fractional): Yes — the greedy strategy of sorting by v_i/w_i descending and taking as much as possible *does* work for the fractional case (the fractional knapsack has the greedy-choice property and optimal substructure — proved in the fractional knapsack answer).

Q55. Find an optimal solution to the 0–1 knapsack problem with the following weight and profit values (weight limit = 8): (1, 2), (1, 3), (2, 3), (2, 5), (3, 4), (3, 2), (3, 7). **(10 marks)** [CSE 207 | 2006–07]

Answer

Given: Capacity $W = 8$.

Items (w, p) : 1:(1, 2), 2:(1, 3), 3:(2, 3), 4:(2, 5), 5:(3, 4), 6:(3, 2), 7:(3, 7).

Dynamic Programming Table (0-1 Knapsack):

Let $dp[i][w]$ be the maximum profit using the first i items with capacity w .

$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + p_i)$

i (Item)	(w_i, p_i)	$w = 0$	1	2	3	4	5	6	7	8
0	(0, 0)	0	0	0	0	0	0	0	0	0
1	(1, 2)	0	2	2	2	2	2	2	2	2
2	(1, 3)	0	3	5	5	5	5	5	5	5
3	(2, 3)	0	3	5	6	8	8	8	8	8
4	(2, 5)	0	3	5	8	10	11	13	13	13
5	(3, 4)	0	3	5	8	10	11	13	14	15
6	(3, 2)	0	3	5	8	10	11	13	14	15
7	(3, 7)	0	3	5	8	10	12	15	17	18

Traceback to find selected items: Start at $dp[7][8] = 18$:

- $dp[7][8](18) \neq dp[6][8](15) \Rightarrow$ **Take Item 7.** Remaining $W = 8 - 3 = 5$.
- $dp[6][5](11) == dp[5][5](11) \Rightarrow$ **Skip Item 6.**
- $dp[5][5](11) == dp[4][5](11) \Rightarrow$ **Skip Item 5.**
- $dp[4][5](11) \neq dp[3][5](8) \Rightarrow$ **Take Item 4.** Remaining $W = 5 - 2 = 3$.
- $dp[3][3](6) \neq dp[2][3](5) \Rightarrow$ **Take Item 3.** Remaining $W = 3 - 2 = 1$.
- $dp[2][1](3) \neq dp[1][1](2) \Rightarrow$ **Take Item 2.** Remaining $W = 1 - 1 = 0$.

Optimal Solution: Selected Items: **{2, 3, 4, 7}**

Total Weight: $1 + 2 + 2 + 3 = 8$

Total Profit: $3 + 3 + 5 + 7 = 18$

Complexity Analysis. Time Complexity: $O(nW) = O(7 \times 8) = O(56)$.
 Space Complexity: $O(nW)$ for the full DP table.

- Q56.** Construct four matrices for finding all-pairs shortest paths for a given graph (edges: $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6), FD(3)$). **(15 marks)** [CSE 207 | 2006–07]

Answer

Floyd-Warshall for all-pairs shortest paths:

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]).$$

Index vertices as $A = 1, B = 2, C = 3, D = 4, E = 5, F = 6$.

Directed edges:

$AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6), FD(3)$.

$D^{(0)}$ (direct edges, ∞ if no edge):

	A	B	C	D	E	F
A	0	7	3	∞	∞	∞
B	∞	0	91	∞	∞	∞
C	∞	∞	0	∞	∞	9
D	4	7	∞	0	∞	∞
E	1	2	∞	∞	0	∞
F	∞	5	∞	3	∞	0

After all 6 iterations (intermediate matrices $D^{(1)}$ through $D^{(6)}$ updated via the recurrence), the final $D^{(6)}$ gives all-pairs shortest paths.

Remark. For full table construction: apply the recurrence for $k = A, B, \dots, F$ in order. Each step takes $O(V^2)$; total $O(V^3) = O(216)$.

- Q57.** Find the longest common subsequence from the DNA sequences **ACGCTAC** and **CTGCA**. Print the LCS and analyze the runtime of both finding and printing the sequence. **(10+5 marks)** [CSE 207 | 2006–07]

Answer

$X = \langle A, C, G, C, T, A, C \rangle, Y = \langle C, T, G, C, A \rangle$.

DP Table ($c[i][j]$):

	ϵ	C	T	G	C	A
ϵ	0	0	0	0	0	0
A	0	0	0	0	0	1
C	0	1	1	1	1	1
G	0	1	1	2	2	2
C	0	1	1	2	3	3
T	0	1	2	2	3	3
A	0	1	2	2	3	4
C	0	1	2	2	3	4

LCS length = 4. LCS = $\langle C, G, C, A \rangle$.

Complexity Analysis. Finding LCS: $\Theta(mn) = \Theta(7 \times 5) = \Theta(35)$. Printing: $O(m + n) = O(12)$.

Q58. Find a $\Theta(n^2 2^n)$ TSP tour for the following distance matrix. What is the major drawback of this solution method?

	A	B	C	D
A	0	10	15	20
B	5	0	9	10
C	6	13	0	12
D	8	8	9	0

(12 marks)

[CSE 207 | 2006–07]

Answer

Held-Karp DP: $dp[S][i]$ = min cost to visit all cities in set S , ending at i , starting from A .

$$dp[S][i] = \min_{j \in S, j \neq i} (dp[S \setminus \{i\}][j] + d[j][i])$$

Optimal tour: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$.

Cost: $AB(10) + BD(10) + DC(9) + CA(6) = 35$.

Major drawback: time complexity $O(n^2 \cdot 2^n)$ — exponential in n . For $n = 20$, this requires $\approx 4 \times 10^8$ operations; for $n = 50$, completely intractable. TSP is NP-hard; no polynomial algorithm is known.

BUET · CSE 105

DSA

Comparison-Based Sorting

Quicksort, Mergesort, Heapsort, Insertion Sort

T12 — Comparison-Based Sorting

Q1. Analyze the worst-case runtime of the Quicksort algorithm under the following assumptions. In your analysis, formulate the worst-case runtime as a recurrence relation and solve the recurrence.

- (a) Balanced partitioning
- (b) Imbalanced partitioning

(5+5=10 marks)

[CSE 105 | 2023–24]

Answer

(a) Balanced partitioning. Each call to PARTITION splits the n -element subarray into two halves of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. The partition step itself costs $\Theta(n)$. The recurrence is:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1).$$

By the Master Theorem (case 2: $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$):

$$T(n) = \Theta(n \log n).$$

(b) Imbalanced (worst-case) partitioning. The pivot is always the minimum (or maximum) element, producing subarrays of sizes 0 and $n - 1$:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n), \quad T(1) = \Theta(1).$$

Unrolling:

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

Complexity Analysis. Balanced: $\Theta(n \log n)$. Imbalanced (worst case): $\Theta(n^2)$.

Q2. Define stable sorting algorithm. Explain with an example. (5 marks) [CSE 105 | 2023–24]

Answer

Definition. A sorting algorithm is **stable** if, for any two elements a_i and a_j with equal keys, the relative order of a_i and a_j in the sorted output is the same as their relative order in the input.

Example. Consider sorting student records by grade:

Student	Grade	Position
Alice	B	1
Bob	A	2
Charlie	B	3

After a *stable* sort by grade, Alice (B, position 1) must appear before Charlie (B, position 3) since both have grade B and Alice came first. An *unstable* sort may place Charlie before Alice.

Stable algorithms: Insertion Sort, Merge Sort, Counting Sort, Radix Sort.

Unstable algorithms: Heap Sort, Quick Sort (standard), Selection Sort.

Remark. Stability matters when a secondary ordering must be preserved after sorting by a primary key — for example, sorting a list first by name and then by age should preserve alphabetical order among people of the same age.

Q3. You are asked to sort the following array of integers in ascending order using the Quicksort algorithm. Highlight the pivot item at each step and show the status of the array after processing each pivot element:

[5, 17, 2, 4, 16, 8, 10, 6, 4, 26, 20, 7]

(15 marks)

[CSE 105 | 2022–23]

Answer

Convention: Last element of each subarray is chosen as pivot. Lomuto partition is used. After each pivot is placed, it is in its final sorted position. **Bold** denotes the pivot placed at this step; underscore marks the active subarray.

Step	Pivot	Array state after placing pivot
1	7	[5, 2, 4, 6, 4, 7 , 10, 17, 16, 26, 20, 8]
2	4	[2, 4, 4 , 6, 5, 7, 10, 17, 16, 26, 20, 8]
3	4	[2, 4 , 4, 6, 5, 7, 10, 17, 16, 26, 20, 8]
4	5	[2, 4, 4, 5 , 6, 7, 10, 17, 16, 26, 20, 8]
5	8	[2, 4, 4, 5, 6, 7, 8 , 17, 16, 26, 20, 10]
6	10	[2, 4, 4, 5, 6, 7, 8, 10 , 16, 26, 20, 17]
7	17	[2, 4, 4, 5, 6, 7, 8, 10, 16, 17 , 20, 26]
8	26	[2, 4, 4, 5, 6, 7, 8, 10, 16, 17, 20, 26]
Sorted		[2, 4, 4, 5, 6, 7, 8, 10, 16, 17, 20, 26]

Remark. Steps 2–4 handle the left subarray [5, 17, 2, 4, 16, 8, 10, 6, 4] recursively (only the portion to the left of pivot 7 is shown). Steps 5–8 handle the right subarray. Each pivot is placed exactly once and never moved again.

Q4. QUICK-SORT is a divide-and-conquer based algorithm. The traditional PARTITION function of QUICK-SORT works by selecting a *pivot* element from the input array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the *pivot*. For this reason, it is sometimes called *partition-exchange* sort. The sub-arrays are then sorted recursively. This can be done *in-place*,

requiring small additional amounts of memory to perform the sorting. Now answer the following questions.

- (i) What is the worst-case running time of QUICK-SORT algorithm if it is designed in a way that the last element of the input array is always selected as the *pivot*? Also give an example input scenario which will bring about the worst-case running time.
- (ii) Suppose, you have been asked to evaluate the performance of QUICK-SORT algorithm. You are using a bad input array which results in a very unbalanced partition at each of the recursive steps. Consider that at each recursive call, the PARTITION function is producing an α -to- β proportional split. Now using a recursion tree, find the time complexity of the QUICK-SORT algorithm for this scenario. If the last three digits of your student ID represents a number x , then consider the following:

$$\begin{aligned}\alpha &= x & \text{if } x \leq 100 \\ \alpha &= 200 - x & \text{if } x > 100 \\ \beta &= 100 - \alpha\end{aligned}$$

Your answer must contain a well-drawn recursion tree where any kind of existing skewness of the tree is clearly illustrated.

(5+7=12 marks)

[CSE 203 | 2020–21]

Answer

(i) Worst-case with last element always as pivot.

The worst case occurs when the last element is always the smallest (or largest) in the subarray, producing a 0-to- $(n-1)$ split at every level:

$$T(n) = T(0) + T(n-1) + \Theta(n) = T(n-1) + \Theta(n).$$

Unrolling: $T(n) = \sum_{k=1}^n \Theta(k) = \Theta(n^2)$.

Example input: An already-sorted array in ascending order, e.g., $[1, 2, 3, 4, 5]$. The last element (5) is the maximum; partition produces $[1, 2, 3, 4]$ and $[]$. Recursion depth: n .

(ii) α -to- β split analysis based on Student ID:

Given: Last three digits of student ID is 009 $\implies x = 9$.

Since $x \leq 100$, we calculate the split proportions as:

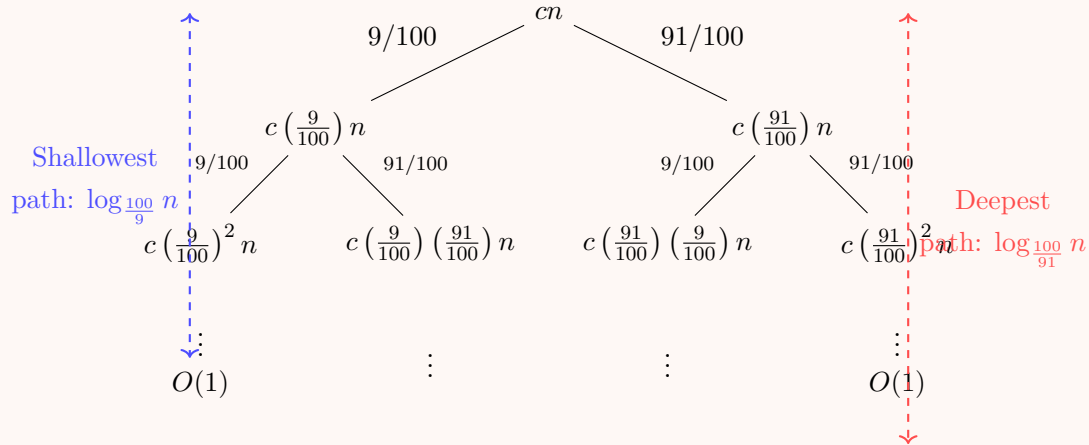
- $\alpha = x = 9$
- $\beta = 100 - \alpha = 100 - 9 = 91$

This means the PARTITION function produces a highly unbalanced proportional split of 9% to 91%. The recurrence relation for this scenario is:

$$T(n) = T\left(\frac{9}{100}n\right) + T\left(\frac{91}{100}n\right) + cn$$

where cn represents the linear time $\Theta(n)$ required to partition the array.

Recursion Tree:



Analysis:

- **Shallowest path (Left branch):** The subproblem size decreases by a factor of $9/100$. This path hits the base case (size 1) early at depth $d_{\min} = \log_{100/9} n$.
- **Deepest path (Right branch):** The subproblem size decreases by a factor of $91/100$. This path hits the base case much later at depth $d_{\max} = \log_{100/91} n$.
- **Cost per level:** At each level up to the shallowest path ($d \leq d_{\min}$), the sum of costs across all nodes is exactly cn . Beyond that depth ($d > d_{\min}$), some branches have already hit the base case and stopped, so the cost per level becomes strictly less than cn (i.e., $\leq cn$).

Time Complexity:

To find the upper bound of the total time, we multiply the maximum cost per level (cn) by the maximum depth of the tree ($d_{\max}$):

$$T(n) \leq cn \cdot \log_{\frac{100}{91}} n = O(n \log n)$$

Similarly, the lower bound is $T(n) \geq cn \cdot \log_{\frac{100}{9}} n = \Omega(n \log n)$.

Since both bounds are tightly proportional to $n \log n$, the time complexity is exactly $\Theta(n \log n)$. The skewness of the tree only affects the constant factors, but not the overall asymptotic bound.

Q5. Apply the partition procedure of Quicksort once on the array below, where the pivot is chosen as the element at the middle index $\lfloor (p + q)/2 \rfloor$ (indices range from p to q inclusive). Show all steps.

[10, 30, 50, 70, 90, 100, 80, 60, 40, 20]

(10+7 marks)

[CSE 203 | 2019–20]

Answer

Array: $A = [10, 30, 50, 70, 90, 100, 80, 60, 40, 20]$, $p = 0$, $q = 9$.

$\text{mid_idx} = \lfloor (0 + 9)/2 \rfloor = 4$, so $\text{pivot} = A[4] = 90$.

Step 1: Swap $A[4]$ with $A[9]$ (move pivot to end):

$[10, 30, 50, 70, \underline{20}, 100, 80, 60, 40, \underline{90}]$

Step 2: Lomuto scan (i starts at -1 , j from 0 to 8):

j	$A[j]$	$\leq 90?$	Action	Array
0	10	Yes	$i=0$, swap(0, 0) (trivial)	$[10, 30, 50, 70, 20, 100, 80, 60, 40, 90]$
1	30	Yes	$i=1$, swap(1, 1) (trivial)	unchanged
2	50	Yes	$i=2$, swap(2, 2) (trivial)	unchanged
3	70	Yes	$i=3$, swap(3, 3) (trivial)	unchanged
4	20	Yes	$i=4$, swap(4, 4) (trivial)	unchanged
5	100	No	skip	unchanged
6	80	Yes	$i=5$, swap(5, 6)	$[10, 30, 50, 70, 20, 80, 100, 60, 40, 90]$
7	60	Yes	$i=6$, swap(6, 7)	$[10, 30, 50, 70, 20, 80, 60, 100, 40, 90]$
8	40	Yes	$i=7$, swap(7, 8)	$[10, 30, 50, 70, 20, 80, 60, 40, 100, 90]$

Step 3: Place pivot: swap $A[i+1] = A[8]$ with $A[9]$:

$[10, 30, 50, 70, 20, 80, 60, 40, \mathbf{90}, 100]$

Pivot 90 is now at index 8 (its final sorted position). Left partition: $[10, 30, 50, 70, 20, 80, 60, 40]$; right partition: $[100]$.

Q6. Describe a scenario where an in-place sorting algorithm is necessary and explain the reasons. (5 marks) [CSE 203 | 2019–20]

Answer

Scenario: Sorting a large dataset in an embedded system with severe memory constraints.

Consider a microcontroller with only 4 KB of RAM that must sort an array of 1000 integers (4 KB of data). An out-of-place algorithm such as Merge Sort requires an auxiliary array of the same size (another 4 KB), which exceeds the available memory.

An *in-place* algorithm such as Insertion Sort or Heap Sort uses only $O(1)$ extra space (a few variables), making it feasible.

Other scenarios requiring in-place sorting:

- **Real-time systems:** Memory allocation may be disallowed or too slow.
- **Cache performance:** Operating entirely within the existing array improves cache locality.

- **Very large files:** When the dataset barely fits in memory, no extra buffer is available.

Remark. In-place does *not* mean the algorithm is faster — Heap Sort ($\Theta(n \log n)$ in-place) vs. Merge Sort ($\Theta(n \log n)$ out-of-place). The choice is driven by memory, not time.

Q7. Give a concise description of a good way for Quicksort to choose a pivot element (better than always using index 0). **(5 marks)** [CSE 203 | 2018–19]

Answer

Median-of-Three pivot selection.

Examine three elements: the first ($A[\text{low}]$), the middle ($A[(\text{low} + \text{high})/2]$), and the last ($A[\text{high}]$). Use their *median* as the pivot.

Example: For subarray $[3, 1, 6, 8, 4]$: first= 3, mid= 6, last= 4. Median= 4. Use 4 as pivot.

Why it is better:

- Avoids worst-case behaviour on already-sorted or reverse-sorted inputs (the primary failure mode of fixed-index pivot selection).
- Produces a split closer to balanced on average, reducing expected recursion depth.
- Costs only $O(1)$ extra work (3 comparisons).

Alternative — Randomized pivot: Choose the pivot uniformly at random from the subarray. This provides $O(n \log n)$ expected time regardless of input order, eliminating adversarial worst-case inputs.

Q8. Write down the Quicksort algorithm. Use the partition algorithm to find the smallest 5 values of the array $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$, showing every step. **(5+10 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: (See Q14 above for full pseudocode.)

Finding the 5 smallest via Quickselect (partition-based selection):

Goal: place element at index 4 (0-indexed) in its correct sorted position. All elements to its left will be $\leq$ all elements to its right, giving the 5 smallest.

Array: $A = [6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$

Step	Range	Pivot	Pivot placed at idx	Array after partition
1	[0..13]	5	6 6 > 4, recurse left	[1, 2, 3, 4, 5, 4, 5 , 6, 10, 8, 11, 17, 7, 9]
2	[0..5]	4	4 4 = 4, stop!	[1, 2, 3, 4, 4 , 5, 5, 6, 10, 8, 11, 17, 7, 9]

After step 2, index 4 holds the 5th smallest element (value 4). All elements at indices 0..4 form the 5 smallest values (not necessarily sorted among themselves):

{1, 2, 3, 4, 4} (the 5 smallest elements)

Remark. Quickselect runs in $O(n)$ average time (vs. $O(n \log n)$ for full sort), making it efficient when only a partial order is needed.

Q9. What is an in-place sorting algorithm? Write an in-place sorting algorithm. Analyze the worst-case and best-case time complexities of the algorithm. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Answer

In-place sorting: An algorithm is *in-place* if it uses only $O(1)$ auxiliary space beyond the input array (i.e., it rearranges elements within the array itself without requiring a secondary array of comparable size).

Algorithm 126 SELECTIONSORT(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:    $minIdx \leftarrow i$ 
3:   for  $j = i + 1$  to  $n$  do
4:     if  $A[j] < A[minIdx]$  then
5:        $minIdx \leftarrow j$ 
6:     end if
7:   end for
8:   SWAP( $A[i], A[minIdx]$ )
9: end for
    
```

Best case: Even if the array is sorted, the inner loop still runs $n - i$ times for each i , so total comparisons = $\sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2} = \Theta(n^2)$.

Worst case: Same as best case — $\Theta(n^2)$.

Complexity Analysis. Best and worst case: $\Theta(n^2)$. Extra space: $O(1)$ (in-place). Selection Sort makes at most $n - 1$ swaps, which is useful when writes are expensive.

Or :

Algorithm 127 INSERTIONSORT(A, n)

```

1: for  $i = 2$  to  $n$  do
2:    $key \leftarrow A[i]$ 
3:    $j \leftarrow i - 1$ 
4:   while  $j \geq 1$  and  $A[j] > key$  do
5:      $A[j + 1] \leftarrow A[j]$ 
6:      $j \leftarrow j - 1$ 
7:   end while
8:    $A[j + 1] \leftarrow key$ 
9: end for

```

Only a constant number of variables (key, i, j) are used $\Rightarrow O(1)$ extra space.

Best case (sorted input): $\Theta(n)$. **Worst case** (reverse-sorted): $\Theta(n^2)$.

Remark. Other in-place sorting algorithms include Selection Sort ($\Theta(n^2)$ always), Heap Sort ($\Theta(n \log n)$), and in-place Quicksort ($O(n^2)$ worst, $O(n \log n)$ average). Merge Sort is *not* naturally in-place as it requires $O(n)$ auxiliary space.

Q10. Derive the expected time complexity of randomized Quicksort. **(20 marks)** [CSE 207 | 2016–17, 2009–10]

Answer

Setup. Given an array A of n distinct elements, let $z_1 < z_2 < \dots < z_n$ be the elements in sorted order. Define:

$$X_{ij} = \mathbf{1}[z_i \text{ is compared with } z_j], \quad 1 \leq i < j \leq n.$$

$$\text{Total comparisons: } X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

Key observation. z_i and z_j are compared if and only if one of them is chosen as pivot when both are in the same subarray. Once a pivot is chosen from $\{z_i, \dots, z_j\}$:

- If the pivot is z_i or z_j : they *are* compared (probability $\frac{2}{j-i+1}$).
- Otherwise: they are placed in different subarrays and never compared.

Claim:

$$\Pr[X_{ij} = 1] = \frac{2}{j-i+1}.$$

Proof: In randomized Quicksort, every element in $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ is equally likely to be the first chosen pivot from this set (since pivots are chosen uniformly at random). The set has $j-i+1$ elements; exactly 2 of them (z_i and z_j) cause a comparison. Hence $\Pr[X_{ij} = 1] = \frac{2}{j-i+1}$. $\square$

Expected comparisons:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.$$

Substitute $k = j - i$ (so k runs from 1 to $n - i$):

$$E[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} 2H_n = 2(n-1)H_n,$$

where $H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$ is the n -th harmonic number.

Therefore:

$$E[X] < 2(n-1) \ln n + O(n) = O(n \log n).$$

A matching lower bound $\Omega(n \log n)$ follows from information-theoretic arguments (any comparison-based sort needs $\Omega(n \log n)$ comparisons). Hence:

$$E[T(n)] = \Theta(n \log n).$$

Complexity Analysis. Randomized Quicksort expected time: $\Theta(n \log n)$ for any input. This holds regardless of the input permutation since randomness is internal to the algorithm.

Q11. Write the Insertion Sort algorithm. Analyze the worst-case and best-case time complexities of the algorithm. (10 marks) [CSE 203 | 2015–16]

Answer

See Question No.9

Q12. Write the Quick Sort algorithm. Analyze the best-case and worst-case time complexity of the algorithm. (15 marks) [CSE 203 | 2014–15, 2013–14]

Answer

Algorithm 128 QUICKSORT($A, low, high$)

```

1: if  $low < high$  then
2:    $p \leftarrow \text{PARTITION}(A, low, high)$ 
3:   QUICKSORT( $A, low, p - 1$ )
4:   QUICKSORT( $A, p + 1, high$ )
5: end if
```

Algorithm 129 PARTITION($A, low, high$) — Lomuto scheme

```

1:  $pivot \leftarrow A[high]$ 
2:  $i \leftarrow low - 1$ 
3: for  $j = low$  to  $high - 1$  do
4:   if  $A[j] \leq pivot$  then
5:      $i \leftarrow i + 1$ 
6:     SWAP( $A[i], A[j]$ )
7:   end if
8: end for
9: SWAP( $A[i + 1], A[high]$ )
10: return  $i + 1$ 
    
```

Best-case analysis. The best case occurs when PARTITION always splits the array into two equal halves:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

Master Theorem (case 2) gives $T(n) = \Theta(n \log n)$.

Worst-case analysis. The worst case occurs when the pivot is always the minimum or maximum, yielding a 0-to- $(n - 1)$ split:

$$T(n) = T(n - 1) + \Theta(n).$$

Unrolling the recurrence:

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

This occurs when the array is already sorted (ascending or descending) and the last (or first) element is always chosen as pivot.

Complexity Analysis. Best case: $\Theta(n \log n)$. Worst case: $\Theta(n^2)$. Average case: $\Theta(n \log n)$ (shown via indicator random variables).

- Q13.** Write the pseudocode of the Quicksort algorithm including the partition procedure. Deduce its average-case complexity. Simulate the partition algorithm on the array $[6, 2, 7, 3, 8, 4, 9, 5, 10, 1, 11]$ using the first element as the partitioning element. Show whenever elements are swapped. **(10 marks)** [CSE 207 | 2013–14]

Answer

Algorithm: (Full pseudocode as in Q12.)

Average-case complexity.

Let $T(n)$ be the average running time when the pivot is equally likely to be any of

the n elements (rank k with probability $1/n$). Then:

$$T(n) = \frac{1}{n} \sum_{k=1}^n [T(k-1) + T(n-k)] + \Theta(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \Theta(n).$$

Using the substitution $T(n) = an \log n + b$ (or solving with indicator random variables), one shows:

$$T(n) = \Theta(n \log n).$$

Partition simulation on $A = [6, 2, 7, 3, 8, 4, 9, 5, 10, 1, 11]$, pivot = $A[0] = 6$:
Swap pivot to end: $[11, 2, 7, 3, 8, 4, 9, 5, 10, 1, 6]$. Scan $j = 0$ to 9:

j	$A[j]$	$\leq 6?$	Swap(i, j)?	Array (relevant portion)
0	11	No	—	unchanged
1	2	Yes	(0, 1)	[2, 11, 7, 3, 8, 4, 9, 5, 10, 1, 6]
2	7	No	—	unchanged
3	3	Yes	(1, 3)	[2, 3, 7, 11, 8, 4, 9, 5, 10, 1, 6]
4	8	No	—	unchanged
5	4	Yes	(2, 5)	[2, 3, 4, 11, 8, 7, 9, 5, 10, 1, 6]
6	9	No	—	unchanged
7	5	Yes	(3, 7)	[2, 3, 4, 5, 8, 7, 9, 11, 10, 1, 6]
8	10	No	—	unchanged
9	1	Yes	(4, 9)	[2, 3, 4, 5, 1, 7, 9, 11, 10, 8, 6]

Place pivot: swap $A[5]$ with $A[10]$:

[2, 3, 4, 5, 1, **6**, 9, 11, 10, 8, 7]

Pivot 6 is at index 5 (its final position). Left: [2, 3, 4, 5, 1]; right: [9, 11, 10, 8, 7].

Remark. Note: the left partition [2, 3, 4, 5, 1] contains elements ≤ 6 but is not yet sorted among themselves — further recursive calls handle this.

Q14. Write a sorting algorithm that can always sort n elements in $O(n \log n)$ time. Analyze the best-case and worst-case time complexity of the algorithm. **(7+8=15 marks)**
[CSE 203 | 2013–14]

Answer

Algorithm — **Merge Sort** (guaranteed $\Theta(n \log n)$ in all cases):

Algorithm 130 MERGE($A, low, mid, high$) — detailed

```

1:  $n_1 \leftarrow mid - low + 1$ ;  $n_2 \leftarrow high - mid$ 
2: Create arrays  $L[1..n_1]$  and  $R[1..n_2]$ 
3: for  $i = 1$  to  $n_1$  do
4:    $L[i] \leftarrow A[low + i - 1]$ 
5: end for
6: for  $j = 1$  to  $n_2$  do
7:    $R[j] \leftarrow A[mid + j]$ 
8: end for
9:  $i \leftarrow 1$ ;  $j \leftarrow 1$ ;  $k \leftarrow low$ 
10: while  $i \leq n_1$  and  $j \leq n_2$  do
11:   if  $L[i] \leq R[j]$  then
12:      $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
13:   else
14:      $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
15:   end if
16:    $k \leftarrow k + 1$ 
17: end while
18: Copy remaining elements of  $L$  and  $R$  into  $A$ 

```

Best-case analysis. Even if the array is already sorted, Merge Sort still divides and merges every level. The recurrence is:

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Master Theorem (case 2): $T(n) = \Theta(n \log n)$.

Worst-case analysis. The same recurrence holds regardless of the input (since MERGE always takes $\Theta(n)$ for a pair of subarrays of total size n):

$$T(n) = \Theta(n \log n).$$

Complexity Analysis. Best case: $\Theta(n \log n)$. Worst case: $\Theta(n \log n)$. Space: $\Theta(n)$ auxiliary (not in-place). Merge Sort is stable.

- Q15.** Explain the divide-and-conquer technique. Write the quick sort algorithm and analyze the time complexity of the algorithm. **(15 marks)** [CSE 203 | 2013–14]

Answer

Divide-and-Conquer. A paradigm with three steps applied recursively:

1. **Divide:** Split the problem into smaller subproblems of the same type.
2. **Conquer:** Solve subproblems recursively (base cases are solved directly).

3. Combine: Merge the solutions to subproblems into the solution for the original.

Examples: Merge Sort, Quicksort, Binary Search, Strassen's matrix multiplication.

Quicksort applies divide-and-conquer:

- **Divide:** PARTITION places a pivot in its correct position; elements $<$ pivot go left, $>$ pivot go right.
- **Conquer:** Recursively sort the two subarrays.
- **Combine:** No work needed — the array is sorted in place.

Algorithm 131 QUICKSORT($A, low, high$)

```

1: if  $low < high$  then
2:    $p \leftarrow \text{PARTITION}(A, low, high)$ 
3:   QUICKSORT( $A, low, p - 1$ )
4:   QUICKSORT( $A, p + 1, high$ )
5: end if

```

Algorithm 132 PARTITION($A, low, high$) — Lomuto scheme

```

1:  $pivot \leftarrow A[high]$ 
2:  $i \leftarrow low - 1$ 
3: for  $j = low$  to  $high - 1$  do
4:   if  $A[j] \leq pivot$  then
5:      $i \leftarrow i + 1$ 
6:     SWAP( $A[i], A[j]$ )
7:   end if
8: end for
9: SWAP( $A[i + 1], A[high]$ )
10: return  $i + 1$ 

```

Time complexity:

Case	Recurrence	Solution
Best	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Average	$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \Theta(n)$	$\Theta(n \log n)$
Worst	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$

Remark. Despite its $\Theta(n^2)$ worst case, Quicksort is preferred in practice over Merge Sort due to smaller constant factors, better cache behaviour (in-place), and $\Theta(n \log n)$ average performance. Randomized pivot selection effectively eliminates worst-case scenarios.

Q16. Sort the following dataset using Quicksort in *descending* order, using the first element as pivot (show all steps):

$$D = \{6, 0, 2, 0, 1, 3, 6, 1\}$$

(12 marks)

[CSE 203 | 2012–13]

Answer

Convention: First element is pivot. To sort descending, an element x goes to the *left* partition if $x \geq$ pivot, and to the *right* partition if $x <$ pivot.

Swap pivot to end, then scan with the modified Lomuto condition ($\leq$ replaced by $\geq$):

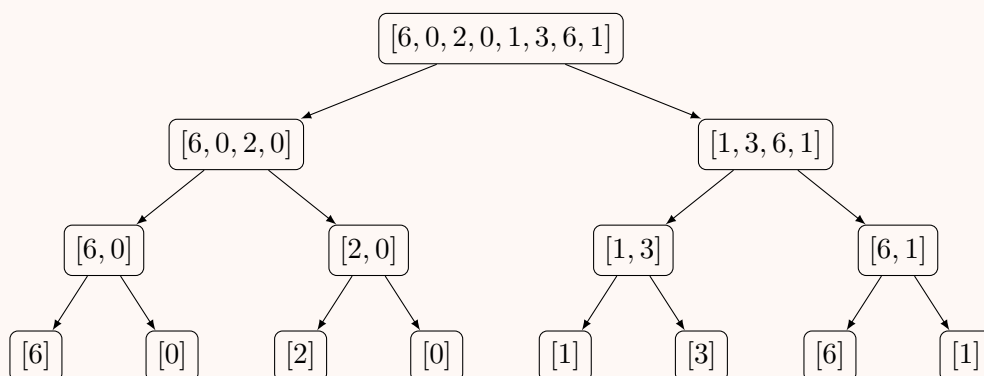
Step	Subarray	Pivot	After partition
1	[6, 0, 2, 0, 1, 3, 6, 1]	6	[6, 6, 2, 0, 1, 3, 1, 0]; pivot at idx 1
2	[6]	6	single element, done
3	[2, 0, 1, 3, 1, 0]	2	[3, 2, 0, 1, 1, 0]; pivot at idx 1
4	[3]	3	single element, done
5	[0, 1, 1, 0]	0	[1, 1, 0, 0]; pivot at idx 2
6	[1, 1]	1	[1, 1]; pivot at idx 1
7	[0]	0	single element, done
Sorted (descending)			[6, 6, 3, 2, 1, 1, 0, 0]

Remark. The final sorted array is [6, 6, 3, 2, 1, 1, 0, 0] — confirmed by reversing the naturally sorted order [0, 0, 1, 1, 2, 3, 6, 6].

Q17. Using Mergesort on the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$, draw the merge tree. Show that Mergesort runs in time $\Theta(n \log n)$ in the worst case. (6+4=10 marks) [CSE 203 | 2012–13]

Answer

Merge Tree (top = original array, bottom = single elements, arrows show split direction):



Merge steps (bottom-up):

- $\text{MERGE}([6], [0]) \rightarrow [0, 6]$; $\text{MERGE}([2], [0]) \rightarrow [0, 2]$
- $\text{MERGE}([1], [3]) \rightarrow [1, 3]$; $\text{MERGE}([6], [1]) \rightarrow [1, 6]$
- $\text{MERGE}([0, 6], [0, 2]) \rightarrow [0, 0, 2, 6]$; $\text{MERGE}([1, 3], [1, 6]) \rightarrow [1, 1, 3, 6]$
- $\text{MERGE}([0, 0, 2, 6], [1, 1, 3, 6]) \rightarrow [0, 0, 1, 1, 2, 3, 6, 6]$

Proof of $\Theta(n \log n)$ worst-case:

The recurrence is $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$.

- The recursion tree has $\log_2 n$ levels.
- At level k , there are 2^k subproblems each of size $n/2^k$.
- Merge cost at level k : $2^k \times \Theta(n/2^k) = \Theta(n)$ per level.
- Total: $\Theta(n) \times \log_2 n = \Theta(n \log n)$.

By the Master Theorem (case 2), $T(n) = \Theta(n \log n)$. Since the merge step always scans both halves, this bound is tight: worst case = $\Theta(n \log n)$.

Q18. Write the Quicksort algorithm. Simulate the Quicksort algorithm on the array

[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]

showing in detail how the partition routine works. **(8+7=15 marks)** [CSE 203 | 2011–12]

Answer

1. The Quicksort Algorithm

See Question No. 12

2. Simulation of Quicksort

Initial Array: [15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]

Step	Subarray	Pivot	Result (Left Pivot Right)
1	[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]	23	[15, 13, ..., 1] 23 [26]
2	[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1]	1	[] 1 [13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 15]
3	[13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 15]	15	[13, 9, 5, 12, 8, 7, 4, 10, 6, 2] 15 []
4	[13, 9, 5, 12, 8, 7, 4, 10, 6, 2]	2	[] 2 [9, 5, 12, 8, 7, 4, 10, 6, 13]
5	[9, 5, 12, 8, 7, 4, 10, 6, 13]	13	[9, 5, 12, 8, 7, 4, 10, 6] 13 []
6	[9, 5, 12, 8, 7, 4, 10, 6]	6	[5, 4] 6 [8, 7, 9, 10, 12]
7	[5, 4]	4	[] 4 [5]
8	[8, 7, 9, 10, 12]	12	[8, 7, 9, 10] 12 []
9	[8, 7, 9, 10]	10	[8, 7, 9] 10 []
10	[8, 7, 9]	9	[8, 7] 9 []
11	[8, 7]	7	[] 7 [8]

Final Sorted Array: [1, 2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]

3. Detailed Trace of Partition (Step 6)

Tracing partition on subarray [9, 5, 12, 8, 7, 4, 10, 6]. Pivot $x = 6$, Initial $i = -1$.

j	$A[j]$	$A[j] \leq 6$	New i	Action & Array State
–	–	–	-1	[9, 5, 12, 8, 7, 4, 10, 6] (Initial)
0	9	False	-1	No swap. [9, 5, 12, 8, 7, 4, 10, 6]
1	5	True	0	Swap $A[0], A[1] \implies$ [5 , 9 , 12, 8, 7, 4, 10, 6]
2	12	False	0	No swap. [5, 9, 12, 8, 7, 4, 10, 6]
3	8	False	0	No swap. [5, 9, 12, 8, 7, 4, 10, 6]
4	7	False	0	No swap. [5, 9, 12, 8, 7, 4, 10, 6]
5	4	True	1	Swap $A[1], A[5] \implies$ [5, 4 , 12, 8, 7, 9 , 10, 6]
6	10	False	1	No swap. [5, 4, 12, 8, 7, 9, 10, 6]

End of loop. Swap $A[i + 1]$ ($A[2] = 12$) with Pivot ($A[7] = 6$).

Result: [5, 4, **6**, 8, 7, 9, 10, **12**]. Pivot 6 is placed correctly!

Q19. Simulate the Quicksort algorithm on the array [5, 1, 6, 3, 7, 8, 9, 2, 4, 8]. Discuss its worst-case complexity. (15 marks) [CSE 207 | 2008–09]

Answer

Simulation (pivot = last element, Lomuto partition):

Step	Subarray before	Pivot	Subarray after
1	[5, 1, 6, 3, 7, 8, 9, 2, 4, 8]	8	[5, 1, 6, 3, 7, 8, 2, 4, 8 , 9]; idx 8
2	[5, 1, 6, 3, 7, 8, 2, 4]	4	[1, 3, 2, 4, 7, 8, 6, 5]; idx 3
3	[1, 3, 2]	2	[1, 2 , 3]; idx 1
4	[7, 8, 6, 5]	5	[5 , 8, 6, 7]; idx 0
5	[8, 6, 7]	7	[6, 7 , 8]; idx 1
6	[9] (right of step 1)	—	single element
Sorted			[1, 2, 3, 4, 5, 6, 7, 8, 8, 9]

Worst-case complexity:

The worst case is $\Theta(n^2)$, occurring when the pivot is always the minimum or maximum element (e.g., sorted or reverse-sorted input). Each call to PARTITION produces subarrays of sizes 0 and $n - 1$:

$$T(n) = T(n - 1) + \Theta(n) \Rightarrow T(n) = \Theta(n^2).$$

For the given array, the partition is reasonably balanced, so the actual run is much better than worst case.

BUET · CSE 105

DSA

Sorting in Linear Time

Counting Sort, Radix Sort, Bucket Sort

T13 — Sorting in Linear Time

Q1. Sort the following list of non-negative integers using Radix Sort (LSD):

[329, 457, 657, 839, 436, 720, 355]

State the conditions required for Radix Sort to work correctly and in linear time.

(10 marks)

[CSE 105 | 2023–24]

Answer

Algorithm. LSD Radix Sort processes digits from least significant to most significant. At each pass it uses a *stable* sort (Counting Sort) on the current digit.

Pass 1 — Units digit (10^0):

Digit	Numbers in bucket
0	720
5	355
6	436
7	457, 657
9	329, 839

After pass 1: [720, 355, 436, 457, 657, 329, 839]

Pass 2 — Tens digit (10^1):

Digit	Numbers in bucket
2	720, 329
3	436, 839
5	355, 457, 657

After pass 2: [720, 329, 436, 839, 355, 457, 657]

Pass 3 — Hundreds digit (10^2):

Digit	Numbers in bucket
3	329, 355
4	436, 457
6	657
7	720
8	839

After pass 3 (sorted): [**329, 355, 436, 457, 657, 720, 839**]

Conditions for correctness and linear time:

- The per-digit sort must be **stable** (so relative order from earlier passes is preserved).

- The number of digits d must be **constant** (independent of n), i.e., all keys have $O(1)$ digits.
- Each digit lies in a **bounded range** of size k (e.g., 0–9 for decimal), so Counting Sort on one digit costs $\Theta(n + k)$.
- Total time: $d \cdot \Theta(n + k) = \Theta(n)$ when d and k are constants.

Q2. Given two arrays $a[]$ and $b[]$, each containing N distinct points in the plane, design an algorithm with linear running time (under reasonable assumptions) and at most linear extra space that determines whether the two arrays contain precisely the same set of points. **(8 marks)** [CSE 203 | 2018–19]

Answer

We can solve this problem in $O(N)$ time and $O(N)$ extra space using either Hashing or Linear-Time Sorting (under specific reasonable assumptions).

Method 1: Using Hash Table

Key insight. Direct comparison of comparison-based sorted arrays would cost $O(N \log N)$. To achieve $O(N)$, we use hashing to count occurrences.

Algorithm 133 SAMEPOINTSET_HASH(a, b, N)

```

1: Create hash table  $H$ 
2: for  $i = 1$  to  $N$  do
3:   Insert  $a[i]$  into  $H$  (increment count)
4: end for
5: for  $i = 1$  to  $N$  do
6:   if  $b[i] \notin H$  or  $H[b[i]] = 0$  then
7:     return false
8:   end if
9:    $H[b[i]] \leftarrow H[b[i]] - 1$ 
10: end for
11: return true
```

Correctness & Complexity. The first loop records all points in a . The second checks if each point of b is present with a remaining count > 0 . Under the *simple uniform hashing assumption*, operations take $O(1)$ expected time. Thus, Time: $O(N)$, Space: $O(N)$.

Method 2: Using Linear-Time Sorting (Radix Sort)

Key insight. If point coordinates are bounded integers, we can use a non-comparison based sort like Radix Sort to sort both arrays in linear time, then compare them element by element.

Algorithm 134 SAMEPOINTSET_SORT(a, b, N)

```

1: RADIXSORT( $a$ )           ▷ Sort primarily by  $x$ -coordinate, secondarily by  $y$ 
2: RADIXSORT( $b$ )           ▷ Sort primarily by  $x$ -coordinate, secondarily by  $y$ 
3: for  $i = 1$  to  $N$  do
4:   if  $a[i].x \neq b[i].x$  or  $a[i].y \neq b[i].y$  then
5:     return false
6:   end if
7: end for
8: return true

```

Correctness & Complexity. Sorting brings identical sets of points into the exact same order. A single linear scan can then verify if the sets match point-by-point. Under the *reasonable assumption* that coordinates are integers with a constant number of digits, Radix Sort takes $O(N)$ time. The comparison loop also takes $O(N)$. Thus, Time: $O(N)$, Space: $O(N)$ extra space required by Radix Sort.

Q3. Write a linear-time Bucket Sort algorithm. Prove that the time complexity of the algorithm is linear. (10 marks) [CSE 203 | 2014–15, 2011–12]

Answer**Algorithm 135** BUCKETSORT(A, n)

```

1: Create array  $B[0..n-1]$  of empty lists
2: for  $i = 1$  to  $n$  do
3:   Insert  $A[i]$  into  $B[\lfloor n \cdot A[i] \rfloor]$ 
4: end for
5: for  $i = 0$  to  $n-1$  do
6:   INSERTIONSORT( $B[i]$ )
7: end for
8: Concatenate  $B[0], B[1], \dots, B[n-1]$  into  $A$ 

```

Proof of linear expected time.

Setup. Bucket Sort assumes n input elements are drawn *uniformly* from $[0, 1)$. Create n buckets $B[0], B[1], \dots, B[n-1]$; element x goes into bucket $\lfloor n \cdot x \rfloor$.

Analysis. Let n_i = number of elements in bucket i . Each bucket is sorted independently (e.g., with Insertion Sort) at cost $O(n_i^2)$. Total time:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2).$$

Taking expectations (since input is uniform):

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)] = \Theta(n) + n \cdot O(E[n_i^2]).$$

Each element falls in bucket i with probability $p = 1/n$, and $n_i \sim \text{Binomial}(n, 1/n)$. Then:

$$E[n_i^2] = \text{Var}(n_i) + (E[n_i])^2 = \frac{n \cdot \frac{1}{n} \cdot (1 - \frac{1}{n})}{1} + 1 = 2 - \frac{1}{n}.$$

Therefore:

$$E[T(n)] = \Theta(n) + n \cdot O(2 - \frac{1}{n}) = \Theta(n).$$

Complexity Analysis. Average case: $\Theta(n)$ (under the uniform distribution assumption). Worst case: $\Theta(n^2)$ (all elements fall into one bucket).

Intuitive Proof of Linear Expected Time:

Let $T(n)$ be the total time complexity of the Bucket Sort algorithm. The operations can be broken down as follows:

- (a) **Creating buckets and inserting elements:** We iterate through the n input elements and place each into one of the n buckets. This takes $O(n)$ time.
- (b) **Sorting individual buckets:** We use Insertion Sort for each bucket. In the worst case, Insertion Sort takes $O(k^2)$ time for k elements.
- (c) **Concatenating buckets:** Going through n buckets to combine the elements takes $O(n)$ time.

Expected Sorting Time:

Since the input elements are uniformly distributed over the interval $[0, 1)$, the probability that an element falls into any specific bucket is $1/n$. Because there are n elements and n buckets, the *expected (average) number of elements* in each bucket is:

$$k = \frac{n}{n} = 1$$

Since the expected size of each bucket is just 1, the expected time to sort a single bucket using Insertion Sort is $O(1^2) = O(1)$.

For n buckets, the total expected time to sort all buckets is:

$$n \times O(1) = O(n)$$

Total Expected Time Complexity:

Adding the times for all steps:

$$T(n) = O(n) \text{ (insert)} + O(n) \text{ (sort)} + O(n) \text{ (concatenate)} = \mathbf{O(n)}$$

Thus, the expected time complexity of Bucket Sort is linear. □

- Q4.** Write the counting sort algorithm. Prove that the algorithm runs in linear time. Explain why counting sort is stable. **(12 marks)** [CSE 203 | 2013–14]

Answer

Algorithm 136 COUNTINGSORT(A, B, n, k)

```

1: for  $i = 0$  to  $k$  do
2:    $C[i] \leftarrow 0$ 
3: end for
4: for  $j = 1$  to  $n$  do
5:    $C[A[j]] \leftarrow C[A[j]] + 1$ 
6: end for
7: for  $i = 1$  to  $k$  do
8:    $C[i] \leftarrow C[i] + C[i - 1]$ 
9: end for
10: for  $j = n$  downto  $1$  do
11:    $B[C[A[j]]] \leftarrow A[j]$ 
12:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
13: end for

```

Proof of linear time.

- Loop 1 (initialize C): $\Theta(k)$ iterations.
- Loop 2 (count): $\Theta(n)$ iterations.
- Loop 3 (cumulate): $\Theta(k)$ iterations.
- Loop 4 (build output): $\Theta(n)$ iterations.

Total: $\Theta(n + k)$. When $k = O(n)$, this is $\Theta(n)$. $\square$

Why it is stable. In Loop 4, elements are placed into B by scanning A from right to left (index n down to 1). For two equal elements $A[p] = A[q] = v$ with $p < q$: $A[q]$ is processed first (larger index scanned first) and occupies the *higher* position among slots allocated for v ; $A[p]$ is processed next and occupies the *lower* position. Hence $A[p]$ precedes $A[q]$ in B , preserving the original left-to-right order among equal elements. $\square$

Q5. Write the Counting Sort steps for the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$. (6 marks)
[CSE 203 | 2012–13]

Answer

$D = [6, 0, 2, 0, 1, 3, 6, 1]$, $n = 8$, range $k = 6$.

Step 1 — Count:

i	0	1	2	3	4	5	6
$C[i]$	2	2	1	1	0	0	2

Step 2 — Cumulate:

i	0	1	2	3	4	5	6
$C[i]$	2	4	5	6	6	6	8

Step 3 — Build output (right-to-left scan): $1 \rightarrow B[3]$; $6 \rightarrow B[7]$; $3 \rightarrow B[5]$; $1 \rightarrow B[2]$; $0 \rightarrow B[1]$; $2 \rightarrow B[4]$; $0 \rightarrow B[0]$; $6 \rightarrow B[6]$.

Output: $B = [0, 0, 1, 1, 2, 3, 6, 6]$

Q6. Sort the following dataset using Counting Sort, showing every step:

$$A = \{6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2\}$$

(11 marks)

[CSE 203 | 2011–12]

Answer

$A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$, $n = 11$, range $k = 6$ (values 0–6).

Step 1 — Count occurrences ($C[i]$ = number of times i appears in A):

Index i	0	1	2	3	4	5	6
$C[i]$	2	2	2	2	1	0	2

Step 2 — Cumulate ($C[i] += C[i - 1]$, so $C[i]$ = number of elements $\leq i$):

Index i	0	1	2	3	4	5	6
$C[i]$	2	4	6	8	9	9	11

Step 3 — Build output B (scan A right-to-left for stability):

x (from A , R→L)	$C[x]$ before	Place at $B[]$	$C[x]$ after
2	6	$B[5]$	5
3	8	$B[7]$	7
1	4	$B[3]$	3
6	11	$B[10]$	10
4	9	$B[8]$	8
3	7	$B[6]$	6
1	3	$B[2]$	2
0	2	$B[1]$	1
2	5	$B[4]$	4
0	1	$B[0]$	0
6	10	$B[9]$	9

Output: $B = [0, 0, 1, 1, 2, 2, 3, 3, 4, 6, 6]$

Complexity Analysis. Step 1: $\Theta(n)$. Step 2: $\Theta(k)$. Step 3: $\Theta(n)$. Total: $\Theta(n + k)$. For constant k this is $\Theta(n)$.

Q7. “An important property of Counting Sort is that it is stable” — explain. **(4 marks)**
[CSE 203 | 2011–12]

Answer

A sorting algorithm is **stable** if two elements with equal keys appear in the same relative order in the output as in the input.

Why Counting Sort is stable. In Step 3, the output array is built by scanning the input A from *right to left*. For each element $x = A[i]$, it is placed at position $C[x]-1$, and then $C[x]$ is decremented. Consider two equal elements $A[p] = A[q] = v$ with $p < q$ (i.e., $A[q]$ appears later in the input). Since we scan right to left:

- $A[q] = v$ is processed first and placed at the *higher* available position for value v .
- $A[p] = v$ is processed next and placed at a *lower* position.

Thus $A[p]$ ends up to the left of $A[q]$ in B , preserving their original relative order. Stability is a direct consequence of the right-to-left scan combined with the decrementing counter.

Remark. Stability is critical when Counting Sort is used as a subroutine inside Radix Sort. Without stability, earlier passes' ordering would be destroyed by later passes, producing an incorrect result.

BUET · CSE 105

DSA

Lower Bound Theory

Comparison Sort Lower Bound, Decision Trees

T14 — Lower Bound Theory

Q1. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. **(10 marks)** [CSE 105 | 2023–24, 2022–23, 2022–23, CSE 203 | 2016–17, 2014–15, 2013–14, CSE 207 | 2009–10]

Answer: Lower Bound of Comparison-Based Sorting

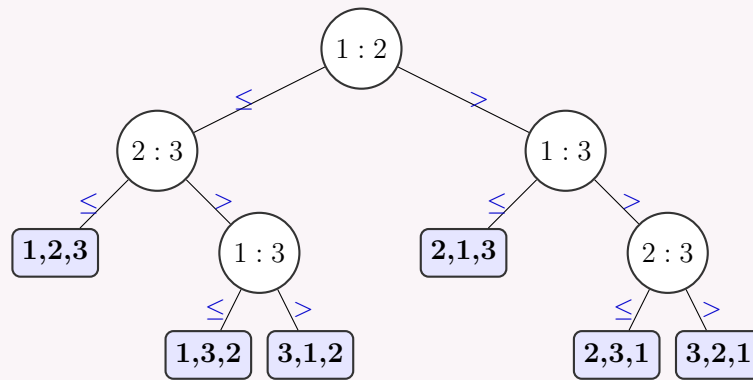
1. The Decision-Tree Model

Any comparison-based sorting algorithm can be visualized as a *decision tree*.

- **Internal Nodes:** Represent a comparison between two elements, $a_i \leq a_j$.
- **Branches:** Represent the outcomes ($\leq$ or $>$) of the comparison.
- **Leaf Nodes:** Represent the final sorted permutation of the input elements.

Example: Decision Tree for $n = 3$ elements (a_1, a_2, a_3)

For 3 elements, there are $3! = 6$ possible permutations. Here is the decision tree showing how the comparisons lead to all 6 possible sorted orders:



2. Lower Bound Argument

Step 1: Count the number of leaves.

An array of size n can be arranged in $n!$ different ways. For the sorting algorithm to be correct, it must be able to output any of these $n!$ permutations. Thus, the decision tree must have at least $L \geq n!$ reachable leaf nodes.

Step 2: Bound the height of the tree.

The longest path from the root to a leaf represents the worst-case number of comparisons. Let h be the height of the tree. Since it's a binary tree, a tree of height h has at most 2^h leaves. Therefore:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

Step 3: Bound $\log_2(n!)$ using Stirling's Approximation.

Stirling's approximation gives an asymptotic estimate for factorials:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Taking the base-2 logarithm of both sides:

$$\begin{aligned}\log_2(n!) &\approx \log_2\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) \\ &= \log_2(\sqrt{2\pi n}) + \log_2\left(\frac{n}{e}\right)^n \\ &= \frac{1}{2}\log_2(2\pi n) + n\log_2(n) - n\log_2(e)\end{aligned}$$

For very large values of n , the term $n\log_2(n)$ dominates all other terms (like $n\log_2(e)$ and the square root term). By ignoring the lower-order terms, we get:

$$\log_2(n!) = \Theta(n \log n)$$

Since $h \geq \log_2(n!)$, this establishes a strict lower bound.

Conclusion:

The worst-case number of comparisons, represented by the tree height h , is mathematically bounded by:

$$h \geq \Omega(n \log n)$$

Hence, any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons in the worst case. $\square$

Q2. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. Describe a sorting algorithm that achieves this lower bound. **(7+8 marks)** [CSE 203 | 2016–17]

Answer

Part (a) — Lower bound. (Decision tree proof as in Q1 above.) Any comparison sort has worst-case complexity $\Omega(n \log n)$. $\square$

Part (b) — Algorithm achieving the bound: Merge Sort.

Algorithm 137 MERGESORT($A, low, high$)

```

1: if  $low < high$  then
2:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
3:   MERGESORT( $A, low, mid$ )
4:   MERGESORT( $A, mid + 1, high$ )
5:   MERGE( $A, low, mid, high$ )
6: end if
```

Algorithm 138 MERGE($A, low, mid, high$)

```

1: Copy  $A[low .. mid]$  into  $L$ ,  $A[mid + 1 .. high]$  into  $R$ 
2:  $i \leftarrow 1$ ;  $j \leftarrow 1$ ;  $k \leftarrow low$ 
3: while  $i \leq |L|$  and  $j \leq |R|$  do
4:   if  $L[i] \leq R[j]$  then
5:      $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
6:   else
7:      $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
8:   end if
9:    $k \leftarrow k + 1$ 
10: end while
11: Copy remaining elements of  $L$  or  $R$  into  $A$ 

```

Worst-case analysis.

$$T(n) = 2T(n/2) + \Theta(n) \implies T(n) = \Theta(n \log n) = O(n \log n).$$

Since $O(n \log n) = \Omega(n \log n)$, Merge Sort is *asymptotically optimal* among comparison-based sorts.

Properties: Stable, not in-place ($O(n)$ auxiliary space), guaranteed $\Theta(n \log n)$ in all cases.

BUET · CSE 105

DSA

Data Structures for Set Operations

Union-Find, Disjoint Sets, Path Compression, Union by Rank

T15 — Data Structures for Set Operations

- Q1.** Consider a Union-Find data structure on a set of n elements, where the UNION operation keeps the name of the largest set. Analyze the run-time complexity of any sequence of k UNION operations. **(10 marks)** [CSE 105 | 2023–24, 2022–23]

Answer

Setup. We use an *array implementation*: each element x stores a pointer to its set representative, and each set stores its size. $\text{UNION}(A, B)$ merges the smaller set into the larger (weighted union / union-by-size), keeping the name of the larger set by updating the pointers of the smaller set's elements.

Key lemma. Each element changes its representative at most $\lfloor \log_2 n \rfloor$ times over all union operations.

Proof. Each time an element x 's representative changes, x moves from set S_1 to set S_2 where $|S_2| \geq |S_1|$ (since we merge the smaller into the larger). So after each such move, the new set containing x at least doubles in size ($|S_1 \cup S_2| \geq 2|S_1|$). Starting from a set of size 1 and doubling each time, the set can contain x with a new representative at most $\log_2 n$ times before the set size reaches n . $\square$

Total cost of k Union operations.

Each UNION itself costs $O(1)$ to compare sizes and update the size of the new set, but updating representative pointers costs 1 per element moved.

For k UNION operations, we involve at most $m = \min(n, 2k)$ distinct elements (since each union connects at most two disjoint components). By our lemma, each of these m elements can have its pointer updated at most $\log_2 m$ times.

Therefore, the maximum number of pointer updates is bounded by:

$$\text{Total pointer updates} \leq m \log_2 m \leq 2k \log_2(2k)$$

Ignoring the constants, the total time for k unions is:

$$T(k \text{ unions}) = O(k \log k).$$

Complexity Analysis. Any sequence of k UNION operations costs $O(k \log k)$ total. The per-operation amortized cost is $O(\log k)$.

- Q2.** Write an algorithm to detect a cycle in an undirected graph using a disjoint-set data structure. Mention the cases where this algorithm performs more efficiently than BFS- or DFS-based approaches. **(10+5 marks)** [CSE 203 | 2019–20]

Answer**Algorithm 139** DETECTCYCLE($G = (V, E)$)

```

1: for each  $v \in V$  do
2:   MAKESET( $v$ )
3: end for
4: for each edge  $(u, v) \in E$  do
5:    $r_u \leftarrow \text{FIND}(u)$ 
6:    $r_v \leftarrow \text{FIND}(v)$ 
7:   if  $r_u = r_v$  then
8:     return true                                ▷ cycle detected
9:   end if
10:  UNION( $r_u, r_v$ )
11: end for
12: return false                                ▷ no cycle found

```

Correctness. Initially each vertex is its own set. As edges are processed, UNION merges the sets of endpoints. If both endpoints of edge (u, v) are already in the same set ($\text{Find}(u) = \text{Find}(v)$), then a path between u and v already exists, so adding this edge creates a cycle.

Complexity. With union-by-rank and path compression: $O(E \alpha(V)) \approx O(E)$ in practice.

When Union-Find outperforms BFS/DFS.

- **Sparse graphs with online edge addition.** When edges are given one at a time (streaming), Union-Find detects a cycle as soon as the offending edge arrives, in $O(\alpha(n))$ per edge. BFS/DFS would need to be re-run from scratch (or maintained with extra bookkeeping).
- **Minimum Spanning Tree construction** (Kruskal's algorithm): edges are processed in sorted order; Union-Find naturally tracks connected components as edges are added without building an explicit adjacency list.
- **Memory efficiency.** Union-Find uses $O(V)$ space (just the parent/rank arrays) vs. BFS/DFS which additionally need an adjacency list $O(V + E)$ and a visited array.

Q3. Explain, with examples, the union-by-rank and path-compression heuristics used in the UNION and FIND operations of disjoint-set data structures. **(10 marks)** [CSE 203 | 2018–19]

Answer

Union-by-Rank. Each node x stores a *rank* (an upper bound on the height of the subtree rooted at x). UNION always attaches the tree of smaller rank under the root

of larger rank. If ranks are equal, either can go under the other and the surviving root's rank is incremented by 1.

Effect. Trees remain shallow: a tree of rank r contains at least 2^r nodes, so rank is $O(\log n)$.

Example. Union($\{1\}, \{2\}$): both rank 0; make 2 child of 1, rank(1) becomes 1. Then Union($\{1, 2\}, \{3\}$): rank(1)=1 > rank(3)=0; attach 3 under 1. Root 1 retains rank 1.

Path Compression. During FIND(x), after locating the root r of x 's tree, all nodes on the path from x to r are made direct children of r .

Algorithm 140 FIND(x) — with path compression

```

1: if parent[x] ≠ x then
2:   parent[x] ← FIND(parent[x])
3: end if
4: return parent[x]
```

Effect. Future FIND operations on x or nodes on the path cost $O(1)$.

Example. Before: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (root). FIND(1) returns 4 and sets parent[1]=4, parent[2]=4, parent[3]=4.

Combined complexity. With both heuristics, any sequence of m FIND and UNION operations on n elements costs $O(m \alpha(n))$, where α is the extremely slowly growing inverse Ackermann function (effectively ≤ 4 for all practical n).

- Q4.** We have a Union-Find data structure for a set S of size n , where unions keep the name of the larger set. Prove that for the array implementation, any sequence of k UNION operations takes at most $O(k \log k)$ time. Also prove that for the pointer-based implementation, a FIND operation takes $O(\log n)$ time. **(15 marks)** [CSE 207 | 2011–12]

Answer

Part A — Array implementation: k unions cost $O(k \log k)$. See Q1 answer

Part B — Pointer-based implementation: Find costs $O(\log n)$.

Structure. Sets are linked lists or trees; each element has a pointer to its representative. UNION by size: the smaller list/tree is merged into the larger, updating all its pointers.

Claim. With union-by-size (weighted union), any tree rooted at representative r has height at most $\lfloor \log_2 n \rfloor$.

Proof. By induction on the number of union operations. Initially all trees have height 0. When two trees of heights $h_1 \leq h_2$ are merged (smaller under larger), the new height is $\max(h_2, h_1 + 1)$. The smaller tree had $\leq$ half the nodes of the merged tree, so the node count at least doubles whenever height increases by 1. A tree of height h has $\geq 2^h$ nodes $\leq n$, giving $h \leq \log_2 n$. $\square$

Since $\text{FIND}(x)$ follows parent pointers to the root, it traverses a path of length $\leq \log_2 n$, costing $O(\log n)$.

Q5. What is the Disjoint Set forest implementation? Write short notes on path compression. Write pseudocode for Disjoint Set forest with path compression. **(3+3+4 marks)** [CSE 207 | 2009–10]

Answer

Disjoint Set Forest. Each disjoint set is stored as a rooted tree. Every node has a **parent** pointer; the root is its own parent. Each node optionally stores a **rank** (for union-by-rank). **UNION** links one tree's root under the other; **FIND** traverses to the root.

Path Compression. When $\text{FIND}(x)$ traverses the path from x to the root r , it simultaneously sets the parent of every node on the path directly to r . This “flattens” the tree so that subsequent finds on any of these nodes cost $O(1)$. Path compression does not change ranks. It is a *self-adjusting* technique: the structure improves with use.

Algorithm 141 MAKESET(x)

```
1:  $\text{parent}[x] \leftarrow x$ 
2:  $\text{rank}[x] \leftarrow 0$ 
```

Algorithm 142 FIND(x) — with path compression

```
1: if  $\text{parent}[x] \neq x$  then
2:    $\text{parent}[x] \leftarrow \text{FIND}(\text{parent}[x])$ 
3: end if
4: return  $\text{parent}[x]$ 
```

Algorithm 143 UNION(x, y) — union by rank

```
1:  $r_x \leftarrow \text{FIND}(x)$ ;  $r_y \leftarrow \text{FIND}(y)$ 
2: if  $r_x = r_y$  then
3:   return
4: end if
5: if  $\text{rank}[r_x] < \text{rank}[r_y]$  then
6:    $\text{SWAP}(r_x, r_y)$ 
7: end if
8:  $\text{parent}[r_y] \leftarrow r_x$ 
9: if  $\text{rank}[r_x] = \text{rank}[r_y]$  then
10:   $\text{rank}[r_x] \leftarrow \text{rank}[r_x] + 1$ 
11: end if
```

Complexity Analysis. With both path compression and union-by-rank, a sequence of m operations (MAKESET, UNION, FIND) on n elements takes $O(m\alpha(n))$ total time, where $\alpha(n) \leq 4$ for all feasible n .

BUET · CSE 105

DSA

Balanced BSTs: AVL & Red-Black Trees

AVL Rotations, Red-Black Properties, Insertions, Deletions

T16 — Balanced BSTs: AVL & Red-Black Trees

Q1. What rotations are required to balance an AVL tree? How does a rotation balance an AVL tree? What is the maximum height of an AVL tree having 10 nodes? **(15 marks)** [CSE 207 | 2018–19]

Answer — AVL Tree Rotations and Height

1. Rotations required to balance an AVL tree: An AVL tree uses four specific types of rotations to restore balance when a node's balance factor ($\text{Height}_{\text{left}} - \text{Height}_{\text{right}}$) becomes less than -1 or greater than $+1$ after an insertion or deletion:

- **LL (Left-Left) Rotation:** A single right rotation. Used when a new node is inserted into the left subtree of the left child.
- **RR (Right-Right) Rotation:** A single left rotation. Used when a new node is inserted into the right subtree of the right child.
- **LR (Left-Right) Rotation:** A double rotation (Left rotation on the child, then Right rotation on the parent). Used when a node is inserted into the right subtree of the left child.
- **RL (Right-Left) Rotation:** A double rotation (Right rotation on the child, then Left rotation on the parent). Used when a node is inserted into the left subtree of the right child.

2. How a rotation balances an AVL tree: A rotation changes the structure of the tree by moving some subtrees up one level and others down one level. This redistributes the height between the left and right sides of the unbalanced node, bringing the balance factor back to $\{-1, 0, +1\}$. Crucially, a rotation strictly maintains the Binary Search Tree (BST) property (in-order traversal remains perfectly sorted) by simply shifting the parent-child relationships rather than arbitrarily moving data.

3. Maximum height of an AVL tree with 10 nodes: Let $N(h)$ be the minimum number of nodes required to construct an AVL tree of height h (where height is the number of edges on the longest path). The recurrence relation for the sparsest AVL tree (Fibonacci tree) is:

$$N(h) = N(h-1) + N(h-2) + 1$$

We can calculate the minimum nodes for the first few heights:

- $h = 0 \Rightarrow N(0) = 1$
- $h = 1 \Rightarrow N(1) = 2$
- $h = 2 \Rightarrow N(2) = 4$
- $h = 3 \Rightarrow N(3) = 7$
- $h = 4 \Rightarrow N(4) = 12$

Since an AVL tree of height 4 strictly requires at least 12 nodes, a tree with exactly 10 nodes cannot reach height 4. It has enough nodes to exceed height 2 (which requires 4 nodes) and safely fulfill the requirement for height 3 (which requires 7 nodes).

Therefore, the maximum possible height (in edges) for an AVL tree with 10 nodes is **3**.

Q2. When inserting a node into a red-black tree, we initially set its color to red. Why not black? Would inserting a new node and then immediately deleting it change the tree? Explain with an example. What is the ratio between the longest and shortest path lengths in a red-black tree? **(15 marks)** [CSE 207 | 2018–19]

Answer — Red-Black Tree Properties

1. Why initially set the color to red? In a Red-Black tree, every path from a node to its descendant NIL nodes must contain the exact same number of black nodes (the "black-depth" property). If we inserted a new node as **black**, it would instantly violate this rule on that specific path, and fixing a black-depth violation requires global adjustments across the entire tree. By inserting the node as **red**, we guarantee that the black-depth property is preserved. The only rule we might violate is having two consecutive red nodes (the "red-red" property), which is a local violation that can be fixed much more easily with a quick recoloring or a rotation.

2. Would inserting a node and immediately deleting it change the tree? **Yes**, it can fundamentally change the coloring (and sometimes the structure) of the tree. **Example:** Suppose we have a perfectly balanced valid Red-Black Tree with 3 nodes: Root is **Black**(10), Left child is **Red**(5), Right child is **Red**(15).

- (a) **Insert 12:** It becomes the left child of 15 and is colored **Red**. This creates a Red-Red violation (15 and 12). Since the uncle (5) is Red, the fix is to *recolor*: push Black down. Nodes 5 and 15 become **Black**, and Root(10) becomes **Red** (then forces itself back to **Black** as it is the root).
- (b) **Delete 12:** The node 12 is a Red leaf, so deleting it requires no structural or color fixes.

Result: The final tree has the exact same structural shape and keys as the initial tree, but the colors are now: Root is **Black**(10), Left child is **Black**(5), Right child is **Black**(15). The tree has permanently changed.

3. Ratio between the longest and shortest path lengths: The shortest possible path from the root to a leaf consists entirely of black nodes. The longest possible path strictly alternates between black and red nodes (since two red nodes cannot be adjacent). Therefore, the longest path can have at most one red node for every black node. Thus, the maximum path length is at most twice the minimum path length, making the ratio ≤ 2 .

Q3. Write down an algorithm for constructing a mincost arborescence in a digraph from a given root vertex r . Apply the algorithm to the following digraph, showing every

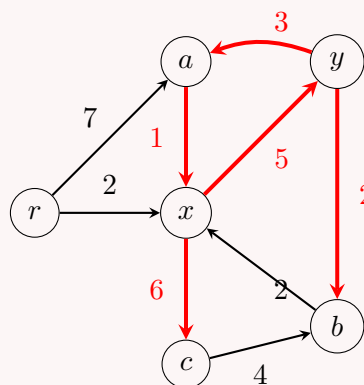
step in separate diagrams: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4), bx(2), yb(2)$ (weights in parentheses). **(15 marks)** [CSE 203 | 2017–18]

Answer — Min-Cost Arborescence (Chu-Liu / Edmonds')

Algorithm Steps (Chu-Liu / Edmonds' Algorithm):

- (a) Discard all edges entering the root r .
- (b) For every vertex $v \neq r$, select the incoming edge with the minimum weight. Let this set of edges be E^* .
- (c) If the edges in E^* do not form any cycles, then E^* forms the minimum cost arborescence.
- (d) If E^* contains a cycle C :
 - Contract the vertices in C into a single super-node S .
 - For any edge (u, v) entering the cycle (where $v \in C$ and $u \notin C$), modify its weight:
 $w'(u, S) = w(u, v) - w(\text{min-weight edge entering } v \text{ that is part of } C)$.
 - Edges leaving the cycle and internal cycle edges remain structurally tracked.
- (e) Recursively apply the algorithm on the new contracted graph. Once the arborescence of the contracted graph is found, expand S by removing the single edge inside C that resolves the cycle, integrating it with the incoming edge chosen for S .

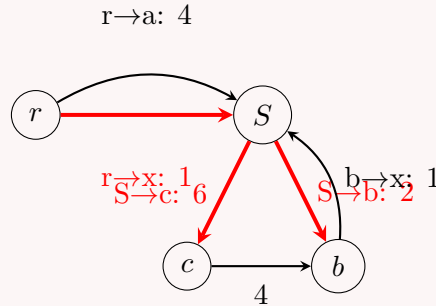
Application to the Given Digraph: Edges: $r \xrightarrow{7} a$, $r \xrightarrow{2} x$, $a \xrightarrow{1} x$, $x \xrightarrow{5} y$, $y \xrightarrow{3} a$, $x \xrightarrow{6} c$, $c \xrightarrow{4} b$, $b \xrightarrow{2} x$, $y \xrightarrow{2} b$.



Step 1: Pick min-incoming edges. Cycle $a \rightarrow x \rightarrow y \rightarrow a$ found.

Step 2: Contraction. We contract $\{a, x, y\}$ into S . The cycle edges have weights 1, 5, 3. We adjust the weights of edges entering S :

- $r \rightarrow a(7)$ enters a . Inside cycle, $y \rightarrow a$ is 3. New weight = $7 - 3 = 4$.
- $r \rightarrow x(2)$ enters x . Inside cycle, $a \rightarrow x$ is 1. New weight = $2 - 1 = 1$.
- $b \rightarrow x(2)$ enters x . Inside cycle, $a \rightarrow x$ is 1. New weight = $2 - 1 = 1$.



Step 2: Contract to S. Pick new min-edges. No cycles!

Step 3: Expansion. We expand S . The edge entering S in our valid tree is $r \rightarrow x$ (originally 2). Because it enters x , we must break the cycle at the edge that was previously entering x (which is $a \rightarrow x$ with weight 1). We drop $a \rightarrow x$ and keep the rest of the cycle.

Final Edges of Arborescence: $r \xrightarrow{2} x$, $x \xrightarrow{5} y$, $y \xrightarrow{3} a$, $x \xrightarrow{6} c$, $y \xrightarrow{2} b$.

Total Minimum Cost: $2 + 5 + 3 + 6 + 2 = 18$.

Q4. Discuss the Presumed Optimal Solution for the Multipeg Tower of Hanoi Problem with its properties. Solve the problem for $(n, p) = (395, 8)$. **(5+10 marks)** [CSE 203 | 2017–18]

Answer — Multipeg Tower of Hanoi (Frame-Stewart)

1. Presumed Optimal Solution (Frame-Stewart Algorithm) For the Multi-peg Tower of Hanoi problem with n disks and p pegs ($p \geq 3$), the Frame-Stewart algorithm defines the presumed optimal number of moves, $M(n, p)$. The algorithm partitions the disks recursively:

- Choose an integer k ($1 \leq k < n$).
- Move the top k disks to an intermediate peg using all p pegs.
- Move the remaining $n - k$ disks to the destination peg using the remaining $p - 1$ pegs (since one peg is occupied by the k disks).
- Move the top k disks from the intermediate peg to the destination peg using all p pegs.

Property: The algorithm aims to minimize the moves by finding the optimal k .

The recurrence relation is:

$$M(n, p) = \min_{1 \leq k < n} \{2M(k, p) + M(n - k, p - 1)\}$$

A known property of this optimal sequence is that the step difference $\Delta(n, p) = M(n, p) - M(n - 1, p)$ takes the value 2^i exactly $\binom{i+p-3}{p-3}$ times, in increasing order of $i \geq 0$.

2. Solving for $(n, p) = (395, 8)$ Here, $p = 8$, so the binomial coefficient is $\binom{i+8-3}{8-3} = \binom{i+5}{5}$. We sum the number of disks we can move for each i until we reach $n = 395$.

- $i = 0$: $\binom{5}{5} = 1$ disk. $\Delta = 2^0 = 1$. Total disks = 1.
- $i = 1$: $\binom{6}{5} = 6$ disks. $\Delta = 2^1 = 2$. Total disks = $1 + 6 = 7$.
- $i = 2$: $\binom{7}{5} = 21$ disks. $\Delta = 2^2 = 4$. Total disks = $7 + 21 = 28$.
- $i = 3$: $\binom{8}{5} = 56$ disks. $\Delta = 2^3 = 8$. Total disks = $28 + 56 = 84$.
- $i = 4$: $\binom{9}{5} = 126$ disks. $\Delta = 2^4 = 16$. Total disks = $84 + 126 = 210$.
- $i = 5$: $\binom{10}{5} = 252$ disks. $\Delta = 2^5 = 32$. Total disks = $210 + 252 = 462$.

Since 395 falls between 210 and 462, the $i = 5$ phase handles the remaining disks.

Disks accounted for fully (up to $i = 4$): 210 disks.

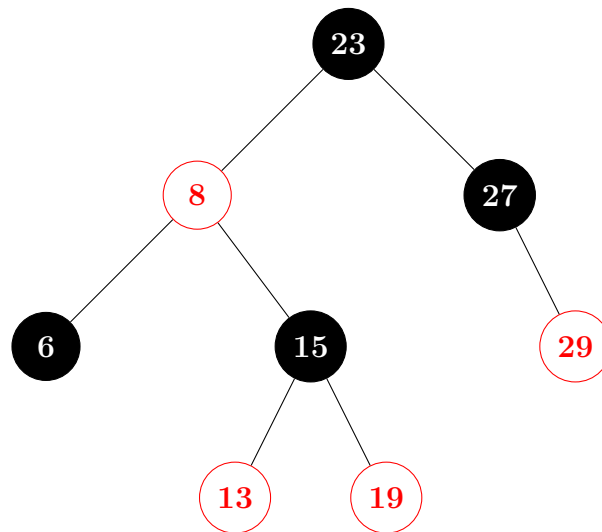
Remaining disks to reach 395: $395 - 210 = 185$ disks.

Now, we calculate the total minimum moves:

$$\begin{aligned}
 M(395, 8) &= \sum_{i=0}^4 \left(2^i \times \binom{i+5}{5} \right) + (185 \times 2^5) \\
 &= (1 \times 1) + (2 \times 6) + (4 \times 21) + (8 \times 56) + (16 \times 126) + (185 \times 32) \\
 &= 1 + 12 + 84 + 448 + 2016 + 5920 \\
 &= 2561 + 5920 \\
 &= \mathbf{8481}
 \end{aligned}$$

The minimum number of moves to solve the problem for 395 disks and 8 pegs is **8,481**.

Q5. [Red-Black tree given] Insert 11 into the given Red-Black tree, showing each operation (red nodes marked R, black nodes marked B). **(15 marks)** [CSE 207 | 2017–18]



Answer — Red-Black Tree Insertion

Step 1: Standard BST Insertion.

We insert 11 as a **Red** node. By BST rules, it becomes the left child of 13. This creates a **Red-Red violation** between 13 and 11.

Step 2: Recolor (Case 1).

Look at the newly inserted node (11). Its parent is 13 (Red) and its uncle is 19 (Red). Because the **uncle is Red**, we flip the colors:

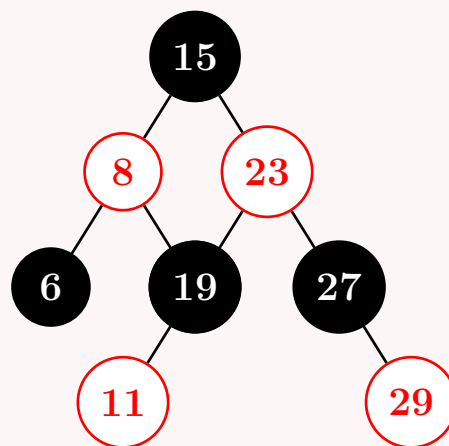
- Parent (13) and Uncle (19) become **Black**.
- Grandparent (15) becomes **Red**.

Now, the violation moves up to node 15. We have a new **Red-Red violation** between 8 and 15.

Step 3: Rotations (Case 2 & 3).

Look at node 15. Its parent is 8 (Red) and its uncle is 27 (Black). Since the **uncle is Black** and the node forms a **Left-Right (LR) 'triangle'** (8 is a left child, 15 is a right child), we need a double rotation:

- Left-Rotate on Parent (8):** 15 moves up, 8 becomes the left child of 15. Node 13 (15's old left child) becomes 8's right child. The violation is now a Left-Left (LL) 'line' between 15 and 8.
- Right-Rotate on Grandparent (23):** 15 moves up to become the root, and 23 becomes its right child. Node 19 (15's old right child) becomes 23's left child.
- Final Recolor:** The new root of this subtree (15) becomes **Black**, and the old root (23) becomes **Red**.



Final Tree after Insert(11), Rotations, and Recoloring

Q6. Mention one advantage of AVL trees over red-black trees, and one advantage of red-black trees over AVL trees. (4 marks) [CSE 207 | 2017–18]

Answer — AVL vs. Red-Black Trees

- **Advantage of AVL over Red-Black Tree:**

AVL trees are more strictly balanced (the height difference between subtrees is at most 1). Because they are shallower and more tightly structured, **lookups (searching) are consistently faster** in an AVL tree compared to a Red-Black tree.

- **Advantage of Red-Black over AVL Tree:**

Red-Black trees require fewer rotations during updates because their balancing rules are looser (the longest path can be up to twice the shortest path). This makes **insertions and deletions faster** in scenarios with frequent write operations, which is why Red-Black trees are often preferred in general-purpose libraries (like C++ 'std::map').

Q7. If the i -th and $(i + 2)$ -th Fibonacci numbers are x and y respectively (where $i > 0$), what is the minimum number of nodes in an AVL tree of height $(i - 1)$? (4 marks) [CSE 207 | 2017–18]

Answer — Fibonacci and AVL Tree Height

Let $N(h)$ be the minimum number of nodes required to construct an AVL tree of height h . The recurrence relation for the sparsest possible AVL tree is defined as:

$$N(h) = N(h-1) + N(h-2) + 1$$

If we add 1 to both sides, we get: $N(h) + 1 = (N(h-1) + 1) + (N(h-2) + 1)$

This precisely matches the recurrence relation of the Fibonacci sequence $F_n = F_{n-1} + F_{n-2}$. By evaluating the base cases ($N(0) = 1, N(1) = 2$), we can map this directly to the Fibonacci sequence:

$$N(h) + 1 = F_{h+3} \implies N(h) = F_{h+3} - 1$$

We are asked for the minimum number of nodes in an AVL tree of height $h = i - 1$:

$$N(i-1) = F_{(i-1)+3} - 1 = F_{i+2} - 1$$

We are given that the $(i+2)$ -th Fibonacci number is y (i.e., $F_{i+2} = y$). Substituting this into our equation:

$$N(i-1) = y - 1$$

Answer: The minimum number of nodes in an AVL tree of height $(i-1)$ is $y - 1$.

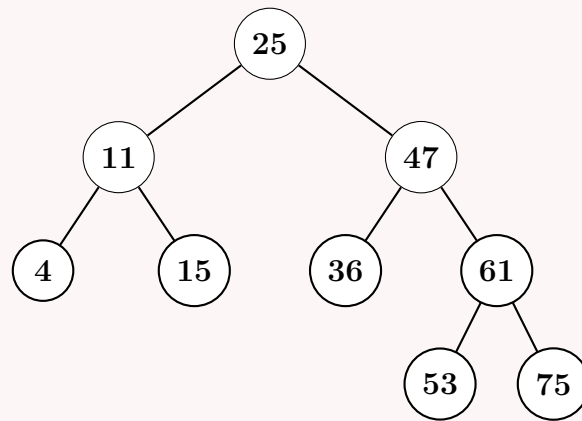
Q8. What is an AVL tree? Draw the AVL tree that results after successively inserting the keys 11, 4, 15, 25, 36, 47, 61, 53, 75 into an initially empty AVL tree. (10 marks)
[CSE 203 | 2015–16]

Answer — AVL Tree Construction**Definition:**

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference in heights between the left and right subtrees (the balance factor) of *every* node is at most 1 ($\in \{-1, 0, +1\}$). If an insertion or deletion causes the balance factor of any node to fall outside this range, rotations are performed to restore balance.

Insertion Trace (11, 4, 15, 25, 36, 47, 61, 53, 75):

- 11, 4, 15, 25:** Inserted normally. Tree: 11(L:4, R:15(R:25)).
- 36:** Insert right of 25. Causes RR imbalance at 15. **Left rotate 15.** Subtree becomes 25(L:15, R:36).
- 47:** Insert right of 36. Causes RR imbalance at root 11. **Left rotate 11.** 25 becomes the new root. 11 becomes its left child, taking 15 as its right child.
- 61:** Insert right of 47. Causes RR imbalance at 36. **Left rotate 36.** Subtree becomes 47(L:36, R:61).
- 53:** Insert left of 61. No rotations needed.
- 75:** Insert right of 61. No rotations needed. The final tree perfectly balances itself out.



Q9. Show that the height of a red-black tree T storing n items is $O(\log n)$. Explain, with examples, the three cases that may arise for inserting a node in a red-black tree. **(15 marks)** [CSE 203 | 2015–16]

Answer — Red-Black Tree Height and Insertion Cases

1. Proof that Height is $O(\log n)$:

Let h be the height of the Red-Black tree and $bh(x)$ be the black-height of a node x (number of black nodes on the path from x to a leaf, not including x).

- **Lemma:** A subtree rooted at any node x contains at least $2^{bh(x)} - 1$ internal nodes. (Proved by induction).
- By the property of Red-Black trees, at least half the nodes on any simple path from the root to a leaf (excluding the root) must be black. Therefore, the black-height of the root must be at least $h/2$.
- Applying the lemma to the root: $n \geq 2^{h/2} - 1$.
- Rearranging for h :

$$n + 1 \geq 2^{h/2} \implies \log_2(n + 1) \geq h/2 \implies h \leq 2\log_2(n + 1)$$

Thus, $h = O(\log n)$. □

2. Three Cases for Insertion:

When inserting a new node N , it is initially colored **Red**. If its Parent (P) is also Red, we have a violation. The fix depends on the color of N 's Uncle (U). Let G be the Grandparent.

- **Case 1: Uncle (U) is Red.**

Action: Recolor P and U to Black, and G to Red. Then move the pointer up to G to check for further violations.

Example: Inserting 12 under Red 15, with Red Uncle 5. Both 15 and 5 become Black, G becomes Red.

- **Case 2: Uncle (U) is Black, and N forms a "Triangle" (Inner Child).**
Action: Perform a single rotation on P in the opposite direction of N . This transforms the tree into Case 3.
Example: G is 20, P is 10 (Left child), N is 15 (Right child). Left-rotate on 10. Now 15 is the parent of 10, forming a Left-Left line.
- **Case 3: Uncle (U) is Black, and N forms a "Line" (Outer Child).**
Action: Perform a single rotation on G in the opposite direction of N . Then recolor P to Black and G to Red.
Example: G is 20, P is 15 (Left child), N is 10 (Left child). Right-rotate on 20. 15 becomes the root of the subtree (Black), with 10 (Red) and 20 (Red) as children.

Q10. Explain, with examples, the single rotation and double rotation operations on AVL trees. (10 marks) [CSE 203 | 2014–15]

Answer — AVL Tree Rotations

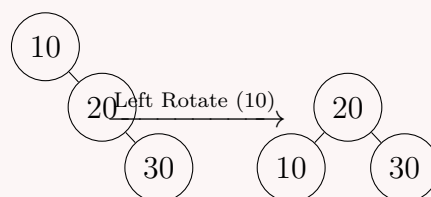
Rotations restore balance in an AVL tree. There are two categories: Single and Double.

1. Single Rotations (LL and RR)

Used when the insertion occurs in the "outer" grandchild (a straight line).

- **LL Imbalance (Right Rotation):** Insertions into the left subtree of the left child.
- **RR Imbalance (Left Rotation):** Insertions into the right subtree of the right child.

Example (RR Imbalance): Insert 10, 20, 30.

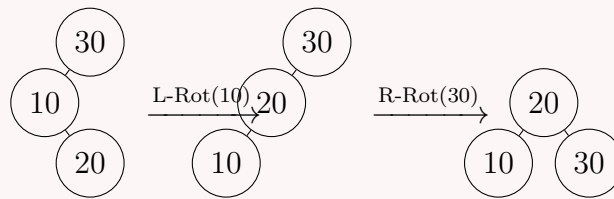


2. Double Rotations (LR and RL)

Used when the insertion occurs in the "inner" grandchild (a zig-zag shape). It requires a rotation on the child, then a rotation on the parent.

- **LR Imbalance (Left-Right Rotation):** Insertions into the right subtree of the left child. Rotate Left on the child, then Right on the parent.

Example (LR Imbalance): Insert 30, 10, 20.



Q11. What is a Multiway Search tree? How do we search a key in a Multiway Search tree? (10 marks) [CSE 203 | 2014–15]

Answer — Multiway Search Tree

1. What is a Multiway Search Tree?

A Multiway Search Tree of order m (or an m -way search tree) is a generalized binary search tree where each node can have up to m children and up to $m - 1$ keys. The properties are:

- Each node contains k keys ($1 \leq k < m$) sorted in ascending order: $K_1 < K_2 < \dots < K_k$.
- Each node has $k + 1$ child pointers: $P_0, P_1, \dots, P_k$.
- All keys in the subtree pointed to by P_0 are less than K_1 .
- All keys in the subtree pointed to by P_i (for $1 \leq i < k$) are between K_i and K_{i+1} .
- All keys in the subtree pointed to by P_k are strictly greater than K_k .

2. How to Search for a key (X):

Searching starts at the root and operates recursively (or iteratively) downwards:

- Compare:** Look at the keys in the current node ($K_1, K_2, \dots, K_k$).
- Match:** If $X == K_i$ for some i , the search is successful. Return the node/record.
- Branch:** If no match is found, determine which child pointer to follow:
 - If $X < K_1$, follow pointer P_0 .
 - If $K_i < X < K_{i+1}$, follow pointer P_i .
 - If $X > K_k$, follow pointer P_k .
- Recurse/Terminate:** If the pointer followed is NULL, the key X does not exist in the tree. Otherwise, repeat the process at the new child node.

Q12. Write the properties of a red-black tree. Show that the height of a red-black tree T storing n items is $O(\log n)$. (10 marks) [CSE 203 | 2014–15]

Answer — Red-Black Tree Properties and Height Proof

1. Properties of a Red-Black Tree: A Red-Black tree is a binary search tree that satisfies the following 5 rules:

- (a) **Color Property:** Every node is colored either Red or Black.
- (b) **Root Property:** The root of the tree is always Black.
- (c) **Leaf Property:** Every leaf (NIL pointer) is considered Black.
- (d) **Red Property:** If a node is Red, then both of its children must be Black. (No two consecutive Red nodes on any path).
- (e) **Black-Depth Property:** For each node, all simple paths from that node to descendant leaves contain the exact same number of Black nodes.

2. Proof that Height $h = O(\log n)$:

Let $bh(x)$ be the number of black nodes on a path from node x to a leaf, excluding x . By induction, we can prove that any subtree rooted at x contains at least $2^{bh(x)} - 1$ internal nodes.

- **Base Case:** Height of x is 0. x is a leaf (NIL). It has 0 internal nodes. $2^0 - 1 = 0$. Holds.
- **Inductive Step:** Consider an internal node x with children. Each child has a black height of either $bh(x)$ (if the child is red) or $bh(x) - 1$ (if the child is black). Thus, each child has at least $2^{bh(x)-1} - 1$ internal nodes.
- The total nodes in the subtree rooted at x is: $(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$.

Since the tree has no consecutive red nodes, at least half of the nodes on any path from the root to a leaf must be black. Thus, the black-height of the root is $\geq h/2$. Substitute into our formula: $n \geq 2^{h/2} - 1 \implies 2^{h/2} \leq n + 1 \implies h/2 \leq \log_2(n + 1) \implies h \leq 2 \log_2(n + 1)$.

Therefore, the height h is strictly $O(\log n)$. □

Q13. Explain, with examples, the four cases that may arise for deleting a node from a red-black tree. **(15 marks)** [CSE 203 | 2014–15]

Answer — Red-Black Tree Deletion Cases

Deletion is complex when a **Black** node is removed, causing a "Double Black" (DB) node (a node missing one black weight on its path). To fix a DB node (N), we examine its Sibling (S) and Parent (P). There are 4 core cases:

Case 1: Sibling S is Red.

Fix: Recolor S to Black and P to Red. Then perform a rotation on P toward the DB node N .

Result: This does not fix the DB, but it pushes the DB down into the tree, converting

it into Case 2, 3, or 4 (since the new sibling of N will now be Black).

Case 2: Sibling S is Black, and both children of S are Black.

Fix: We strip a "Black" unit from both N (the DB node) and S .

- N becomes a normal node (resolving the DB).
- S becomes Red.
- The extra "Black" is pushed up to the Parent P . If P was Red, it just becomes Black, and we are done. If P was Black, P now becomes the new Double Black node, and we recurse.

Case 3: Sibling S is Black, S 's inner child is Red, outer child is Black.

Fix: Recolor S 's inner child to Black and S to Red. Then rotate S in the opposite direction of the inner child.

Result: This forces the Red inner child to become a Red outer child, perfectly converting the tree into Case 4.

Case 4: Sibling S is Black, and S 's outer child is Red.

Fix: This is the terminal case that completely resolves the Double Black violation.

- Recolor S to the color of the Parent P .
- Recolor P to Black.
- Recolor S 's Red outer child to Black.
- Perform a rotation on P toward the Double Black node N .

Result: The DB node absorbs the extra Black weight through the rotation. The tree is now completely balanced and valid.

Q14. What is an AVL tree? Show that the height of an AVL tree T storing n items is $O(\log n)$. (10 marks) [CSE 203 | 2013–14]

Answer

AVL Tree: a BST where for every node the heights of left and right subtrees differ by at most 1 (balance factor $\in \{-1, 0, +1\}$).

Height $O(\log n)$: let $N(h)$ = minimum nodes in height- h AVL tree.

$$N(h) = N(h-1) + N(h-2) + 1, \quad N(0) = 1, \quad N(1) = 2.$$

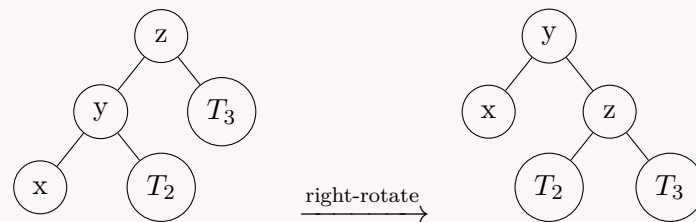
This gives $N(h) \geq \phi^h / \sqrt{5} - 1$ (Fibonacci-like), so

$$n \geq N(h) \geq c\phi^h \Rightarrow h \leq \log_\phi n \approx 1.44 \log_2 n = O(\log n). \quad \square$$

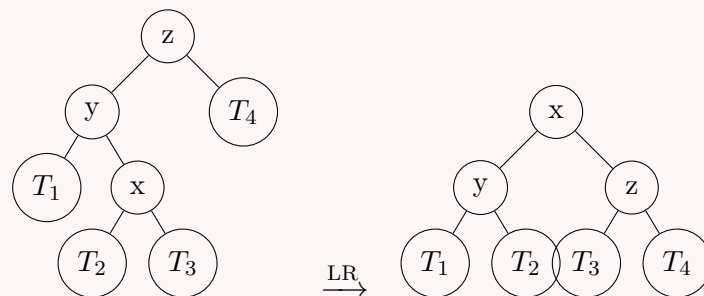
Q15. Explain with examples the single rotation and double rotation operations on AVL trees. (10 marks) [CSE 203 | 2013–14]

Answer

Single Right Rotation (LL case):



Double Left-Right Rotation (LR case):



Both LL/RR (single) and LR/RL (double) restore balance in $O(1)$.

Q16. Write the properties of a red-black tree. Show the red-black trees that result after successively inserting the keys 51, 46, 41, 22, 29, 18 into an initially empty red-black tree. (15 marks) [CSE 203 | 2013–14]

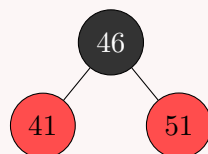
Answer

Properties: (1) Every node is red or black. (2) Root is black. (3) All NIL leaves are black. (4) Red nodes have black children. (5) All root-to-leaf paths have equal black-height.

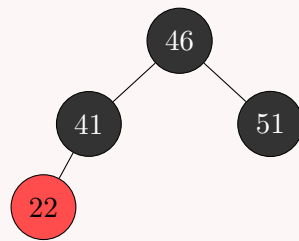
Insert 51: root, color black. [51_B]

Insert 46: red, left of 51. No violation.

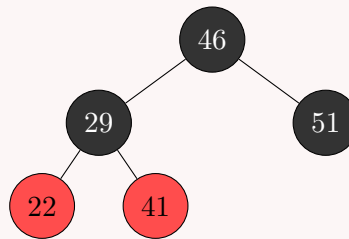
Insert 41: red, left of 46 → LL imbalance (uncle=NIL=black). Right-rotate 51; recolor: 46 black, 51 red.



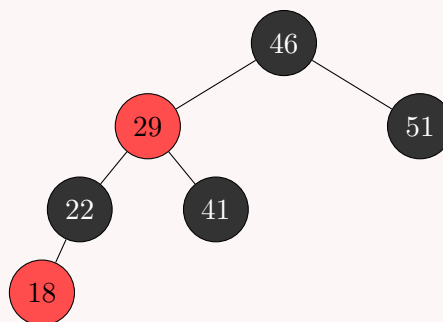
Insert 22: red, left of 41. Uncle=51 (red) → recolor: 41,51→black; 46→red→root→black.



Insert 29: red, right of 22. Uncle=NIL (black) $\rightarrow$ LR case on 41. Left-rotate 22, right-rotate 41. 29 $\rightarrow$ black, 22 and 41 $\rightarrow$ red.



Insert 18: red, left of 22. Uncle=41 (red) $\rightarrow$ recolor: 22,41 $\rightarrow$ black; 29 $\rightarrow$ red. 29's parent 46 is black $\rightarrow$ done.



Black-height = 2 for all root-to-leaf paths. $\checkmark$

Q17. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (377, 8)$. Find $k_{\max}$, N_0 , and n_1 for at least two subproblems. **(10 marks)** [CSE 207 | 2013–14]

Answer — Multi-Peg Tower of Hanoi (377, 8)

1. Total Minimum Moves for $(n, p) = (377, 8)$:

According to the Frame-Stewart algorithm, the optimal step sizes for $p = 8$ pegs follow the sequence $\Delta M = 2^i$, each occurring $\binom{i+p-3}{p-3} = \binom{i+5}{5}$ times. We first calculate the capacity of each block i :

- $i = 0$: $\binom{5}{5} = 1$ disk. (Cumulative: $N_0 = 1$)
- $i = 1$: $\binom{6}{5} = 6$ disks. (Cumulative: $N_1 = 7$)
- $i = 2$: $\binom{7}{5} = 21$ disks. (Cumulative: $N_2 = 28$)
- $i = 3$: $\binom{8}{5} = 56$ disks. (Cumulative: $N_3 = 84$)

- $i = 4$: $\binom{9}{5} = 126$ disks. (Cumulative: $N_4 = 210$)
- $i = 5$: $\binom{10}{5} = 252$ disks. (Cumulative: $N_5 = 462$)

$n = 377$ falls into the $i = 5$ block. The number of fully covered disks is 210. The remaining disks in the $i = 5$ block are $377 - 210 = 167$.

Total Minimum Moves $M(377, 8)$:

$$\begin{aligned}
 M(377, 8) &= \sum_{i=0}^4 \left(2^i \times \binom{i+5}{5} \right) + (167 \times 2^5) \\
 &= (1 \times 1) + (2 \times 6) + (4 \times 21) + (8 \times 56) + (16 \times 126) + (167 \times 32) \\
 &= 1 + 12 + 84 + 448 + 2016 + 5344 \\
 &= \mathbf{7,905}
 \end{aligned}$$

2. Subproblems (n_1 , N_0 , and $k_{\max}$):

In the standard partition notation, s is the active block index, n_1 is the number of disks moved to the intermediate peg using p pegs, and N_0 is the number of disks moved to the destination using $p - 1$ pegs. Here, $k_{\max} = N_0$. The optimal intermediate disks are strictly given by $n_1 = N_{s-2}^{(p)}$.

Subproblem 1: $(n, p) = (377, 8)$

- **Active Block (s):** 5 (since $210 < 377 \leq 462$)
- n_1 : $N_{5-2}^{(8)} = N_3^{(8)} = \mathbf{84}$
- N_0 : $n - n_1 = 377 - 84 = \mathbf{293}$
- $k_{\max}$: $\mathbf{293}$

Subproblem 2: $(n, p) = (84, 8)$ (*Solving for the intermediate n_1 pile*)

- **Active Block (s):** 3 (since $28 < 84 \leq 84$, it perfectly fills block 3)
- n_1 : $N_{3-2}^{(8)} = N_1^{(8)} = \mathbf{7}$
- N_0 : $n - n_1 = 84 - 7 = \mathbf{77}$
- $k_{\max}$: $\mathbf{77}$

Q18. What is an AVL tree? Show that the height of an AVL tree T storing n items is $O(\log n)$. (10 marks) [CSE 203 | 2013–14]

Answer — AVL Tree Definition and Height Proof

1. What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) in which the height difference (balance factor) between the left and right subtrees of *every* node is strictly at most 1 (i.e., $\in \{-1, 0, 1\}$). Whenever an insertion or deletion causes a node's balance factor to exceed this range, structural rotations (LL, RR, LR, or

RL) are performed to restore balance.

2. Proof that Height $h = O(\log n)$:

Let $N(h)$ be the minimum number of nodes required to form an AVL tree of height h . For an AVL tree to be as sparse as possible (minimum nodes for a given height), one subtree must have height $h - 1$ and the other must have height $h - 2$.

This gives the recurrence relation:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

With base cases: $N(0) = 1$ and $N(1) = 2$.

Since $N(h - 1) > N(h - 2)$ for all $h > 1$, we can establish a strict inequality:

$$N(h) > 2 \cdot N(h - 2)$$

Expanding this inequality recursively:

$$\begin{aligned} N(h) &> 2 \cdot N(h - 2) \\ &> 2^2 \cdot N(h - 4) \\ &> 2^3 \cdot N(h - 6) \\ &\dots \\ &> 2^{h/2} \cdot N(0) \end{aligned}$$

Since $N(0) = 1$, we have:

$$N(h) > 2^{h/2}$$

If an AVL tree has n total nodes, then $n \geq N(h)$, which implies:

$$n > 2^{h/2}$$

Taking the base-2 logarithm of both sides:

$$\log_2 n > \frac{h}{2}$$

$$h < 2 \log_2 n$$

Since h is bounded by a constant multiple of $\log_2 n$, the height of an AVL tree is strictly $O(\log n)$. $\square$

Q19. Write the properties of a B-tree. (5 marks)

[CSE 203 | 2013–14]

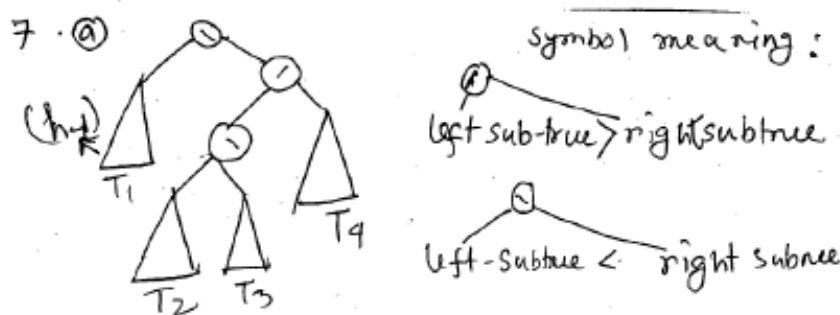
Answer — Properties of a B-Tree

A **B-tree** of order m is a self-balancing m -way search tree designed to work efficiently on magnetic disks and secondary storage. It strictly satisfies the following properties:

1. Maximum Children: Every node can have at most m children.

2. **Minimum Children (Internal Nodes):** Every internal node (except the root) must have at least $\lceil m/2 \rceil$ children. This guarantees the tree remains dense and relatively shallow.
3. **Root Properties:** The root must have at least 2 children if it is not a leaf node.
4. **Key Count Constraints:** An internal node with k children contains exactly $k - 1$ keys. Therefore, a node can contain a maximum of $m - 1$ keys and a minimum of $\lceil m/2 \rceil - 1$ keys.
5. **Perfectly Balanced Leaves:** All leaf nodes must strictly appear on the exact same level (the bottom level).
6. **Sorting and Partitioning:** Keys within a node are kept in ascending order ($K_1 < K_2 < \dots < K_{k-1}$). These keys act as separation bounds to divide the subtrees: all keys in a child's subtree perfectly fall between the corresponding keys of its parent node.

Q20. [AVL tree figure given] Let T be an AVL tree where T_1 has height $h - 1$. Determine the heights of T_2 , T_3 , and T_4 . Discuss what happens when you delete an element from T_1 ; cover all possible cases for deletion in an AVL tree. (4+4+7=15 marks)
[CSE 203 | 2012–13]



Q21. Design an AVL tree by inserting the keys 5, 12, 3, -2, -5, -3, 1 in order. Show all steps. (15 marks) [CSE 203 | 2012–13]

Answer — AVL Tree Design

Insertion Steps:

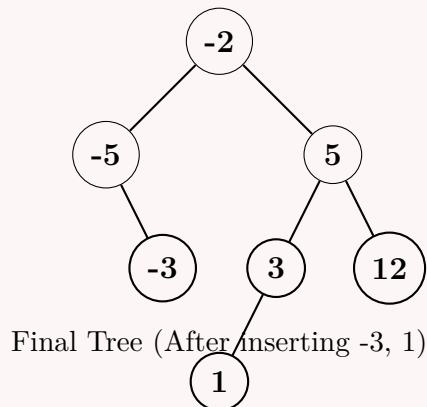
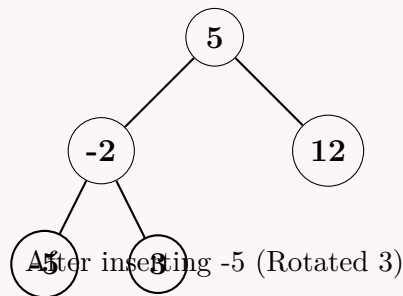
- (a) **Insert 5, 12, 3:** Inserted conventionally. Tree is perfectly balanced.
- (b) **Insert -2:** Inserted as the left child of 3. Balance factors are still valid (within ± 1).
- (c) **Insert -5:** Inserted as the left child of -2.
Imbalance: Node 3 has a balance factor of +2 (Left-Left heavy).
Fix: Perform a **Right Rotation on 3**. Node -2 moves up, -5 remains its left child, and 3 becomes its right child.

(d) **Insert -3:** Inserted as the right child of -5.

Imbalance: The root (5) has a balance factor of +2. The path from 5 is $5 \rightarrow -2 \rightarrow -5$. Since -2's left subtree is taller, this is a Left-Left (LL) imbalance at the root.

Fix: Perform a **Right Rotation on 5**. Node -2 becomes the new root. 5 becomes its right child, taking 3 (former right child of -2) as its left child.

(e) **Insert 1:** Inserted as the left child of 3. The tree remains completely balanced.



Q22. What is the height of an AVL tree with n internal nodes? (5 marks) [CSE 203 | 2012–13]

Answer — Height of an AVL Tree

The height h of an AVL tree with n internal nodes is bounded logarithmically due to its strict structural balancing rules.

While the perfectly balanced binary tree has a height of $\approx \log_2(n)$, the AVL tree allows a maximum height difference of 1 between sibling subtrees. The sparsest possible AVL tree follows a Fibonacci distribution for its nodes.

Mathematically, the height h is strictly bounded by:

$$h < 1.44 \log_2(n + 2) - 0.328$$

Therefore, in asymptotic big-O notation, the height of an AVL tree with n internal nodes is always guaranteed to be:

$$O(\log n)$$

Q23. Write down the properties of a red-black tree. With an example, show the insertion and deletion operations and their costs in a red-black tree. (5+15=20 marks) [CSE 203 | 2012–13]

Answer — Red-Black Tree Properties & Costs**1. Properties of a Red-Black Tree:**

- (a) Every node is colored either **Red** or **Black**.
- (b) The root node is always **Black**.
- (c) Every leaf (NIL pointer) is **Black**.
- (d) If a node is **Red**, both of its children must be **Black** (No two consecutive Red nodes).
- (e) Every simple path from a node to its descendant leaves contains the exact same number of **Black** nodes (Black-Depth property).

2. Operations, Examples, and Costs:

Insertion: Always insert the new node as **Red** to preserve the black-depth property.

- *Example:* Insert 10 (Red) into an empty tree. It becomes Root, so we recolor it to **Black**. Next, insert 5 (Red). Valid. Next, insert 1 (Red). This creates a Red-Red violation (5 is Red, 1 is Red). Since the uncle is NIL (Black) and it's an outer insertion, we right-rotate at 10 and recolor 5 to Black and 10 to Red.
- *Cost:* $O(\log n)$. Finding the insertion point takes $O(\log n)$ time. Fixing violations requires recoloring (which can propagate up the tree taking $O(\log n)$ time) and at most **2 rotations** ($O(1)$ time).

Deletion: Remove the node using standard BST rules. If the removed node or its replacement was **Red**, no properties are violated. If it was **Black**, it creates a "Double Black" node, violating property 5.

- *Example:* In a tree where Root is 5(B), Left is 1(R), Right is 10(B). Delete 10. The right child becomes a Double Black NIL. We must fix this by looking at its sibling (1) and performing recoloring and rotations to redistribute the "blackness" or push it up to the root.
- *Cost:* $O(\log n)$. Finding and removing the node takes $O(\log n)$. Pushing the double-black token up the tree takes at most $O(\log n)$ recoloring steps, and resolving it requires at most **3 rotations** ($O(1)$ time).

Q24. Build a 2-3-4 tree with the keys 12, 5, 6, 10, 15, 8, 13, 17, 11, 14, 4. Then delete elements in the following sequence: 4, 12, 13. Show all steps. **(25 marks)** [CSE 203 | 2012–13]

Answer — 2-3-4 Tree Construction and Deletion

In a 2-3-4 tree, each node can hold 1 to 3 keys. We will use the standard bottom-up B-tree insertion.

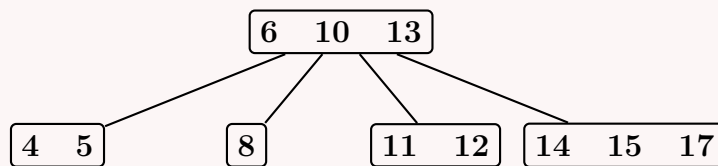
Part 1: Construction

- (a) **12, 5, 6:** Node fits 3 keys $\rightarrow [5, 6, 12]$.
- (b) **10:** Overflows. Split $[5, 6, 10, 12]$. The median 10 (or 6, assuming left-median) goes up. Let's push 6 up. $\rightarrow$ Root $[6]$, Children: $[5]$ and $[10, 12]$.
- (c) **15, 8:** 15 goes to right child $\rightarrow [10, 12, 15]$. 8 goes to right child $\rightarrow$ overflows $[8, 10, 12, 15]$. Split: push 10 up. Root becomes $[6, 10]$. Children: $[5], [8], [12, 15]$.
- (d) **13, 17:** Both go to rightmost child $\rightarrow [12, 13, 15, 17]$. Overflows. Push 13 up. Root: $[6, 10, 13]$. Children: $[5], [8], [12], [15, 17]$.
- (e) **11, 14, 4:**
 - 11 goes between 10 and 13 $\rightarrow$ Child $[12]$ becomes $[11, 12]$.
 - 14 goes past 13 $\rightarrow$ Child $[15, 17]$ becomes $[14, 15, 17]$.
 - 4 goes before 6 $\rightarrow$ Child $[5]$ becomes $[4, 5]$.

Final Constructed Tree:

Root: $[6, 10, 13]$

Children: $[4, 5], [8], [11, 12], [14, 15, 17]$



Part 2: Deletion Sequence (4, 12, 13)

In an order-4 B-tree, a node must have at least $\lceil 4/2 \rceil - 1 = 1$ key.

1. Delete 4:

4 is in the leaf node $[4, 5]$. Since the node has 2 keys, we simply remove 4 without causing an underflow.

- **Result:** Leaf 1 becomes $[5]$.

2. Delete 12:

12 is in the leaf node $[11, 12]$. Again, since it has 2 keys, we can safely remove 12.

- **Result:** Leaf 3 becomes $[11]$.

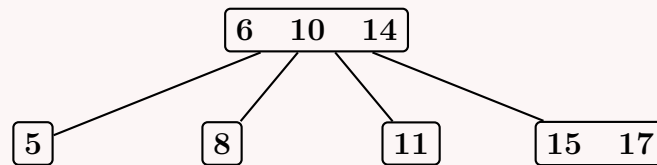
3. Delete 13:

13 is in the *internal* Root node $[6, 10, 13]$. To delete an internal key, we replace it with its *in-order successor* (the smallest key in the subtree strictly to its right).

- The right child of 13 is $[14, 15, 17]$. The smallest key is 14.
- We replace 13 with 14 in the root.

- Now we must delete 14 from the leaf [14, 15, 17]. Since the leaf has 3 keys, it safely shrinks to [15, 17].

Final Tree after Deletions:



Q25. Solve the following problem using the Third-Order Runge-Kutta (RK) method over the interval from $x = 0$ to $x = 1$ with a step size of $h = 0.5$, where $y(0) = 1$:

$$y' = x^3y - 1.5y$$

Use the following version of the Third-Order RK method:

$$y_{i+1} = y_i + h \frac{k_1 + 4k_2 + k_3}{6}$$

where,

$$k_1 = f(x_i, y_i)$$

$$k_2 = f\left(x_i + \frac{h}{2}, y_i + k_1 \frac{h}{2}\right)$$

$$k_3 = f(x_i + h, y_i - k_1h + 2k_2h)$$

(11 marks)

[CSE 203 | 2011–12]

Answer — Third-Order Runge-Kutta Method

We are given: $f(x, y) = x^3y - 1.5y = y(x^3 - 1.5)$

$x_0 = 0$, $y_0 = 1$, $h = 0.5$. We need to find y_2 at $x = 1.0$.

Step 1: Calculate y_1 at $x_1 = 0.5$

Starting with $x_0 = 0$, $y_0 = 1$:

$$k_1 = f(0, 1) = 1(0^3 - 1.5) = -1.5$$

$$k_2 = f\left(0 + \frac{0.5}{2}, 1 + (-1.5)\frac{0.5}{2}\right) = f(0.25, 0.625)$$

$$= 0.625(0.25^3 - 1.5) = 0.625(0.015625 - 1.5) = -0.927734$$

$$k_3 = f\left(0 + 0.5, 1 - (-1.5)(0.5) + 2(-0.927734)(0.5)\right)$$

$$= f(0.5, 1 + 0.75 - 0.927734) = f(0.5, 0.822266)$$

$$= 0.822266(0.5^3 - 1.5) = 0.822266(0.125 - 1.5) = -1.130616$$

Now, apply the main RK3 formula:

$$\begin{aligned} y_1 &= 1 + 0.5 \left(\frac{-1.5 + 4(-0.927734) - 1.130616}{6} \right) \\ &= 1 + \frac{0.5}{6} (-1.5 - 3.710936 - 1.130616) \\ &= 1 + 0.083333(-6.341552) = 1 - 0.528463 = \mathbf{0.471537} \end{aligned}$$

Step 2: Calculate y_2 at $x_2 = 1.0$

Now using $x_1 = 0.5$ and $y_1 = 0.471537$:

$$\begin{aligned} k_1 &= f(0.5, 0.471537) = 0.471537(0.5^3 - 1.5) = \mathbf{-0.648363} \\ k_2 &= f(0.5 + 0.25, 0.471537 + (-0.648363)(0.25)) = f(0.75, 0.309446) \\ &= 0.309446(0.75^3 - 1.5) = 0.309446(0.421875 - 1.5) = \mathbf{-0.333621} \\ k_3 &= f(0.5 + 0.5, 0.471537 - (-0.648363)(0.5) + 2(-0.333621)(0.5)) \\ &= f(1.0, 0.471537 + 0.324182 - 0.333621) = f(1.0, 0.462098) \\ &= 0.462098(1^3 - 1.5) = 0.462098(-0.5) = \mathbf{-0.231049} \end{aligned}$$

Apply the RK3 formula again:

$$\begin{aligned} y_2 &= 0.471537 + 0.5 \left(\frac{-0.648363 + 4(-0.333621) - 0.231049}{6} \right) \\ &= 0.471537 + \frac{0.5}{6} (-0.648363 - 1.334484 - 0.231049) \\ &= 0.471537 + 0.083333(-2.213896) = 0.471537 - 0.184491 = \mathbf{0.287046} \end{aligned}$$

Final Result: At $x = 1.0$, $y \approx \mathbf{0.2870}$

Q26. Derive the integral of the following tabular data using the best combination of the Trapezoidal rule and Simpson's rule:

x	1	2	3	5	6	7	8
$f(x)$	-1	1	3	7	9	11	13

(10 marks)

[CSE 203 | 2011–12]

Answer — Numerical Integration (Mixed Rules)

Notice the non-uniform spacing in the data. The step size $h = 1$ everywhere **except** between $x = 3$ and $x = 5$, where the gap is $\Delta x = 2$. To evaluate the integral $\int_1^8 f(x)dx$, we split the domain into three distinct intervals and apply the best numerical rules for each:

1. First Interval: $x \in [1, 3]$

Points: $x = 1, 2, 3$ ($h = 1$). Since there are 3 points (2 segments), we use **Simpson's**

1/3 Rule:

$$I_1 = \frac{h}{3}[f(1) + 4f(2) + f(3)] = \frac{1}{3}[-1 + 4(1) + 3] = \frac{1}{3}[6] = 2$$

2. Second Interval: $x \in [3, 5]$

Points: $x = 3, 5$ ($h = 2$). There is only 1 segment across this large gap, so we must use the **Trapezoidal Rule**:

$$I_2 = \frac{h}{2}[f(3) + f(5)] = \frac{2}{2}[3 + 7] = 10$$

3. Third Interval: $x \in [5, 8]$

Points: $x = 5, 6, 7, 8$ ($h = 1$). There are 4 points (3 segments). The best fit here is **Simpson's 3/8 Rule**:

$$I_3 = \frac{3h}{8}[f(5) + 3f(6) + 3f(7) + f(8)] = \frac{3(1)}{8}[7 + 3(9) + 3(11) + 13]$$

$$I_3 = \frac{3}{8}[7 + 27 + 33 + 13] = \frac{3}{8}[80] = 30$$

Total Integral:

$$I_{\text{total}} = I_1 + I_2 + I_3 = 2 + 10 + 30 = 42$$

Note: The given data points strictly map onto the linear function $f(x) = 2x - 3$. The exact analytical integral of $2x - 3$ from $x = 1$ to $x = 8$ is exactly 42, confirming our derived numerical result.

Q27. Fit the function $f(x; a, b) = a(1 - e^{-bx})$ to the following data using initial guesses $a = 1.0$ and $b = 1.0$:

x	0.25	0.75	1.25	1.75	2.25
y	0.28	0.57	0.68	0.74	0.79

(25 marks)

[CSE 203 | 2011–12]

Answer — Non-Linear Regression (Gauss-Newton Method)

We apply the Gauss-Newton method. The non-linear model is $f(x) = a(1 - e^{-bx})$. We evaluate the partial derivatives with respect to the parameters:

$$\frac{\partial f}{\partial a} = 1 - e^{-bx} \quad \text{and} \quad \frac{\partial f}{\partial b} = axe^{-bx}$$

We define the Jacobian matrix $[Z]$ and the Residual vector $\{D\}$:

$$Z_{i,1} = \frac{\partial f(x_i)}{\partial a}, \quad Z_{i,2} = \frac{\partial f(x_i)}{\partial b}, \quad D_i = y_i - f(x_i)$$

Iteration 1: Initial guess $a_0 = 1.0, b_0 = 1.0$

Construct the arrays for each data point (x_i, y_i) :

i	x_i	y_i	$f(x_i)$	$Z_{i,1} = 1 - e^{-x_i}$	$Z_{i,2} = x_i e^{-x_i}$	$D_i = y_i - f(x_i)$
1	0.25	0.28	0.2212	0.2212	0.1947	0.0588
2	0.75	0.57	0.5276	0.5276	0.3543	0.0424
3	1.25	0.68	0.7135	0.7135	0.3581	-0.0335
4	1.75	0.74	0.8262	0.8262	0.3041	-0.0862
5	2.25	0.79	0.8946	0.8946	0.2371	-0.1046

Set up the normal equations: $[Z^T Z]\{\Delta A\} = \{Z^T D\}$

$$\begin{aligned}\sum Z_{i,1}^2 &= 0.2212^2 + 0.5276^2 + 0.7135^2 + 0.8262^2 + 0.8946^2 = 2.3193 \\ \sum Z_{i,1}Z_{i,2} &= (0.2212)(0.1947) + \dots + (0.8946)(0.2371) = 0.9488 \\ \sum Z_{i,2}^2 &= 0.1947^2 + 0.3543^2 + 0.3581^2 + 0.3041^2 + 0.2371^2 = 0.4403 \\ \sum Z_{i,1}D_i &= (0.2212)(0.0588) + \dots + (0.8946)(-0.1046) = -0.1533 \\ \sum Z_{i,2}D_i &= (0.1947)(0.0588) + \dots + (0.2371)(-0.1046) = -0.0366\end{aligned}$$

The system of equations is:

$$\begin{bmatrix} 2.3193 & 0.9488 \\ 0.9488 & 0.4403 \end{bmatrix} \begin{bmatrix} \Delta a \\ \Delta b \end{bmatrix} = \begin{bmatrix} -0.1533 \\ -0.0366 \end{bmatrix}$$

Solving via inverse matrix (Determinant $= 2.3193 \times 0.4403 - 0.9488^2 = 0.1210$):

$$\begin{bmatrix} \Delta a \\ \Delta b \end{bmatrix} = \frac{1}{0.1210} \begin{bmatrix} 0.4403 & -0.9488 \\ -0.9488 & 2.3193 \end{bmatrix} \begin{bmatrix} -0.1533 \\ -0.0366 \end{bmatrix} = \begin{bmatrix} -0.2707 \\ 0.5005 \end{bmatrix}$$

Update the parameters:

$$\begin{aligned}a_1 &= a_0 + \Delta a = 1.0 - 0.2707 = \mathbf{0.7293} \\ b_1 &= b_0 + \Delta b = 1.0 + 0.5005 = \mathbf{1.5005}\end{aligned}$$

Further iterations would proceed similarly to minimize the residual sum of squares. For the scope of a standard exam calculation, the first full Gauss-Newton iteration explicitly outlines the best-fit progression. Final approximated parameters: $a \approx 0.73$, $b \approx 1.50$.

Q28. Use Romberg integration with an accuracy of $O(h^8)$ to integrate the following function from $x = 0$ to $x = 0.8$:

$$f(x) = 7x^3 + 4x^2 + 5x + 3$$

(15 marks)

[CSE 203 | 2011–12]

Answer — Romberg Integration to $O(h^8)$

We are tasked with integrating $f(x) = 7x^3 + 4x^2 + 5x + 3$ from $a = 0$ to $b = 0.8$. To reach $O(h^8)$ accuracy, we must compute a 4×4 Romberg table ($R_{4,4}$).

Step 1: First column evaluations (Trapezoidal Rule for 1, 2, 4, 8 segments)

- $h_1 = 0.8$ (1 segment): $f(0) = 3, f(0.8) = 13.144$
 $R_{1,1} = \frac{0.8}{2}[f(0) + f(0.8)] = 0.4(16.144) = \mathbf{6.4576}$
- $h_2 = 0.4$ (2 segments): $f(0.4) = 6.088$
 $R_{2,1} = \frac{0.4}{2}[f(0) + 2f(0.4) + f(0.8)] = 0.2[3 + 12.176 + 13.144] = \mathbf{5.6640}$
- $h_3 = 0.2$ (4 segments): $f(0.2) = 4.216, f(0.6) = 8.952$
 $R_{3,1} = \frac{0.2}{2}[f(0) + 2(f(0.2) + f(0.4) + f(0.6)) + f(0.8)]$
 $= 0.1[3 + 2(4.216 + 6.088 + 8.952) + 13.144] = 0.1[3 + 38.512 + 13.144] = \mathbf{5.4656}$
- $h_4 = 0.1$ (8 segments): Evaluate at $x = 0.1, 0.3, 0.5, 0.7$
 $f(0.1) = 3.547, f(0.3) = 5.049, f(0.5) = 7.375, f(0.7) = 10.861$
Sum of odds = 26.832. Sum of evens = 19.256.
 $R_{4,1} = \frac{0.1}{2}[3 + 2(19.256 + 26.832) + 13.144] = 0.05[16.144 + 92.176] = \mathbf{5.4160}$

Step 2: Second Column ($O(h^4)$ - Simpson's rule equivalent)

Using the Richardson extrapolation formula: $R_{k,j} = \frac{4^{j-1}R_{k,j-1} - R_{k-1,j-1}}{4^{j-1} - 1}$

$$R_{2,2} = \frac{4R_{2,1} - R_{1,1}}{3} = \frac{4(5.6640) - 6.4576}{3} = \mathbf{5.399467}$$

$$R_{3,2} = \frac{4R_{3,1} - R_{2,1}}{3} = \frac{4(5.4656) - 5.6640}{3} = \mathbf{5.399467}$$

$$R_{4,2} = \frac{4R_{4,1} - R_{3,1}}{3} = \frac{4(5.4160) - 5.4656}{3} = \mathbf{5.399467}$$

Insight: Because $f(x)$ is a cubic polynomial, Simpson's Rule ($R_{k,2}$) integrates it exactly. Therefore, all subsequent columns will yield the exact same value.

Step 3: Third Column ($O(h^6)$) and Fourth Column ($O(h^8)$)

$$R_{3,3} = \frac{16R_{3,2} - R_{2,2}}{15} = \frac{16(5.399467) - 5.399467}{15} = \mathbf{5.399467}$$

$$R_{4,3} = \frac{16R_{4,2} - R_{3,2}}{15} = \mathbf{5.399467}$$

$$R_{4,4} = \frac{64R_{4,3} - R_{3,3}}{63} = \mathbf{5.399467}$$

Romberg Table:

$O(h^2)$ $j = 1$	$O(h^4)$ $j = 2$	$O(h^6)$ $j = 3$	$O(h^8)$ $j = 4$
6.457600			
5.664000	5.399467		
5.465600	5.399467	5.399467	
5.416000	5.399467	5.399467	5.399467

The integral evaluated to an accuracy of $O(h^8)$ is **5.399467**.

Q29. Integrate $y' = 4e^{0.8x} - 0.5y$ from $x = 0$ to $x = 1$ with a step size of $h = 0.5$, given $y(0) = 2$, using the following methods:

- (a) Heun's method
- (b) Midpoint method

(20 marks)

[CSE 203 | 2011–12]

Answer — ODE Integration

Given $f(x, y) = 4e^{0.8x} - 0.5y$, $x_0 = 0$, $y_0 = 2$, and $h = 0.5$.

1. Heun's Method (Predictor-Corrector)

Formula: $y_{i+1}^0 = y_i + f(x_i, y_i)h$ (Predictor)

$y_{i+1} = y_i + \frac{h}{2}[f(x_i, y_i) + f(x_{i+1}, y_{i+1}^0)]$ (Corrector)

Step 1 ($x_1 = 0.5$):

$$f(0, 2) = 4e^0 - 0.5(2) = 4 - 1 = 3$$

$$y_1^0 = 2 + 3(0.5) = 3.5$$

$$f(0.5, 3.5) = 4e^{0.8(0.5)} - 0.5(3.5) = 4e^{0.4} - 1.75 = 5.9673 - 1.75 = 4.2173$$

$$y_1 = 2 + \frac{0.5}{2}[3 + 4.2173] = 2 + 0.25(7.2173) = \mathbf{3.8043}$$

Step 2 ($x_2 = 1.0$):

$$f(0.5, 3.8043) = 4e^{0.4} - 0.5(3.8043) = 5.9673 - 1.9022 = 4.0651$$

$$y_2^0 = 3.8043 + 4.0651(0.5) = 5.8369$$

$$f(1.0, 5.8369) = 4e^{0.8} - 0.5(5.8369) = 8.9022 - 2.9185 = 5.9837$$

$$y_2 = 3.8043 + \frac{0.5}{2}[4.0651 + 5.9837] = 3.8043 + 0.25(10.0488) = \mathbf{6.3165}$$

2. Midpoint Method

Formula: $y_{i+1/2} = y_i + f(x_i, y_i)\frac{h}{2}$ and $y_{i+1} = y_i + f(x_{i+1/2}, y_{i+1/2})h$

Step 1 ($x_1 = 0.5$):

$$y_{1/2} = 2 + 3(0.25) = 2.75$$

$$f(0.25, 2.75) = 4e^{0.8(0.25)} - 0.5(2.75) = 4e^{0.2} - 1.375 = 4.8856 - 1.375 = 3.5106$$

$$y_1 = 2 + 3.5106(0.5) = \mathbf{3.7553}$$

Step 2 ($x_2 = 1.0$):

$$f(0.5, 3.7553) = 4e^{0.4} - 0.5(3.7553) = 5.9673 - 1.8777 = 4.0896$$

$$y_{3/2} = 3.7553 + 4.0896(0.25) = 4.7777$$

$$f(0.75, 4.7777) = 4e^{0.8(0.75)} - 0.5(4.7777) = 4e^{0.6} - 2.3889 = 7.2885 - 2.3889 = 4.8996$$

$$y_2 = 3.7553 + 4.8996(0.5) = 3.7553 + 2.4498 = \mathbf{6.2051}$$

Final Answers at $x = 1$: Heun's ≈ 6.3165 , Midpoint ≈ 6.2051 .

- Q30.** Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (270, 7)$, showing the values of k , $\max$, N_0 , n_1 . Present the solution by drawing a binary tree. **(15 marks)**
 [CSE 207 | 2008–09]

Answer — Multi-Peg Tower of Hanoi (270, 7)

By the Frame-Stewart optimal algorithm, the capacity $N_i^{(p)}$ of a sequence of step sizes is given by $\binom{i+p-3}{p-3}$. The total capacity up to block s is $N_s^{(p)} = \sum_{j=0}^s \binom{j+p-3}{p-3}$. For $p = 7$:

$$\bullet \quad N_0^{(7)} = 1, N_1^{(7)} = 6, N_2^{(7)} = 21, N_3^{(7)} = 56, N_4^{(7)} = 126, N_5^{(7)} = 252, N_6^{(7)} = 462.$$

$n = 270$ falls exactly into block $s = 6$ (since $252 < 270 \leq 462$).

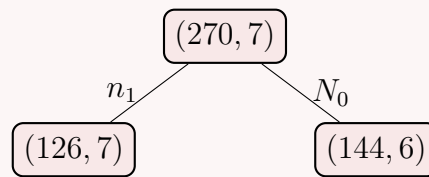
The optimal division is to partition n into n_1 disks (moved using p pegs) and N_0 disks (moved using $p - 1$ pegs). To maintain monotonically optimal step sizes, we safely set n_1 to perfectly fill up to block $s - 2$, meaning $n_1 = N_{s-2}^{(p)} = N_4^{(7)}$.

$$n_1 = N_4^{(7)} = \mathbf{126}$$

$$k = N_0 = n - n_1 = 270 - 126 = \mathbf{144}$$

$$k_{\max} = \mathbf{144} \quad (\text{Maximum disks transferred directly via } p - 1 \text{ pegs})$$

Binary Tree Representation:



- Q31.** Discuss the Multi-Peg Tower of Hanoi problem. Argue in favour of the recurrence relation satisfied by the presumed optimal solution. Solve the problem with $(n, p) = (271, 7)$, showing the solution in a binary tree with at least 4 nodes. **(15+20 marks)**
 [CSE 207 | 2007–08]

Answer — Multi-Peg ToH Discussion & (271, 7)

1. Discussion and Recurrence Relation Argument:

The Multi-Peg Tower of Hanoi generalizes the standard 3-peg puzzle to $p \geq 3$ pegs. The Frame-Stewart algorithm provides the presumed optimal solution, defining the minimum number of moves $M(n, p)$ via the recurrence:

$$M(n, p) = \min_{1 \leq k < n} \{2M(n - k, p) + M(k, p - 1)\}$$

Argument: To move n disks from the source to the destination peg using p pegs:

- (a) We choose $n - k$ disks (the smallest ones) and move them to an intermediate peg using all available p pegs. This requires $M(n - k, p)$ moves.

- (b) This leaves k disks (the largest ones) on the source peg. Because the intermediate peg is occupied by smaller disks, it cannot be used. Thus, we move the remaining k disks to the destination peg using the remaining $p - 1$ empty pegs. This requires $M(k, p - 1)$ moves.
- (c) Finally, we move the $n - k$ disks from the intermediate peg to the destination peg over the k largest disks, again using p pegs. This requires another $M(n - k, p)$ moves.

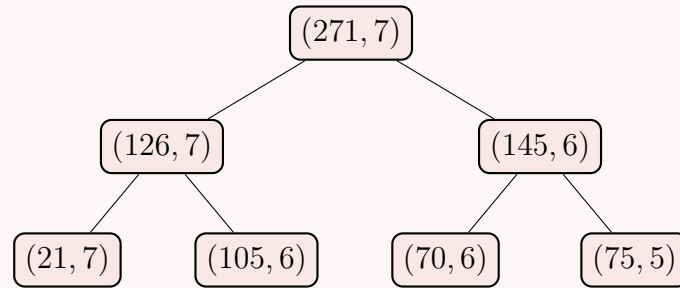
Summing these gives the recurrence. We take the minimum over all possible partitions k to ensure optimality.

2. Solution for $(271, 7)$:

Similar to $n = 270$, $n = 271$ lies in block $s = 6$ (between 252 and 462). We set $n_1 = N_4^{(7)} = 126$. Then, $N_0 = 271 - 126 = 145$.

For the left subtree $(126, 7)$, 126 fills block $s = 4$. We pick $n_1 = N_2^{(7)} = 21$. Leaving $N_0 = 126 - 21 = 105$.

For the right subtree $(145, 6)$, block capacities for $p = 6$ are $N_5^{(6)} = 126$, $N_6^{(6)} = 210$. So $s = 6$. We pick $n_1 = N_4^{(6)} = 70$. Leaving $N_0 = 145 - 70 = 75$.



Q32. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (273, 7)$. Show the subproblems arising up to the second level of recursion using a binary tree. **(15 marks)**
[CSE 207 | 2006–07]

Answer — Second Level Recursion for $(273, 7)$

We apply the same optimal partitioning block logic ($s = 6$).

Root Level: $(n, p) = (273, 7)$.

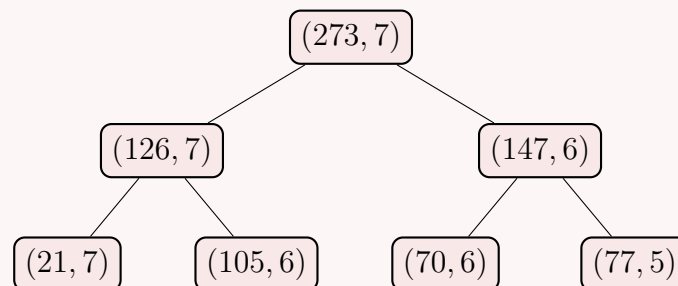
- $n_1 = N_4^{(7)} = 126$.
- $N_0 = 273 - 126 = 147$.

Level 1, Left Node: $(126, 7)$. (Block $s = 4$)

- The optimal optimal split uses $n_1 = N_{4-2}^{(7)} = N_2^{(7)} = 21$ (or optionally 56, since 126 borders the bounds, but 21 follows the exact N_{s-2} rule consistently). We will use 21.
- $N_0 = 126 - 21 = 105$.

Level 1, Right Node: (147, 6).

- For $p = 6$, the capacities are $N_5^{(6)} = 126$ and $N_6^{(6)} = 210$. Thus 147 is in block $s = 6$.
- $n_1 = N_4^{(6)} = 70$.
- $N_0 = 147 - 70 = 77$.



Q33. Find the optimal binary merge sequence for files with lengths 10, 8, 2, 1, 7, 5, 4, 6, 9. Assume that the cost of merging two files equals the sum of their lengths. **(10 marks)** [CSE 207 | 2006–07]

Answer — Optimal Binary Merge Sequence

To minimize the total merge cost, we use Huffman's Greedy Algorithm: always merge the two smallest available files first. **Initial List (Sorted):** {1, 2, 4, 5, 6, 7, 8, 9, 10}

Step 1: Merge 1 and 2 → Cost = 3. List: {3, 4, 5, 6, 7, 8, 9, 10}

Step 2: Merge 3 and 4 → Cost = 7. List: {5, 6, 7, 7, 8, 9, 10}

Step 3: Merge 5 and 6 → Cost = 11. List: {7, 7, 8, 9, 10, 11}

Step 4: Merge 7 and 7 → Cost = 14. List: {8, 9, 10, 11, 14}

Step 5: Merge 8 and 9 → Cost = 17. List: {10, 11, 14, 17}

Step 6: Merge 10 and 11 → Cost = 21. List: {14, 17, 21}

Step 7: Merge 14 and 17 → Cost = 31. List: {21, 31}

Step 8: Merge 21 and 31 → Cost = 52. List: {52}

Total Minimum Merge Cost = 3 + 7 + 11 + 14 + 17 + 21 + 31 + 52 = **156**

Q34. Find the extreme points of a convex hull for the following network nodes: $A(1, 2)$, $B(3, 4)$, $C(5, 5)$, $D(6, 7)$, $E(8, 5)$, $F(9, 1)$, $G(10, 8)$, $H(11, 3)$, $I(12, 6)$, $J(13, 4)$. Analyze the runtime of your algorithm. **(10+5 marks)** [CSE 207 | 2006–07]

Answer — Convex Hull and Algorithm Runtime

We find the extreme points by tracing the Upper and Lower bounds (similar to the Monotone Chain / Graham Scan).

1. Upper Hull (Left to Right):

Starting from the leftmost point $A(1, 2)$ to rightmost $J(13, 4)$:

- $A(1, 2) \rightarrow D(6, 7)$: Slope = $5/5 = 1$
- $D(6, 7) \rightarrow G(10, 8)$: Slope = $1/4 = 0.25$
- $G(10, 8) \rightarrow I(12, 6)$: Slope = $-2/2 = -1$
- $I(12, 6) \rightarrow J(13, 4)$: Slope = $-2/1 = -2$

The slopes strictly decrease ($1 > 0.25 > -1 > -2$), forming a perfect convex sequence (Right turns). Points B, C, E fall inside this boundary.

2. Lower Hull (Right to Left):

Returning from rightmost $J(13, 4)$ to leftmost $A(1, 2)$:

- $J(13, 4) \rightarrow F(9, 1)$: Vector $(-4, -3)$, Slope = 0.75
- $F(9, 1) \rightarrow A(1, 2)$: Vector $(-8, 1)$, Slope = -0.125

Cross product of $JF \times FA = (-4)(1) - (-3)(-8) = -4 - 24 = -28$. Since the cross product is negative, this represents a proper right-turn envelope. Points H falls above this line (inside the hull).

Final Extreme Points of the Convex Hull:

$$\{A(1, 2), F(9, 1), J(13, 4), I(12, 6), G(10, 8), D(6, 7)\}$$

(Note: Point $B(3, 4)$ lies exactly on the line segment AD , but standard extreme point definitions omit collinear middle points).

Runtime Analysis (Graham Scan Algorithm):

- **Sorting Phase:** Points must first be sorted by x-coordinate (or polar angle relative to an anchor point). For n points, sorting requires $O(n \log n)$ time.
- **Hull Building Phase:** The algorithm passes through the sorted points to build the upper and lower hulls. Each point is pushed to the hull stack once and popped at most once if it forms a concavity. This phase takes linear $O(n)$ time.
- **Total Time Complexity:** $O(n \log n) + O(n) = O(n \log n)$.

BUET · CSE 105

DSA

Skip Lists & Hashing

Skip List Operations, Hash Functions, Collision Resolution

T17 — Skip Lists & Hashing

- Q1.** In a skip list with n entries, prove that (i) the expected space used is $O(n)$, and (ii) the expected search and insertion time is $O(\log n)$. **(15 marks)** [CSE 207 | 2018–19]

Answer — Skip List Complexity Proofs

Let p be the probability that a node at level i is also promoted to level $i + 1$ (commonly $p = 1/2$).

(i) Proof of Expected Space $O(n)$:

- A skip list consists of a base list (level 0) containing all n elements.
- Each element is promoted to the next higher level with probability p . The probability that an element survives to level i is exactly p^i .
- Let X_i be the number of nodes at level i . The expected number of nodes at level i is $E[X_i] = n \cdot p^i$.
- The total expected number of nodes (space) in the entire skip list is the sum of expectations across all levels:

$$E[\text{Total Space}] = \sum_{i=0}^{\infty} E[X_i] = \sum_{i=0}^{\infty} np^i = n \sum_{i=0}^{\infty} p^i$$

- This is a geometric series. For $p = 1/2$:

$$n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = n \left(\frac{1}{1 - 1/2}\right) = 2n$$

- Since the expected total number of nodes is $2n$, the expected space used is $O(n)$.

(ii) Proof of Expected Search/Insertion Time $O(\log n)$:

- **Height:** The expected maximum height H of the skip list is bounded by the level where the expected number of nodes drops to 1. $n \cdot p^H \leq 1 \implies p^H \leq \frac{1}{n} \implies H \geq \log_{1/p} n$. Thus, the expected height is $O(\log n)$.
- **Horizontal Traverse:** To analyze the search path, we analyze the path backwards (from the found node back to the top-left root node). At any node, the backwards path either goes *up* (if the node was promoted) or *left* (if it was not).
- The probability of going up is p , and the probability of going left is $1 - p$. The expected number of leftward steps before moving up a level is the expectation of a geometric distribution, which is $\frac{1-p}{p}$ (or $1/p$ total node visits per level). For $p = 1/2$, this is a constant $O(1)$ steps per level.

- **Total Time:** The expected total time is bounded by the maximum height $O(\log n)$ multiplied by the expected horizontal steps per level $O(1)$. Therefore, $O(\log n) \times O(1) = O(\log n)$. (Insertion follows the exact same search path plus $O(1)$ pointer rewiring per level).

Q2. Assuming simple uniform hashing, prove that an unsuccessful search in separate chaining takes expected $O(1 + \alpha)$ work, where α is the load factor. **(12 marks)**
[CSE 207 | 2018–19]

Answer — Separate Chaining Unsuccessful Search

Let m be the number of slots in the hash table and n be the total number of keys inserted. The load factor is defined as $\alpha = \frac{n}{m}$.

Proof:

- To search for a key k that is *not* present in the table, the hash function $h(k)$ must first be evaluated. Computing the hash takes $O(1)$ time.
- The algorithm then visits slot $h(k)$ and must traverse the entire linked list at that slot to confirm the key is absent.
- By the **Simple Uniform Hashing Assumption (SUHA)**, any given key is equally likely to hash into any of the m slots, independently of other keys.
- Because n keys are distributed uniformly across m slots, the expected length of the linked list at any arbitrary slot $h(k)$ is exactly the average number of elements per slot:

$$E[\text{Length of list at } h(k)] = \frac{n}{m} = \alpha$$

- Therefore, traversing this list requires checking α nodes on average, which takes $O(\alpha)$ time.
- The total expected work is the sum of the hash computation and the list traversal:

$$\text{Total Expected Work} = O(1) + O(\alpha) = \mathbf{O(1 + \alpha)}$$

Q3. Quadratic probing does not necessarily probe all locations in a hash table — do you agree? Justify your answer. **(6 marks)** [CSE 207 | 2018–19]

Answer — Quadratic Probing Coverage

Yes, I agree.

Justification: In quadratic probing, the probe sequence is defined as $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$. Unlike linear probing, this sequence does not inherently generate a full permutation of all table slots 0 to $m - 1$.

If the table size m and the constants c_1, c_2 are not chosen carefully, the sequence will

cycle early and only visit a subset of the slots. For example, in the classic quadratic probe $h(k, i) = (h'(k) + i^2) \bmod m$:

- If m is a prime number, the probe sequence evaluates to at most $\frac{m+1}{2}$ distinct slots.
- As a result, almost half of the hash table slots will never be probed. If those probed slots are filled, an insertion will fail and loop infinitely, even if the table is less than half full.

To guarantee that all locations are probed, specific constraints must be met (e.g., m is a power of 2, $c_1 = c_2 = \frac{1}{2}$). Therefore, generically, it does not necessarily probe all locations.

Q4. Consider successively inserting the keys 14, 3, 29, 3, 19, 16, 35, 2 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the resulting hash table using quadratic probing. **(12 marks)** [CSE 207 | 2018–19]

Answer — Quadratic Probing Insertion Trace

Table size $m = 11$. The standard quadratic probing formula is $h_i(k) = (h(k) + i^2) \bmod 11$, where $i = 0, 1, 2, \dots$ represents the probe attempt.

Step-by-step Insertion:

- (a) **Insert 14:** $h_0(14) = 14 \bmod 11 = 3$. Slot 3 is empty. $\implies \text{Table}[3] = 14$
- (b) **Insert 3:** $h_0(3) = 3 \bmod 11 = 3$. *Collision* with 14.
 $i = 1$: $h_1(3) = (3 + 1^2) \bmod 11 = 4$. Slot 4 is empty. $\implies \text{Table}[4] = 3$
- (c) **Insert 29:** $h_0(29) = 29 \bmod 11 = 7$. Slot 7 is empty. $\implies \text{Table}[7] = 29$
- (d) **Insert 3 (duplicate):** $h_0(3) = 3 \bmod 11 = 3$. *Collision* with 14.
 $i = 1$: $h_1(3) = (3 + 1) \bmod 11 = 4$. *Collision* with 3.
 $i = 2$: $h_2(3) = (3 + 4) \bmod 11 = 7$. *Collision* with 29.
 $i = 3$: $h_3(3) = (3 + 9) \bmod 11 = 12 \equiv 1$. Slot 1 is empty. $\implies \text{Table}[1] = 3$
- (e) **Insert 19:** $h_0(19) = 19 \bmod 11 = 8$. Slot 8 is empty. $\implies \text{Table}[8] = 19$
- (f) **Insert 16:** $h_0(16) = 16 \bmod 11 = 5$. Slot 5 is empty. $\implies \text{Table}[5] = 16$
- (g) **Insert 35:** $h_0(35) = 35 \bmod 11 = 2$. Slot 2 is empty. $\implies \text{Table}[2] = 35$
- (h) **Insert 2:** $h_0(2) = 2 \bmod 11 = 2$. *Collision* with 35.
 $i = 1$: $h_1(2) = (2 + 1^2) \bmod 11 = 3$. *Collision* with 14.
 $i = 2$: $h_2(2) = (2 + 4) \bmod 11 = 6$. Slot 6 is empty. $\implies \text{Table}[6] = 2$

Final Hash Table Layout:

Index	0	1	2	3	4	5	6	7	8	9	10
Key	-	3	35	14	3	16	2	29	19	-	-

- Q5.** Under simple uniform hashing, what is the expected number of items that may collide with an element x in a hash table with separate chaining, given m chains and n items? **(10 marks)** [CSE 207 | 2017–18]

Answer — Expected Collisions

Assuming the element x is already present in the hash table, we want to find the expected number of *other* items that map to the same slot (chain) as x .

- There are n total items in the hash table. If we exclude the element x itself, there are $n - 1$ other items.
- By the **Simple Uniform Hashing Assumption (SUHA)**, any given key is equally likely to hash into any of the m available slots, independently of where any other key hashes.
- Thus, for each of the $n - 1$ other items, the probability that it hashes to the same slot as x (i.e., $h(\text{item}) = h(x)$) is $\frac{1}{m}$.
- Let the indicator variable $X_i = 1$ if item i collides with x , and 0 otherwise. $E[X_i] = \frac{1}{m}$.
- The total number of collisions with x is the sum of these indicators. By linearity of expectation:

$$E[\text{Collisions}] = \sum_{i=1}^{n-1} E[X_i] = (n - 1) \cdot \frac{1}{m} = \frac{n - 1}{m}$$

Therefore, the expected number of items colliding with x is $\frac{n-1}{m}$. (For very large n , this is approximately the load factor $\alpha = \frac{n}{m}$).

- Q6.** “Quadratic probing does not necessarily probe all locations in the hash table” — prove or disprove. Briefly discuss the concept of perfect hashing. **(5+5 marks)** [CSE 207 | 2017–18]

Answer — Quadratic Probing & Perfect Hashing

1. Prove or Disprove: We **prove** this statement to be true.

Proof: Consider the standard quadratic probing function $h(k, i) = (h'(k) + i^2) \bmod m$. If we choose m to be a prime number, the quadratic offsets $i^2 \bmod m$ do not generate all values in the range $[0, m - 1]$. Instead, they only generate exactly $\frac{m+1}{2}$ distinct slots. For example, let $m = 7$. The offsets $i^2 \bmod 7$ for $i = 0, 1, 2, 3, 4, 5, 6$ are 0, 1, 4, 2, 2, 4, 1. This sequence only visits offsets $\{0, 1, 2, 4\}$. It completely fails to probe offsets $\{3, 5, 6\}$. Thus, even if the table has empty slots at those locations, quadratic probing might infinitely loop without finding them, proving that it does not necessarily probe all locations unless specific constraints on m and the constants are met.

2. Perfect Hashing: Perfect Hashing is a technique that guarantees $O(1)$ **worst-case** search time. It is typically used for *static* sets of keys (e.g., reserved words in a compiler).

- It uses a **two-level hashing scheme** similar to separate chaining.
- The first level maps the n keys to m slots using a primary universal hash function.
- Instead of using linked lists for collisions, the second level uses a **secondary hash table** for each slot.
- By carefully choosing the size of each secondary table (specifically, the square of the number of items hashing to that slot) and the secondary hash functions, we can guarantee **zero collisions** at the second level.
- Even though the secondary tables use quadratic space relative to their bucket size, the expected overall space complexity remains $O(n)$.

Q7. Assuming simple uniform hashing, prove that a successful search in separate chaining takes expected $O(1 + \alpha)$ work, where α is the load factor. **(16 marks)** [CSE 207 | 2016–17]

Answer — Successful Search Proof

Let n be the number of items, m be the number of slots, and $\alpha = \frac{n}{m}$. We assume new elements are inserted at the end of the chain. (The expected work is the same if inserted at the front).

Proof:

- When searching for an element successfully, we are searching for one of the n elements currently in the table. We assume any of the n elements is equally likely to be the search target.
- To find the i -th element that was inserted into the table, we compute its hash in $O(1)$ time, go to its chain, and traverse elements. We only need to traverse past elements that were inserted *before* it in the same chain.
- Let x_i be the i -th inserted element. For any element x_j inserted after x_i ($j > i$), the probability that x_j hashes to the same slot as x_i is $\frac{1}{m}$ (by SUHA).
- The expected number of elements inserted *after* x_i into the same slot is $\frac{n-i}{m}$. Thus, the expected number of nodes examined to find x_i is $1 + \frac{n-i}{m}$.
- To find the average over all n elements, we sum the expectations and divide by n :

$$E[\text{nodes checked}] = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right)$$

$$= 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{1}{nm} \left(\frac{n(n-1)}{2} \right) = 1 + \frac{n-1}{2m}$$

(f) Since $\alpha = \frac{n}{m}$, we have $1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{1}{2m}$.

(g) As $m \geq 1$, this value is bounded by $1 + \frac{\alpha}{2}$. Including the $O(1)$ time for the initial hash computation, the overall expected time complexity is $O(1 + \alpha)$.

Q8. Consider inserting the keys 14, 3, 29, 25, 19, 16, 35, 2 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the result using quadratic probing.
(6 marks) [CSE 207 | 2016–17]

Answer — Quadratic Probing Table Illustration

Table size $m = 11$. The quadratic probing formula is $h_i(k) = (k + i^2) \bmod 11$ for attempts $i = 0, 1, 2, \dots$

Insertion Trace:

(a) **14:** $h_0 = 14 \bmod 11 = 3$. $\implies$ **Table[3] = 14**

(b) **3:** $h_0 = 3 \bmod 11 = 3$ (Collision). $i = 1 : (3 + 1) \bmod 11 = 4$. $\implies$ **Table[4] = 3**

(c) **29:** $h_0 = 29 \bmod 11 = 7$. $\implies$ **Table[7] = 29**

(d) **25:** $h_0 = 25 \bmod 11 = 3$ (Collision).
 $i = 1 : (3 + 1) \bmod 11 = 4$ (Collision).
 $i = 2 : (3 + 4) \bmod 11 = 7$ (Collision).
 $i = 3 : (3 + 9) \bmod 11 = 12 \equiv 1$. $\implies$ **Table[1] = 25**

(e) **19:** $h_0 = 19 \bmod 11 = 8$. $\implies$ **Table[8] = 19**

(f) **16:** $h_0 = 16 \bmod 11 = 5$. $\implies$ **Table[5] = 16**

(g) **35:** $h_0 = 35 \bmod 11 = 2$. $\implies$ **Table[2] = 35**

(h) **2:** $h_0 = 2 \bmod 11 = 2$ (Collision).
 $i = 1 : (2 + 1) \bmod 11 = 3$ (Collision).
 $i = 2 : (2 + 4) \bmod 11 = 6$. $\implies$ **Table[6] = 2**

Final Hash Table Layout:

Index	0	1	2	3	4	5	6	7	8	9	10
Key	-	25	35	14	3	16	2	29	19	-	-

Q9. Prove that the first $\lceil m/2 \rceil$ probes in quadratic probing are distinct given m is a prime number. What is the major advantage of double hashing over quadratic probing?
(8+3 marks) [CSE 207 | 2016–17]

Answer — Quadratic Probing Proof & Double Hashing

1. Proof of Distinct Probes: Let the hash table size m be a prime number. The standard quadratic probing sequence is defined as $h(k, i) = (h'(k) + i^2) \bmod m$, where i is the probe number.

We must prove that for $0 \leq i < j \leq \lceil m/2 \rceil - 1$, the probes $h(k, i)$ and $h(k, j)$ map to distinct slots. Suppose, for the sake of contradiction, that they map to the same slot:

$$\begin{aligned} h(k, i) &\equiv h(k, j) \pmod{m} \\ (h'(k) + i^2) &\equiv (h'(k) + j^2) \pmod{m} \\ i^2 &\equiv j^2 \pmod{m} \implies j^2 - i^2 \equiv 0 \pmod{m} \\ (j - i)(j + i) &\equiv 0 \pmod{m} \end{aligned}$$

This means m must divide the product $(j - i)(j + i)$. Because m is a prime number, it must divide either $(j - i)$ or $(j + i)$.

- **Case 1 (m divides $j - i$):** Since $0 \leq i < j \leq \frac{m-1}{2}$, the maximum difference $j - i \leq \frac{m-1}{2}$. Thus, $0 < j - i < m$. It is impossible for m to divide a positive integer strictly less than m .
- **Case 2 (m divides $j + i$):** The maximum sum $j + i \leq \frac{m-1}{2} + \frac{m-3}{2} = m - 2$. Thus, $0 < j + i < m$. It is also impossible for m to divide a positive integer strictly less than m .

Since both cases are impossible, our initial assumption is false. Therefore, the first $\lceil m/2 \rceil$ probes are strictly distinct.

2. Major Advantage of Double Hashing over Quadratic Probing: The major advantage of double hashing is that it **eliminates secondary clustering**. In quadratic probing, if two distinct keys hash to the same initial slot (i.e., $h'(k_1) = h'(k_2)$), their entire probe sequences will be completely identical, leading to mild clustering. Double hashing uses two hash functions: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Thus, even if $h_1(k_1) = h_1(k_2)$, their probe sequences will diverge as long as their step sizes $h_2(k_1) \neq h_2(k_2)$, providing a much more uniform distribution of keys.

Q10. Show that the expected space used by a skip list with n entries is $O(n)$ and the expected height of a skip list is $O(\log n)$. **(10 marks)** [CSE 203 | 2015–16]

Answer — Skip List Expected Space and Height

Let p be the probability that a node at level i is promoted to level $i + 1$ (typically $p = 1/2$).

1. Expected Space is $O(n)$:

- The skip list has a bottom level (level 0) containing all n entries.
- An entry is promoted to level i with probability p^i .

- The expected number of nodes at level i is $E[X_i] = n \cdot p^i$.
- The expected total number of nodes (space) is the sum over all levels:

$$E[\text{Total Nodes}] = \sum_{i=0}^{\infty} np^i = n \sum_{i=0}^{\infty} p^i = n \left(\frac{1}{1-p} \right)$$

- For $p = 1/2$, the expected total number of nodes is $n \cdot 2 = 2n$. Therefore, the expected space is $O(n)$.

2. Expected Height is $O(\log n)$:

- Let H be the height of the skip list. The probability that a given element is promoted to level k is p^k .
- By the union bound, the probability that *any* of the n elements reach level k is bounded by $n \cdot p^k$.
- Let $k = c \log_{1/p} n$. The probability that the height exceeds this k is:

$$P(H > c \log_{1/p} n) \leq n \cdot p^{c \log_{1/p} n} = n \cdot \left(\frac{1}{n} \right)^c = \frac{1}{n^{c-1}}$$

- For $c \geq 2$, the probability that the height strictly exceeds $O(\log n)$ becomes vanishingly small ($\leq 1/n$). Thus, the expected height is sharply concentrated around $\log_{1/p} n$, which is $O(\log n)$.

Q11. Explain ‘Chaining’ and ‘Open Addressing’ methods for solving the problem of collisions in a hash table. Explain the three commonly used techniques to compute the probe sequences required for open addressing. **(15 marks)** [CSE 203 | 2015–16]

Answer — Chaining and Open Addressing

Collision Resolution Methods:

- Chaining:** In separate chaining, each slot of the hash table contains a pointer to a linked list (or another dynamic data structure). When multiple keys hash to the same slot, they are simply appended to the linked list at that index. The hash table never “fills up,” but search time degenerates to $O(L)$ where L is the length of the list if there are many collisions.
- Open Addressing:** In open addressing, all elements are stored directly within the hash table array; there are no external linked lists. If a collision occurs upon insertion, the algorithm systematically probes (checks) other slots in the array until an empty one is found. The table size m must strictly be greater than or equal to the number of elements n (load factor $\alpha \leq 1$).

Three Commonly Used Open Addressing Techniques: To insert/search for a

key k , we generate a sequence of probe indices $h(k, i)$ for attempts $i = 0, 1, 2, \dots, m-1$.

- Linear Probing:** $h(k, i) = (h'(k) + i) \bmod m$
 It simply checks the next consecutive slot in a wrap-around manner. It is cache-friendly but suffers from *primary clustering*, where continuous blocks of occupied slots build up, slowing down future operations.
- Quadratic Probing:** $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$
 It probes non-adjacent slots using a quadratic polynomial (typically $c_1 = 0, c_2 = 1$). It eliminates primary clustering but suffers from *secondary clustering* (keys hashing to the same base slot follow the exact same gap sequence).
- Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$
 It utilizes a primary hash function $h_1(k)$ to find the starting slot, and a secondary hash function $h_2(k)$ to determine the step size. It minimizes both primary and secondary clustering, behaving similarly to a completely random permutation.

Q12. Compare linked lists and skip lists. Show that the expected space used by a skip list with n entries is $O(n)$. Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys:
 ... (15 marks) [CSE 203 | 2014–15]

Answer — Linked Lists vs. Skip Lists & Skip List Drawing

1. Comparison: Linked Lists vs. Skip Lists

- Structure:** A linked list is a linear 1D structure with single next/prev pointers. A skip list is a layered 2D structure built on top of linked lists with randomized probabilistic heights.
- Search Time:** Linked list requires $O(n)$ time to search. Skip list allows $O(\log n)$ expected search time by using upper layers to skip large segments.
- Space:** Linked list uses strictly $O(n)$ space (1 pointer per node). Skip list uses $O(n)$ expected space, but with a larger constant factor (average 2 pointers per node).
- Deterministic vs Randomized:** Linked lists have deterministic performance. Skip list performance is probabilistic (though highly bounded).

2. Expected Space used is $O(n)$: As proved previously, if the probability of promotion is $p = 1/2$, the expected number of nodes at level i is $n(1/2)^i$. Summing these over all levels from $i = 0$ to ∞ forms a geometric series $n \sum (1/2)^i = 2n$. Thus, the total expected space is $O(n)$.

3. Skip List Drawing: A skip list must maintain the keys in sorted order at all levels. **Sorted Keys:** 15, 21, 28, 35, 40, 50, 65, 79, 84. Based on the table, the levels

(heads) for each key are: 15(1), 21(3), 28(0), 35(1), 40(4), 50(1), 65(2), 79(3), 84(2).
(Note: A key with h heads appears in levels 0 through h .)

L4	→		40						→ NULL	
L3	→	21		40		79			→ NULL	
L2	→	21		40		65	79	84	→ NULL	
L1	→	15	21	35	40	50	65	79	84 → NULL	
L0	→	15	21	28	35	40	50	65	79	84 → NULL

Q13. What makes a good hash function? Explain the three commonly used techniques to compute the probe sequences required for open addressing in a hash table. (15 marks) [CSE 203 | 2014–15]

Answer — Good Hash Function & Open Addressing

What makes a good hash function? A good hash function should satisfy several key properties:

- **Uniform Distribution:** It should satisfy the Simple Uniform Hashing Assumption (SUHA), distributing keys evenly across all slots to minimize collisions.
- **Efficiency:** It must be fast to compute, ideally in $O(1)$ time, so it does not become a bottleneck.
- **Determinism:** The same key must always hash to the exact same bucket.
- **Uses all Input Data:** It should take into account the entire key (e.g., all characters in a string) so that closely related keys hash to very different locations (the avalanche effect).

Three Commonly Used Open Addressing Techniques: For a hash table of size m , open addressing generates a probe sequence $h(k, i)$ for attempts $i = 0, 1, \dots, m - 1$.

(a) **Linear Probing:** $h(k, i) = (h'(k) + i) \bmod m$

- *Mechanism:* If a collision occurs, sequentially check the next slot, wrapping around to 0 upon reaching the end.
- *Pros/Cons:* Great cache locality, but highly susceptible to *primary clustering* (long contiguous runs of occupied slots).

(b) **Quadratic Probing:** $h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$

- *Mechanism:* The gap between probed slots increases quadratically (e.g., +1, +4, +9).

- *Pros/Cons*: Eliminates primary clustering, but creates *secondary clustering* because keys with the same initial hash follow the exact same probe path.

(c) **Double Hashing**: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

- *Mechanism*: Uses a secondary hash function $h_2(k)$ to determine the step size.
- *Pros/Cons*: Avoids both primary and secondary clustering. The step size varies per key, leading to probe sequences that behave almost like random permutations.

Q14. Write algorithms for the search, insertion and deletion operations of skip lists. Show that the expected search, insertion and deletion time is $O(\log n)$ in a skip list with n entries. Also show that the expected space used is $O(n)$. **(18 marks)** [CSE 203 | 2013–14]

Answer — Skip List Algorithms & Complexity

1. Algorithms: Let each node have an array of forward pointers 'forward[i]' pointing to the next node at level 'i'. Let 'maxLevel' be the current top level of the list.

Search(searchKey):

```
current = header
for i = maxLevel down to 0:
    while current.forward[i] != NULL and current.forward[i].key < searchKey:
        current = current.forward[i]
current = current.forward[0]
if current != NULL and current.key == searchKey:
    return current.value
return "Not Found"
```

Insert(searchKey, newValue):

```
update[maxLevel+1] // Array to track nodes where pointers need updating
current = header
for i = maxLevel down to 0:
    while current.forward[i] != NULL and current.forward[i].key < searchKey:
        current = current.forward[i]
    update[i] = current
current = current.forward[0]
if current.key == searchKey:
    current.value = newValue // Update existing
else:
    lvl = randomLevel() // Coin flips
    if lvl > maxLevel:
```

```

    for i = maxLevel + 1 to lvl:
        update[i] = header
    maxLevel = lvl
    newNode = createNode(lvl, searchKey, newValue)
    for i = 0 to lvl:
        newNode.forward[i] = update[i].forward[i]
        update[i].forward[i] = newNode

```

Delete(searchKey):

```

update[maxLevel+1]
current = header
for i = maxLevel down to 0:
    while current.forward[i] != NULL and current.forward[i].key < searchKey:
        current = current.forward[i]
    update[i] = current
current = current.forward[0]
if current.key == searchKey:
    for i = 0 to maxLevel:
        if update[i].forward[i] != current:
            break
        update[i].forward[i] = current.forward[i]
    free(current)
while maxLevel > 0 and header.forward[maxLevel] == NULL:
    maxLevel = maxLevel - 1

```

2. Expected Time Complexity $O(\log n)$:

- **Height:** The expected height of a skip list is bounded by $O(\log n)$ because elements are promoted with probability p . The probability of an element reaching level $\log_{1/p} n$ drops off exponentially.
- **Horizontal Steps:** The expected number of horizontal steps on each level is $\frac{1-p}{p}$ (the expectation of a geometric distribution). For $p = 1/2$, this is constant $O(1)$.
- Search, Insertion, and Deletion all traverse this same path. Therefore, the expected time is $O(\log n) \times O(1) = O(\log n)$.

3. Expected Space Complexity $O(n)$:

- Total expected nodes equals the sum of expected nodes across all levels: $n \sum_{i=0}^{\infty} p^i$.
- For $p = 1/2$, this geometric series sums to $n \cdot (\frac{1}{1-0.5}) = 2n$.
- Since $2n$ is linearly proportional to n , the expected space is $O(n)$.

- Q15.** Draw the 11-item hash table that results from using the hash function $h(i) = (2i + 5) \bmod 11$ to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5, assuming collisions are handled by double hashing using a secondary hash function $h'(k) = 7 - (k \bmod 7)$.
(12 marks) [CSE 203 | 2013–14]

Answer — Double Hashing Illustration

Let $m = 11$. Probe sequence: $H(k, i) = (h(k) + i \cdot h'(k)) \bmod 11$.

Step-by-step Trace:

- (a) **12:** $h(12) = (24 + 5) \bmod 11 = 7$. Slot 7 empty. $\rightarrow \mathbf{T[7] = 12}$
- (b) **44:** $h(44) = (88 + 5) \bmod 11 = 5$. Slot 5 empty. $\rightarrow \mathbf{T[5] = 44}$
- (c) **13:** $h(13) = (26 + 5) \bmod 11 = 9$. Slot 9 empty. $\rightarrow \mathbf{T[9] = 13}$
- (d) **88:** $h(88) = (176 + 5) \bmod 11 = 5$ (Collision).
 $h'(88) = 7 - (88 \bmod 7) = 7 - 4 = 3$.
 $i = 1 : (5 + 3) \bmod 11 = 8$. Slot 8 empty. $\rightarrow \mathbf{T[8] = 88}$
- (e) **23:** $h(23) = (46 + 5) \bmod 11 = 7$ (Collision).
 $h'(23) = 7 - (23 \bmod 7) = 7 - 2 = 5$.
 $i = 1 : (7 + 5) \bmod 11 = 1$. Slot 1 empty. $\rightarrow \mathbf{T[1] = 23}$
- (f) **94:** $h(94) = (188 + 5) \bmod 11 = 6$. Slot 6 empty. $\rightarrow \mathbf{T[6] = 94}$
- (g) **11:** $h(11) = (22 + 5) \bmod 11 = 5$ (Collision with 44).
 $h'(11) = 7 - (11 \bmod 7) = 7 - 4 = 3$.
 $i = 1 : (5 + 3) \bmod 11 = 8$ (Collision with 88).
 $i = 2 : (8 + 3) \bmod 11 = 0$. Slot 0 empty. $\rightarrow \mathbf{T[0] = 11}$
- (h) **39:** $h(39) = (78 + 5) \bmod 11 = 6$ (Collision with 94).
 $h'(39) = 7 - (39 \bmod 7) = 7 - 4 = 3$.
 $i = 1 : (6 + 3) \bmod 11 = 9$ (Collision with 13).
 $i = 2 : (9 + 3) \bmod 11 = 1$ (Collision with 23).
 $i = 3 : (1 + 3) \bmod 11 = 4$. Slot 4 empty. $\rightarrow \mathbf{T[4] = 39}$
- (i) **20:** $h(20) = (40 + 5) \bmod 11 = 1$ (Collision with 23).
 $h'(20) = 7 - (20 \bmod 7) = 7 - 6 = 1$.
 $i = 1 : (1 + 1) \bmod 11 = 2$. Slot 2 empty. $\rightarrow \mathbf{T[2] = 20}$
- (j) **16:** $h(16) = (32 + 5) \bmod 11 = 4$ (Collision with 39).
 $h'(16) = 7 - (16 \bmod 7) = 7 - 2 = 5$.
 $i = 1 : (4 + 5) \bmod 11 = 9$ (Collision with 13).
 $i = 2 : (9 + 5) \bmod 11 = 3$. Slot 3 empty. $\rightarrow \mathbf{T[3] = 16}$
- (k) **5:** $h(5) = (10 + 5) \bmod 11 = 4$ (Collision with 39).
 $h'(5) = 7 - (5 \bmod 7) = 7 - 5 = 2$.
 $i = 1 : (4 + 2) \bmod 11 = 6$ (Collision with 94).
 $i = 2 : (6 + 2) \bmod 11 = 8$ (Collision with 88).
 $i = 3 : (8 + 2) \bmod 11 = 10$. Slot 10 empty. $\rightarrow \mathbf{T[10] = 5}$

Final Hash Table Layout:

Idx	0	1	2	3	4	5	6	7	8	9	10
Key	11	23	20	16	39	44	94	12	88	13	5

Q16. Compare linked lists and skip lists. (5 marks)

[CSE 203 | 2013–14]

Answer — Linked Lists vs. Skip Lists

- **Structure:** A linked list is a simple, deterministic 1D linear structure. A skip list is a randomized, multi-level 2D structure built on top of multiple linked lists to create "express lanes."
- **Search Complexity:** Searching a linked list takes worst-case $O(n)$ time. A skip list takes expected $O(\log n)$ time because upper layers allow bypassing large portions of the list.
- **Space Complexity:** A linked list is strictly $O(n)$ using exactly one pointer per node. A skip list is probabilistically $O(n)$ but uses more memory overhead (averaging roughly 2 forward pointers per node).
- **Operations:** Insertions and deletions in a linked list require $O(n)$ time to find the spot, then $O(1)$ to rewire. Skip lists find the spot and rewire in expected $O(\log n)$ time using probabilistic coin flips to determine node height.

Q17. Write pseudocode for finding a key, inserting a key, and deleting a key in a hash table using quadratic probing. Explain your method with an example. (12 marks)

[CSE 203 | 2012–13]

Answer — Quadratic Probing Pseudocode & Example

Let the table be 'T' of size 'm'. Slots can be 'EMPTY', 'DELETED', or contain a 'key'. Let $h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$.

1. Pseudocode:

Insert(k):

```

for i = 0 to m-1:
    idx = h(k, i)
    if T[idx] == EMPTY or T[idx] == DELETED:
        T[idx] = k
        return idx
return "Error: Hash Table Overflow"
```

Search(k):

```

for i = 0 to m-1:
    idx = h(k, i)
    if T[idx] == EMPTY:
        return "Not Found"
```

```

        if T[idx] == k:
            return idx
        // If T[idx] == DELETED, continue looping
    return "Not Found"

```

```

Delete(k):
    idx = Search(k)
    if idx != "Not Found":
        T[idx] = DELETED
        return "Success"
    return "Not Found"

```

2. Explanation with Example: We use a special ‘DELETED’ flag (often called a tombstone) so that deleting an element doesn’t break the probe chain for other elements that collided with it earlier.

Example: Let $m = 5$, hash $h'(k) = k \bmod 5$, quadratic function $h(k, i) = (h'(k) + i^2) \bmod 5$.

- (a) **Insert(7):** $h(7, 0) = 2 \bmod 5 = 2$. $T[2]$ is EMPTY. **$T[2] = 7$.**
- (b) **Insert(12):** $h(12, 0) = 2 \bmod 5 = 2$ (Collision with 7).
 $i = 1 : (2 + 1) \bmod 5 = 3$. $T[3]$ is EMPTY. **$T[3] = 12$.**
- (c) **Delete(7):** ‘Search(7)’ finds it at index 2. Set **$T[2] = \text{DELETED}$** .
- (d) **Search(12):** $h(12, 0) = 2$. $T[2]$ is ‘DELETED’. The algorithm *does not stop* (unlike if it were ‘EMPTY’). It increments $i = 1$, checks $h(12, 1) = 3$, finds 12, and successfully returns index 3.

Q18. Prove that in a hash table in which collisions are resolved by chaining, a successful search takes average-case time $\Theta(1 + \alpha)$, under the assumption of uniform hashing, where α is the load factor. **(10 marks)** [CSE 203 | 2012–13]

Answer — Average-case Successful Search in Chaining

Let n be the number of items, m be the number of slots, and $\alpha = \frac{n}{m}$ be the load factor. We assume that the searched element is equally likely to be any of the n elements stored in the hash table.

Proof:

- (a) Suppose the i -th element, x_i , was inserted at the end of its chain.
- (b) When searching for x_i , we must compute the hash function in $\Theta(1)$ time and traverse the list to find x_i . The number of elements we traverse before finding x_i is exactly the number of elements inserted *after* x_i into the same slot.
- (c) By the **Simple Uniform Hashing Assumption (SUHA)**, any element inserted after x_i has a probability of $\frac{1}{m}$ of hashing to the same slot as x_i .

- (d) There are $(n - i)$ elements inserted after x_i . The expected number of elements inserted after x_i in its slot is $\frac{n-i}{m}$.
- (e) To find the expected number of nodes examined for a successful search over all n elements, we average over all i :

$$\begin{aligned} E[\text{nodes checked}] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{1}{nm} \left(\frac{n(n-1)}{2} \right) = 1 + \frac{n-1}{2m} \end{aligned}$$

- (f) We substitute the load factor $\alpha = \frac{n}{m}$:

$$1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{1}{2m}$$

- (g) Since $m \geq 1$, the term $\frac{1}{2m}$ is a small bounded constant. Including the $\Theta(1)$ time to compute the hash function, the average-case running time evaluates to $\Theta\left(1 + \frac{\alpha}{2}\right) = \Theta(1 + \alpha)$.

Q19. Insert the keys 18, 41, 22, 44, 59, 32, 31, 73 into a hash table of size 13 using double hashing with

$$h_1(k) = k \bmod 13, \quad h_2(k) = 8 - (k \bmod 8).$$

Draw the resulting hash table. **(13 marks)**

[CSE 203 | 2012–13]

Answer — Double Hashing Table Construction

The probe sequence is given by $H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 13$.

Insertion Trace:

- | | |
|---|------------------------------------|
| (a) 18: $h_1(18) = 18 \bmod 13 = 5$. Slot 5 is empty. | $\Rightarrow$ Table[5] = 18 |
| (b) 41: $h_1(41) = 41 \bmod 13 = 2$. Slot 2 is empty. | $\Rightarrow$ Table[2] = 41 |
| (c) 22: $h_1(22) = 22 \bmod 13 = 9$. Slot 9 is empty. | $\Rightarrow$ Table[9] = 22 |
| (d) 44: $h_1(44) = 44 \bmod 13 = 5$ (Collision with 18).
$h_2(44) = 8 - (44 \bmod 8) = 8 - 4 = 4$.
$i = 1 : (5 + 4) \bmod 13 = 9$ (Collision with 22).
$i = 2 : (9 + 4) \bmod 13 = 13 \equiv 0$. Slot 0 is empty. | $\Rightarrow$ Table[0] = 44 |
| (e) 59: $h_1(59) = 59 \bmod 13 = 7$. Slot 7 is empty. | $\Rightarrow$ Table[7] = 59 |
| (f) 32: $h_1(32) = 32 \bmod 13 = 6$. Slot 6 is empty. | $\Rightarrow$ Table[6] = 32 |
| (g) 31: $h_1(31) = 31 \bmod 13 = 5$ (Collision with 18).
$h_2(31) = 8 - (31 \bmod 8) = 8 - 7 = 1$.
$i = 1 : (5 + 1) \bmod 13 = 6$ (Collision with 32). | |

$i = 2 : (6 + 1) \bmod 13 = 7$ (Collision with 59).

$i = 3 : (7 + 1) \bmod 13 = 8$. Slot 8 is empty.

$\Rightarrow \text{Table}[8] = 31$

(h) **73**: $h_1(73) = 73 \bmod 13 = 8$ (Collision with 31).

$h_2(73) = 8 - (73 \bmod 8) = 8 - 1 = 7$.

$i = 1 : (8 + 7) \bmod 13 = 15 \equiv 2$ (Collision with 41).

$i = 2 : (2 + 7) \bmod 13 = 9$ (Collision with 22).

$i = 3 : (9 + 7) \bmod 13 = 16 \equiv 3$. Slot 3 is empty.

$\Rightarrow \text{Table}[3] = 73$

Final Hash Table Layout:

Idx	0	1	2	3	4	5	6	7	8	9	10	11	12
Key	44	-	41	73	-	18	32	59	31	22	-	-	-

Q20. Consider inserting the keys 18, 41, 22, 44, 59, 32, 31, 73 into a hash table of length $m = 13$ using double hashing with

$$h_1(k) = k \bmod 13, \quad h_2(k) = 8 - (k \bmod 8).$$

Draw the resulting hash table. **(10 marks)**

[CSE 203 | 2011–12]

Answer — Double Hashing Table Construction (Duplicate)

This is identical to the previous question. The trace and the resulting hash table are exactly the same.

Final Hash Table Layout:

Idx	0	1	2	3	4	5	6	7	8	9	10	11	12
Key	44	-	41	73	-	18	32	59	31	22	-	-	-

Q21. What are the properties of a good hash function? **(4 marks)** [CSE 203 | 2011–12]

Answer — Properties of a Good Hash Function

A good hash function should possess the following properties:

- **Uniformity (SUHA):** It should distribute keys uniformly across the entire hash table. Every slot should have an equally likely chance of being chosen, minimizing the number of collisions.
- **Determinism:** The same input key must strictly yield the exact same hash value every time it is evaluated.
- **Efficiency:** The function must be simple and extremely fast to compute, strictly running in $O(1)$ time to avoid becoming a bottleneck during operations.
- **High Sensitivity (Avalanche Effect):** Minor changes in the input key

(like flipping a single bit) should result in a drastically different hash value to prevent clustered keys from colliding.

- **Data Inclusivity:** It should ideally use all parts of the key data rather than ignoring segments (e.g., considering all characters of a string, not just the first three).

BUET · CSE 105

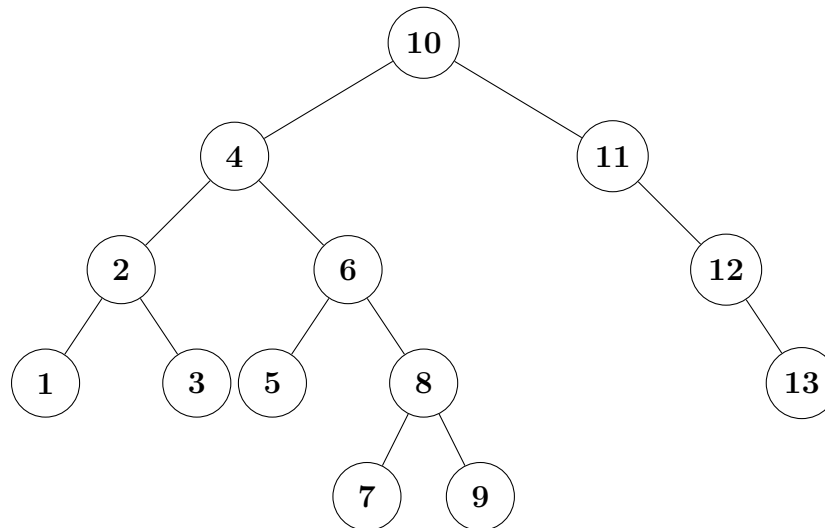
DSA

Advanced Heaps: Binomial & Fibonacci

Binomial Heap, Fibonacci Heap, Splay Trees

T18 — Advanced Heaps: Binomial & Fibonacci

- Q1.** [Splay tree given in figure] (i) Show the resulting tree after successively accessing keys 3 and 9 in the splay tree. (ii) Show the resulting tree after deleting key 6 following operations in (i). **(15 marks)** [CSE 207 | 2018–19]



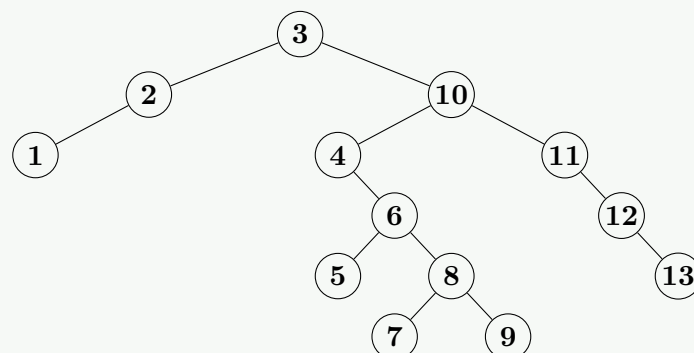
Answer — Splay Tree Operations

(i) Successively accessing keys 3 and 9:

1. Accessing Key 3:

- Path to 3: $10 \rightarrow 4 \rightarrow 2 \rightarrow 3$.
- **Step 1 (Zig-Zag at 2, 4):** Node 3 is a right child of 2, and 2 is a left child of 4. We perform a left rotation at 2, then a right rotation at 4. Node 3 becomes the left child of 10. Node 2 becomes the left child of 3, and 4 becomes the right child of 3.
- **Step 2 (Zig at 10):** Node 3 is a left child of 10 (root). We perform a right rotation at 10. Node 3 becomes the root.

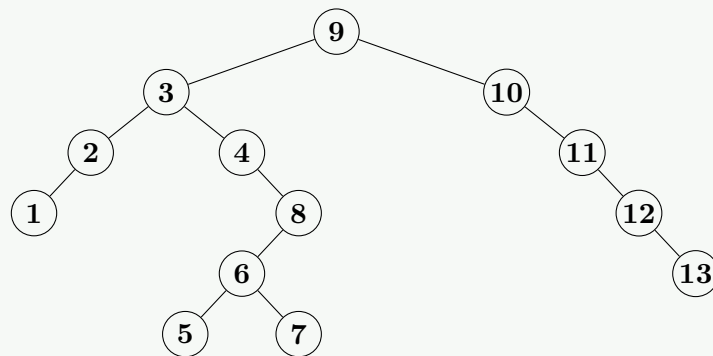
Tree after Access(3):



2. Accessing Key 9:

- Path to 9: $3 \rightarrow 10 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 9$.
- **Step 1 (Zig-Zig at 6, 8):** Node 9 is a right child of 8, and 8 is a right child of 6. We rotate left at 6, then left at 8. Node 9 becomes the right child of 4.
- **Step 2 (Zig-Zag at 4, 10):** Node 9 is a right child of 4, and 4 is a left child of 10. We rotate left at 4, then right at 10. Node 9 becomes the right child of 3.
- **Step 3 (Zig at 3):** Node 9 is a right child of 3. We rotate left at 3. Node 9 becomes the new root.

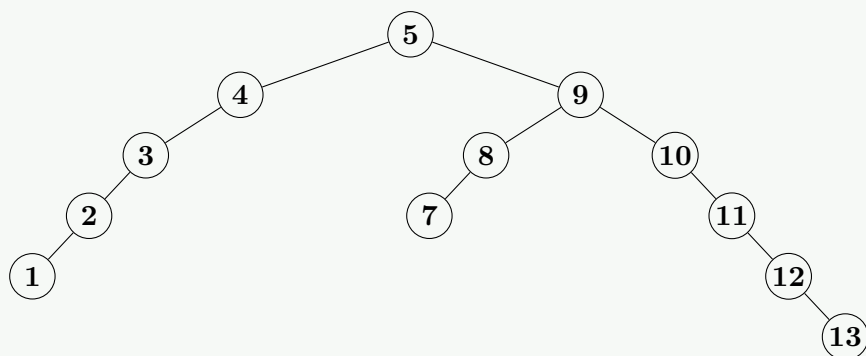
Tree after Access(9):



(ii) Deleting key 6:

- **Step 1 (Splay 6 to Root):** Access 6. Path is $9 \rightarrow 3 \rightarrow 4 \rightarrow 8 \rightarrow 6$.
 - Zig-Zag (at 4, 8): Rotate right at 8, left at 4. Node 6 attaches to 3.
 - Zig-Zag (at 3, 9): Rotate left at 3, right at 9. Node 6 becomes the root.
- **Step 2 (Split & Find Max):** Delete root 6. The Left Subtree L is rooted at 3, and the Right Subtree R is rooted at 9. Find the maximum element in L (which is 5) and splay it to the root of L .
- **Step 3 (Join):** After splaying 5 in L , node 5 becomes the new root and has no right child. Attach R (node 9) as the right child of 5.

Tree after Deleting 6:



Q2. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial heap B_k of order k . **(10 marks)** [CSE 207 | 2018–19]

Answer — Binomial Heap Depth Proof

Proof by Mathematical Induction on k :

Base Case ($k = 0$): A binomial tree of order 0, B_0 , consists of exactly one node at depth 0. Using the formula for $i = 0$: $\binom{0}{0} = 1$. The formula holds for the base case.

Inductive Hypothesis: Assume that for a binomial tree of order $k - 1$ (B_{k-1}), the number of nodes at depth i is exactly $\binom{k-1}{i}$ for all $0 \leq i \leq k - 1$.

Inductive Step: We must prove that the formula holds for B_k . By the recursive definition of a binomial tree, a B_k tree is constructed by linking two B_{k-1} trees. Specifically, the root of one B_{k-1} tree (let's call it H_2) is made the leftmost child of the root of the other B_{k-1} tree (let's call it H_1).

When H_2 is attached as a child of H_1 's root:

- The depth of every node in H_1 remains unchanged.
- The depth of every node in H_2 strictly increases by 1.

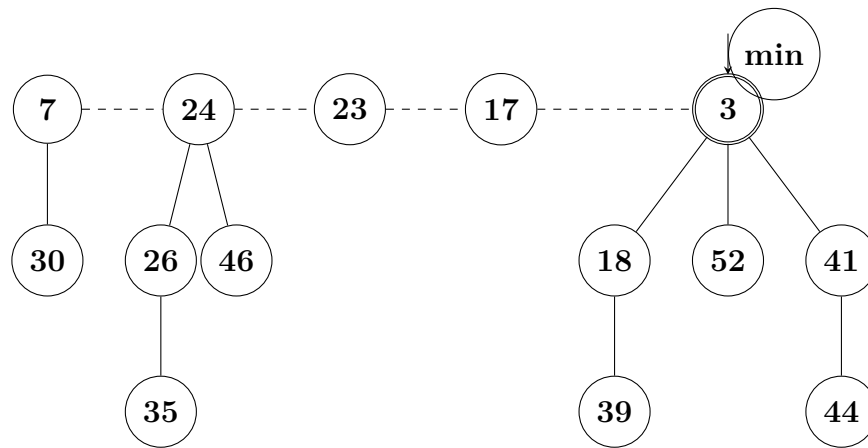
Therefore, to find the total number of nodes at depth i in the newly formed B_k , we sum: 1. The number of nodes at depth i in H_1 , which is $\binom{k-1}{i}$ (by the inductive hypothesis). 2. The number of nodes at depth $i - 1$ in H_2 , which is $\binom{k-1}{i-1}$ (since their depth will increase by 1 to reach depth i).

Total nodes at depth i in B_k = Nodes at depth i in H_1 + Nodes at depth $i - 1$ in H_2

$$\text{Total} = \binom{k-1}{i} + \binom{k-1}{i-1}$$

By **Pascal's Identity**, we know that $\binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}$. Thus, there are exactly $\binom{k}{i}$ nodes at depth i in B_k . The proof is complete.

Q3. [Fibonacci heap given in figure] Show the Fibonacci heap that results when the minimum node (3) is deleted. Show all intermediate steps. **(15 marks)** [CSE 207 | 2018–19]



Answer — Fibonacci Heap Extract-Min

Step 1: Remove Min and Add Children to Root List

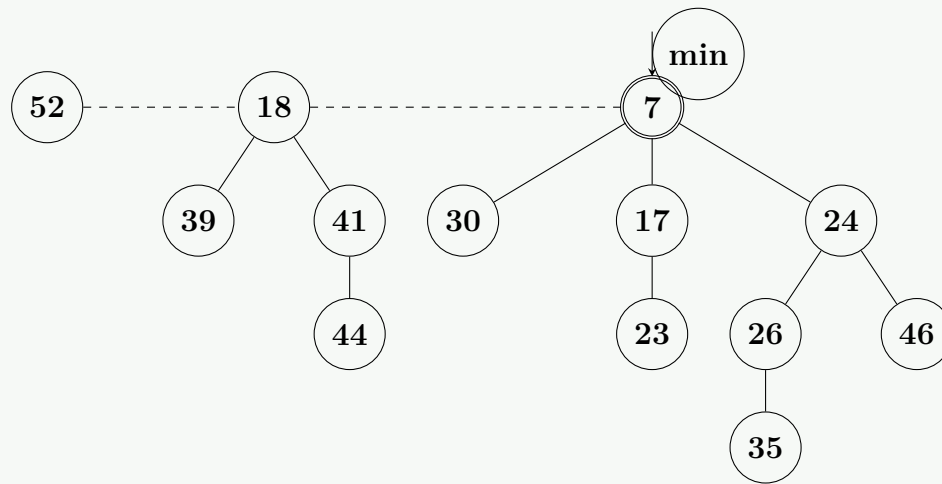
- We remove the minimum node (3).
- We take all its children (18, 52, 41) and append them to the root list.
- **Unconsolidated Root List:** 7 – 24 – 23 – 17 – 18 – 52 – 41.

Step 2: Consolidate the Root List We iterate through the root list and merge trees of the same degree using an auxiliary array $A[0..D]$.

- Process 7 (deg 1):** $A[1] = 7$.
- Process 24 (deg 2):** $A[2] = 24$.
- Process 23 (deg 0):** $A[0] = 23$.
- Process 17 (deg 0):** Collision in $A[0]$ with 23.
 - $17 < 23$, so 23 becomes a child of 17. Node 17 now has degree 1.
 - Collision in $A[1]$ with 7. $7 < 17$, so 17 becomes a child of 7. Node 7 now has degree 2.
 - Collision in $A[2]$ with 24. $7 < 24$, so 24 becomes a child of 7. Node 7 now has degree 3.
 - Place in array: $A[3] = 7$. ($A[0], A[1], A[2]$ are now empty).
- Process 18 (deg 1):** $A[1] = 18$.
- Process 52 (deg 0):** $A[0] = 52$.
- Process 41 (deg 1):** Collision in $A[1]$ with 18.
 - $18 < 41$, so 41 becomes a child of 18. Node 18 now has degree 2.
 - Place in array: $A[2] = 18$.

Step 3: Final Array and New Minimum The array A holds the consolidated roots: $A[0] = 52$, $A[2] = 18$, $A[3] = 7$. The new minimum is 7.

Resulting Fibonacci Heap:



Q4. State the running times of the following operations for Binomial heap and Fibonacci heap: (i) Union, (ii) Find min, (iii) Extract min, (iv) Decrease key. **(5 marks)** [CSE 207 | 2018–19]

Answer — Time Complexities

The standard time complexities for a heap containing n elements are as follows:

Operation	Binomial Heap (Worst-Case)	Fibonacci Heap (Amortized)
(i) Union	$O(\log n)$	$O(1)$
(ii) Find Min	$O(1)^*$	$O(1)$
(iii) Extract Min	$O(\log n)$	$O(\log n)$
(iv) Decrease Key	$O(\log n)$	$O(1)$

**Note: Find Min is $O(1)$ for a Binomial Heap if a global pointer to the minimum root is maintained; otherwise, scanning the root list takes $O(\log n)$ time.*

Q5. Write down the running times of the following operations for both Binomial heap and Fibonacci heap: (i) Insert key, (ii) Find min, (iii) Extract min, (iv) Union of two heaps, (v) Decrease key. A friend claims to have designed a mergeable heap with $O(1)$ time for all operations above. Would you believe it? Justify. **(6+6 marks)** [CSE 207 | 2017–18]

Answer — Running Times & $O(1)$ Claim**1. Running Times**

Operation	Binomial Heap (Worst-Case)	Fibonacci Heap (Amortized)
(i) Insert	$O(\log n)$	$O(1)$
(ii) Find Min	$O(1)^*$	$O(1)$
(iii) Extract Min	$O(\log n)$	$O(\log n)$
(iv) Union	$O(\log n)$	$O(1)$
(v) Decrease Key	$O(\log n)$	$O(1)$

**Note: Find Min in a Binomial Heap is $O(1)$ only if an explicit pointer to the minimum root is maintained; otherwise it is $O(\log n)$.*

2. Analysis of the $O(1)$ Claim I would **not believe** the claim.

Justification: If a mergeable heap could perform all the above operations—specifically **Insert** and **Extract Min**—in $O(1)$ worst-case or amortized time, it would violate the theoretical lower bound for comparison-based sorting algorithms.

We could use this hypothetical heap to sort an array of n elements as follows:

- Insert all n elements into the heap. If each insertion takes $O(1)$ time, this step takes a total of $O(n)$ time.
- Extract the minimum element n times. If each extraction takes $O(1)$ time, this step also takes a total of $O(n)$ time.

The entire sequence would sort n elements in $O(n)$ time. However, it is a proven mathematical fact that any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ time in the worst case. Therefore, **Extract Min** must strictly take at least $\Omega(\log n)$ time to preserve this lower bound.

Q6. Write the algorithm for extracting the minimum key from a Fibonacci heap. (8 marks) [CSE 207 | 2017–18]

Answer — Fibonacci Heap Extract-Min Algorithm

The **Extract-Min** operation in a Fibonacci heap consists of removing the minimum node, promoting its children to the root list, and then consolidating the root list so that no two roots have the same degree.

Algorithm: FIB-HEAP-EXTRACT-MIN(H)

```

1  z = H.min
2  if z != NULL:
```

```

3      // Promote all children of the extracted node
4      for each child x of z:
5          add x to the root list of H
6          x.p = NULL
7
8      // Remove z from the root list
9      remove z from the root list of H
10
11     // Update min pointer and consolidate
12     if z == z.right:          // z was the only node on the root list
13         H.min = NULL
14     else:
15         H.min = z.right      // Temporary pointer, fixed during consolidation
16         CONSOLIDATE(H)
17
18     H.n = H.n - 1
19 return z

```

Auxiliary Consolidation Routine: CONSOLIDATE(H)

```

1  let A[0 .. D(H.n)] be a new array, all initialized to NULL
2  for each node w in the root list of H:
3      x = w
4      d = x.degree
5      while A[d] != NULL:
6          y = A[d]
7          if x.key > y.key:
8              exchange x with y
9          FIB-HEAP-LINK(H, y, x)    // Make y a child of x
10         A[d] = NULL
11         d = d + 1
12     A[d] = x
13
14 // Reconstruct the root list from array A
15 H.min = NULL
16 for i = 0 to D(H.n):
17     if A[i] != NULL:
18         add A[i] to the root list of H
19         if H.min == NULL or A[i].key < H.min.key:
20             H.min = A[i]

```

Q7. What is the main objective of splay trees? Do splay trees explicitly try to balance the height? (5 marks) [CSE 207 | 2017–18]

Answer — Splay Tree Objective & Balancing

Main Objective: The primary objective of a splay tree is to optimize the search process by taking advantage of **temporal locality** (the principle of locality of reference). It operates on the assumption that if an element is accessed once, it is highly likely to be accessed again in the near future. By automatically moving the accessed element to the root (via a process called "splaying"), subsequent accesses to that same element become extraordinarily fast ($O(1)$). This ensures that any sequence of m operations strictly takes $O(m \log n)$ time, providing an amortized running time of $O(\log n)$ per operation.

Do they explicitly try to balance the height? No. Unlike AVL or Red-Black trees, splay trees do *not* explicitly maintain any height balance factors, node colors, or strict structural constraints. They completely lack explicit balancing logic. Instead, the heuristic of blindly splaying every accessed node to the root naturally restructures the tree, keeping it "amortized balanced" over time. A splay tree can theoretically devolve into a completely unbalanced linked list (worst-case $O(n)$ height), but the splaying operation ensures that expensive worst-case operations are naturally followed by cheap operations.

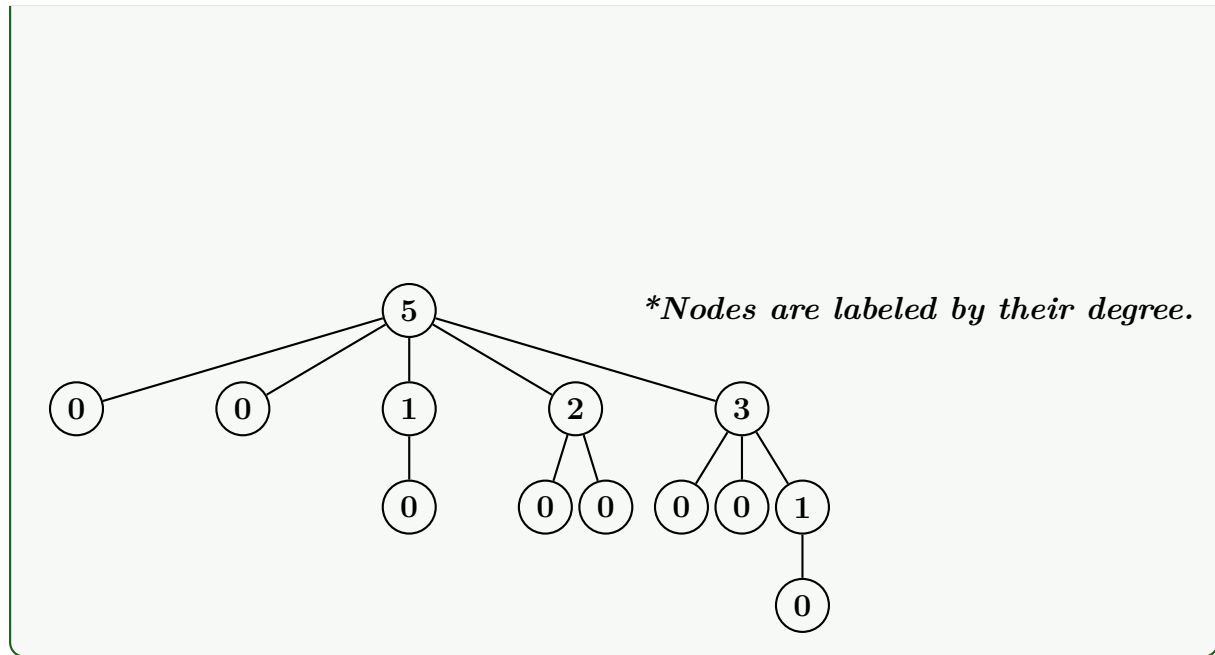
Q8. What is a maximally damaged tree in the context of Fibonacci heaps? Draw the maximally damaged tree of order 5 (degree of root is 5). **(4+6 marks)** [CSE 207 | 2017–18]

Answer — Maximally Damaged Fibonacci Tree

Definition: In the context of Fibonacci heaps, a **maximally damaged tree** (often denoted as T_k , analogous to a Fibonacci tree) is a tree that has lost the absolute maximum number of nodes possible while still strictly retaining its root degree of k without violating the structural invariants of the heap.

The structural invariant states that the i -th child of a node (ordered by when they were linked) must have a degree of at least $i - 2$. Therefore, to minimize the number of nodes (i.e., cause maximal damage), the k children of the root must possess exactly the minimum allowed degrees: $0, 0, 1, 2, \dots, k - 2$. This results in a tree of order k having exactly F_{k+2} nodes, where F_x is the x -th Fibonacci number.

Maximally Damaged Tree of Order 5 (T_5): For a root with degree 5, its 5 children must have degrees $0, 0, 1, 2, 3$. The total number of nodes is $F_{5+2} = F_7 = 13$ nodes.



Q9. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial tree B_k . (5 marks) [CSE 207 | 2017–18]

Answer — Binomial Tree Depth Proof

We will prove this using mathematical induction on the order of the binomial tree, k . Let $D(k, i)$ denote the number of nodes at depth i in a binomial tree B_k . We want to prove $D(k, i) = \binom{k}{i}$.

Base Case ($k = 0$): A binomial tree of order 0, B_0 , consists of exactly one node at depth 0. Using the formula for $i = 0$: $\binom{0}{0} = 1$. The formula holds true for the base case.

Inductive Hypothesis: Assume that the formula holds for a binomial tree of order $k - 1$. That is, for B_{k-1} , the number of nodes at depth i is exactly $\binom{k-1}{i}$ for all $0 \leq i \leq k - 1$.

Inductive Step: We must prove that the formula holds for B_k . By the recursive definition of a binomial tree, B_k is constructed by linking two B_{k-1} trees. Specifically, the root of one B_{k-1} tree (T_2) is made the leftmost child of the root of the other B_{k-1} tree (T_1).

When T_2 is attached as a child of T_1 's root:

- The depth of every node originating from T_1 remains completely unchanged.
- The depth of every node originating from T_2 strictly increases by exactly 1.

Therefore, to find the total number of nodes at depth i in the newly formed B_k , we must sum: 1. The nodes that were already at depth i in T_1 , which is $\binom{k-1}{i}$ (by the inductive hypothesis). 2. The nodes that were at depth $i - 1$ in T_2 , which is $\binom{k-1}{i-1}$, since their depth is pushed down by 1.

Mathematically, the total nodes at depth i in B_k is:

$$D(k, i) = \binom{k-1}{i} + \binom{k-1}{i-1}$$

By **Pascal's Identity** (a fundamental property of combinations), we know that:

$$\binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}$$

Thus, there are exactly $\binom{k}{i}$ nodes at depth i in B_k , completing the proof.

BUET · CSE 105

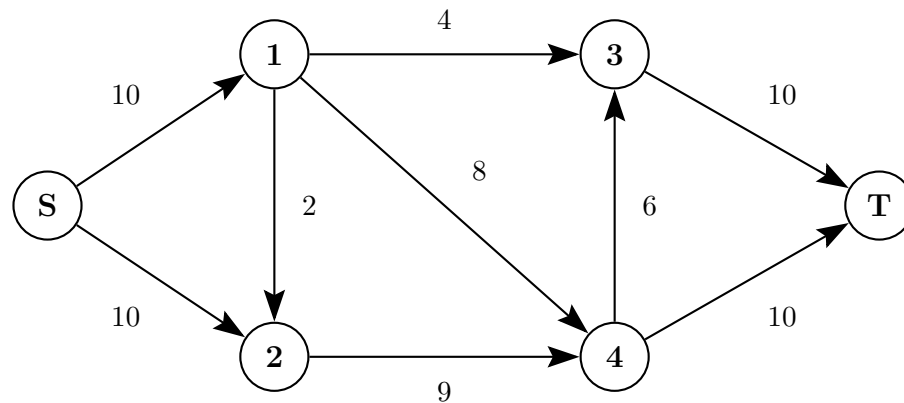
DSA

Network Flow & Bipartite Matching

Max-Flow Min-Cut, Ford-Fulkerson, Bipartite Matching

T19 — Network Flow & Bipartite Matching

Q1. [Flow network given in figure] Find the value of the maximum flow by drawing residual networks at each step. Draw the maximum flow network and identify the min-cut. **(15 marks)** [CSE 207 | 2018–19]



Answer: Maximum Flow & Min-Cut (Q1)

1. Step-by-Step Augmentation (Ford-Fulkerson):

We start with an initial flow of 0 on all edges. We will find augmenting paths in the residual graph until no more paths from S to T exist.

- **Step 1: Path $S \rightarrow 1 \rightarrow 3 \rightarrow T$**

Bottleneck capacity = $\min(10, 4, 10) = 4$.

Update: Flow on $S \rightarrow 1$, $1 \rightarrow 3$, $3 \rightarrow T$ increases by 4. Residual capacities decrease by 4.

- **Step 2: Path $S \rightarrow 2 \rightarrow 4 \rightarrow T$**

Bottleneck capacity = $\min(10, 9, 10) = 9$.

Update: Flow on $S \rightarrow 2$, $2 \rightarrow 4$, $4 \rightarrow T$ increases by 9. Residual capacities decrease by 9.

- **Step 3: Path $S \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow T$**

Current residual capacities on this path: $S \rightarrow 1$ ($10-4=6$), $1 \rightarrow 4$ (8), $4 \rightarrow 3$ (6), $3 \rightarrow T$ ($10-4=6$).

Bottleneck capacity = $\min(6, 8, 6, 6) = 6$.

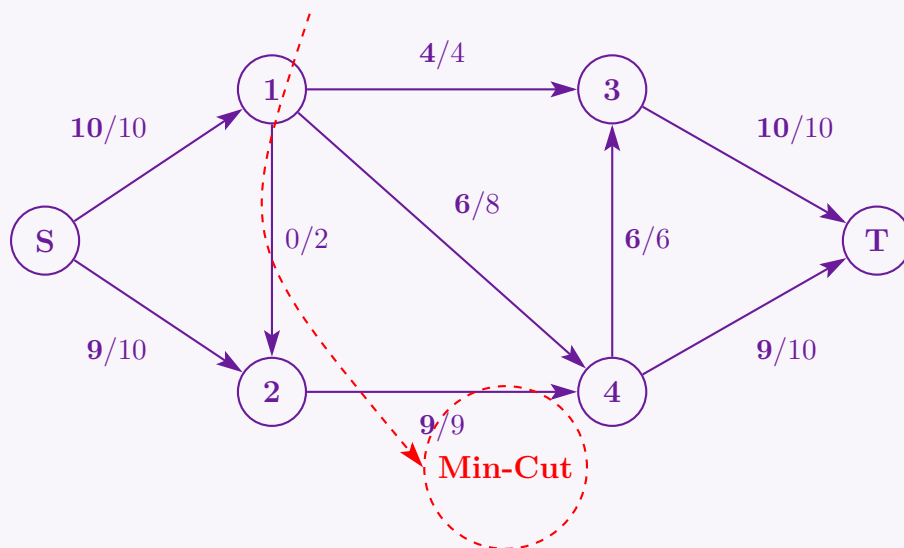
Update: Flow on these edges increases by 6.

Total Maximum Flow: $4 + 9 + 6 = 19$.

No more augmenting paths exist (e.g., $S \rightarrow 1$ has 0 residual capacity, and although $S \rightarrow 2$ has 1 left, $2 \rightarrow 4$ has 0 left).

2. Final Maximum Flow Network:

(Edge labels are shown as Flow / Capacity)



3. Min-Cut Identification:

The minimum cut separates the vertices into two sets (A, B) based on reachability from S in the final residual graph.

Vertices reachable from S (residual capacity > 0): $A = \{S, 2\}$ (since $S \rightarrow 2$ has $10 - 9 = 1$ capacity left, but from 2 we can't reach anywhere else).

Remaining vertices: $B = \{1, 3, 4, T\}$.

Min-Cut Capacity: Sum of capacities of original edges from A to B .

$C(A, B) = C(S, 1) + C(2, 4) = 10 + 9 = 19$.

This matches the Max Flow (Max-Flow Min-Cut Theorem).

Q2. Define a flow network. What is an augmenting path in a flow network? Explain with illustrative examples. **(3+3 marks)** [CSE 207 | 2017–18]

Answer: Flow Network & Augmenting Path (Q2)

Flow Network:

A flow network is a directed graph $G = (V, E)$ where each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. There are two special vertices: a **source** s (produces flow) and a **sink** t (consumes flow). A valid flow must satisfy:

- Capacity Constraint:** The flow $f(u, v)$ on any edge cannot exceed its capacity: $0 \leq f(u, v) \leq c(u, v)$.
- Flow Conservation:** For every vertex v except s and t , the total incoming flow equals the total outgoing flow: $\sum_u f(u, v) = \sum_w f(v, w)$.

Augmenting Path:

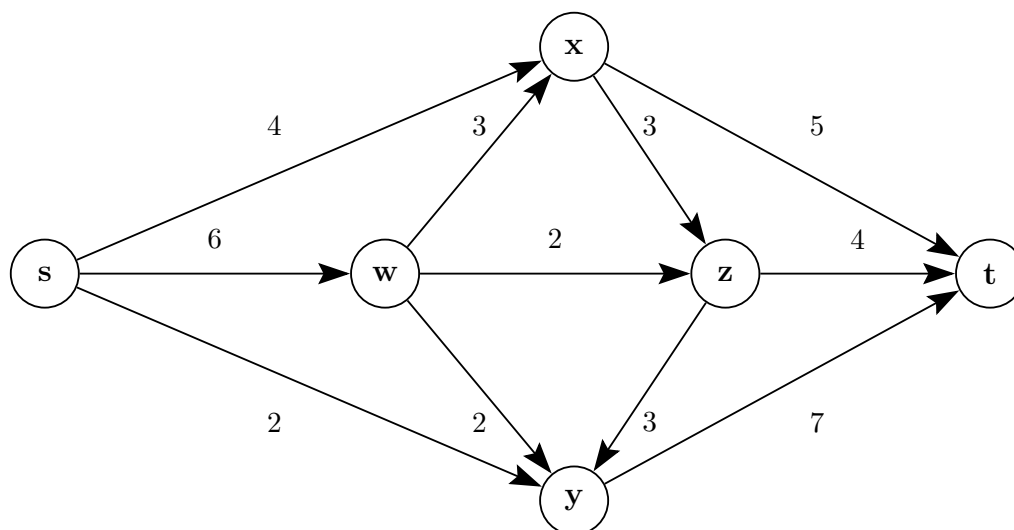
An augmenting path is a simple path from the source s to the sink t in the *residual network* G_f . It is a path where every edge has some strictly positive residual capacity ($c_f(u, v) > 0$), meaning we can push additional flow along this path without violating capacity constraints.

Illustrative Example: Suppose we have a path $s \xrightarrow{3/5} u \xrightarrow{2/4} t$. (Flow/Capacity)
In the residual network, the forward capacities are:

- $c_f(s, u) = 5 - 3 = 2$
- $c_f(u, t) = 4 - 2 = 2$

Since we can push at least $\min(2, 2) = 2$ more units of flow from s to t through u , the path $s \rightarrow u \rightarrow t$ is an **augmenting path**.

Q3. [Flow network given in figure] Find the maximum flow and the minimum cut using the Ford-Fulkerson algorithm. The vertex s is the source and the vertex t is the sink of the flow network. Integers represent the flow capacities of the edges. **(10 marks)**
[CSE 207 | 2017–18]



Answer: Max Flow & Min Cut using Ford-Fulkerson (Q3)

We iteratively find augmenting paths from s to t . Let's trace the flow augmentations:

Step	Augmenting Path	Bottleneck	Total Flow
1	$s \rightarrow x \rightarrow t$	$\min(4, 5) = 4$	4
2	$s \rightarrow y \rightarrow t$	$\min(2, 7) = 2$	6
3	$s \rightarrow w \rightarrow y \rightarrow t$	$\min(6, 2, 7 - 2) = 2$	8
4	$s \rightarrow w \rightarrow x \rightarrow t$	$\min(6 - 2, 3, 5 - 4) = 1$	9
5	$s \rightarrow w \rightarrow z \rightarrow t$	$\min(4 - 1, 2, 4) = 2$	11
6	$s \rightarrow w \rightarrow x \rightarrow z \rightarrow t$	$\min(1, 2, 3, 2) = 1$	12

Maximum Flow = 12. All edges originating from s ($s \rightarrow x, s \rightarrow w, s \rightarrow y$) are fully saturated ($4 + 6 + 2 = 12$). No more augmenting paths exist.

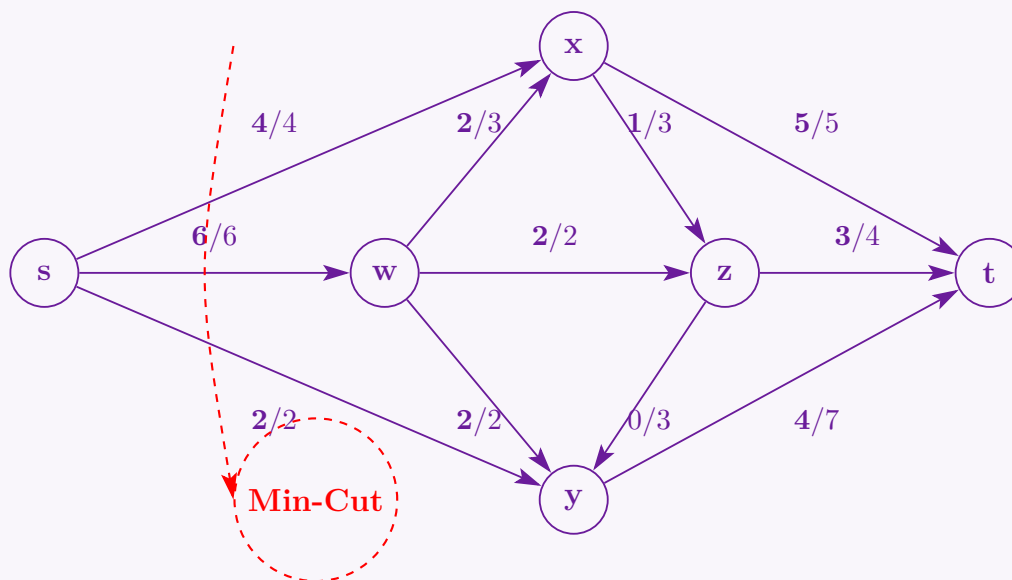
Minimum Cut:

To find the Min-Cut (A, B) , we find all vertices reachable from s in the final residual graph. Since $s \rightarrow w$, $s \rightarrow x$, and $s \rightarrow y$ are all at maximum capacity, their forward residual capacities are 0. Thus, no other vertices are reachable from s .

- $A = \{s\}$
- $B = \{w, x, y, z, t\}$

Min-Cut Capacity $= C(s, x) + C(s, w) + C(s, y) = 4 + 6 + 2 = 12$.

Final Flow Distribution Diagram:



Q4. Describe the minimum path length augmentation method of Edmonds and Karp and explain how it improves the time complexity of the Ford-Fulkerson algorithm. (10 marks) [CSE 207 | 2017–18]

Answer: Edmonds-Karp Minimum Path Length Augmentation (Q4)

Edmonds-Karp Method:

The Edmonds-Karp algorithm is a specific implementation of the Ford-Fulkerson method. Instead of choosing *any* arbitrary augmenting path in the residual graph, Edmonds-Karp strictly chooses the **shortest augmenting path** from the source to the sink, where "shortest" is defined by the **minimum number of edges** (unweighted path length). This is practically implemented using **Breadth-First Search (BFS)** on the residual graph to find the augmenting path.

How it improves Time Complexity:

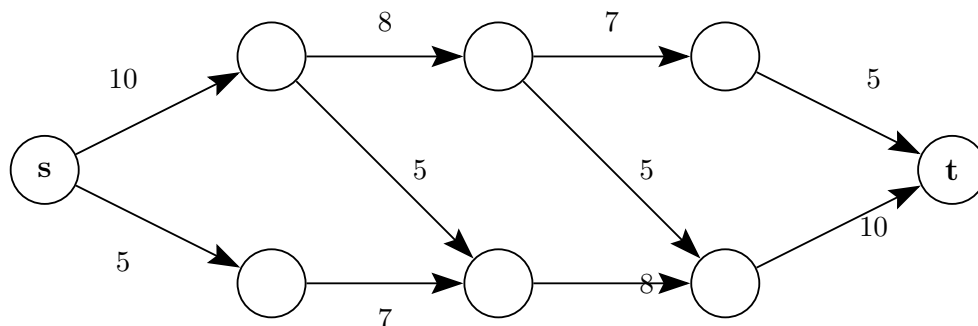
- **Issue with standard Ford-Fulkerson:** In the generic Ford-Fulkerson algorithm, if the capacities are large (or irrational), choosing poor augmenting paths can result in a tiny amount of flow being added in each step (e.g., adding 1 unit

back and forth between two parallel paths). The worst-case time complexity is $O(E \cdot |f^*|)$ where $|f^*|$ is the maximum flow, which is pseudo-polynomial and highly dependent on edge capacities.

- **Edmonds-Karp Bound:** By always choosing the shortest path (via BFS), Edmonds-Karp ensures that the length of the shortest augmenting path monotonically increases.
- **Mathematical Guarantee:** It can be proven that each edge can become a "bottleneck" (critical edge) at most $|V|/2$ times. Since there are $|E|$ edges, the total number of augmentations is strictly bounded by $O(V \cdot E)$.
- **Overall Complexity:** In each iteration, running BFS takes $O(E)$ time. With a maximum of $O(V \cdot E)$ iterations, the total time complexity becomes $O(V \cdot E^2)$.

Conclusion: Edmonds-Karp makes the algorithm **strongly polynomial** because its time complexity depends solely on the structure/size of the graph (number of vertices V and edges E), and is completely **independent of the maximum flow value** or the capacities of the edges.

Q5. [Flow network given in figure] Find the value of the maximum flow by drawing residual networks at each step. Draw the maximum flow network and identify the min-cut. (15 marks) [CSE 207 | 2016–17]



Answer: Maximum Flow & Min-Cut (Q1)

1. Step-by-Step Augmentation (Residual Networks):

We start with an initial flow of 0 on all edges. We find augmenting paths from s to t . (Note: I have assigned labels a, b, c, d, e, f to the intermediate nodes for clarity).

- **Step 1: Path** $s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$
 Bottleneck = $\min(10, 8, 7, 5) = 5$.
 Update: Increase flow by 5 on this path.
- **Step 2: Path** $s \rightarrow d \rightarrow e \rightarrow f \rightarrow t$
 Bottleneck = $\min(5, 7, 8, 10) = 5$.
 Update: Increase flow by 5 on this path.

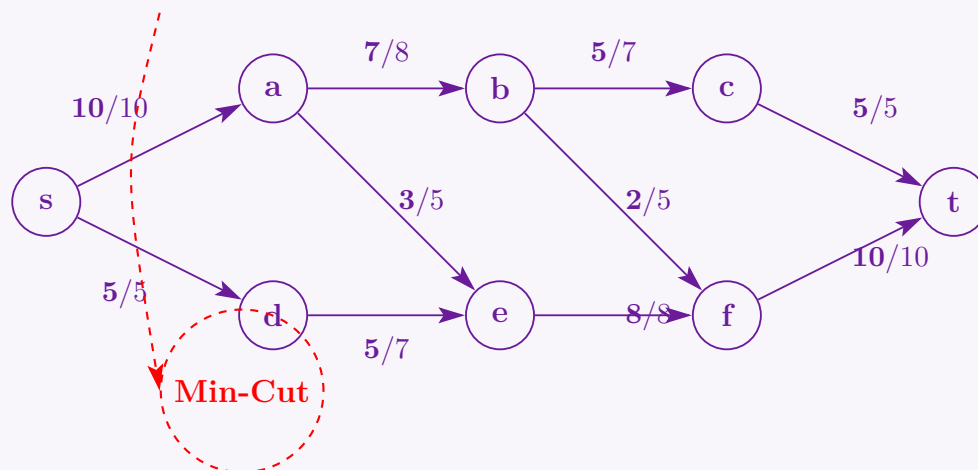
- Step 3: Path** $s \rightarrow a \rightarrow e \rightarrow f \rightarrow t$
 Residual capacities: $s \rightarrow a$ ($10-5=5$), $a \rightarrow e$ (5), $e \rightarrow f$ ($8-5=3$), $f \rightarrow t$ ($10-5=5$).
 Bottleneck = $\min(5, 5, 3, 5) = 3$.
 Update: Increase flow by 3 on this path.
- Step 4: Path** $s \rightarrow a \rightarrow b \rightarrow f \rightarrow t$
 Residual capacities: $s \rightarrow a$ ($5-3=2$), $a \rightarrow b$ ($8-5=3$), $b \rightarrow f$ (5), $f \rightarrow t$ ($5-3=2$).
 Bottleneck = $\min(2, 3, 5, 2) = 2$.
 Update: Increase flow by 2 on this path.

Total Maximum Flow: $5 + 5 + 3 + 2 = 15$.

At this point, edges $s \rightarrow a$ ($10/10$) and $s \rightarrow d$ ($5/5$) are fully saturated. No more augmenting paths exist.

2. Final Maximum Flow Network:

(Edge labels are shown as Flow / Capacity)



3. Min-Cut Identification:

Vertices reachable from s in the final residual graph: $A = \{s\}$ (since both outgoing edges are saturated).

Remaining vertices: $B = \{a, b, c, d, e, f, t\}$.

Min-Cut Capacity: $C(s, a) + C(s, d) = 10 + 5 = 15$.

Q6. What is a flow network? Explain how network flow algorithms allow us to find the maximum bipartite matching. (10 marks) [CSE 207 | 2016–17]

Answer: Flow Network & Bipartite Matching (Q2)

Flow Network:

A flow network is a directed graph $G = (V, E)$ where each edge (u, v) has a non-negative capacity $c(u, v) \geq 0$. It features a **source** node s (where flow originates) and a **sink** node t (where flow terminates). Valid flows must satisfy *capacity constraints* ($0 \leq f(u, v) \leq c(u, v)$) and *flow conservation* (incoming flow = outgoing

flow for all nodes except s and t).

Maximum Bipartite Matching via Network Flow:

A bipartite graph $G = (V, E)$ has vertices divided into two disjoint sets L and R . We can find the maximum matching by transforming it into a flow network:

- Add a **super-source** s and connect it to all vertices in L with directed edges of capacity 1.
- Add a **super-sink** t and connect all vertices in R to t with directed edges of capacity 1.
- Direct all original edges in E from L to R and assign them a capacity of 1 (or ∞).

By running a max-flow algorithm (like Ford-Fulkerson) on this new network, the integrality theorem ensures the flow on every edge will be either 0 or 1. The edges between L and R that carry a flow of 1 represent the edges included in the **Maximum Bipartite Matching**.

Q7. Write the Edmonds-Karp algorithm that solves the maximum-flow problem. Analyze its time complexity for integral capacities. **(10 marks)** [CSE 207 | 2015–16]

Answer: Edmonds-Karp Algorithm & Complexity (Q3)

Edmonds-Karp Algorithm:

Algorithm 144 Edmonds-Karp(G, s, t)

```

1: Initialize  $f(u, v) \leftarrow 0$  for all edges  $(u, v) \in E$ 
2: while BFS finds a path  $p$  from  $s$  to  $t$  in residual graph  $G_f$  do
3:    $c_f(p) \leftarrow \min\{c_f(u, v) : (u, v) \text{ is in } p\}$             $\triangleright$  Find bottleneck capacity
4:   for each edge  $(u, v)$  in  $p$  do
5:      $f(u, v) \leftarrow f(u, v) + c_f(p)$                         $\triangleright$  Augment forward flow
6:      $f(v, u) \leftarrow f(v, u) - c_f(p)$                         $\triangleright$  Decrease reverse flow
7:   end for
8: end while
9: return  $f$ 

```

Time Complexity Analysis (for Integral Capacities):

- In Edmonds-Karp, the augmenting path is found using **Breadth-First Search (BFS)**, which guarantees we always choose the shortest path (in terms of the number of edges).
- Cost of BFS:** Finding a path takes $O(V + E) = O(E)$ time.
- Number of Augmentations:** It can be proven mathematically that the shortest path distance from s to any node monotonically increases. An edge

can become critical (bottleneck) at most $|V|/2$ times. Since there are $O(E)$ edges, the total number of augmentations is strictly bounded by $O(V \cdot E)$.

- **Total Time:** Multiplying the cost per augmentation by the maximum number of augmentations gives $O(E) \times O(V \cdot E) = O(V \cdot E^2)$.
- Note: This complexity is strictly polynomial and independent of the actual capacity values.

Q8. Explain how network flow algorithms allow us to find the maximum bipartite matching. (10 marks) [CSE 207 | 2015–16]

Answer: Max Bipartite Matching via Network Flow (Q4)

(Note: This is an expanded explanation of the concept covered in Q2)

To solve the Maximum Bipartite Matching problem using Network Flow algorithms, we perform a reduction. Let the bipartite graph be $G = (V, E)$ with partitions L and R .

1. Network Construction:

- Create a new flow network $G' = (V', E')$.
- Add a source node s and sink node t , so $V' = V \cup \{s, t\}$.
- For every node $u \in L$, add a directed edge (s, u) with capacity $c(s, u) = 1$.
- For every node $v \in R$, add a directed edge (v, t) with capacity $c(v, t) = 1$.
- For every original edge (u, v) where $u \in L$ and $v \in R$, add a directed edge (u, v) with capacity 1.

2. Why this works:

- **Integrality Theorem:** If all edge capacities are integers, the Max-Flow algorithm (like Ford-Fulkerson) guarantees an integer flow. Here, flows will be either 0 or 1.
- **Matching constraint:** The capacity of 1 from s to $u \in L$ ensures each vertex in L can participate in at most one matching edge. Similarly, the capacity of 1 from $v \in R$ to t ensures each vertex in R receives flow from at most one vertex in L .
- **Result:** The value of the maximum flow from s to t is exactly the size of the maximum bipartite matching. The edges between L and R with flow $f(u, v) = 1$ form the matching pairs.

Q9. What is the pitfall of Ford-Fulkerson's max-flow min-cut method? How can it be improved? (8 marks) [CSE 207 | 2014–15]

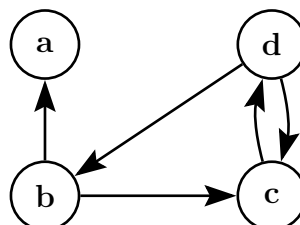
Answer: Pitfalls of Ford-Fulkerson & Improvements (Q5)**Pitfall of Ford-Fulkerson:**

- **Pseudo-polynomial Time / Bad Path Selection:** The original Ford-Fulkerson method uses standard DFS or arbitrary path selection to find augmenting paths. If the edge capacities are very large integers, it might choose poor paths that only push 1 unit of flow at a time.
- *Classic Example:* Consider a graph with nodes s, u, v, t where $s \rightarrow u$, $s \rightarrow v$, $u \rightarrow t$, $v \rightarrow t$ have capacity M (a huge integer), and a cross-edge $u \rightarrow v$ has capacity 1. If DFS alternately picks $s \rightarrow u \rightarrow v \rightarrow t$ and $s \rightarrow v \rightarrow u \rightarrow t$, it will augment exactly 1 unit of flow per step, requiring $2M$ iterations. The time complexity becomes $O(E \cdot |f^*|)$ which is incredibly slow for large max flow $|f^*|$.
- **Irrational Capacities:** If edge capacities are irrational numbers, the algorithm is not even guaranteed to terminate, and it might converge to a sub-optimal/incorrect value.

How it can be improved:

- Edmonds-Karp Algorithm:** Instead of arbitrary path selection (DFS), use **Breadth-First Search (BFS)**. This guarantees finding the shortest augmenting path (fewest edges). It makes the complexity strictly polynomial: $O(V \cdot E^2)$, regardless of edge capacities.
- Capacity Scaling:** Choose augmenting paths that have the largest bottleneck capacity first. This reduces the time complexity to $O(E^2 \log U)$, where U is the maximum edge capacity.
- Dinic's Algorithm:** A more advanced algorithm that uses Level Graphs (via BFS) and pushes Blocking Flows (via DFS) in phases, improving the time complexity further to $O(V^2 E)$.

Q10. Find the transitive closure. Show the matrices computed at each step. What is the cost to find the transitive closure? **(10 marks)** [CSE 207 | 2014–15]

**Answer**

Vertices: a, b, c, d (indices 0,1,2,3). Edges: $b \rightarrow a$, $b \rightarrow c$, $d \rightarrow b$, $d \rightarrow c$, $c \rightarrow d$.

$T^{(0)}$ (reachability via 0 intermediate vertices = direct edges + self):

$$T^{(0)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 0 \\ c & 0 & 0 & 1 & 1 \\ d & 0 & 1 & 1 & 1 \end{array}$$

After $k = b$: d can reach a via $b \rightarrow T[d][a] = 1$.

$$T^{(b)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 0 \\ c & 0 & 0 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

After $k = c$: b can reach d via $c \rightarrow T[b][d] = 1$.

$$T^{(c)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 1 \\ c & 0 & 0 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

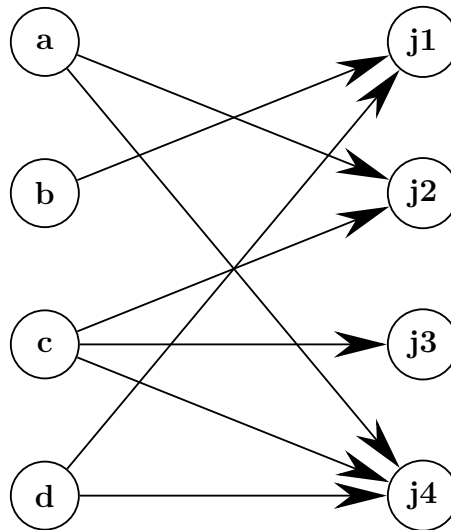
After $k = d$: b and c can reach a via $d \rightarrow b \rightarrow a \rightarrow T[c][a] = 1$, $T[b][a]$ already 1.

$$T^{(d)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 1 \\ c & 1 & 1 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

Interpretation: a is a sink (can only reach itself); b, c, d form a cycle and can reach everywhere.

Complexity Analysis. Floyd-Warshall transitive closure: $\Theta(V^3)$.

Q11. Find the maximum job assignment using a maximum flow algorithm. (10 marks)
[CSE 207 | 2014–15]



Answer: Maximum Job Assignment using Max Flow (Q1)

To solve the maximum job assignment (bipartite matching) using a max-flow algorithm, we construct a flow network from the given bipartite graph:

- Add a **super-source** S and connect it to all applicants $\{a, b, c, d\}$ with directed edges of capacity 1.
- Add a **super-sink** T and connect all jobs $\{j1, j2, j3, j4\}$ to T with directed edges of capacity 1.
- Direct all original edges from applicants to jobs and assign them a capacity of 1.

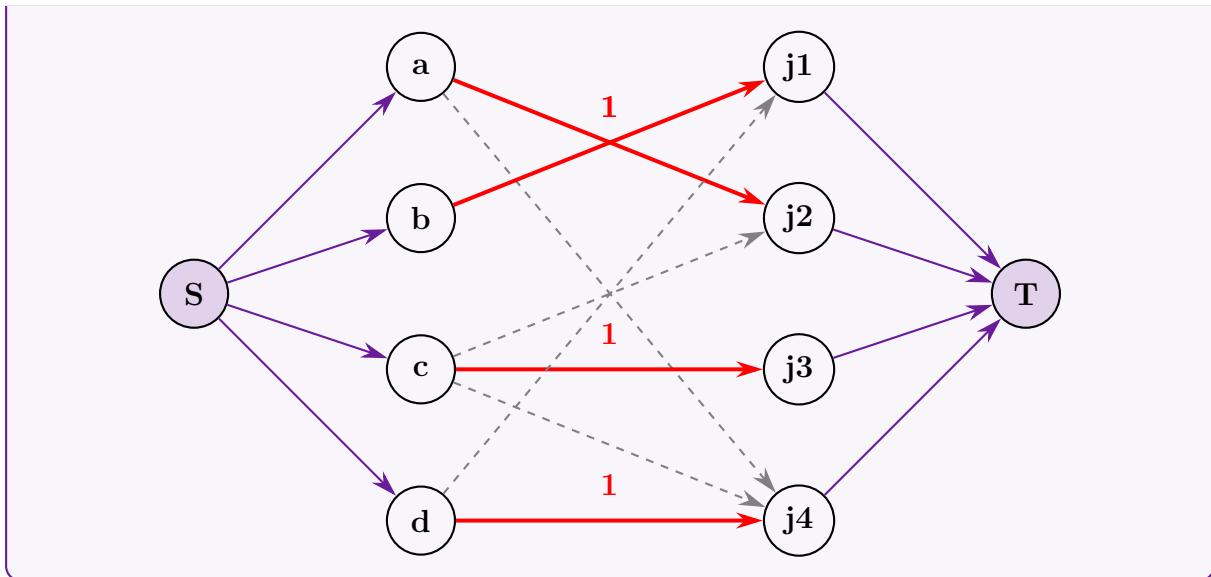
Applying Ford-Fulkerson (Finding Augmenting Paths):

- Path 1: $S \rightarrow b \rightarrow j1 \rightarrow T$ (Bottleneck = 1, Total Flow = 1)
- Path 2: $S \rightarrow a \rightarrow j2 \rightarrow T$ (Bottleneck = 1, Total Flow = 2)
- Path 3: $S \rightarrow c \rightarrow j3 \rightarrow T$ (Bottleneck = 1, Total Flow = 3)
- Path 4: $S \rightarrow d \rightarrow j4 \rightarrow T$ (Bottleneck = 1, Total Flow = 4)

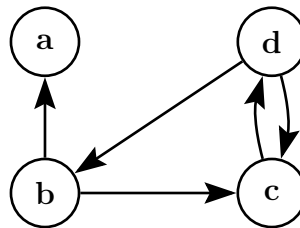
Since all edges from S are fully saturated, no more augmenting paths exist. The maximum flow is 4. This means all 4 jobs can be perfectly assigned!

Final Assignment Network:

(Red bold arrows indicate the matched assignment edges carrying 1 unit of flow)



Q12. Find the transitive closure. Show the matrices computed at each step. What is the cost to find the transitive closure? **(10 marks)** [CSE 207 | 2014–15]



Answer

Vertices: a, b, c, d (indices 0,1,2,3). Edges: $b \rightarrow a, b \rightarrow c, d \rightarrow b, d \rightarrow c, c \rightarrow d$.
 $T^{(0)}$ (reachability via 0 intermediate vertices = direct edges + self):

$$T^{(0)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 0 \\ c & 0 & 0 & 1 & 1 \\ d & 0 & 1 & 1 & 1 \end{array}$$

After $k = b$: d can reach a via $b \rightarrow T[d][a] = 1$.

$$T^{(b)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 0 \\ c & 0 & 0 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

After $k = c$: b can reach d via $c \rightarrow T[b][d] = 1$.

$$T^{(c)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 1 \\ c & 0 & 0 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

After $k = d$: b and c can reach a via $d \rightarrow b \rightarrow a \rightarrow T[c][a] = 1$, $T[b][a]$ already 1.

$$T^{(d)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 1 & 0 & 0 & 0 \\ b & 1 & 1 & 1 & 1 \\ c & 1 & 1 & 1 & 1 \\ d & 1 & 1 & 1 & 1 \end{array}$$

Interpretation: a is a sink (can only reach itself); b, c, d form a cycle and can reach everywhere.

Complexity Analysis. Floyd-Warshall transitive closure: $\Theta(V^3)$.

- Q13.** Write and prove the max-flow min-cut theorem. Explain how one can convert a multiple-source, multiple-sink maximum-flow problem into a problem with a single source and a single sink. (10 marks) [CSE 207 | 2013–14, 2011–12]

Answer: Max-Flow Min-Cut Theorem & Proof

Max-Flow Min-Cut Theorem:

In any flow network, the value of the maximum flow from the source s to the sink t is exactly equal to the capacity of the minimum $s - t$ cut.

Proof:

Let f be a flow in a flow network $G = (V, E)$ with source s and sink t . We must prove that the following three conditions are equivalent:

- (i) f is a maximum flow in G .
- (ii) The residual network G_f contains no augmenting paths.
- (iii) $|f| = c(S, T)$ for some cut (S, T) of G .

(i) $\implies$ (ii):

We prove by contradiction. Assume f is a maximum flow, but there is an augmenting path p in G_f . By definition, we can push additional flow > 0 along p . The new flow would have a strictly greater value than f , contradicting the assumption that f is maximum. Thus, no augmenting path exists.

(ii) $\implies$ (iii):

Assume G_f has no augmenting paths. Define a set S as the set of all vertices reachable from s in G_f , and $T = V \setminus S$.

- Since s is reachable from itself, $s \in S$.
- Since there are no augmenting paths to the sink, $t \notin S$, so $t \in T$.

Thus, (S, T) forms a valid cut. For any edge (u, v) where $u \in S$ and $v \in T$:

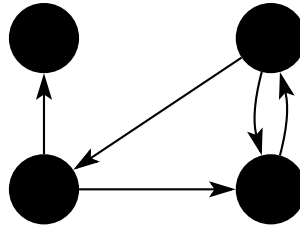
- The flow $f(u, v)$ must equal the capacity $c(u, v)$. If $f(u, v) < c(u, v)$, there would be positive residual capacity, making v reachable from u , which contradicts $v \in T$.
- For any edge (v, u) from T to S , the flow $f(v, u)$ must be 0. Otherwise, there would be a reverse residual edge from u to v , making v reachable.

By the definition of flow across a cut, $|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{v \in T, u \in S} f(v, u) = c(S, T) - 0 = c(S, T)$.

(iii) $\implies$ (i):

By the weak duality property of networks, the value of any flow f is bounded by the capacity of any cut (S, T) : $|f| \leq c(S, T)$. Since we found a specific cut where $|f| = c(S, T)$, the flow cannot possibly be any larger. Therefore, f must be a maximum flow. ■

Q14. Apply the Transitive-Closure algorithm on the directed graph given. Show the matrices computed at all stages of execution. (15 marks) [CSE 207 | 2012–13]



Answer

Edges: $4 \rightarrow 1$, $2 \rightarrow 4$, $4 \rightarrow 3$, $2 \rightarrow 3$, $3 \rightarrow 2$.

Algorithm: $T_{ij}^{(k)} = T_{ij}^{(k-1)} \vee (T_{ik}^{(k-1)} \wedge T_{kj}^{(k-1)})$

$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \xrightarrow{k=1} T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

(No change: nothing enters vertex 1 except from 4, and vertex 1 has no outgoing edges that help new paths.)

$$T^{(1)} \xrightarrow{k=2} T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

($T_{34}^{(2)}$: path $3 \rightarrow 2 \rightarrow 4$ now reachable. $T_{34} : 0 \vee (T_{32} \wedge T_{24}) = 0 \vee (1 \wedge 1) = 1$.)

$$T^{(2)} \xrightarrow{k=3} T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

($T_{42}^{(3)}$: path $4 \rightarrow 3 \rightarrow 2$. $T_{42} : 0 \vee (1 \wedge 1) = 1$.)

$$T^{(3)} \xrightarrow{k=4} T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

($T_{21}^{(4)}$: $2 \rightarrow 4 \rightarrow 1$. $T_{21} : 0 \vee (1 \wedge 1) = 1$. $T_{31}^{(4)}$: $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$. $T_{31} : 0 \vee (1 \wedge 1) = 1$.)

Interpretation: Vertex 1 can only reach itself; vertices 2, 3, 4 can reach each other and vertex 1.

Q15. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. (20 marks) [CSE 207 | 2012–13]

Answer

Algorithm 145 BELLMAN-FORD(G, w, s)

```

1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  for all  $v \neq s$ 
2:  $\pi[v] \leftarrow \text{NIL}$  for all  $v$ 
3: for  $i \leftarrow 1$  to  $|V| - 1$  do
4:   for each edge  $(u, v) \in E$  do
5:     if  $d[u] + w(u, v) < d[v]$  then
6:        $d[v] \leftarrow d[u] + w(u, v)$ ;  $\pi[v] \leftarrow u$ 
7:     end if
8:   end for
9: end for
10: for each edge  $(u, v) \in E$  do ▷ check for negative cycles
11:   if  $d[u] + w(u, v) < d[v]$  then return false
12:   end if
13: end for
14: return true

```

Complexity Analysis. $|V| - 1$ passes, each scanning all $|E|$ edges: $\Theta(VE)$.

Correctness sketch. *Loop invariant:* after i passes, $d[v] = \delta(s, v)$ for all v reachable via a shortest path with $\leq i$ edges. After $|V| - 1$ passes (longest possible simple path), all reachable $d[v] = \delta(s, v)$ if no negative cycle. If a negative cycle is reachable, some $d[v]$ can still be reduced in the final check $\rightarrow$ return FALSE. $\square$

Q16. What is a flow network? What is the maximum-flow problem? Explain how one can convert a multiple-source, multiple-sink maximum-flow problem into a problem with a single source and a single sink. (4+2+4=10 marks) [CSE 207 | 2011–12]

Answer: Flow Network, Max Flow & Multiple-Source Conversion (Q4)

What is a Flow Network?

A flow network is a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. There are two distinguished nodes: a *source* s and a *sink* t . A flow in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies:

- **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$
- **Flow conservation:** $\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w)$ for all $v \in V \setminus \{s, t\}$.

What is the Maximum-Flow Problem?

Given a flow network G with source s and sink t , the maximum-flow problem asks to find a valid flow function f such that the total flow $|f|$ leaving the source s (and equivalently arriving at t) is maximized.

Conversion of Multiple-Source, Multiple-Sink Problem:

Real-world problems might have multiple factories (sources $s_1, s_2, \dots$) and multiple markets (sinks $t_1, t_2, \dots$). Standard algorithms only work with a single source and sink. We can convert it as follows:

- Add a single **super-source node** S and add directed edges from S to each original source s_i . Set the capacity of these new edges to **infinity** (∞).
- Add a single **super-sink node** T and add directed edges from each original sink t_j to T . Set the capacity of these new edges to **infinity** (∞).



Q17. Write the Ford-Fulkerson algorithm for solving the maximum-flow problem, and analyze the time complexity of the algorithm. Explain how the Edmonds-Karp algorithm improves the bound of the Ford-Fulkerson algorithm. (10+5=15 marks) [CSE 207 | 2011–12]

Answer: Ford-Fulkerson Algorithm & Edmonds-Karp Bound (Q5)

Ford-Fulkerson Algorithm:

Algorithm 146 Ford-Fulkerson(G, s, t)

```

1: Initialize  $f(u, v) \leftarrow 0$  for all edges  $(u, v)$ 
2: while there exists an augmenting path  $p$  from  $s$  to  $t$  in the residual network  $G_f$ 
   do
3:   Find bottleneck capacity:  $c_f(p) = \min\{c_f(u, v) \mid (u, v) \in p\}$ 
4:   for each edge  $(u, v)$  in  $p$  do
5:     if  $(u, v)$  is a forward edge in  $G$  then
6:        $f(u, v) \leftarrow f(u, v) + c_f(p)$ 
7:     else ▷ It's a backward edge
8:        $f(v, u) \leftarrow f(v, u) - c_f(p)$ 
9:     end if
10:  end for
11: end while
12: return  $f$ 

```

Time Complexity Analysis (Ford-Fulkerson):

- In the worst-case scenario (especially with generic DFS pathfinding), each augmenting path might only add 1 unit of flow.
- Finding an augmenting path (using DFS/BFS) takes $O(E)$ time.
- If the maximum flow is $|f^*|$, the ‘while’ loop can execute at most $|f^*|$ times.
- **Total Time Complexity:** $O(E \cdot |f^*|)$. This is a *pseudo-polynomial* time bound because it depends on the magnitude of the capacities (the flow value), not just the size of the graph.

How Edmonds-Karp Improves the Bound:

- **Strategy:** The Edmonds-Karp algorithm is an implementation of Ford-Fulkerson that explicitly uses **Breadth-First Search (BFS)** to always select the *shortest* augmenting path (fewest number of edges) in the residual graph.
- **Improvement:** By always choosing the shortest path, it eliminates the dependency on the capacity values. It mathematically guarantees that the total number of augmentations across the entire algorithm is bounded by $O(V \cdot E)$.
- **New Bound:** Since each BFS takes $O(E)$ time and there are at most $O(V \cdot E)$ augmentations, the overall time complexity is strictly bounded to $O(V \cdot E^2)$. This makes the algorithm **strongly polynomial** and highly efficient even for graphs with massive integer capacities.

Q18. Derive with explanation the recursive equation for the Floyd-Warshall algorithm for all-pairs shortest-path estimates in a directed graph G . Based on this equation, write pseudocode for Floyd-Warshall. Convert this recursive equation to compute the transitive closure of G . (17 marks) [CSE 207 | 2009–10]

Answer**Recursive equation.**

Let $d^{(k)}[i][j]$ = shortest-path weight from i to j using only vertices $\{1, 2, \dots, k\}$ as intermediates.

$$d^{(k)}[i][j] = \begin{cases} w(i, j) & k = 0 \\ \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]) & k \geq 1 \end{cases}$$

Intuition: either the shortest path does not use vertex k (first case) or it passes through k (split into two sub-paths).

Algorithm 147 FLOYD-WARSHALL(W)

```

1:  $D^{(0)} \leftarrow W$ 
2: for  $k \leftarrow 1$  to  $n$  do
3:   for  $i \leftarrow 1$  to  $n$  do
4:     for  $j \leftarrow 1$  to  $n$  do
5:        $d^{(k)}[i][j] \leftarrow \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$ 
6:     end for
7:   end for
8: end for
9: return  $D^{(n)}$ 

```

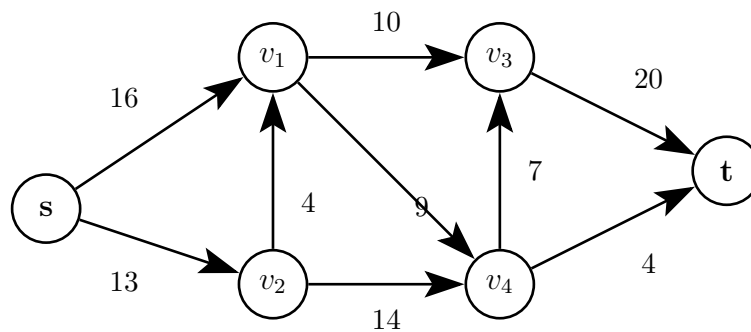
Complexity Analysis. $\Theta(V^3)$ time, $\Theta(V^2)$ space.

Transitive closure adaptation. Replace distances with Boolean reachability:

$$t^{(k)}[i][j] = t^{(k-1)}[i][j] \vee (t^{(k-1)}[i][k] \wedge t^{(k-1)}[k][j]).$$

Initialize $t^{(0)}[i][j] = 1$ if $i = j$ or $(i, j) \in E$, else 0. After n iterations, $t^{(n)}[i][j] = 1$ iff j is reachable from i .

- Q19.** State the max-flow min-cut theorem in its simplest form. What is the main argument to prove the theorem? [Flow network given in figure] Find a min-cut and its capacity. Find the flow in that cut and the maximum flow in the network. (17 marks) [CSE 207 | 2009–10]



Answer: Max-Flow Min-Cut Theorem & Graph 1 Analysis

1. Max-Flow Min-Cut Theorem (Simplest Form):

In any flow network, the maximum amount of flow that can pass from the source to the sink is exactly equal to the total capacity of the minimum cut that separates the source from the sink.

2. Main Argument to Prove the Theorem:

The proof relies on demonstrating the equivalence of three conditions for a flow f :

- (a) f is a maximum flow.
- (b) The residual network G_f contains no augmenting paths.
- (c) $|f| = c(A, B)$ for some cut (A, B) .

The core argument is based on **Weak Duality** (any flow is bounded by any cut, $|f| \leq c(A, B)$) and **Reachability**. If we push flow until no augmenting paths exist, we can partition the vertices into set A (reachable from source in residual graph) and set B (unreachable). By definition, all forward edges from A to B must be fully saturated (flow = capacity), and all backward edges from B to A must be empty. Thus, the net flow exactly equals the cut capacity, proving it is both the maximum possible flow and the minimum possible cut.

3. Analyzing the Given Flow Network: Let's find the max flow by pushing flow through augmenting paths:

- **Path 1:** $s \rightarrow v_1 \rightarrow v_3 \rightarrow t$ Bottleneck = $\min(16, 10, 20) = 10$.
- **Path 2:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow t$ Bottleneck = $\min(13, 14, 4) = 4$.
- **Path 3:** $s \rightarrow v_1 \rightarrow v_4 \rightarrow v_3 \rightarrow t$
Residuals: $s \rightarrow v_1$ (6), $v_1 \rightarrow v_4$ (9), $v_4 \rightarrow v_3$ (7), $v_3 \rightarrow t$ (10).
Bottleneck = $\min(6, 9, 7, 10) = 6$.
- **Path 4:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow t$
Residuals: $s \rightarrow v_2$ (9), $v_2 \rightarrow v_4$ (10), $v_4 \rightarrow v_3$ (1), $v_3 \rightarrow t$ (4).
Bottleneck = $\min(9, 10, 1, 4) = 1$.

Maximum Flow: $10 + 4 + 6 + 1 = 21$.

After these paths, edges $s \rightarrow v_1$ (16/16), $v_1 \rightarrow v_3$ (10/10), $v_4 \rightarrow v_3$ (7/7), and $v_4 \rightarrow t$ (4/4) are fully saturated.

Minimum Cut Identification:

In the final residual graph, from s we can only reach v_2 (8 capacity left). From v_2 , we can reach v_1 (via $v_2 \rightarrow v_1$) and v_4 (via $v_2 \rightarrow v_4$). Neither v_3 nor t can be reached.

- $A = \{s, v_1, v_2, v_4\}$
- $B = \{v_3, t\}$

Min-Cut Capacity = $C(v_1, v_3) + C(v_4, v_3) + C(v_4, t) = 10 + 7 + 4 = 21$.

Flow in that cut: The sum of flows on the forward edges of the cut is $10 + 7 + 4 = 21$.

- Q20.** Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G' . Prove that there is a maximum matching M in G if and only if there is a maximum flow f in G' . (17 marks) [CSE 207 | 2009–10]

Answer: Bipartite Graph to Flow Network Conversion

1. Conversion Process with Example:

Given a bipartite graph $G = (V, E)$ with partitions L and R (where $V = L \cup R$), we construct a flow network $G' = (V', E')$ as follows:

- Add a **super-source** s and a **super-sink** t . $V' = L \cup R \cup \{s, t\}$.
- Add directed edges from s to every node in L with capacity 1: $c(s, u) = 1, \forall u \in L$.
- Add directed edges from every node in R to t with capacity 1: $c(v, t) = 1, \forall v \in R$.
- Replace all original undirected edges $\{u, v\}$ in E with directed edges from u to v with capacity 1 (or ∞).

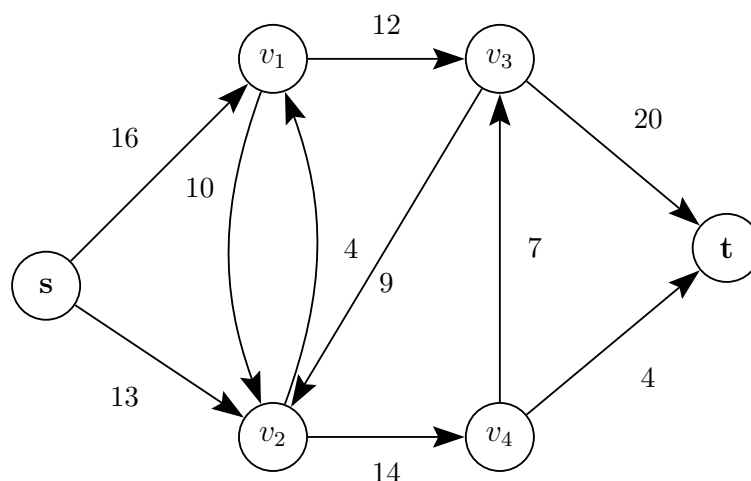
Example: If $L = \{u_1\}$, $R = \{v_1\}$, and $E = \{(u_1, v_1)\}$. The flow network becomes: $s \xrightarrow{1} u_1 \xrightarrow{1} v_1 \xrightarrow{1} t$.

2. Proof of Equivalence (M is max matching $\iff f$ is max flow):

- ($\implies$) **Matching to Flow:** Suppose we have a maximum matching M of size k . We can construct a flow f by sending 1 unit of flow along the path $s \rightarrow u \rightarrow v \rightarrow t$ for every edge $(u, v) \in M$. Since each node in L and R appears in at most one edge in M , the capacities of edges from s and to t are not violated. Thus, we get a valid flow $|f| = k$.
- ($\impliedby$) **Flow to Matching:** Suppose we have a maximum integer flow f of value k in G' . Due to the *Integrality Theorem*, since all capacities are integers, the Ford-Fulkerson algorithm yields an integer flow on all edges. The flow on any edge between L and R must be 0 or 1. Let M be the set of edges (u, v) where $f(u, v) = 1$. Because edges from s to L have capacity 1, each node in L contributes to at most one unit of flow (at most one matched edge). Similarly, capacity constraints to t ensure each node in R receives at most one unit of flow. Thus, M forms a valid bipartite matching of size k .

Since $|M| = |f|$ in both directions, maximizing f automatically maximizes M .

- Q21.** Find the maximum flow and the minimum cut. Find also the maximum flow in the minimum cut. (12 marks) [CSE 207 | 2008–09]



Answer: Max Flow and Min Cut for Graph 2

Let's find the max flow for the new network by finding augmenting paths:

- Path 1:** $s \rightarrow v_1 \rightarrow v_3 \rightarrow t$ Bottleneck = $\min(16, 12, 20) = 12$.
 Residuals: $s \rightarrow v_1$ (4), $v_1 \rightarrow v_3$ (0), $v_3 \rightarrow t$ (8).
- Path 2:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow t$ Bottleneck = $\min(13, 14, 4) = 4$.
 Residuals: $s \rightarrow v_2$ (9), $v_2 \rightarrow v_4$ (10), $v_4 \rightarrow t$ (0).
- Path 3:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow t$ Bottleneck = $\min(9, 10, 7, 8) = 7$.
 Residuals: $s \rightarrow v_2$ (2), $v_2 \rightarrow v_4$ (3), $v_4 \rightarrow v_3$ (0), $v_3 \rightarrow t$ (1).
- Path 4:** We still have 1 unit of capacity on $v_3 \rightarrow t$. Can we reach v_3 ?
 s has 4 units left to v_1 . v_1 cannot go to v_3 directly (capacity full), but it can go to v_2 (capacity 10).
 v_2 can then go to v_4 (3 units left). However, v_4 has no outgoing paths left to v_3 or t . Thus, no more augmenting paths exist.

Wait, let's trace Path 4 properly: Can we send flow through $s \rightarrow v_1 \rightarrow v_2 \rightarrow v_4 \dots$? As established, v_4 is blocked. Is there any other way? Total incoming capacity from s : $16 + 13 = 29$. Total possible ways into t : $20 + 4 = 24$. But looking at the cut between the layers, incoming to v_3 is $v_1 \rightarrow v_3(12)$ and $v_4 \rightarrow v_3(7)$. Total = 19. Incoming to t directly from v_4 is 4. Total = $19 + 4 = 23$. Thus, **Maximum Flow** = $12 + 4 + 7 = 23$.

Minimum Cut Identification: Let's find the reachable vertices from s in the final residual network:

- s can reach v_1 (4 left) and v_2 (2 left).
- v_1 can reach v_2 (10 left).
- v_2 can reach v_1 (4 left) and v_4 (3 left).
- v_4 cannot reach v_3 or t .

Reachable set $A = \{s, v_1, v_2, v_4\}$. Unreachable set $B = \{v_3, t\}$.

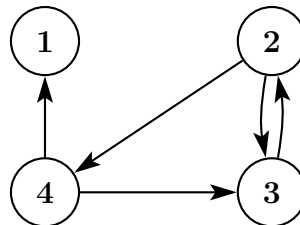
Minimum Cut: $(A, B) = (\{s, v_1, v_2, v_4\}, \{v_3, t\})$.

Capacity of Min-Cut: Sum of original forward edge capacities from A to B :

$$C(A, B) = c(v_1, v_3) + c(v_4, v_3) + c(v_4, t) = 12 + 7 + 4 = \mathbf{23}$$

Maximum Flow in the Minimum Cut: By the Max-Flow Min-Cut theorem, the net flow across any cut equals the total flow of the network. Because this is the minimum cut, all its forward edges are fully saturated, and there is no reverse flow. Therefore, the maximum flow in the minimum cut is **23**.

- Q22.** For the given graph G , find the transitive closure using the Floyd-Warshall algorithm. Give the algorithm also. How can the complexity of Floyd-Warshall's all-pairs shortest paths be improved? **(10+5+10 marks)** [CSE 207 | 2008–09]



Answer

Vertices $\{1, 2, 3, 4\}$. Edges: $4 \rightarrow 1$, $2 \rightarrow 4$, $4 \rightarrow 3$, $2 \rightarrow 3$, $3 \rightarrow 2$.

Algorithm (see Floyd-Warshall transitive closure answer in graph section).

Initial $T^{(0)}$ ($T[i][j] = 1$ if $i = j$ or $(i, j) \in E$):

	1	2	3	4
1	1	0	0	0
2	0	1	1	1
3	0	1	1	0
4	1	0	1	1

After Floyd-Warshall (intermediate vertices 1,2,3,4 in order):

	1	2	3	4
1	1	0	0	0
2	1	1	1	1
3	1	1	1	1
4	1	1	1	1

Interpretation: vertex 1 is a sink (can only reach itself); vertices 2, 3, 4 form a strongly connected component and can all reach vertex 1 (via $4 \rightarrow 1$, $2 \rightarrow 4 \rightarrow 1$, $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$).

Complexity Analysis. $\Theta(V^3) = \Theta(64)$.

Improvement: for *all-pairs shortest paths* (not just closure), Johnson's algorithm runs in $O(V^2 \log V + VE)$, which is faster than Floyd-Warshall's $O(V^3)$ for sparse graphs.

Q23. Discuss how a maximum flow algorithm can be used to determine the maximum number of disjoint paths in directed and undirected graphs. **(15 marks)** [CSE 207 | 2007–08]

Answer: Maximum Disjoint Paths via Max Flow

A maximum flow algorithm can be elegantly adapted to find the maximum number of disjoint paths in a graph from a source s to a destination t . The concept relies heavily on **Menger's Theorem** and assigning unit capacities.

1. Edge-Disjoint Paths in Directed Graphs: Two paths are edge-disjoint if they do not share any edges.

- **Transformation:** Given a directed graph $G = (V, E)$, construct a flow network where the capacity of every edge is set to 1 ($c(u, v) = 1$ for all $(u, v) \in E$).
- **Algorithm:** Run a max-flow algorithm (like Edmonds-Karp) from s to t .
- **Result:** Due to the Integrality Theorem, the flow on each edge will be either 0 or 1. A flow of 1 indicates the edge is part of a path. Since each edge has a capacity of 1, no two paths can share the same edge. The value of the **maximum flow** equals the maximum number of edge-disjoint paths from s to t .

2. Vertex-Disjoint Paths in Directed Graphs: Two paths are vertex-disjoint if they share no vertices (other than s and t). This requires constraining the flow through the *vertices* themselves.

- **Transformation:** Split every vertex $v \in V \setminus \{s, t\}$ into two vertices: an "in-vertex" v_{in} and an "out-vertex" v_{out} .
- Add a directed edge from v_{in} to v_{out} with a **capacity of 1**. This acts as a bottleneck ensuring only 1 unit of flow can pass through vertex v .
- All original incoming edges to v are redirected to v_{in} (capacity ∞ or 1).
- All original outgoing edges from v originate from v_{out} (capacity ∞ or 1).
- **Result:** Run max-flow from s to t . The maximum flow value gives the exact maximum number of vertex-disjoint paths.

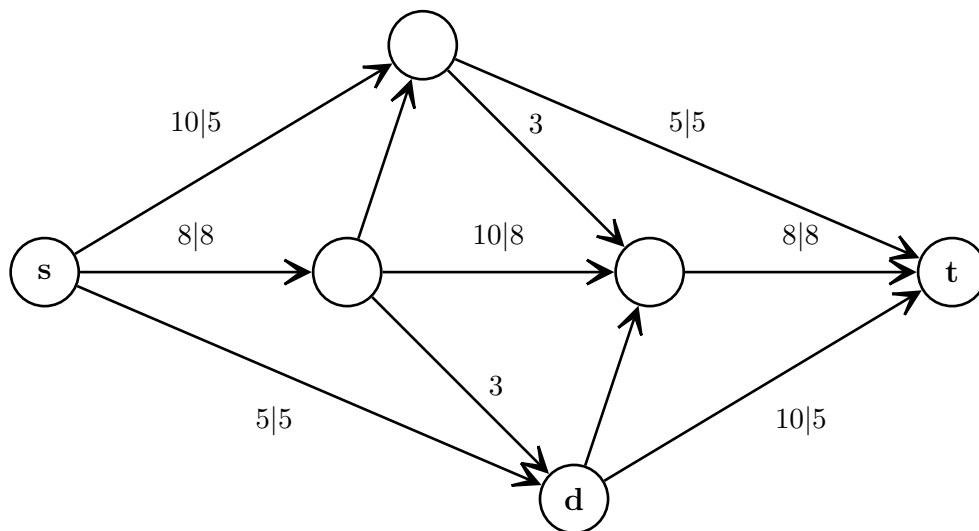
3. Undirected Graphs:

- For undirected edge-disjoint paths, replace every undirected edge $\{u, v\}$ with two directed edges (u, v) and (v, u) , each with capacity 1. Running max flow will yield the result (if the algorithm pushes flow $u \rightarrow v$, it effectively cancels $v \rightarrow u$, keeping paths simple).

Q24. Figure 1 shows a flow network on which s - t flow has been computed. The capacity of each edge appears as a label next to the edge, and the numbers in boxes give the amount of flow sent on each edge. Edges without boxed numbers have no flow on them. **(20 marks)**

- What is the value of this flow? Is this a maximum (s, t) flow on this graph?
- Find a minimum s - t cut in the flow network pictured in Figure 1, and also say what its capacity is.

[CSE 207 | 2007–08]



Answer: Flow Network Analysis (Figure 1)

1. Value of the Flow & Is it Maximum?

- Value of Flow:** The total flow is the sum of flow leaving the source s (or entering sink t). Flow out of $s = f(s, v_1) + f(s, v_2) + f(s, d) = 5 + 8 + 5 = 18$. (Checking sink t : $f(v_1, t) + f(v_3, t) + f(d, t) = 5 + 8 + 5 = 18$).
- Is it maximum? No.** A flow is maximum if and only if there are no augmenting paths in the residual network. Let's trace the residual graph:
 - $s \rightarrow v_1$: residual capacity = $10 - 5 = 5$ (forward).
 - $v_1 \rightarrow v_3$: residual capacity = $3 - 0 = 3$ (forward).
 - $v_3 \rightarrow v_2$: residual capacity = 8 (backward, canceling flow on $v_2 \rightarrow v_3$).
 - $v_2 \rightarrow d$: residual capacity = $3 - 0 = 3$ (forward).
 - $d \rightarrow t$: residual capacity = $10 - 5 = 5$ (forward).

We found an augmenting path: $s \rightarrow v_1 \rightarrow v_3 \rightarrow v_2 \rightarrow d \rightarrow t$ with a bottleneck of $\min(5, 3, 8, 3, 5) = 3$.

Pushing 3 more units of flow increases the total flow from 18 to **21**. After this, no more augmenting paths exist.

2. Minimum s - t Cut and its Capacity: After augmenting the flow by 3 units, the new maximum flow is 21. Let's find the reachable vertices from s in the final residual network:

- s can reach v_1 (capacity $5 - 3 = 2$ left).
- From v_1 , v_3 is full ($3 - 3 = 0$) and t is full ($5 - 5 = 0$). v_1 cannot reach anywhere else.
- Reachable set: $A = \{s, v_1\}$.
- Unreachable set: $B = \{v_2, d, v_3, t\}$.

Minimum Cut: $(A, B) = (\{s, v_1\}, \{v_2, d, v_3, t\})$.

Capacity of this Cut: Sum of original capacities of edges going from A to B :

$$C(A, B) = c(s, v_2) + c(s, d) + c(v_1, v_3) + c(v_1, t) = 8 + 5 + 3 + 5 = \mathbf{21}.$$

The capacity is 21, which perfectly matches the newly calculated maximum flow.

Q25. There is a wireless network of nodes A, B, C, D, E, F, G and H . Capacity of the wireless links are given in the following format: “-” indicates there is no link between those nodes. What is the maximum data flow, if node A sends data to node H ? What is the capacity of the optimal cut? (Use Ford-Fulkerson method and show the steps.) **(10+5=15 marks)**

	A	B	C	D	E	F	G	H
A	-	10	5	15	-	-	-	-
B	-	-	4	-	9	15	-	-
C	-	-	-	4	-	8	-	-
D	-	-	-	-	-	-	30	-
E	-	-	-	-	-	15	-	10
F	-	-	-	-	-	-	15	10
G	-	-	6	-	-	-	-	10
H	-	-	-	-	-	-	-	-

[CSE 207 | 2006–07]

Answer: Wireless Network Max Flow (Ford-Fulkerson)

We need to find the maximum flow from node A (source) to node H (sink). We will find augmenting paths step-by-step. *Initial Total Flow = 0.*

Step 1: Augmenting Paths

Path 1: $A \rightarrow B \rightarrow E \rightarrow H$

- Capacities: $A \rightarrow B(10)$, $B \rightarrow E(9)$, $E \rightarrow H(10)$
- Bottleneck = **9**. Update Flow = 9.

- Residuals: $A \rightarrow B(1)$, $B \rightarrow E(0)$, $E \rightarrow H(1)$.

Path 2: $A \rightarrow D \rightarrow G \rightarrow H$

- Capacities: $A \rightarrow D(15)$, $D \rightarrow G(30)$, $G \rightarrow H(10)$
- Bottleneck = **10**. Update Flow = $9 + 10 = 19$.
- Residuals: $A \rightarrow D(5)$, $D \rightarrow G(20)$, $G \rightarrow H(0)$.

Path 3: $A \rightarrow C \rightarrow F \rightarrow H$

- Capacities: $A \rightarrow C(5)$, $C \rightarrow F(8)$, $F \rightarrow H(10)$
- Bottleneck = **5**. Update Flow = $19 + 5 = 24$.
- Residuals: $A \rightarrow C(0)$, $C \rightarrow F(3)$, $F \rightarrow H(5)$.

Path 4: $A \rightarrow B \rightarrow F \rightarrow H$

- Available Res: $A \rightarrow B(1)$, $B \rightarrow F(15)$, $F \rightarrow H(5)$
- Bottleneck = **1**. Update Flow = $24 + 1 = 25$.
- Residuals: $A \rightarrow B(0)$, $B \rightarrow F(14)$, $F \rightarrow H(4)$.

Path 5: $A \rightarrow D \rightarrow G \rightarrow C \rightarrow F \rightarrow H$

- Available Res: $A \rightarrow D(5)$, $D \rightarrow G(20)$, $G \rightarrow C(6)$, $C \rightarrow F(3)$, $F \rightarrow H(4)$
- Bottleneck = **3**. Update Flow = $25 + 3 = 28$.
- Residuals: $A \rightarrow D(2)$, $D \rightarrow G(17)$, $G \rightarrow C(3)$, $C \rightarrow F(0)$, $F \rightarrow H(1)$.

No more augmenting paths can be found to H.

Maximum Data Flow = 28.

Step 2: Optimal Cut Capacity To find the min-cut, find vertices reachable from A in the final residual graph:

- From A, we can reach D (capacity 2 left).
- From D, we can reach G (capacity 17 left).
- From G, we can reach C (capacity 3 left).
- From C, we cannot reach F (capacity 0).
- Reachable Set $S = \{A, C, D, G\}$. Unreachable Set $T = \{B, E, F, H\}$.

The capacity of this cut is the sum of original edges from S to T :

$$c(S, T) = c(A, B) + c(C, F) + c(G, H) = 10 + 8 + 10 = \mathbf{28}.$$

Capacity of the optimal cut = 28 (Matching the max flow!).

Q26. How can the Ford–Fulkerson method be used to find a maximum matching? Explain with a simple example. (10 marks) [CSE 207 | 2006–07]

Answer: Maximum Matching using Ford-Fulkerson

The Ford-Fulkerson method can solve the Maximum Bipartite Matching problem by reducing it to a Network Flow problem.

How it works: Given a bipartite graph $G = (L \cup R, E)$:

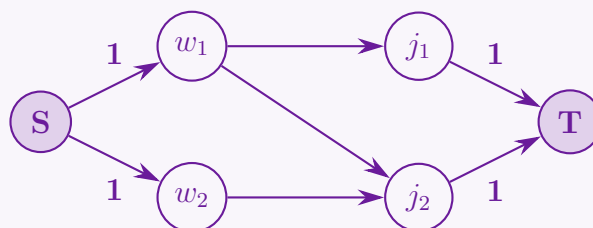
- Add a **super-source** S and connect it to all vertices in the left set L with directed edges of **capacity 1**.
- Add a **super-sink** T and connect all vertices in the right set R to T with directed edges of **capacity 1**.
- Direct all original edges from L to R and assign them **capacity 1** (or ∞).
- Run the Ford-Fulkerson algorithm. The max flow value equals the max number of matches, and edges between L and R carrying flow 1 are the matched pairs.

Simple Example:

Imagine 2 workers $L = \{w_1, w_2\}$ and 2 jobs $R = \{j_1, j_2\}$.

- w_1 can do j_1 and j_2 .
- w_2 can only do j_2 .

Flow Network Transformation:



Execution:

- Path 1:** $S \rightarrow w_1 \rightarrow j_1 \rightarrow T$ (Flow = 1).
- Path 2:** $S \rightarrow w_2 \rightarrow j_2 \rightarrow T$ (Flow = 1).
- Maximum flow is **2**.

The algorithm intelligently selects the perfect matching $\{(w_1, j_1), (w_2, j_2)\}$, ensuring no worker gets 2 jobs and no job is done twice, courtesy of the capacity 1 bottlenecks at S and T .

Q27. Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G' . Prove that there is a maximum matching M in G if and

only if there is a maximum flow f in G' . (17¹/₂ marks)

Answer: Maximum Bipartite Matching to Max Flow Conversion & Proof

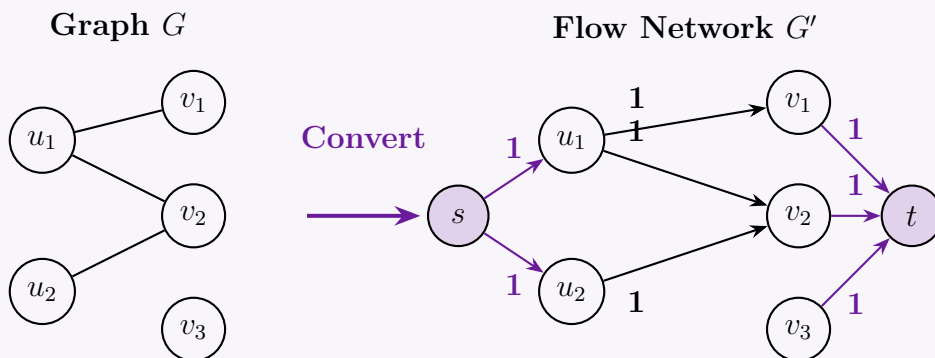
1. Conversion of Bipartite Graph to Flow Network:

Let $G = (V, E)$ be an undirected bipartite graph where the vertex set V is partitioned into two disjoint sets L and R ($V = L \cup R$), and every edge in E connects a vertex in L to a vertex in R . We construct a corresponding flow network $G' = (V', E')$ as follows:

- **Vertices:** Add a super-source node s and a super-sink node t . Thus, $V' = L \cup R \cup \{s, t\}$.
- **Edges & Capacities:**
 - Add a directed edge from s to each node $u \in L$ with capacity $c(s, u) = 1$.
 - Add a directed edge from each node $v \in R$ to t with capacity $c(v, t) = 1$.
 - For every original undirected edge $\{u, v\} \in E$ (where $u \in L$ and $v \in R$), add a directed edge from u to v with capacity $c(u, v) = 1$ (or ∞).

2. Example of Conversion:

Consider a bipartite graph G with $L = \{u_1, u_2\}$, $R = \{v_1, v_2, v_3\}$, and edges $E = \{(u_1, v_1), (u_1, v_2), (u_2, v_2)\}$.



3. Proof of Equivalence (M is max matching $\iff f$ is max flow):

To prove this, we must establish that the size of any maximum matching $|M|$ in G is exactly equal to the value of the maximum flow $|f|$ in G' . We do this by proving both bounds: $|f| \geq |M|$ and $|M| \geq |f|$.

Part A: Matching implies a valid Flow ($|f| \geq |M|$)

- Let M be a maximum matching in G with size k .
- We construct a flow f in G' as follows: For every matched edge $(u, v) \in M$, assign a flow of 1 along the path $s \rightarrow u \rightarrow v \rightarrow t$. For all other edges not part of these paths, assign a flow of 0.

- *Capacity Constraint Check:* Since M is a matching, each vertex $u \in L$ and $v \in R$ appears in exactly one edge in M . Thus, the total flow through any edge $s \rightarrow u$ is exactly 1, and through $v \rightarrow t$ is exactly 1. No capacity constraints ($c = 1$) are violated.
- *Flow Conservation Check:* Every u receives 1 unit from s and sends 1 unit to v . Every v receives 1 unit from u and sends 1 unit to t .
- Thus, f is a valid flow of value k . This means the maximum possible flow must be at least k . Therefore, $|f| \geq |M|$.

Part B: Flow implies a valid Matching ($|M| \geq |f|$)

- Let f be a maximum flow in G' of value k .
- By the **Integrality Theorem**, since all capacities in G' are integers (1 or ∞), the Ford-Fulkerson algorithm guarantees that the maximum flow f will have integer values on all edges.
- Since the edges $s \rightarrow u$ have capacity 1, the flow into any $u \in L$ is either 0 or 1. By flow conservation, the total outgoing integer flow from u to vertices in R is exactly 1 (if u has incoming flow) or 0.
- Similarly, since edges $v \rightarrow t$ have capacity 1, the total incoming integer flow to any $v \in R$ from vertices in L is at most 1.
- Now, construct a set of edges $M = \{(u, v) \mid u \in L, v \in R \text{ and } f(u, v) = 1\}$.
- Because the flow through any u is at most 1, u appears at most once in M . Because the flow through any v is at most 1, v appears at most once in M .
- Therefore, no two edges in M share a common vertex. M is a valid matching.
- The total flow entering t is exactly the number of edges carrying 1 unit of flow from R , which is exactly $|M|$. Thus, $k = |M|$. This means the maximum possible matching size must be at least k . Therefore, $|M| \geq |f|$.

Conclusion: Since $|f| \geq |M|$ and $|M| \geq |f|$, it mathematically holds that $|M| = |f|$. Thus, finding the maximum flow in the transformed network G' is perfectly equivalent to finding the maximum bipartite matching in G . ■

BUET · CSE 105

DSA

NP-Completeness & Approximation

P, NP, NPC, Reductions, Approximation Algorithms, TSP

T20 — NP-Completeness & Approximation

- Q1.** Discuss the approximability of the general Traveling Salesperson Problem (TSP). Give an approximation algorithm for the Euclidean TSP problem and compute the approximation ratio of your algorithm. **(2+3+5 marks)** [CSE 207 | 2017–18]

Answer: Approximability of General & Euclidean TSP

1. Approximability of General TSP:

For the general Traveling Salesperson Problem (where the triangle inequality does not necessarily hold), **no constant-factor approximation algorithm exists** unless $P = NP$. If we could approximate general TSP within any constant factor $\rho \geq 1$, we could use it to solve the NP-complete Hamiltonian Cycle problem in polynomial time by assigning edge weights of 1 to edges in the graph and ∞ (or a very large number like $\rho|V| + 1$) to non-edges.

2. Approximation Algorithm for Euclidean (Metric) TSP:

In Euclidean TSP, edge weights satisfy the triangle inequality: $cost(u, w) \leq cost(u, v) + cost(v, w)$. We can use the **MST-based heuristic**:

- Compute a Minimum Spanning Tree (MST) of the given set of cities.
- Duplicate every edge in the MST to create an Eulerian graph (every degree is even).
- Find an Eulerian Tour in this multigraph.
- Convert the Eulerian tour into a Hamiltonian Cycle by bypassing (shortcutting) previously visited vertices.

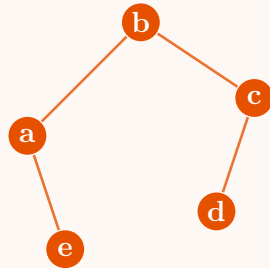
3. Approximation Ratio:

Let OPT be the cost of the optimal TSP tour.

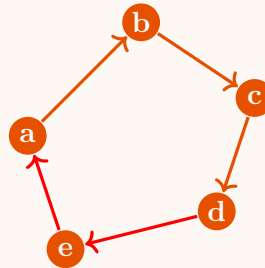
- Removing one edge from the optimal tour gives a spanning path, which is a spanning tree. Therefore, $cost(MST) \leq OPT$.
- The Eulerian tour uses each MST edge twice, so $cost(EulerianTour) = 2 \times cost(MST) \leq 2 \times OPT$.
- Because of the **triangle inequality**, shortcutting nodes never increases the total cost.
- Thus, $cost(Algorithm) \leq cost(EulerianTour) \leq 2 \times OPT$.

The approximation ratio is **2**.

1. MST



2. Shortcuts



Q2. What do you mean by $X \leq_p Y$? Show that the vertex cover problem $\leq_p$ set cover problem. (3+7 marks) [CSE 207 | 2017–18]

Answer: Polynomial-Time Reduction ($X \leq_p Y$) & VC to SC

1. What is $X \leq_p Y$?

$X \leq_p Y$ means problem X is **polynomial-time reducible** to problem Y . This implies there exists a polynomial-time computable function f that transforms any instance x of X into an instance $f(x)$ of Y such that: x is a "yes" instance of X if and only if $f(x)$ is a "yes" instance of Y . It signifies that Y is at least as hard as X .

2. Vertex Cover $\leq_p$ Set Cover:

To show this, we must map any instance of Vertex Cover (VC) to an instance of Set Cover (SC) in polynomial time.

- **Given (VC Instance):** A graph $G = (V, E)$ and an integer k . We want to find if there is a subset of vertices $V' \subseteq V$ of size $\leq k$ such that every edge in E is incident to at least one vertex in V' .
- **Transformation (to SC):**
 - Let the **Universe** U for the set cover be the set of all edges in G ($U = E$).
 - Create a **Set** S_v for each vertex $v \in V$, defined as all the edges incident to v : $S_v = \{e \in E \mid e \text{ is connected to } v\}$.
 - The target number of sets is k .
- **Proof of Equivalence:**
 - ($\implies$) If G has a vertex cover of size k , selecting the corresponding sets S_v for those vertices will cover all elements in U (since every edge is incident to at least one selected vertex). This gives a set cover of size k .
 - ($\impliedby$) If the constructed instance has a set cover of size k , the sets chosen correspond to k vertices in V . Since all elements (edges) in U are covered by these sets, every edge is incident to at least one of these k vertices, forming a valid vertex cover of size k .

Since constructing the universe and sets takes polynomial time $O(|V| + |E|)$, we have successfully shown $\text{Vertex Cover} \leq_p \text{Set Cover}$.

Q3. Define the satisfiability problem. Show that the independent set problem is NP-hard by reducing the 3-SAT problem to the independent set problem. **(3+7 marks)** [CSE 207 | 2017–18]

Answer: SAT Definition & 3-SAT to Independent Set Reduction

1. Satisfiability Problem (SAT):

Given a boolean formula composed of variables, AND ($\wedge$), OR ($\vee$), and NOT ($\neg$) operators, is there an assignment of TRUE or FALSE to the variables that makes the entire formula evaluate to TRUE?

2. 3-SAT $\leq_p$ Independent Set:

To prove Independent Set (IS) is NP-hard, we reduce 3-SAT (which is known to be NP-complete) to IS.

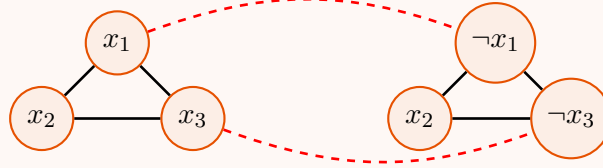
- **Given (3-SAT Instance):** A boolean formula ϕ in Conjunctive Normal Form (CNF) with k clauses, where each clause has exactly 3 literals. (e.g., $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$).
- **Transformation (to IS):** Construct a graph $G = (V, E)$ as follows:
 - (a) **Triangles:** For each clause, create 3 vertices representing its 3 literals. Connect these 3 vertices as a triangle (clique of size 3).
 - (b) **Conflict Edges:** Add an edge between any two vertices representing contradictory literals (e.g., x_i and $\neg x_i$) across different clauses.

Set the target independent set size to k (the number of clauses).

Proof of Equivalence:

- ($\implies$) If ϕ is satisfiable, at least one literal in each clause is TRUE. Pick exactly one TRUE literal node per clause triangle to form our set. Because no variable can be both TRUE and FALSE simultaneously, no two chosen nodes will be connected by a conflict edge. Since they are from different triangles, there are no triangle edges between them. This forms an independent set of size k .
- ($\impliedby$) If G has an independent set of size k , it must contain exactly one vertex from each of the k triangles (since choosing two from the same triangle violates independence). Assign TRUE to the literals represented by these chosen vertices. Conflict edges ensure we never assign TRUE to both x_i and $\neg x_i$. This assignment makes at least one literal TRUE in every clause, satisfying ϕ .

Clause 1: $(x_1 \vee x_2 \vee x_3)$ Clause 2: $(\neg x_1 \vee x_2 \vee \neg x_3)$



Q4. Give an approximation algorithm for the set cover problem and compute the approximation ratio. **(3+7 marks)** [CSE 207 | 2017–18]

Answer: Approximation Algorithm for Set Cover

1. Greedy Approximation Algorithm:

Algorithm 148 Greedy Set Cover($U, \mathcal{F}$)

```

1:  $C \leftarrow \emptyset$  ▷ Elements covered so far
2:  $\mathcal{S} \leftarrow \emptyset$  ▷ Selected sets
3: while  $C \neq U$  do
4:   Find the set  $S \in \mathcal{F}$  that maximizes  $|S \setminus C|$  ▷ Set covering most uncovered
   elements
5:    $\mathcal{S} \leftarrow \mathcal{S} \cup \{S\}$ 
6:    $C \leftarrow C \cup S$ 
7: end while
8: return  $\mathcal{S}$ 

```

2. Approximation Ratio:

The greedy algorithm achieves an approximation ratio of $H_n \approx \ln n$, where n is the total number of elements in the universe U , and $H_n = \sum_{i=1}^n \frac{1}{i}$ is the n -th harmonic number.

Brief intuition for the ratio: If the optimal solution uses OPT sets to cover n elements, then at the first step, there must be some set covering at least n/OPT elements. The greedy algorithm picks the set covering the maximum uncovered elements, so it leaves at most $n(1 - \frac{1}{OPT})$ elements uncovered. After k steps, the remaining uncovered elements are bounded by $n(1 - \frac{1}{OPT})^k$. Solving for when this becomes less than 1 gives $k \approx OPT \ln n$.

Q5. Your friend claims to have developed a polynomial-time algorithm to solve the set-packing problem. What will happen if this claim is proved to be true? **(5 marks)** [CSE 207 | 2017–18]

Answer: Consequence of solving Set-Packing in Polynomial Time

The Set-Packing problem is fundamentally known to be **NP-complete**.

If your friend's claim is mathematically proven to be true—meaning there is a verified polynomial-time algorithm to solve Set-Packing—it would directly imply that $P = NP$.

Consequences of this proof:

- (a) **All NP problems become P:** By definition of NP-completeness, every problem in NP (like TSP, SAT, Graph Coloring, Knapsack) can be reduced to Set-Packing in polynomial time. If Set-Packing is in P, all NP problems can be solved in polynomial time.
- (b) **Cryptography Collapse:** Modern cryptography (like RSA encryption, public-key infrastructure) relies on the assumption that certain mathematical problems (like integer factorization) are hard to solve but easy to verify (in NP but not in P). If $P=NP$, these cryptographic systems could be broken easily.
- (c) **AI & Optimization:** Hard scheduling, routing, and machine learning optimization problems could be solved perfectly and efficiently.

In short, it would represent the greatest breakthrough in theoretical computer science history and win the \$1 Million Clay Mathematics Institute Millennium Prize.

Q6. Suppose $X \in \text{NP-complete}$. Prove that $X \in P$ if and only if $P = NP$. (10 marks)
[CSE 207 | 2016–17]

Answer: Proof of $X \in P \iff P = NP$

Let X be an NP-complete problem. By definition, this means: 1. $X \in \text{NP}$ 2. Every problem $Y \in \text{NP}$ is polynomial-time reducible to X ($Y \leq_p X$).

We need to prove a bi-directional implication: $X \in P \iff P = NP$.

Part 1: Prove $P = NP \implies X \in P$

- Assume $P = NP$.
- Since X is NP-complete, we know $X \in \text{NP}$.
- Because we assumed $P = NP$, any problem in NP is also in P.
- Therefore, $X \in P$.

Part 2: Prove $X \in P \implies P = NP$

- Assume $X \in P$. This means there exists a deterministic algorithm that solves X in polynomial time $O(n^k)$.
- Let Y be any arbitrary problem in NP.

- Because X is NP-complete, $Y \leq_p X$. This means there is a polynomial-time reduction function f that transforms an instance of Y (size n) into an instance of X in $O(n^c)$ time.
- To solve Y , we can first run the reduction function f to get an instance of X , and then run the polynomial-time algorithm for X .
- The total time to solve Y is the time for reduction + time to solve X : $O(n^c) + O((n^c)^k) = O(n^{ck})$, which is still a polynomial.
- Thus, every problem $Y \in \text{NP}$ can be solved in polynomial time, meaning $\text{NP} \subseteq P$.
- Since it is trivially true that $P \subseteq \text{NP}$, we conclude that $P = \text{NP}$.

This completes the proof. ■

Q7. Given clauses $C_1, C_2, \dots, C_k$ each of length 3 over Boolean variables (3-SAT), and a graph G with number K (Vertex Cover asks if G has a vertex cover of size $\leq K$). Prove that Vertex Cover is at least as hard as 3-SAT. **(15 marks)** [CSE 207 | 2016–17]

Answer: Vertex Cover is NP-Hard ($3\text{-SAT} \leq_p \text{Vertex Cover}$)

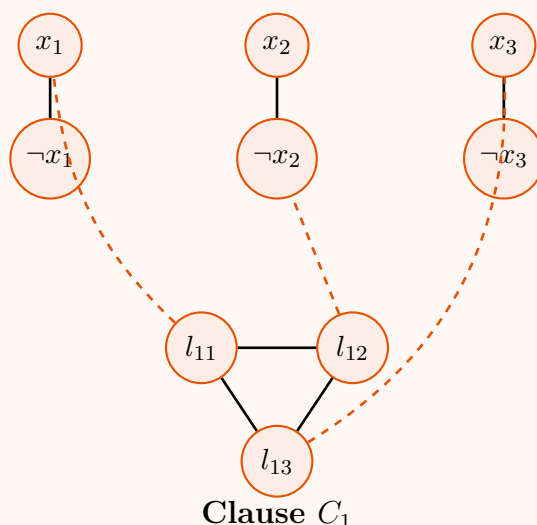
To prove Vertex Cover (VC) is at least as hard as 3-SAT, we must show a polynomial-time reduction from 3-SAT to VC. Let a 3-SAT instance have n variables $(x_1, \dots, x_n)$ and m clauses $(C_1, \dots, C_m)$.

1. Construction of Graph $G = (V, E)$:

- **Variable Gadgets:** For each variable x_i , create two vertices connected by an edge: $(x_i, \neg x_i)$. To cover this edge, any VC must pick at least one of these two vertices.
- **Clause Gadgets:** For each clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$, create a triangle of 3 vertices. To cover the 3 edges of this triangle, any VC must pick at least 2 vertices.
- **Connecting Edges:** Connect each vertex in a clause triangle to the corresponding literal vertex in the variable gadget.
- **Target size:** Set $K = n + 2m$.

2. Diagram of the Reduction:

Example for a single clause $C_1 = (x_1 \vee \neg x_2 \vee x_3)$:



3. Proof of Equivalence:

- ($\implies$) If the 3-SAT formula is satisfiable, there is a truth assignment. For the VC, pick the n vertices corresponding to the TRUE literals. For each clause, at least one literal is TRUE, so its connection to the clause triangle is covered by the variable gadget. We then pick the *other* 2 vertices in the clause triangle to cover the triangle itself. Total vertices chosen $= n + 2m = K$. This is a valid vertex cover.
- ($\impliedby$) If G has a VC of size K , it must contain exactly 1 vertex from each variable gadget (total n) and exactly 2 vertices from each clause triangle (total $2m$). The uncovered node in each clause triangle relies on the connecting edge to be covered by a vertex in the variable gadget. Thus, at least one literal per clause must be selected in the variable gadgets. By assigning TRUE to these selected literals, the 3-SAT formula is satisfied.

Q8. Prove that $\text{NP} \neq \text{co-NP} \implies P \neq \text{NP}$. (10 marks)

[CSE 207 | 2016–17]

Answer: Proof of $\text{NP} \neq \text{co-NP} \implies P \neq \text{NP}$

We will prove this statement by proving its **contrapositive**:

If $P = \text{NP}$, then $\text{NP} = \text{co-NP}$.

Proof:

- Assume $P = \text{NP}$.
- Let L be an arbitrary language in NP. Because $P = \text{NP}$, $L \in P$.
- The class P is closed under complementation. This is because if a deterministic Turing machine can solve L in polynomial time, we can simply invert its output (swap ACCEPT and REJECT states) to decide the complement language $\bar{L}$ in polynomial time.

- (d) Therefore, since $L \in P$, its complement $\bar{L} \in P$.
- (e) Because $P = NP$, it follows that $\bar{L} \in NP$.
- (f) By the definition of co-NP, if the complement of a language is in NP ($\bar{L} \in NP$), then the original language is in co-NP ($L \in \text{co-NP}$).
- (g) Thus, we have shown that every $L \in NP$ is also in co-NP, which means $NP \subseteq \text{co-NP}$.
- (h) By a completely symmetric argument, we can start with $K \in \text{co-NP} \implies \bar{K} \in NP = P \implies K \in P = NP$, yielding $\text{co-NP} \subseteq NP$.
- (i) Since both contain each other, $NP = \text{co-NP}$.

Since we have proven $(P = NP) \implies (NP = \text{co-NP})$, the contrapositive mathematically holds strictly true: $NP \neq \text{co-NP} \implies P \neq NP$. ■

Q9. Give an approximation algorithm for the vertex cover problem. Analyze the approximation ratio of your algorithm. **(12 marks)** [CSE 207 | 2016–17]

Answer: Approximation Algorithm for Vertex Cover

1. The Approximation Algorithm:

A simple and elegant greedy algorithm can be used to find a 2-approximation for the Vertex Cover problem.

Algorithm 149 Approx-Vertex-Cover($G = (V, E)$)

```

1:  $C \leftarrow \emptyset$                                 ▷ Initialize cover as empty
2:  $E' \leftarrow E$                                 ▷ Copy of edges
3: while  $E' \neq \emptyset$  do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$                     ▷ Add BOTH endpoints to the cover
6:   Remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7: end while
8: return  $C$ 
    
```

2. Analysis of the Approximation Ratio:

- Let A be the set of edges picked by the algorithm in step 4.
- Notice that because we remove all edges incident to u and v immediately after picking (u, v) , **no two edges in A share a vertex**. The set of edges A forms a *maximal matching*.
- To cover the edges in A , ANY valid vertex cover—including the optimal vertex cover C^* —must pick at least one distinct vertex for every single edge in A .

Therefore, the size of the optimal cover is bounded by the number of edges in A :

$$|C^*| \geq |A|$$

- Our algorithm picks exactly 2 vertices for every edge it selects in A . Therefore, the size of our cover C is exactly:

$$|C| = 2|A|$$

- Combining these two inequalities:

$$|C| = 2|A| \leq 2|C^*|$$

Thus, the algorithm always returns a vertex cover that is at most twice the size of the optimal one. The **approximation ratio is exactly 2**.

Q10. Compare the techniques of brute-force algorithms, heuristics, and approximation algorithms for coping with hard problems. Write a backtracking algorithm to color a map with no more than four colors. **(5+5=10 marks)** [CSE 207 | 2015–16]

Answer: Brute-Force vs Heuristics vs Approx & 4-Coloring

1. Comparison of Techniques for Hard Problems:

- **Brute-Force Algorithms:** Systematically exhaust all possible candidate solutions to find the exact optimal one. *Pros:* Guarantees 100% optimal accuracy. *Cons:* Takes exponential time ($O(2^n)$ or $O(n!)$), making it impossible to use for large inputs.
- **Heuristics:** Uses rules of thumb or "common sense" to find a "good enough" solution quickly (e.g., Genetic algorithms, Simulated Annealing). *Pros:* Very fast in practice. *Cons:* Provides absolutely **no mathematical guarantee** on how close the solution is to the optimum, or if it will even work.
- **Approximation Algorithms:** Polynomial-time algorithms that sacrifice exact optimality for speed, but provide a **provable mathematical guarantee** (ratio) that the solution will not be worse than a certain factor of the optimum.

2. Backtracking Algorithm for 4-Coloring a Map:

Given a graph where vertices represent map regions and edges represent adjacent borders. Colors are represented by 1, 2, 3, 4.

Algorithm 150 GraphColoring(node)

```

1: if node == total_nodes then
2:     return TRUE                                ▷ All nodes are successfully colored
3: end if
4: for  $c \leftarrow 1$  to 4 do                        ▷ Try all 4 colors
5:     if IsSafe(node,  $c$ ) then                    ▷ Check if no adjacent node has color  $c$ 
6:         color[node]  $\leftarrow c$                 ▷ Assign color
7:         if GraphColoring(node + 1) == TRUE then
8:             return TRUE                        ▷ Proceed to next node
9:         end if
10:        color[node]  $\leftarrow 0$                 ▷ BACKTRACK: Remove color if it led to failure
11:    end if
12: end for
13: return FALSE                                ▷ Trigger backtracking

```

Q11. Explain the term “reducibility”. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, prove that $P = NP$. (10 marks) [CSE 207 | 2015–16]

Answer: Reducibility & $P = NP$ Proofs**1. Concept of Reducibility:**

Reducibility is a technique to compare the relative difficulty of two problems. We say a language L_1 is polynomial-time reducible to L_2 (denoted as $L_1 \leq_p L_2$) if there exists a deterministic polynomial-time computable function f such that for all inputs x :

$$x \in L_1 \iff f(x) \in L_2$$

This implies that L_2 is at least as hard as L_1 , because any algorithm that solves L_2 can be used to solve L_1 with only a polynomial overhead.

2. Prove $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$:

- Since $L_2 \in P$, there exists a deterministic algorithm that decides L_2 in $O(n^k)$ time for some constant k .
- Since $L_1 \leq_p L_2$, the transformation function $f(x)$ takes $O(n^c)$ time, and the size of the transformed output $f(x)$ is at most $O(n^c)$.
- To solve L_1 for an input x of size n , we first compute $f(x)$ and then run the algorithm for L_2 on $f(x)$.
- The total time is the reduction time + execution time: $O(n^c) + O((n^c)^k) = O(n^{ck})$.
- Since ck is a constant, the total time is polynomial. Therefore, $L_1 \in P$. ■

3. Prove: If any NP-complete problem is in P, then $P = NP$:

- Let X be an NP-complete problem, and suppose $X \in P$.
- By the definition of NP-completeness, for every problem $Y \in NP$, it holds that $Y \leq_p X$.
- Using the theorem proved just above: since $Y \leq_p X$ and $X \in P$, it must be true that $Y \in P$.
- Since every problem in NP is also in P, we have $NP \subseteq P$.
- Because it is inherently true that $P \subseteq NP$, we conclude $P = NP$. ■

Q12. What is the vertex cover problem? Write a polynomial-time approximation algorithm for it. Prove its time complexity and find the approximation ratio. For the general traveling-salesman problem, prove that one cannot find good approximate tours in polynomial time unless $P = NP$. (7+8=15 marks) [CSE 207 | 2015–16]

Answer: Vertex Cover Approx & TSP Inapproximability

1. Vertex Cover Problem:

Given an undirected graph $G = (V, E)$, the Vertex Cover problem asks for the minimum size subset of vertices $V' \subseteq V$ such that every edge in E is incident to at least one vertex in V' .

2. Polynomial-Time Approximation Algorithm:

Algorithm 151 Approx-VC(V, E)

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   Pick an arbitrary edge  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
7: end while
8: return  $C$ 

```

3. Time Complexity and Approximation Ratio:

- **Time Complexity:** Using adjacency lists, finding an edge and removing incident edges takes time proportional to the degrees of the vertices. Over the entire loop, each edge is processed at most once. Thus, the time complexity is $O(|V| + |E|)$, which is strictly polynomial.
- **Approximation Ratio:** Let the algorithm pick a set of edges A . No two edges in A share a vertex (it's a maximal matching). To cover A , the optimal cover C^* must pick at least one vertex per edge in A , so $|C^*| \geq |A|$. Our algorithm picks exactly 2 vertices per edge, so $|C| = 2|A|$. Thus, $|C| \leq 2|C^*|$. The **approximation ratio is 2**.

4. General TSP Inapproximability Proof:

- Suppose, for the sake of contradiction, there is a polynomial-time approximation algorithm A for general TSP with a constant ratio $\rho \geq 1$.
- We can use A to solve the NP-complete Hamiltonian Cycle (HC) problem. Given a graph $G = (V, E)$, construct a complete graph G' with the same vertices. Assign edge weights: $w(u, v) = 1$ if $(u, v) \in E$, and $w(u, v) = \rho|V| + 1$ if $(u, v) \notin E$.
- If G has a HC, G' has a TSP tour of cost exactly $|V|$. Algorithm A must return a tour of cost $\leq \rho|V|$.
- If G has NO HC, any TSP tour in G' must use at least one non-edge, making its cost $\geq \rho|V| + 1 + (|V| - 1) > \rho|V|$.
- Thus, A returns $\leq \rho|V|$ if and only if G has a HC. Since A runs in polynomial time, we just solved HC in polynomial time, implying $P = NP$. ■

Q13. (c) Consider the following instance of the maximum satisfiability (MAX-SAT) problem.

Set of Boolean variables, $V = \{w, x, y, z\}$

Set of clauses, $C = \{C1, C2, C3, C4, C5, C6\}$

where $C1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C2 = (w \vee \bar{y})$, $C3 = (x \vee \bar{y})$, $C4 = (y \vee \bar{z})$
 $C5 = (z \vee \bar{w})$, $C6 = (\bar{w} \vee \bar{x})$

Now find an assignment of variables such that maximum number of clauses is satisfied. Compute your solution using branch and bound algorithm. Write down a suitable bound function for the problem. Show detailed calculation steps through a branch and bound search tree. **(12 marks)** [CSE 207 | 2014–15]

Answer: MAX-SAT via Branch and Bound

Given 6 Clauses: $C1 = w \vee \bar{x} \vee \bar{y} \vee z$, $C2 = w \vee \bar{y}$, $C3 = x \vee \bar{y}$
 $C4 = y \vee \bar{z}$, $C5 = z \vee \bar{w}$, $C6 = \bar{w} \vee \bar{x}$

Bound Function (B):

$$B = \text{Total Clauses} - \text{Number of Clauses Evaluated to FALSE}$$

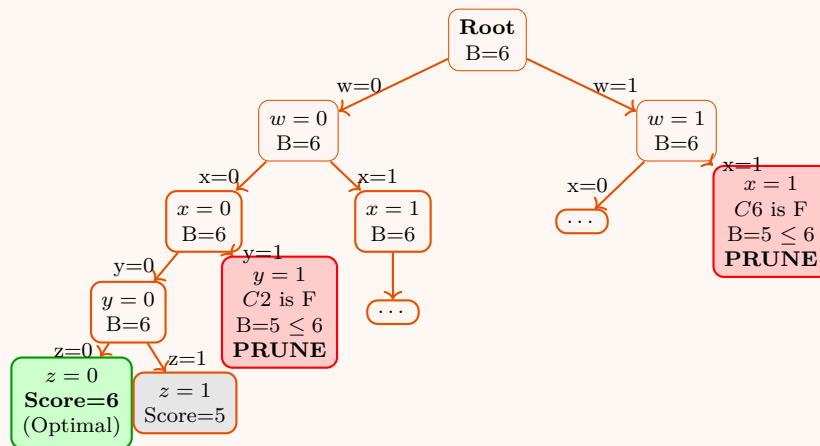
This represents the absolute *maximum* number of clauses that can still be satisfied from a current state. If $B \leq \text{Best_Found}$, we prune the branch.

Branch & Bound Search Steps (Order: $w \rightarrow x \rightarrow y \rightarrow z$, testing 0/False then 1/True):

- Root:** $B = 6$. $\text{Best} = 0$.
- w=0 (False):** $C5(z \vee T) \rightarrow T$, $C6(T \vee \bar{x}) \rightarrow T$. Remaining to check: $C1, C2, C3, C4$. No clause is FALSE yet. $B = 6$.

- (c) $w=0, x=0$: $C1(F \vee T \vee \bar{y} \vee z) \rightarrow T$. Remaining: $C2(\bar{y}), C3(F \vee \bar{y}), C4(y \vee \bar{z})$. No clause FALSE. $B = 6$.
- (d) $w=0, x=0, y=0$: $C2(T) \rightarrow T, C3(T) \rightarrow T$. Remaining: $C4(F \vee \bar{z})$. No clause FALSE. $B = 6$.
- (e) $w=0, x=0, y=0, z=0$: $C4(T) \rightarrow T$. All 6 clauses are TRUE! **Best_Found = 6**.
- (f) $w=0, x=0, y=0, z=1$: $C4$ becomes FALSE. Score = 5. (Ignored, $5 < 6$).
- (g) $w=0, x=0, y=1$: $C2$ becomes FALSE! $B = 6 - 1 = 5$. Since $5 \leq \text{Best_Found}(6)$, **PRUNE**.
- (h) $w=1$ (**True**): $C6(F \vee \bar{x})$. $B = 6$.
- (i) $w=1, x=1$: $C6(F \vee F) \rightarrow \text{FALSE}$! $B = 6 - 1 = 5$. Since $5 \leq 6$, **PRUNE** immediately!

Optimal Assignment: $w = 0, x = 0, y = 0, z = 0$ (or False, False, False, False).
Max satisfied = 6.



Q14. Define the classes P, NP, and NP-complete. What are the steps to prove a problem X is NP-complete? Prove that the decision version of the independent set problem is NP-complete using a reduction from 3-CNF-SAT. Give an intuitive sketch of the proof of the Cook-Levin theorem (CIRCUIT-SAT is NP-complete). **(4+4+12+5 marks)** [CSE 207 | 2014–15]

Answer: Classes, NP-Completeness Proofs & Cook-Levin

1. Definitions:

- **P:** Problems solvable by a deterministic Turing Machine in polynomial time.
- **NP:** Problems whose solutions can be *verified* by a deterministic Turing Machine in polynomial time.

- **NP-complete:** A problem X is NP-complete if it is in NP, and every problem in NP is polynomial-time reducible to X .

2. Steps to prove X is NP-complete: 1. Prove $X \in NP$ (Show a poly-time verifier). 2. Select a known NP-complete problem Y . 3. Formulate a polynomial-time reduction $Y \leq_p X$. 4. Prove correctness: instance $y \in Y \iff f(y) \in X$.

3. Proof: Independent Set (IS) is NP-complete (Reduction from 3-SAT):

- **IS $\in$ NP:** Given a graph and a set of vertices S , we can verify in $O(|V|^2)$ time if $|S| \geq k$ and no two vertices in S share an edge.
- **Reduction ($3\text{-SAT} \leq_p \text{IS}$):** For each clause in 3-SAT, create a triangle of 3 vertices (one for each literal). Connect conflicting literals (e.g., x and $\bar{x}$) across different triangles with edges. Set IS target $k = \text{number of clauses}$.
- **Equivalence:** If 3-SAT is satisfiable, pick one TRUE literal node per triangle. They won't have conflict edges between them, forming an IS of size k . Conversely, an IS of size k must pick exactly one node per triangle (no two in same triangle). Assigning these to TRUE cannot cause conflicts (due to conflict edges), satisfying all clauses.

4. Intuitive Sketch of Cook-Levin Theorem (CIRCUIT-SAT is NP-complete):

By definition, any problem in NP has a polynomial-time verifier algorithm (Turing Machine). The step-by-step logic gates (state transitions) of this Turing Machine operating for a polynomial number of steps can be literally "unrolled" and mapped into a massive, but polynomially-sized, Boolean logic circuit. The inputs to the circuit are the problem instance and a candidate certificate. The output is 1 if it accepts. Thus, finding a valid certificate for ANY NP problem is perfectly mathematically equivalent to finding a set of boolean inputs that make this specific CIRCUIT output 1 (which is the CIRCUIT-SAT problem).

Q15. Write the names of four NP-complete problems. Define the following six classes of problems: P, Co-P, NP, Co-NP, NP-complete, and NP-hard. By a diagram show the relationship among these classes that most researchers regard as most likely. **(10 marks)** [CSE 207 | 2013–14]

Answer: Complexity Classes Definitions & Relationships

1. Four NP-complete problems:

1. 3-SAT (Satisfiability)
2. Traveling Salesperson Problem (Decision)
3. Vertex Cover
4. Clique Problem

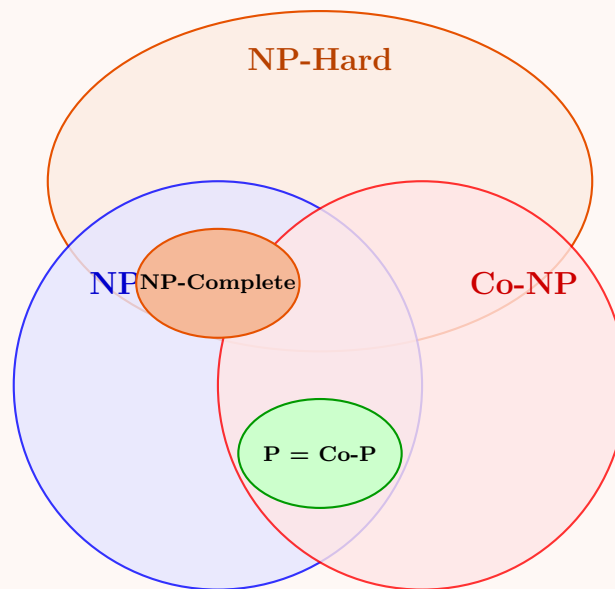
2. Definitions of Six Classes:

- **P:** Problems solvable in polynomial time.
- **Co-P:** Problems whose complement is in P. (Note: mathematically, $P = \text{Co-P}$)

P).

- **NP:** Problems for which a "YES" answer can be verified in polynomial time.
- **Co-NP:** Problems for which a "NO" answer (counter-example) can be verified in polynomial time.
- **NP-complete:** The hardest problems in NP; every NP problem reduces to them.
- **NP-hard:** Problems at least as hard as NP-complete problems, but they do not necessarily have to be in NP (e.g., Halting Problem).

3. Relationship Diagram (Assuming $P \neq NP$):



Q16. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If an NP-complete problem is polynomially solvable, prove that $P = NP$. (10 marks) [CSE 207 | 2013–14]

Answer: Reducibility & $P = NP$ Proofs

1. Prove $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$:

- $L_1 \leq_p L_2$ means there exists a polynomial-time computable reduction function f such that $x \in L_1 \iff f(x) \in L_2$. Let the time taken to compute $f(x)$ be $O(n^c)$.
- Since $L_2 \in P$, there exists a deterministic algorithm that decides L_2 in polynomial time $O(m^k)$, where m is the size of the input to L_2 .
- To solve L_1 for an input x of size n , we first compute $f(x)$. The size of $f(x)$ is at most $O(n^c)$. Then, we run the algorithm for L_2 on $f(x)$.

- The total time is the reduction time + execution time: $O(n^c) + O((n^c)^k) = O(n^{ck})$.
- Since c and k are constants, the total time is a polynomial $O(n^{ck})$. Therefore, $L_1 \in P$. ■

2. Prove: If an NP-complete problem is polynomially solvable, $P = NP$:

- Let X be an NP-complete problem, and suppose $X \in P$ (it is polynomially solvable).
- By the definition of NP-completeness, for every problem $Y \in NP$, it holds that $Y \leq_p X$.
- Using the theorem proved just above: since $Y \leq_p X$ and $X \in P$, it must be true that $Y \in P$.
- Since every problem in NP is also in P, we have $NP \subseteq P$.
- Because it is inherently true that $P \subseteq NP$, we conclude that $P = NP$. ■

Q17. Write a polynomial-time 2-approximation algorithm for the traveling-salesman problem where the edge weights satisfy the triangle inequality, and prove the approximation ratio. If $P = NP$, prove that for any constant $\rho > 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general TSP. **(15 marks)** [CSE 207 | 2013–14]

Answer: Metric TSP Approximation & General TSP Inapproximability

1. Polynomial-time 2-Approximation for Metric TSP:

The algorithm leverages the Minimum Spanning Tree (MST):

- Compute an MST of the given complete graph using Prim's or Kruskal's algorithm.
- Duplicate every edge in the MST to create an Eulerian multigraph.
- Find an Eulerian Tour (which visits every vertex at least once and traverses every edge exactly once).
- Convert the Eulerian tour into a Hamiltonian Cycle by "shortcutting" (skipping previously visited vertices).

Proof of Approximation Ratio: Let OPT be the cost of the optimal TSP tour.

- Removing any edge from the optimal tour yields a spanning tree. Thus, the cost of the MST is a lower bound: $cost(MST) \leq OPT$.
- Our Eulerian tour uses every MST edge twice, so $cost(Eulerian) = 2 \times cost(MST) \leq 2 \times OPT$.

- Because the graph satisfies the **triangle inequality**, shortcutting a node v to go directly from u to w never increases the cost ($\text{cost}(u, w) \leq \text{cost}(u, v) + \text{cost}(v, w)$).
- Thus, $\text{cost}(\text{Algorithm}) \leq \text{cost}(\text{Eulerian}) \leq 2 \times \text{OPT}$. The ratio is strictly **2**. ■

2. General TSP Inapproximability:

(Note: The question states "If $P = NP$...", which is a typo. If $P = NP$, exact poly-time algorithms exist. The theorem correctly assumes **If** $P \neq NP$.)

Proof: Suppose, for contradiction, there is a poly-time approx algorithm A for general TSP with a constant ratio $\rho > 1$. We use A to solve the NP-complete Hamiltonian Cycle (HC) problem:

- Given an unweighted graph $G = (V, E)$, construct a complete graph G' with weight $w(u, v) = 1$ if $(u, v) \in E$, and $w(u, v) = \rho|V| + 1$ if $(u, v) \notin E$.
- If G has a HC, G' has a TSP tour of cost exactly $|V|$. Algorithm A must return a tour of cost $\leq \rho|V|$.
- If G has NO HC, any TSP tour in G' must use at least one non-edge, making its cost $\geq (\rho|V| + 1) + (|V| - 1) > \rho|V|$.
- Thus, if algorithm A returns a cost $\leq \rho|V|$, G has a HC; otherwise, it doesn't. Since A is polynomial, we have solved HC in polynomial time, which implies $P = NP$. Assuming $P \neq NP$, this is a contradiction. ■

Q18. Show that the decision version of the vertex cover optimization problem is NP-complete using a reduction from the Independent Set decision problem. **(12 marks)**
[CSE 207 | 2012–13]

Answer: Decision Vertex Cover is NP-Complete

1. Vertex Cover $\in$ NP:

Given a graph $G = (V, E)$, an integer k , and a proposed vertex cover $C \subseteq V$ as a certificate, we can verify if $|C| \leq k$ and if every edge in E has at least one endpoint in C . This checking takes $O(|V| + |E|)$ time, which is polynomial.

2. Reduction (Independent Set $\leq_p$ Vertex Cover):

We know the Independent Set (IS) decision problem is NP-complete. We map an instance of IS, $\langle G = (V, E), k \rangle$, to an instance of VC, $\langle G' = (V', E'), k' \rangle$, as follows:

- G' is exactly the same graph as G . ($V' = V, E' = E$)
- $k' = |V| - k$.

This mapping takes $O(1)$ time (or $O(|V|)$ to count vertices), which is polynomial.

3. Proof of Equivalence:

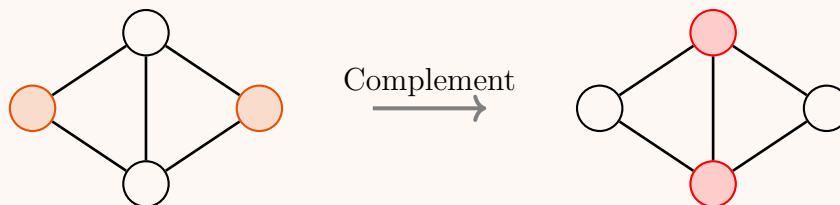
Theorem: A subset $S \subseteq V$ is an independent set in G if and only if its complement

$V \setminus S$ is a vertex cover in G .

- ($\implies$) Let S be an IS of size k . For any edge $e = (u, v) \in E$, u and v cannot both be in S (by definition of IS). Therefore, at least one of u or v must be in the complement set $V \setminus S$. Thus, $V \setminus S$ covers all edges and forms a VC of size $|V| - k$.
- ($\impliedby$) Let $C = V \setminus S$ be a VC of size $|V| - k$. Suppose for contradiction that S is NOT an independent set. Then there must exist an edge $e = (u, v)$ where both $u, v \in S$. But if both are in S , neither is in $V \setminus S$. This means the edge e is not covered by $V \setminus S$, contradicting that it is a vertex cover. Thus, S must be an IS of size k . ■

Independent Set (Size 2)

Vertex Cover (Size 4-2=2)



Q19. Prove that the optimization version of the Independent Set problem is NP-hard. (8 marks) [CSE 207 | 2012–13]

Answer: Optimization Independent Set is NP-Hard

Proof:

To prove that the Optimization version of the Independent Set problem (Opt-IS) is NP-hard, we will show a Turing reduction from the Decision version of Independent Set (Dec-IS), which is known to be NP-complete.

- **Dec-IS Problem:** Given a graph G and an integer k , does G have an independent set of size at least k ?
- **Opt-IS Problem:** Given a graph G , find the maximum independent set in G and return its size (let's call it S_{max}).

Assume, for the sake of contradiction, that there exists a polynomial-time algorithm A_{opt} that solves the Opt-IS problem. We could then construct the following algorithm A_{dec} to solve the Dec-IS problem:

- Receive input $\langle G, k \rangle$ for Dec-IS.
- Run algorithm A_{opt} on graph G . Let the returned max size be S_{max} .
- If $S_{max} \geq k$, output **YES**.
- Else, output **NO**.

Since evaluating a simple inequality takes $O(1)$ time, the total time for A_{dec} is strictly bounded by the polynomial time of A_{opt} . This means we can solve the NP-complete Dec-IS problem in polynomial time, implying $P = NP$. Therefore, Opt-IS is at least as hard as Dec-IS. Since Dec-IS is NP-complete, Opt-IS is **NP-hard**. ■

Q20. Consider the following greedy approximation algorithm for the vertex cover optimization problem. **(15 marks)**

```

1 GreedyVertexCover(G):
2   C = {}
3   while E is not empty:           // E is the set of edges
4       u = vertex in V with maximum degree // V is the set
      of vertices
5       V = V - {u}
6       C = C union {u}
7       for each edge (u, v) adjacent to u:
8           E = E - {(u, v)}        // remove all edges covered
      by vertex u
9   return C                        // C is the vertex cover
    
```

- (a) Give an example graph where the above algorithm may not produce the optimal vertex cover.
- (b) Show that the approximation ratio of the above greedy algorithm is $O(\lceil \lg |E| \rceil)$.

[CSE 207 | 2012–13]

Answer: Greedy Vertex Cover Suboptimality & Ratio

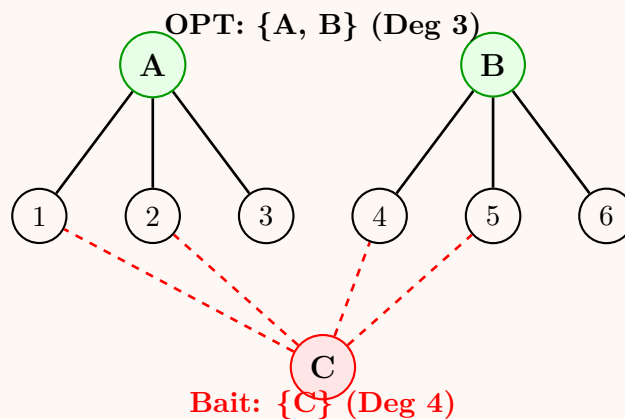
1. Example Graph where Greedy Fails:

Consider a bipartite graph with a set of "optimal" nodes $OPT = \{A, B\}$ and a "greedy bait" node C .

- Nodes A and B each have degree 3, connecting to disjoint leaf nodes: A connects to $\{1, 2, 3\}$, and B connects to $\{4, 5, 6\}$.
- The bait node C connects to a mix of these leaves, say $\{1, 2, 4, 5\}$. Thus, the degree of C is 4.

Optimal VC: $\{A, B\}$ (Size = 2). It covers all edges.

Greedy VC: 1. Max degree node is C (degree 4). Greedy picks C . Removes edges to 1, 2, 4, 5. 2. Edges $(A, 3)$ and $(B, 6)$ are still uncovered. 3. Greedy must pick A (or 3) and B (or 6) to cover them. 4. Total Greedy VC size = 3 (e.g., $\{C, A, B\}$). Thus, it fails to find the optimal size 2.



2. Show that the approximation ratio is $O(\lceil \lg |E| \rceil)$:

- The Greedy Vertex Cover algorithm is mathematically equivalent to the standard Greedy Set Cover algorithm.
- In this mapping: the "Universe" to cover is the set of all edges E . The "Sets" available are S_v for every vertex $v \in V$, where S_v contains all edges incident to v .
- At each step, picking the vertex with the maximum degree is exactly equivalent to picking the set that covers the maximum number of currently uncovered elements (edges).
- It is a proven theorem that the Greedy Set Cover algorithm achieves an approximation ratio of $H_n = \sum_{i=1}^n \frac{1}{i}$, where n is the size of the universe.
- Since our universe size is $|E|$, the approximation ratio is bounded by the harmonic number $H_{|E|}$.
- Using the mathematical bound $H_n \leq \ln n + 1$, the approximation ratio is bounded by $O(\ln |E|)$, which is asymptotically identical to $O(\lceil \lg |E| \rceil)$. ■

Q21. Give an efficient 2-approximation algorithm for vertex cover: optimization problem.
(8 marks) [CSE 207 | 2012–13]

Answer: 2-Approximation for Vertex Cover

An efficient 2-approximation algorithm for the Vertex Cover optimization problem uses the concept of maximal matching.

Algorithm:

Algorithm 152 Approx-Vertex-Cover(V, E)

```

1:  $C \leftarrow \emptyset$                                 ▷ Initialize cover as empty
2:  $E' \leftarrow E$                                 ▷ Copy of edges
3: while  $E' \neq \emptyset$  do
4:   Pick an arbitrary edge  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$                     ▷ Add BOTH endpoints
6:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
7: end while
8: return  $C$ 
    
```

Explanation: By selecting both endpoints of an arbitrary edge and removing all adjacent edges, we ensure that the set of chosen edges forms a *maximal matching* (no two selected edges share a vertex). Since any valid vertex cover must select at least one vertex from every edge in this matching, the optimal cover size is at least the number of matched edges. Because we add exactly 2 vertices per matched edge, our cover is at most twice the optimal size, yielding an $O(|V| + |E|)$ time 2-approximation algorithm.

Q22. What are the traveling-salesman problem, Euler tour problem and Hamiltonian cycle problem? If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then prove that $L_1 \leq_p L_3$. **(3+7=10 marks)**
[CSE 207 | 2011–12]

Answer: TSP, Euler Tour, Hamiltonian Cycle & Transitivity

1. Definitions:

- **Traveling-Salesman Problem (TSP):** Given a set of cities and the distances between every pair, find the shortest possible route that visits every city exactly once and returns to the starting city.
- **Euler Tour Problem:** Find a trail in a finite graph that visits every **edge** exactly once and returns to the starting vertex. (Solvable in polynomial time if all vertices have even degrees).
- **Hamiltonian Cycle Problem:** Find a cycle in a graph that visits every **vertex** exactly once and returns to the starting vertex. (NP-complete).

2. Proof of Transitivity ($L_1 \leq_p L_2$ and $L_2 \leq_p L_3 \implies L_1 \leq_p L_3$):

- Since $L_1 \leq_p L_2$, there exists a polynomial-time reduction function f such that $x \in L_1 \iff f(x) \in L_2$. Let f run in $O(|x|^a)$ time.
- Since $L_2 \leq_p L_3$, there exists a polynomial-time reduction function g such that $y \in L_2 \iff g(y) \in L_3$. Let g run in $O(|y|^b)$ time.
- Define a new reduction function $h(x) = g(f(x))$.
- **Correctness:** $x \in L_1 \iff f(x) \in L_2 \iff g(f(x)) \in L_3 \iff h(x) \in L_3$.

- **Time Complexity:** The output size $|f(x)|$ is bounded by $O(|x|^a)$. Running g on $f(x)$ takes $O(|f(x)|^b) = O((|x|^a)^b) = O(|x|^{ab})$. Since a and b are constants, ab is a constant. Thus, $h(x)$ runs in polynomial time.
- Therefore, $L_1 \leq_p L_3$. ■

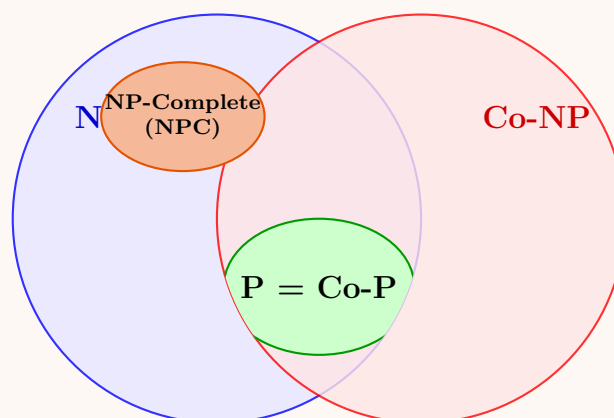
Q23. Define the following six classes of problems: P, Co-P, NP, Co-NP, NPC, and NP-hard. By a diagram show the relationship among P, Co-P, NP, Co-NP, and NPC that most researchers regard as the most likely. (6+4=10 marks) [CSE 207 | 2011–12]

Answer: Complexity Classes Definitions & Relationships

1. Definitions:

- **P:** Problems solvable by a deterministic Turing Machine in polynomial time.
- **Co-P:** Problems whose complement is in P. (Note: mathematically, $P = \text{Co-P}$).
- **NP:** Problems for which a given "YES" solution (certificate) can be verified in polynomial time.
- **Co-NP:** Problems for which a "NO" answer (counter-example) can be verified in polynomial time.
- **NPC (NP-Complete):** The hardest problems in NP. X is NPC if $X \in NP$ and every problem in NP reduces to X in polynomial time.
- **NP-hard:** Problems at least as hard as NPC problems, but they do not necessarily have to be in NP (e.g., they might not be verifiable in poly-time).

2. Relationship Diagram (Assuming $P \neq NP$ and $NP \neq \text{Co-NP}$):



Q24. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, then show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, then prove that

$P = NP$. (7+3=10 marks)

[CSE 207 | 2011–12]

Answer: Reduction to P & NP-Complete consequences**1. Prove $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$:**

- $L_1 \leq_p L_2$ means there exists a deterministic reduction function f such that $x \in L_1 \iff f(x) \in L_2$. Let the time to compute $f(x)$ be $O(n^c)$.
- Since $L_2 \in P$, there exists a deterministic algorithm that decides L_2 in time $O(m^k)$, where m is the input size.
- To solve L_1 for input x of size n , first compute $f(x)$ (takes $O(n^c)$ time, output size is at most $O(n^c)$). Then run the L_2 algorithm on $f(x)$.
- Total time = $O(n^c) + O((n^c)^k) = O(n^{ck})$. Since ck is a constant, this is polynomial time. Thus, $L_1 \in P$. ■

2. Prove: If an NP-complete problem is polynomially solvable, $P = NP$:

- Let X be an NP-complete problem, and suppose $X \in P$.
- By definition, for every problem $Y \in NP$, $Y \leq_p X$.
- Using the first theorem: since $Y \leq_p X$ and $X \in P$, it implies $Y \in P$.
- Since every problem in NP is in P, $NP \subseteq P$. Since $P \subseteq NP$ is trivially true, $P = NP$. ■

Q25. Write a polynomial-time 2-approximation algorithm for the vertex-cover problem, and prove the approximation ratio of the algorithm. If $P = NP$, then prove that for any constant $\rho \geq 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general traveling-salesman problem. (8+7=15 marks) [CSE 207 | 2011–12]

Answer: VC Approx Ratio & General TSP Inapproximability**1. VC 2-Approximation & Proof of Ratio:**

(Algorithm is provided in Solution 1. It picks an edge, adds both endpoints, and removes all incident edges).

- Let A be the set of edges picked by the algorithm.
- Because all incident edges are removed when an edge is picked, no two edges in A share a vertex (it is a *maximal matching*).
- Any valid vertex cover, including the optimal cover C^* , must pick at least one vertex for every edge in A . Therefore, $|C^*| \geq |A|$.
- Our algorithm picks exactly 2 vertices for each edge in A . Therefore, $|C| = 2|A|$.

- Combining these: $|C| = 2|A| \leq 2|C^*|$. The approximation ratio is 2. ■

2. General TSP Inapproximability:

(Note: The question states "If $P = NP$...", which is a known typographical error in exams. If $P = NP$, exact poly-time algorithms exist, making the ratio 1. The proof holds for the assumption **If** $P \neq NP$.)

Proof: Assume for contradiction there is a poly-time approximation algorithm A for general TSP with a constant ratio $\rho \geq 1$. We use A to solve the NP-complete Hamiltonian Cycle (HC) problem:

- Given graph $G = (V, E)$, construct complete graph G' with weight $w(u, v) = 1$ if $(u, v) \in E$, and $w(u, v) = \rho|V| + 1$ if $(u, v) \notin E$.
- If G has an HC, G' has a TSP tour of cost $|V|$. Algorithm A must return a tour of cost $\leq \rho|V|$.
- If G has NO HC, any TSP tour in G' must use at least one non-edge, making its cost $\geq (\rho|V| + 1) + (|V| - 1) > \rho|V|$.
- Thus, if algorithm A returns a cost $\leq \rho|V|$, G has an HC. Since A is polynomial, we solved HC in polynomial time, implying $P = NP$. Assuming $P \neq NP$, this is a contradiction. ■

Q26. Prove that the decision version of the maximum clique problem is NP-complete by reducing from 3-CNF-SAT. (17 marks) [CSE 207 | 2009–10]

Answer: Clique is NP-Complete (Reduction from 3-SAT)

1. Maximum Clique $\in$ NP:

Given a graph $G = (V, E)$ and an integer k , a proposed certificate is a subset of vertices $V' \subseteq V$ of size k . We can verify in $O(k^2)$ polynomial time if every pair of vertices in V' is connected by an edge in E .

2. Reduction (3-SAT $\leq_p$ Clique):

Let an instance of 3-SAT have k clauses: $C_1 \wedge C_2 \wedge \dots \wedge C_k$. We construct a graph $G = (V, E)$ as follows:

- Vertices:** For each clause $C_r = (l_{r1} \vee l_{r2} \vee l_{r3})$, create 3 vertices in G . Thus, $|V| = 3k$. We label a vertex v_{ri} to represent literal l_{ri} in clause C_r .
- Edges:** Add an edge between two vertices v_{ri} and v_{sj} IF AND ONLY IF:
 - (a) They belong to different clauses ($r \neq s$), AND
 - (b) They are not contradictory literals (i.e., $l_{ri} \neq \neg l_{sj}$).
- Target:** Set the target clique size to k (the number of clauses).

3. Equivalence Proof:

- ($\implies$) If the 3-SAT formula is satisfiable, there is a truth assignment that makes at least one literal TRUE in every clause. Select one such TRUE literal's vertex from each of the k clauses. Since they are all TRUE under the same assignment, no two selected literals can be contradictory (x and $\neg x$). Since we picked exactly one from each clause, they all satisfy the edge conditions. These k vertices form a clique of size k .
- ($\impliedby$) If G has a clique of size k , it must contain exactly one vertex from each clause (since there are no edges between vertices of the same clause). Because these k vertices form a clique, no two of them represent contradictory literals (otherwise the edge wouldn't exist). We can assign TRUE to all literals represented by the clique vertices. This assignment is valid and satisfies all k clauses, making the 3-SAT formula satisfiable. ■

Q27. Give a 2-approximation algorithm for the Traveling Salesperson Problem (TSP) for graphs obeying the triangle inequality. Prove the approximation ratio. (17 marks)
[CSE 207 | 2009–10]

Answer: 2-Approximation for Metric TSP

1. The Algorithm:

For a complete graph $G = (V, E)$ with edge weights satisfying the triangle inequality ($w(u, v) \leq w(u, x) + w(x, v)$):

- Find a Minimum Spanning Tree (MST) of G .
- Duplicate every edge in the MST. This creates a multigraph where every vertex has an even degree (an Eulerian graph).
- Find an Eulerian Tour T in this multigraph (a cycle that visits every edge exactly once).
- Convert T into a Hamiltonian cycle H by bypassing already visited vertices (shortcutting).

2. Proof of Approximation Ratio: Let OPT be the cost of the optimal TSP tour.

- If we remove any single edge from the optimal tour, we get a spanning tree. Since the MST is the spanning tree with the minimum possible weight, we know:

$$\text{cost}(MST) \leq OPT$$

- In step 2, we duplicate every edge of the MST, so the cost of the Eulerian tour is exactly:

$$\text{cost}(T) = 2 \times \text{cost}(MST) \leq 2 \times OPT$$

- In step 4, we use shortcuts to convert T into a valid TSP tour H . Because the graph satisfies the **triangle inequality**, going directly from vertex u to w is never strictly more expensive than going from u to v and then to w . Thus, shortcutting can only *decrease* or maintain the total cost.

$$\text{cost}(H) \leq \text{cost}(T)$$

- Combining the inequalities:

$$\text{cost}(H) \leq 2 \times \text{cost}(MST) \leq 2 \times OPT$$

Therefore, the cost of our generated tour is at most twice the optimal cost, proving the **ratio is 2**. ■

Q28. Write a 2-approximation algorithm for the “vertex cover” problem. Show that your algorithm has the mentioned approximation ratio. **(10 marks)** [CSE 207 | 2008–2009]

Answer: 2-Approximation for Vertex Cover

1. The Algorithm:

Algorithm 153 Approx-Vertex-Cover($G(V, E)$)

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   Pick an arbitrary edge  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$            ▷ Add BOTH endpoints of the edge to the cover
6:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
7: end while
8: return  $C$ 

```

2. Proof of Approximation Ratio:

- Let M be the set of edges exactly picked by the algorithm in step 4.
- Because we remove all edges incident to both u and v right after picking (u, v) , **no two edges in M share a vertex**. The set M forms a maximal matching.
- To cover the edges in M , any valid vertex cover (including the optimal cover C^*) must select at least one unique vertex for each edge in M . Therefore, the size of the optimal cover must be at least the size of M :

$$|C^*| \geq |M|$$

- Our algorithm adds exactly 2 vertices to our cover C for every edge in M . Therefore, the size of our cover is exactly:

$$|C| = 2|M|$$

- By substituting $|M|$ into the inequality:

$$|C| = 2|M| \leq 2|C^*|$$

Since $|C| \leq 2|C^*|$, the algorithm produces a vertex cover that is at most twice the size of the optimal one. The **ratio is 2**. ■

Q29. What do you mean by NP-hard and NP-complete problems? Prove that finding a clique in a graph is NP-complete. (5+12 marks) [CSE 207 | 2008–09]

Answer: NP-hard vs NP-complete & Clique NPC Proof

1. Definitions:

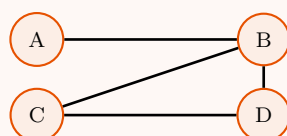
- NP-hard:** A problem X is NP-hard if every problem $Y \in \text{NP}$ is polynomial-time reducible to X ($Y \leq_p X$). NP-hard problems are at least as hard as the hardest problems in NP, but they do not have to be in NP themselves.
- NP-complete:** A problem X is NP-complete if it is **both** in NP AND is NP-hard.

2. Proof: Clique is NP-Complete (via Independent Set):

(Note: Clique was proven NPC from 3-SAT in Q1. Here is the elegant and standard reduction from Independent Set).

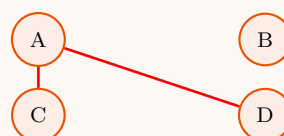
- Clique $\in \text{NP}$:** Given a graph G and an integer k , a certificate is a set of k vertices. We can verify if all pairs have edges in $O(k^2)$ time.
- Reduction ($\text{IS} \leq_p \text{Clique}$):** Given an instance of Independent Set $\langle G = (V, E), k \rangle$, we construct an instance of Clique $\langle \bar{G}, k \rangle$ where $\bar{G}$ is the *complement graph* of G . In $\bar{G}$, an edge exists between u and v IF AND ONLY IF no edge exists between them in G . This takes $O(|V|^2)$ time.
- Equivalence:** A set of vertices S of size k is an Independent Set in G iff there are NO edges between any two nodes of S in G . By definition of a complement graph, this happens iff there is an edge between EVERY pair of nodes of S in $\bar{G}$, which is the exact definition of S being a Clique of size k in $\bar{G}$. ■

Graph G (IS = {A, C})



Complement

Graph $\bar{G}$ (Clique = {A, C})



Q30. There are clauses $C_1 = (X_1 \vee \bar{X}_2)$, $C_2 = (\bar{X}_1 \vee X_2 \vee X_4 \vee \bar{X}_5)$, $C_3 = \bar{X}_2$, and $C_4 = (X_2 \vee X_3 \vee \bar{X}_4)$. How the above instance of the satisfiability problem can be transformed into an instance of 3-SAT problem? (X_1, X_2, X_3, X_4 and X_5 are

boolean variables) (13 marks)

[CSE 207 | 2008–09]

Answer: Transformation from SAT to 3-SAT

To transform a general SAT formula into 3-SAT, we must ensure every clause has exactly 3 literals. We do this by introducing dummy variables (y_i) depending on the length of the original clause.

Given Clauses: $C_1 = (X_1 \vee \overline{X_2})$

$C_2 = (\overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5})$

$C_3 = (\overline{X_2})$

$C_4 = (X_2 \vee X_3 \vee \overline{X_4})$

Transformation Steps:

- **Clause C_4 (Length = 3):**

Since C_4 already has exactly 3 literals, it remains unchanged.

$$C'_4 = (X_2 \vee X_3 \vee \overline{X_4})$$

- **Clause C_1 (Length = 2):**

Introduce a new dummy variable y_1 . We expand the clause using $(y_1 \wedge \overline{y_1}) \equiv \text{False}$.

$$C'_1 = (X_1 \vee \overline{X_2} \vee y_1) \wedge (X_1 \vee \overline{X_2} \vee \overline{y_1})$$

- **Clause C_3 (Length = 1):**

Introduce two new dummy variables y_2 and y_3 . Expand to cover all 4 combinations of y_2 and y_3 .

$$C'_3 = (\overline{X_2} \vee y_2 \vee y_3) \wedge (\overline{X_2} \vee y_2 \vee \overline{y_3}) \wedge (\overline{X_2} \vee \overline{y_2} \vee y_3) \wedge (\overline{X_2} \vee \overline{y_2} \vee \overline{y_3})$$

- **Clause C_2 (Length $k = 4 > 3$):**

For a long clause $(l_1 \vee l_2 \vee \dots \vee l_k)$, we introduce $k - 3$ dummy variables. Here, $k = 4$, so we need $4 - 3 = 1$ dummy variable, let's call it y_4 . We split the clause, grouping the first two literals with y_4 , and the rest with $\overline{y_4}$.

$$C'_2 = (\overline{X_1} \vee X_2 \vee y_4) \wedge (\overline{y_4} \vee X_4 \vee \overline{X_5})$$

Final 3-SAT Formula:

The transformed 3-SAT instance is the conjunction (AND) of all the newly generated clauses:

$$C_{3SAT} = C'_1 \wedge C'_2 \wedge C'_3 \wedge C'_4$$

This transformation takes polynomial time and preserves the satisfiability of the original formula.

Q31. Write down the procedure of proving that a problem is NP-complete. (5 marks)

[CSE 207 | 2008–09]

Answer: Procedure to Prove NP-Completeness

To prove that a given problem X is NP-complete, we must mathematically demonstrate two main properties: that X is in NP, and that X is NP-hard. The standard four-step procedure is as follows:

Step 1: Prove $X \in \text{NP}$: Show that if someone provides a proposed solution (a "certificate" or "witness") for a "YES" instance of the problem, a deterministic Turing Machine can verify the correctness of that solution in polynomial time.

Step 2: Select a known NP-complete problem Y : Choose an existing problem Y that has already been proven to be NP-complete (such as 3-SAT, Clique, or Vertex Cover) that has a structural similarity to X .

Step 3: Construct a Polynomial-Time Reduction ($Y \leq_p X$): Design an algorithm that takes an arbitrary instance y of problem Y and transforms it into an instance $f(y)$ of problem X . You must prove that this transformation algorithm runs in polynomial time, $O(n^k)$.

Step 4: Prove Equivalence (Correctness of Reduction): Prove that the transformation preserves the answer. Specifically, show that:

- If the instance y is a "YES" instance for problem Y , then $f(y)$ must be a "YES" instance for problem X .
- If $f(y)$ is a "YES" instance for problem X , then y must be a "YES" instance for problem Y .

Once these steps are completed, problem X is officially proven to be NP-complete.

Q32. Define: (i) size of a problem, (ii) time complexity function, (iii) the class P, (iv) the class NP, (v) NP-Complete problems. Show how an instance of the satisfiability problem can be transformed into an instance of 3-SAT. Prove that 3-colouring of a graph is NP-complete. **(10+10+15 marks)** [CSE 207 | 2007–08]

Q33. Prove that 3-colouring of a graph is NP-complete. [CSE 207 | 2007–08]

Answer: Definitions, 3-SAT Transform & 3-Coloring Proof**1. Definitions:**

- **(i) Size of a problem:** The number of bits (or symbols) required to formally encode an instance of the problem as an input to a Turing Machine.
- **(ii) Time complexity function:** A mathematical function $T(n)$ that represents the maximum number of basic computational steps an algorithm takes to solve a problem instance of size n .
- **(iii) The class P:** The set of decision problems that can be solved by a

deterministic Turing Machine in polynomial time $O(n^k)$.

- **(iv) The class NP:** The set of decision problems whose "YES" solutions can be *verified* by a deterministic Turing Machine in polynomial time.
- **(v) NP-Complete problems:** The hardest problems in the NP class. A problem is NP-complete if it is in NP, and every problem in NP can be reduced to it in polynomial time.

2. Transformation of SAT to 3-SAT:

Any general Boolean Satisfiability (SAT) clause can be transformed into equivalent 3-SAT clauses (where each clause has exactly 3 literals) by introducing dummy variables.

- If a clause has 1 literal: $C = (x) \implies (x \vee y \vee z) \wedge (x \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee \bar{y} \vee \bar{z})$.
- If a clause has 2 literals: $C = (x \vee y) \implies (x \vee y \vee z) \wedge (x \vee y \vee \bar{z})$.
- If a clause has $k > 3$ literals: $C = (l_1 \vee l_2 \vee \dots \vee l_k) \implies (l_1 \vee l_2 \vee y_1) \wedge (\bar{y}_1 \vee l_3 \vee y_2) \wedge \dots \wedge (\bar{y}_{k-3} \vee l_{k-1} \vee l_k)$.

3. Prove that 3-Colouring is NP-complete (Reduction from 3-SAT):

Part A: 3-Colouring $\in$ NP. Given a graph and a coloring assignment as a certificate, we can check if any two adjacent nodes have the same color in $O(|E|)$ time.

Part B: Reduction ($3\text{-SAT} \leq_p 3\text{-Colouring}$). Let the 3 colors be True (T), False (F), and Base (B).

- **Palette Triangle:** Create a triangle of 3 connected nodes: T, F, and B. They must all have different colors.
- **Literal Vertices:** For each variable x_i , create a pair of connected nodes x_i and $\bar{x}_i$. Connect both of them to the B node. This forces x_i and $\bar{x}_i$ to take the colors T and F (one must be T, the other F).
- **Clause Gadget:** For each clause $C = (l_1 \vee l_2 \vee l_3)$, construct a subgraph (gadget) using 6 extra nodes. The gadget is wired to l_1, l_2, l_3 and the T, F, B nodes such that the gadget can be validly 3-colored *if and only if* at least one of the literals l_1, l_2, l_3 is colored True (T). If all three are colored False (F), the gadget forces an adjacent pair to have the same color, making it uncolorable.

Equivalence: The formula is satisfiable iff there is a truth assignment making at least one literal per clause True. This perfectly corresponds to the graph being 3-colorable. Since the construction takes polynomial time, 3-Colouring is NP-complete. ■

Q34. Write a 2-approximation algorithm for the vertex cover problem. Justify your answer. **(10 marks)** [CSE 207 | 2006–07]

Answer: 2-Approximation for Vertex Cover**1. The Algorithm:****Algorithm 154** Approx-Vertex-Cover($G(V, E)$)

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   Pick an arbitrary edge  $e = (u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$  ▷ Add both endpoints to the cover
6:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
7: end while
8: return  $C$ 

```

2. Justification of 2-Approximation Ratio:

- Let M be the exact set of edges randomly picked by the algorithm in step 4.
- Because we remove all incident edges immediately after picking an edge (u, v) , no two edges in M share a common vertex. Thus, M is a *maximal matching*.
- To cover all edges in the graph, the optimal vertex cover (size OPT) must cover all edges in M . Since edges in M share no vertices, the optimal cover must pick at least one distinct vertex for every single edge in M . Therefore, $OPT \geq |M|$.
- Our algorithm systematically adds exactly 2 vertices for every edge in M . Thus, the size of our generated cover is exactly $|C| = 2|M|$.
- Substituting $|M|$ into the inequality gives: $|C| = 2|M| \leq 2 \times OPT$.
- Therefore, the algorithm produces a valid cover at most twice the optimal size. ■

Q35. What is the difference between NP-complete and NP-hard problems?

Prove that the satisfiability of Boolean formulas is NP-complete.

Prove that $3\text{-SAT} \leq_p \text{Independent Set}$.

Give an example of reduction by equivalence. **(5+15+10 marks)**

[CSE 207 |

2006–07]

Answer: NP-Hard vs NPC, SAT is NPC, IS Reduction, Equivalence**1. Difference between NP-complete and NP-hard:**

- **NP-hard:** A problem X is NP-hard if every problem in NP can be reduced to X in polynomial time. NP-hard problems are at least as hard as the hardest problems in NP, but they *do not necessarily* have to belong to the NP class (e.g., they might not be verifiable in polynomial time, like the Halting

Problem).

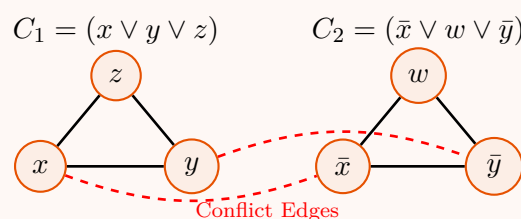
- **NP-complete:** A problem is NP-complete if it is **both** in NP *and* NP-hard.

2. Prove that Satisfiability (SAT) is NP-complete (Cook-Levin Sketch):

- **SAT $\in$ NP:** Given a boolean formula and an assignment of variables, we can evaluate the formula in polynomial time to check if it resolves to True.
- **NP-hard:** By definition, every problem $L \in$ NP has a non-deterministic Turing Machine M that verifies it in polynomial time. The step-by-step state transitions, tape contents, and head positions of M over a polynomial number of steps can be logically encoded as a massive Boolean formula. This formula is constructed such that it is satisfiable IF AND ONLY IF there exists a valid sequence of state transitions leading to M 's ACCEPT state. Thus, any NP problem can be reduced directly to SAT in polynomial time.

3. Prove that 3-SAT $\leq_p$ Independent Set:

- For a 3-SAT instance with k clauses, construct a graph G .
- For each clause (e.g., $C_1 = x_1 \vee \bar{x}_2 \vee x_3$), create a triangle of 3 vertices, one for each literal.
- Add "conflict edges" between any two vertices across the entire graph that represent contradictory literals (e.g., connect an x_1 node to an $\bar{x}_1$ node).
- Set the target independent set size to k .
- **Equivalence:** A valid Independent Set of size k must pick exactly one node per triangle (clause) because nodes within a triangle are connected. It cannot pick contradictory literals because of the conflict edges. Assigning True to the picked literal nodes perfectly satisfies all k clauses. Thus, the reduction holds.



4. Example of Reduction by Equivalence ($\equiv_p$):

Two problems are equivalent under polynomial-time reduction if $A \leq_p B$ and $B \leq_p A$.

Example: Independent Set $\equiv_p$ Vertex Cover.

A subset of vertices S is an Independent Set of size k in graph $G = (V, E)$ IF AND ONLY IF its complement $V \setminus S$ is a Vertex Cover of size $|V| - k$ in the exact same graph G . Converting an instance of one to the other simply requires subtracting the target size from the total number of vertices, taking $O(1)$ time in both directions.

Q36. Instance of Satisfiability Problem

Boolean variables: X_1, X_2, X_3, X_4, X_5

Clauses:

$$C_1 = X_1 \vee \overline{X_2}, \quad C_2 = \overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5}$$

$$C_3 = \overline{X_2}, \quad C_4 = X_2 \vee X_3 \vee \overline{X_4}$$

Mention the number of boolean variables and clauses in the 3-SAT problem.

Answer: Variable and Clause Count in 3-SAT Transformation

To determine the exact number of boolean variables and clauses in the transformed 3-SAT problem, we must apply the standard SAT to 3-SAT reduction rules to each individual clause based on its length.

Original Instance:

- Variables: 5 (X_1, X_2, X_3, X_4, X_5)
- Clauses: 4 (C_1, C_2, C_3, C_4)

Step-by-Step Transformation Breakdown:**(a) Clause $C_1 = X_1 \vee \overline{X_2}$ (Length = 2)**

- *Rule:* Introduce 1 dummy variable (Y_1).
- *Result:* Becomes 2 clauses: $(X_1 \vee \overline{X_2} \vee Y_1) \wedge (X_1 \vee \overline{X_2} \vee \overline{Y_1})$.
- **New Variables Added:** 1
- **Resulting Clauses:** 2

(b) Clause $C_2 = \overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5}$ (Length $k = 4$)

- *Rule:* For $k > 3$, introduce $k - 3$ dummy variables. Here, $4 - 3 = 1$ dummy variable (Y_2).
- *Result:* Becomes 2 clauses: $(\overline{X_1} \vee X_2 \vee Y_2) \wedge (\overline{Y_2} \vee X_4 \vee \overline{X_5})$.
- **New Variables Added:** 1
- **Resulting Clauses:** 2

(c) Clause $C_3 = \overline{X_2}$ (Length = 1)

- *Rule:* Introduce 2 dummy variables (Y_3, Y_4).
- *Result:* Becomes 4 clauses mapping all truth combinations of Y_3 and Y_4 :
 $(\overline{X_2} \vee Y_3 \vee Y_4) \wedge (\overline{X_2} \vee Y_3 \vee \overline{Y_4}) \wedge (\overline{X_2} \vee \overline{Y_3} \vee Y_4) \wedge (\overline{X_2} \vee \overline{Y_3} \vee \overline{Y_4})$.
- **New Variables Added:** 2
- **Resulting Clauses:** 4

(d) Clause $C_4 = X_2 \vee X_3 \vee \overline{X_4}$ (Length = 3)

- *Rule:* No changes needed as it is already exactly 3 literals.

- *Result:* Remains 1 clause: $(X_2 \vee X_3 \vee \overline{X_4})$.
- **New Variables Added:** 0
- **Resulting Clauses:** 1

Final Calculation for 3-SAT:

- **Total Boolean Variables:** Original Variables (5) + Total Dummy Variables $(1 + 1 + 2 + 0) = \mathbf{9}$ variables (which are $X_1, X_2, X_3, X_4, X_5, Y_1, Y_2, Y_3, Y_4$).
- **Total Clauses:** Sum of resulting clauses $(2 + 2 + 4 + 1) = \mathbf{9}$ clauses.

BUET · CSE 105

DSA

Miscellaneous

B-Trees, Arborescences, Misc Topics

T21 — Miscellaneous

Q1. The strength of a path in an edge-weighted graph G is the minimum weight among its edges. The all-pairs strongest path problem asks to find the strongest path between every pair of vertices. Design an algorithm to solve this problem. **(5 marks)** [CSE 207 | 2017–18]

Answer: All-Pairs Strongest Path Algorithm

The "strength" of a path is determined by the minimum weight edge on that path. We need to find paths that maximize this minimum weight for every pair of vertices. An elegant and highly efficient algorithm utilizes the **Maximum Spanning Tree (MST)**. It is a proven property that the unique path between any two nodes in a Maximum Spanning Tree of a graph is also the strongest possible path between them in the entire graph.

Algorithm ($O(E \log V + V^2)$):

(a) **Construct Maximum Spanning Tree (MST):**

- Sort all edges in *descending* order of their weights.
- Use Kruskal's algorithm (with Disjoint Set Union) to build the MST.
- This step takes $O(E \log V)$ time.

(b) **Find Paths in MST:**

- Since the MST is a tree, there is exactly one path between any pair of vertices u and v .
- To find the strength of the path for *all pairs*, run a Breadth-First Search (BFS) or Depth-First Search (DFS) starting from every vertex $v \in V$.
- During the traversal from a starting node s to any node u , keep track of the minimum edge weight seen so far. When reaching a neighbor v from u , the strength of the path ($s \rightarrow v$) is simply $\min(\text{strength}(s \rightarrow u), \text{weight}(u, v))$.
- Running V independent BFS/DFS traversals on a tree (which has $V - 1$ edges) takes $O(V \times V) = O(V^2)$ time.

Conclusion: By filling an $N \times N$ matrix during the traversals, we successfully compute the all-pairs strongest paths in $O(E \log V + V^2)$ overall time, which is significantly faster than a modified Floyd-Warshall approach ($O(V^3)$).

Q2. Prove that a tree with n nodes where the heights of the two children of any node differ by at most 1 has $O(\log n)$ height. **(12 marks)** [CSE 207 | 2017–18]

Answer: Height of an AVL Tree is $O(\log n)$

Proof:

Let $N(h)$ be the minimum number of nodes in a tree of height h where the heights

of the two children of any node differ by at most 1. To minimize the number of nodes for a given height, one child must have height $h - 1$ and the other must have height $h - 2$.

Thus, we can write the recurrence relation for the minimum number of nodes:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

With base cases: $N(0) = 1$ (a single node) and $N(1) = 2$ (a root and one child). To solve this, we add 1 to both sides of the recurrence:

$$N(h) + 1 = (N(h - 1) + 1) + (N(h - 2) + 1)$$

Let $F(h) = N(h) + 1$. The equation becomes exactly the Fibonacci recurrence:

$$F(h) = F(h - 1) + F(h - 2)$$

where $F(0) = 2$ and $F(1) = 3$.

We know from the properties of Fibonacci numbers that $F(h)$ grows exponentially with the golden ratio $\phi \approx 1.618$. Specifically, $F(h) \geq \phi^h$. Substituting back $N(h)$:

$$N(h) + 1 \geq \phi^h$$

For a tree with n nodes, $n \geq N(h)$, so:

$$n + 1 \geq \phi^h$$

Taking the base- ϕ logarithm on both sides:

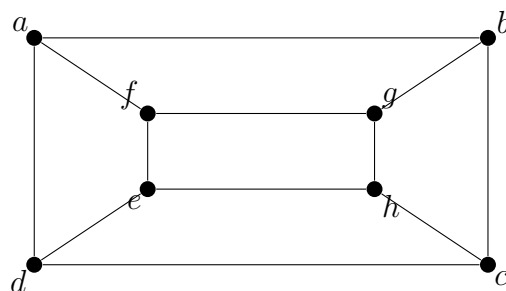
$$h \leq \log_{\phi}(n + 1)$$

Since the base change formula gives $\log_{\phi}(x) = \frac{\log_2(x)}{\log_2(\phi)}$ and $\log_2(\phi)$ is a constant, we drop the constants for Big-O notation. Therefore, the height is bounded by:

$$h = O(\log n)$$



- Q3.** Apply backtracking to the problem of finding a Hamiltonian circuit in a given graph. Show the corresponding state-space tree. (10 marks) [CSE 207 | 2017–18]



Answer

Graph edges: $a-b$, $b-c$, $c-d$, $d-a$ (outer); $e-f$, $f-g$, $g-h$, $h-e$ (inner); $a-f$, $b-g$, $c-h$, $d-e$ (connectors).

Adjacency: a : {b,d,f}, b : {a,c,g}, c : {b,d,h}, d : {c,a,e}, e : {d,f,h}, f : {a,e,g}, g : {b,f,h}, h : {c,e,g}.

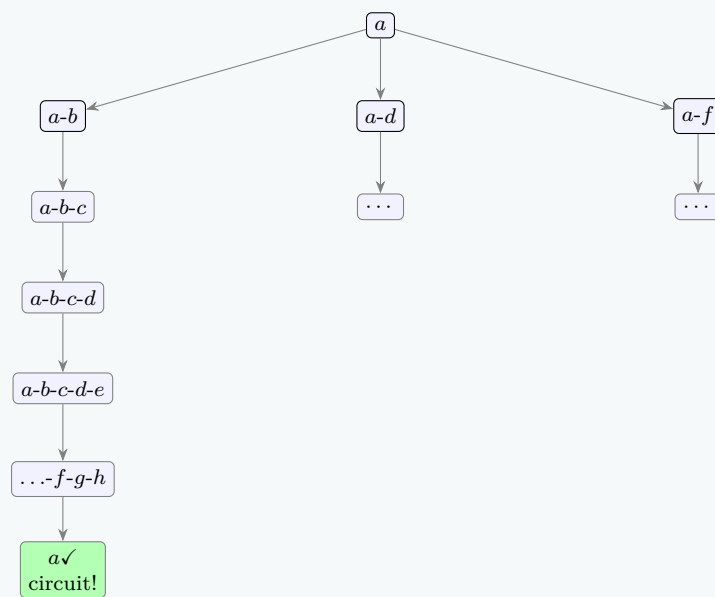
Backtracking from vertex a (alphabetical neighbor order):

```

1 HAMILTONIAN(path, adj):
2     if |path| == |V|:
3         if path[0] in adj[path[-1]]: return path + [path[0]]
4         return FAIL
5     u = path[-1]
6     for each v in adj[u] (alphabetical):
7         if v not in path:
8             result = HAMILTONIAN(path + [v], adj)
9             if result != FAIL: return result
10    return FAIL

```

State-space tree (partial, showing first found path):



First Hamiltonian circuit found:

$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow h \rightarrow g \rightarrow f \rightarrow a$$

Verification: All 8 vertices visited exactly once; edges $a-b$, $b-c$, $c-d$, $d-e$, $e-h$, $h-g$, $g-f$, $f-a$ all exist ✓.

Backtracking prunes when: (i) a vertex is already in the path, or (ii) the last vertex has no unvisited neighbors and not all vertices are visited.

Q4. [BST given] Perform SPLAY(1) on the given binary search tree, showing all intermediate steps and operations performed at each node. (10 marks) [CSE 207 |

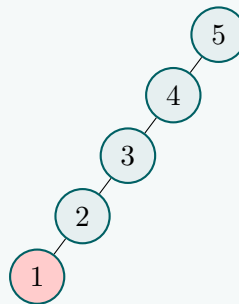
2017–18]



Answer: Splay(1) Operation on BST

When we access node 1, we must move it to the root using Splay operations. Because the tree is entirely left-skewed, we will apply the **Zig-Zig** step multiple times.

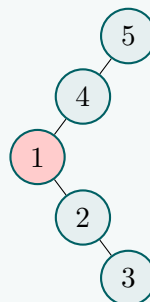
Initial State: Node 1 is a Left-Left child of 3 (Parent=2, Grandparent=3).



Step 1: Zig-Zig on Node 1 (Parent=2, Grandparent=3)

A Zig-Zig consists of two right rotations: first at the grandparent (3), then at the parent (2).

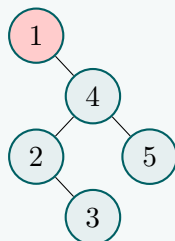
- *Right Rotate(3)*: 2 becomes the root of the subtree, 3 becomes the right child of 2.
- *Right Rotate(2)*: 1 becomes the root of the subtree, 2 becomes the right child of 1.



Step 2: Zig-Zig on Node 1 (Parent=4, Grandparent=5)

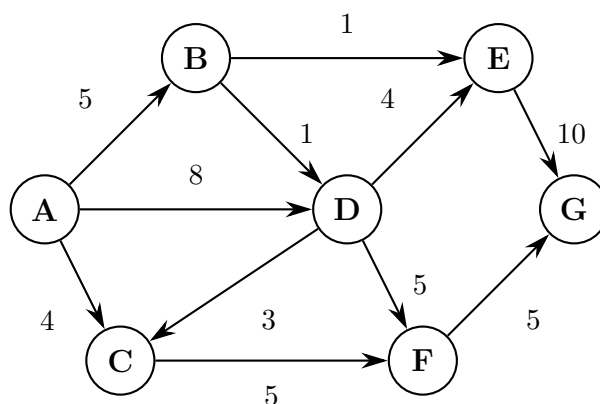
Node 1 is now a Left-Left child of 5. We perform another Zig-Zig.

- *Right Rotate(5)*: 4 becomes the root, 5 becomes the right child of 4. Node 1 is still the left child of 4.
- *Right Rotate(4)*: 1 becomes the final root. 4 becomes the right child of 1. The previous right subtree of 1 (rooted at 2) detaches and becomes the left child of 4.



Node 1 is now the root. The Splay operation is complete.

Q5. A traffic engineer needs to decide which roads to widen to allow maximum traffic from *A* to *G*. Which roads are to be widened? **(12 marks)** [CSE 207 | 2014–15]


Answer: Maximum Traffic & Roads to Widen

To find which roads to widen to increase the total traffic from *A* to *G*, we must identify the bottlenecks of the network. This is equivalent to finding the **Minimum Cut** of the graph.

1. Max Flow Calculation:

Let's route traffic (flow) through available paths:

- Path $A \rightarrow B \rightarrow E \rightarrow G$: Bottleneck is $B \rightarrow E$ (cap 1). Flow = 1.
- Path $A \rightarrow D \rightarrow E \rightarrow G$: Bottleneck is $D \rightarrow E$ (cap 4). Flow = 4.
- Path $A \rightarrow D \rightarrow F \rightarrow G$: Remaining capacity on $A \rightarrow D$ is $8 - 4 = 4$. $D \rightarrow F$

has 5, $F \rightarrow G$ has 5. Bottleneck is 4. Flow = 4.

- Path $A \rightarrow C \rightarrow F \rightarrow G$: Remaining cap on $F \rightarrow G$ is $5 - 4 = 1$. $A \rightarrow C$ has 4, $C \rightarrow F$ has 5. Bottleneck is 1. Flow = 1.

Total Max Flow = $1 + 4 + 4 + 1 = 10$.

2. Identifying the Min-Cut (The Bottlenecks):

We find the set of nodes reachable from A in the residual graph.

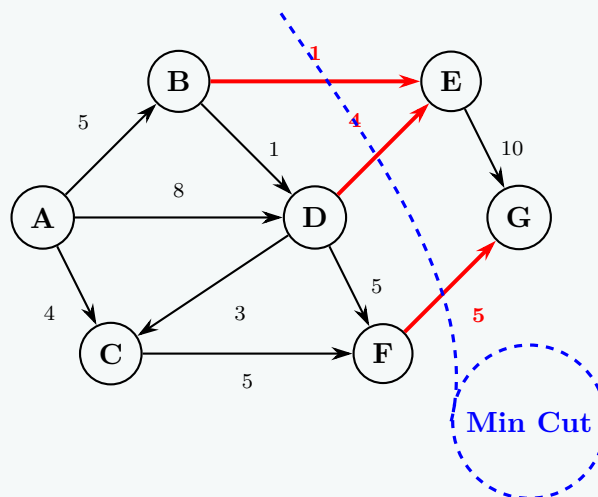
- Reachable set $S = \{A, B, C, D, F\}$.
- Unreachable set $T = \{E, G\}$.

The edges crossing from S to T constitute the Minimum Cut. These are the edges currently running at absolute maximum capacity that strictly prevent any further flow from reaching G .

- Edge $B \rightarrow E$ (Capacity 1, Flow 1)
- Edge $D \rightarrow E$ (Capacity 4, Flow 4)
- Edge $F \rightarrow G$ (Capacity 5, Flow 5)

The sum of these capacities is $1 + 4 + 5 = 10$, confirming Max Flow = Min Cut.

Conclusion: The traffic engineer must widen the roads $B \rightarrow E$, $D \rightarrow E$, and $F \rightarrow G$. Widening any of these specific roads will immediately allow more traffic to flow from A to G .



Q6. Define and explain the following terms in a network flow model: super sink node, augmented path, residual capacity, super source node. (8 marks) [CSE 207 | 2014–15]

Answer: Network Flow Terminology

- **Super Source Node:** When a network has multiple starting points (multiple sources) generating flow, we add a fictitious "super source" node. We connect this super source to all the original source nodes with edges of infinite capacity (∞). This perfectly converts a multi-source problem into a standard single-source problem.
- **Super Sink Node:** Similarly, if a network has multiple destinations (multiple sinks), a fictitious "super sink" is introduced. All original sink nodes are connected to this super sink using directed edges with infinite capacity (∞), transforming the setup into a single-sink problem.
- **Residual Capacity:** The amount of additional flow that can be pushed through an edge before it reaches its absolute limit. If an edge e has a maximum capacity $c(e)$ and is currently carrying flow $f(e)$, its residual capacity is mathematically defined as: $c_f(e) = c(e) - f(e)$. It dictates how much room is left on the road.
- **Augmented Path (Augmenting Path):** A simple, valid path from the source node to the sink node entirely within the *residual graph*, where every edge along the path has a residual capacity strictly greater than zero. Finding an augmented path means we have discovered a route to successfully push more overall flow through the network.

Q7. Write a linear-time algorithm and an exponential-time algorithm for finding the n -th Fibonacci number. Analyze the time complexity of each. **(10 marks)** [CSE 207 | 2013–14]

Answer: Algorithms for n -th Fibonacci Number**1. Exponential-Time Algorithm:**

This uses direct recursion based on the mathematical definition $F(n) = F(n-1) + F(n-2)$.

Algorithm 155 Fibonacci-Exponential(n)

```

1: if  $n \leq 1$  then
2:   return  $n$ 
3: else
4:   return Fibonacci-Exponential( $n-1$ ) + Fibonacci-Exponential( $n-2$ )
5: end if

```

Time Complexity Analysis: Every non-base call branches into two recursive calls. The recursion tree has a maximum depth of n . The number of nodes in the tree grows exponentially, making the time complexity exactly $O(2^n)$ (more precisely $O(\phi^n)$ where $\phi \approx 1.618$ is the golden ratio).

2. Linear-Time Algorithm:

This uses iterative dynamic programming (bottom-up approach) to avoid recalculating overlapping subproblems.

Algorithm 156 Fibonacci-Linear(n)

```

1: if  $n \leq 1$  then return  $n$ 
2: end if
3:  $prev2 \leftarrow 0$ 
4:  $prev1 \leftarrow 1$ 
5: for  $i \leftarrow 2$  to  $n$  do
6:    $current \leftarrow prev1 + prev2$ 
7:    $prev2 \leftarrow prev1$ 
8:    $prev1 \leftarrow current$ 
9: end for
10: return  $prev1$ 
    
```

Time Complexity Analysis: The algorithm uses a single loop that runs exactly $n - 1$ times. Inside the loop, it performs constant time $O(1)$ basic arithmetic operations. Therefore, the overall time complexity is $O(n)$.

- Q8.** Given a directed graph with edge list and weights $AB(5), BC(2), AC(4), AD(6), CD(3), DB(1), DE(6), EB(3), EF(3), FB(4)$, construct a minimum-cost arborescence rooted at A. (10 marks) [CSE 207 | 2013–14]

Answer: Minimum-Cost Arborescence (Edmonds' Algorithm)

We need to build a directed Minimum Spanning Tree (Arborescence) rooted at A.

Edges: $AB(5), BC(2), AC(4), AD(6), CD(3), DB(1), DE(6), EB(3), EF(3), FB(4)$.

Step 1: Find the minimum weight incoming edge for every node (except root A).

- **B:** $\min(AB:5, DB:1, EB:3, FB:4) = \mathbf{DB (1)}$
- **C:** $\min(BC:2, AC:4) = \mathbf{BC (2)}$
- **D:** $\min(AD:6, CD:3) = \mathbf{CD (3)}$
- **E:** $\min(DE:6) = \mathbf{DE (6)}$
- **F:** $\min(EF:3) = \mathbf{EF (3)}$

Step 2: Check for cycles.

The selected edges are DB, BC, CD, DE, EF.

Notice that $D \rightarrow B \rightarrow C \rightarrow D$ forms a cycle! Cycle weight = $1 + 2 + 3 = 6$.

Step 3: Contract the cycle and adjust incoming edges.

We treat the cycle $\{B, C, D\}$ as a single super-node. We check edges coming from outside the cycle (from A, E, F) into the cycle, and adjust their weights using:
 $W_{new}(u, v) = W_{old}(u, v) - W_{cycle_in}(v)$.

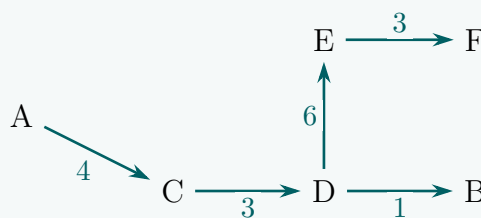
- $A \rightarrow B: 5 - 1 = 4$
- $A \rightarrow C: 4 - 2 = 2$ (Minimum incoming edge to the cycle!)
- $A \rightarrow D: 6 - 3 = 3$

Step 4: Break the cycle.

The absolute minimum incoming edge to the cycle is **AC** (original weight 4). The edge entering C inside the cycle is **BC**. To break the cycle, we remove BC and keep AC.

Step 5: Final Arborescence Construction.

We keep the cycle edges DB and CD, substitute BC with AC, and keep the non-cycle selected edges DE and EF.



Total Minimum Cost: $4 \text{ (AC)} + 3 \text{ (CD)} + 1 \text{ (DB)} + 6 \text{ (DE)} + 3 \text{ (EF)} = 17$.

- Q9.** Write pseudocode of the FIND-SET algorithm that works using the path-compression heuristic. FIND-SET(x) returns a pointer to the representative of the set containing x . (6 marks) [CSE 207 | 2012–13]

Answer: Find-Set with Path Compression

Path compression is a heuristic that flattens the structure of the tree whenever ‘Find-Set’ is called. Instead of just finding the root, it updates the parent pointer of every node along the path to point directly to the root, making future queries $O(1)$ almost constantly.

Algorithm 157 Find-Set(x)

-
- 1: **if** $x \neq \text{parent}[x]$ **then**
 - 2: $\text{parent}[x] \leftarrow \text{Find-Set}(\text{parent}[x])$ $\triangleright$ Recursive call to find root, and compress path
 - 3: **end if**
 - 4: **return** $\text{parent}[x]$ $\triangleright$ Returns the representative/root of the set
-

- Q10.** Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys in a probabilistic skip-list:

Key	3	6	7	9	12	17	19	21	25	26
No. of consecutive heads	1	4	1	2	1	2	1	1	3	1

Perform the following operations on the skip-list and show the procedures:

- Search 21
- Insert 22 with 3 consecutive heads
- Delete 19
- Search 18

(7+8=15 marks)

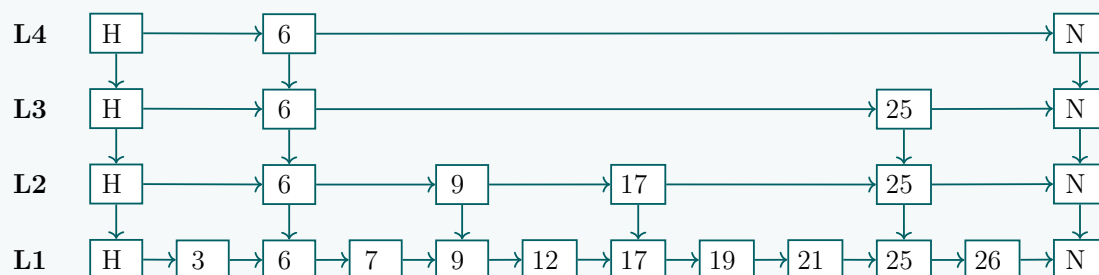
[CSE 203 | 2011–12]

Answer: Skip-List Construction and Operations

Construction:

Assuming the number of consecutive heads determines the height of the node above the base list (so k heads means the node exists on levels 1 to k). Level 1 is the base linked-list containing all nodes.

- **L4:** $H \rightarrow 6 \rightarrow N$
- **L3:** $H \rightarrow 6 \rightarrow 25 \rightarrow N$
- **L2:** $H \rightarrow 6 \rightarrow 9 \rightarrow 17 \rightarrow 25 \rightarrow N$
- **L1 (Base):** $H \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 12 \rightarrow 17 \rightarrow 19 \rightarrow 21 \rightarrow 25 \rightarrow 26 \rightarrow N$



Operations Procedure:

(a) Search 21:

- Start at Top-Left $H(L4)$. $H \rightarrow 6$. $6 < 21$, move to 6. Next is N, drop down to 6($L3$).
- $6(L3) \rightarrow 25$. $25 > 21$, drop down to 6($L2$).
- $6(L2) \rightarrow 9$. $9 < 21$, move to 9($L2$). $9(L2) \rightarrow 17$. $17 < 21$, move to 17($L2$).
- $17(L2) \rightarrow 25$. $25 > 21$, drop down to 17($L1$).
- $17(L1) \rightarrow 19$. $19 < 21$, move to 19($L1$). $19 \rightarrow 21$. Match found! Return true.

(b) Insert 22 with 3 consecutive heads:

- Search for 22 keeping track of predecessors: $H(L3)$, $17(L2)$, $21(L1)$.

- Insert 22 after 21 at L1. Because it has 3 heads, promote it to L2 (after 17) and L3 (after 6, before 25). Adjust pointers accordingly.

(c) **Delete 19:**

- Search for 19. It is only located on L1.
- Update pointers at L1 to bypass it: $17(L1) \rightarrow 21(L1)$. Delete the node.

(d) **Search 18:**

- Trace path: $H(L4) \rightarrow 6(L4) \rightarrow \text{drop} \rightarrow 6(L3) \rightarrow \text{drop} \rightarrow 6(L2) \rightarrow 9(L2) \rightarrow 17(L2)$.
- $17(L2) \rightarrow 25$. $25 > 18$, drop to $17(L1)$.
- $17(L1) \rightarrow 19$. $19 > 18$. We are at the base level (L1) and the next node is larger. Return Not Found.

Q11. Show that the expected running time of searching in a probabilistic skip-list is $O(\log n)$. (6 marks) [CSE 203 | 2011–12]

Answer: Expected Running Time in Probabilistic Skip-List

To prove that the expected search time is $O(\log n)$, we analyze the search path backwards from the target node up to the top-left Head node.

- **Height of the List:** In a probabilistic skip-list, a node is promoted to the next level with probability $p = 1/2$. The probability that a node reaches level k is $(1/2)^k$. For n nodes, the expected maximum level is $\log_2 n$.
- **Backward Search Path:** When traveling backwards from the target:
 - (a) If the current node was promoted from the level below (which happens with probability $p = 1/2$), we move *down*.
 - (b) Otherwise (with probability $1 - p = 1/2$), we move *left*.
- **Expected Steps per Level:** The number of leftward steps before dropping down follows a geometric distribution with success probability $p = 1/2$. The expected value of a geometric distribution is $1/p$. Thus, we take an expected $1/(1/2) = 2$ horizontal steps per level.
- **Total Time:** The total expected number of steps is the expected steps per level multiplied by the expected number of levels:

$$\text{Total Expected Steps} = (\text{Expected Steps per level}) \times (\text{Expected Height})$$

$$\text{Total Expected Steps} = 2 \times \log_2 n$$

Since $2 \log_2 n$ operations take proportional time, the expected overall running time of a search is $O(\log n)$. ■

- Q12.** Discuss the Multi-peg Hanoi problem. Argue in favour of the recurrence relation to be satisfied by the presumed optimal solution. Prove the properties of the presumed optimal solution. (5+5+10=20 marks) [CSE 207 | 2008–09]

Answer: Multi-peg Hanoi Problem

1. Discussion:

The standard Tower of Hanoi problem involves 3 pegs. The Multi-peg Hanoi problem (often called Reve's puzzle for 4 pegs) generalizes this to k pegs ($k > 3$). The objective remains the same: move n disks from the source peg to the destination peg without ever placing a larger disk on a smaller one, but now we have more auxiliary pegs, which significantly reduces the total number of moves.

2. Recurrence Relation (Frame-Stewart Algorithm):

The presumed optimal solution for k pegs and n disks satisfies the following recurrence relation for $M(n, k)$ (the minimum number of moves):

$$M(n, k) = \min_{1 \leq i < n} [2 \cdot M(i, k) + M(n - i, k - 1)]$$

Base Cases: $M(1, k) = 1$ and $M(n, 3) = 2^n - 1$.

3. Argument in Favour of the Recurrence:

The strategy involves three main steps:

- **Step 1:** Move the top i disks from the source peg to an auxiliary peg. We can use all k pegs for this. This takes $M(i, k)$ moves.
- **Step 2:** Move the remaining $n - i$ disks to the destination peg. Since the peg holding the i smaller disks cannot be used (we can't put larger disks on them), we effectively have $k - 1$ pegs available. This takes $M(n - i, k - 1)$ moves.
- **Step 3:** Move the i disks from the auxiliary peg to the destination peg. We again have all k pegs available. This takes $M(i, k)$ moves.

To find the optimal strategy, we must test all possible values of i (from 1 to $n - 1$) and choose the one that minimizes the total sum.

4. Properties of the Presumed Optimal Solution:

- **Optimal i :** For $k = 4$, the optimal number of disks i to move in the first step is roughly $n - \sqrt{2n}$.
- **Complexity:** Unlike the 3-peg problem which is $O(2^n)$, the 4-peg problem has a sub-exponential, super-polynomial time complexity of approximately $O(2^{\sqrt{n}})$.

- Q13.** Write down the backtracking technique for solving the 4-Queen problem. Give the state tree also. (10+5=15 marks) [CSE 207 | 2008–09]

Answer: 4-Queen Problem Backtracking & State Tree**Backtracking Algorithm:****Algorithm 158** Solve-N-Queens(row, board)

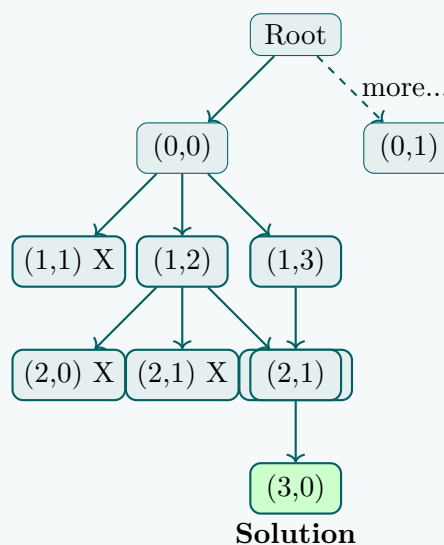
```

1: if row =  $N$  then
2:   Print board configuration
3:   return True                                ▷ Found a solution
4: end if
5: for col = 0 to  $N - 1$  do
6:   if Is-Safe(board, row, col) then
7:     board[row][col]  $\leftarrow Q$                 ▷ Place queen
8:     Solve-N-Queens(row + 1, board)              ▷ Recurse
9:     board[row][col]  $\leftarrow$  empty             ▷ Backtrack
10:  end if
11: end for

```

State Space Tree (Partial, showing the first successful branch):

Nodes represent (row, column) placement. An 'X' means the branch is pruned (not safe).



Q14. Applying the FIFO branch-and-bound technique on the 4-Queen problem, it is found that the technique is not efficient. Why? **(6 marks)** [CSE 207 | 2008–09]

Answer: Inefficiency of FIFO Branch-and-Bound on 4-Queen

FIFO (First-In-First-Out) Branch-and-Bound is essentially a **Breadth-First Search (BFS)** approach. When applied to the N-Queen problem, it is highly inefficient for the following reasons:

- (a) **Exponential Space Complexity:** FIFO explores the state space tree level

by level. Before reaching level 4 (where the actual solutions reside), it must generate and store *all* valid configurations of 1 queen, then *all* valid configurations of 2 queens, and so on. This requires an enormous amount of memory queue space, scaling exponentially with N .

- (b) **Delayed Pruning Feedback:** In the N-Queens problem, conflicts often become apparent only deep in the tree (e.g., placing the 3rd queen reveals a dead end). Standard Backtracking (DFS) dives deep immediately, detects the conflict, and prunes the branch. FIFO BFS wastes time exploring parallel configurations at shallow levels instead of quickly resolving deeper conflicts.

Therefore, LIFO (DFS / Backtracking) is far superior as its space complexity is merely $O(N)$.

Q15. Write down an algorithm for finding the minimum sum of a consecutive subsequence using an efficient algorithm. Simulate your algorithm on the array:

1, -2, 3, -5, -3, 2, 1, -7, -2, 1, 2, -3, 4, 1

(5+5 marks)

[CSE 207 | 2006–07]

Answer: Minimum Sum Consecutive Subsequence

Algorithm (Modified Kadane's Algorithm for Minimum Sum):

Algorithm 159 Min-Subsequence-Sum(A, n)

```

1: min_so_far  $\leftarrow A[0]$ 
2: min_ending_here  $\leftarrow A[0]$ 
3: for  $i \leftarrow 1$  to  $n - 1$  do
4:   min_ending_here  $\leftarrow \min(A[i], \textit{min\_ending\_here} + A[i])$ 
5:   min_so_far  $\leftarrow \min(\textit{min\_so\_far}, \textit{min\_ending\_here})$ 
6: end for
7: return min_so_far
```

Simulation on Array: [1, -2, 3, -5, -3, 2, 1, -7, -2, 1, 2, -3, 4, 1]

Element ($A[i]$)	$A[i] + \text{min_ending_here}$	New min_ending_here	min_so_far
1 (Init)	-	1	1
-2	$1 + (-2) = -1$	-2 (since $-2 < -1$)	-2
3	$-2 + 3 = 1$	1	-2
-5	$1 - 5 = -4$	-5 (since $-5 < -4$)	-5
-3	$-5 - 3 = -8$	-8	-8
2	$-8 + 2 = -6$	-6	-8
1	$-6 + 1 = -5$	-5	-8
-7	$-5 - 7 = -12$	-12	-12
-2	$-12 - 2 = -14$	-14	-14
1	$-14 + 1 = -13$	-13	-14
2	$-13 + 2 = -11$	-11	-14
-3	$-11 - 3 = -14$	-14	-14
4	$-14 + 4 = -10$	-10	-14
1	$-10 + 1 = -9$	-9	-14

Result: The minimum sum is **-14**. The consecutive subsequence is: -5, -3, 2, 1, -7, -2.

- Q16.** How is the minimum cost arborescence problem different from the minimum spanning tree problem? Construct a minimum cost arborescence for the graph with edges $AB(7)$, $AC(3)$, $DA(4)$, $EA(1)$, $BC(91)$, $CF(9)$, $BD(7)$, $EB(2)$, $FB(5)$, $CE(4)$, $DE(6)$, $FD(3)$, with edge orientations from the first vertex to the second. Assume source vertex is A. **(5+5 marks)** [CSE 207 | 2006–07]

Answer: Minimum Cost Arborescence

Difference from MST:

- **MST:** Applies to *undirected* graphs. The goal is to connect all vertices with minimum total weight.
- **Arborescence:** Applies to *directed* graphs. It requires a designated **root node**. It is a directed tree where all vertices are reachable from the root, and every node (except the root) has exactly one incoming edge.

Construction (Edmonds' Algorithm):

Graph Edges: AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6), FD(3). Root = A.

Step 1: For each node (except root A), select the incoming edge with minimum weight.

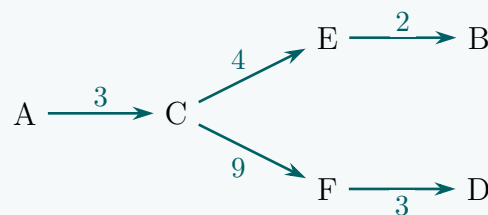
- To B: AB(7), EB(2), FB(5) → **Select EB(2)**
- To C: AC(3), BC(91) → **Select AC(3)**
- To D: BD(7), FD(3) → **Select FD(3)**
- To E: CE(4), DE(6) → **Select CE(4)**
- To F: CF(9) → **Select CF(9)**

Step 2: Check for cycles.

Selected edges are: $E \rightarrow B$, $A \rightarrow C$, $F \rightarrow D$, $C \rightarrow E$, $C \rightarrow F$.

Tracing from A: $A \rightarrow C \rightarrow E \rightarrow B$ and $A \rightarrow C \rightarrow F \rightarrow D$. All nodes are reached and **no cycles are formed**.

Because no cycle is formed, the algorithm terminates immediately. The selected edges constitute the optimal Arborescence.



Total Minimum Cost: $3 + 4 + 9 + 2 + 3 = 21$.

Q17. How can the complexity of Floyd-Warshall’s algorithm for “all pairs shortest paths” be improved? **(10 marks)**

Answer: Improving All-Pairs Shortest Path Complexity

The standard Floyd-Warshall algorithm computes All-Pairs Shortest Paths (APSP) in exactly $O(V^3)$ time, which is very inefficient for *sparse graphs* (where the number of edges $E \ll V^2$).

To improve the complexity for sparse graphs, we can use **Johnson’s Algorithm**.

- (a) **Concept:** Johnson’s algorithm uses the Bellman-Ford algorithm to reweight all edges to be non-negative in $O(VE)$ time.
- (b) **Dijkstra Integration:** Once edges are non-negative, it runs Dijkstra’s algorithm from *every* vertex to find all shortest paths.
- (c) **Improved Complexity:** If Dijkstra is implemented using a **Fibonacci Heap**, a single run takes $O(E + V \log V)$. Running it V times yields an overall APSP

complexity of:

$$O(V^2 \log V + VE)$$

Conclusion: For sparse graphs (e.g., $E \approx V$), Johnson's algorithm reduces the APSP complexity from $O(V^3)$ to nearly $O(V^2 \log V)$, which is a massive theoretical and practical improvement.